Distributed Systems by Design

The concepts, numbers and code under every system design

First edition, October 2026

Contents

About this book

Part I. Thinking in systems

1 From requirements to a design
2 The machine under the code
3 Concurrency: threads, async and race conditions
4 Networks for system designers

Part II. Interfaces and architecture

5 APIs: styles, formats and contracts
6 Application architecture

Part III. Building blocks

7 Scalability: from one server to many
8 Load balancers and gateways
9 Caching and Redis
10 CDNs and the edge
11 Rate limiting, backpressure and resilience
12 Storage and databases
13 Database types: relational, NoSQL and beyond
14 Message queues, streams and workers
15 Compute in the cloud: containers, Kubernetes and functions

Part IV. Distribution

16 Replication
17 Partitioning and consistent hashing
18 Consistency, clocks and consensus
19 The algorithms inside real systems
20 Web3: blockchains as distributed systems
21 Big data: batch, streams and lakehouses

Part V. Safe and reliable

22 Security
23 Reliability, observability and operations

Part VI. Putting it together

24 External systems, tools and AI agents
25 Encore end to end: from requirements to code and cloud
26 Hard problems by service, and how to solve them
27 The system design interview

Appendix A. Numbers to know
Appendix B. Formulas
Appendix C. Cloud service map
Appendix D. Glossary
Selected reading

About this book

Every system design, from a URL shortener to a global payment network, is built from the same few ideas: a request crosses a network, waits in a queue, reads from a cache or a database, and changes some state that other machines must agree on. This book teaches those ideas one at a time, in the order a request meets them, so that a blank whiteboard turns into a sequence of familiar decisions.

It is the concepts book under its sibling, *50 System Designs*. That book works through fifty case studies; this one explains the building blocks those case studies reuse — load balancers, caches, CDNs, rate limiters, databases, queues, replication, sharding, consensus — and the numbers that decide between them. Where a case study uses a concept, the text points to it as "*50 System Designs* #25 (Ticket Booking)".

How each chapter teaches

Every concept follows the same path:

1. **A picture first.** An everyday analogy and a small diagram, so the idea has a shape before it has details.
2. **The mechanism.** How it works, step by step.
3. **The numbers.** The equation that governs it, with a worked example. System design is quantitative: "add a cache" becomes "a 95 percent hit ratio cuts database reads from 6,000 to 300 per second".
4. **Code or configuration.** A short example you can run: Python for algorithms and services, SQL for data, Redis commands, Kubernetes YAML, HTTP traces.
5. **Where it runs.** Which real systems implement it, and which managed cloud service you would rent instead of building it.
6. **The trade-off.** What you give up, and when the alternative wins.

Design questions appear inside the chapters, where the concept has just been taught, in the form an interviewer would ask them, each with a compact worked answer. The appendices collect the numbers to know, every formula, a map of equivalent services across AWS, Google Cloud and Azure, and a glossary.

The running example: Encore

One product runs through the whole book. **Encore** is a concert-ticket marketplace. Fans browse and search events, pick seats on a seat map, hold seats for ten minutes, pay through an external payment provider, and receive tickets by e-mail and in the app; organizers watch live sales dashboards; a support agent built on a language model answers questions and can request refunds.

Encore is ordinary most of the day — 4 million daily users and about 1,500 requests per second — and extraordinary for ten minutes at a time. When a 50,000-seat stadium show goes on sale, a million fans arrive at once, the seat map is read 200,000 times a second, and 30,000 fans a second try to hold seats that only one of them can have. That single event needs nearly everything in this book: a CDN and caches for the reads, a waiting room and rate limits for admission, careful concurrency control so no seat is sold twice, idempotent payments, queues and workers for tickets and e-mail, sharding for the hot inventory, replication across regions, and observability to know it all worked. All Encore numbers are illustrative and are kept identical across chapters, so a figure estimated in Chapter 1 is the figure cached, sharded and paid for later.

How to read it

Read Part I first: requirements, the machine, concurrency and the network are the vocabulary of everything after. Parts II to V can be read in order or by need. Part VI puts it together: how external systems and AI agents fit into a distributed design, Encore built end to end in code and cloud, and a method for the design interview. Readers preparing for an interview in a few days can read the interview chapter first and follow its links back.

Facts about products and cloud services reflect their documentation in October 2026. Services change quickly; treat named limits and defaults as things to confirm, and the reasoning around them as the durable part.

Part I. Thinking in systems

How to turn a problem into requirements and numbers, and what the machine, the operating system and the network actually do with a request.

In this part:

- From requirements to a design
- The machine under the code
- Concurrency: threads, async and race conditions
- Networks for system designers

1 From requirements to a design

Every system design starts with the same question: what exactly must this system do, and how well must it do it? This chapter turns a vague brief ("build a ticket marketplace") into a short list of functions, a set of measurable targets, and a page of numbers that tell you which parts of the design are easy and which part will decide everything. The picture to hold on to is a funnel: words go in at the top, numbers come out at the bottom, and the numbers choose the architecture.

1.1 The picture

Think of an architect asked to design a stadium. Before drawing a single beam, she asks how many people it must seat, how fast it must empty in an emergency, how many toilets the half-time rush needs, and what the budget is. The answers do not describe the building, but they rule out most buildings. Requirements and estimates do the same for software: they rule out designs before you draw boxes.

```
+-----------------------------+  "Build Encore" (words)
 \ functional requirements    /   browse, hold, pay, ticket
  \ non-functional targets   /    p99 < 200 ms, 99.95%, no oversell
   \ estimates              /     200,000 reads/s, 30,000 holds/s
    +---------------------+       (numbers)
              |
              v
API -> data model -> high-level design -> deep dives -> trade-offs
```

The funnel narrows: dozens of possible features become five core flows, and those flows become a handful of numbers. For Encore, one number dominates all others. On a normal day the whole marketplace serves 6,000 requests per second at peak; during one flash sale the seat map alone receives 200,000 per second. A design that handles the normal day is easy. The design is about the flash sale.

1.2 Functional and non-functional requirements

A restaurant menu lists the dishes; the health inspector's checklist says how hot the fridge must be, how fast orders go out, and what happens when the power fails. **Functional requirements** are the menu: the operations a user or another system can perform. **Non-functional requirements** are the checklist: the qualities those operations must have. Two systems with identical menus (two ticket sites) can need completely different designs because their checklists differ.

1.2.1 Functional requirements

Write them as verbs on nouns, from the user's point of view, and mark what is out of scope. For Encore:

- Fans browse and search events by artist, city and date.
- Fans view a seat map with live availability for one event.
- Fans **hold** up to a few seats for 10 minutes; unpaid holds expire and the seats return to sale.
- Fans pay through an external payment provider and receive tickets (QR codes) by e-mail and in the app.
- Organizers create events and watch live sales dashboards.
- Fans ask the support agent about orders and request refunds.
- Out of scope for the first design: resale, dynamic pricing, venue management.

Scoping is a design decision. Every requirement you drop removes a component; every one you add (resale, say) adds a ledger, a fraud model and a second inventory. Say explicitly which flows are core: here, view seat map, hold, pay, issue ticket.

1.2.2 Non-functional requirements

Each quality below is a promise with a number attached. A requirement without a number ("fast", "highly available") cannot be tested, so cannot be designed for.

Quality	Question it answers	Encore target (illustrative)
Latency	How long does one request take?	seat map p99 under 200 ms
Throughput	How many requests per second?	200,000 seat-map reads/s
Availability	What fraction of time does it work?	checkout 99.95% per month
Durability	Can data, once accepted, be lost?	a paid order is never lost
Consistency	Do all readers see the latest write?	a seat is sold at most once
Cost	What does a unit of work cost?	infra cost per ticket in cents
Security	Who may do what?	only the buyer sees a ticket
Compliance	Which laws and standards apply?	PCI DSS, GDPR deletion

A few notes that change designs:

- **Latency is a distribution, not an average.** Quote percentiles: p50 for the typical user, p99 for the unlucky one in a hundred. A page that makes 20 backend calls is slow far more often than any one backend: $1 - 0.99^{20} \approx 0.18$, so almost one page load in five waits for at least one backend's p99.
- **Availability and durability are different promises.** S3 Standard is designed for 99.99% availability and 99.999999999% (eleven nines) durability of objects over a year: an object can be briefly unreachable but is almost never lost.
- **Consistency is per operation, not per system.** The seat map may be a second stale; the hold must be exact.
- **Compliance moves boundaries.** Card numbers never touch Encore's servers: the payment provider's hosted fields tokenise them, which shrinks the PCI DSS scope to almost nothing (Chapter 22).

> **Design question.** Which Encore operations need strong consistency, and which can serve stale data?
>
> List each operation with the cost of a stale answer. Hold and purchase: a stale read sells one seat twice, an angry fan and a refund, so these need a single authoritative write path (a transaction or a compare-and-set on the seat row). Seat map: a seat shown green for one second after it was taken costs one failed hold attempt, which the hold path rejects cleanly, so serve it from a cache refreshed every second. Search and event pages: minutes of lag are harmless, so index asynchronously and cache at the CDN. Dashboards: a few seconds of lag is the requirement itself. The result is the shape of the whole design: one small strongly consistent core (seat inventory) surrounded by large, cheap, eventually consistent read paths.
>
> See also *50 System Designs* #25 (Ticket Booking).

External systems carry their own requirements into yours. The payment provider's availability caps checkout's availability; its webhooks arrive at least once, so Encore must process them idempotently; the e-mail and SMS providers have sending quotas that set how fast tickets can go out after a sale. The support agent adds requirements of a different kind: a model call takes seconds rather than milliseconds, its cost is counted in tokens, and its tools (look up order, request refund) must be authorized per user and safe to retry. *Designing Agentic Systems* treats those in depth; here they are rows in the same requirements table.

1.3 SLIs, SLOs and SLAs

A car has a speedometer, the road has a speed limit, and the law sets a fine. The speedometer is the **service level indicator** (SLI): a measurement. The limit you choose to drive under is the **service level objective** (SLO): an internal target. The fine is the **service level agreement** (SLA): a contract with consequences, usually service credits. Sensible drivers keep a margin below the limit; sensible teams set the SLO stricter than the SLA.

```
SLI  measured    "99.93% of holds succeeded in < 500 ms this month"
SLO  target      "99.95% of holds succeed in < 500 ms, 30-day window"
SLA  contract    "below 99.9% we refund 10% of the organizer's fee"
```

Pick the journeys that matter (view seat map, hold, pay). Define each SLI as good events over valid events, measured as close to the user as possible (at the load balancer or in the client, not inside the database). Choose a target and a window, for example 99.95% over a rolling 30 days. The gap between the target and 100% is the **error budget**: while budget remains, ship features; when it is spent, spend engineering time on reliability.

$$\text{SLI} = \frac{\text{good events}}{\text{valid events}} \qquad \text{error budget} = (1 - \text{SLO}) \times \text{valid events} \qquad \text{burn rate} = \frac{\text{bad fraction}}{1 - \text{SLO}}$$

Worked example: at Encore's average 1,500 requests/s, a 30-day window holds $1500 \times 2,592,000 \approx 3.9$ billion requests. A 99.95% SLO allows about 1.9 million bad ones. A burn rate of 1 spends the budget in exactly 30 days; a burn rate of 10 spends it in 3 days and should page someone (Chapter 23).

The SLI below counts a request as good when it did not fail on the server and finished in under 500 ms. The traffic is synthetic.

```
import random

random.seed(7)
# One minute of checkout requests: (status, latency in ms)
reqs = [(500 if random.random() < 0.0004 else 200,
         random.lognormvariate(4.6, 0.5)) for _ in range(90_000)]

def good(status, ms, threshold_ms=500):
    return status < 500 and ms < threshold_ms

sli = sum(good(s, ms) for s, ms in reqs) / len(reqs)
slo = 0.9995
budget = (1 - slo) * len(reqs)          # bad requests allowed
spent = sum(not good(s, ms) for s, ms in reqs)
print(f"SLI {sli:.5f}  budget {budget:.0f}  spent {spent}")
print(f"burn rate {spent / budget:.2f}")  # 1.0 = exactly on budget
```

It prints `SLI 0.99911  budget 45  spent 80` and a burn rate of 1.78: this minute consumed budget almost twice as fast as the SLO allows. In production the inputs come from metrics. With Prometheus histograms, the p99 latency of the last five minutes is:

```
histogram_quantile(0.99,
  sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
```

Amazon CloudWatch Application Signals (Google Cloud Monitoring, Azure Monitor) defines SLOs on latency and availability metrics and tracks the remaining budget; Prometheus with Grafana, Datadog and Honeycomb do the same from your own metrics. Cloud SLAs cap what you can promise on top of them: DynamoDB commits to 99.99% monthly uptime for standard tables and 99.999% for global tables; the S3 SLA pays service credits when monthly uptime drops below 99.9%. An SLA is a refund policy, not a guarantee: your design must survive the months it is missed.

Every extra nine costs roughly an order of magnitude more effort: more replicas, more regions, slower and more careful releases. Set SLOs from what users notice, not from what is technically possible, and set them per journey. Encore's checkout deserves 99.95%; the organizer dashboard can live with 99.5%.

1.4 Availability in nines

A shop that promises to be open "99.9% of the time" may be closed almost nine hours a year. Translating nines into minutes makes the promise concrete, and shows why the fourth and fifth nines are a different kind of engineering: at 99.999% there is no time for a human to wake up, log in and fix anything.

$$\text{downtime} = (1 - A) \times T$$

Availability	Downtime per year	Per 30-day month
99%	3.65 days	7.2 hours
99.9%	8.76 hours	43.2 minutes
99.95%	4.38 hours	21.6 minutes
99.99%	52.6 minutes	4.3 minutes
99.999%	5.3 minutes	26 seconds

Components combine in two ways. A request that needs every component in a chain (serial) is only as available as the product; a request that can use any one of several redundant copies (parallel) fails only when all fail:

$$A_{\text{serial}} = \prod_i A_i \qquad A_{\text{parallel}} = 1 - \prod_i (1 - A_i)$$

```
from math import prod

serial = prod([0.9999, 0.9995, 0.999])   # gateway, inventory, payments
parallel = 1 - prod([1 - 0.999] * 2)     # two independent regions
print(f"serial {serial:.4%}  parallel {parallel:.4%}")
print(f"{(1 - 0.9995) * 365 * 24 * 60:.0f} min/yr down at 99.95%")
```

Encore's checkout passes through the gateway (99.99%), seat inventory (99.95%) and the payment provider (99.9%), all illustrative: the chain gives 99.84%, below the 99.95% checkout target before Encore writes a line of its own code. Two fixes follow from the equations: remove components from the critical path (issue tickets asynchronously instead of inside the checkout call), or make them parallel (a second payment provider as a fallback). Two regions at 99.9% each would give 99.9999% only if their failures were independent; shared deployments, shared configuration and shared DNS make them correlated, so treat the parallel formula as an upper bound (Chapter 23).

1.5 The read/write ratio

A library lends thousands of books a day and buys a few dozen. It puts popular titles on open shelves near the door and keeps one careful catalogue. Systems work the same way: the ratio of reads to writes decides whether the design is about copying data close to readers (caches, replicas, CDNs) or about absorbing and ordering writes (partitions, queues, logs).

Compute the ratio per service, not for the whole system, and compute it for the peak, not the average. Then let it pick the tools:

- **Read-heavy (100:1 and up):** cache aggressively, add read replicas, push static content to a CDN, accept some staleness.
- **Balanced:** the database must handle both; indexes help reads and slow writes, so choose them with care.
- **Write-heavy:** append to a log or queue, batch, partition by key, read later from a derived store.

$$\text{read/write ratio} = \frac{\text{reads per second}}{\text{writes per second}}$$

Encore service	Reads/s	Writes/s	Ratio	Shape
Whole site, normal day	1,500	about 1.6 orders	about 950:1	cache and CDN
Seat map, flash sale	200,000	30,000 holds	about 7:1	hot key, contention
Clickstream	batch only	11,574 events	write-only	append to a log

The first row comes from 50 million orders a year divided by 31.5 million seconds. The flash-sale row is the surprise: the ratio collapses from about 950:1 to about 7:1 on a single event, and all 30,000 holds per second compete for the same 50,000 rows. That is a contention problem, not a capacity problem, and no cache solves it (Chapter 3, Chapter 17).

Measure the ratio instead of guessing it. Over a request log in Postgres:

```sql
WITH counts AS (
  SELECT service,
         SUM(CASE WHEN method = 'GET' THEN 1 ELSE 0 END) AS reads,
         SUM(CASE WHEN method = 'GET' THEN 0 ELSE 1 END) AS writes
  FROM request_log
  WHERE ts > now() - INTERVAL '1 hour'
  GROUP BY service)
SELECT service, reads, writes,
       ROUND(1.0 * reads / NULLIF(writes, 0), 1) AS ratio
FROM counts
ORDER BY writes;
```

`NULLIF` turns a zero write count into `NULL`, so a read-only service shows an empty ratio instead of a division error. The same query runs in SQLite with `datetime('now', '-1 hour')` in place of the interval arithmetic.

Read-heavy paths end up in CloudFront (Cloud CDN, Azure Front Door), ElastiCache for Valkey or Redis OSS (Memorystore, Azure Managed Redis) and Aurora or RDS read replicas; write-heavy paths in Kafka, Kinesis (Pub/Sub, Event Hubs) and DynamoDB. The trade-off is staleness against cost: every copy that makes reads cheap is a copy that can be out of date.

1.6 Back-of-the-envelope estimation

Before a road trip you estimate: 600 km at 100 km/h is six hours, plus two stops, so leave at eight. Nobody needs the exact minute; the estimate decides whether to drive or fly. Back-of-the-envelope estimation does the same for a system: within a factor of two or three, is this one server, ten, or a thousand? Is the data a gigabyte (fits in memory) or a petabyte (needs a distributed store)?

Write down the inputs (users, actions per user, object sizes), convert them to rates per second (a day is 86,400 seconds, about 10^5; a year about 3.15×10^7), multiply by a peak factor, then turn rates into storage, bandwidth and memory. Round hard and compare each result with what one machine does (Chapter 2).

$$\mathrm{QPS}_{\mathrm{avg}} = \frac{\mathrm{DAU} \times \text{requests per user per day}}{86,400} \qquad \mathrm{QPS}_{\mathrm{peak}} = \mathrm{QPS}_{\mathrm{avg}} \times \text{peak factor}$$

$$\begin{aligned} \text{storage per year} &= \text{items per year} \times \text{bytes per item} \times \text{replicas} \\ \text{bandwidth} &= \text{requests/s} \times \text{bytes per response} \\ \text{cache memory} &= \text{hot items} \times \text{bytes per item} \times \text{overhead} \end{aligned}$$

Applied to Encore (inputs from the running example; the 10 KB event record and the overhead factor of 2 are illustrative assumptions):

Quantity	Calculation	Result
Average load	4 M DAU x 32 requests / 86,400 s	about 1,500/s
Daily peak	1,500/s x peak factor 4	6,000/s
Orders	50 M x 2 KB x 3 replicas	300 GB a year
Ticket PDFs	50 M x 50 KB (S3 replicates)	2.5 TB a year
Seat state, upper bound	300,000 events x 80,000 seats x 100 B	2.4 TB a year
Event pages, flash sale	1 M fans / 600 s x 200 KB	2.7 Gbit/s at the CDN
Catalog cache	300,000 events x 10 KB x 2	6 GB of RAM

Read the results as decisions. Orders fit in one relational database for years. Ticket PDFs belong in object storage. The catalog fits in one cache node. The event-page bandwidth is a CDN's job, not the origin's. The only number that looks hard is the flash sale.

Keep the estimate in a script so that changing an assumption updates every figure:

```python
DAY, YEAR = 86_400, 31_536_000
KB, MB, GB, TB = 1e3, 1e6, 1e9, 1e12

def qps(dau, requests_per_user, peak_factor=1):
    return dau * requests_per_user / DAY * peak_factor

def storage_per_year(items_per_year, item_bytes, replicas=1):
    return items_per_year * item_bytes * replicas

def bandwidth_gbps(rps, response_bytes):
    return rps * response_bytes * 8 / 1e9

print(f"average {qps(4e6, 32):,.0f}/s  peak {qps(4e6, 32, 4):,.0f}/s")
print(f"orders  {storage_per_year(50e6, 2 * KB, 3) / GB:,.0f} GB/yr")
print(f"tickets {storage_per_year(50e6, 50 * KB) / TB:,.1f} TB/yr")
print(f"seats   {storage_per_year(300_000 * 80_000, 100) / TB:,.1f} TB/yr")
print(f"pages   {bandwidth_gbps(1e6 / 600, 200 * KB):,.1f} Gbit/s")
print(f"map     {bandwidth_gbps(200_000, 50_000 / 8):,.0f} Gbit/s")
print(f"cache   {300_000 * 10 * KB * 2 / GB:,.0f} GB")
```

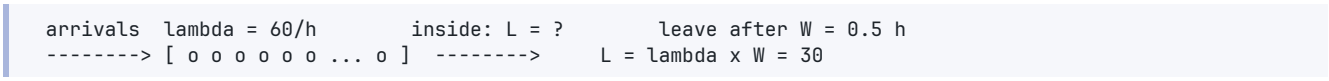
It prints about 1,481/s average and 5,926/s peak (round to 1,500 and 6,000), 300 GB, 2.5 TB, 2.4 TB, 2.7 Gbit/s, 10 Gbit/s and 6 GB. Bits versus bytes is the classic slip: network links are sold in bits per second, so multiply bytes by 8.

> **Design question.** How much bandwidth and memory does Encore's seat map need during the flash sale?
>
> 200,000 reads/s means each of 1,000,000 fans refreshes about every 5 seconds. Sent as JSON at an illustrative 20 bytes per seat, one map of 50,000 seats is 1 MB, and the total is 200 GB/s (1.6 Tbit/s): impossible from an origin and expensive even from a CDN. Encode availability as a bitmap instead, one bit per seat: 6.25 KB per map and 1.25 GB/s (10 Gbit/s) in total, cached at the edge for one second. Memory is trivial: 50,000 seat rows of 100 bytes are 5 MB, so the authoritative state for the hottest event fits in one Redis node. The bottleneck is therefore neither bandwidth nor memory but the 30,000 hold attempts per second on one event, which the waiting room must smooth.
>
> See also *50 System Designs* #25 (Ticket Booking).

Estimates are only as good as their assumptions, so state each one aloud and keep the arithmetic in powers of ten. Precision is wasted effort: whether the answer is 5,926 or 6,000 changes nothing, but a forgotten factor of 8 or 1,000 changes the design.

1.7 Little's law

A café serves 60 customers an hour and each stays 30 minutes, so on average 30 people are inside. You never counted them; arrival rate times time spent gives the crowd. That is **Little's law**, and it holds for any stable system regardless of how arrivals or service times are distributed: requests in a server, connections in a pool, messages in a queue, fans in a waiting room.



$$L = \lambda W$$

L is the average number in the system, λ the average arrival (equals departure) rate, W the average time each item spends inside. Rearranged, it sizes everything that holds work in progress:

- In-flight requests at Encore's daily peak: $6,000/s \times 50$ ms = 300 concurrent requests, spread across the API fleet.
- Seat-map reads in the flash sale: $200,000/s \times 20$ ms = 4,000 in flight at once.
- Database connections for holds: $30,000/s \times 2$ ms per transaction = 60 busy connections, if every hold is a short transaction (latencies illustrative).

When *W* grows because a dependency slows down, *L* grows with it at the same λ: a 2 ms transaction that becomes 200 ms needs 6,000 connections, which no pool has. This is why slow dependencies exhaust thread pools and connection pools, and why timeouts matter (Chapter 11).

A simulation with random arrivals and random stays confirms the law without assuming it:

```
import bisect
import random

random.seed(1)
lam, mean_w, t, visits = 200.0, 0.05, 0.0, []  # arrivals/s, seconds
while t < 600:                                 # ten simulated minutes
    t += random.expovariate(lam)               # Poisson arrivals
    visits.append((t, random.expovariate(1 / mean_w)))  # (arrive, stay)
starts = [a for a, _ in visits]

def inside(now):          # count visitors present at one instant
    lo = bisect.bisect_left(starts, now - 2)   # stays are far below 2 s
    hi = bisect.bisect_right(starts, now)
    return sum(a + w > now for a, w in visits[lo:hi])

snapshots = [inside(random.uniform(10, 590)) for _ in range(20_000)]
W = sum(w for _, w in visits) / len(visits)
print(f"counted L {sum(snapshots) / len(snapshots):.2f}"
      f"  lambda*W {len(visits) / t * W:.2f}")
```

It counts the people inside at 20,000 random instants and prints `counted L 10.00  lambda*W 10.03`. Little's law gives averages only; for how waiting time explodes as utilization approaches 100%, see the queueing section of Chapter 2.

> **Design question.** How many fans per second should Encore's waiting room admit during the flash sale?
>
> Start from what the core can take, not from the crowd. Suppose seat inventory was load-tested at 6,000 hold attempts/s and runs at half that, 3,000/s, and an admitted fan makes about 3 hold attempts in a 2-minute seat-picking session, one every 40 seconds (all illustrative). Then the flow can hold $L = 3,000 \times 40 = 120,000$ shoppers at once. With a session of $W = 120$ s, Little's law gives the admission rate $\lambda = L/W = 120,000/120 = 1,000$ fans/s. Draining 1,000,000 fans at that rate takes about 17 minutes, far longer than 50,000 seats last, so the waiting room must also tell queued fans the moment the event sells out. The trade-off: a slower admission rate protects inventory and makes the sale fair, at the price of a long, visible queue; Chapter 11 builds the waiting room itself.
>
> See also *50 System Designs* #25 (Ticket Booking) and #34 (E-Commerce Checkout and Inventory).

1.8 The design method

Requirements and numbers feed a fixed sequence of steps. Each step narrows the next, the way a house goes from brief to floor plan to wiring. The same sequence structures a 45-minute interview (Chapter 27) and a full design document (Chapter 25).

```
requirements -> estimates -> API -> data model -> high-level design
     |                                                    |
     +---- trade-offs <-------- deep dives <-------------+
```

1. **Requirements:** core flows, non-functional targets with numbers, out of scope.
2. **Estimates:** peak QPS per flow, read/write ratio, storage, bandwidth, memory.
3. **API:** the few calls that carry the core flows (Chapter 5).
4. **Data model:** the entities, their keys and which store owns each (Chapter 12).
5. **High-level design:** boxes and arrows for the request path of each core flow.
6. **Deep dives:** the one or two places where the numbers say the design is hard.
7. **Trade-offs:** what each choice gives up, and when you would choose otherwise.

For Encore, steps 3 and 4 fit on a page. The hold call carries an idempotency key so that a retried request cannot create two holds:

```
POST /v1/events/42/holds HTTP/1.1
Idempotency-Key: 5b0c2e7a-9f1d-4c55-8a1e-2f6b1d0c9e44

{"seat_ids": ["A-12-7", "A-12-8"]}

HTTP/1.1 201 Created

{"hold_id": "h_91", "expires_at": "2026-10-14T10:05:00Z"}
```

The seat row is the unit of contention, and a conditional update makes the hold atomic without a separate lock (tested on SQLite with `datetime('now', '+10 minutes')`; the statement below is Postgres syntax):

```
CREATE TABLE seats (
  event_id   BIGINT NOT NULL,
  seat_id    TEXT   NOT NULL,           -- 'A-12-7': section, row, seat
  status     TEXT   NOT NULL DEFAULT 'free',  -- free, held, sold
  hold_id    TEXT,
  held_until TIMESTAMPTZ,
  PRIMARY KEY (event_id, seat_id)
);

-- A hold succeeds only if the seat is free or its hold has expired.
UPDATE seats
SET status = 'held', hold_id = 'h_91',
    held_until = CURRENT_TIMESTAMP + INTERVAL '10 minutes'
WHERE event_id = 42 AND seat_id = 'A-12-7'
  AND (status = 'free'
       OR (status = 'held' AND held_until < CURRENT_TIMESTAMP));
```

The first caller updates one row; a second caller updates zero rows and gets “seat taken”. Step 5 then draws the path, and the estimates mark the hot spot:

```
 fans -> CDN (event pages, seat-map bitmap, 1 s TTL)
      -> waiting room (admits about 1,000 fans/s)
      -> API gateway -> seat inventory (3,000 holds/s) -> Postgres + Redis
                     -> orders -> payment provider (idempotency keys)
                     -> queue -> notification workers -> e-mail, SMS, push
```

Step 6 goes where the numbers point: the flash sale (waiting room, holds that never oversell, the hot partition for one event). Everything else is standard parts, and saying so is part of a good design.

1.9 Numbers to remember

Figure	Value
Seconds in a day / a year	86,400 (about 10^5) / about 3.15×10^7
1 request/s sustained	about 86,000 a day, 31.5 million a year
99.9% / 99.99% / 99.999%	8.8 h / 53 min / 5.3 min of downtime a year
99.9% per 30-day month	43 minutes
Little’s law	$L = \lambda W$
Encore normal load	1,500/s average, 6,000/s peak (factor 4)
Encore flash sale	200,000 seat-map reads/s, 30,000 holds/s
Encore storage per year	orders 100 GB (300 GB with 3 replicas), PDFs 2.5 TB, seats at most 2.4 TB
Bytes to bits	multiply by 8

1.10 Pitfalls

- Designing for the average: size for the peak; Encore's flash-sale seat-map reads alone are 33 times the whole site's daily peak.
- Targets without numbers ("fast", "scalable"): write a percentile, a rate or a ratio for each.
- One consistency level for the whole system: decide per operation; keep the strongly consistent core small.
- Averages for latency: use p99 and remember that fan-out multiplies tail latency.
- Treating cloud SLAs as guarantees: they are refund policies; design for the months they are missed.
- Multiplying nines of correlated components as if independent: shared deployments, configuration and DNS fail together.
- Exact arithmetic: round to powers of ten and spend the time on the bottleneck instead.
- Skipping scope: name what is out of scope before drawing boxes.

1.11 Remember

- Functional requirements say what the system does; non-functional ones say how well, and each needs a number.
- SLI measures, SLO targets, SLA contracts; the error budget is 1 − SLO.
- Availability is downtime per year: 99.9% is about 8.8 hours, 99.99% about 53 minutes.
- Estimate QPS, storage, bandwidth and memory in powers of ten, at the peak.
- Little's law, $L = \lambda W$, sizes pools, fleets, queues and waiting rooms.
- Follow the method: requirements, estimates, API, data model, high-level design, deep dives, trade-offs.

2 The machine under the code

Every box on a system design diagram is a real machine: cores that run a few billion steps a second, a ladder of memories where each rung is ten to a thousand times slower than the one above, an operating system that turns a socket into a file descriptor, and a queue in front of every resource. This chapter answers one question: *what can one machine really do, and why does it stop doing it well before 100 percent busy?* Hold one picture in mind: a request is a chef's order, and almost all of its time is spent walking to fetch ingredients or waiting in line, not cooking.

2.1 The picture

A core is a chef. Ingredients in the chef's hands (registers) cost nothing to use; the cutting board and the counter (L1 and L2 cache) are a second or a few away; the shared shelf (L3) is about twenty seconds away; the walk-in fridge (RAM) is close to two minutes. Scale one nanosecond to one second and the same walk becomes absurd: a read from an NVMe SSD is a day trip, a round trip to another server in the same data center is a week, a hard-disk seek is three months, and a round trip from Frankfurt to Virginia is nearly three years. A design is fast when it keeps the chef at the counter.

```
rung                 latency     size (2026 server)   at 1 ns = 1 s
-------------------  ----------  -------------------  -------------
registers            < 1 ns      ~ 1 KiB per core     instant
L1 cache             ~ 1 ns      32-64 KiB per core   1 second
L2 cache             ~ 4 ns      1-2 MiB per core     4 seconds
L3 cache             10-40 ns    32-500 MiB/socket    ~ 20 seconds
DRAM                 ~ 100 ns    64 GiB - 4 TiB       ~ 2 minutes
NVMe SSD (4 KiB)     20-100 us   1-30 TiB             ~ 1 day
network, same zone   ~ 0.5 ms    -                    ~ 6 days
HDD seek             5-10 ms     up to ~ 30 TB        ~ 3 months
network, FRA-IAD     ~ 90 ms     -                    ~ 3 years
       ^ faster, smaller, dearer per byte
       v slower, bigger, cheaper per byte
```

Each rung exists because the one above it is too small and the one below it too slow. Caches, CDNs, replicas and sharding in later chapters are the same idea applied to whole machines: keep hot data on a faster rung, close to the work.

2.2 Cores, clocks and pipelines

2.2.1 The picture

A pipeline is an assembly line: while one instruction is being executed, the next is being decoded and a third is being fetched. A modern core runs several such lines side by side (it is *superscalar*), reorders instructions to skip past ones that are waiting for memory (*out-of-order execution*), and guesses which way each `if` will go (*branch prediction*) so the line never stalls.

```
cycle:      1      2      3      4      5      6
instr A   fetch  decode  exec   mem    write
instr B          fetch  decode  exec   mem    write
instr C                 fetch  decode  exec   mem
```

2.2.2 Mechanism

- **Clock.** Server cores run at roughly 2 to 4 GHz, so one cycle is 0.25 to 0.5 ns. Clock speeds stopped climbing in the mid-2000s, when power and heat ran out; chips got more cores instead. That is why software got concurrent (Chapter 3) and then distributed.
- **Instructions per cycle (IPC).** A wide core can retire four or more instructions per cycle when data is in cache, and well under one when it keeps waiting for DRAM. Memory, not arithmetic, usually decides IPC.

- **Branch misprediction.** A wrong guess flushes the pipeline and costs about 10 to 20 cycles (a few nanoseconds). Hot loops over predictable, sorted data run faster for this reason.
- **SIMD.** Vector instructions (AVX-512 on x86, NEON and SVE on Arm) process 4 to 16 numbers per instruction; they power compression, hashing, JSON parsing and columnar analytics.
- **Hyperthreads and vCPUs.** Simultaneous multithreading (SMT) runs two hardware threads on one core and shares its execution units. On Intel and AMD cloud instances one vCPU is one hardware thread (half a core); on AWS Graviton, which has no SMT, one vCPU is a full core.

2.2.3 Numbers

The compute capacity of a server, and the cores a service needs, follow from the CPU time each request burns:

$$\text{instructions/s} = \text{cores} \times f_{\text{clock}} \times \text{IPC} \qquad \text{cores needed} = \frac{\lambda \times s_{\text{cpu}}}{\rho_{\text{target}}}$$

Here λ is the request rate, s_{cpu} the CPU time per request and ρ_{target} the utilization you are willing to run at. Sixteen cores at 3 GHz and an IPC of 2 execute about 96 billion instructions per second. Encore's API tier at its daily peak of 6,000 requests/s, at an illustrative 2 ms of CPU per request, keeps 12 cores busy; at a 60 percent utilization target it needs 20 cores, for example three 8-vCPU Graviton instances. The section on queueing explains the 60 percent.

2.2.4 Where it runs

Cloud servers are Intel Xeon, AMD EPYC or Arm: AWS Graviton4 (Arm Neoverse V2), with Google Axion and Microsoft Azure Cobalt as the GCP and Azure Arm counterparts. Instance families encode the ratio of memory to cores: on AWS, `c` (compute, about 2 GiB per vCPU), `m` (general, 4 GiB) and `r` (memory, 8 GiB). Arm instances usually cost less per unit of work but need multi-architecture container images. A single-threaded server such as Redis prefers a few fast cores; a Postgres or a JVM service with many busy connections prefers many cores.

2.3 Caches and the memory hierarchy

Memory moves in **cache lines** of 64 bytes (128 on Apple's M-series). Touch one byte and the whole line comes in; the next 63 bytes are then free. Two habits of programs make caches work: *temporal locality* (what you used, you will use again soon) and *spatial locality* (what is next to it, you will use next). Hardware prefetchers notice sequential or strided walks and fetch lines before you ask. Random pointer chasing defeats them, which is why an array beats a linked list, a B-tree node is sized to a page, and analytics stores data by column.

On a multicore chip each core has private L1 and L2 caches and shares an L3. A coherence protocol (MESI and its variants) keeps copies consistent: when one core writes a line, other copies are invalidated. Two threads writing *different* variables that happen to share a line bounce it between cores at L3 speed; this **false sharing** can make a counter per thread slower than one shared counter until each is padded to its own line.

The average cost of a memory access is the hit time plus the miss rate times the miss penalty, the same equation that governs every cache in this book:

$$t_{\text{avg}} = t_{\text{hit}} + m \times t_{\text{miss}}$$

With an L3 hit at 20 ns and a DRAM penalty of 100 ns, a 1 percent miss rate gives 21 ns, 5 percent gives 25 ns, and 20 percent gives 40 ns. Small changes in the miss rate move the average far more than a faster hit. The program below does identical work twice, reading a 128 MB array once in order and once in a random order:

```python
# Same work, different access order: sequential vs random.
import random
import time
from array import array

N = 16_000_000                      # 16 M x 8-byte ints = 128 MB
data = array("q", range(N))         # one contiguous block of memory
seq = array("q", range(N))          # visit 0, 1, 2, ...
rnd = array("q", range(N))
random.Random(42).shuffle(rnd)      # visit in random order


def walk(order: array) -> float:
    start = time.perf_counter()
    total = 0
    for i in order:
        total += data[i]
    return time.perf_counter() - start


t_seq, t_rnd = walk(seq), walk(rnd)
print(f"sequential {t_seq / N * 1e9:5.1f} ns per element")
print(f"random     {t_rnd / N * 1e9:5.1f} ns per element")
print(f"random is {t_rnd / t_seq:.1f}x slower")
```

On the Arm Neoverse V2 server (the core inside Graviton4) used to test this chapter it printed 63 ns against 167 ns per element: random order is 2.7 times slower for the same arithmetic. The interpreter's own cost per step hides most of the gap; compiled code shows a far larger one. You can read your own machine's ladder from Linux:

```bash
nproc                                # cores (vCPUs) visible to you
getconf PAGESIZE                     # virtual memory page, bytes
getconf LEVEL1_DCACHE_LINESIZE       # cache line, bytes
cd /sys/devices/system/cpu/cpu0/cache
for i in index*; do
  echo "L$(cat $i/level) $(cat $i/type) $(cat $i/size)"
done
```

The test machine reports 8 vCPUs, 4,096-byte pages, 64-byte lines, 64 KiB of L1 data cache, 2 MiB of L2 and 36 MiB of shared L3.

Where it runs. Redis stores small hashes, lists and sorted sets in compact contiguous encodings (listpacks) rather than pointer-linked nodes, partly for cache locality. Kafka, LSM-tree engines and Parquet files win by turning random access into sequential scans. The same ladder repeats one level up: the Postgres buffer pool and the OS page cache are caches of disk in RAM, and Redis and CDNs are caches of databases and origins.

2.4 Latency numbers every engineer should know

These figures are orders of magnitude for 2026 server hardware and cloud networks, in the spirit of the list Jeff Dean and Peter Norvig popularised around 2010. Memorise the exponent, not the digits; the ratios have stayed stable while the absolute numbers drifted.

Operation	Typical (2026)	Order
L1 cache hit	1 ns	10^0 ns
L2 cache hit	4 ns	10^0 ns
Uncontended mutex lock and unlock	20 ns	10^1 ns
L3 cache hit	10-40 ns	10^1 ns
DRAM access	100 ns	10^2 ns
Trivial system call	0.1-0.5 us	10^2 ns
Compress 1 KB (LZ4, Snappy)	1-2 us	10^3 ns

Operation	Typical (2026)	Order
Thread context switch	1-5 us	10^3 ns
Read 1 MB sequentially from RAM	10-50 us	10^4 ns
Random 4 KiB read, local NVMe SSD	20-100 us	10^4 ns
Round trip within one zone	0.1-0.5 ms	10^5 ns
Read 1 MB sequentially from NVMe	0.15-0.3 ms	10^5 ns
Round trip between zones in a region	0.5-2 ms	10^6 ns
Random I/O on network block storage (EBS gp3)	~1 ms	10^6 ns
HDD seek	5-10 ms	10^7 ns
Round trip Frankfurt to Virginia	~90 ms	10^8 ns
Round trip California to Europe	~150 ms	10^8 ns

Four lessons fall out of the table. A DRAM access is about a thousand times faster than an SSD read, and an SSD read is about a hundred times faster than a disk seek. A network round trip inside a zone costs about as much as several SSD reads, so a remote cache only pays when it saves a slower call. A cross-region round trip costs about a hundred million L1 hits; never put one inside a loop. And the speed of light sets a floor: light in fibre covers about 200 km per millisecond, so Frankfurt to Virginia (about 6,500 km) can never round-trip in less than about 65 ms; Chapter 4 does the full latency maths.

> **Design question.** Encore's seat-map service makes five sequential calls to render one page: catalog, seat inventory, prices, the user's holds and the waiting-room status. Each is a same-zone round trip of 0.5 ms plus 1 ms of work. Is that acceptable?
>
> Sequential calls add: $5 \times 1.5 = 7.5$ ms before any database time, and each extra hop adds its own tail latency. Three fixes, in order of value: issue the independent calls in parallel (the page then costs about one call, 1.5 ms, plus the slowest one's tail); batch what one service owns into a single request; and cache the read-mostly parts (catalog, prices) in process memory, turning a 0.5 ms round trip into a 100 ns lookup. Never let a cross-region call into this path: one 90 ms round trip costs more than the whole budget. The trade-off of in-process caches is staleness, bounded by a short TTL (Chapter 9).

2.5 Storage and network: RAM, SSD, HDD and the wire

Think of four ways to fetch a book. RAM is the book on your desk. An SSD is a library with thousands of clerks: one request takes a while, but many clerks work at once. A hard disk is one clerk on roller skates who must skate to the shelf (seek) and wait for the shelf to spin round (rotation). The network is a courier to another building: quick to start, but every trip has a fixed cost.

```
          latency     parallelism          random 4 KiB    sequential
RAM       ~100 ns     many channels        tens of M/s     10s-100s GB/s
NVMe SSD  20-100 us   64K queues x 64K     100K-1M+ IOPS   3-14 GB/s
EBS gp3   ~1 ms       network-attached     3K-80K IOPS     0.125-2 GiB/s
HDD       5-10 ms     one arm              100-200 IOPS    ~200-280 MB/s
NIC       ~0.1 ms RTT many flows           -               10-200 Gbps
```

SSDs. Flash is read and written in pages of a few KiB but erased only in large blocks, so the drive's firmware (the flash translation layer) writes new data elsewhere and garbage-collects old blocks in the background. That makes writes cheaper than they look but adds internal *write amplification* and occasional latency spikes during garbage collection. **NVMe** drives sit on PCIe (about 7 GB/s for a PCIe 4.0 x4 drive, about 14 GB/s for PCIe 5.0) and expose up to 65,535 I/O queues, each up to 65,536 commands deep, so every core can submit I/O without a lock. An SSD reaches its rated IOPS only when many requests are in flight.

Hard disks. A 7,200 rpm disk needs on average half a revolution, 4.2 ms, before the sector arrives, plus a 4 to 8 ms seek. Random IOPS are therefore a hundred or two; sequential throughput is fine. Disks survive where bytes must be cheap and reads are large and sequential: backups, cold object storage, logs.

Network block storage. A cloud volume such as Amazon EBS (Google Persistent Disk and Hyperdisk, Azure Managed Disks) is a disk on the other end of the data-center network, replicated inside its availability zone. It survives the loss of the instance; local instance-store NVMe is far faster but disappears when the instance stops. As of October 2026 an EBS gp3 volume includes 3,000 IOPS and 125 MiB/s, and can be provisioned up to 80,000 IOPS and 2,000 MiB/s; io2 Block Express targets an average under 500 us per 16 KiB I/O.

Three equations size any storage. Little's law (Chapter 1) links IOPS to the number of I/Os in flight (the queue depth Q) and the latency per I/O; throughput is IOPS times the I/O size; and a disk's random IOPS are the inverse of its mechanical delay:

$$\text{IOPS} = \frac{Q}{t_{\text{io}}} \qquad \text{throughput} = \text{IOPS} \times \text{I/O size} \qquad \text{IOPS}_{\text{HDD}} \approx \frac{1}{t_{\text{seek}} + t_{\text{rotation}}}$$

At 80 us per read, an NVMe drive at queue depth 1 delivers only 12,500 IOPS; at queue depth 32 it can approach 400,000, if the drive is rated that high. A gp3 volume at 80,000 IOPS of 16 KiB pages moves 1,250 MiB/s, inside its 2,000 MiB/s cap. A disk with a 6 ms seek and 4.2 ms rotation manages about 98 random IOPS.

Durability has a price. A `write` lands in the OS page cache and returns at memory speed; only `fsync` waits until the device confirms the data is stored. Databases call it on every commit (Postgres flushes its write-ahead log; Redis does when `appendfsync always` is set), so commit latency is storage latency:

```
# What durability costs: buffered writes vs writes forced to the device.
import os
import time

BLOCK = b"x" * 4096
N = 500


def run(path: str, sync: bool) -> float:
    fd = os.open(path, os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o644)
    start = time.perf_counter()
    for _ in range(N):
        os.write(fd, BLOCK)
        if sync:
            os.fsync(fd)        # wait until the device says it is stored
    elapsed = time.perf_counter() - start
    os.close(fd)
    os.remove(path)
    return elapsed / N * 1e6


print(f"write only     {run('t.bin', False):8.1f} us per 4 KiB")
print(f"write + fsync  {run('t.bin', True):8.1f} us per 4 KiB")
```

On the EBS-backed test machine a buffered write took 1.8 us and a write plus `fsync` 1.8 to 2.6 ms, a thousand times more. A single writer that fsyncs each commit is capped at a few hundred commits per second, which is why databases use *group commit*: one flush makes many transactions durable together.

Trade-offs. Choose local NVMe for caches, replicas and stores that replicate themselves (Cassandra, Kafka, Redis replicas); choose network volumes for a primary database that must survive its instance; choose object storage (S3, Google Cloud Storage, Azure Blob Storage) for large immutable files such as Encore's 50 KB ticket PDFs.

2.6 The operating system: processes, threads and system calls

The kernel is the building manager; programs are tenants. A tenant cannot touch the wiring (devices, other tenants' memory) and must ask the manager through a hatch, the **system call**: `read`, `write`, `send`, `fsync`, `mmap`. Crossing the hatch costs about 0.1 to 0.5 us before any work is done, so fast servers batch work into fewer, larger calls.

- **Process.** A running program with its own virtual address space, its own file descriptors and its own security identity. Processes are isolated: one crashing does not corrupt another. Postgres runs one process per client connection, which is why its `max_connections` defaults to 100 and why production deployments put a pooler (PgBouncer, or RDS Proxy on AWS) in front.
- **Thread.** A path of execution inside a process: it shares the heap and file descriptors with its siblings and has its own stack (8 MiB of reserved virtual memory by default on Linux, of which only touched pages use RAM) and registers. Threads are cheap to create but share everything, which is the source of every race in Chapter 3.
- **Scheduler.** The kernel time-slices runnable threads onto cores (Linux uses the EEVDF scheduler since kernel 6.6). A context switch costs 1 to 5 us directly and more indirectly, because the next thread finds the caches cold. Ten thousand threads, each blocked on a socket, spend more time being switched than working.

2.7 Files, sockets and epoll

Everything a Linux process talks to is a **file descriptor** (fd), a small integer: a file, a pipe, a TCP connection, a timer. Every open connection costs one fd at each end, and each process has a limit (`ulimit -n`; often 1,024 by default for interactive shells, raised to hundreds of thousands or more for servers). A gateway holding 100,000 WebSockets needs 100,000 fds; the error "too many open files" means the limit was hit.

The **page cache** is the kernel's cache of file contents in otherwise free RAM. A second read of a file is a memory read; a write marks a page dirty and is flushed later (or at `fsync`). Kafka leans on it deliberately: it writes segments sequentially and serves consumers that keep up straight from the page cache, using `sendfile` to copy file pages to the socket without passing through user space (when TLS is off). Postgres keeps its own buffer pool (`shared_buffers`, commonly about 25 percent of RAM) on top of the page cache.

Epoll solves the ten-thousand-connections problem. Instead of one blocked thread per socket, one thread registers every socket with the kernel and asks "which are ready?"; the answer costs time proportional to the ready sockets, not to all of them (the older `select` and `poll` scan every fd on every call). It is the engine under Nginx, Redis, Envoy, Node.js, Netty and Python's `asyncio`. The BSD and macOS counterpart is kqueue; Linux's newer `io_uring` (kernel 5.1 and later) goes further, with shared submission and completion rings that batch many operations into one system call.

```
# One thread, many sockets: an echo server on the OS readiness API.
import selectors
import socket

sel = selectors.DefaultSelector()       # epoll on Linux, kqueue on macOS
server = socket.create_server(("127.0.0.1", 8007))
server.setblocking(False)
sel.register(server, selectors.EVENT_READ)
print(type(sel).__name__, "listening on 8007")

while True:
    for key, _ in sel.select():         # sleep until some fd is ready
        sock = key.fileobj
        if sock is server:              # new connection: one more fd
            conn, _ = server.accept()
            conn.setblocking(False)
            sel.register(conn, selectors.EVENT_READ)
        elif data := sock.recv(4096):   # ready, so recv never blocks
            sock.sendall(data)          # fine for small replies
        else:                           # empty read: client closed
            sel.unregister(sock)
            sock.close()
```

It printed `EpollSelector listening on 8007` and echoed 500 concurrent connections correctly from a single thread. The catch is the same as for Redis: one slow handler stalls every connection on the loop, so CPU-heavy work goes to a pool of worker threads or processes.

2.8 Virtual memory and NUMA

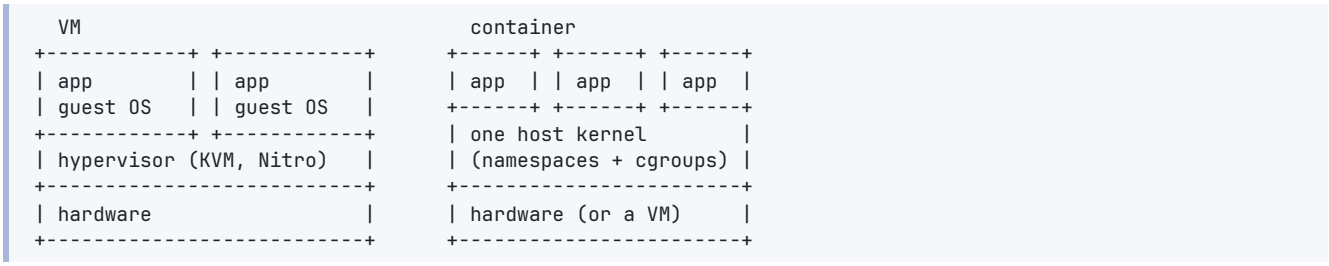
Every process believes it owns a vast, private, contiguous memory. In reality the kernel maps each 4 KiB **page** of that virtual space to a physical frame, through page tables that the CPU walks in hardware and caches in a small translation lookaside buffer (TLB). Huge pages (2 MiB or 1 GiB on x86-64) cover more memory per TLB entry and help large databases and JVMs.

Three consequences matter in design. Memory is allocated lazily, on first touch, so a process can reserve more than it uses, and when RAM truly runs out the kernel's out-of-memory killer ends the process with the highest OOM score, usually the largest. A page that must come back from swap costs a disk read, so latency-sensitive servers run with little or no swap. And `fork` shares pages copy-on-write: Redis forks a child to write an RDB snapshot, and under heavy writes each modified page is duplicated, so a Redis node can briefly need up to twice its dataset in RAM (Redis recommends `vm.overcommit_memory = 1` so that the snapshot's fork does not fail).

NUMA. A two-socket server is two computers glued together: each socket has its own memory controllers, and reaching the other socket's RAM is roughly 1.3 to 2 times slower than local RAM. The largest cloud instances span sockets; smaller ones usually sit inside one NUMA node. A database on a large instance runs best when its threads and memory stay on the same node (`numactl`, or the kernel's automatic NUMA balancing); Redis's own benchmark guide warns that results vary with process placement on multi-socket machines.

2.9 Containers and virtual machines on the hardware

A virtual machine is a house with its own foundations: a hypervisor emulates hardware, and each guest boots its own kernel. A container is an apartment in a shared building: an ordinary process on the host kernel, fenced by **namespaces** (its own view of process IDs, network, mounts and users) and metered by **cgroups** (CPU, memory and I/O limits).



- **Isolation.** A VM boundary is enforced by the CPU's virtualization extensions; a container shares the kernel, so a kernel bug can cross it. Clouds therefore run containers inside VMs: an EKS (GKE, AKS) node is an EC2 (Compute Engine, Azure) VM. Firecracker microVMs, which power AWS Lambda, boot in under 125 ms with under 5 MiB of overhead per VM; gVisor (GKE Sandbox) puts a user-space kernel between container and host.
- **Overhead.** On modern hardware a VM's CPU overhead is small; AWS Nitro moves network and storage virtualization onto dedicated cards. What you still share is the physical machine: L3 cache, memory bandwidth and the NIC, the source of *noisy neighbours*.
- **CPU limits throttle.** Linux enforces a container CPU limit as a quota per 100 ms period. A container limited to 1 CPU running 4 busy threads spends its quota in 25 ms and is frozen for the remaining 75 ms, which shows up as tail latency, not as high average CPU. Set requests for scheduling, size thread pools to the limit, and keep limits generous for latency-critical services (Chapter 15).
- **Memory limits kill.** Exceeding a cgroup memory limit invokes the OOM killer inside the container. Page cache is charged to the container as well, although the kernel reclaims it before killing anything.

2.10 Throughput, latency and Amdahl's law

Latency is how long one request takes; **throughput** is how many complete per second. A motorway lane has the same latency at 3 a.m. and at rush hour until it fills; adding lanes raises throughput but not the speed of one car. Little's law ties the two together through concurrency: in-flight requests = throughput × latency. **Batching** trades one for the

other: Postgres group commit, Kafka producer batching (`linger.ms`) and Redis pipelining raise throughput by making some requests wait a little for company.

Adding cores (or servers) speeds up only the part of the work that can run in parallel. If a fraction p parallelises perfectly over N workers and the rest is serial, **Amdahl's law** gives the speedup:

$$\text{speedup}(N) = \frac{1}{(1-p) + \frac{p}{N}} \qquad \lim_{N \to \infty} \text{speedup} = \frac{1}{1-p}$$

With 95 percent parallel work, 8 workers give 5.9 times, 16 give 9.1 times and 64 give only 15.4 times; no number of workers beats 20 times. The serial 5 percent is the lock, the single leader, the one hot row or the one Redis shard that every request touches. Gunther's Universal Scalability Law adds a coherence term for workers that must coordinate with each other, which makes throughput not just flatten but *fall* past a peak; that is what happens when hundreds of connections fight over one lock. In distributed design, Amdahl's law is the argument for partitioning work so that no single serial step sits on the hot path (Chapter 17).

2.11 Utilization and queueing

A supermarket checkout at 50 percent busy has a short line; at 90 percent the line goes out of the door, although the cashier works less than twice as hard. The cause is randomness: customers arrive in clumps and some carts are full, and an idle minute lost early is never recovered.

```
utilization   mean response time, in multiples of the service time
   50 %       2.0  ##
   60 %       2.5  ###
   70 %       3.3  ###
   80 %       5.0  #####
   90 %      10.0  ##########
   95 %      20.0  ####################
```

For one server with random (Poisson) arrivals at rate λ and exponential service of mean S, the **M/M/1** model gives the utilization, the mean response time (waiting plus service), the mean waiting time and the mean number in the system:

$$\rho = \lambda S \qquad W = \frac{S}{1-\rho} \qquad W_q = \frac{\rho S}{1-\rho} \qquad L = \frac{\rho}{1-\rho}$$

At 50 percent utilization a request takes 2 service times; at 80 percent, 5; at 90 percent, 10; at 95 percent, 20. The tail is worse than the mean: M/M/1 response time is exponentially distributed, so its 99th percentile is $\ln 100 \approx 4.6$ times the mean. With a 2 ms service time at 80 percent busy, the mean is 10 ms and the p99 is 46 ms. Variance hurts as well: for general service times (M/G/1) the waiting time scales with $(1 + C_s^2)/2$, where C_s is the coefficient of variation of service time, so a few slow requests inflate everyone's wait. The simulation below checks the formula:

```python
# Queueing in front of one server: simulate M/M/1 and compare to theory.
import random

S = 0.002                                  # 2 ms mean service time


def simulate(rho: float, n: int = 200_000, seed: int = 7) -> float:
    rng = random.Random(seed)
    lam = rho / S                          # arrivals per second
    clock = free_at = total = 0.0
    for _ in range(n):
        clock += rng.expovariate(lam)      # next arrival
        start = max(clock, free_at)        # wait if the server is busy
        free_at = start + rng.expovariate(1 / S)
        total += free_at - clock           # waiting + service
    return total / n


for rho in (0.5, 0.8, 0.9, 0.95):
    theory = S / (1 - rho)
    print(f"rho={rho:.2f}  simulated {simulate(rho) * 1e3:5.1f} ms"
          f"  theory {theory * 1e3:5.1f} ms")
```

It printed 4.0, 9.9, 20.7 and 41.4 ms against theory's 4, 10, 20 and 40 ms. **Pooling** softens the curve: with c servers sharing one queue (the M/M/c model, solved by the Erlang C formula) and the same 2 ms service time at 80 percent, 10 servers give a mean of 2.4 ms and 100 servers 2.002 ms. The 80 percent cliff is therefore sharpest for anything that is a *single* queue: one disk, one database primary's WAL, one single-threaded Redis, one hot partition, one lock. That is why services aim at 50 to 70 percent utilization at peak, why one hot key hurts more than its share of traffic suggests, and why above 100 percent ($\lambda > 1/S$) the queue grows without bound until something times out or is shed (Chapter 11).

> **Design question.** In the first minute of the flash sale, hold attempts reach 30,000/s. If each costs 2 ms of CPU in the inventory service, how many cores should be ready?
>
> Busy cores are $\lambda S = 30,000 \times 0.002 = 60$. At a 60 percent target that is 100 cores; at 80 percent, 75. The pooled stateless tier can run hot, because 75 or more cores behind one load balancer behave much like one shared queue, whose waiting time Erlang C puts at well under a millisecond. The real danger is the single queues behind it: the shard that owns the stadium event, and its lock or Lua script. Keep that path tiny (a microsecond-scale Redis operation, not a 2 ms transaction), cap admission with the waiting room so λ never exceeds what the shard can serve, and pre-scale the stateless tier before 09:55 because autoscaling reacts in minutes. See also *50 System Designs* #25 (Ticket Booking).

2.12 What a single server can do

Before adding servers, know what one does. The figures below are rough orders of magnitude for a modern cloud instance with the data in memory, gathered from vendor documentation and common practice, not benchmarks; your handler, payload and client decide the real number, so load-test your own.

One instance	Rough capacity	Limited by
Nginx, static or cached responses	10,000s-100,000s req/s	CPU, NIC
Python API worker, real handler	100s-1,000s req/s	CPU per request, GIL
Postgres, indexed reads in RAM	10,000s-100,000s queries/s	CPU, connections
Postgres, durable commits	1,000s-10,000s per second	WAL fsync, group commit
Redis, simple commands	100,000s ops/s, 1M+ pipelined	one core, network
Mostly idle TCP connections	100,000s-millions	RAM, fd limits

Redis's own benchmark page shows about 180,000 SET requests/s without pipelining and 1.5 million with a pipeline of 16, on modest hardware. The lesson of the table is that one machine is big: most services in a design are stateless copies for availability long before they are copies for capacity, and the parts that truly need distribution are the hot data paths.

> **Design question.** Could one machine hold the seat state for the 50,000-seat stadium show during the flash sale?
>
> Size first: $50,000 \times 100$ bytes is 5 MB, which fits in the L3 cache of one server. Writes: 30,000 holds/s against a Redis primary whose hold operation costs on the order of 10 us gives $\rho \approx 0.3$, comfortable for a single queue. Reads are the problem: 200,000 seat-map reads/s would saturate that one core. So do not read the primary; publish a seat bitmap every second (50,000 bits is 6,250 bytes) to replicas or the CDN, and let fans read the snapshot: 200,000 reads/s of 6.25 KB is 10 Gbps, spread over edge locations. The trade-offs: the shard is a single point of failure (a replica with automatic failover covers it, see Chapter 16) and fans see availability up to a second stale, which is fine because the hold itself is checked on the primary. See also *50 System Designs* #25 (Ticket Booking).

2.13 In Encore

- **Seat inventory** is the single queue of the system: a 5 MB working set on one Redis shard per hot event, so every operation on it must be microseconds, never a remote call inside a lock.
- **Orders** grow by $50 \times 10^6 \times 2$ KB = 100 GB a year: one Postgres primary on a network volume holds years of them, and its commit latency is the volume's `fsync`, a millisecond or two, amortised by group commit.
- **Ticket PDFs** (50 KB each) go to object storage, not database rows, so the database's RAM stays free for hot rows.
- **The API tier** is stateless and pooled, sized by $\lambda S/\rho_{\text{target}}$ at 60 percent and pre-scaled before the 10:00 sale.
- **Notification workers and the support agent** are I/O-bound: an e-mail provider call takes about 100 ms and a model call several seconds, billions of L1 hits. One event loop per process keeps thousands of such calls in flight; the queue that matters is on the provider side, which is why the model API is rate-limited by tokens and long agent jobs go onto a queue (Chapter 14; *Designing Agentic Systems* covers the agent runtime).

2.14 Numbers to remember

Figure	Value
L1, L2, L3, DRAM	1, 4, 10-40, 100 ns
Random 4 KiB read, NVMe	20-100 us
Round trip, same zone	0.1-0.5 ms
`fsync` on network block storage	1-3 ms
HDD seek	5-10 ms
Round trip, Frankfurt to Virginia	~90 ms
Cache line, page	64 B, 4 KiB
EBS gp3 IOPS, included and maximum	3,000 and 80,000
M/M/1 response at 50, 80, 90 % busy	2, 5, 10 service times
Amdahl ceiling	$1/(1-p)$
Redis simple commands, one instance	100,000s per second

2.15 Pitfalls

- Chatty calls in sequence (one request per item, five services in a row) -> batch, parallelise, or cache in process; count round trips, not lines of code.

- Running a single-queue resource (a primary, a hot shard, a lock) at 90 percent -> target 50 to 70 percent at peak and shed or queue excess load.
- Watching average CPU while containers are throttled -> alert on throttled periods and p99 latency, and size thread pools to the CPU limit.
- One thread per connection for tens of thousands of clients -> an event loop (epoll, asyncio) plus a small worker pool for CPU work.
- Assuming `write` means durable -> commit or `fsync`, and let group commit amortise it.
- Adding servers to speed up a serial step -> Amdahl caps the gain; remove the serial step by partitioning the hot key.
- Hitting "too many open files" or a Redis snapshot OOM in production -> raise fd limits for servers and leave memory headroom for copy-on-write.

2.16 Remember

- Each rung of the memory ladder is ten to a thousand times slower than the one above; keep hot data high and close.
- Memorise orders of magnitude: 100 ns RAM, 100 us SSD, 0.5 ms same-zone round trip, 10 ms disk seek, 100 ms across an ocean.
- IOPS = queue depth / latency, and durability costs an `fsync`: storage design is latency arithmetic.
- Response time grows as $S/(1-\rho)$: single queues fall off a cliff past 80 percent, pooled servers degrade more gracefully.
- Amdahl's law: the serial fraction, often one hot row or shard, sets the ceiling on scale-out.
- One machine is big: count what a single server can do before you distribute.

3 Concurrency: threads, async and race conditions

At 10:00:00 on the morning of Encore's flash sale, two fans in different cities click seat A1 in the same millisecond. Two requests land on two servers, each running dozens of threads, and both ask the same question: "is A1 free?" This chapter answers what happens next. It explains how one machine does many things at once (threads, processes, coroutines), why doing things at once breaks programs that are correct one at a time (race conditions), the tools that put order back (locks, semaphores, compare-and-swap, transactions), and how a service should wait for slow work, by blocking, by awaiting, or by answering "202, come back later".

3.1 The picture

A restaurant kitchen. The cooks are CPU cores: four cooks can chop four onions at the same instant (parallelism). The orders on the rail are tasks: one cook can have a soup simmering, bread proving and a steak resting all at once, switching between them while each one waits (concurrency). The trouble starts at the shared fridge: two cooks each see the last lobster, each promise it to a table, and one table is disappointed. That is a race condition, and the fix is the same in a kitchen and in code: one cook takes the lobster *and* marks it gone in a single step that nobody can interrupt.

```
two fans, one seat, no coordination
time  request ana (server 1)        request ben (server 2)     seat A1
----  ------------------------      ------------------------   -------
 t1   read A1 -> free                                          free
 t2                                 read A1 -> free            free
 t3   write A1 = ana, reply "yours"                            ana
 t4                                 write A1 = ben, "yours"    ben
result: two fans told "yours", one seat, ana's write lost
```

Every fix in this chapter removes the gap between the read at t1 and the write at t3: either nobody else may enter it (locking), or the write fails if the world changed inside it (compare-and-swap, optimistic concurrency), or there is only ever one writer, so the gap is harmless.

3.2 Concurrency and parallelism

Concurrency is about structure: many tasks are in progress at the same time, and their steps interleave. **Parallelism** is about execution: several steps run at the same physical instant on different cores. One core can be highly concurrent (a Redis server juggles thousands of client connections on one main thread); a parallel program needs several cores (a matrix multiply split across eight cores).

Services are mostly concurrent rather than parallel, because a request spends most of its life waiting: for the database, for Redis, for the payment provider. The amount of concurrency a server needs follows from Little's law (Chapter 1): the number of requests in flight equals arrival rate times time in the system.

$$L = \lambda W \qquad \text{CPU cores needed} = \lambda C$$

Here λ is requests per second, W the latency of one request and C the CPU time it burns. Illustrative Encore numbers: a seat-hold request lasts W = 20 ms, of which C = 2 ms is CPU and 18 ms is waiting on the database. At 30,000 holds/s the fleet has L = 600 requests in flight but needs only 60 cores of CPU. Ten times more requests are *waiting* than *running*, so the design question is how to wait cheaply, which is what threads, coroutines and async APIs are for.

3.3 Processes, threads and coroutines

- A **process** has its own address space, file descriptors and at least one thread. Processes share nothing by default, so a crash or memory bug stays inside one process; they talk through pipes, sockets or shared memory. Postgres runs one process per connection; Gunicorn and uWSGI run several Python worker processes.

- A **thread** is a schedulable stream of execution inside a process. Threads share the heap, so they communicate by reading and writing the same objects, which is fast and is also the source of every race in this chapter. The operating system schedules them preemptively: it can pause a thread between any two machine instructions.
- A **coroutine** (green thread, fiber, goroutine, virtual thread) is a task scheduled in user space by a runtime, not by the kernel. In Python's `asyncio`, a coroutine runs until it reaches `await`, then hands control back to the event loop: switching is *cooperative* and happens only at visible points.

	Process	OS thread	Coroutine
Scheduled by	kernel	kernel	runtime
Memory each	MBs	stack, ~80 KB used	~1 KB
Switch cost	microseconds	~5 us handoff	~1.6 us
Preempted	yes	yes	no (asyncio)
Practical count	tens	hundreds to thousands	100,000+

These figures are measured, not folklore: on one Linux machine with Python 3.13, 1,000 idle threads added about 84 KB of resident memory each (each also reserves 8 MB of virtual address space for its stack, the Linux default), while 100,000 idle `asyncio` tasks cost under 1 KB each; a handoff between two threads through a queue took about 5 microseconds and an `asyncio` task switch about 1.6. Go's goroutines start with a 2 KB stack that grows, and Java 21's virtual threads are cheap enough to run one per request, which gives the blocking programming style the cost profile of coroutines.

3.3.1 Python's GIL and free-threaded builds

CPython's default build has a **global interpreter lock** (GIL): only one thread executes Python bytecode at a time. Threads still help with I/O, because the GIL is released while a thread waits on a socket or a disk, and C extensions such as NumPy release it during heavy computation. They do not help pure-Python CPU work: four threads computing hashes take as long as one. For that, use processes (`multiprocessing`, `ProcessPoolExecutor`), or since Python 3.14 the `InterpreterPoolExecutor`, which runs each worker in its own sub-interpreter with its own GIL.

The GIL is going away in stages. PEP 703 introduced a **free-threaded** build (`--disable-gil`, installed as `python3.13t`, `python3.14t` and so on). It was experimental in 3.13 and, under PEP 779, became officially supported but optional in 3.14; as of Python 3.15 (October 2026) it is still not the default, and making it default is left to a future PEP. The 3.15 documentation quotes a single-threaded overhead of about 1 to 8 percent depending on platform. Check at runtime with `sys._is_gil_enabled()`.

The GIL never made your code thread-safe. It makes individual bytecodes atomic, not your read-check-write sequences, so the race in the next section happens with or without it. It does make some races *rare*, which is worse: they pass every test and appear in production, or on the day you move to a free-threaded build.

3.4 Race conditions

A **race condition** is a bug where the result depends on the timing of concurrent steps. It needs three ingredients: shared state, at least one writer, and a sequence of steps that must happen together but can be interleaved. A **data race**, the low-level cousin, is two threads touching the same memory without synchronization, at least one of them writing; in C, C++ and Go it is undefined or unsafe behaviour, and in Python it is "only" a logic bug.

Races come in two recurring shapes:

- **Lost update (read-modify-write).** Two tasks read a value, both compute a new one, both write; the second write silently erases the first. A view counter that loses increments, a balance that loses a deposit.
- **Check-then-act (time of check to time of use).** A task checks a condition, then acts on it, but the condition changed in between. "If the seat is free, hold it"; "if the file does not exist, create it"; "if the user has no refund yet, refund".

Here is the opening picture as a program. Two threads play Ana and Ben; the `sleep` stands for anything that takes time between the check and the act, such as a database round trip or a log call:

```
import threading
import time

seat = {"A1": None}               # seat id -> fan who holds it
winners = []

def hold(fan: str) -> None:
    if seat["A1"] is None:        # 1. check
        time.sleep(0.001)         # 2. gap: a DB round trip, a log call
        seat["A1"] = fan          # 3. act
        winners.append(fan)

fans = [threading.Thread(target=hold, args=(f,)) for f in ("ana", "ben")]
for t in fans: t.start()
for t in fans: t.join()
print("told they won:", winners, "| seat belongs to:", seat["A1"])
```

Every run prints `told they won: ['ana', 'ben'] | seat belongs to: ben`. Both fans saw a free seat, both were told they won, and Ana's write was lost. Remove the `sleep` and the bug does not go away, it only becomes rare: the gap shrinks from a millisecond to a few bytecodes. The same holds for `count += 1`, which compiles to a load, an add and a store; on a GIL build four threads adding a million each usually reach exactly 4,000,000, which proves nothing, and the operation is not atomic on any build.

How often does a narrow window bite? If attempts on one seat arrive at random at rate λ and the gap between check and act lasts w, the chance that another attempt lands inside the gap is

$$P(\text{collision}) = 1 - e^{-\lambda w}$$

Illustrative flash-sale numbers: suppose the best 1,000 seats each receive $\lambda = 15$ hold attempts per second in the first second and the gap is $w = 2$ ms. Then $P = 1 - e^{-0.03} \approx 3$ percent per attempt, and about 440 of those 15,000 attempts would land inside someone else's gap. A bug that a developer clicking alone will never see becomes hundreds of double-sold seats in one second.

> **Design question.** Encore's tests hold seats one at a time and never see a double booking. How do you show the hold path is race-free before the flash sale?
>
> Test the invariant, not the happy path: "for one seat, at most one hold succeeds, however many arrive". Start N threads (or N async clients against a staging API) that wait on a `threading.Barrier(N)` and then hit the same seat together, and assert exactly one success. Widen the window in a test build by injecting a delay between the read and the write; a correct design stays correct with any delay, a racy one fails at once. Run it against the real database, because the fix usually lives there. Finally keep a backstop the race cannot pass: a unique constraint on (event, seat) in the tickets table, so even a bug in the application cannot write two tickets for one seat. See also *50 System Designs* #25 (Ticket Booking).

3.5 Synchronization primitives

A **critical section** is the code between the check and the act: the steps that must run as one, with no other task interleaving on the same state. Synchronization primitives are the turnstiles at its door. Picture the single key to a bathroom on a petrol-station keyring: whoever holds the key is inside, everyone else waits.

3.5.1 Mutexes and reentrant locks

A **mutex** (mutual-exclusion lock, `threading.Lock`) lets one thread at a time hold it. Wrapping the check and the act in one lock fixes the seat race; every run now prints `told they won: ['ana'] | seat belongs to: ana`:

```
lock = threading.Lock()

def hold(fan: str) -> None:
    with lock:                    # critical section starts
        if seat["A1"] is None:
            time.sleep(0.001)
            seat["A1"] = fan
            winners.append(fan)   # critical section ends
```

A lock costs little when nobody else wants it (about 135 ns for an acquire and release of a Python `Lock`, measured; tens of nanoseconds in C or Java) and a lot when many threads want it, because they queue: a critical section of 1 ms under one lock caps that code path at 1,000 operations per second no matter how many cores you add. That is Amdahl's law (Chapter 2) applied to a lock, and why locks should be fine-grained (one per seat or per event, not one per service) and held for as short a time as possible, never across a network call if you can avoid it.

A **reentrant lock** (`threading.RLock`) can be acquired again by the thread that already holds it, which lets a locked method call another locked method on the same object; a plain `Lock` would deadlock against itself. A **read-write lock** lets many readers in together but a writer alone, which suits read-heavy data such as a configuration snapshot; Python's standard library has none, Java has `ReentrantReadWriteLock`, Go has `sync.RWMutex`.

3.5.2 Semaphores

A **semaphore** is a lock with a counter: up to N holders at once. It is the tool for "at most N concurrent calls", which is how you protect a dependency. Suppose (illustratively) Encore's payment provider accepts 100 concurrent requests per merchant and the orders service runs 5 pods. Each pod caps itself at 20:

```
pay_slots = threading.BoundedSemaphore(20)   # 5 pods x 20 = 100

def charge(order):
    with pay_slots:               # blocks while 20 calls are in flight
        return provider.charge(order)       # ~1.5 s round trip
```

Little's law gives the ceiling: 100 slots divided by a 1.5 s call is about 66 charges per second, enough for a flash sale that sells 50,000 seats in about 10 minutes at 2 seats per order (about 42 orders per second). A per-process semaphore only limits one process; a fleet-wide limit needs a shared counter, such as the Redis limiters of Chapter 11. `BoundedSemaphore` raises an error if released more times than acquired, which catches bugs a plain `Semaphore` hides. In `asyncio` the same tool is `asyncio.Semaphore`.

3.5.3 Condition variables

A **condition variable** lets a thread sleep until some state changes, without spinning. The consumer waits for "the buffer is not empty", the producer for "the buffer is not full"; `wait()` atomically releases the lock and sleeps, and `notify()` wakes a sleeper, which re-checks the condition with the lock held:

```
cond = threading.Condition()

def get(items):
    with cond:
        while not items:          # a loop, never an if: wakeups can be
            cond.wait()           # spurious or stolen by another consumer
        return items.pop(0)
```

You rarely write this yourself: `queue.Queue` is a bounded buffer built on exactly these conditions, and it is the right way to hand work between threads. Semaphores, events, barriers and queues are all built on a mutex plus conditions.

3.5.4 Atomics and compare-and-swap

Locks make others wait. **Atomic** operations avoid waiting: the CPU performs a small read-modify-write as one indivisible instruction. The most important is **compare-and-swap** (CAS): "set the value to `new` only if it still equals `expected`; tell me whether it worked". On failure you re-read and retry. x86 provides it as `LOCK CMPXCHG`, ARMv8.1 as `CAS`; Java exposes it as `AtomicInteger.compareAndSet`, C++ as `std::atomic::compare_exchange_strong`, Go as `atomic.CompareAndSwapInt64`. Python has no public atomics, so a lock stands in for the instruction here:

```
class AtomicInt:
    def __init__(self, value=0):
        self._value, self._guard = value, threading.Lock()
    def load(self):
        return self._value
    def compare_and_swap(self, expected, new):
        with self._guard:
            if self._value != expected:
                return False          # someone changed it: caller retries
            self._value = new
            return True

def increment(cell, times):
    for _ in range(times):
        while True:                   # the CAS retry loop
            old = cell.load()
            if cell.compare_and_swap(old, old + 1):
                break
```

Four threads running `increment(cell, 100_000)` end at exactly 400,000 every time. Learn this loop well: read, compute, write-if-unchanged, retry on conflict. It reappears at every scale in this chapter, as a SQL `UPDATE ... WHERE version = ?`, as Redis `WATCH`, as DynamoDB condition expressions and as HTTP `If-Match`. CAS has one subtle trap, the **ABA problem**: the value goes from A to B and back to A, and CAS cannot tell; a version counter that only ever increases fixes it.

3.6 Deadlock, livelock and starvation

Picture a four-way junction where every car has nosed in and waits for the car on its right: nobody can move, forever. A **deadlock** is a cycle of tasks, each holding a resource and waiting for one held by the next. It needs all four Coffman conditions at once, so removing any one prevents it:

Condition	Meaning	How to break it
Mutual exclusion	resource held by one at a time	lock-free or shared reads
Hold and wait	hold one, ask for another	take all at once, or none
No preemption	nobody can take it away	timeouts, abort and retry
Circular wait	A waits for B waits for A	one global lock order

Encore meets it directly. Ana selects seats A1 and A2, Ben selects A2 and A1; Ana locks A1, Ben locks A2, and each waits for the other. The standard fix is a global order: always lock seats sorted by id, so both fans go for A1 first and the cycle cannot form. A timeout is the second line of defence:

```
def lock_seats(seat_ids: list[str]) -> list[str] | None:
    taken = []
    for sid in sorted(seat_ids):              # global order: no cycle
        if not locks[sid].acquire(timeout=0.5):
            for t in taken: locks[t].release()    # back off: release all
            return None
        taken.append(sid)
    return taken                              # caller releases when done
```

Databases detect deadlocks for you. Postgres checks for a cycle once a lock wait exceeds `deadlock_timeout` (1 s by default) and aborts one transaction with SQLSTATE `40P01`; MySQL InnoDB detects them immediately by default. Your code must catch that error and retry the transaction, and sorting the rows you update (`ORDER BY seat_id ... FOR UPDATE`) makes the error rare.

A **livelock** is the polite version: two people in a corridor both step left, then both step right, forever, busy but making no progress. Two clients retrying on a conflict at exactly the same intervals do this; random jitter in the backoff breaks the symmetry (Chapter 11). **Starvation** is a task that never gets its turn because others always win: an unfair lock that

the same busy thread keeps re-acquiring, readers that keep a writer out of a read-write lock, a low-priority queue that high-priority work always pre-empts. The fixes are fair (FIFO) locks, writer preference and ageing priorities.

3.7 Thread pools and sizing

Creating a thread per request is slow under load and unbounded: a traffic spike becomes 50,000 threads and an out-of-memory kill. A **thread pool** keeps a fixed set of worker threads and a queue of tasks, which bounds memory and turns overload into queueing you can measure. In Python, `concurrent.futures.ThreadPoolExecutor` defaults to `min(32, os.process_cpu_count() + 4)` workers (since 3.13). How many threads should a pool have? The classic rule (from *Java Concurrency in Practice*) is

$$N_{\text{threads}} = N_{\text{cores}} \times (1 + \frac{W}{C})$$

where W is the time a task waits and C the time it computes. CPU-bound work wants about one thread per core; I/O-bound work wants many more. With the hold request from earlier (C = 2 ms, W = 18 ms) on a 4-core pod: $N = 4 \times (1 + 9) = 40$ threads. The pod's CPU caps it at 4/0.002 = 2,000 holds/s, and Little's law agrees: 2,000/s times 20 ms is 40 requests in flight. So 30,000 holds/s need 15 such pods (illustrative).

Two consequences are easy to miss. First, every thread that can talk to the database needs a connection: 15 pods of 40 threads is 600 Postgres connections, and Postgres's `max_connections` defaults to 100 because each connection is a process. Put a pooler such as PgBouncer or Amazon RDS Proxy (on GCP and Azure a PgBouncer, which Azure Database for PostgreSQL builds in) between them. Second, the queue in front of the pool must be bounded; when it is full, reject with 503 or 429 at once rather than letting requests wait until their callers have given up (Chapter 11).

3.8 Async I/O and event loops

Picture a waiter with twenty tables. He does not stand at table 1 until the food is eaten; he takes an order, hands it to the kitchen, and serves whichever table is ready next. An **event loop** is that waiter: one thread runs many tasks, and whenever a task would wait for I/O it parks itself and the loop runs another.

```
           +-----------------------------+
           | event loop (one thread)     |
 ready  --> | run task until it awaits    | --> task parks on socket
 queue     +-----------------------------+
   ^                      |
   |       epoll_wait(): "which sockets are ready?"
   +---------- kernel wakes tasks whose I/O completed
```

The mechanism underneath is the operating system's readiness API: `epoll` on Linux, `kqueue` on BSD and macOS, IOCP on Windows (and `io_uring`, a newer completion-based Linux interface). The loop registers thousands of sockets and asks the kernel in one call which ones can make progress; `asyncio` on Linux uses `epoll` through the `selectors` module. Because a coroutine yields only at `await`, code between two awaits cannot be interleaved by another task on the same loop, which removes most low-level races (but not check-then-act across an `await`).

```
import asyncio

async def lookup(name: str, ms: int) -> str:
    await asyncio.sleep(ms / 1000)      # stands in for a network call
    return name

async def event_page(event_id: int) -> list[str]:
    async with asyncio.timeout(0.5):    # one deadline for the whole page
        return await asyncio.gather(
            lookup("event", 40), lookup("venue", 30), lookup("prices", 50))

async def main() -> None:
    print(await event_page(42))           # ~0.05 s, not 0.12 s
    await asyncio.gather(*(event_page(i) for i in range(1000)))

asyncio.run(main())
```

One page takes as long as its slowest call (50 ms) rather than the sum (120 ms), and 1,000 pages at once finished in about 0.07 s on one thread in a test run. The trap is a blocking call inside a coroutine: `time.sleep(0.2)` or a synchronous database driver freezes every task on the loop. Ten coroutines that each call `time.sleep(0.2)` take 2.0 s; with `await asyncio.to_thread(time.sleep, 0.2)` they take 0.2 s.

When async beats threads: very many concurrent connections that mostly wait, such as WebSockets, Server-Sent Events, long polls, Encore's waiting room with a million connected fans, or fan-out to many backends. **When threads are fine or better**: moderate concurrency (hundreds), libraries that only block, and code that is easier to read sequentially; Java virtual threads and Go goroutines give you both, a blocking style on an event-driven runtime. **Neither helps CPU-bound work**: that needs more cores and processes. Event loops run Node.js (libuv), Nginx, Envoy, HAProxy, Netty, Redis's command loop, and Python's FastAPI on Uvicorn.

3.9 Synchronous and asynchronous requests

The same choice appears one level up, between services. A **synchronous** request is a phone call: the caller waits on the line until the answer comes. An **asynchronous** request is ordering at a counter and getting a buzzer: the caller gets an acknowledgement now and the result later, by checking back, by a call-back, or by a push.

Inside one program, asynchrony shows up as **callbacks** (a function the runtime calls when the result is ready) and **futures** or promises (a placeholder for a result that does not exist yet: `concurrent.futures.Future`, `asyncio.Task`, a JavaScript `Promise`). Between services, it shows up in the API itself. A synchronous call is simplest when the work is fast and the caller cannot continue without the answer: "hold seat A1" must return yes or no. Its costs are that latencies add along the chain, a slow dependency ties up the caller's threads (Little's law again: if the payment provider slows from 1.5 s to 15 s, the same traffic needs ten times as many waiting threads), and every call needs a **timeout**. Timeouts must shrink as you go deeper, so an inner call never outlives the request that is waiting for it:

$$T_{\text{inner}} < T_{\text{caller}} - \text{time already spent}$$

Illustrative Encore budget: app 10 s, gateway 8 s, orders service 6 s, payment call 4 s. gRPC carries this as a deadline that propagates with the call; with plain HTTP you pass the remaining budget in a header and enforce it yourself.

An **asynchronous API** accepts the work, answers `202 Accepted` at once with a `Location` header pointing at a status resource, and does the slow part in a worker; the client polls the status URL, waiting as long as `Retry-After` says. Encore uses it for rendering ticket PDFs and QR codes after payment. Run with FastAPI's `TestClient`, the `POST` below returns `202` with `{"status": "pending"}` and the following `GET /jobs/<id>` returns `{"status": "done", ...}`. In production the job record lives in a database and the work runs on a queue worker (Chapter 14), not inside the web process:

```
import uuid
from fastapi import BackgroundTasks, FastAPI, Response

app = FastAPI()
jobs: dict[str, dict] = {}              # use a database in production

def render_tickets(job_id: str, order_id: str) -> None:
    jobs[job_id] = {"status": "done", "url": f"/orders/{order_id}/tickets"}

@app.post("/orders/{order_id}/tickets", status_code=202)
def request_tickets(order_id: str, bg: BackgroundTasks, resp: Response):
    job_id = uuid.uuid4().hex
    jobs[job_id] = {"status": "pending"}
    bg.add_task(render_tickets, job_id, order_id)
    resp.headers["Location"] = f"/jobs/{job_id}"
    resp.headers["Retry-After"] = "2"
    return jobs[job_id]

@app.get("/jobs/{job_id}")
def job_status(job_id: str):
    return jobs[job_id]
```

Polling has a price that follows from one division: polling load equals waiting clients divided by the poll interval. If 25,000 orders (50,000 seats at 2 per order) wait for tickets and each app polls every 2 s, that is 12,500 requests/s of "not yet". Raise the interval with backoff, honour `Retry-After`, or push the result instead: a webhook to a server, Server-Sent Events or a WebSocket to an app, or a mobile push notification (Chapter 5).

> **Design question.** After a fan pays, the payment provider takes anywhere from 1 to 30 seconds to confirm. Should Encore's "pay" endpoint wait for it?
>
> No. Holding a request open for 30 s ties up a connection and a thread per fan (25,000 orders at up to 30 s each is up to 750,000 thread-seconds of waiting in ten minutes) and breaks on any mobile network hiccup. Instead: `POST /orders/{id}/payment` with an `Idempotency-Key` records the intent, calls the provider with a short timeout, and returns `202` with the order's status URL. The provider's webhook (signature-verified) moves the order to `paid`; a worker issues tickets; the app learns of it by push, with polling every few seconds as the fallback. The seat hold's 10-minute TTL covers the wait. The trade-off is more states to handle (`pending`, `paid`, `failed`, `expired`) in exchange for bounded resources. See also *50 System Designs* #24 (Payment System).

AI agents make the case for asynchronous APIs stronger. A model call takes seconds and an agent run can take minutes, so Encore's support agent runs as a job: the chat client gets a `202` or a stream of partial output over SSE, the run's state is checkpointed so a crashed worker can resume, and every tool call carries a timeout and an idempotency key, so a retried "request refund" step cannot refund twice. *Designing Agentic Systems* covers the agent side in depth.

3.10 Handling races in services

A `threading.Lock` protects memory inside one process. Encore's seat inventory runs on 15 pods across two availability zones; their locks know nothing of each other. In a service, the shared state lives in a database or in Redis, so the coordination must live there too. Six tools cover almost every case:

Technique	Idea	Best when
Conditional write	one atomic `UPDATE ... WHERE`	single-row check-then-act
Pessimistic lock	`SELECT ... FOR UPDATE`	multi-row, high conflict
Optimistic version	write only if version unchanged	low conflict, long think time
Redis compare-and-set	`SET NX`, `WATCH`, Lua	fast, TTL-based holds
Idempotency key	same request, same effect	retries, payments, tools
Single writer	one owner per key	hot keys, ordered streams

3.10.1 Conditional writes and transactions

The simplest fix moves the check *into* the write, so the database performs check and act as one atomic step. The `WHERE status = 'free'` clause is a compare-and-swap; the affected-row count tells you who won. Four threads hammering this statement in SQLite always produce exactly one winner (`{'ana': True, 'ben': False, 'cy': False, 'dee': False}`), and the statement behaves the same way in Postgres and MySQL:

```
UPDATE seats
SET    status = 'held', holder = :fan, version = version + 1
WHERE  event_id = 42 AND seat_id = 'A1' AND status = 'free';
-- rowcount 1: you hold it; rowcount 0: someone else got there first
```

When the decision spans several rows or needs application logic between the read and the write, take a **pessimistic** row lock inside a transaction. `SELECT ... FOR UPDATE` (Postgres, MySQL; SQLite has no row locks) makes other writers of those rows wait until you commit; sorting the rows avoids the deadlock from earlier, and `NOWAIT` or `SKIP LOCKED` lets you fail fast or take the next free seat instead of waiting:

```sql
BEGIN;
SELECT seat_id, status FROM seats
WHERE  event_id = 42 AND seat_id IN ('A1', 'A2')
ORDER  BY seat_id
FOR UPDATE;            -- other writers wait; the app checks both are free
UPDATE seats SET status = 'held', holder = 'ana'
WHERE  event_id = 42 AND seat_id IN ('A1', 'A2');
COMMIT;
```

A transaction alone is not a lock. Under Postgres's default Read Committed isolation, a plain `SELECT` followed by an `UPDATE` in the same transaction still races; Repeatable Read turns the lost update into a serialization error (SQLSTATE `40001`) that you retry. Isolation levels and their anomalies are in Chapter 12.

3.10.2 Optimistic concurrency with version numbers

Optimistic concurrency assumes conflicts are rare: read the row and its `version`, do your work without holding a lock, then write `... WHERE id = ? AND version = :read_version` and increment the version. Zero rows updated means someone else wrote first, so re-read and retry; in the test above, the second of two writers holding the same version got 0 rows. The same idea is HTTP's `ETag` with `If-Match` (a mismatch returns `412 Precondition Failed`), DynamoDB's `ConditionExpression`, S3's conditional writes (`If-Match` on an ETag and `If-None-Match: *` for create-only, with 412 on failure) and Kubernetes' `resourceVersion`. If each attempt conflicts with probability p, the expected number of attempts is

$$E[\text{attempts}] = \frac{1}{1-p}$$

At $p = 0.03$ (the hot-seat estimate above) that is 1.03 attempts, so optimism is nearly free; at $p = 0.5$ it is 2, and under heavier contention retries pile up and a pessimistic lock or a single writer wins.

3.10.3 Compare-and-set in Redis

Redis executes commands one at a time on its main thread, so each command is atomic. `SET` with `NX` (only if absent) and `EX` (expiry) is a complete seat hold with Encore's 10-minute TTL in one command; tested with fakeredis:

```
SET hold:42:A1 ana NX EX 600   -> OK      (ana holds A1 for 600 s)
SET hold:42:A1 ben NX EX 600   -> (nil)   (already held)
TTL hold:42:A1                 -> 600
```

For read-compute-write on Redis data, `WATCH key`, then `MULTI ... EXEC` executes the queued writes only if the watched key did not change, otherwise `EXEC` returns nil (redis-py raises `WatchError`) and you retry: CAS again. A Lua script or a Redis Function runs atomically on the server and is usually simpler and faster than WATCH loops (Chapter 9).

3.10.4 Idempotency keys

Retries create their own race: the client times out, retries, and the first request was in fact still running. An **idempotency key** is a client-chosen unique id for one logical operation; the server records the key with the result and returns the stored result for any repeat. `SET idem:<key> in-progress NX EX 86400` in Redis or a unique index on `idempotency_key` in the orders table makes the first request the only one that acts. Encore sends one to the payment provider on every charge, so a retried charge is not a second charge (Chapter 5).

3.10.5 Single writer per key

The most scalable way to avoid races is to have nothing to race over: route every write for one key to one owner that processes them in order. Partition a Kafka topic of hold requests by `event_id`, and one consumer per partition applies them sequentially; or give each event an actor (Akka, Orleans) or a shard owner. Redis's single command thread is this idea applied to a whole server. The cost is a throughput ceiling per key, which for a 50,000-seat flash sale means splitting the event by section (Chapter 17).

Distributed locks (a Redis or etcd lock held across services) look like the obvious tool but are subtle: a paused process can wake up after its lock expired and write anyway. Leases and fencing tokens, which make them safe, are in Chapter 18.

3.11 In Encore: two fans, one seat

Back to 10:00:00. Ana and Ben pass the waiting room (Chapter 11) and load the seat map from a cache that may be a second stale (Chapter 9); both see A1 as free, which is fine, because the map is a hint and the hold is the truth. Both click. The seat-inventory service runs `SET hold:42:A1 <fan> NX EX 600` in Redis: Ana's command executes first and returns `OK`, Ben's returns nil, and Ben gets `409 Conflict` with three nearby free seats in the same response. A multi-seat hold runs as one Lua script that checks every seat and sets all of them or none, so there is no partial hold and no lock ordering to get wrong. At checkout, the durable step in Postgres is a conditional write, `UPDATE seats SET status = 'sold' WHERE ... AND holder = 'ana' AND status = 'held'`, inside the same transaction that inserts the order, and the unique index on tickets `(event_id, seat_id)` is the backstop that no bug can pass. Payment carries an idempotency key and returns `202`; tickets arrive by push.

> **Design question.** 30,000 hold attempts per second arrive in the first minute. Should holds take Postgres row locks or go to Redis?
>
> Each `SELECT ... FOR UPDATE` holds a row lock for a network round trip or more, and hot seats serialize behind it, while a pool of a few hundred database connections serves the whole fleet; a durable write per click also costs WAL and replication. Redis evaluates `SET NX EX` in memory as one atomic command, and a single shard handles on the order of 100,000 simple commands per second, so the hold burst fits with headroom. The trade-off: Redis is the source of truth for holds only (a lost hold costs a fan ten minutes, not a double sale), Postgres stays the source of truth for sales via the conditional write. One subtlety: a multi-key Lua script needs all its keys in one Redis Cluster slot, so keys use a hash tag such as `hold:{42}:A1`, which puts one event on one shard; for a stadium show, tag by section instead. See also *50 System Designs* #25 (Ticket Booking) and #34 (E-Commerce Checkout and Inventory).

3.12 Numbers to remember

Timings and memory below were measured on one Linux machine with Python 3.13 (orders of magnitude, not benchmarks); Encore figures are illustrative.

Quantity	Value
Pool size, I/O-bound	cores x (1 + W/C)
Encore holds	600 in flight, 60 cores, 15 pods
Race window hit	$1 - e^{-\lambda w}$, about 3%
Python `Lock` acquire+release	about 135 ns
Thread handoff, task switch	about 5 us, 1.6 us
Idle thread, idle task	about 84 KB, under 1 KB
`ThreadPoolExecutor` default	min(32, cpus + 4)
Postgres `deadlock_timeout`	1 s
Postgres `max_connections`	100 by default

3.13 Pitfalls

- Trusting `count += 1` or a check-then-act because tests pass -> test the invariant with a barrier and an injected delay, and make the operation atomic.
- Holding a lock across a network call -> do the slow call outside the lock, or move the coordination into the database or Redis.
- Locking rows or locks in arbitrary order -> sort by id; catch deadlock errors (`40P01`) and retry.
- Calling a blocking library inside `async def` -> use an async driver or `asyncio.to_thread`.

- Thread count without a connection budget -> size pools per pod and put PgBouncer or RDS Proxy in front of Postgres.
- Timeouts missing, or growing deeper in the chain -> shrink them inward and propagate the deadline.
- Using a process-local lock in a multi-pod service -> conditional writes, Redis CAS or a single writer per key.

3.14 Remember

- Concurrency is many tasks in progress; parallelism is many running at once. Services need mostly concurrency, because requests mostly wait.
- A race is a gap between a check and an act on shared state. Close it: lock it, make the write conditional, or have one writer.
- Compare-and-swap is the pattern at every scale: CPU `CMPXCHG`, SQL `WHERE version = ?`, Redis `WATCH` and `SET NX`, HTTP `If-Match`.
- Threads for moderate concurrency and blocking code; async for very many idle connections; processes for CPU-bound Python (until free-threading is the default).
- Answer fast work synchronously with a timeout; answer slow work with `202 Accepted`, a status resource and a push.

4 Networks for system designers

When a fan in Frankfurt taps “Hold these seats”, what happens between the phone and Encore’s seat-inventory service in Virginia, and why does it take 300 milliseconds instead of 3? This chapter follows one request down through the layers and across the ocean: names become addresses, addresses become connections, connections become encrypted streams, and streams carry HTTP. Each step costs round trips, and round trips, not bandwidth, are what a system designer spends.

4.1 The picture

Think of sending a parcel abroad. You write the recipient’s name (a domain), look up the street address (DNS), hand the parcel to a courier who promises delivery and re-sends anything lost (TCP), seal it in a tamper-proof envelope (TLS), and inside write a letter in a shared language (HTTP). Each layer only talks to the same layer on the other side and trusts the layer below to carry its bytes.

```
layer        what it adds                    unit        example
-----------  ------------------------------  ----------  ----------------
application  meaning: methods, paths, status  message     HTTP, gRPC, DNS
security     secrecy, integrity, identity     record      TLS 1.3
transport    ports, reliability, ordering     segment     TCP, UDP, QUIC
network      addresses, routing hop by hop    packet      IPv4, IPv6
link         one hop on a wire or radio       frame       Ethernet, Wi-Fi
```

Two rules of thumb carry most designs. First, **latency is set by distance and round trips**: light in fiber covers about 200 km per millisecond, so every extra round trip to a far region costs tens of milliseconds no matter how fat the pipe is. Second, **connections are expensive and requests are cheap**: open a connection once, keep it warm, and send many requests over it.

4.2 Layers, addresses and ports

IP addresses name machines (strictly, network interfaces). IPv4 addresses are 32 bits (`10.20.37.14`), IPv6 addresses 128 bits (`2001:db8::1`). A **prefix** in CIDR notation groups addresses: `10.20.0.0/16` fixes the first 16 bits and leaves the rest free. Routers forward a packet hop by hop by matching its destination against the longest prefix in their table; nothing along the way guarantees delivery or order.

$$\text{addresses in a /p block} = 2^{32-p} \qquad 2^{32-16} = 65,536 \qquad 2^{32-20} = 4,096$$

Ports name programs on a machine: 16 bits, so 0 to 65,535. A TCP connection is identified by its **4-tuple** (source IP, source port, destination IP, destination port). A client picks a source port from the ephemeral range (on Linux 32768 to 60999, so 28,232 ports), which is why one client machine can hold only about 28,000 simultaneous connections to one server address and port, a limit that bites proxies and NAT gateways first.

Private ranges (RFC 1918: `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`) are not routed on the internet. **NAT** (network address translation) lets many private machines share one public address by rewriting the source address and port on the way out and keeping a table to reverse it on the way back. NAT is why a phone behind a home router cannot accept inbound connections and why servers push to clients over connections the client opened.

In the cloud, a **VPC** (virtual private cloud; Google Cloud VPC, Azure Virtual Network) is your private address space in a region, cut into **subnets**, each living in one availability zone. A subnet is *public* if its route table sends `0.0.0.0/0` to an internet gateway, *private* otherwise; private subnets reach the internet through a NAT gateway (Google Cloud NAT, Azure NAT Gateway). AWS reserves five addresses in every subnet. Plan generously, because a VPC’s ranges are hard to change once workloads run in it:

```python
import ipaddress

vpc = ipaddress.ip_network("10.20.0.0/16")
print(vpc, vpc.num_addresses, "addresses")       # 65536

# Carve /20 subnets: 3 zones x (public, private-app, private-data)
subnets = vpc.subnets(new_prefix=20)
plan = {}
for zone in ("a", "b", "c"):
    for tier in ("public", "app", "data"):
        plan[f"{tier}-{zone}"] = next(subnets)

for name, net in plan.items():
    usable = net.num_addresses - 5          # AWS reserves 5 per subnet
    print(f"{name:10} {str(net):16} {usable:5} usable")   # 4091 each
```

Nine /20 subnets use 36,864 of the 65,536 addresses and leave room for growth. On Amazon EKS the default VPC CNI gives every pod a VPC address, so a busy cluster consumes addresses far faster than the node count suggests; size the app subnets for pods, not machines.

4.3 DNS: from a name to an address

DNS is the internet's phone book, distributed as a tree of delegations and held together by caches. Nobody holds the whole book; each server knows who to ask next.

```
phone (stub resolver)
  | 1. api.encore.example?              (cache miss everywhere)
  v
recursive resolver (ISP, 8.8.8.8, VPC resolver)
  | 2. ask a root server   -> "ask the .example TLD servers"
  | 3. ask the TLD server  -> "ask ns1.encore-dns.net"
  | 4. ask authoritative   -> "api.encore.example A 203.0.113.10 TTL 60"
  v
resolver caches the answer for 60 s; the OS and browser cache it too
```

The steps:

- The application calls the OS resolver (`getaddrinfo`), which checks its cache and `/etc/hosts`, then sends a query, usually over UDP port 53, to a **recursive resolver**.
- The recursive resolver walks the tree from the **root** (13 named root server identities, each served by many anycast instances), to the **TLD** servers, to the domain's **authoritative** servers, caching each referral.
- The answer carries a **TTL** (time to live, in seconds). Every cache along the path may reuse it until it expires.

Record types worth knowing: `A` (IPv4), `AAAA` (IPv6), `CNAME` (an alias to another name), `NS` (delegation), `TXT` (verification, SPF), `SRV` (service and port), and on Route 53 the **alias** record, which points at an AWS resource such as a load balancer and can sit at the zone apex where a `CNAME` cannot.

The TTL is a dial between load and agility. With N recursive resolvers querying a name, the authoritative servers see about one query per resolver per TTL, and a change takes up to a TTL (plus client caches that misbehave) to reach everyone:

$$\text{authoritative QPS} \approx \frac{N_{\text{resolvers}}}{\text{TTL}} \qquad t_{\text{switch}} \le t_{\text{detect}} + \text{TTL}$$

With 10,000 resolvers worldwide (illustrative), a 60-second TTL costs about 167 queries/s at the authoritative servers, a 300-second TTL about 33/s. Managed DNS absorbs either easily, so the real cost of a long TTL is slow failover. Applications see only the result, through `socket.getaddrinfo(host, port)`, which returns every IPv6 and IPv4 address to try in order.

Where it runs. Amazon Route 53 (Google Cloud DNS, Azure DNS) hosts authoritative zones and adds routing policies: weighted, latency-based, geolocation and failover with health checks, which makes DNS the first, coarse load balancer (Chapter 8). Inside a VPC, the provider's resolver answers private zones; inside Kubernetes, CoreDNS resolves

`service.namespace.svc.cluster.local`.

Trade-offs. DNS-based failover is only as fast as the TTL and the slowest cache, and a long-lived connection never re-resolves at all: a client holding a pooled connection to an old address keeps using it after the record changes. For fast failover, put a stable address in DNS (an anycast or load-balancer address) and move traffic behind it.

4.4 TCP: a reliable byte stream

TCP turns unreliable packets into an ordered, reliable stream of bytes, like a courier who numbers every page, waits for receipts and re-sends what went missing. The price is a setup round trip and caution about speed.

```
client                          server
  | ---- SYN (seq=x) ---------->  |     1 RTT before any data
  | <--- SYN-ACK (seq=y,ack=x+1)  |
  | ---- ACK + first request -->  |
  | <--- data, data, data ------  |     window grows each RTT
  | ---- FIN ------------------>  |     closer waits in TIME_WAIT
```

Mechanism. The three-way handshake agrees on starting sequence numbers. Every byte is numbered; the receiver acknowledges what arrived; missing bytes are re-sent after a duplicate-ACK signal or a retransmission timeout. Two windows limit how much is in flight: the receiver's **flow-control window** (what it can buffer) and the sender's **congestion window** (`cwnd`, what it believes the path can carry).

Slow start and congestion. A new connection starts with an initial window of 10 segments (RFC 6928), about 14.6 KB with a 1,460-byte MSS, and doubles `cwnd` every round trip until it sees loss; then it backs off and grows more gently (additive increase, multiplicative decrease). Linux defaults to **CUBIC**; Google's **BBR** instead estimates bottleneck bandwidth and RTT and is common on CDNs and Google's own services. The number of round trips to deliver S bytes from a cold start, and the steady-state ceiling under random loss p (Mathis et al.), are:

$$\text{rounds} = \log_2\left(\frac{S}{IW \cdot MSS} + 1\right) \qquad \text{rate} \quad \frac{MSS}{RTT}\,\frac{1.22}{\sqrt{p}}$$

Encore's 200 KB event page is 137 segments, so a cold connection needs 4 round trips of data after the handshakes. With 0.01 percent loss, one TCP flow across a 90 ms ocean tops out near 16 Mbit/s, while the same flow to a CDN edge 10 ms away is capped near 143 Mbit/s: throughput falls as RTT rises, which is the second reason to serve from nearby.

Head-of-line blocking. TCP delivers bytes in order. If one packet is lost, everything behind it waits in the kernel for the retransmission, even bytes belonging to unrelated requests multiplexed on the same connection. This is the flaw HTTP/3 removes.

Connection state. The side that closes first keeps the 4-tuple in `TIME_WAIT` (60 seconds on Linux) so late packets cannot corrupt a new connection. A service that opens and closes thousands of short connections per second to one backend exhausts ephemeral ports this way; reuse connections instead. Sockets expose the knobs (`TCP_CONGESTION` is Linux-only); `TCP_NODELAY` disables Nagle's algorithm, which holds small writes back to batch them, so latency-sensitive clients (Redis, gRPC, databases) set it:

```
import socket

srv = socket.create_server(("127.0.0.1", 0))           # listen
host, port = srv.getsockname()

cli = socket.create_connection((host, port), timeout=2.0)  # 3-way handshake
conn, peer = srv.accept()
cli.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)  # no Nagle delay
cli.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)  # probe idle peers
algo = cli.getsockopt(socket.IPPROTO_TCP, socket.TCP_CONGESTION, 16)
print("4-tuple:", cli.getsockname(), "->", cli.getpeername())
print("congestion control:", algo.rstrip(b"\0").decode())  # cubic or bbr
cli.sendall(b"hold seat A-12")
print(conn.recv(100))                                      # b'hold seat A-12'
```

4.5 UDP: datagrams without promises

UDP is a postcard: an address, a port, a checksum and the payload, with no handshake, no retransmission, no ordering and no congestion control. Each datagram arrives once, never, or (rarely) twice, and out of order. That is exactly right when a late answer is worthless or the application wants its own rules: DNS queries (one packet each way), voice and video (WebRTC media, *50 System Designs* #36 (Video Conferencing)), game state, metrics such as StatsD, and QUIC, which builds a better TCP on top of UDP in user space. Keep datagrams under the path MTU (1,500 bytes on most of the internet, minus headers) to avoid fragmentation; AWS allows 9,001-byte jumbo frames inside a VPC, but traffic through an internet gateway or a VPN drops back to 1,500 bytes or less.

4.6 TLS 1.3: the secure channel

TLS is the sealed, signed envelope: the client checks that the server is who it claims to be (a certificate chain up to a trusted authority, matching the hostname), both sides agree on fresh keys, and every record is then encrypted and integrity-checked.

```
client                                  server
  | -- ClientHello: key share, SNI, ALPN -> |
  | <- ServerHello: key share,              |   1 RTT, then
  |    {certificate, verify, finished} --   |   application data
  | -- {finished} + first HTTP request ---> |
resumption with 0-RTT: request rides in the first flight (replayable)
```

TLS 1.3 (RFC 8446) cut the full handshake from two round trips (TLS 1.2) to **one** by sending the client's Diffie-Hellman key share in the first message, and allows **0-RTT** resumption, where a returning client sends its request with the first flight. 0-RTT data can be replayed by an attacker, so accept it only for idempotent requests such as `GET /events/42`, never for `POST /holds`. The ClientHello carries **SNI** (the hostname, so one IP can serve many certificates) and **ALPN** (which protocol to speak next, such as `h2`).

```
import ssl

ctx = ssl.create_default_context()          # verifies cert chain + hostname
ctx.minimum_version = ssl.TLSVersion.TLSv1_3
ctx.set_alpn_protocols(["h2", "http/1.1"])   # offer HTTP/2 first
print(ctx.verify_mode.name, ctx.check_hostname, ctx.minimum_version.name)
# CERT_REQUIRED True TLSv1_3
```

Where it runs. TLS usually terminates at the edge: CloudFront (Google Cloud CDN, Azure Front Door), an Application Load Balancer, or an Envoy or Nginx proxy, with certificates from AWS Certificate Manager (Google Certificate Manager, Azure Key Vault). Inside the VPC, a service mesh can add **mutual TLS**, where both sides present certificates (Chapter 22). The handshake's CPU cost is small on modern hardware; its round trip is what hurts, which is why terminating TLS close to the user matters.

4.7 HTTP/1.1, HTTP/2 and HTTP/3

Picture three ways to run a checkout lane. HTTP/1.1 is one cashier per lane, one customer at a time, so browsers open about six lanes per host. HTTP/2 is one lane where the cashier interleaves many customers' items, but a single dropped item stops the whole belt. HTTP/3 gives each customer a separate belt on the same lane.

```
HTTP/1.1  conn1: [req A][resp A][req B][resp B]     conn2: [req C][resp C]
HTTP/2    one TCP conn: A1 B1 C1 A2 C2 B2 ...  (lost packet stalls all)
HTTP/3    one QUIC conn: stream A | stream B | stream C  (loss stalls one)
```

	HTTP/1.1	HTTP/2	HTTP/3
Transport	TCP	TCP	QUIC over UDP
Requests per connection	one at a time	many streams	many streams
Headers	text	binary, HPACK	binary, QPACK

	HTTP/1.1	HTTP/2	HTTP/3
Head-of-line blocking	per connection	at the TCP layer	per stream only
Setup with TLS 1.3	TCP + TLS: 2 RTT	2 RTT	1 RTT (0 on resume)
Survives IP change	no	no	yes (connection IDs)

HTTP/1.1 connections are persistent by default (**keep-alive**), so the handshakes are paid once and many requests follow, but each connection carries one request at a time. **HTTP/2** (RFC 9113) splits messages into binary frames on numbered streams, multiplexes them over one connection and compresses repeated headers. **HTTP/3** (RFC 9114) runs over **QUIC** (RFC 9000), which lives in user space over UDP, integrates the TLS 1.3 handshake into its own and recovers lost packets per stream. Its connection IDs let a phone move from Wi-Fi to cellular without reconnecting, a real gain for fans walking into a stadium. The wire looks the same at the HTTP level in all three:

```
GET /v1/events/42/seatmap HTTP/1.1
Host: api.encore.example

HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: public, max-age=1, stale-while-revalidate=5
Alt-Svc: h3=":443"; ma=86400
```

The `Alt-Svc` header tells the browser that HTTP/3 is available, so the next connection can use it. **Where it runs.** CloudFront, Cloudflare and Google's front ends speak HTTP/3 to clients; back-end hops inside a VPC are mostly HTTP/1.1 or HTTP/2, where low loss makes QUIC's advantages small.

4.8 Latency math

Three numbers describe a path: the **round-trip time** (RTT), the **bandwidth** B, and the loss rate. Light in fiber travels at about two thirds of its speed in vacuum, roughly 200,000 km/s, which sets a floor no hardware can beat. Frankfurt to Northern Virginia is about 6,550 km on the great circle:

$$RTT_{min} = \frac{2d}{v} = \frac{2 \times 6{,}550\ \text{km}}{200{,}000\ \text{km/s}} \approx 65\ \text{ms}$$

Real routes are longer and pass through routers, so a measured RTT between AWS `eu-central-1` and `us-east-1` is typically around 90 ms (illustrative; measure your own path). The time to fetch S bytes is the sum of setup round trips, data round trips and wire time; the **bandwidth-delay product** is how many bytes must be in flight to keep the pipe full:

$$T = T_{\text{dns}} + k \quad RTT + \text{rounds} \quad RTT + \frac{S}{B} + T_{\text{server}} \qquad BDP = B \times RTT$$

Here k = 2 for a new TCP plus TLS 1.3 connection, 1 for QUIC, 0 for a warm connection. At 1 Gbit/s and 90 ms the BDP is 11.25 MB: a TCP flow needs that much window (and socket buffer) to use the link, which is why bulk cross-region copies use parallel streams. A calculator with Encore's numbers (50 Mbit/s for the fan, 20 ms of server time for a hold):

```python
import math

MSS, IW = 1460, 10                        # bytes, initial window segments

def fetch_ms(size, rtt, mbps, warm=False, server_ms=0, quic=False):
    """Time to fetch `size` bytes: setup + slow-start rounds + wire time."""
    setup = 0 if warm else (1 if quic else 2) * rtt   # TCP+TLS1.3 or QUIC
    segs = math.ceil(size / MSS)
    rounds = 1 if warm else math.ceil(math.log2(segs / IW + 1))
    wire = size * 8 / (mbps * 1e6) * 1000
    return setup + rounds * rtt + wire + server_ms

PAGE, HOLD = 200_000, 1_000
for name, rtt in [("Virginia origin", 90), ("Frankfurt edge", 10)]:
    print(f"{name:16} page cold {fetch_ms(PAGE, rtt, 50):5.0f} ms"
          f"  hold warm {fetch_ms(HOLD, rtt, 50, True, 20):4.0f} ms"
          f"  hold cold {fetch_ms(HOLD, rtt, 50, False, 20):4.0f} ms")
# Virginia origin  page cold   572 ms  hold warm  110 ms  hold cold  290 ms
# Frankfurt edge   page cold    92 ms  hold warm   30 ms  hold cold   50 ms
```

The 200 KB page costs 572 ms from Virginia, of which only 32 ms is wire time: the rest is six round trips. Bandwidth barely matters for small objects; round trips dominate until objects reach megabytes.

> **Design question.** Fans in Europe complain that "Hold" feels slow during the flash sale. Encore's seat inventory lives in `us-east-1`. What do you change?
>
> A cold hold from Frankfurt costs about 290 ms: 180 ms of handshakes, 90 ms for the request and 20 ms of work. Moving inventory to Europe splits a single source of truth per event, so keep it in one region and remove round trips instead. Terminate TCP and TLS at a nearby edge (CloudFront or AWS Global Accelerator, 10 ms away) and let the edge reuse warm connections to the origin: 20 + 10 + 90 + 20 = 140 ms. Pre-warm the connection when the seat map opens, so the click itself pays one round trip. Enable HTTP/3 for mobile fans on lossy radio. The trade-off: one more hop to operate and pay for, and the 90 ms ocean crossing stays, because each event's seats have one home region. See also *50 System Designs* #25 (Ticket Booking).

4.9 Connection pooling

A pool is a taxi rank: cars wait with engines running, a passenger takes the first one, and it returns to the rank instead of being scrapped. Opening a connection costs one or two round trips plus server work (a Postgres connection forks a backend process); reusing one costs nothing. A pool keeps idle connections open, hands one to each request and caps the total so a traffic spike queues in the client instead of drowning the server. Size it with Little's law (Chapter 1): connections in use equal the request rate times how long each request holds a connection.

$$C = \lambda \times W \qquad 30,000 \text{ holds/s} \times 5 \text{ ms} = 150 \quad (225 \text{ with } 1.5\text{x headroom})$$

```python
import math
import httpx

def pool_size(rate_per_s, hold_ms, headroom=1.5):
    """Little's law: connections busy = arrival rate x time each is held."""
    return math.ceil(rate_per_s * hold_ms / 1000 * headroom)

print(pool_size(30_000, 5))          # holds -> Postgres: 225
print(pool_size(1_000, 400))         # checkouts -> payment provider: 600

client = httpx.Client(               # one client per process, reused
    base_url="https://payments.example",
    limits=httpx.Limits(max_connections=600,
                        max_keepalive_connections=600,
                        keepalive_expiry=30),
    timeout=httpx.Timeout(2.0, connect=0.5),
)
```

The 1,000 checkouts/s and 400 ms provider latency are illustrative. httpx defaults to 100 connections, 20 kept alive and a 5-second keep-alive expiry, and many libraries have similar small defaults: a slow dependency then makes requests queue for a connection, which looks like the dependency being slower still. Keep the keep-alive expiry below the server's or load balancer's idle timeout (60 seconds by default on an AWS ALB), or the client will send requests on connections the other side already closed.

> **Design question.** Before the waiting room existed, Encore ran its API and seat-inventory tiers on 60 pods, each with a pool of 20 Postgres connections. During a flash sale the autoscaler added 60 more pods and the database started refusing connections. Why, and what is the fix?
>
> 120 pods x 20 = 2,400 connections, while Postgres defaults to `max_connections = 100` and each connection is a process with its own memory. The work needs only about 150 busy connections even at the open door's 30,000 holds/s x 5 ms (about 15 at the 3,000 holds/s the waiting room admits). Put a pooler between pods and database, PgBouncer in transaction mode or Amazon RDS Proxy (Azure Database for PostgreSQL has built-in PgBouncer; Cloud SQL Enterprise Plus offers Managed Connection Pooling), and give it about 225 server connections. Pods can then scale freely. The trade-off: transaction pooling breaks session state such as `SET`, session advisory locks and `LISTEN`, so the code must avoid them. See also *50 System Designs* #34 (E-Commerce Checkout and Inventory).

4.10 Pushing to clients: long polling, SSE and WebSockets

NAT and firewalls mean the server cannot call the phone; the phone must open a connection and the server answers through it. Three ways to keep that line open, from a waiter you keep asking to a phone call left open:

```
long polling  C: any news? ....(held up to 30 s).... S: yes, X  (repeat)
SSE           C: GET /events   S: event, event, event ...  (one way)
WebSocket     C: GET + Upgrade S: 101   C <-> S frames both ways
```

	Long polling	Server-Sent Events	WebSocket
Direction	server to client	server to client	both
Protocol	plain HTTP	HTTP, `text/event-stream`	RFC 6455 after HTTP upgrade
Reconnect	client loops	built in, `Last-Event-ID`	write it yourself
Proxies and CDNs	everywhere	usually fine	needs upgrade support
Best for	rare events, legacy	feeds, progress, LLM tokens	chat, games, collaboration

SSE is the simplest real-time channel: an ordinary response that never ends, with events separated by blank lines. A runnable sketch of Encore's waiting-room position feed:

```python
import asyncio, json
import httpx
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()

@app.get("/v1/queue/{ticket}/events")
async def queue_events(ticket: str):
    async def stream():
        for n, ahead in enumerate([1200, 600, 0]):   # position updates
            data = json.dumps({"ahead": ahead})
            yield f"id: {n}\nevent: position\ndata: {data}\n\n"
            await asyncio.sleep(0.01)
    return StreamingResponse(stream(), media_type="text/event-stream")

async def main():
    transport = httpx.ASGITransport(app=app)
    async with httpx.AsyncClient(transport=transport,
                                 base_url="http://encore") as c:
        async with c.stream("GET", "/v1/queue/t-77/events") as r:
            async for line in r.aiter_lines():
                if line.startswith("data:"):
                    print(line)            # data: {"ahead": 1200} ...

asyncio.run(main())
```

A WebSocket starts as HTTP and switches protocols; after the `101`, both sides exchange small frames on the same TCP connection:

```
GET /v1/live HTTP/1.1
Host: api.encore.example
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13

HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaq9kYGzzhZRbK+xOo=
```

Where it runs. AWS ALB and API Gateway WebSocket APIs (Google Cloud Load Balancing, Azure Web PubSub) carry WebSockets; SSE passes through any HTTP proxy that does not buffer responses. LLM APIs stream tokens over SSE, and an agent that runs for minutes reports progress the same way; the sibling book *Designing Agentic Systems* covers streaming agent output.

> **Design question.** One million fans sit in Encore's waiting room for ten minutes. How do you tell each one their position and when it is their turn?
>
> The traffic is one-way, low-rate and must survive flaky mobile networks: SSE fits, with WebSockets reserved for features that need client-to-server messages. One million open connections at an illustrative 50,000 per gateway node is 20 nodes, each mostly idle memory. Send a position update every 10 to 15 seconds (67,000 to 100,000 events/s in total) and a comment line as heartbeat so the ALB's 60-second idle timeout never fires. On reconnect, `Last-Event-ID` resumes the stream, and the queue position lives in Redis, not in the gateway, so any node can serve any fan. The admission token itself is checked at the edge (Chapter 10). The trade-off: a fleet of long-lived connections makes deployments slower, since nodes must drain. See also *50 System Designs* #25 (Ticket Booking) and #6 (Chat System).

4.11 gRPC

gRPC is RPC over HTTP/2 with Protocol Buffers: you define a service in a `.proto` file, generate client and server stubs in each language, and calls look like local functions. Each call is an HTTP/2 stream, so one connection carries thousands of concurrent calls, and four shapes are built in: unary, server streaming, client streaming and bidirectional streaming. Deadlines travel with the call, so a downstream service knows when to give up.

```
service SeatInventory {
  rpc Hold(HoldRequest) returns (HoldReply);              // unary
  rpc WatchSeats(WatchRequest) returns (stream SeatDelta); // server stream
}
```

Its sharp edge is load balancing: a long-lived HTTP/2 connection pins every call from one client to one server, so an L4 balancer (a Kubernetes `ClusterIP` Service, an NLB) spreads connections, not calls. Balance per request at L7 with Envoy, a service mesh, an ALB with gRPC target groups or client-side balancing (Chapter 8). Browsers cannot speak raw gRPC and need gRPC-Web or a JSON gateway; Chapter 5 compares it with REST and GraphQL.

4.12 Proxies and anycast

A **forward proxy** acts for clients (a corporate egress proxy, or an egress gateway that lets Encore's support agent call only allow-listed APIs); a **reverse proxy** acts for servers, terminating TLS, routing by path, caching, compressing and hiding the backends (Nginx, Envoy, HAProxy, an ALB, a CDN). **Anycast** announces the same IP prefix from many locations through BGP, so the internet's routing delivers each packet to the nearest one: DNS root servers, Cloudflare, Google's front end and AWS Global Accelerator work this way, and an attack spreads across every site instead of flooding one (Chapter 10).

4.13 Private networking in the cloud

Services in one VPC talk over private addresses, filtered by **security groups** (stateful, per interface) and network ACLs (stateless, per subnet). Between VPCs, **peering** (Google Cloud VPC Network Peering, Azure VNet peering) joins two networks without transitive routing; **Transit Gateway** (Google Network Connectivity Center, Azure Virtual WAN) is a hub for many. To reach a managed service without the internet, use **private endpoints**: gateway endpoints for S3 and DynamoDB, and **PrivateLink** interface endpoints (Google Private Service Connect, Azure Private Link) for everything else, including partners such as a payment provider that offers one. Private endpoints also cut NAT gateway charges, which are billed per gigabyte processed, and cross-zone and cross-region traffic is billed too, so chatty services belong in the same zone where their availability allows.

4.14 In Encore

A fan's hold request, end to end, with the chapter's numbers:

```
phone --(HTTP/3, 10 ms RTT)--> CloudFront PoP, Frankfurt: TLS ends here
  --(warm HTTP/2 pool, 90 ms RTT)--> ALB, us-east-1, public subnets
  --(HTTP/2 over the VPC)--> gateway --> seat-inventory pods (app subnets)
  --(pooled TCP, < 1 ms)--> Redis and RDS Proxy -> Postgres (data subnets)
  --(PrivateLink or NAT)--> payment provider; SES and SNS via endpoints
```

DNS uses a 60-second TTL on alias records pointing at stable edge and load-balancer addresses, so failover between the two regions (Chapter 16) moves traffic behind those addresses rather than waiting on caches. The waiting room streams positions over SSE; the organizer dashboard uses SSE too; the mobile app opens one HTTP/3 connection and multiplexes everything over it. Encore's support agent reaches the order service through the internal gateway and the outside world only through an egress proxy with an allow-list.

4.15 Numbers to remember

Figure	Value
Light in fiber	about 200 km per ms
Frankfurt to Virginia RTT	about 90 ms (65 ms floor)
TCP initial window	10 segments, about 14.6 KB
New connection, TCP + TLS 1.3	2 RTT before the request
QUIC / HTTP/3 new connection	1 RTT (0 on resumption)
Ethernet MTU, VPC jumbo frames	1,500 / 9,001 bytes
Linux ephemeral ports	32768 to 60999 (28,232)
TIME_WAIT on Linux	60 s
AWS ALB idle timeout default	60 s
Postgres `max_connections` default	100

4.16 Pitfalls

- Opening a new connection per request -> one long-lived client per process with a sized pool.
- Long DNS TTLs for failover, or clients that never re-resolve -> short TTLs on stable anycast or load-balancer addresses.
- Keep-alive longer than the server's idle timeout -> client expiry below the balancer's 60 s, plus retries on idempotent calls only.
- gRPC behind an L4 balancer, all calls landing on one pod -> L7 or client-side balancing.
- 0-RTT accepted for `POST` -> allow early data only for idempotent `GET`s.
- Long-lived WebSockets without heartbeats or draining -> pings under the idle timeout, graceful drain on deploy.
- Every pod connecting straight to Postgres -> PgBouncer or RDS Proxy sized by Little's law.

4.17 Remember

- Latency is distance and round trips: count RTTs first; bandwidth matters only for large objects.
- A new HTTPS connection costs two round trips (one with QUIC); reuse connections and terminate TLS near the user.
- HTTP/2 multiplexes streams but keeps TCP's head-of-line blocking; HTTP/3 on QUIC removes it and survives network changes.
- Pools, sized with $C = \lambda W$, protect databases and dependencies from your own autoscaler.
- Push to clients with SSE for one-way feeds and WebSockets for two-way traffic, with heartbeats and resumable state outside the gateway.
- Keep private traffic private: subnets per tier and zone, security groups, peering and private endpoints.

Part II. Interfaces and architecture

How components talk to each other, and how an application is cut into pieces that teams can build, deploy and scale.

In this part:

- APIs: styles, formats and contracts
- Application architecture

5 APIs: styles, formats and contracts

An API is a promise between two programs that are built, deployed and scaled by different people at different times. This chapter answers one question: for each conversation in a system — browser to backend, service to service, provider to you, backend to a million waiting fans — which style, which wire format and which contract rules make that promise cheap to keep and hard to break?

5.1 The picture

Think of a restaurant. The **menu** is the contract: what you may order and what comes back. Ordering at the counter and waiting is **request/response**. A pager that buzzes when your food is ready is a **callback** (a webhook). The screen that keeps showing order numbers as they come up is a **stream**. And the kitchen shouting "table 7 is free" to anyone listening is an **event**. Each suits a different task; a good restaurant uses all four. In Encore every arrow below is an API: clients speak REST over HTTPS, internal services speak gRPC, the waiting room streams positions over Server-Sent Events, the payment provider calls back through a webhook, and services announce facts on an event bus.

```
browser/app --REST/JSON-------> API gateway --gRPC/Protobuf--> seat inventory
    ^   ^                          |                        \-> orders
    |   +---SSE: queue position----+                              |
    |                                                             v
    +---push/e-mail <-- notification workers <--events (Avro)-- event bus
                                                                  ^
payment provider --webhook (signed JSON)--> payments service -----+
```

5.2 Choosing a style

Style	Shape	Requirements it meets	Avoid when
REST	resources + HTTP verbs	universal clients, HTTP caching	chatty fine-grained calls
gRPC	typed procedures, HTTP/2	low latency, strict types, streams	browsers without a proxy
GraphQL	one endpoint, client picks fields	flexible reads, fewer round trips	simple CRUD, heavy HTTP caching
SSE	server pushes text events	one-way push, auto-resume	client must send often
WebSocket	two-way message channel	two-way low latency	one-way updates (use SSE)
Webhook	provider POSTs to you	cross-company notification, no polling	caller needs an answer now
Event API	publish facts to a log/bus	decoupling, fan-out, replay	caller needs the result

The requirements decide. **Latency and coupling**: a call that must answer within the user's click is request/response (REST or gRPC); work that may finish later is asynchronous (202 Accepted, a queue, or an event). **Who initiates**: if the server knows first (a seat freed, a payment settled), push it (SSE, WebSocket, webhook) instead of letting clients poll. **Audience**: public clients want REST and JSON — every language, every proxy and every CDN understands them; internal services want gRPC and Protobuf for types, speed and streaming. **Shape of reads**: many different screens composing data from many services is GraphQL's home ground. **Fan-out**: one fact, many consumers that the producer should not know about, is an event.

5.3 REST over HTTP

REST models the system as **resources** named by URLs (`/v1/events/4211`, `/v1/holds/h_8f3k`) and manipulates them with a small, fixed set of HTTP methods. The uniform interface is the point: caches, proxies, gateways and client libraries all understand GET, ETag and 304 without knowing anything about tickets.

Method	Safe	Idempotent	Use
GET, HEAD	yes	yes	read a resource or collection
PUT	no	yes	create or replace at a known URL
DELETE	no	yes	remove (repeat: still gone)
POST	no	no	create in a collection, run an action
PATCH	no	no	partial update

Safe means no intended side effect, so crawlers and prefetchers may call it; *idempotent* means sending it twice has the same effect as once, so a client may retry it after a timeout (RFC 9110). POST and PATCH are neither, which is why payments and holds need an idempotency key (below). Status codes tell the client what to do next, not only what happened: `200 OK`, `201 Created` (with `Location`), `202 Accepted` (work continues), `204 No Content`, `304 Not Modified`; `400` malformed, `401` not authenticated, `403` not allowed, `404` missing, `409 Conflict` (seat already held), `412 Precondition Failed` (stale ETag), `422` semantically invalid, `429 Too Many Requests` (with `Retry-After`); `500` bug, `502/504` upstream failed or timed out, `503` overloaded. The rule for clients: retry 429, 502, 503 and 504 with backoff (Chapter 11); never blindly retry other 4xx.

Good REST design is mostly naming: plural nouns for collections, IDs in the path, filters in the query string, verbs only for actions that are not CRUD (`POST /v1/holds/h_8f3k:confirm` in Google's style, or `POST /v1/holds/h_8f3k/confirmation`). Google's public API Improvement Proposals (AIPs) are a worked rulebook for resource-oriented design, used across Google Cloud. REST runs everywhere: behind Amazon API Gateway (Google Cloud API Gateway or Apigee, Azure API Management), an ALB or Envoy, served by FastAPI, Spring or Express.

5.4 RPC and gRPC

RPC (remote procedure call) makes a network call look like a function call: `inventory.HoldSeats(req)`. **gRPC** is the dominant modern form. You describe services and messages in a `.proto` file; the compiler generates typed client stubs and server skeletons in a dozen languages; calls travel as Protobuf messages over HTTP/2, one stream per call, many calls multiplexed on one TCP connection (Chapter 4).

```
syntax = "proto3";
package encore.inventory.v1;

service SeatInventory {
  rpc HoldSeats(HoldSeatsRequest) returns (Hold);              // unary
}
message HoldSeatsRequest {
  int64 event_id = 1;
  repeated string seat_ids = 2;
  string request_id = 3;          // idempotency key, travels with the call
}
message Hold {
  string id = 1;
  repeated string seat_ids = 2;
  int64 expires_at_unix = 3;
  reserved 4;                     // was "price"; never reuse the number
}
```

gRPC offers four call shapes: unary, server streaming (`rpc WatchSeats(WatchRequest) returns (stream SeatChange)` pushes seat changes as they happen), client streaming and bidirectional streaming. Every call should carry a **deadline**; the server sees the remaining time and passes it on, so a request whose user has given up stops consuming capacity three services down. Errors use a fixed set of 17 status codes (`OK`, `INVALID_ARGUMENT`, `NOT_FOUND`, `ALREADY_EXISTS`, `FAILED_PRECONDITION`, `RESOURCE_EXHAUSTED`, `DEADLINE_EXCEEDED`, `UNAVAILABLE`, ...); `UNAVAILABLE` is the code meant to be retried with backoff.

Two operational facts matter in designs. Browsers cannot speak native gRPC, so public clients need gRPC-Web or Connect through a proxy such as Envoy, or a REST facade (gRPC transcoding at the gateway). And because one HTTP/2 connection carries many calls, an L4 load balancer pins all of a client's calls to one backend; use an L7 balancer (Envoy,

ALB with gRPC target groups) or client-side balancing (Chapter 8). gRPC runs inside Kubernetes service meshes, Google's internal Stubby lineage, Google Cloud APIs, etcd and many databases' client protocols. Older RPC styles (JSON-RPC, Thrift, SOAP) follow the same idea with different encodings; JSON-RPC 2.0 is also the message layer of the Model Context Protocol (MCP) that agents use to call tools. In Encore the gateway keeps REST/JSON toward browsers and calls seat inventory over gRPC: at 30,000 hold attempts per second, Protobuf messages several times smaller than JSON (measured below), multiplexed connections and deadline propagation that cancels abandoned holds are worth the cost — L7 balancing for HTTP/2, schema discipline for `.proto` files, and `grpcurl` instead of curl for debugging.

5.5 GraphQL

REST gives each resource its own URL, so the event page of the Encore app may need five round trips (event, venue, price tiers, artist, my tickets) and receives fields it never shows. GraphQL turns this around: one endpoint (`POST /graphql`), a typed **schema** of the whole graph, and the client writes a **query** naming exactly the fields it wants, for example `query { event(id: 4211) { name venue { name city } priceTiers { name priceCents } } }`. The server walks the query and calls a **resolver** per field, which may fetch from any service.

Mutations change data (`holdSeats(eventId:, seatIds:)`) and subscriptions push updates, usually over WebSockets. The famous cost is the **N+1 problem**: a list of N events, each resolving its venue, issues one query for the list and N for the venues. A per-request batching loader (DataLoader) collects the keys and fetches them in one call, so $1 + N$ queries become 1 + 1. For a search page of 50 events that is 51 backend calls against 2. The other costs: every request is a POST to one URL, so CDN and HTTP caching do not work out of the box (persisted queries sent as GET with a hash restore it); a malicious client can write a deeply nested query, so production servers enforce depth and **query cost** limits; and per-field authorization must be designed in. GraphQL suits a **backend for frontend** that composes many services for many screens (Chapter 6); it runs on Apollo Server and Router, AWS AppSync, Hasura and GitHub's public GraphQL API. For a simple CRUD API or a hot, cacheable read like Encore's seat map, REST is simpler and faster.

5.6 Pushing to clients: WebSockets and SSE

When the server learns something first, the client has three ways to hear about it. **Polling** asks every T seconds; **long polling** asks and the server holds the request open until there is news; a **persistent connection** lets the server push. Polling costs clients/T requests per second whether or not anything changed: a million fans polling the waiting room every 5 seconds is $10^6/5 = 200,000$ requests/s — as much as the whole seat-map peak — almost all answering "no change".

Server-Sent Events (SSE) is one long HTTP response with `Content-Type: text/event-stream`. The server writes events as lines; the browser's `EventSource` parses them, reconnects automatically and sends `Last-Event-ID` so the server can resume. **WebSocket** (RFC 6455) instead starts as an HTTP request with `Upgrade: websocket`, gets `101 Switching Protocols`, and then both sides exchange framed messages until one closes.

```
id: 1842
event: position
data: {"position": 48211, "eta_s": 540}

id: 1843
event: admitted
data: {"token": "wr_9c1f...", "expires_in_s": 600}
```

	Polling	SSE	WebSocket
Direction	client pulls	server to client	both ways
Reconnect and resume	trivial	built in (`Last-Event-ID`)	you build it
Proxies, CDNs, HTTP/2	everything works	works; multiplexed on HTTP/2	needs upgrade support
Fits	rare changes	feeds, progress, LLM tokens	chat, games, live editing

Both persistent options move the cost from requests to **open connections**: each holds a socket, buffers and a little state on a server, and a deploy or crash drops thousands at once, so clients must reconnect with jitter. Browsers open at most six HTTP/1.1 connections per host, which caps SSE tabs unless it runs over HTTP/2. Managed WebSocket endpoints (Amazon API Gateway WebSocket APIs, Azure Web PubSub; on Google Cloud, Cloud Run or a load balancer in front of your own servers) handle the connection fleet; Socket.IO, Phoenix and Centrifugo are common self-hosted layers. *50 System Designs* #6 (Chat System) builds a WebSocket gateway in depth.

> **Design question.** How should 1,000,000 fans in Encore's waiting room learn their queue position and their admission?
>
> Requirements: one-way updates, a position that changes every few seconds, admission within seconds of their turn, and survival of reconnects. Polling every 5 s costs 200,000 requests/s for mostly unchanged answers. SSE fits: one HTTP response per fan, updates pushed every 5 to 10 s (jittered), the admission token sent as a final event, `Last-Event-ID` for resume. At an illustrative 50,000 open connections per edge server that is 20 servers, and the stream can be served by a stateless fleet that reads positions from Redis. The trade-off: a fleet of long-lived connections to drain carefully on deploys; keep a polling fallback for clients behind hostile proxies. See also *50 System Designs* #25 (Ticket Booking).

5.7 Webhooks and event APIs

A **webhook** is an API you host for someone else to call: you register a URL, and the provider POSTs an event to it when something happens. Encore's payment provider confirms a payment this way, seconds or minutes after the customer authorised it. Because anyone on the internet can POST to that URL, and because the provider retries until it sees a `2xx`, a correct receiver does four things: verify a **signature**, reject **stale timestamps** (replay protection), **deduplicate** by event ID, and **acknowledge fast** — store the event, return 200, process from a queue.

```
import hashlib, hmac, time
SECRET = b"whsec_encore_demo"            # shared with the payment provider

def sign(body: bytes, ts: int) -> str:
    mac = hmac.new(SECRET, f"{ts}.".encode() + body, hashlib.sha256)
    return f"t={ts},v1={mac.hexdigest()}"

def verify(body: bytes, header: str, now: float) -> bool:
    parts = dict(p.split("=", 1) for p in header.split(","))
    ts = int(parts["t"])
    if abs(now - ts) > 300:               # reject replays older than 5 min
        return False
    expected = sign(body, ts).split("v1=")[1]
    return hmac.compare_digest(expected, parts["v1"])

body = b'{"id":"evt_91","type":"payment.succeeded","order":"o_77"}'
hdr = sign(body, int(time.time()))
print(verify(body, hdr, time.time()),                  # True
      verify(body, hdr, time.time() + 600))            # False: too old
```

The signature covers the timestamp and the **raw bytes** of the body, so verify before any JSON parsing reorders keys; `compare_digest` avoids a timing leak. This mirrors Stripe's `Stripe-Signature: t=...,v1=...` scheme (Stripe retries undelivered events for up to three days); the Standard Webhooks specification defines the same idea with `webhook-id`, `webhook-timestamp` and `webhook-signature` headers. Webhooks are at-least-once and unordered, so never trust them alone: reconcile against the provider's API on a schedule (Chapter 24).

Event APIs go one step further: the producer publishes facts (`order.paid`) to a broker — Kafka, Amazon SNS/SQS, EventBridge (Google Pub/Sub or Eventarc, Azure Event Grid or Event Hubs) — and any number of consumers subscribe without the producer knowing them (Chapter 14). The contract is the event schema plus the channel. **AsyncAPI** (version 3.1.0) is to event APIs what OpenAPI is to REST; **CloudEvents** standardises the envelope (`id`, `source`, `type`, `specversion`, plus `time` and `data`).

```
asyncapi: 3.1.0
info: {title: Encore order events, version: 1.4.0}
channels:
  orderPaid:
    address: orders.paid
    messages:
      orderPaid:
        contentType: application/json
        payload:
          type: object
          required: [order_id, total_cents]
          properties: {order_id: {type: string}, total_cents: {type: integer}}
operations:
  publishOrderPaid: {action: send, channel: {$ref: '#/channels/orderPaid'}}
```

5.8 Formats on the wire

A format decides three things: how big a message is, how fast it is to read and write, and what happens when the two sides disagree about its shape.

Format	Encoding	Schema	Self-describing	Typical home
JSON	text	optional (JSON Schema)	yes, names in every record	public APIs, browsers, logs
MessagePack	binary JSON	none	yes	caches, compact client protocols
Protobuf	binary, numbered fields	required (`.proto`)	no	gRPC, internal storage
Avro	binary, no field tags	required, sent or registered	no	Kafka, Hadoop, lakehouse files

JSON repeats every field name in every record and spells numbers as decimal text. MessagePack keeps the names but writes values in binary. Protobuf replaces each name with a small tag (field number plus wire type) and writes integers as **varints**, 7 bits per byte:

$$\text{bytes}(v) = max(1, \frac{log_2 v \ + 1}{7})$$

A seat ID of 100,000 needs 17 bits, so 3 bytes instead of 6 characters; field numbers 1 to 15 fit their tag in one byte (the formula holds for $v \geq 1$; zero takes one byte). Avro writes no tags at all — values follow the schema's field order — so it is smallest, but the reader must have the writer's schema. Measured sizes for a 1,000-seat section of Encore's seat map (seat ID, section, row, number, state, price), encoded with common Python libraries (json, msgpack, protobuf, fastavro):

Format	Size	Per seat	Gzipped
JSON (compact)	87.4 KB	87 B	5.8 KB
MessagePack	63.0 KB	63 B	5.2 KB
Protobuf	17.3 KB	17 B	4.2 KB
Avro	11.0 KB	11 B	3.3 KB

Protobuf and Avro are 5 to 8 times smaller than JSON raw, but compression closes most of the gap (1.8 times between JSON and Avro here), because repeated field names compress extremely well. The bigger win of binary formats is CPU: no text parsing, no number formatting. Choose JSON at the edge, where debuggability and universality matter; Protobuf between services; Avro or Protobuf with a schema registry on Kafka; columnar Parquet for analytics (Chapter 21).

5.8.1 Schema evolution

Producers and consumers deploy at different times, so for a while old readers see new data and new readers see old data. **Backward compatible** means new readers can read old data; **forward compatible** means old readers can read new data; you want both.

Change	JSON	Protobuf	Avro
Add an optional field	safe if readers ignore unknowns	safe (unknown fields skipped)	safe with a default
Remove a field	safe if readers tolerate absence	safe; `reserved` the number	safe if it had a default
Rename a field	breaks	safe (names not on the wire)	use `aliases`
Change a field's type	breaks	only compatible pairs	only promotions (int to long)

The Confluent Schema Registry (also AWS Glue Schema Registry, Azure Event Hubs schema registry) stores every version, checks each new one against a compatibility mode — BACKWARD by default, or FORWARD, FULL, their TRANSITIVE variants, or NONE — and stamps every Kafka message with a 5-byte prefix (magic byte plus schema ID) so readers fetch the writer's schema.

> **Design question.** What format should Encore use to send the seat map of a 50,000-seat stadium to 200,000 readers per second?
>
> Split the static from the dynamic. The layout (coordinates, sections, prices) changes rarely: serve it as a versioned JSON file from the CDN with a long TTL (Chapter 10). The live part is one of four states per seat, 2 bits: $50,000 \times 2/8 = 12,500$ bytes as a packed bitmap, against 4.4 MB as JSON objects (289 KB gzipped). At 200,000 reads/s the bitmap is still 2.5 GB/s (20 Gbit/s), so cache it at the edge with a 1 to 2 s TTL and send seat-level deltas over SSE to clients already on the page. The trade-off: a custom binary encoding needs a versioned spec and client decoders. See also *50 System Designs* #25 (Ticket Booking).

5.9 The contract

A style says how calls travel; the contract says what a well-behaved call looks like. These rules come up in every design review and most interviews.

5.9.1 Idempotency keys

A client sends `POST /holds`, the network drops the response, the client retries: without protection the fan now holds four seats. The fix: the client generates a unique key per logical operation (a UUID) and sends it as `Idempotency-Key`; the server stores key, a hash of the request and the response, and replays the stored response on a retry. The IETF draft for the header (now expired) recommends `400` when the key is missing, `409` while the first request is still running and `422` when the key comes back with a different body; Stripe keeps keys for at least 24 hours and accepts up to 255 characters. Claim the key atomically before doing the work:

```
SET idem:user42:7f3c9a2e "pending" NX EX 86400     -> OK (first), nil (retry)
SET idem:user42:7f3c9a2e "a91f|201|{...}" XX KEEPTTL  (store the response)
```

Scope keys per user, so one client cannot replay another's response. Sizing for the flash sale: $30,000/\text{s} \times 60\ \text{s} = 1.8$ million keys in the first minute; at about 500 bytes each (illustrative) that is 0.9 GB, one Redis node. The same rule applies inside the system: the hold request carries the key to seat inventory, and orders pass it on to the payment provider (Chapter 3).

5.9.2 Pagination

Offset pagination (`?offset=20000&limit=100`) makes the database walk past every skipped row. **Cursor** (keyset) pagination returns an opaque token holding the last row's sort key, and the next page seeks straight to it through an index:

$$\text{cost}_{\text{offset}}(p) = O(pL + L), \qquad \text{cost}_{\text{cursor}}(p) = O(\log n + L)$$

An organizer exporting all 50,000 orders of the stadium show in pages of $L = 100$: offset touches $\sum_{p=0}^{499}(100p + 100) = 12,525,000$ rows, cursor touches 50,000 plus 500 index seeks — 250 times less work. Offsets also skip or repeat rows when new orders arrive mid-scan; cursors do not.

```sql
-- next page after the cursor (created_at, id) = (1790000100, 300)
SELECT id, created_at, total_cents FROM orders
WHERE event_id = 4211 AND (created_at, id) > (1790000100, 300)
ORDER BY created_at, id
LIMIT 100;  -- index on (event_id, created_at, id); works in Postgres, SQLite
```

Return `{"items": [...], "next_page_token": "eyJ..."}` and keep the token opaque (base64 of the key, ideally signed), so you can change its contents later. Offsets remain fine for small, bounded lists where users jump to page 7. Put filters and sorts in the query string with a documented grammar (`?status=paid&sort=-created_at`, or Google's `filter=` string in AIP-160), allow only indexed fields, and cap `limit`. Let clients trim responses with field selection (`?fields=id,status`, Google's `FieldMask`), which saves bandwidth without the full weight of GraphQL.

5.9.3 Versioning

Additive changes — a new optional field, a new endpoint, a new enum value that clients were told to tolerate — are not breaking; everything else is. Put the major version in the path (`/v1/`, as Google does) or in a header; Stripe pins each account to a dated version (`Stripe-Version: 2026-...`) and upgrades it on request. Run old and new versions side by side, announce removal with the `Deprecation` (RFC 9745) and `Sunset` (RFC 8594) headers, and watch traffic per version before switching off.

5.9.4 Errors: problem details

RFC 9457 (which replaced RFC 7807) defines one JSON shape for errors, served as `application/problem+json`, so every client handles every error the same way:

```http
{
  "type": "https://api.encore.example/problems/seat-unavailable",
  "title": "Seat unavailable",
  "status": 409,
  "detail": "Already held or sold: A-12",
  "seat_ids": ["A-12"]
}
```

`type` is a stable identifier clients branch on; `title` is for humans; extension members such as `seat_ids` carry machine-readable details. Never leak stack traces.

5.9.5 Rate-limit headers

When a client exceeds its quota, answer `429 Too Many Requests` with `Retry-After` (RFC 9110). To let well-behaved clients slow down before they hit the wall, advertise the quota. The de facto headers are `X-RateLimit-Limit`, `-Remaining` and `-Reset` (GitHub style); the IETF draft (draft-ietf-httpapi-ratelimit-headers-11, May 2026, not yet an RFC) standardises them as structured fields, for example `RateLimit-Policy: "holds";q=5;w=60` (5 holds per 60 s) and `RateLimit: "holds";r=0;t=12` (none left, window resets in 12 s). The algorithms behind the numbers are in Chapter 11.

5.9.6 Authentication

API keys identify a calling application (server-to-server, partners): long random strings with a readable prefix, stored hashed, scoped and rotatable. **OAuth 2.0 bearer tokens** (`Authorization: Bearer <token>`, RFC 6750) identify a user or a client acting with delegated rights: users log in through the authorization code flow with PKCE, machines use client credentials. Access tokens are short-lived (minutes); a JWT can be validated locally from the issuer's public keys, while an opaque token needs an introspection call. Inside the network, services authenticate each other with mTLS. Details, flows and pitfalls are in Chapter 22.

5.9.7 ETags and conditional requests

An **ETag** is a version tag for a representation. Reads send `If-None-Match` and get `304 Not Modified` with no body when nothing changed; writes send `If-Match` and get `412 Precondition Failed` if someone else changed the resource first — optimistic concurrency over HTTP (servers may demand it with `428 Precondition Required`). The organizer's second browser tab loses cleanly instead of overwriting a newer price change:

```
PATCH /v1/events/4211 HTTP/1.1
If-Match: "e4211-v17"

{"on_sale": false}

HTTP/1.1 412 Precondition Failed
ETag: "e4211-v18"
```

5.10 Paradigms for different tasks

Task	Paradigm	Mechanism	Encore example
answer within a click	request/response	REST or gRPC, deadline	hold seats, show event
work that takes seconds to hours	long-running operation	`202` + status resource	order export, refund
result trickles out	streaming	SSE, gRPC stream, chunked	queue position, LLM tokens
many small items at once	batch	one call, per-item results	fetch 40 tickets
millions of records	bulk	async job + object storage	import a venue, export sales
tell others something happened	event	publish to a bus or webhook	`order.paid`

Long-running operations. Never hold an HTTP request open for minutes: gateways and load balancers cut it (Amazon API Gateway REST APIs at 29 s by default), clients retry and duplicate the work. Accept the job, return a handle, and let the client poll it, subscribe to it, or receive a callback:

```
POST /v1/events/4211/exports HTTP/1.1
Idempotency-Key: 2b7e...

HTTP/1.1 202 Accepted
Location: /v1/operations/op_51c
Retry-After: 5

GET /v1/operations/op_51c HTTP/1.1

HTTP/1.1 200 OK

{"id": "op_51c", "done": true,
 "result": {"url": "https://exports.encore.example/op_51c.csv?X-Amz-..."}}
```

Google formalises this as the `Operation` resource (AIP-151); AWS Step Functions and Temporal run the job behind it durably (Chapter 14). **Batch** endpoints (`POST /v1/tickets:batchGet`) save round trips but must report per-item success, because item 17 can fail while the rest succeed. **Bulk** transfers never pass through the API servers at all: the API hands out a presigned object-storage URL and the client uploads or downloads directly from S3 (Cloud Storage, Azure Blob Storage).

> **Design question.** An organizer clicks "Export all orders" for the stadium show. How should the API behave?
>
> 50,000 orders at about 2 KB each is 100 MB of order data: too slow for one request and too large for a gateway payload (10 MB on Amazon API Gateway). Make it a long-running operation: `POST /exports` with an idempotency key returns `202` and an operation ID; a worker reads the orders with cursor pagination from a replica, writes a CSV

> to S3, and marks the operation done with a presigned URL valid for 15 minutes; the dashboard polls every 5 s (honouring `Retry-After`) or gets an SSE "done" event. The trade-off: a job table, a worker pool and expiry rules instead of one endpoint — in exchange for retries that cannot double the work.

Agents and LLM services. Model APIs stream tokens to the user over SSE; an agent task that may run for minutes (Encore's support agent checking an order and requesting a refund that waits for human approval) is a long-running operation with a status resource; every tool the agent calls is an ordinary API that must be idempotent, authorized and rate-limited, because a model retries and improvises more than any human client. MCP servers expose those tools over JSON-RPC. *Designing Agentic Systems* covers the agent side; Chapter 24 covers the plumbing.

5.11 API gateways

An API gateway is the front door that enforces the contract once instead of in every service: TLS termination, authentication (validating JWTs or API keys), routing by path and version, rate limits and quotas per key, request validation against the OpenAPI schema, REST-to-gRPC transcoding, response caching, CORS, and logging with a request ID. Managed choices are Amazon API Gateway (Google Cloud API Gateway or Apigee, Azure API Management); self-hosted ones are Envoy-based gateways, Kong, NGINX and Traefik. Know the managed limits: Amazon API Gateway defaults to 10,000 requests/s per account per Region with a 5,000-request burst bucket, a 10 MB payload and a 29 s integration timeout for REST APIs; its WebSocket APIs cap connections at 2 hours and 10 minutes idle. Encore's flash-sale hold rate of 30,000/s is three times that default quota, so either raise it well in advance or put the hot path behind an ALB and Envoy fleet you size yourself. Gateway placement and load balancing are in Chapter 8.

5.12 In Encore: holds and orders

Encore's public API is REST/JSON behind the gateway; each endpoint states its contract.

Endpoint	Purpose	Contract
`GET /v1/events/{id}`	event page data	cacheable, ETag
`GET /v1/events/{id}/seat-state`	packed seat states	CDN, 1 to 2 s TTL
`POST /v1/events/{id}/holds`	hold seats for 10 min	idempotency key, admission token, 201 or 409
`POST /v1/orders`	pay for a hold	idempotency key, 202
`GET /v1/orders/{id}`	order status and tickets	ETag, poll or push
`POST /webhooks/payments`	provider callback	signature, dedupe by event ID

A fan holds two seats, the response is lost, the app retries, and a second fan tries one of the same seats (traces trimmed; produced by a small FastAPI implementation of the endpoint):

```
POST /v1/events/4211/holds HTTP/1.1
Idempotency-Key: 7f3c9a2e-1d4b-4c8e-9f60-2a7b5e1c0d93

{"seat_ids": ["A-12", "A-13"]}

HTTP/1.1 201 Created
Location: /v1/holds/h_1283bbf9

{"id": "h_1283bbf9", "event_id": 4211, "seat_ids": ["A-12", "A-13"],
 "status": "held", "expires_at": 1791960801}

(same request again, same key)  ->  HTTP/1.1 201 Created
                                    Idempotent-Replayed: true  (same body)
(other fan, other key, ["A-12"]) -> HTTP/1.1 409 Conflict
                                    Content-Type: application/problem+json
```

Paying is asynchronous, because the provider confirms later through the webhook: `POST /v1/orders` with `{"hold_id": "h_1283bbf9"}` and a new idempotency key returns `202 Accepted`, `Location: /v1/orders/o_77` and `"status": "pending_payment"`. The orders service passes its own idempotency key to the provider, the signed `payment.succeeded` webhook moves `o_77` to `paid`, an `order.paid` event wakes the ticket and notification workers, and the app learns it by polling `GET /v1/orders/o_77` with `If-None-Match` or by a push. The contract lives in an OpenAPI document (version 3.1 is the widely supported one; 3.2 is current), from which the gateway validates requests and client SDKs are generated; the full flow, with the database schema and race-condition tests, is built in Chapter 25:

```
openapi: 3.1.1
info: {title: Encore Public API, version: 1.0.0}
paths:
  /v1/events/{event_id}/holds:
    post:
      parameters:
        - {name: event_id, in: path, required: true, schema: {type: integer}}
        - name: Idempotency-Key
          in: header
          required: true
          schema: {type: string, maxLength: 255}
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [seat_ids]
              properties:
                seat_ids: {type: array, items: {type: string}, maxItems: 8}
      responses:
        '201': {description: Seats held for 10 minutes}
        '409':
          description: A seat is already held or sold
          content:
            application/problem+json:
              schema: {$ref: '#/components/schemas/Problem'}
components:
  schemas:
    Problem:
      type: object
      required: [type, status]
      properties: {type: {type: string}, status: {type: integer}}
```

5.13 Numbers to remember

Figure	Value
Idempotency key retention (Stripe)	at least 24 h, up to 255 chars
Offset vs cursor, 50,000 rows in pages of 100	12.5 million vs 50,000 rows read
Polling cost	clients / interval: 1M fans every 5 s = 200,000 req/s
Seat map, 1,000 seats	JSON 87 KB, Protobuf 17 KB, Avro 11 KB; gzipped 3 to 6 KB
Protobuf varint	7 bits per byte; tags of fields 1 to 15 take 1 byte
Amazon API Gateway defaults	10,000 req/s, 5,000 burst, 29 s, 10 MB
Webhook replay window	5 min timestamp tolerance (Stripe default)

5.14 Pitfalls

- Retrying a POST without an idempotency key -> double holds and double charges; require the key and store the response.

- Offset pagination on large or changing tables -> slow deep pages and skipped rows; use opaque cursors over an index.
- Breaking changes inside `/v1` (renamed field, new required field) -> only add; ship `/v2` side by side with `Deprecation` and `Sunset` headers.
- Reusing a Protobuf field number -> old readers decode garbage silently; mark removed numbers `reserved`.
- Long jobs as one HTTP request -> gateway timeouts and duplicate work; return `202` with an operation resource.
- Trusting a webhook body unverified or processing it inline -> forged events and provider retries; verify the signature on raw bytes, dedupe, acknowledge, queue.

5.15 Remember

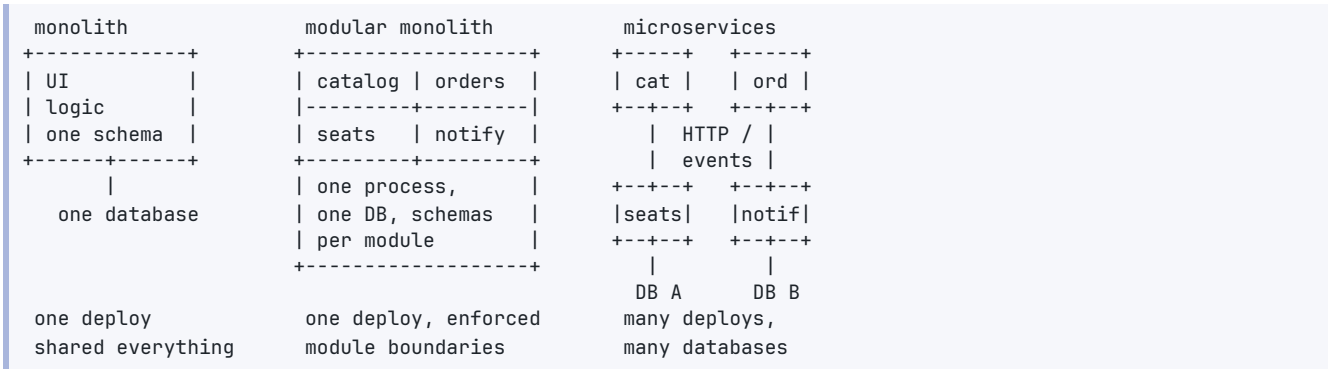
- Pick the style from the requirements: answer now (REST, gRPC), answer later (202 plus status, webhook), server knows first (SSE, WebSocket), many unknown listeners (events).
- JSON at the edge, Protobuf between services, Avro or Protobuf with a registry on streams; compression narrows the size gap, schemas guard evolution.
- Every unsafe operation needs an idempotency key; every list needs cursor pagination; every error uses one problem format.
- The contract is the product: version additively, advertise limits, use ETags for caching and optimistic concurrency.
- Gateways enforce the contract once — and have quotas of their own to plan for.

6 Application architecture

An application is more than the code that answers one request: it is a set of pieces with boundaries, owners and contracts. This chapter answers one question — where do you draw the lines inside a system, and what crosses them? — in one picture: boxes that each own their data, joined by a few calls that must answer now and many events that may arrive later. It covers monoliths and microservices, how to find service boundaries, the layers inside one service, how services talk and who owns which data, event-driven designs, CQRS and event sourcing, serverless, backends for frontends, service meshes, configuration, and Encore's service map.

6.1 The picture

Think of a restaurant. A food truck is a monolith: one cook does everything, nothing is lost between stations, and it works well until the queue goes round the block. A large kitchen is a brigade: a grill station, a pastry station, a cold station, each with its own fridge. Stations pass orders on tickets, not by reaching into each other's fridges. The brigade serves far more covers, at the price of a pass, tickets and a head chef who coordinates. The food court is the extreme: independent stalls, each with its own staff and opening hours.



Moving right buys independent deployment and scaling per piece; it costs network calls, partial failure and distributed data. Every section below is about paying that cost only where it buys something.

6.2 Monolith, modular monolith and microservices

A **monolith** is one deployable unit: one codebase, one process type, usually one database. Calls between parts are function calls (nanoseconds, never lost), transactions span the whole data model, and one test run covers everything. It starts failing when many teams queue behind one release train, when one hot path (Encore's seat inventory) needs a hundred times the capacity of the rest, or when a memory leak in the PDF renderer takes checkout down with it.

A **modular monolith** keeps one deployment but cuts the code into modules with enforced boundaries: each module has a public interface, private internals and its own tables (often its own schema). It gives most of the design benefits of services with none of the network cost, and it is the cheapest place to discover where the boundaries really are. Boundaries that are not enforced erode, so enforce them in CI:

```
# .importlinter  (run: lint-imports; fails the build on a violation)
[importlinter]
root_package = encore

[importlinter:contract:inventory-private]
name = Other modules use only encore.inventory.api
type = forbidden
source_modules =
    encore.orders
    encore.catalog
forbidden_modules =
    encore.inventory.repository
```

Microservices are independently deployable services, each owning its data and exposing it only through an API or events. They buy independent releases, independent scaling, fault isolation and freedom of technology per service. The bill arrives as network calls that can be slow or fail, data spread across databases, distributed tracing, versioned contracts and a platform team to run it all, and the bill has a number: a request that must pass through n services in series is up only when all of them are up, and its latency is the sum of theirs:

$$A_{\text{chain}} = \prod_{i=1}^{n} A_i \qquad L_{\text{chain}} = \sum_{i=1}^{n} L_i$$

With five components at 99.95% each (gateway, waiting room, seat inventory, Redis, Postgres), a hold succeeds 99.75% of the time: about 1,313 minutes, nearly 22 hours, of failed holds a year instead of 263 minutes for one component. Every synchronous hop also adds a network round trip and serialization, roughly 0.2 to 1 ms inside one zone (illustrative), against nanoseconds for a function call. Chapter 23 develops this availability math.

Pick a monolith for one team and an unclear domain, a modular monolith for several teams that can share one release cadence, and microservices when teams block each other or parts have very different load or failure needs. Where it runs: monoliths on a VM group or one container service behind a load balancer (Amazon ECS on AWS Fargate; Google Cloud Run; Azure Container Apps); microservices usually on Kubernetes (Amazon EKS; GKE; AKS) or many small container services. Shopify runs its core commerce application as a large modular Rails monolith; Packwerk (Ruby), Spring Modulith (Java) and import-linter (Python) check module boundaries.

6.3 Drawing service boundaries

Cut along the business, not along the database tables. Domain-driven design calls a region with one consistent vocabulary a **bounded context**. The word “event” shows why it matters at Encore: in the catalog an event is a show with a date, venue and description; in seat inventory it is a grid of 80,000 seat states; in analytics an “event” is a click. Three models, three contexts, three owners. A service per bounded context keeps each model small and lets each change without negotiating with the others. Inside a context, an **aggregate** is the cluster of data that must change together in one transaction, such as one event’s seat states plus its active holds. Aggregates are the consistency boundary: anything that must be atomic stays inside one service and one database; anything else may be eventually consistent and cross a boundary as an event.

Four tests for a good boundary:

- **Change together, deploy together.** Code that changes in the same pull requests belongs in the same service.
- **One transaction.** If two pieces need a joint atomic write, do not split them.
- **Different load or failure profile.** Seat inventory takes 30,000 holds/s for a minute; the catalog takes a few writes an hour. That difference justifies a split.
- **One team.** Conway’s law: systems mirror the communication structure of the organizations that build them. Coordination paths grow with the square of the number of teams:

$$\text{paths}(n) = \frac{n(n-1)}{2}$$

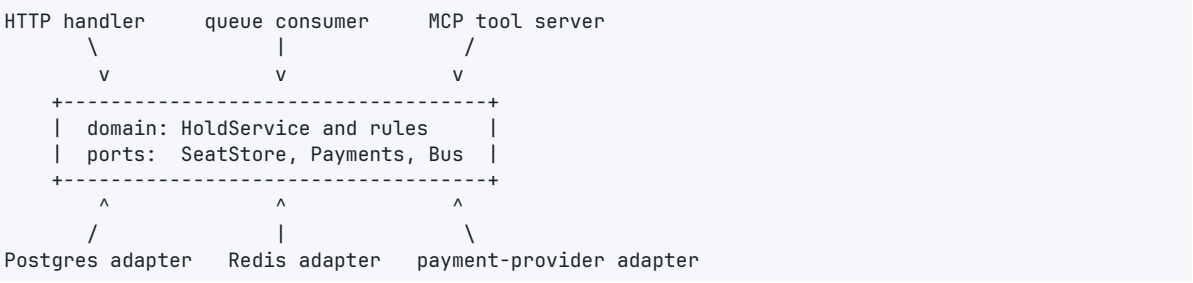
Three teams have 3 paths; eight teams have 28; twenty have 190. Services aligned to teams turn most of those paths into stable API contracts. Encore’s resulting contexts appear in the service map at the end of this chapter. The classic mistake is the **entity service**: a “seat service”, a “user service” and an “order service” that are thin CRUD wrappers over tables. Every feature then calls all of them in a chain, and the result is a **distributed monolith** — the coupling of a monolith with the failure modes of a network.

> **Design question.** Encore launches with eight engineers. Should it start with microservices?
>
> No. Eight engineers are one or two teams (at most 1 coordination path), and the domain will move for a year. Start with a modular monolith: modules for catalog, inventory, orders, notifications, with one Postgres and a schema per module, boundaries checked in CI. Extract only what has a different load or failure profile, and do it early for the obvious ones: seat inventory (30,000 holds/s in a flash sale against 6,000 requests/s for the whole site on a

> normal peak), the waiting room, and the notification workers (slow external providers). Trade-off: a later extraction needs data migration, but a wrong early split costs more, because every wrong boundary becomes a synchronous chain. See also *50 System Designs* #25 (Ticket Booking).

6.4 Layered and hexagonal architecture

Inside one service, the same question repeats at a smaller scale: which code may depend on which? The **layered** answer stacks presentation (HTTP handlers), application services (use cases), domain (rules) and data access (repositories), and each layer calls only the one below. It is easy to teach and fine for CRUD, but the domain ends up depending on the database layer, so business rules cannot be tested without a database. The **hexagonal** answer (ports and adapters, Alistair Cockburn) turns the dependency around. The domain sits in the middle and declares **ports**: interfaces in its own vocabulary, such as "try to hold this seat until 10:15". **Adapters** on the outside implement them: a Postgres adapter, a Redis adapter, an in-memory fake for tests, an HTTP handler that drives the domain, a client for the payment provider. Dependencies point inward only; onion and clean architecture are the same idea with more rings.



A port is a `Protocol`; the adapters are plain classes. The domain service below holds a seat for 10 minutes and knows nothing about HTTP or SQL; the SQL adapter makes the hold safe with a conditional `UPDATE` that succeeds for only one of two racing fans (Chapter 3).

```python
# seats_core.py: domain service + port + two adapters (hexagonal)
import sqlite3, time, uuid
from typing import Protocol

HOLD_SECONDS = 600
class SeatTaken(Exception): pass

class SeatStore(Protocol):  # port: what the domain needs, in its words
    def try_hold(self, event: str, seat: str, user: str, now: float,
                 until: float) -> bool: ...

class SqlSeatStore:  # adapter: SQL; a conditional UPDATE is the guard
    def __init__(self, conn: sqlite3.Connection):
        self.conn = conn
    def try_hold(self, event, seat, user, now, until):
        cur = self.conn.execute(
            "UPDATE seats SET holder = ?, held_until = ?"
            " WHERE event_id = ? AND seat = ? AND status = 'free'"
            " AND (holder IS NULL OR held_until < ?)",
            (user, until, event, seat, now))
        self.conn.commit()
        return cur.rowcount == 1

class MemorySeatStore:  # adapter: in memory, for unit tests
    def __init__(self, seats):
        self.holds = {s: (None, 0.0) for s in seats}
    def try_hold(self, event, seat, user, now, until):
        holder, held_until = self.holds.get(seat, ("?", float("inf")))
        if holder is not None and held_until >= now:
            return False
        self.holds[seat] = (user, until)
        return True

class HoldService:  # business rules: no HTTP, no SQL, no Redis
    def __init__(self, store: SeatStore, clock=time.time):
        self.store, self.clock = store, clock
    def hold(self, event: str, seat: str, user: str) -> dict:
        now = self.clock()
        if not self.store.try_hold(event, seat, user, now,
                                   now + HOLD_SECONDS):
            raise SeatTaken(seat)
        return {"hold_id": str(uuid.uuid4()), "seat": seat,
                "expires_at": now + HOLD_SECONDS}

if __name__ == "__main__":  # a unit test that needs no database
    svc = HoldService(MemorySeatStore(["A1"]), clock=lambda: 1000.0)
    print(svc.hold("e1", "A1", "ana")["expires_at"])  # 1600.0
    try:
        svc.hold("e1", "A1", "ben")
    except SeatTaken:
        print("A1 already held")
```

Running it prints `1600.0` and `A1 already held`, with no database in sight. Swapping the Postgres adapter for a Redis one, or the payment provider for another, touches one class.

The adapter for an external system is also its **anti-corruption layer**: the payment provider's statuses, error codes and webhooks are translated at the edge into Encore's own terms (`PaymentCaptured`, `PaymentDeclined`), so a provider change does not ripple through the domain (Chapter 24). The same shape fits AI agents: Encore's support agent reaches orders through an MCP tool server that is one more driving adapter over the same application services, with the same authorization checks as the mobile app, never a side door into the tables. *Designing Agentic Systems* covers tool design in depth. Trade-off: ports and adapters add interfaces and wiring. For a service that is thin CRUD over one table, layering is enough; use hexagonal where rules are rich or external systems are many.

6.5 How a request flows through the code

Follow one hold request from the fan's phone to the database and back. Each layer has one job, and the arrows are the only places where things can go wrong.

```
phone -> CDN -> API gateway (auth, rate limit) -> load balancer -> pod
  handler     parse JSON, check token, validate input, map errors to HTTP
     |
  service     rules: hold 10 minutes, a per-fan seat limit, one transaction
     |
  repository  SQL or Redis commands; returns domain objects, not rows
     |
  Postgres / Redis / outbox table -> queue -> notification workers
```

Cross-cutting concerns sit around the handler as middleware rather than inside it: a request ID and trace context (Chapter 23), authentication (Chapter 22), a deadline for the whole request, and structured access logs. The deadline propagates: if the gateway allows 2 s, the service must give its database call less than what remains, or a slow dependency holds threads long after the client has given up. The FastAPI service below wires the previous block (saved as `seats_core.py`) behind an HTTP endpoint. `Depends` is the composition root: the one place that chooses adapters, so a test can override it with `app.dependency_overrides` and inject the in-memory store.

```
# app.py: handler -> service -> repository, wired with FastAPI Depends
import sqlite3
from fastapi import Depends, FastAPI, HTTPException
from pydantic import BaseModel
from seats_core import HoldService, SeatTaken, SqlSeatStore

db = sqlite3.connect(":memory:", check_same_thread=False)
db.executescript("""
CREATE TABLE seats (event_id TEXT, seat TEXT, status TEXT DEFAULT 'free',
  holder TEXT, held_until REAL, PRIMARY KEY (event_id, seat));
INSERT INTO seats (event_id, seat) VALUES ('e1', 'A1'), ('e1', 'A2');
""")
app = FastAPI()

class HoldIn(BaseModel):
    seat: str

def hold_service() -> HoldService:  # composition root: pick adapters
    return HoldService(SqlSeatStore(db))

@app.post("/events/{event_id}/holds", status_code=201)
def create_hold(event_id: str, body: HoldIn, user: str = "demo",
                svc: HoldService = Depends(hold_service)):
    try:  # the handler only translates HTTP <-> domain
        return svc.hold(event_id, body.seat, user)
    except SeatTaken:
        raise HTTPException(409, f"seat {body.seat} is already held")
```

Run it with `uvicorn app:app` (or FastAPI's `TestClient`): the first `POST` for seat A1 returns `201`, a second fan's returns `409 {'detail': 'seat A1 is already held'}`. In production, the user comes from the verified token rather than a query parameter, and errors use the problem-details format of Chapter 5.

6.6 Configuration, secrets and the twelve-factor app

The same container image should run in a laptop, staging and both production regions; only its configuration differs. The **twelve-factor app** (written at Heroku in 2011) states the rules that make a service easy to deploy, scale and replace: one codebase, many deploys; declared, isolated dependencies; config in the environment; backing services as attached resources; strictly separate build, release and run stages; stateless processes; port binding; concurrency by adding processes; disposability (fast start, graceful shutdown); dev/prod parity; logs as event streams to stdout; admin tasks as one-off processes.

Statelessness is the factor that matters most for scale: a pod that keeps no session in memory can be killed, moved or multiplied at any moment, which is what autoscalers and rolling deploys assume (Chapter 15). Read configuration once at startup, validate it, and refuse to start when it is wrong:

```
# settings.py: twelve-factor config, read once from the environment
import os
from dataclasses import dataclass

@dataclass(frozen=True)
class Settings:
    database_url: str
    payments_api_key: str  # injected from a secret store, never in git
    hold_seconds: int = 600
    def __repr__(self):  # never print a secret into the logs
        return f"Settings(db={self.database_url!r}, key=***)"
def load() -> Settings:
    missing = {"DATABASE_URL", "PAYMENTS_API_KEY"} - os.environ.keys()
    if missing:  # fail fast at startup, not on the first payment
        raise SystemExit(f"missing config: {sorted(missing)}")
    return Settings(os.environ["DATABASE_URL"],
                    os.environ["PAYMENTS_API_KEY"],
                    int(os.environ.get("HOLD_SECONDS", "600")))
```

Secrets (database passwords, the payment provider's API key, signing keys) are configuration with a blast radius. Keep them in a secret store, AWS Secrets Manager (Google Cloud Secret Manager; Azure Key Vault), inject them at runtime as environment variables or mounted files, rotate them, and grant each service only its own. Kubernetes Secrets are only base64-encoded unless encryption at rest is configured for etcd. Non-secret, slow-changing settings live in AWS Systems Manager Parameter Store or AWS AppConfig (Google Cloud Parameter Manager; Azure App Configuration); fast-changing ones, such as "waiting room on for event 42", are feature flags (*50 System Designs* #49, Feature Flag and Config Service). Chapter 22 covers secret management in depth.

6.7 Synchronous and asynchronous communication

A synchronous call is a phone call: the caller waits, and both sides must be available at the same moment. An asynchronous message is a letter in a mailbox: the sender moves on, and the receiver reads it when it can. Synchronous calls (HTTP, gRPC) give an immediate answer and couple the caller's latency and availability to the callee's. Asynchronous messages (a queue or a topic) decouple them in time, absorb bursts and survive a consumer outage, at the price of eventual consistency, at-least-once delivery and idempotent consumers (Chapter 14).

Fan-out makes synchronous coupling worse. If each of n parallel calls is slower than its p99 with probability $p = 0.01$, the request is slow whenever any one of them is:

$$P(\text{slow request}) = 1 - (1 - p)^n$$

With $n = 5$ that is 4.9%; with $n = 10$, 9.6%; with $n = 50$, 39.5%. At 50 calls, two page loads in five see somebody's p99. Hedged requests, timeouts with fallbacks, and fewer synchronous dependencies are the cures. The rule of thumb: call synchronously when the user is waiting for the answer and the decision cannot be made without it (can this fan hold seat A1?); send a message for everything that is a consequence (e-mail the ticket, render the PDF, update analytics). Long tasks the user starts but does not wait for use 202 Accepted plus a status resource (Chapter 5); this is also how Encore queues long support-agent jobs.

> **Design question.** When an order is paid, ticketing, notifications and analytics must react. Should the order service call them?
>
> No: three synchronous calls make checkout as available as the product of four services, and a slow e-mail provider would slow payments. The order service commits the order and an `OrderPaid` row in an outbox table in one transaction; a relay publishes it to Amazon SNS with an Amazon SQS queue per consumer (Google Pub/Sub; Azure Service Bus), or to Kafka. Each consumer processes at its own pace and deduplicates by order ID, since delivery is

> at least once. Volume is small (50 million orders a year is about 1.6/s on average, far more during a flash sale), so the cost is latency of seconds, not throughput. Trade-off: the e-mail arrives a few seconds later and the flow is harder to trace. See also *50 System Designs* #20 (Notification System).

6.8 Data ownership per service

Each service owns its data and is the only one that reads or writes its tables; others go through its API or its events. A shared database is a hidden, unversioned API: a column rename in orders breaks the analytics job nobody told. Ownership leaves three questions, each with a standard answer:

- **Reading another service's data**: call its API, or keep a local read-only copy fed by its events (search keeps its own index of catalog events).
- **Joining across services**: compose in an API layer (the BFF below) or build a read model that subscribes to both.
- **Writing across services**: no distributed transaction; a saga of local transactions with compensations (Chapter 18).

"Database per service" means a logical boundary, not necessarily a server per service: early on, separate schemas with separate credentials in one Postgres cluster are enough, and the hot service moves to its own cluster when its load demands it. Agents follow the same rule: the support agent's tools call the orders API, inheriting its authorization and audit, never a direct database connection.

6.9 Event-driven architecture

In an event-driven architecture, services announce facts in the past tense (`EventPublished`, `SeatSold`, `OrderPaid`) and other services react; the producer does not know who listens. Adding a consumer, such as fraud scoring next year, needs no change to orders. Events come in two weights: an **event notification** carries IDs and makes consumers call back for details; **event-carried state transfer** carries the data, so consumers keep local copies and never call back, at the price of larger messages and versioned schemas.

When a business process spans services, there are two ways to run it. In **choreography**, each service reacts to the previous one's event; it is loosely coupled but the flow exists only in people's heads. In **orchestration**, one workflow (AWS Step Functions; Google Cloud Workflows; Azure Logic Apps or Durable Functions; or Temporal) calls each step and handles compensations; it is easy to see and change, at the price of a central component. Use choreography for fan-out of facts and orchestration for multi-step processes with failure handling, such as checkout. Where it runs: Amazon EventBridge (Google Eventarc; Azure Event Grid) for routing events between services, SNS with SQS for fan-out to queues, Kafka or Amazon MSK (Azure Event Hubs also speaks the Kafka protocol) for high-volume ordered logs. Register event schemas (Confluent Schema Registry, EventBridge schema registry) and evolve them only in backward-compatible ways (Chapter 5).

6.10 CQRS and event sourcing

CQRS (command query responsibility segregation) splits the model that changes data from the models that answer questions. Commands go to a write model shaped for invariants; queries go to read models shaped for each screen, updated from the write side's events. Encore's seat map is the textbook case: hold attempts peak at 30,000/s and must be exact, while seat-map reads peak at 200,000/s and may be a second stale. The write side is the inventory database; the read side is one bitmap per event, 1 bit per seat (free or not), so even an 80,000-seat event fits in $80,000/8 = 10,000$ bytes, 10 KB, served from Redis or the CDN with a 1-second TTL (Chapter 9, Chapter 10). Reads never touch the write database, and each side scales on its own. The price is lag: a fan may see a seat as free that was held 500 ms ago, so the hold command, not the map, is the source of truth.

Event sourcing goes further: the write model stores the events themselves, and current state is a fold over them. It gives a complete audit trail, time travel and the ability to build new read models by replaying history, which is why ledgers and wallets use it (*50 System Designs* #44, Digital Wallet). Replays grow with history, so services keep snapshots:

```python
# event sourcing: the log is the truth; state is a fold over the log
def apply(seats: dict, ev: dict) -> dict:
    if ev["type"] in ("SeatHeld", "SeatSold"):
        seats[ev["seat"]] = (ev["type"], ev["user"])
    elif ev["type"] == "HoldExpired":
        seats.pop(ev["seat"], None)
    return seats

def rebuild(log, snapshot=({}, 0)):  # snapshot = (state, version)
    seats, version = dict(snapshot[0]), snapshot[1]
    for ev in log[version:]:  # replay only what the snapshot lacks
        apply(seats, ev)
    return seats, len(log)

log = [{"type": "SeatHeld", "seat": "A1", "user": "ana"},
       {"type": "SeatHeld", "seat": "A2", "user": "ben"},
       {"type": "HoldExpired", "seat": "A2"},
       {"type": "SeatSold", "seat": "A1", "user": "ana"}]
print(rebuild(log)[0])      # {'A1': ('SeatSold', 'ana')}
print(rebuild(log[:2])[0])  # time travel: the state after two events
snap = rebuild(log[:3])     # snapshot at version 3
print(rebuild(log, snap))   # replays one event, not four
```

A stadium show with 50,000 seats and about four events per seat (held, expired, held, sold) has 200,000 events; at an illustrative 100,000 events/s, a full rebuild takes 2 s, and a snapshot every few thousand events bounds it to milliseconds. Where it runs: Kafka or Amazon Kinesis (Pub/Sub; Event Hubs) as the log, KurrentDB (formerly EventStoreDB), Axon, or an append-only Postgres table; DynamoDB Streams feeding read models. Trade-offs: schemas of old events live forever, "delete this user" (GDPR) needs crypto-shredding or rewriting, and every read needs a projection. Use CQRS without event sourcing in most services; reserve event sourcing for domains where history is the product.

6.11 Serverless architecture

Serverless is renting a taxi instead of owning a car: you pay per ride, someone else maintains the fleet, and at rush hour you may wait. A serverless architecture composes functions (AWS Lambda; Cloud Run functions; Azure Functions) with managed services that also scale to zero and bill per use: API Gateway (Google API Gateway or Apigee; Azure API Management), SQS, DynamoDB, S3, EventBridge, Step Functions. There are no servers to patch, scaling is automatic and idle cost is near zero; the costs are cold starts, execution limits (a Lambda invocation runs at most 15 minutes, with up to 10,240 MB of memory and 6 MB synchronous payloads) and per-request pricing that loses to containers at high, steady load (Chapter 15).

A standard Lambda environment serves one request at a time, so concurrency follows Little's law, and Lambda also caps synchronous invocations at 10 requests per second per environment:

$$C = max(\lambda \quad D, \frac{\lambda}{10})$$

> **Design question.** Should Encore's hold endpoint run on Lambda?
>
> At the flash-sale peak, λ = 30,000 holds/s and an illustrative D = 40 ms give λD = 1,200, but the invocation cap needs $\lambda/10$ = 3,000 environments. The default quota is 1,000 concurrent executions per region (raisable to tens of thousands), and each function adds at most 1,000 environments every 10 seconds, while the spike arrives within seconds at 10:00. So: put holds on pre-scaled containers (ECS or EKS sized before the sale) behind the waiting room, and use Lambda where it shines: ticket-PDF rendering and e-mail jobs from SQS, webhooks from the payment provider, scheduled cleanup. Even the whole site's normal peak of 6,000 requests/s needs only 600 environments. Trade-off: the containers cost money while idle. See also *50 System Designs* #25 (Ticket Booking).

6.12 Backend for frontend

The web app, the mobile app and the organizers' dashboard need different shapes of the same data: the phone wants one small payload per screen over a slow network, the web page wants more, the dashboard wants live aggregates. A **backend for frontend** (BFF) is a thin service per client type, owned by that client's team, that calls the domain services, aggregates and trims the responses, and absorbs client-specific concerns (versions of old app builds still in the field). It holds no domain rules and no database of its own. Its cost is duplicated glue code and, because it fans out, the tail-latency math above; a GraphQL gateway (Apollo, AWS AppSync) is the alternative when many clients need flexible queries over one schema. Gateway, BFF and services stack in that order: the gateway handles cross-cutting edge concerns, the BFF handles one client's composition (Chapter 8).

6.13 Service mesh and sidecars

Every service needs timeouts, retries, mutual TLS, load balancing and telemetry on its calls. Writing them into every codebase in every language drifts; a **service mesh** moves them into the network. In the sidecar model, a proxy (usually Envoy) runs next to each pod and intercepts its traffic; a control plane (Istio; or Linkerd, which uses its own Rust proxy) pushes policy to all proxies. A retry policy becomes configuration:

```yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata: {name: seat-inventory}
spec:
  hosts: [seat-inventory.encore.svc.cluster.local]
  http:
  - timeout: 800ms            # whole call, all retries included
    retries: {attempts: 2, perTryTimeout: 300ms,
              retryOn: "connect-failure,refused-stream,unavailable"}
    route:
    - destination: {host: seat-inventory.encore.svc.cluster.local}
```

The price is CPU, memory and a hop at each end of every call. Istio's own benchmark (version 1.24) measures about 0.20 vCPU and 60 MB per sidecar per 1,000 requests per second; for 30,000 holds/s through two sidecars per hop that is about 12 vCPU for one hop. Istio's **ambient** mode replaces sidecars with a per-node ztunnel for mTLS and L4 (about 0.06 vCPU per 1,000 requests/s in the same benchmark) plus optional waypoint proxies for L7 policy. Managed options: Google Cloud Service Mesh, the Istio-based add-on for AKS, and on AWS, Amazon ECS Service Connect or Amazon VPC Lattice; AWS App Mesh reached end of support on 30 September 2026. Choose a mesh when you run dozens of services in several languages and need mTLS everywhere; with a handful of services, a shared client library or the platform's built-in service discovery costs less.

6.14 In Encore: the service map

```
web, mobile, organizer dashboard
          |
  CDN (event pages, images, seat-map bitmaps)
          |
  API gateway: auth, rate limits -------- waiting room (flash sales)
          |
  BFFs: web, mobile, dashboard
    |          |              |                  |
 catalog    search        seat inventory      orders ---- payment adapter
 Postgres   OpenSearch    Postgres + Redis    Postgres     -> provider
    |  ^        ^              |                  |
    |  +- catalog events       +-- SeatSold -->---+
    |                                             | OrderPaid (outbox)
    +------------------> topic <------------------+
                          |
    ticketing (PDF to S3) | notifications (queue) | analytics (lakehouse)
                          |
  support agent -> MCP tool server -> orders API (refund requests)
```

Each line in the map is a decision from this chapter: one owner per table, synchronous calls only where a fan waits for an answer, events for every consequence, a CQRS read model for the seat map, an adapter in front of each external system, and Lambda only for the bursty workers. Chapter 25 builds this map in code and cloud.

6.15 Numbers to remember

Figure	Value
Serial availability, 5 x 99.95%	99.75%, about 22 h down a year
Fan-out slow probability, p = 1%	4.9% at 5 calls, 9.6% at 10
Team coordination paths	$n(n-1)/2$: 28 for 8 teams
Same-zone RPC vs function call	about 0.2 to 1 ms vs nanoseconds
Lambda limits	15 min, 10,240 MB, 1,000 concurrent by default
Lambda scaling	+1,000 environments per 10 s per function
Istio sidecar (1.24 benchmark)	0.20 vCPU, 60 MB per 1,000 requests/s
Seat-map bitmap, 80,000 seats	10 KB

6.16 Pitfalls

- Microservices on day one -> a modular monolith with enforced boundaries; extract on evidence.
- Entity services and long synchronous chains -> services per bounded context; events for consequences.
- A shared database between services -> one owner per table; APIs, events or read models for everyone else.
- Dual writes (save to the database, then publish) -> the transactional outbox (Chapter 14).
- Retries at every layer of a mesh -> retry at one layer with a budget and a total timeout (Chapter 11).
- Secrets in images, repositories or logs -> a secret store, runtime injection, redacted `repr`.

6.17 Remember

- Draw boundaries around business capabilities and transactions, not tables; align them with teams.
- Inside a service, the domain owns ports and the adapters plug in: HTTP, SQL, Redis, the payment provider, agent tools.
- Call synchronously when the user waits for the answer; publish events for everything that follows.
- Every service owns its data; reads go through APIs, events or read models; cross-service writes are sagas.
- Every hop costs availability and tail latency: multiply availabilities, compute $1 - (1 - p)^n$, and remove hops before adding retries.

Part III. Building blocks

How a system grows from one server to many, and the components every design reuses: load balancers, caches, CDNs, rate limiters, storage engines and the many kinds of database, queues and workers, and the cloud compute they run on.

In this part:

- Scalability: from one server to many
- Load balancers and gateways
- Caching and Redis
- CDNs and the edge
- Rate limiting, backpressure and resilience
- Storage and databases
- Database types: relational, NoSQL and beyond
- Message queues, streams and workers
- Compute in the cloud: containers, Kubernetes and functions

7 Scalability: from one server to many

A system is scalable when you can serve more load by adding resources, at a cost that grows no faster than the load, without redesigning it each time. This chapter answers how Encore grows from one server that handles its first customers to a two-region system that survives a million fans arriving in ten minutes, and it answers the question every design review asks early: do the servers have to be stateless? The short answer is: the request-serving tier almost always should be, because that is what lets you add, kill and replace copies freely; but state has to live somewhere, and for game servers, chat gateways, stream processors, matching engines and one-big-box databases, keeping state in the server is the right design. The whole chapter fits in one picture: clone what holds no state, split what does, and find the one serial step that stops the curve.

7.1 The picture

A restaurant gets busier. First the owner buys a bigger stove and hires a faster cook: that is **vertical scaling**, and it stops when there is no bigger stove. Then more cooks work side by side: **horizontal scaling**. It works only if any cook can pick up any order, which is true when orders are written on tickets hanging on the rail, not kept in one cook's head. A kitchen whose cooks hold nothing between orders is **stateless**; the rail, the pantry and the cash register are the **state**, and they become the next bottleneck.

```
vertical:   [ small box ] -> [ BIGGER BOX ] -> [ BIGGEST BOX ]  (ceiling)

horizontal:            clients
                          |
                    load balancer
              +-----------+-----------+
              |           |           |
          [ app 1 ]   [ app 2 ]   [ app N ]    stateless: clone freely
              |           |           |
     +-------+-----+------+------+-----+-------+
     |             |             |             |
  Redis         Postgres      object store   queue     state: split
(sessions,     (orders,       (ticket PDFs,  (work     carefully
 holds)         seats)         images)        to do)
```

The top half of the diagram scales by copying; the bottom half scales by replication, caching and partitioning, which the rest of this part and Chapter 16 and Chapter 17 teach.

7.2 Vertical and horizontal scaling

Vertical scaling (scale up) moves the workload to a bigger machine: more cores, more memory, faster disks. Nothing in the code changes, transactions stay local, and there is no network between components. **Horizontal scaling** (scale out) runs more copies of a component behind a load balancer (Chapter 8) or splits its data across machines. It has no hard ceiling and survives the loss of a machine, but the work must be divisible and the state must live outside the copies.

In the cloud, vertical scaling is cheaper than its reputation. Within one instance family the price is linear in size: on AWS an m8i.large (2 vCPU, 8 GiB) lists at about $0.106 an hour and an m8i.96xlarge (384 vCPU, 1,536 GiB) at about $20.32, 192 times the resources for 192 times the price (on-demand Linux, US East, October 2026). The real limits are elsewhere:

- **The ceiling.** The largest general-purpose box has 384 vCPU; the largest memory-optimized ones (U7i) reach 1,920 vCPU and 32 TiB, at hundreds of dollars an hour (u7in-32tb.224xlarge: about $361, over $260,000 a month). Past that, there is no bigger box.
- **One failure domain.** One machine lives in one availability zone; when it fails, everything fails.

- **Resizing is downtime.** Changing an EC2 instance type means stop and start; a managed database (RDS; Cloud SQL in GCP; Azure Database for PostgreSQL) resizes with a restart, or with a failover to a resized standby in a Multi-AZ setup: a short outage either way.
- **Diminishing returns inside the box.** More cores fight over the same locks, memory bandwidth and NUMA nodes; the Universal Scalability Law below applies inside one machine as well as across many.

The number of copies a horizontal tier needs is the capacity equation from Chapter 2, plus spares for failure:

$$N = \frac{\lambda}{c\,u} + N_{\text{spare}}$$

where λ is the peak request rate, c the rate one copy sustains at full utilization and u the target utilization. Encore's API pods (1 vCPU, 2 ms of CPU per request) sustain c = 500 requests/s; at the 6,000 requests/s daily peak and u = 0.6 that is 20 pods. Spread across three zones, surviving the loss of one zone means the other two must carry the peak, so run 30 (10 per zone).

Vertical scale goes further than most people expect. In 2016 Stack Overflow served 209 million HTTP requests a day (about 2,400 per second on average, more than Encore's 1,500) from 9 primary web servers and 4 SQL Servers in two clusters with 384 and 768 GB of RAM, with question pages rendering in about 23 ms on average (Nick Craver, "Stack Overflow: The Architecture, 2016 Edition"). Note the shape: the stateless web tier was already horizontal, and the database was scaled up and kept in memory. That shape, **scale out the stateless tier, scale up the database until it hurts**, is the right default for most systems.

> **Design question.** Encore is launching next month. Should it start with twenty microservices on Kubernetes, or one big server?
>
> Neither extreme. CPU demand at the daily peak is 6000 × 2 ms = 12 busy cores, which one 48-vCPU machine could carry, so capacity does not force scale-out. Availability does: one machine is one zone and every deploy is an outage. Run the modular monolith (Chapter 6) as stateless containers, at least two per zone in three zones, behind an ALB (Cloud Load Balancing in GCP; Application Gateway in Azure). Keep the database vertical: one managed Postgres primary with a synchronous standby in a second zone, sized so the working set fits in RAM; add Redis for sessions and holds. You get horizontal scale and zone failover for the tier that is cheap to clone, and one simple transactional database for the part that is hard to split. Revisit when a measured limit (write rate, data size, team size) binds. See also *50 System Designs* #25 (Ticket Booking).

7.3 What a stateless server is

A cloakroom attendant who hands you a numbered ticket is stateless: any attendant can fetch your coat with the ticket, because the coat and its number are on the rack, not in anyone's memory. A **stateless server** keeps no client-specific state between requests that another copy would need. Each request carries, or points to, everything required to serve it: a signed token or a session ID, the resource IDs, an idempotency key. Everything durable lives in **backing services** (databases, caches, object stores, queues), the twelve-factor "processes" rule from Chapter 6.

Stateless does not mean the process holds nothing. It holds code, configuration, connection pools, compiled templates and caches. The rule is that whatever it holds is **disposable**: derived from a backing service, rebuildable, and harmless to lose. A local cache of the venue map is fine; the only copy of a fan's seat hold is not. The practical test is one question: *can I kill any instance at any moment, or add ten, without a user noticing more than one retried request?*

What passing that test buys:

- **Any balancing algorithm works**, with no sticky sessions (Chapter 8).
- **Autoscaling, rolling deploys and spot instances** become routine, because removing a copy loses nothing.
- **Failure handling is a retry**, provided writes are idempotent (Chapter 5).

The cost of failing it is easy to compute. When a copy dies, the work in flight is lost (Little's law: requests in flight are rate times time), and so is whatever state it held:

$$\text{in-flight lost} = \lambda_{\text{copy}} \times W \qquad \text{state lost} = \frac{S_{\text{total}}}{N}$$

An Encore API pod at 300 requests/s with a 50 ms mean response time has 15 requests in flight; a stateless pod turns those into 15 client retries. If the same pods kept the 4 million daily users' sessions in memory, each of 20 pods would hold 200,000 of them, and one crash would log out 200,000 fans, perhaps in the middle of a checkout.

7.4 Do you always need stateless servers?

No. Stateless application servers are the *default* for request/response tiers because they are the cheapest way to get horizontal scale and painless failure. They are not a law, and the state does not disappear: making the API stateless moves its state into Redis and Postgres, which are stateful servers you must now scale. Choose a stateful server when one of these is true:

- **One machine is enough and simplicity wins.** A single Postgres or SQLite-backed application, an internal tool, a batch job, or the Stack Overflow database tier: keeping state in process memory is the fastest design there is. You pay with one failure domain, so add a standby, not a cluster.
- **The state is the product and must sit next to the compute.** A multiplayer **game server** runs an authoritative simulation tens of times a second; shipping the world state to Redis on every tick would cost more than the game. Fleets such as Amazon GameLift or Agones on Kubernetes place each match on one server and route its players there.
- **The connection is the state.** A **chat or WebSocket gateway** holds an open TCP connection per user; that connection cannot move, so the gateway is stateful by nature, with a registry (user to gateway) in a shared store so other services can reach the user (*50 System Designs* #6 (Chat System)).
- **Per-key state is read on every event. Stream processors** keep aggregates, windows and joins in local state tied to the partitions they own: Flink keyed state with checkpoints to S3, Kafka Streams state stores backed by changelog topics. Fetching that state over the network per event would cut throughput by orders of magnitude (Chapter 21).
- **A single writer per key is the correctness model.** A stock exchange **matching engine** keeps each order book in memory on one thread so that price-time priority is exact (*50 System Designs* #43 (Stock Exchange Matching Engine)); a collaborative editor keeps each document's live session on one server (#11). Actor frameworks (Microsoft Orleans, Akka Cluster Sharding) generalise this: one live object per key, placed and moved by the runtime.

Stateful done well follows four rules: **partition** the state by a key; **route** each request to the owner of its key (consistent hashing, partition assignment, or a directory, see Chapter 17); make the state **recoverable** (a replica, a checkpoint or a changelog, so a crash costs seconds, not data); and make it **movable**, so rebalancing and deploys hand keys over cleanly. In Kubernetes these servers run as StatefulSets with stable names and their own volumes (Chapter 15).

Workload	Stateful server?	Where the truth lives	Routing
REST or gRPC API	no	DB, cache	any copy
WebSocket gateway	connections only	shared store	sticky per connection
Game match	yes	match server, snapshots	match ID to server
Stream processor	yes, per partition	local state + changelog	partition owner
Matching engine	yes, per symbol	memory + replicated log	symbol to engine
Database primary	yes	its disk, its replicas	shard map

> **Design question.** Encore's waiting room keeps 1,000,000 fans informed of their place in line during the flash sale. Stateless servers or stateful ones?
>
> Polling a stateless API every 5 seconds costs $10^6/5 = 200,000$ requests/s, as much as the seat-map peak, almost all answering "no change" (Chapter 5). Server-Sent Events or WebSockets cut that to one long-lived connection per fan, which makes the gateway stateful in connections: at an illustrative 50,000 connections per pod, 20 pods. Keep the *truth* elsewhere: each fan's arrival time is a member of a Redis sorted set, the admission cursor is one Redis key advanced by about 1,000 fans/s, and the gateway only pushes "your number is 41,207" (the back of the line waits $10^6/1000 = 1000$ s, about 17 minutes). When a gateway pod dies, its 50,000 fans reconnect with jittered

> backoff to another pod and read their place from Redis; nobody loses their place. Deploys must drain connections slowly, and the gateway count follows connections, not requests. Stateful at the edge, stateless in the truth. See also *50 System Designs* #25 (Ticket Booking).

7.5 Where the state goes

Making a server stateless is an act of moving state, so decide for each kind where it lands:

State	Where it goes	Why there
Login session	Redis with a TTL, or a signed token	fast, shared, expires
Seat holds	Redis with a TTL + durable row	atomic, timed, survives restart
Orders, users, events	the database	transactions, durability
Uploads, ticket PDFs	object storage (S3)	cheap, huge, by URL
Work to do later	a queue or workflow engine	survives the worker
Live connection	the gateway + a registry	cannot move
Derived data	a local or shared cache	disposable by definition

Sessions have two standard homes. A **server-side session** is a random ID in a cookie pointing at a Redis hash: every request costs one round trip inside the zone (0.1 to 0.5 ms), the session is revocable at once by deleting the key, and Redis becomes a dependency you must replicate. A **self-contained token** (a signed JWT) carries the user ID, scopes and expiry, so any copy verifies it locally with no lookup; the price is revocation, which is why access tokens typically live 5 to 15 minutes beside a revocable refresh token (Chapter 22). Memory is rarely the issue: 4 million daily users at 1 KB per session is 4 GB.

Files go to object storage, and the server should not even proxy the bytes: it hands the client a presigned URL to upload or download directly from S3 (Cloud Storage in GCP; Blob Storage in Azure), so a 50 KB ticket PDF never occupies an API pod.

The demo below runs two “instances” of a FastAPI app that hold no session in process memory. A fan logs in on pod A, adds a seat to the selection, pod A disappears, and pod B carries on with the same session, because both read and write one Redis (fakeredis here, ElastiCache or MemoryDB in production; Memorystore in GCP; Azure Managed Redis). The selection is a convenience; the authoritative seat hold lives in the inventory service.

```
# Two "instances" of a stateless API share session state in Redis.
import secrets

import fakeredis
from fastapi import Cookie, FastAPI, HTTPException
from fastapi.testclient import TestClient

SESSION_TTL = 1800                      # 30 minutes idle


def create_app(name: str, r) -> FastAPI:
    app = FastAPI()                     # no session dict in the process

    @app.post("/login")
    def login(user_id: str):
        sid = secrets.token_urlsafe(16)
        r.hset(f"sess:{sid}", "user", user_id)
        r.expire(f"sess:{sid}", SESSION_TTL)
        return {"sid": sid, "served_by": name}

    @app.post("/cart")
    def add_seat(seat: str, sid: str = Cookie()):
        key = f"sess:{sid}"
        if not r.exists(key):
            raise HTTPException(401, "session expired")
        r.hset(key, f"seat:{seat}", 1)   # one atomic command, no race
        r.expire(key, SESSION_TTL)       # sliding expiry
        seats = sorted(f[5:].decode() for f in r.hkeys(key)
                       if f.startswith(b"seat:"))
        return {"cart": seats, "served_by": name}

    return app


shared = fakeredis.FakeServer()          # stands in for ElastiCache
a = TestClient(create_app("pod-a", fakeredis.FakeRedis(server=shared)))
b = TestClient(create_app("pod-b", fakeredis.FakeRedis(server=shared)))

sid = a.post("/login", params={"user_id": "u42"}).json()["sid"]
a.cookies.set("sid", sid)
b.cookies.set("sid", sid)
print(a.post("/cart", params={"seat": "A-12"}).json())
del a                                    # pod-a is scaled in or crashes
print(b.post("/cart", params={"seat": "A-13"}).json())
```

It printed `{'cart': ['A-12'], 'served_by': 'pod-a'}` and then `{'cart': ['A-12', 'A-13'], 'served_by': 'pod-b'}`. Replace the Redis calls with a module-level dictionary and pod B answers 401: that is the bug sticky sessions paper over. Each update is one atomic command (`HSET` of a field), not a read-modify-write of the whole session, so two tabs adding seats at once cannot overwrite each other (Chapter 3).

7.6 The scale cube

A restaurant chain grows in three ways: it opens identical kitchens side by side (more of the same), it splits the menu so a pastry kitchen bakes and a grill kitchen grills (by function), and it gives each neighbourhood its own branch (by customer). Abbott and Fisher's **scale cube** (*The Art of Scalability*, 2009) names the same three axes for software:

```
          Z: split by data (shards: event, tenant, region)
          ^
          |      +-----------------+
          |     /                 /|
          |    +-----------------+ |
          |    |                 | |
          |    |  where Encore   | +
          |    |  ends up        |/
          |    +-----------------+
          +--------------------------> X: clone (N identical copies)
         /
        v
      Y: split by function (catalog, search, inventory, orders)
```

- **X axis, clone.** Run *N* identical copies behind a load balancer. Each copy sees all the data, so X scales request handling and availability, not data size or write throughput. It is cheap if the copies are stateless.
- **Y axis, split by function.** Cut the system into services by capability or domain (Chapter 6). Each service scales, deploys and fails on its own, and a team owns it; the cost is network calls and distributed data.
- **Z axis, split by data.** Each copy runs the same code but owns a subset of the data, chosen by a key: event, user, tenant, region. Z is the only axis that divides data size and write load (Chapter 17); the cost is routing and cross-shard queries.

The axes multiply. A server's share of load and data is roughly

$$\lambda_{\text{server}} \approx \frac{f_y \lambda}{N_x N_z} \qquad S_{\text{server}} \approx \frac{S_y}{N_z}$$

where f_y is the fraction of requests that reach this function, S_y the data that function owns, N_x the copies per shard and N_z the shards. Behind the waiting room, seat inventory runs at 3,000 holds/s and is load-tested at 6,000; with its 12 pods arranged as 4 shards of 3 copies, each pod sees 250 holds/s in normal sale traffic and 500 at the tested rate, well under the 1,000 one pod sustains. Notice what X alone cannot do: the 3 copies of one shard all share that shard's single Redis primary, so only Z relieves it.

```python
# The scale cube as one routing decision per axis.
import hashlib
import random

SERVICES = {"/events": "catalog", "/search": "search",          # Y axis
            "/holds": "seat-inventory", "/orders": "orders"}
REPLICAS = {"catalog": 6, "search": 4, "seat-inventory": 3,   # X, per shard
            "orders": 6}
SHARDS = {"seat-inventory": 4}                                  # Z axis


def route(path: str, key: str) -> str:
    prefix = "/" + path.split("/")[1]
    svc = SERVICES[prefix]
    target = svc
    if svc in SHARDS:                       # Z: the key picks the shard
        h = int.from_bytes(hashlib.md5(key.encode()).digest()[:8], "big")
        target += f"-shard{h % SHARDS[svc]}"
    replica = random.randrange(REPLICAS[svc])   # X: any copy will do
    return f"{target}/pod{replica}"


random.seed(1)
print(route("/events/42", "event:42"))
print(route("/holds/42", "event:42#section:A"))
print(route("/holds/42", "event:42#section:B"))
```

It printed `catalog/pod1`, `seat-inventory-shard0/pod2` and `seat-inventory-shard1/pod0`: the path chose the service (Y), the key chose the shard (Z), and any copy of that shard would do (X). In production the three decisions sit in different places: path rules in the API gateway or ALB, the shard map in a smart client or proxy, and replica choice in

the load balancer or service mesh. Kubernetes Deployments and EC2 Auto Scaling groups (managed instance groups in GCP; virtual machine scale sets in Azure) do X; DynamoDB, Vitess, Citus and Redis Cluster do Z for you.

7.7 The scaling path for Encore

Systems grow along a well-worn path. Each step removes one bottleneck and adds one moving part, so take a step when its trigger is *measured*, not when it is fashionable:

```
1 one box -> 2 app | DB -> 3 LB + N app -> 4 cache -> 5 read replicas
  -> 6 CDN -> 7 queues + workers -> 8 waiting room -> 9 shards
  -> 10 two regions -> cells
```

Step	Removes	Encore's trigger (illustrative)
One server, app + DB	nothing yet	first organizers, tens of req/s
Separate database	app and DB fighting for RAM	DB working set needs the memory
LB + stateless app copies	one box's CPU; one failure	6,000 req/s peak = 20 pods
Cache	repeated reads	200,000 seat-map reads/s
Read replicas	cache misses on one primary	10,000 reads/s after a 95% hit ratio
CDN	origin bandwidth, distance	200 GB of pages in 10 minutes
Queues and workers	slow work in the request	PDF 200 ms + e-mail 100 ms per ticket
Waiting room	an open door	30,000 attempts/s raw
Sharding	hot rows of one event	3,000 to 6,000 holds/s
Second region	a region outage; far users	SLO above one region's

A few of the numbers deserve the arithmetic. The cache turns 200,000 seat-map reads/s into 10,000 at a 95 percent hit ratio, and at an illustrative 10,000 reads/s per replica and 60 percent utilization that is 3 Postgres replicas instead of 35 (Chapter 9, Chapter 16). A million fans each loading a 200 KB event page in ten minutes is 200 GB, 2.7 Gbit/s on average and several times that in the first minute, which belongs on CloudFront (Cloud CDN in GCP; Front Door in Azure), not on the origin (Chapter 10). Selling 50,000 seats in ten minutes is 83 tickets/s; at 200 ms per PDF that keeps 17 workers busy, work that must not sit inside the checkout request (Chapter 14). At the door, 30,000 hold attempts per second would be beyond one durable Postgres primary (thousands to tens of thousands of commits per second); the waiting room admits about 1,000 fans/s, about 3 hold attempts each, so seat inventory runs at 3,000 holds/s and is load-tested at 6,000 (Chapter 11). Even that rate lands on the rows of one event, so seat inventory is the one component that shards, by `event#section`, while orders at 83 writes/s stay on one primary for years.

The path has a step nobody draws: **capacity you do not own**. Adding pods does not raise the payment provider's rate limit, the e-mail provider's sending quota or a model API's tokens-per-minute limit. Encore's support agent runs as a stateless service that checkpoints each conversation in the database, so it scales out like the API, but every replica draws from the same LLM quota; past a point, more replicas only produce more 429s. Treat each external limit as a fixed-size shard: queue in front of it, rate-limit to it, cache what you can (Chapter 11; *Designing Agentic Systems* covers model gateways and agent runtimes in depth).

7.8 Elasticity and autoscaling

Scalability is the ability to handle more load by adding resources; **elasticity** is adding and removing them automatically as load changes, so you pay for the trough, not the peak. Elasticity needs stateless (or cheaply movable) copies, a metric that tracks demand, and capacity that arrives in time. Kubernetes' Horizontal Pod Autoscaler applies

$$N_{\text{desired}} = N_{\text{current}} \times \frac{m_{\text{current}}}{m_{\text{target}}}$$

so 20 pods at 90 percent CPU against a 60 percent target become 30. Encore's daily cycle between 1,500 and 6,000 requests/s lets the API tier float between about 8 and 30 pods, enough at either end to lose one of three zones. The flash sale does not wait for that loop: autoscaling reacts in minutes and new capacity needs nodes, images and warm caches, so Encore schedules the scale-out before 09:55 (seat inventory to 12 pods: 8 for the tested 6,000 holds/s at 1,000 per pod and 75 percent, plus the zone spare) and lets the waiting room hold demand at what the pre-scaled fleet can serve. Chapter 15 covers the HPA, target tracking, scale-to-zero and a simulation of the lag.

7.9 Why throughput stops rising

Ten people in a meeting room get more done than one, until they queue for the single whiteboard (**contention**) and spend their time bringing each other up to date (**coherency**). The first cost grows with the number of people; the second with the number of *pairs*, which grows with its square. Amdahl's law (Chapter 2) models only the first, so it predicts that throughput flattens. Neil Gunther's **Universal Scalability Law** (USL) adds the second, and predicts what load tests show: throughput peaks and then falls.

$$X(N) = \frac{\lambda N}{1 + \sigma(N-1) + \kappa N(N-1)} \qquad N^* = \frac{\sqrt{1-\sigma}}{\kappa}$$

$X(N)$ is the throughput of N copies, λ the throughput of one, σ the contention (the fraction of work that is serialised: one lock, one leader, one hot row), and κ the coherency cost (pairwise coordination: invalidating every copy's cache, lock handoffs, gossip). With $\kappa = 0$ the USL is Amdahl's law with $\sigma = 1 - p$, and throughput approaches the ceiling λ/σ. With $\kappa > 0$ it peaks at N^*; beyond it, every added copy *reduces* throughput.

```python
# Universal Scalability Law: where adding copies stops paying.
import math


def usl(n: int, lam: float, sigma: float, kappa: float) -> float:
    return lam * n / (1 + sigma * (n - 1) + kappa * n * (n - 1))


def peak(sigma: float, kappa: float) -> int:
    return round(math.sqrt((1 - sigma) / kappa))


for name, sigma, kappa in [("one hot lock", 0.15, 0.005),
                           ("by section", 0.015, 0.0003)]:
    curve = {n: round(usl(n, 1000, sigma, kappa)) for n in (4, 8, 13, 30)}
    print(f"{name:12} peak at N={peak(sigma, kappa):3}  {curve}")
```

It printed a peak at 13 copies and 3,631 holds/s for the first case, with 30 copies doing worse (3,093) than 13; the second case reaches 7,131 holds/s with 8 copies and keeps rising to 57. You do not guess σ and κ: run the load test at several sizes, then fit the two numbers by regression to the measured throughputs (Gunther's method). The fitted values tell you *which* fix to look for, a serial step (σ) or chatter between copies (κ).

> **Design question.** The flash-sale load test aims at 6,000 holds/s, but seat-inventory throughput stops at about 3,600 however many pods you add, and 30 pods are slower than 13. Why, and what do you change?
>
> The curve that rises, peaks and falls is the USL with both terms present; fitting the test gives roughly $\sigma = 0.15$ and $\kappa = 0.005$ for pods that each sustain 1,000 holds/s alone. Contention: every hold also decrements one "seats left" counter on the event row, so all pods serialise on it. Coherency: each pod broadcasts seat-map invalidations to every other pod, $N(N-1)$ messages. Fix both: make each hold touch only its seat's `event#section` key (100 sections divide the hot spot), compute "seats left" asynchronously from the hold stream, and replace pod-to-pod invalidation with one seat bitmap published every second. The refit, $\sigma = 0.015$ and $\kappa = 0.0003$, gives 7,100 holds/s with 8 pods, above the 6,000 target, and the 12 pods that run on the day keep that with a zone lost. Trade-off: the "seats left" figure on the page lags by a second or two; the hold itself stays exact. See also *50 System Designs* #34 (E-Commerce Checkout and Inventory).

7.10 Scaling reads and scaling writes

A library scales borrowing by buying more copies of a popular book; it cannot scale *writing* that book by hiring more authors to type into the same manuscript. Reads scale by **copying**: caches, read replicas, CDNs and precomputed views, because any copy can answer. Writes scale only by **splitting** (shards, each owning its keys) or by **doing less per write** (batching, group commit, append-only logs, LSM storage). The asymmetry is in one pair of equations:

$$\text{read capacity} = n_{\text{copies}} \times c_r \qquad \text{write capacity} = P_{\text{shards}} \times c_w$$

Replicas do not appear in the second: every replica replays every write, so adding one adds read capacity and zero write capacity. A queue in front of the database does not add write capacity either; it absorbs a burst and drains it at c_w, which helps only if the *average* write rate fits (Chapter 14). Encore's profile is lopsided like most consumer systems: 200,000 seat-map reads/s against 30,000 hold attempts/s at the door, of which the waiting room admits 3,000/s, so the seat map is served from a published snapshot and caches, and only the holds touch the sharded primary. Separating the two paths entirely is CQRS (Chapter 6).

7.11 Shared-nothing

In a **shared-nothing** architecture each node owns its CPU, memory and disk, and nodes cooperate only by messages over the network. It is the architecture behind DynamoDB, Cassandra, Kafka, Vitess, Elasticsearch, Spanner and every stateless web tier. Its alternatives share something: **shared memory** is one big machine (vertical scaling), and **shared disk** puts several compute nodes on one storage layer, as Aurora does, where one writer and up to 15 read replicas attach to the same distributed volume (Oracle RAC is the classic). Shared-nothing scales almost linearly, σ and κ near zero in the USL, *as long as each request touches one node*. Every cross-node join, transaction or broadcast reintroduces the terms, which is why partition keys are chosen so the hot path stays on one shard (Chapter 17).

7.12 Cells and shuffle sharding

Ships are built with bulkheads, so one breach floods one compartment, not the hull. A **cell** is a complete, independent copy of the stack (gateway, services, cache, database) that serves a fixed subset of customers; a thin **cell router** maps each key, such as an organizer or event, to its cell, and cells share no state. A bad deploy, a poison request or a runaway tenant takes down one cell, so with C cells the **blast radius** is $1/C$ of customers. Cells also cap scale per unit: once a cell is load-tested at its maximum size, growth means adding cells, never discovering a new limit in production. AWS describes the pattern in the whitepaper "Reducing the Scope of Impact with Cell-Based Architecture" (2023); deploy to one cell first and the cell becomes a natural canary (Chapter 23).

Shuffle sharding shrinks the blast radius further without more hardware. Instead of cutting n workers into fixed groups, give each customer a random combination of k of them. Two customers share *all* their workers with probability

$$P(\text{identical shard}) = \frac{1}{\binom{n}{k}}$$

Colm MacCarthaigh's Amazon Builders' Library example has 8 workers in shards of 2: 28 combinations, so a poison customer harms 1⁄28 of the others fully instead of 1⁄4. Route 53 gives each customer domain 4 of 2,048 virtual name servers, 730 billion combinations. For Encore's organizer dashboards:

```python
# Shuffle sharding: each organizer gets 4 of 20 dashboard pods.
import hashlib
import math
import random

PODS, K = 20, 4


def shard(org_id: str) -> frozenset[int]:
    seed = hashlib.sha256(org_id.encode()).digest()
    return frozenset(random.Random(seed).sample(range(PODS), K))


bad = shard("org-poison")                 # its traffic crashes its 4 pods
orgs = [shard(f"org-{i}") for i in range(10_000)]
overlap = [len(s & bad) for s in orgs]
print("combinations:", math.comb(PODS, K))
for j in range(K + 1):
    print(f"share {j} pods: {overlap.count(j):5}")
```

It printed 4,845 combinations, and of 10,000 other organizers, 3,770 share no pod with the poison one, 4,646 share one, 1,452 two, 131 three and 1 all four (the exact expectation is 2). With five fixed groups of four pods, 2,000 organizers would have lost everything. Customers who share one to three pods keep working if the client retries on another pod of its shard. The trade-offs: a router that must be simpler and more available than anything behind it, capacity planned per cell, and cross-cell features (a global search, an all-organizer report) built as separate aggregation paths.

7.13 In Encore

- **Stateless and cloned (X):** the API gateway, the BFFs, catalog, orders and search front ends, and the support agent's runtime; 8 to 30 API pods across three zones.
- **Stateful by design:** the waiting-room gateways (connections only; places in line live in Redis), the clickstream stream processors (per-partition state with changelogs), and the databases themselves.
- **Split by function (Y):** catalog, search, seat inventory, orders and payments, notifications, analytics, each scaling on its own metric.
- **Admitted, then split by data (Z):** the waiting room turns 30,000 hold attempts/s at the door into 3,000/s (6,000 tested); seat inventory alone shards, by `event#section`, on 12 pods, because one event's writes concentrate on a few rows; orders stay vertical on one Postgres primary with replicas.
- **Copied for reads:** seat-map snapshots in caches and on the CDN, event pages and images at the edge.
- **Isolated:** organizer dashboards shuffle-sharded over pods; a dedicated cell for a stadium-sized sale once the business has several running at once.

7.14 Numbers to remember

Figure	Value
Cloud price within one family	linear: m8i $0.106 to $20.32 an hour
Largest EC2 instance	1,920 vCPU, 32 TiB (U7i)
Stack Overflow, 2016	~2,400 req/s average; 9 web + 4 SQL servers
Copies needed	$\lambda/(c\,u)$ + spares
Encore API pods at peak	20, or 30 to survive a zone
Lost when a stateless pod dies	λW: 15 requests to retry
Sessions, 4M users at 1 KB	4 GB in Redis
USL peak	$N^* = \sqrt{(1-\sigma)/\kappa}$
Ceiling with $\kappa = 0$	λ/σ

Figure	Value
Cell blast radius	$1/C$ of customers
Shuffle shard, identical	$1/\binom{n}{k}$: 1 in 4,845 for 4 of 20
Route 53 shuffle shards	4 of 2,048: 730 billion

7.15 Pitfalls

- Sessions, uploads or holds in pod memory or on local disk -> Redis, signed tokens, S3 and the database; prove it by killing pods during a load test.
- Sticky sessions to hide server state -> externalise the state; keep affinity only for live connections and owned partitions.
- Adding app servers when the database is the limit -> fit the USL from a load test and remove the serial step.
- Treating “stateless” as dogma for gateways, games, stream processors or matching engines -> partition the state, route by key, make it recoverable and movable.
- Read replicas to fix write load -> shard, batch or do less per write.
- Sharding before a bigger box, a cache and replicas -> take each step on a measured trigger.
- Reactive autoscaling for a scheduled spike -> pre-scale before 09:55 and admit through the waiting room.
- Forgetting capacity you do not own -> queues and rate limits in front of the payment, e-mail and model providers.
- One global deployment -> cells with a thin router, and shuffle sharding for shared pools.

7.16 Remember

- Scale out the stateless tier, scale up the database until a measured limit forces a split.
- A stateless server holds nothing another copy would need: kill any copy, add ten, and users see at most a retry.
- Stateless is the default, not a law: game servers, chat gateways, stream processors and matching engines keep state in the server, partitioned, routed by key, recoverable and movable.
- The scale cube: clone (X) for requests and availability, split by function (Y) for teams and independent scaling, split by data (Z) for size and writes.
- Reads scale by copying, writes by splitting; the USL’s σ and κ tell you why throughput stops rising and which fix applies.
- Cells and shuffle sharding cap the blast radius: $1/C$ for cells, $1/\binom{n}{k}$ for shuffle shards.

8 Load balancers and gateways

One machine cannot serve 200,000 seat-map reads a second, so Encore runs hundreds of copies of each service. That raises the question of this chapter: when a request arrives, *which copy* should serve it, how do we stop sending traffic to a copy that is dying, and where do we check who the caller is before any copy does work? Load balancers answer the first two; API gateways answer the third. Together they are the front door of every design.

8.1 The picture

Picture the check-in hall of a busy airport. At the entrance, a dispatcher waves each passenger to a desk: the next one in turn, the one with the shortest queue, or the one for their airline. A desk that closes gets no more passengers, and the people already standing at it are allowed to finish. Further in, passport control checks every traveller's documents before anyone reaches the gates. The dispatcher is the **load balancer**; passport control is the **API gateway**; the desks are the service replicas, called **backends**, **targets** or **upstreams** depending on the product.

```
fans (web, mobile)
     |
     v
DNS: latency-based routing -----> region A  or  region B
     |
     v
CDN edge (CloudFront)           images, static files, cached pages
     |
     v
load balancer (ALB, layer 7)    TLS, health checks, path routing
     |
     v
API gateway (Envoy pods)        auth, rate limits, routing
     |                |               |
     v                v               v
  catalog       seat-inventory     orders     (many replicas each)
```

Every arrow that fans out is a balancing decision. DNS picks a region, the CDN picks an edge, the load balancer picks a gateway pod, and the gateway (or a sidecar) picks a service replica. The same handful of algorithms runs at each level.

8.2 Layer 4 and layer 7

A load balancer decides per **connection** or per **request**, and that one difference splits the field in two.

```
layer 4 (transport)                   layer 7 (application)
client --TCP--> LB --same TCP--> B1   client --TLS/HTTP--> LB --new conn--> B1
sees: IPs, ports, protocol            sees: method, path, headers, cookies
decides once per connection           decides once per request
```

A **layer 4** balancer looks only at the 5-tuple (source IP and port, destination IP and port, protocol). It hashes or assigns the flow to a backend and forwards its packets, either rewriting addresses (NAT) or passing them through untouched so the backend sees the client's real IP. It never parses HTTP, so it is fast, protocol-agnostic (TCP, UDP, QUIC, database protocols) and cheap per byte. AWS Network Load Balancer is one (GCP: passthrough Network Load Balancer; Azure: Load Balancer), as are Linux IPVS and Google's Maglev.

A **layer 7** balancer terminates the client connection, reads each HTTP request, and opens or reuses its own connections to backends. Because it understands requests, it can route `/v1/holds` to seat inventory and `/v1/events` to catalog, retry a failed idempotent GET on another replica, add `X-Forwarded-For`, compress, cache and enforce limits. AWS Application Load Balancer is one (GCP: Application Load Balancer; Azure: Application Gateway), as are Nginx, HAProxy in HTTP mode and Envoy. The price is CPU per request and a second connection hop.

The connection-versus-request difference bites with **HTTP/2 and gRPC**, which multiplex thousands of requests over one long-lived connection. Behind a layer 4 balancer, every request on that connection lands on the same backend:

$$\text{backends that receive traffic} \le \text{number of client connections}$$

If Encore's 20 gateway pods each hold one gRPC connection to seat inventory, at most 20 of its 500 flash-sale replicas do any work, whatever the algorithm. A Kubernetes `Service` behaves like this, because kube-proxy balances connections, not requests. The fix is per-request balancing at layer 7: a proxy such as Envoy or a service-mesh sidecar, or a client-side balancer over a headless service.

Choose	When
Layer 4	Non-HTTP protocols, extreme packet rates, TLS passthrough, static IPs
Layer 7	Path or header routing, gRPC, retries, auth, per-request fairness
Both	L4 in front for scale and static IPs, L7 fleet behind it for logic

8.3 Balancing algorithms

The algorithms form a ladder: each rung uses more information about the backends and costs more to run.

8.3.1 Round robin and weighted round robin

Round robin hands requests to backends in a fixed cycle: 1, 2, 3, 1, 2, 3. It needs no state beyond a counter and is the default in Nginx, HAProxy and AWS ALB. It is fair only when requests cost the same and backends are equal. When a fleet mixes machine sizes, **weighted round robin** gives backend i a weight w_i and a share of traffic

$$\text{share}_i = \frac{w_i}{\sum_j w_j}$$

so a new 8-vCPU node with weight 2 next to two 4-vCPU nodes with weight 1 takes 2/4 = 50 percent. Nginx interleaves the picks with a "smooth" variant, so a heavy backend never receives a burst of consecutive requests:

```
def smooth_wrr(weights: dict[str, int], n: int) -> list[str]:
    """Nginx-style smooth weighted round robin."""
    current = {name: 0 for name in weights}
    total = sum(weights.values())
    picks = []
    for _ in range(n):
        for name, w in weights.items():
            current[name] += w          # everyone earns its weight
        best = max(current, key=current.get)
        current[best] -= total          # the winner pays the total
        picks.append(best)
    return picks

print(smooth_wrr({"a": 5, "b": 1, "c": 1}, 7))
# ['a', 'a', 'b', 'a', 'c', 'a', 'a']
```

8.3.2 Least connections and least outstanding requests

Round robin is blind: if one replica is stuck in a slow garbage collection, it keeps receiving its turn. **Least connections** sends each new connection to the backend with the fewest open ones, weighted when sizes differ:

$$i^* = \operatorname{argmin}_i \frac{c_i}{w_i}$$

where c_i counts active connections (or, at layer 7, in-flight requests). A slow backend accumulates in-flight work and automatically receives less. AWS ALB calls this *least outstanding requests*; Nginx has `least_conn`; HAProxy has `leastconn` and recommends it for long sessions such as LDAP or SQL connections. It is the natural choice for Encore's live sales dashboards, whose Server-Sent Events streams stay open for hours: round robin would balance the number of *new* streams, not the number of *open* ones.

8.3.3 Least response time

Least response time goes one rung higher and tracks latency, usually as an exponentially weighted moving average updated after each response with latency x:

$$x \quad \alpha x + (1 - \alpha) x, \qquad 0 < \alpha \le 1$$

A backend's score combines its average latency and its in-flight count, for example $x\,(c_i + 1)$, and the lowest score wins. Linkerd balances HTTP and gRPC this way (EWMA), and Envoy and gRPC can weight endpoints from server-reported load (ORCA). The risk is herding: every balancer sees the same fast replica, sends it everything, and it stops being fast. The next section shows the standard cure.

8.4 Power of two choices

Picture a supermarket with fifty tills. Joining a random queue sometimes puts you behind a full trolley; scanning all fifty takes too long. Glance at **two** random tills and join the shorter one, and the bad outcome almost disappears. That is the **power of two choices**: pick $d = 2$ backends at random, send the request to the less loaded of the pair.

Why it works is a classic balls-into-bins result (Azar, Broder, Karlin and Upfal; Mitzenmacher). Throw n requests at n servers. With one random choice the busiest server ends up with about

$$\max \text{load}_{d=1} \approx \frac{\ln n}{\ln \ln n}$$

requests, while choosing the lighter of $d \ge 2$ gives

$$\max \text{load}_{d} = \frac{\ln \ln n}{\ln d} + O(1)$$

an exponential improvement: from $\ln n$ to $\ln \ln n$. Going from two choices to three gains only the constant $\ln 3 / \ln 2 \approx 1.6$; almost all the benefit comes from the second sample. Under heavy load, $m \quad n$ requests, the gap between the busiest and the average server grows like $\sqrt{2(m/n)\ln n}$ for random placement but stays near $\ln \ln n / \ln 2$ for two choices, however many requests arrive (Berenbrink et al., 2000). For $n = 1,000$ that predicts a busiest server near $100 + 37 = 137$ at an average of 100 for random, against about $100 + 3$ for two choices. A simulation agrees:

```
import random

def max_load(servers: int, requests: int, d: int, seed: int = 7) -> int:
    """Send each request to the least loaded of d random servers."""
    rng = random.Random(seed)
    load = [0] * servers
    for _ in range(requests):
        picks = [rng.randrange(servers) for _ in range(d)]
        best = min(picks, key=load.__getitem__)
        load[best] += 1
    return max(load)

n = 1_000
for requests in (n, 100 * n):
    random_max = max_load(n, requests, d=1)
    two_max = max_load(n, requests, d=2)
    print(f"avg {requests // n:>3}: random max {random_max:>3},"
          f" two choices max {two_max:>3}")
# avg   1: random max   6, two choices max   3
# avg 100: random max 138, two choices max 102
```

(The $\ln n / \ln \ln n \approx 3.6$ formula is asymptotic and undershoots the observed 6 at this small n; the two-choice bound of about 3 is already tight.)

Two choices also beats "always pick the global least loaded" in a fleet of balancers. Each balancer's view is slightly stale, so a global minimum sends a burst from *every* balancer to the same idle server; random pairs spread that burst. That is why Envoy's `LEAST_REQUEST` samples two hosts by default, HAProxy's `random` uses two draws by default ("Power of Two Random Choices" in its manual), and Nginx offers `random two least_conn`.

> **Design question.** Encore runs 20 gateway pods in front of 500 seat-inventory pods during the flash sale. Should the gateways share a global view of every pod's load to pick the least loaded one?
>
> No. A shared view needs a coordination store on the hot path (200,000 lookups a second) and is stale by the time it is read, so all 20 gateways chase the same "idle" pod and overload it in turn. Give each gateway its own in-flight counts and use least-request with two random choices. Each pod then sees roughly $200,000/500 = 400$ requests a second and the busiest pod stays within a few in-flight requests of the average instead of tens of percent above it. The trade-off is that each gateway's decisions are only locally optimal, which the math above shows is enough. See also *50 System Designs* #40 (Load Balancer and API Gateway).

8.5 Hashing: IP hash, consistent hashing and Maglev

Sometimes the same caller *should* reach the same backend: a cache shard that holds an event's seat map, or a server holding a WebSocket's state. Hashing maps a key (client IP, user ID, event ID, the connection's 5-tuple) to a backend without storing a table of assignments.

IP hash computes `hash(ip) mod N`. Nginx's `ip_hash` uses the first three octets of an IPv4 address, so a whole /24 lands together, and HAProxy's `balance source` does the same for full addresses. Its flaw is the modulus. When N changes to $N+1$, a key keeps its backend only if both remainders agree, so the fraction that moves is

$$\text{moved}_{\text{mod}} = 1 - \frac{1}{N+1} = \frac{N}{N+1}$$

Growing from 10 to 11 backends remaps about 91 percent of keys; every cache behind the balancer goes cold at once. **Consistent hashing** places backends and keys on a ring and gives each key to the next backend clockwise, so adding or removing one of N backends moves only about $1/N$ of the keys, and virtual nodes even out the arcs (Chapter 17 builds the ring). Nginx offers `hash $request_uri consistent` (ketama) and Envoy offers `RING_HASH`.

Maglev, Google's software load balancer (Eisenbud et al., 2016), trades a little of that stability for speed and evenness. Each backend derives a permutation of the slots of a lookup table of prime size M from two hashes (an offset and a skip); backends take turns claiming their next preferred free slot until the table is full. A lookup is then one array access, `table[hash(key) % M]`, and every backend owns M/N slots to within one:

```
import hashlib
from collections import Counter

def h(s: str, salt: str) -> int:
    return int.from_bytes(hashlib.sha256((salt + s).encode()).digest()[:8])

def maglev(backends: list[str], m: int = 65537) -> list[str]:
    """Lookup table of prime size m (Eisenbud et al., NSDI 2016)."""
    prefs = [(h(b, "o") % m, h(b, "s") % (m - 1) + 1) for b in backends]
    table, nxt, filled = [""] * m, [0] * len(backends), 0
    while True:
        for i, (offset, skip) in enumerate(prefs):  # backends take turns
            slot = (offset + nxt[i] * skip) % m
            while table[slot]:                       # next preferred slot
                nxt[i] += 1
                slot = (offset + nxt[i] * skip) % m
            table[slot], filled = backends[i], filled + 1
            if filled == m:
                return table

pods = [f"seat-{i}" for i in range(10)]
before, after = maglev(pods), maglev(pods[:-1])     # seat-9 dies
counts = Counter(before)
print("slots per pod:", min(counts.values()), "to", max(counts.values()))
moved = sum(a != b and a != "seat-9" for a, b in zip(before, after))
print(f"survivor slots remapped: {moved / len(before):.2%}")
# slots per pod: 6553 to 6554
# survivor slots remapped: 0.18%
```

Removing one of ten pods reassigns its 10 percent of slots, as it must, and disturbs only about 0.2 percent of the others. Google fronts Maglev machines with ECMP routing and adds a connection-tracking table so a live TCP flow never switches backend. Google Cloud's passthrough Network Load Balancers run on Maglev, and Envoy's `MAGLEV` policy uses a table of 65,537 entries; its documentation notes it builds and looks up faster than a ring hash but is less stable when hosts change.

Hashing has one weakness: a hot key stays hot. All 200,000 reads a second for the stadium show carry the same event ID, so hashing by event sends them to one replica. Hash by a key with many values (user, session, connection), and treat a known hot key separately (Chapter 9 and Chapter 17 show how to split it).

8.6 Health checks and connection draining

A balancer is only as good as its list of live backends. It keeps the list two ways. **Active health checks** probe each backend on a schedule (`GET /readyz` every few seconds) and flip it to unhealthy after k consecutive failures. **Passive checks**, called outlier detection in Envoy, watch real traffic and eject a backend that returns a run of 5xx errors or times out. The time a dead backend keeps receiving traffic is roughly

$$T_{\text{detect}} \approx k \times T_{\text{interval}} + T_{\text{timeout}} \qquad \text{failed requests} \approx \frac{\lambda}{N} T_{\text{detect}}$$

With AWS ALB's defaults (interval 30 s, unhealthy threshold 2, timeout 5 s) detection takes about a minute. If a service takes 6,000 requests a second across 10 replicas and one crashes, about $600 \times 60 = 36,000$ requests fail before the balancer notices. A 5-second interval cuts that to about $600 \times 10 = 6,000$; passive detection ejects the replica after its first five 5xx responses. Use both, in Envoy for example:

```
clusters:
- name: seat_inventory
  type: STRICT_DNS
  connect_timeout: 0.25s
  lb_policy: LEAST_REQUEST            # two random choices by default
  load_assignment:
    cluster_name: seat_inventory
    endpoints:
    - lb_endpoints:
      - endpoint:
          address:
            socket_address: {address: seat-inventory, port_value: 8080}
  health_checks:                      # active: probe every 5 s
  - timeout: 1s
    interval: 5s
    unhealthy_threshold: 2
    healthy_threshold: 2
    http_health_check: {path: /readyz}
  outlier_detection:                  # passive: watch real responses
    consecutive_5xx: 5
    base_ejection_time: 30s
    max_ejection_percent: 30
```

Two rules keep health checks from causing outages. First, the readiness probe should test the replica itself, not every dependency: if `/readyz` fails whenever the database blips, every replica fails together and the pool empties. Balancers guard against that case by **failing open**: an ALB target group whose targets are all unhealthy sends traffic to all of them, and Envoy ignores health status once fewer than 50 percent of hosts are healthy (its panic threshold). Second, cap ejections (`max_ejection_percent`) so a bad deploy cannot eject most of the fleet.

Connection draining is the polite exit. A replica that is being removed (scale-in, deploy) stops receiving *new* requests but finishes its in-flight ones within a deadline: the ALB *deregistration delay*, 300 seconds by default. In Kubernetes the pod receives `SIGTERM` at the same moment it is removed from the endpoint list, and the balancers learn of the removal a little later, so a pod that exits at once drops requests that are still arriving. The usual recipe is a `preStop` hook that sleeps 5 to 15 seconds, then a graceful shutdown that refuses new work and finishes the rest, all inside `terminationGracePeriodSeconds` (30 seconds by default). Set the balancer's drain deadline slightly above the longest

normal request; long-lived streams (WebSockets, SSE) need the server to tell clients to reconnect elsewhere. New replicas get the mirror image, **slow start**, which ramps their share up over a configured window (ALB, HAProxy and Envoy all offer it) while caches and JIT compilers warm up.

8.7 Sticky sessions and why to avoid them

Sticky sessions (session affinity) pin a client to one backend, usually with a cookie the balancer sets: ALB's duration-based cookie is `AWSALB`, and the first request is placed by the normal algorithm. They exist so a server can keep a user's session in local memory. The cost shows up exactly when it hurts. Load drifts uneven, because one heavy user stays on one replica. New replicas added for a surge receive only *new* sessions, so scale-out barely relieves the old ones (the ALB documentation suggests expiring cookies to rebalance). And when a replica dies, its users lose their state.

The fix is to make the servers **stateless**: keep sessions in a signed token or in Redis (Chapter 9), so any replica can serve any request. Keep affinity only where the state is inherently local, such as a WebSocket connection (sticky by nature) or a cache shard, and then prefer consistent hashing on a key you control over a cookie. Encore's seat holds live in Redis and the database, never in a pod, so a fan whose pod dies mid-checkout retries on another pod and finds the hold intact.

8.8 TLS termination

Every HTTPS connection begins with a handshake that costs round trips and CPU. **TLS termination** decides where that happens: at the balancer (which decrypts, routes on HTTP and forwards plaintext or re-encrypts to the backend), or at the backend (**TLS passthrough**, where a layer 4 balancer routes on the unencrypted SNI host name and never sees the content). Terminating at the edge centralises certificates (AWS Certificate Manager; GCP Certificate Manager; Azure Key Vault), offloads crypto from the services, and is what makes layer 7 routing possible. Inside the VPC, re-encrypt or use mutual TLS from a service mesh when compliance or zero-trust policy requires it (Chapter 22).

The placement also changes latency. A new HTTPS request over TCP and TLS 1.3 needs one round trip for TCP, one for TLS and one for the request itself:

$$T_{\text{first byte}} \approx 3 \times \text{RTT}_{\text{client-terminator}} + \text{RTT}_{\text{terminator-origin}}$$

A fan in Frankfurt talking straight to Virginia (RTT about 90 ms) waits about $3 \times 90 = 270$ ms. Terminating at an edge 10 ms away, which holds a warm connection to the origin, gives about $3 \times 10 + 90 = 120$ ms. TLS 1.2 adds one more round trip, and HTTP/3 over QUIC folds the transport and TLS handshakes into one (Chapter 4). A minimal Nginx terminator in front of the catalog service:

```
upstream catalog {
    least_conn;
    server 10.0.1.10:8080 max_fails=3 fail_timeout=10s;
    server 10.0.2.10:8080 max_fails=3 fail_timeout=10s;
    keepalive 64;                     # reuse upstream connections
}
server {
    listen 443 ssl;
    http2 on;
    ssl_certificate     /etc/ssl/encore.pem;
    ssl_certificate_key /etc/ssl/encore.key;
    ssl_protocols       TLSv1.2 TLSv1.3;
    location /v1/events/ {
        proxy_pass http://catalog;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

8.9 Global load balancing

Within a region, a balancer picks a replica. Across regions, something must pick the *region*, before any connection reaches a balancer. Two mechanisms do it.

DNS steering (GeoDNS). The authoritative DNS server answers the same name with different addresses depending on who asks. Amazon Route 53 (GCP: Cloud DNS routing policies; Azure: Traffic Manager) offers latency-based, geolocation, geoproximity, weighted, IP-based and failover routing, each optionally tied to health checks. It works with any protocol, but failover is only as fast as caches allow:

$$T_{\text{failover}} \approx T_{\text{detect}} + \text{TTL} + T_{\text{stale caches}}$$

With 30-second health checks, a failure threshold of 3 and a 60-second TTL, that is about 90 + 60 = 150 seconds, plus however long resolvers and clients ignore the TTL.

Anycast. Many edge sites announce the *same* IP address over BGP, and the internet's routing delivers each packet to the nearest one; if a site fails, its announcement is withdrawn and traffic flows to the next nearest within seconds, without waiting for DNS. AWS Global Accelerator gives two static anycast IPs in front of regional endpoints; Google Cloud's global external Application Load Balancer exposes a single anycast IP for backends in many regions; Azure Front Door and Cloudflare are anycast networks with layer 7 features. CDNs use the same two techniques to choose a PoP (Chapter 10).

> **Design question.** Encore runs in two regions. How should fans reach the right one, and what happens when a region fails during a sale?
>
> Reads (event pages, search, seat maps) are served active-active from both regions, so steer by latency: Route 53 latency records with health checks, or Global Accelerator for static IPs and failover in seconds rather than minutes. Writes for one event's seats must not be split, because two regions selling the same seat is overselling; each event's inventory has a home region, and the gateway forwards hold and order requests there. On a regional failure, traffic shifts to the survivor, which promotes its replica of the inventory (Chapter 16). The trade-off: fans far from an event's home region pay one cross-region round trip (about 90 ms) on each hold. See also *50 System Designs* #25 (Ticket Booking).

8.10 API gateways

A load balancer asks "which replica?". An **API gateway** asks "should this request be served at all, and by which service?". It is a layer 7 proxy with an opinion about every request: it **authenticates** (API keys, OAuth 2.0 bearer tokens, JWT signature checks), **routes** by path, method, header or version to services, **limits** rates per user, key or tenant (Chapter 11), **transforms** (header injection, request and response rewriting, protocol translation such as REST to gRPC), and **observes** (access logs, metrics, trace IDs). Doing this once at the edge keeps every service free of the same boilerplate. A rejected hold never reaches seat inventory:

```
POST /v1/events/ev_42/holds HTTP/2
Host: api.encore.example
Authorization: Bearer eyJhbGciOiJFUzI1NiJ9...
Idempotency-Key: 6f1c2a9e-hold-1

HTTP/2 429 Too Many Requests
Retry-After: 2
RateLimit-Policy: "holds";q=5;w=10
Content-Type: application/problem+json

{"type": "https://encore.example/errors/rate-limited", "status": 429}
```

Managed gateways include Amazon API Gateway (GCP: API Gateway and Apigee; Azure: API Management); self-run ones include Kong, Envoy Gateway and the Kubernetes Gateway API implementations. Know their limits before a flash sale: Amazon API Gateway throttles each account at 10,000 requests a second per region by default, with a token-

bucket burst of 5,000, and REST API integrations time out after 29 seconds by default. The gateway is also where a backend for frontend (BFF) lives when web and mobile need different shapes of the same data (Chapter 6).

> **Design question.** Hold attempts reach 30,000 a second in the first minute of the sale. Can they go through Amazon API Gateway?
>
> Not at the default quota: 30,000 exceeds the 10,000 per second account limit three times over, and the limit is shared with every other API in the account and region. Options: request a quota increase well ahead and load-test it; or put the hot path (holds, seat maps) on ALB plus a fleet of Envoy gateway pods that scale with the sale, keeping the managed gateway for partner and admin APIs. Better still, the waiting room admits fans at a controlled rate, so the hold path sees a planned load rather than the raw stampede (Chapter 11). The trade-off of self-running Envoy is operating it: upgrades, autoscaling, and configuration safety. See also *50 System Designs* #40 (Load Balancer and API Gateway).

External systems and agents. The gateway is also the door for systems Encore does not control. The payment provider's webhooks arrive through it, and a route-specific policy verifies their signatures before the orders service sees them; outbound calls to the provider and the e-mail and SMS vendors leave through an egress proxy with its own timeouts and limits (Chapter 24). LLM traffic adds new balancing problems. A *model gateway* routes requests across model providers and self-hosted model servers, falls back when one is down, and limits by tokens rather than requests. Self-hosted model replicas are best balanced on server signals such as queue length and prefix-cache contents, which the Kubernetes Gateway API Inference Extension uses (Envoy AI Gateway adds provider routing and token limits), and streamed answers over SSE call for least-request balancing because one request can occupy a replica for many seconds. The support agent's tools and MCP servers sit behind the same gateway as any other service, with the same authentication and quotas; *Designing Agentic Systems* covers the agent side.

8.11 Where they run

System	Layer	Algorithms and notes
Nginx	L7 (L4 via `stream`)	Round robin default; `least_conn`, `ip_hash`, `hash ... consistent`, `random two`
HAProxy	L4 and L7	`roundrobin` default; `leastconn`, `source`, `uri`, `random` (2 draws)
Envoy	L4 and L7	`LEAST_REQUEST` (two choices), `RING_HASH`, `MAGLEV`; outlier detection
AWS ALB	L7	Round robin, least outstanding requests, weighted random
AWS NLB	L4	Flow hash, static IPs, TLS listeners, client IP preserved
Google Cloud LB	L4 and L7	Envoy-based Application LBs; Maglev passthrough; global anycast IP
Azure	L4, L7, DNS	Load Balancer, Application Gateway, Front Door, Traffic Manager
Cloudflare	L7 edge	Anycast network; balances across origin pools
Kubernetes	L4 and L7	`Service` (per connection), Ingress and Gateway API (per request)

Envoy is the common engine of modern stacks: Istio's sidecars, Envoy Gateway, and Google Cloud's Application Load Balancers all run it.

8.12 Capacity sizing

Size the fleet behind a balancer from the peak rate λ, the measured capacity c of one replica, a target utilization u that keeps queues short (Chapter 2), and enough headroom to lose one of Z availability zones:

$$N = \frac{\lambda}{cu} \cdot \frac{Z}{Z-1}$$

With an illustrative $c = 1,000$ requests a second per seat-map pod (found by load test), $u = 0.6$ and $Z = 3$: the daily peak of 6,000 requests a second needs $N = 15$ pods, and the flash sale's 200,000 seat-map reads a second need $N = 500$, about 167 per zone, unless the CDN absorbs most of them (Chapter 10).

Size the balancer too. AWS ALB bills and scales in Load Balancer Capacity Units, each worth 25 new connections a second, 3,000 active connections, 1 GB processed per hour (EC2, container and IP targets) or 1,000 rule evaluations a second; the busiest dimension counts. For the sale, 1,000,000 fans holding connections need $10^6/3,000 \approx 333$ LCUs; their arrival over ten minutes, about 1,667 new connections a second, needs about 67; but 200,000 reads a second of a 5 KB seat-map delta (illustrative) is 1 GB a second, 3,600 GB an hour, so 3,600 LCUs. Bytes, not requests, dominate, which is the argument for serving the seat map from the edge. Managed balancers scale out on their own but not instantly; for a scheduled spike, reserve a capacity floor ahead of time (ALB LCU reservation, minimum 100 LCUs) or load-test the warm-up.

8.13 In Encore

Tier	Choice	Why
Region	Route 53 latency records or Global Accelerator	Nearest healthy region; inventory writes go to the event's home region
Edge	CloudFront	TLS near fans; images and event pages cached; waiting-room token check
Regional entry	ALB with least outstanding requests	Path routing, TLS from the edge, WAF, health checks
Gateway	Envoy pods, two-choice least request	Auth, per-user hold limits, gRPC per-request balancing
Seat-inventory cache	Consistent hashing on seat-block key	Keeps cache shards warm; the hot event is split into blocks
Dashboards (SSE)	Least connections, long drain	Streams stay open for hours
Payment webhooks	Dedicated gateway route	Signature check, idempotent handler

8.14 Numbers to remember

Figure	Value
Keys moved, `mod N` to `N + 1`	$N/(N+1)$, 91% for 10 to 11
Keys moved, consistent hashing	about $1/N$
Max load, random vs two choices	$\ln n/\ln\ln n$ vs $\ln\ln n/\ln 2$
Envoy Maglev table	65,537 slots
ALB health check defaults	30 s interval, 2 failures, 5 s timeout
ALB deregistration delay	300 s default
Kubernetes grace period	30 s default
Envoy outlier detection	5 consecutive 5xx, 30 s ejection
Amazon API Gateway	10,000 req/s per account and region, burst 5,000
One ALB LCU	25 new conn/s, 3,000 active, 1 GB/h, 1,000 rules/s

Figure	Value
DNS failover	detection + TTL, often minutes

8.15 Pitfalls

- Balancing gRPC or HTTP/2 at layer 4, so a few replicas take all the traffic -> balance per request with Envoy, a mesh, or a client-side balancer.
- `hash mod N` for cache affinity, so every scale event empties the caches -> consistent hashing or Maglev.
- Round robin over replicas with very uneven request costs -> least outstanding requests or two-choice least request.
- Readiness checks that test every dependency, so one database blip empties the pool -> check the replica itself; rely on fail-open and ejection caps.
- Pods that exit on `SIGTERM` while balancers still send to them -> `preStop` sleep, graceful shutdown, drain deadline above the longest request.
- Sticky sessions holding user state in a pod -> stateless replicas with state in Redis or a token.
- Assuming a managed balancer or gateway absorbs any spike instantly -> check quotas, reserve capacity, load-test the warm-up.
- Retries at the balancer for non-idempotent POSTs, so a hold is placed twice -> retry only idempotent requests or require an `Idempotency-Key`.
- DNS failover with long TTLs -> short TTLs on failover records, or anycast for seconds-level failover.

8.16 Remember

- Layer 4 balances connections and is fast; layer 7 balances requests and can route, retry and authenticate.
- Two random choices and pick the lighter: almost all the benefit of least loaded, none of the herding.
- Hash for affinity, but with consistent hashing or Maglev so membership changes move about 1/*N* of keys.
- Health checks plus outlier detection decide how many requests a dead replica eats; draining decides how many a deploy drops.
- The gateway is the policy point: authentication, routing, rate limits and observability once, at the edge.
- Size both the fleet and the balancer for the peak, with headroom for a lost zone, and know every managed quota.

9 Caching and Redis

How do you serve 200,000 seat-map reads a second when the database behind them can answer a few thousand? You keep a copy of the answer closer to the reader, in faster memory, and you decide how stale that copy may be. This chapter is about that decision: where the copies live, how they are filled and evicted, how they go wrong under load, and how Redis, the most common cache in modern designs, works inside.

9.1 The picture

A cache is the notepad on your desk. The library (the database) has every book, but walking there takes minutes; the notepad holds the dozen facts you looked up this morning and answers in a second. The notepad is small, so you cross out what you have not used lately (eviction). The library can change a fact after you copied it, so you write an expiry date next to each note (TTL) or the librarian phones you when something changes (invalidation). Every layer below answers what it can and passes the rest down, slower and costlier at each step, so fewer requests should get there (latencies illustrative):

```
fan --> browser cache --> CDN edge --> API pod (in-process cache)
  ~0 ms        ~10-30 ms      |
                              v
                        Redis (shared cache)   ~0.5 ms
                              | miss
                              v
                        Postgres (buffer pool, then disk)   ~5 ms
```

9.2 Where caches live

Layer	Holds	Controlled by	Example
Browser, app	responses, images	`Cache-Control`, `ETag`	event images
CDN edge	public responses	TTL, purge	event pages
In-process	hot objects	your code	seat-map snapshot
Shared cache	objects, sessions	your code	Redis, Memcached
DB buffer pool	pages of tables	the database	Postgres buffers

The **browser** and the **CDN** are covered in Chapter 10; they are driven by HTTP headers such as `Cache-Control: max-age=60` and conditional requests with `ETag`. A **reverse proxy** (Nginx `proxy_cache`, Varnish) caches whole responses in front of your service. An **in-process cache** (a Python dict, `functools.lru_cache`, Caffeine in Java) costs nanoseconds but is private to one pod, so 100 pods hold 100 copies that disagree for up to a TTL. A **shared cache** (Redis, Memcached) costs a network round trip, about half a millisecond inside one availability zone, but every pod sees the same copy. The **database buffer pool** caches disk pages in RAM: Postgres `shared_buffers` (default 128 MB, commonly set to about a quarter of RAM, with the OS page cache behind it) and the MySQL InnoDB buffer pool. A query that hits the buffer pool still pays for parsing, planning and locking; a cache hit skips the database entirely.

Cache what is read far more often than it changes, is expensive to compute, and tolerates a little staleness: event details, venue maps, search results, rendered seat maps, sessions. Do not cache the source of truth for anything that must be exact, such as whether a seat is sold.

9.3 Caching patterns

- **Cache-aside** (lazy loading) is the default. The application owns both calls; only requested data is cached; a cache outage degrades to database reads instead of errors. The cost is a slow first read and the invalidation race described below.

- **Read-through** moves the loading into the cache layer or a library, so callers see one interface. Amazon DynamoDB Accelerator (DAX) is a read-through and write-through cache in front of DynamoDB.
- **Write-through** writes the cache and the database in the same request, so reads after a write are fresh; every write pays both latencies, and data that is never read is cached anyway.
- **Write-behind** (write-back) acknowledges after the cache write and flushes in batches. It absorbs write bursts (view counters, analytics) but loses the unflushed writes if the cache node dies, so never use it for orders or payments.
- **Refresh-ahead** reloads hot keys before they expire, so readers never see a miss on them; it wastes work on keys that would not have been read again.

Cache-aside in Python, against Redis (here `fakeredis`) and SQLite standing in for Postgres:

```
import json, random, sqlite3
import fakeredis

r = fakeredis.FakeRedis(decode_responses=True)
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE events (id INTEGER PRIMARY KEY, name TEXT)")
db.execute("INSERT INTO events VALUES (42, 'Stadium Show')")


def get_event(event_id: int) -> dict | None:
    key = f"event:{event_id}"
    if (hit := r.get(key)) is not None:          # 1. try the cache
        return json.loads(hit)
    row = db.execute("SELECT id, name FROM events WHERE id = ?",
                     (event_id,)).fetchone()     # 2. miss: read the DB
    value = {"id": row[0], "name": row[1]} if row else None
    ttl = int(300 * random.uniform(0.9, 1.1))   # jittered TTL
    r.set(key, json.dumps(value), ex=ttl)    # 3. fill
    return value                                 # (None is cached too)

def rename_event(event_id: int, name: str) -> None:
    with db:                                     # 1. write the DB, commit
        db.execute("UPDATE events SET name = ? WHERE id = ?",
                   (name, event_id))
    r.delete(f"event:{event_id}")                # 2. then invalidate

print(get_event(42), r.ttl("event:42"))         # miss, then filled
rename_event(42, "Stadium Show (new date)")
print(get_event(42))                             # fresh value
print(get_event(7))                              # negative result cached
```

Two details carry most of the value. The write path *deletes* the key rather than writing the new value, because a delete is idempotent and cannot install a value computed from an older read. And a missing row is cached as `null` (negative caching), so a bot hammering nonexistent event IDs does not reach the database; give negative entries a shorter TTL in production.

9.4 Eviction: LRU, LFU and TTL

A full cache must forget something to remember something new. **Expiry** removes a key because its TTL ran out (a freshness rule); **eviction** removes a key because memory is full (a space rule). You need both.

- **LRU** (least recently used) forgets what has not been touched for longest. It bets that recent use predicts future use, which holds for most web traffic.
- **LFU** (least frequently used) forgets what is used least often, so a steadily popular key survives a burst of one-off reads (a crawler walking every old event). It needs decay, or yesterday's hit keeps squatting. **TTL-based** eviction forgets what expires soonest; **random** eviction is cheap and fine when access is uniform.

An exact LRU is a hash map plus a doubly linked list ordered by recency; both `get` and `put` are $O(1)$. Python's `OrderedDict` is exactly that pair:

```python
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.data: OrderedDict[str, object] = OrderedDict()

    def get(self, key: str):
        if key not in self.data:
            return None                      # miss
        self.data.move_to_end(key)           # now most recently used
        return self.data[key]

    def put(self, key: str, value) -> None:
        self.data[key] = value
        self.data.move_to_end(key)
        if len(self.data) > self.capacity:
            self.data.popitem(last=False)    # evict least recently used

cache = LRUCache(2)
cache.put("event:1", "Arena")
cache.put("event:2", "Stadium")
cache.get("event:1")                         # touch event:1
cache.put("event:3", "Club")                 # evicts event:2
print(list(cache.data))                      # ['event:1', 'event:3']
```

Redis does not keep a linked list over millions of keys; it **approximates**. On eviction it samples `maxmemory-samples` keys (default 5) and evicts the best candidate, keeping a small pool of good candidates between rounds; 10 samples come very close to true LRU. Its LFU stores a logarithmic (Morris) counter of 8 bits per key that decays every `lfu-decay-time` minutes (default 1). The policy is set with `maxmemory-policy`: the default is `noeviction` (writes fail when memory is full), so a pure cache should set `allkeys-lru` or `allkeys-lfu`. The `volatile-*` variants evict only keys that carry a TTL, and Redis 8.6 added `allkeys-lrm` and `volatile-lrm` (least recently *modified*). `INFO stats` reports `keyspace_hits`, `keyspace_misses` and `evicted_keys`, from which you compute the hit ratio.

9.5 Invalidation and consistency

A cache is a replica with no replication protocol, so it is eventually consistent at best. The classic cache-aside race shows why a TTL must always be the safety net:

```
reader A: GET event:42 -> miss
reader A: SELECT ... -> "old name"            (slow, holds the old row)
writer B: UPDATE ... "new name"; COMMIT
writer B: DEL event:42
reader A: SET event:42 "old name" EX 300      (stale for up to 300 s)
```

Ways to narrow or close the window, from cheapest to strongest:

- **Short TTLs** bound staleness: the worst case equals the TTL. Pick it per data type from the business answer to "how stale may this be?"
- **Delete after commit**, never before, and never update the cache inside the transaction that may roll back.
- **Versioned keys**: store `event:42:v17` and bump the version in the database row; readers that know the current version never read an old entry, and old entries age out.
- **Leases**: on a miss the cache hands the reader a token, and a delete invalidates outstanding tokens, so A's late `SET` is rejected. Facebook's memcache paper (2013) uses leases against both stale sets and stampedes.
- **Change data capture**: a consumer of the database's commit log (Postgres logical replication through Debezium, DynamoDB Streams, or Spanner change streams and the Cosmos DB change feed) deletes keys after every committed change, so invalidation cannot be forgotten by a code path. See Chapter 14.
- **Read-your-writes**: after a user changes their own data, read it from the database (or write it through) for that user for a few seconds, as in Chapter 16.

Design question. An organizer moves a show from Friday to Saturday. How stale may the cached event page be, and how do you bound it?

Event details are read about a thousand times for every write, so cache-aside with a TTL fits. The business answer is "a minute at most; never wrong at checkout". Use a 60-second TTL (plus jitter) on the event object in Redis and `max-age=60` at the CDN, delete the Redis key and purge the CDN path on every edit, and let the CDC consumer repeat the delete in case the editing code path forgets. Checkout reads the date from the orders service's own database, never from the cache, so the ticket is correct even if a page lagged. The trade-off: a 60-second TTL on an illustrative 30,000 on-sale events costs about 500 database reads a second in expiry misses, which is cheap insurance. See also *50 System Designs* #4 (Distributed Cache).

9.6 The numbers: hit ratio, latency and load

Three equations size every cache. With hit ratio h, cache latency L_c and database latency L_d (cache-aside: a miss pays both), and request rate R:

$$L_{\text{eff}} = h\,L_c + (1-h)\,(L_c + L_d) = L_c + (1-h)\,L_d \qquad R_{\text{db}} = (1-h)\,R$$

$$M = N_{\text{hot}} \times (s_{\text{key}} + s_{\text{value}} + s_{\text{overhead}}) \times f$$

Take Encore's illustrative figures L_c = 0.5 ms for Redis and L_d = 5 ms for Postgres, with the flash-sale seat-map rate $R = 200,000$/s:

Hit ratio	Effective latency	Database reads/s
90%	1.0 ms	20,000
95%	0.75 ms	10,000
99%	0.55 ms	2,000

Latency barely moves between 95 and 99 percent, but database load falls fivefold. **Each extra nine of hit ratio divides the database load by ten**; that is why caches are sized for the miss rate the database can survive, not for average latency.

For memory, N_{hot} is the working set (keys read within one TTL), the overhead is Redis's per-key bookkeeping (tens of bytes per key), and f is a headroom factor of about 1.3 to 2 for fragmentation, replication buffers and growth. Encore's illustrative working set: 30,000 on-sale events at 10 KB per cached event document is 300 MB; sessions for 4 million daily users at 1 KB each are 4 GB. With $f = 1.5$ both fit in about 6.5 GB, so a pair of 13 GB nodes (primary and replica) is plenty. Memory is rarely the limit for a cache; throughput on one hot key is, as the next section shows.

9.7 Stampedes and hot keys

When a hot key expires, every request in flight misses at once and they all rebuild the same value: a **cache stampede** (thundering herd). The number of simultaneous rebuilds is Little's law, arrival rate times rebuild time:

$$N_{\text{rebuild}} = R_{\text{key}} \times t_{\text{rebuild}}$$

At 200,000 reads/s on Encore's stadium seat map and a 50 ms rebuild query, one expiry launches $200,000 \times 0.05 = 10,000$ identical queries. Four defences, usually combined:

- **Request coalescing** (single flight): within one process, the first miss starts the load and later misses await the same future. With 100 pods, the 10,000 rebuilds become at most 100.
- **A rebuild lock**: `SET lock:key token NX PX 2000` lets one caller cluster-wide rebuild while the others serve the old copy or wait briefly. Release the lock only if you still own it (compare the token, atomically in Lua).
- **Probabilistic early expiration** (XFetch, Vattani et al., 2015): each reader recomputes early with a probability that rises as expiry approaches, so one lucky request refreshes the key before anyone misses.
- **Jittered TTLs**: 30,000 keys filled together after a deploy, all with TTL 300 s, expire in the same second. With ±10% jitter their expiries spread over 60 s, about 500 misses per second instead of 30,000 at once.

XFetch stores the value with its expiry time t_{exp} and the time δ the last rebuild took. With U uniform on (0, 1] and a tuning knob β (default 1), a reader recomputes when

$$t_{now} - \delta\,\beta\,\ln U \;\geq\; t_{exp} \qquad P(\text{early at } T \text{ before expiry}) = e^{-T/(\delta\beta)}$$

With δ = 0.2 s, a request one second before expiry refreshes with probability $e^{-5} \approx 0.7\%$. At a rate r of requests to the key, the first early refresh comes about $\delta\beta\ln(r\,\delta\beta)$ before expiry: at 1,000 requests/s, $0.2\ln 200 \approx 1.1$ s early, which a simulation confirms. Hot keys refresh early, cold keys simply expire, and no coordination is needed. Below, XFetch decides when to refresh and a short lock makes sure only one caller does it while the rest keep serving the old copy (a cold miss still rebuilds directly; coalesce those in-process):

```
import json, math, random, time
import fakeredis

r = fakeredis.FakeRedis(decode_responses=True)

def fetch(key: str, rebuild, ttl: int = 5, beta: float = 1.0):
    if raw := r.get(key):
        e = json.loads(raw)
        jump = -e["delta"] * beta * math.log(1 - random.random())
        if time.time() + jump < e["exp"]:        # XFetch says: not yet
            return e["v"]
        if not r.set(f"lock:{key}", 1, nx=True, px=2_000):
            return e["v"]                        # someone else refreshes
    start = time.time()
    value = rebuild()                            # one caller recomputes
    entry = {"v": value, "delta": time.time() - start,
             "exp": time.time() + ttl}
    r.set(key, json.dumps(entry), ex=ttl)
    return value

slow = lambda: time.sleep(0.05) or {"event": 42, "free": 41_250}
print(fetch("seatmap:42", slow), fetch("seatmap:42", slow))
```

A **hot key** is the other face of the problem: one key read so often that the single Redis shard owning it saturates, however large the cluster. Hashing cannot spread one key. The fixes are to cache it closer (an in-process copy with a one-second TTL on every pod turns 200,000 reads/s into 100 Redis reads/s for 100 pods), to replicate it (read replicas, or k copies `seatmap:42:#1` to `seatmap:42:#k` chosen at random by readers), or to push it to the CDN. Redis 6+ also offers **server-assisted client-side caching**: with `CLIENT TRACKING ON` the server remembers which keys a connection read and pushes an invalidation when any of them change, so pods can keep in-process copies safely. Find hot keys with `redis-cli --hotkeys` (needs an LFU policy) or client-side metrics.

> **Design question.** How would you keep Encore's seat map fast when 200,000 fans refresh it every second?
>
> The seat map is read-mostly, and a fan tolerates a second of staleness because the hold request is checked against seat inventory, the source of truth. Store availability as a Redis bitmap: 50,000 seats are 6,250 bytes. Even so, 200,000 reads/s of 6.25 KB is 1.25 GB/s, about 10 Gbit/s, from one key on one shard: too much. So each API pod keeps an in-process copy with a 1-second TTL and coalesced refreshes (100 pods mean 100 Redis reads/s), and clients poll with a short `max-age` at the CDN or receive deltas over SSE. Inventory updates the bitmap on every hold and sale. The trade-off is a second of staleness: a fan may click a seat that was taken a moment ago and see "just taken, pick another". See also *50 System Designs* #25 (Ticket Booking).

9.8 Redis data structures

Redis is a server of data structures, not of opaque blobs: you send a command that operates on a structure in place, so a counter increments or a queue pops without a read-modify-write round trip. Every key can carry a TTL.

Type	Shape	Key commands	Encore use
String	bytes, up to 512 MB	`SET`, `GET`, `INCR`	cached JSON, counters

Type	Shape	Key commands	Encore use
Hash	field -> value	`HSET`, `HGETALL`	sessions, hold details
List	linked sequence	`LPUSH`, `BRPOP`, `LMOVE`	simple job queue
Set	unique members	`SADD`, `SCARD`	a fan's held seats
Sorted set	member + score	`ZADD`, `ZRANK`, `ZRANGE`	waiting-room line
Stream	append-only log	`XADD`, `XREADGROUP`	order events
HyperLogLog	12 KB sketch	`PFADD`, `PFCOUNT`	unique visitors
Bitmap	bits in a string	`SETBIT`, `BITCOUNT`	seat availability
Geo	sorted set of geohashes	`GEOADD`, `GEOSEARCH`	venues near me

```
HSET hold:42:A-12 user u7 expires 1791975600     -> 2
ZADD wait:42 1791975300.123 u7 1791975300.456 u9 -> 2
ZRANK wait:42 u9                                 -> 1 (second in line)
XADD orders * order 1001 status paid             -> 1791975301000-0
PFADD uv:42 u7 u9 u7                             -> 1 (PFCOUNT -> 2)
SETBIT seats:42 1234 1                           -> 0 (previous bit)
```

A few internals are worth knowing in interviews. Sorted sets are a skip list plus a hash table, so `ZADD` and `ZRANK` are *O*(*log*n) (see Chapter 19). Small hashes, sets and sorted sets use a compact listpack encoding until they pass a size threshold, which is why many small hashes beat many top-level keys for memory. A HyperLogLog counts distinct items with a standard error of 0.81 percent in at most 12 KB, whatever the cardinality. A bitmap of 50,000 stadium seats is 6,250 bytes. Streams add consumer groups with acknowledgements and a pending-entries list, a small Kafka inside Redis (Chapter 14). Since Redis 7.4, `HEXPIRE` gives individual hash fields their own TTL (Valkey added it in 9.0), and Redis 8 folds JSON, time series, probabilistic structures, the query engine and vector sets into the core distribution.

9.9 Why Redis is fast

Picture one very fast clerk at a counter with a ticket machine. The clerk serves one customer at a time, never leaves the counter, and every request is a few seconds of work, so the line moves quickly and nobody ever needs a lock on the filing cabinet.

```
clients --> sockets --> epoll event loop --> one thread runs commands
                        (I/O threads may          in arrival order,
                         read/write sockets)      one at a time
```

- **Memory, not disk**: a command touches RAM only (about 100 ns per access, see Chapter 2); persistence happens in a forked child or a background fsync.
- **One thread executes commands**, so every command is atomic and there are no locks or context switches on the data path. Optional I/O threads (Redis 6+, much improved in Redis 8 and Valkey 8) parse requests and write replies, but execution stays serial.
- **An event loop** (epoll on Linux) multiplexes tens of thousands of connections on that thread (Chapter 3). Simple structures with known complexity and a compact binary protocol (RESP) keep each command short.

The flip side follows directly: **one slow command stalls everyone**. `KEYS *`, `HGETALL` on a million-field hash, `SMEMBERS` on a huge set or a long Lua script block the server for their whole run. Use `SCAN`, `HSCAN` and `SSCAN` to iterate, `UNLINK` instead of `DEL` for big keys (it frees memory in the background), and watch `SLOWLOG GET`. One instance uses one core for commands, so you scale by running more shards, not bigger machines. Throughput is network-bound more often than CPU-bound. Without pipelining, `redis-benchmark` on ordinary hardware typically reports on the order of 100,000 or more simple operations per second per instance; pipelining 16 commands per round trip pushes that past a million, because the round trip, not the command, dominates. Treat those as orders of magnitude, measured on your own hardware. In redis-py, `r.pipeline(transaction=False)` queues commands and `execute()` sends them in one round trip.

9.10 Persistence, replication and failover

A pure cache can lose everything on restart and refill from the database (at the price of a cold-start surge). Once Redis holds sessions, holds or queues, you need it to survive restarts:

- **RDB snapshots**: Redis forks, and the child writes a compact point-in-time file using copy-on-write memory (`save 60 1000` means every 60 s if 1,000 keys changed). Fast restarts and good backups, but you lose the writes since the last snapshot, and `fork` on a large dataset can pause the server briefly.
- **AOF** (append-only file): every write command is appended and replayed on restart. With the default `appendfsync everysec` you lose at most about one second of writes; `always` fsyncs per batch and is slow; `no` leaves it to the OS. Since Redis 7.0 the AOF is multi-part (a base snapshot plus incremental files) and is rewritten in the background. The documentation recommends RDB plus AOF if you want safety close to a database; on restart the AOF wins because it is more complete.

Replication is asynchronous: the primary streams its command log to replicas and acknowledges clients without waiting, so a primary that dies can take its last acknowledged writes with it. `WAIT numreplicas timeout` blocks until that many replicas have the write, which narrows the window but does not make Redis a consensus system (Chapter 16). **Sentinel** adds automatic failover to a primary-replica setup without sharding: three or more Sentinel processes monitor the primary; when a quorum agrees it is down, one is elected to promote a replica and clients ask Sentinel for the new address. **Redis Cluster** has failover built in. In both, a network partition can leave a stale primary accepting writes until it notices it is in the minority, so those writes can be lost. The lesson for design: Redis is an excellent *fast* store and a weak *durable* one. Keep anything you cannot afford to lose (orders, payments, sold seats) in a database, and treat Redis state as reconstructible or as an accepted risk. AWS MemoryDB is the exception worth knowing: it is Redis- and Valkey-compatible but commits writes to a multi-AZ transaction log before acknowledging.

9.11 Redis Cluster

Redis Cluster shards the keyspace into **16,384 hash slots**. Each primary owns a range of slots and has replicas; the client library caches the slot map and sends each command straight to the owner, with no proxy in between.

$$\text{slot}(k) = \text{CRC16}(k) \bmod 16384$$

```
slots 0-5460         slots 5461-10922      slots 10923-16383
[primary A]+replica  [primary B]+replica   [primary C]+replica
     ^ client hashes the key, sends to the owner;
       a wrong guess gets "-MOVED 3999 10.0.0.2:6379"
```

- The CRC16 variant is XMODEM. A **hash tag** changes what is hashed: if a key contains `{...}` with at least one character inside, only that substring is hashed, so `hold:{evt:42}:A-12` and `user:{evt:42}:u7` land in the same slot.
- **Multi-key commands, transactions and Lua scripts must touch keys in one slot**, or they fail with `CROSSSLOT`. Hash tags are how you make related keys co-located; overuse them and one slot becomes a hot shard.
- **Resharding** moves slots between nodes one key batch at a time. During a move the source answers `ASK` for keys already migrated (a one-off redirect), and after the move `MOVED` (update your map).
- Nodes gossip over a cluster bus on the data port plus 10,000 (for example 16379) and promote a replica when a majority of primaries agree its primary failed. The design target is up to about 1,000 nodes, and replication stays asynchronous.

Fixed slots are a form of the "many fixed partitions" strategy from Chapter 17: adding a node moves some slots, never rehashes keys, and the slot is small enough to move quickly. The slot function in Python reproduces `CLUSTER KEYSLOT`:

```
def crc16(data: bytes) -> int:              # CRC-16/XMODEM
    crc = 0
    for byte in data:
        crc ^= byte << 8
        for _ in range(8):
            crc = (crc << 1) ^ 0x1021 if crc & 0x8000 else crc << 1
            crc &= 0xFFFF
    return crc

def hash_slot(key: str) -> int:
    s = key.find("{")
    if s != -1:
        e = key.find("}", s + 1)
        if e > s + 1:                       # non-empty tag: hash only it
            key = key[s + 1:e]
    return crc16(key.encode()) % 16384

for k in ["somekey", "foo{hash_tag}", "hold:{evt:42}:A-12",
          "user:{evt:42}:u7"]:
    print(k, hash_slot(k))     # 11058, 2515, 6586, 6586
```

9.12 Lua scripts and atomicity

Because one thread executes commands, a Lua script sent with `EVAL` (or a stored Redis Function, `FCALL`, since Redis 7) runs **atomically**: no other command interleaves between its reads and writes. That turns a check-then-act race into one step. `MULTI`/`EXEC` also runs a batch atomically, but it cannot branch on values read inside it; `WATCH` adds optimistic concurrency at the cost of retries (Chapter 3).

Encore's seat hold is the canonical case: "if the seat is free and the fan holds fewer than six seats, hold it for 10 minutes".

```
import fakeredis

r = fakeredis.FakeRedis(decode_responses=True)

HOLD = """
-- KEYS[1] = seat hold key, KEYS[2] = the user's set of holds
-- ARGV[1] = user id, ARGV[2] = hold seconds, ARGV[3] = max holds
if redis.call('EXISTS', KEYS[1]) == 1 then
  return {0, 'taken'}
end
if redis.call('SCARD', KEYS[2]) >= tonumber(ARGV[3]) then
  return {0, 'limit'}
end
redis.call('SET', KEYS[1], ARGV[1], 'EX', ARGV[2])
redis.call('SADD', KEYS[2], KEYS[1])
redis.call('EXPIRE', KEYS[2], ARGV[2])
return {1, 'held'}
"""
hold = r.register_script(HOLD)

ev = "{evt:42}"                     # hash tag: both keys in one slot
print(hold(keys=[f"hold:{ev}:A-12", f"user:{ev}:u7"], args=["u7", 600, 6]))
print(hold(keys=[f"hold:{ev}:A-12", f"user:{ev}:u9"], args=["u9", 600, 6]))
```

The first fan gets `[1, 'held']` and the second `[0, 'taken']`. Rules for scripts: pass every key in `KEYS` (the cluster routes by them, and they must share a slot), keep scripts short because they block the server like any command (a script running past `busy-reply-threshold`, 5 s by default, makes the server answer `BUSY` to others), and load them once with `SCRIPT LOAD` so clients call `EVALSHA` with the hash. The same pattern implements atomic rate limiters (Chapter 11) and safe lock release (delete only if the stored token is mine). A Redis lock is fine for efficiency, such as avoiding duplicate rebuilds; for correctness it needs fencing tokens, as Chapter 18 explains.

> **Design question.** Should seat holds live only in Redis?

> At 30,000 hold attempts/s in the first minute, the Lua script above is the right fast path: atomic, about one round trip, with the 10-minute TTL doing the release for free. But asynchronous replication means a failover can forget a few seconds of holds, and a forgotten hold can lead to a double sale. So Redis is the gate, not the record: the inventory service writes each successful hold to its database (a conditional `UPDATE seats SET state = 'held' WHERE id = ? AND state = 'free'`, or through a queue), and the sale itself is committed only in the database. If Redis loses a hold, the database rejects the second buyer at payment. Shard by event with the hash tag `{evt:42}` so one event's keys share a slot; the stadium show then concentrates on one shard, which 30,000 short scripts a second can still fit on a fast node, with the waiting room capping arrivals. See also *50 System Designs* #25 (Ticket Booking) and #34 (E-Commerce Checkout and Inventory).

9.13 Memcached, Valkey and managed services

Memcached is the simpler sibling: a multithreaded key-value cache of opaque values (1 MB maximum item by default) with a slab allocator and per-slab LRU, no persistence, no replication and no server-side clustering; clients shard keys with consistent hashing. Choose it for a plain, very large, multi-core object cache. Choose Redis when you need data structures, TTLs on rich types, atomic scripts, replication or persistence, which is most designs.

Valkey is the Linux Foundation fork created in March 2024, when Redis moved away from the BSD licence; it continues under BSD-3-Clause from Redis 7.2.4 and is command-compatible. Valkey 8 added multithreaded I/O, 8.1 a more memory-efficient hash table, and 9.0 hash-field expiration; 9.1 shipped in May 2026. Redis itself added the AGPLv3 as a licence option with Redis 8 in 2025. For a design interview the two are interchangeable; the cloud choice is usually about price.

Service	AWS	GCP	Azure
Redis-compatible	ElastiCache	Memorystore	Azure Managed Redis
Engines	Valkey, Redis OSS	Valkey, Redis	Redis Enterprise
Memcached	ElastiCache	deprecated	(none)

AWS adds three variants worth naming: ElastiCache Serverless (no node sizing), MemoryDB (Valkey- and Redis-compatible, durable, primary database) and DAX (a cache built into the DynamoDB API). On Azure, Azure Cache for Redis is being retired (Enterprise tiers in March 2027, Basic, Standard and Premium in September 2028) in favour of Azure Managed Redis, which runs Redis Enterprise and is clustered by default. Google lists Memorystore for Memcached as deprecated.

9.14 Caching LLM answers

Model calls are the slowest and most expensive reads in Encore: seconds of latency and a per-token bill. Three caches apply, in order of safety. **Exact-match caching** keys on a hash of model, prompt and parameters, and suits deterministic jobs such as classifying a support ticket. **Provider prompt caching** reuses the processed prefix of a long, stable prompt (the support agent's instructions and tool definitions) at a discounted input price, so put the stable part first. **Semantic caching** embeds the question and returns a stored answer when a previous question is close enough (cosine similarity above a threshold such as 0.9), using a vector index in Redis or a vector database. The savings follow the same equation as before, cost $= (1 - h) \times$ cost per call, but a semantic hit can be confidently wrong, so scope the cache (public policy answers only, never answers built from one fan's order data), give entries a TTL tied to the policy they quote, and tune the threshold on real paraphrases. Tool results the agent fetches (order status) are ordinary cache-aside data with short TTLs; *Designing Agentic Systems* covers the agent side.

9.15 In Encore

Data	Where	Pattern	TTL	Invalidation
Event images, pages	CDN, browser	HTTP caching	60 s to 1 year	purge, versioned URLs

Data	Where	Pattern	TTL	Invalidation
Event and venue details	Redis	cache-aside	60 s + jitter	delete on edit, CDC
Seat map	pod memory, Redis bitmap	refresh-ahead	1 s	inventory writes bitmap
Seat holds	Redis (Lua) + DB	write-through	600 s	expiry, sale
Sessions	Redis hash	primary copy	30 min idle	logout deletes
Waiting-room line	Redis sorted set	primary copy	sale window	admission pops

The shape generalises: public, slow-changing data goes to the CDN; shared objects go to Redis with cache-aside and short jittered TTLs; the hottest few keys are copied into every pod; and anything that decides money or seats is checked against the database. Encore runs one Redis Cluster per region with a replica per shard in a second availability zone; the stadium show's keys share the `{evt:42}` tag, and the waiting room (Chapter 11) keeps arrivals within what that one shard can serve.

9.16 Numbers to remember

Figure	Value
Redis round trip in one zone	about 0.5 ms
Simple ops per Redis instance	order of 100,000/s; 1,000,000+/s pipelined
Effective latency	$L_c + (1 - h)L_d$
Database load	$(1 - h)R$: each nine divides it by 10
Concurrent rebuilds on expiry	$R_{key} \times t_{rebuild}$
Redis Cluster slots	16,384, CRC16 mod 16384
HyperLogLog	12 KB, 0.81% error
Bitmap of 50,000 seats	6,250 bytes

9.17 Pitfalls

- Caching without a TTL -> every bug in invalidation becomes permanent; always set a TTL as the safety net.
- Updating the cache on write -> races install stale values; delete after commit instead.
- Identical TTLs on keys filled together -> synchronized expiry; add jitter.
- One hot key on one shard -> a saturated node in a large cluster; copy it into pods or replicate the key.
- Leaving `noeviction` on a cache -> writes fail when memory fills; set `allkeys-lru` or `allkeys-lfu`.
- `KEYS *` or huge `HGETALL` in production -> the single thread stalls; use `SCAN` and bounded structures.
- Treating Redis as the record of sales -> asynchronous replication loses writes on failover; commit money and seats in the database.
- Caching per-user answers in a shared semantic cache -> data leaks between users; scope keys by tenant and user.

9.18 Remember

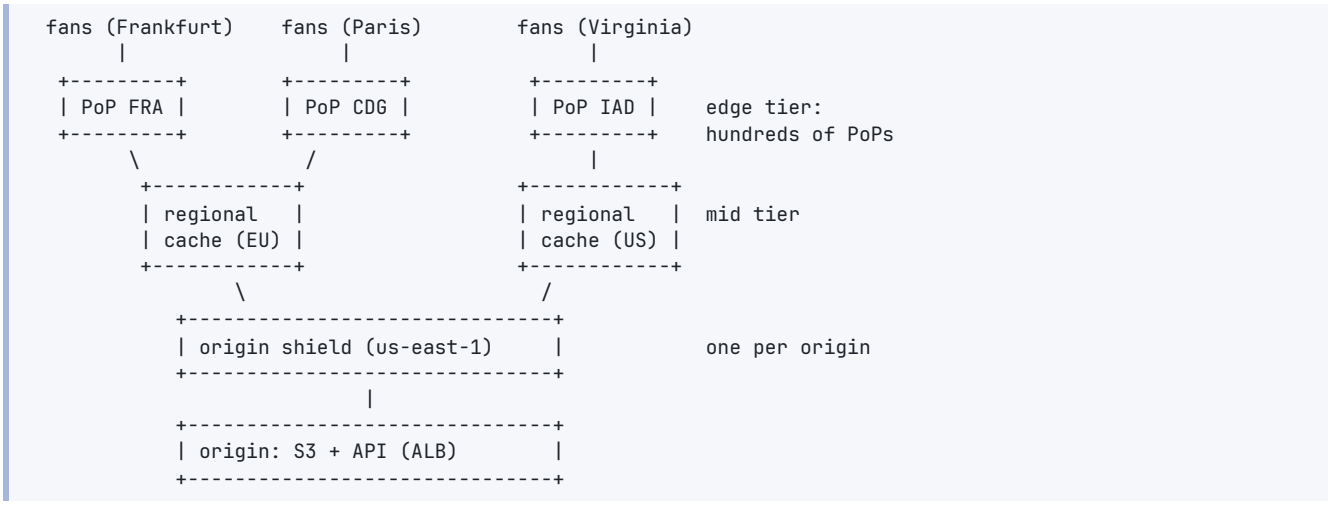
- A cache is a replica without a protocol: choose the staleness you can accept, enforce it with a TTL, and tighten it with delete-on-write or CDC.
- Cache-aside with jittered TTLs is the default; write-through for read-after-write, write-behind only for data you can lose.
- Size by miss rate: $(1 - h)R$ must fit the database, and hot keys need copies, not bigger shards.
- Stampedes are $R \times t$ rebuilds at once: coalesce, lock, refresh early with XFetch, and jitter.
- Redis is fast because one thread runs in-memory commands from an event loop, so one slow command stalls all; Redis Cluster maps keys to 16,384 CRC16 slots, and hash tags co-locate keys for atomic Lua scripts.

10 CDNs and the edge

At 09:58 on flash-sale morning a million fans open the same event page, and the origin servers in Virginia should barely notice. A content delivery network (CDN) makes that possible: it copies the bytes that many people ask for into hundreds of small data centers close to them, answers from there, and forwards only what it cannot answer to the origin. This chapter asks one question: how much work can Encore push to the edge, and what must stay home? The answer covers how a user reaches the nearest point of presence, how the cache decides what is "the same" response and how long to keep it, how tiers of caches shield the origin, how to protect private files, and what code is worth running at the edge itself.

10.1 The picture

A CDN is a chain of neighbourhood newsstands supplied by one printing press. Each newsstand (a **point of presence**, or PoP) keeps the papers its neighbourhood buys; when a customer asks for one it has not stocked, the newsstand phones a regional depot, and only the depot calls the press. The press prints each edition once per depot, not once per reader.



Three properties make the picture work. Requests are **routed** to a nearby PoP, so the round trip is 10 ms instead of 90 ms across the Atlantic. Responses are **cached** under a key, so one origin fetch serves thousands of readers. And the edge is a **programmable proxy**, so it can also terminate TLS, reject attackers and run small functions before a request ever leaves the city it came from.

10.2 Routing a user to a PoP

Two techniques steer a user to a PoP. **DNS steering** is a receptionist who looks at your address and hands you the phone number of the nearest branch. **Anycast** is a single emergency number that every branch answers; the telephone network connects you to the nearest one. DNS steering takes four steps:

1. Encore's DNS maps `www.encore.example` with a CNAME (or a Route 53 alias) to the CDN's name, such as `d111abcdef8.cloudfront.net`.
2. The user's recursive resolver asks the CDN's authoritative DNS for that name.
3. The CDN looks up the resolver's network (or the client subnet, if the resolver sends the EDNS Client Subnet option of RFC 7871) in its latency and health maps and answers with the addresses of the best PoP, with a short TTL.
4. The browser connects to that PoP. If the PoP is overloaded or down, the next DNS answer points elsewhere.

Anycast, step by step. Every PoP announces the same IP prefix over BGP. Internet routers forward packets to the announcement with the best route, which is usually, but not always, the nearest PoP: BGP picks by path and policy, not by measured latency. A PoP that fails or overloads withdraws its announcement and traffic flows to the next one within seconds, with no DNS cache to wait out.

What a Frankfurt resolver sees, step 1 then step 3 (addresses illustrative):

```
www.encore.example.          300  IN  CNAME  d111abcdef8.cloudfront.net.
d111abcdef8.cloudfront.net.   60  IN  A      198.51.100.10   ; PoP FRA
d111abcdef8.cloudfront.net.   60  IN  A      198.51.100.11   ; PoP FRA
```

Behind the short TTL the CDN's DNS takes the healthy PoP with the lowest measured latency from that resolver's network; if FRA fails, the next answer, within 60 s, names AMS.

Where it runs. Amazon CloudFront, with more than 750 PoPs and 15 regional edge caches, steers with DNS to the PoP "that can best serve the request, typically the nearest CloudFront POP in terms of latency"; for allow-listing or apex domains it also offers Anycast static IP lists (21 addresses, or 3 for apex records). Cloudflare and Google Cloud CDN (behind the global external Application Load Balancer's single anycast IP) use anycast. Azure Front Door Standard and Premium steer with DNS to unicast PoP addresses; the classic tier, which used anycast, retires on 31 March 2027. Chapter 4 explains BGP and anycast at the packet level.

Trade-offs. DNS steering sees the resolver, not the user, so a fan in Paris using a resolver in Frankfurt lands one city away unless the resolver sends a client subnet; it reacts at the speed of the TTL. Anycast reacts fast and spreads floods across every PoP, but BGP can send a user to a distant PoP, and a route change mid-connection can break long TCP sessions. Large CDNs mix both.

10.3 Cache keys and Cache-Control

The **cache key** is the label on the newsstand's shelf. Two requests with the same label get the same copy. Put too little on the label and a German fan receives the French page; put too much on it (every fan's session cookie) and the shelf holds four million copies of one page, each read once.

Mechanism. For each request the PoP builds a key from the parts the configuration names, by default the host and the path, optionally selected query parameters, headers and cookies, and looks it up. On a hit it answers from memory or SSD. On a miss it forwards the request (with whatever the origin needs, which may be more than the key) and stores the response under the key, for as long as the response's headers allow. The rule for designing a key is short: **include every input that changes the bytes, and nothing else.** Tracking parameters (`utm_source`, `fbclid`), cookies and `User-Agent` almost never belong in it.

The origin controls storage with `Cache-Control` (RFC 9111):

Directive	Meaning
`public`, `private`	Shared caches may store it, or only the browser may
`no-store`	Nobody stores it
`no-cache`	Store it, but revalidate before every use
`max-age=N`	Fresh for N seconds (browsers and CDNs)
`s-maxage=N`	Overrides `max-age` for shared caches only
`immutable`	Never revalidate while fresh (RFC 8246)
`stale-while-revalidate=N`	Serve stale up to N s while refetching (RFC 5861)
`stale-if-error=N`	Serve stale up to N s if the origin fails

A stale object is not thrown away: the cache sends a **conditional request** with `If-None-Match: "<etag>"`, and an unchanged object comes back as a body-less `304 Not Modified`. Revalidation costs a round trip to the origin but almost no bandwidth.

Numbers. The number of cached variants of one URL is the product of the distinct values of every key component, $\prod_i |V_i|$. An Encore event page keyed on three languages and three encodings (Brotli, gzip, none) has $3 \times 3 = 9$ variants. Let `utm_source` with 50 campaign values into the key and it becomes 450 variants, each fetched from the origin separately and each with a fiftieth of the traffic; let the session cookie in and it is one variant per fan. Hit ratio falls with every factor.

Normalizing the key is a few lines, and a CloudFront Function can run the same logic on the viewer-request event:

```
from urllib.parse import parse_qsl, urlencode, urlsplit

KEEP = {"event", "lang", "w"}       # query parameters that change the bytes


def cache_key(url: str, headers: dict[str, str]) -> str:
    parts = urlsplit(url)
    query = sorted((k, v) for k, v in parse_qsl(parts.query) if k in KEEP)
    enc = headers.get("accept-encoding", "")
    encoding = "br" if "br" in enc else "gzip" if "gzip" in enc else "id"
    return f"{parts.hostname.lower()}{parts.path}?{urlencode(query)}|{encoding}"


a = cache_key("https://WWW.encore.example/e/42?utm_source=x&lang=de",
              {"accept-encoding": "gzip, br"})
b = cache_key("https://www.encore.example/e/42?lang=de&fbclid=abc",
              {"accept-encoding": "br"})
print(a, a == b)    # www.encore.example/e/42?lang=de|br True: one object
```

On CloudFront the key is declared in a **cache policy**, separate from the origin request policy that says what is forwarded:

```
EventPageCachePolicy:
  Type: AWS::CloudFront::CachePolicy
  Properties:
    CachePolicyConfig:
      Name: encore-event-pages
      MinTTL: 0              # never override the origin's no-store
      DefaultTTL: 60         # used when the origin sends no max-age
      MaxTTL: 31536000       # one year, for versioned assets
      ParametersInCacheKeyAndForwardedToOrigin:
        EnableAcceptEncodingGzip: true
        EnableAcceptEncodingBrotli: true
        HeadersConfig:
          HeaderBehavior: none
        CookiesConfig:
          CookieBehavior: none   # cookies never split the cache
        QueryStringsConfig:
          QueryStringBehavior: whitelist
          QueryStrings: [event, lang, w]
```

Where it runs. CloudFront clamps the origin's `max-age` or `s-maxage` between the policy's minimum and maximum TTL and uses the default TTL when the origin is silent; a minimum TTL above zero makes CloudFront cache even responses marked `no-store` or `private`. It ignores `Cache-Control` sent by viewers, so a fan cannot force an origin fetch. Fastly, Akamai and Cloudflare offer the same controls under different names (VCL, property rules, cache rules), and all honour `Vary`, the origin's way of saying which request headers belong in the key.

Trade-offs. A narrow key raises the hit ratio and risks serving one variant to the wrong audience; a wide key is safe and slow. When in doubt, move personalization out of the cached response (a static page plus a small uncached API call for "Hi, Ana") rather than into the key.

10.4 Freshness, purge and versioned URLs

There are three ways to stop a newsstand selling yesterday's paper: print a sell-by time on it (a **TTL**), send a recall order (a **purge**), or publish a new edition under a new name (a **versioned URL**). Good designs use all three, each for a different kind of content.

TTL with stale-while-revalidate. With `max-age=60, stale-while-revalidate=30`, a cached page is fresh for 60 s. A request in the next 30 s gets the stale copy at once while the PoP refetches in the background; after that window, the next request waits for the origin.

```
age:  0s                60s                90s
      |---- fresh ------|--- stale, served --|--- must refetch ---
      hit, no origin     hit + one async      miss: request waits
                         revalidation
```

Without a purge, the worst-case age a fan can see at the CDN is the sum of the two windows, TTL+SWR , or 90 s for this page, a bound worth writing into the requirements; the browser adds its own `max-age`, less the `Age` header the CDN sends.

Purge (invalidation). A purge tells every PoP to drop or mark stale the objects matching a path, a wildcard or a **tag**. Tags (also called surrogate keys) are labels the origin attaches to responses, so one call can drop "everything that shows event 42": the event page, its search snippets and its JSON. Fastly calls them `Surrogate-Key`, Cloudflare `Cache-Tag`, and CloudFront added tag invalidation through a configurable response header:

```
aws cloudfront create-invalidation --distribution-id E2EXAMPLE \
  --paths "#event:42" "/e/42"
```

Purges take seconds to minutes to reach every PoP, and they cost: CloudFront's first 1,000 paths or tags a month are free, then each is billed, and the default quota is 150 paths or tags per second and one wildcard invalidation per second. A purge after every write does not scale; a purge after a rare, important edit does.

Versioned URLs. Name each static file after a hash of its content and it never changes, so it can be cached for a year and never purged:

```
import hashlib
from pathlib import Path


def versioned_name(path: Path) -> str:
    digest = hashlib.sha256(path.read_bytes()).hexdigest()[:10]
    return f"{path.stem}.{digest}{path.suffix}"


print(versioned_name(Path("assets/hero.jpg")))  # hero.<10 hex chars>.jpg
```

A deploy uploads the new assets first, then the HTML that references them; old HTML keeps working because the old files are still there.

Where it runs. Every CDN supports TTLs, conditional revalidation and path purges. CloudFront honours both `stale-while-revalidate` and `stale-if-error`, capping the stale window at the policy's maximum TTL; Google Cloud CDN honours `stale-while-revalidate` but not `stale-if-error`, and instead serves stale content on origin errors for a configurable `serveWhileStale` window (one day by default). Vite emits content-hashed file names by default, and webpack does with `[contenthash]` in the output name.

Trade-offs. TTLs are free and bound staleness, but the bound is the TTL. Purges give freshness on demand, but they are asynchronous, rate-limited and easy to forget for one path. Versioned URLs are free and exact, but only for content referenced by something else; the HTML at the top of the tree still needs a TTL or a purge.

> **Design question.** An organizer moves a show from 20:00 to 21:00. How do fans see the new time within a minute, without purging the whole site?

> Requirements: the event page is cached; the change is rare but must appear fast; images are unaffected. The event page HTML carries `s-maxage=60, stale-while-revalidate=30`, so with no action the worst case is 90 s. The catalog service writes the change and an outbox row in one transaction (Chapter 14); a worker reads the outbox and issues one tag invalidation, `#event:42`, which drops the page, its JSON and its search snippet at every PoP within seconds. Images are content-hashed and stay cached. The trade-off: one more moving part (the purge worker), and a purge that fails silently leaves a 90 s bound, which is acceptable. See also *50 System Designs* #15 (Content Delivery Network).

10.5 Origin shield and tiered caching

Hundreds of newsstands that each phone the press on a miss would keep the press busy all morning. Depots fix that: newsstands call a depot, and only the depot calls the press. In a CDN the depots are **tiers**: a miss at an edge PoP goes to a larger regional cache, and a miss there goes to one designated **origin shield** in front of the origin.

Mechanism.

1. The edge PoP misses and asks its regional (mid-tier) cache, which holds far more objects than any single PoP and keeps the long tail warm.
2. The regional cache misses and asks the shield, a single cache location chosen close to the origin.
3. The shield **collapses** concurrent misses for the same key: the first request goes to the origin, and the others wait for that response instead of each going to the origin. A thousand simultaneous misses become one origin fetch.
4. The response flows back down, filling each tier on the way.

Numbers. Under Poisson arrivals at rate λ for one object cached with TTL T, each cycle is one miss followed by an average of λT hits until expiry, so

$$h = \frac{\lambda T}{1+\lambda T}, \qquad \text{misses per second per cache} = \frac{\lambda}{1+\lambda T} \approx \frac{1}{T} \text{ when } \lambda T \gg 1$$

Two consequences follow. A popular object costs each cache about one origin fetch per TTL no matter how hot it is, so without a shield N PoPs cost the origin about N/T fetches a second and with a shield about $1/T$. And the long tail is where tiers pay most: an Encore event page read 0.01 times a second at each of 40 PoPs, with a 60 s TTL, hits only 0.6/1.6 = 37.5 percent at the edge; the shield sees the combined 0.4 requests a second and hits 24/25 = 96 percent. With 300,000 events a year, most pages are long tail. A short simulation confirms the formula:

```
import random


def ttl_hit_ratio(rate: float, ttl: float, n: int = 200_000) -> float:
    rng, t, expires, hits = random.Random(7), 0.0, -1.0, 0
    for _ in range(n):
        t += rng.expovariate(rate)      # Poisson arrivals
        hits += t < expires
        if t >= expires:
            expires = t + ttl           # miss: fetch and cache for ttl
    return hits / n


for rate, ttl in [(0.01, 60), (0.4, 60), (50, 1)]:
    print(rate, ttl, round(ttl_hit_ratio(rate, ttl), 3),
          round(rate * ttl / (1 + rate * ttl), 3))  # 0.376 vs 0.375, ...
```

Where it runs. CloudFront routes edge misses through regional edge caches and, if enabled per origin, through **Origin Shield** in one AWS Region; it consolidates concurrent requests for the same object "resulting in as few as one request going to your origin". Requests that are dynamic, or use `POST`, `PUT`, `PATCH`, `OPTIONS` or `DELETE`, skip the regional tier and go straight to the origin. Cloudflare calls the feature Tiered Cache, Fastly calls it shielding, Akamai Tiered Distribution.

Trade-offs. Each tier adds one hop to a miss, and Origin Shield is billed per request that passes through it. The shield is a concentration point: the CDN fails over to a secondary shield location, but your origin should still survive a burst of shield misses. For uncacheable traffic a shield adds latency and cost and saves nothing.

10.6 Offload and hit ratio

Two ratios measure a CDN. The **request hit ratio** h is the share of requests answered from cache; the **byte hit ratio**, or **offload**, is the share of bytes. They differ because big objects (images, video segments) cache well and small ones (API calls) often do not: a site can offload 99 percent of its bytes while hitting only 70 percent of its requests. With edge and shield hit ratios h_e and h_s, request rate R, a cache-hit time L_e and an extra fill time L_f on a miss:

$$R_{\text{origin}} = R(1 - h_e)(1 - h_s), \qquad L_{\text{eff}} = L_e + (1 - h)\,L_f$$

At 200,000 requests a second with $h_e = 0.9$ and $h_s = 0.8$, the origin sees $200,000 \times 0.1 \times 0.2 = 4,000$ requests a second instead of 20,000 without the shield: 98 percent offload. With a 10 ms edge and a 90 ms transatlantic fill, a 90 percent hit ratio gives 19 ms on average and 99 percent gives 10.9 ms, so the last few points of hit ratio matter more for the origin's survival than for the average user. CloudFront reports each response in the `x-cache` header (`Hit from cloudfront`, `Miss from cloudfront`) and, with additional metrics enabled, a `CacheHitRate` metric in CloudWatch (Cloud Monitoring, Azure Monitor).

10.7 Static and dynamic acceleration

Static acceleration is everything above: cacheable bytes served from nearby. **Dynamic acceleration** helps responses that can never be cached, such as a hold or a checkout, because the CDN is still a better path to the origin. The fan's TCP and TLS handshakes end at the PoP 10 ms away (split TCP), and the PoP forwards the request over a connection to the origin that is already open, warm and running on the provider's backbone. Counting one round trip for TCP, one for TLS 1.3 and one for the request itself:

$$T_{\text{direct}} = 3\,RTT_o, \qquad T_{\text{edge}} = 3\,RTT_e + RTT_{eo}$$

From Frankfurt to Virginia, $3 \times 90 = 270$ ms direct against $3 \times 10 + 90 = 120$ ms through the edge; HTTP/3 merges the transport and TLS handshakes and saves one more short round trip. Between the two sits **micro-caching**: a dynamic but shared response, such as the seat map's availability, cached for one second, which turns 200,000 identical reads into about one origin fetch per second per cache.

Where it runs. CloudFront forwards all HTTP methods and WebSockets to origins; AWS Global Accelerator (Google's Premium Tier network behind its global load balancers, Azure cross-region Load Balancer) gives anycast entry and backbone transit at layer 4 without caching. Cloudflare sells backbone routing as Argo Smart Routing. **Trade-off:** the CDN terminates TLS, so it sees plaintext; keep card data off this path by sending card entry straight to the payment provider's hosted fields.

10.8 Signed URLs for private content

A ticket PDF must load fast from the edge yet open only for the fan who bought it. A **signed URL** is a cloakroom ticket: the origin, which knows who the fan is, issues a URL that carries an expiry and a signature over it; the edge, which knows nothing about fans, checks the signature and the clock and serves or refuses. No database call happens at the edge. For many files at once, such as the segments of a video, the same signed policy travels in a **signed cookie** instead.

This is Google Cloud CDN's format (HMAC-SHA1 over the URL, base64url-encoded), signed by the orders service and verified at the edge:

```python
import base64
import hashlib
import hmac
import time

KEY_NAME, KEY = "tickets-2026", bytes(range(16))  # 16-byte secret


def sign(url: str, ttl_s: int, now: int) -> str:
    sep = "&" if "?" in url else "?"
    to_sign = f"{url}{sep}Expires={now + ttl_s}&KeyName={KEY_NAME}"
    mac = hmac.new(KEY, to_sign.encode(), hashlib.sha1).digest()
    return f"{to_sign}&Signature={base64.urlsafe_b64encode(mac).decode()}"


def verify(signed: str, now: int) -> bool:          # what the edge does
    to_sign, _, sig = signed.rpartition("&Signature=")
    mac = hmac.new(KEY, to_sign.encode(), hashlib.sha1).digest()
    expires = int(to_sign.split("Expires=")[1].split("&")[0])
    ok = hmac.compare_digest(base64.urlsafe_b64encode(mac).decode(), sig)
    return ok and now < expires


now = int(time.time())
url = sign("https://cdn.encore.example/t/ord_81f2.pdf", 900, now)
print(verify(url, now))                             # True
print(verify(url.replace("81f2", "81f3"), now))     # False: tampered
print(verify(url, now + 901))                       # False: expired
```

Where it runs. CloudFront signs with a public-key pair (RSA 2048 or ECDSA P-256) registered in a trusted key group; a canned-policy URL carries `Expires`, `Signature` and `Key-Pair-Id`, and a custom policy adds start times and IP ranges. The S3 bucket stays private behind Origin Access Control, so the only way in is through the signed edge. Cloudflare, Akamai and Fastly offer token authentication in the same shape. **Trade-offs:** a signed URL is a bearer token until it expires, so a forwarded link works for anyone; keep expiries short (Encore uses 15 minutes for the download link and re-signs on each app open), and never sign with a long-lived key embedded in a client.

10.9 Video delivery

Video is the CDN's heaviest customer and its most cache-friendly one. An encoder cuts each video into **segments** of a few seconds at several bitrates (the **ladder**, say 0.8, 2.5 and 5 Mbit/s) and lists them in a **manifest**: an HLS `.m3u8` playlist or a DASH `.mpd`; CMAF lets both formats share one set of fragmented MP4 segments. The player downloads the manifest, then segment after segment, choosing the bitrate for each from its measured throughput (adaptive bitrate). Segments never change, so they carry year-long TTLs; a live manifest changes with every new segment, so its TTL is shorter than a segment. Egress, concurrent viewers times bitrate, is the multiplication that sizes everything. If Encore streamed a concert live to 100,000 fans at 5 Mbit/s, that is 500 Gbit/s, served almost entirely from edge caches and well above CloudFront's default per-distribution quota of 150 Gbit/s, so it needs a quota increase weeks ahead. Origin Shield matters most here, because just-in-time packagers such as AWS Elemental MediaPackage are expensive per request; Google offers Media CDN for this kind of delivery. *50 System Designs* #8 (Video Platform), #9 (Streaming Service) and #32 (Live Streaming) go deeper.

10.10 Edge compute

The newsstand clerk can do more than hand over papers: check a ticket, point you to another counter, stamp your hand. **Edge compute** runs small functions inside the PoP on each request or response, before the cache lookup or after the origin answers. The two shapes are a fast clerk with no phone (tiny sandboxed functions with no network access, run on every request) and a slower clerk with a phone (full runtimes that can call other services).

	CloudFront Functions	Lambda@Edge	Cloudflare Workers
Runtime	JavaScript (ES 5.1 plus newer features)	Node.js, Python	JavaScript, Wasm (V8 isolates)
Events	viewer request, response	viewer and origin, both ways	every request
Time	sub-millisecond	up to 30 s	CPU 10 ms free; 30 s default paid
Memory, code	2 MB, 10 KB	128 MB viewer, 10 GB origin; 50 MB	128 MB, 64 MiB
Network	no	yes	yes
Scale	millions of requests/s	10,000 requests/s per Region	every PoP

What belongs at the edge is logic that needs only the request and a small, slowly changing configuration: redirects and URL rewrites, cache-key normalization, security headers, geo and language routing, **A/B bucketing** (hash a stable cookie into 100 buckets and rewrite the path, so the variant is cacheable), **authentication checks** that verify a signature rather than look up a session, and the **waiting-room token check**. What does not belong: anything that needs the authoritative seat inventory, long work, or secrets beyond a verification key.

The waiting-room service, which admits fans at the rate the seat-inventory service can absorb (Chapter 11), hands each admitted fan a signed, short-lived token. The edge verifies it on every hold request without a network call and sends everyone else back to the queue. The logic, shown in Python, ports line for line to CloudFront Functions, whose runtime 2.0 offers `crypto.createHmac('sha256', key)` and reads the key from CloudFront KeyValueStore:

```
import base64
import hmac
import json

KEY = b"rotate-me-from-key-value-store"


def b64(data: bytes) -> str:
    return base64.urlsafe_b64encode(data).rstrip(b"=").decode()


def issue(fan: str, event: int, exp: int) -> str:     # waiting-room service
    body = b64(json.dumps({"fan": fan, "ev": event, "exp": exp}).encode())
    return body + "." + b64(hmac.new(KEY, body.encode(), "sha256").digest())


def viewer_request(path: str, token: str, now: int) -> tuple[int, str]:
    if not (path.startswith("/api/events/") and path.endswith("/holds")):
        return 200, "pass"                            # not protected
    event = int(path.split("/")[3])
    body, _, sig = token.partition(".")
    good = b64(hmac.new(KEY, body.encode(), "sha256").digest())
    if sig and hmac.compare_digest(sig, good):
        claims = json.loads(base64.urlsafe_b64decode(body + "=="))
        if claims["ev"] == event and now < claims["exp"]:
            return 200, "pass"                        # forward to origin
    return 302, f"/waiting-room/{event}"              # back of the queue


t = issue("fan_77", 42, exp=1_000_600)
for path, tok, now in [("/api/events/42/holds", t, 1_000_000),     # pass
                       ("/api/events/42/holds", "", 1_000_000),    # 302
                       ("/api/events/43/holds", t, 1_000_000),     # 302
                       ("/api/events/42/holds", t, 1_000_601)]:    # 302
    print(viewer_request(path, tok, now))
```

Where it runs. Besides the three above: Fastly Compute (Wasm), Akamai EdgeWorkers, Google's Service Extensions plugins (Wasm on Application Load Balancers and Media CDN), and Azure Front Door's rules engine for declarative rewrites. **Trade-offs:** edge code is deployed to hundreds of locations at once, so a bug is global within minutes; it sees

only eventually consistent configuration; and its logs are scattered. Keep it small, stateless and covered by tests.

> **Design question.** Why check the waiting-room token in CloudFront Functions rather than Lambda@Edge or the API gateway?
>
> In the first minute 30,000 hold attempts a second arrive, most from fans who are not yet admitted, plus bots. A check at the API gateway in Virginia would let every request cross the ocean and occupy origin capacity before being refused. Lambda@Edge could do it at the edge, but its default of 10,000 requests a second per Region is below the peak and each invocation costs milliseconds. A CloudFront Function verifies an HMAC in under a millisecond at millions of requests a second, so an unadmitted request costs the origin nothing. The trade-off: the function cannot ask whether a token was revoked, so tokens must be short-lived (ten minutes, matching the hold) and bound to one event and one fan. See also *50 System Designs* #25 (Ticket Booking).

10.11 DDoS absorption

A flood that hits one data center sinks it; the same flood spread over hundreds of PoPs is a ripple at each. The edge absorbs attacks in three ways. **Volume:** layer 3 and 4 floods (SYN floods, UDP reflection) land on the CDN's multi-terabit edge, spread by anycast or DNS, and are filtered before they reach any origin. **Caching:** a flood of requests for cacheable pages is served from cache; attackers answer with cache-busting (a random query string per request), which normalized, allow-listed cache keys defeat. **Filtering:** a web application firewall at the edge applies rate-based rules per IP or token, bot detection and challenges to layer 7 floods. None of it helps if the origin is reachable directly, so **cloak the origin**: the S3 bucket accepts only CloudFront through Origin Access Control, and the load balancer only the CloudFront origin-facing managed prefix list (or a VPC origin) plus a secret header.

Where it runs. AWS Shield Standard protects every AWS customer automatically at no additional charge, with the most complete layer 3 and 4 coverage for CloudFront, Route 53 and Global Accelerator; Shield Advanced adds a response team, layer 7 mitigation and cost protection; AWS WAF provides the rules (Google Cloud Armor; Azure DDoS Protection and the Front Door WAF). Rate limiting itself is the subject of Chapter 11.

10.12 In Encore

Encore splits its traffic by how it may be cached, and each class gets its own header from the origin:

```
# versioned image, CSS, JS: cache for a year, never revalidate
Cache-Control: public, max-age=31536000, immutable

# event page HTML: short TTL at the CDN, shorter in the browser
Cache-Control: public, max-age=10, s-maxage=60, stale-while-revalidate=30

# seat-map availability JSON: one-second micro-cache
Cache-Control: public, max-age=0, s-maxage=1, stale-while-revalidate=1

# order page, account, holds: never in a shared cache
Cache-Control: private, no-store
```

Agents and external systems follow the same split. Fans' AI shopping assistants arrive as bots: they get the cached event page like anyone else, and the hold API demands the same waiting-room token, so an agent cannot jump the queue. Responses from the support agent, streamed over Server-Sent Events, are `no-store` and pass through the edge unbuffered. Webhooks from the payment provider enter on their own uncached path, exempt from bot challenges but verified by signature at the origin (Chapter 24).

> **Design question.** How does Encore serve the event page and seat map to 1,000,000 fans arriving between 09:55 and 10:05 without the origin noticing?
>
> Requirements: 200,000 seat-map reads a second; first page loads of $10^6 \times 200$ KB = 200 GB in ten minutes, an average of 2.7 Gbit/s before refreshes. Images are content-hashed with year-long TTLs and were warmed by weeks of browsing. The HTML has a 60 s TTL and a tag purge. Seat availability is a 50,000-bit bitmap (6.25 KB) with `s-`

> `maxage=1` behind Origin Shield, so 200,000 reads a second (10 Gbit/s) cost the origin about one fetch a second. The waiting-room token is checked in a CloudFront Function. The edge has the bandwidth, but 200,000 requests a second is close to CloudFront's default quota of 250,000 per distribution, so Encore raises the quota and load-tests through the CDN before the sale. The trade-off: the seat map can be a second or two stale, which costs a fan one rejected hold, never an oversold seat. See also *50 System Designs* #25 (Ticket Booking).

10.13 Numbers to remember

Figure	Value
RTT to a PoP vs across the Atlantic	about 10 ms vs 90 ms
TTL cache hit ratio	$\lambda T/(1+\lambda T)$
Origin fetches for a hot object	about N/T; $1/T$ with a shield
Origin requests with two tiers	$R(1-h_e)(1-h_s)$
Flash-sale first page loads	200 GB in 10 min, 2.7 Gbit/s
Seat-map availability at peak	6.25 KB x 200,000/s = 10 Gbit/s
CloudFront default per distribution	250,000 requests/s, 150 Gbit/s
CloudFront Functions	sub-ms, 2 MB memory, 10 KB code
Lambda@Edge	10,000 requests/s per Region, 30 s
CloudFront invalidations	1,000 free paths a month; 150/s

10.14 Pitfalls

- Caching a personalized response (a page with a fan's name, a `Set-Cookie`) for everyone -> mark it `private, no-store` and fetch personal data with a separate uncached call.
- A minimum TTL above zero, which makes CloudFront cache `no-store` responses -> keep `MinTTL: 0` and let the origin decide.
- Tracking parameters, cookies or random query strings in the cache key -> allow-list the inputs that change the bytes.
- Purging on every deploy -> content-hash static file names; purge only HTML and rare edits, by tag.
- An origin reachable around the CDN -> Origin Access Control, the origin-facing prefix list and a secret header.
- Discovering CDN quotas during the sale -> raise request and bandwidth quotas in advance and load-test through the edge.
- Long-lived signed URLs or tokens -> expire in minutes, bind to one fan and one resource.

10.15 Remember

- A CDN routes users to a nearby PoP (by DNS or anycast), caches under a key, and proxies everything else over warm connections.
- Design the cache key first: every input that changes the bytes, nothing else.
- Freshness comes from TTLs with `stale-while-revalidate`, purges by tag, and versioned URLs; use each where it fits.
- Tiers and an origin shield turn N/T origin fetches into $1/T$ and rescue the long tail.
- Run only signature checks, rewrites and routing at the edge; the authoritative state stays at the origin.
- For Encore, the edge carries the flash sale's reads and rejects unadmitted holds; the origin sees the few requests that must be exact.

11 Rate limiting, backpressure and resilience

At 10:00 on flash-sale day, a million fans press the same button within a few seconds, a few hundred bots press it thousands of times each, and the payment provider has a slow morning. Every service in Encore has a finite capacity, and past that capacity latency does not degrade gently: queues grow, timeouts fire, clients retry, and the retries become the outage. This chapter answers one question: how does a system say "not now" early, cheaply and fairly, at every layer, so that the work it does accept finishes? The picture is a stadium on match day: turnstiles that let people in at a fixed pace, stewards who close a gate when a stand is full, and a fuse box that cuts a circuit before the wiring melts.

11.1 The picture

Think of the stadium again. Turnstiles at the gates admit people at a steady pace and let a small group through at once (a **rate limiter**). Each turnstile serves one queue per ticket type, so a coach party cannot block everyone else (**fairness by key**). When a stand is full, stewards close its entrance and send people elsewhere instead of letting the crush grow (**load shedding**). The bar tells the cashier to stop selling drinks when the taps cannot keep up (**backpressure**). The floodlights sit on separate circuits with their own fuses, so a short in the scoreboard does not black out the pitch (**bulkheads** and **circuit breakers**). And on big days the club opens a queue in the car park and hands out numbered wristbands (**the waiting room**).

```
fans and bots
     |
     v
[edge: WAF, per-IP limit, waiting-room token]   -> 429 / queue page
     |
     v
[API gateway: per-user, per-key, per-tenant]    -> 429 + Retry-After
     |
     v
[service: concurrency limit, priority shedding] -> 503 + Retry-After
     |               |
     v               v
[seat inventory]  [breaker + bulkhead] -> payment provider (external)
```

Each layer rejects what the next one cannot afford, and each rejection costs less than the work it prevents. A request turned away at the edge costs microseconds; the same request turned away after it has taken a database connection costs milliseconds and a connection that someone else needed.

11.2 Why limit

Four reasons put a limit in front of a service:

- **Protection.** A server near saturation slows down for everyone. In the single-queue model of Chapter 2, mean time in the system is $W = S/(1 - \rho)$ for service time S and utilization ρ, so a service that answers in 10 ms when idle takes 20 ms at 50 percent load and 100 ms at 90 percent.
- **Fairness.** One client, tenant or script must not consume the capacity that thousands of others share.
- **Cost.** Some calls cost money per unit: SMS messages, payment-provider calls, LLM tokens. A limit is a spending cap that acts in real time.
- **Abuse.** Credential stuffing, scraping and seat-hoarding bots are rate problems before they are anything else (Chapter 22).

The tools in this chapter differ in who decides: a **rate limit** is policy per client (answer 429), **load shedding** is the server judging its own capacity (503), **backpressure** is a consumer whose queue is full (block or slow down), and **breakers and bulkheads** are a caller judging a dependency's health (fail fast, fall back).

A rate limit is not a business quota. “At most 8 tickets per fan per show” is a rule about the domain and lives in the orders service with a durable count; “at most 30 hold attempts per minute” is a rule about traffic and lives in a fast, approximate limiter.

11.3 Token bucket

Picture a jar of arcade tokens. An attendant drops in r tokens per second, and the jar holds at most b; extra tokens spill on the floor. Every request takes one token. A client that has been quiet finds a full jar and can spend b tokens at once; a client that hammers the button gets one token every $1/r$ seconds.

The mechanism needs no timer. Store two numbers per key, the token count and the time of the last update. On each request, add the tokens earned since then, cap at b, and admit the request if at least one token (or `cost` tokens) is left. Over any interval of length T the bucket admits at most a full jar plus the refill:

$$\text{tokens}(t) = \min(b,\ \text{tokens}(t_0) + r\,(t - t_0)) \qquad \text{admitted}(T) \le b + rT$$

Encore limits hold attempts per signed-in fan with $r = 0.5$ per second (30 per minute) and $b = 5$ (illustrative). A fan who clicks five seats in a second gets all five; a script that fires 30,000 holds in the first minute gets at most $5 + 0.5 \times 60 = 35$, and every other attempt bounces off the limiter without touching seat inventory.

```
import time


class TokenBucket:
    def __init__(self, rate: float, capacity: float):
        self.rate, self.capacity = rate, capacity
        self.tokens, self.stamp = capacity, time.monotonic()

    def allow(self, cost: float = 1, now: float | None = None) -> bool:
        now = time.monotonic() if now is None else now
        elapsed = max(0.0, now - self.stamp)
        self.tokens = min(self.capacity, self.tokens + elapsed * self.rate)
        self.stamp = now
        if self.tokens >= cost:
            self.tokens -= cost
            return True
        return False


bucket = TokenBucket(rate=0.5, capacity=5)    # 30 holds/min, burst 5
burst = [bucket.allow(now=0.0) for _ in range(7)]
print(burst)                                  # 5 True, then False
print(bucket.allow(now=2.0))                  # one token back after 2 s
```

Requests can cost more than one token: an export call might cost 10, and an LLM call costs its token count (see the last section). The token bucket is the most deployed limiter. Amazon API Gateway (GCP API Gateway or Apigee, Azure API Management) throttles with token buckets, by default 10,000 requests per second per account per Region with a bucket of 5,000 for bursts; Envoy’s local rate limit filter is a token bucket configured as `max_tokens`, `tokens_per_fill` and `fill_interval`; the Claude API uses token buckets for requests and tokens per minute; and AWS SDKs keep a token bucket of retry credits.

The trade-off is that bursts are allowed by design. Choose b to match the burst a real user makes (five seat clicks), not the burst the backend survives; protect the backend with a separate, shared limit.

11.4 Leaky bucket

Picture a bucket with a small hole in the bottom. Requests are poured in at any pace; they leak out of the hole at a constant rate r; if the bucket (capacity b) is full, new water spills over and is rejected. Where the token bucket lets a burst *through*, the leaky bucket turns a burst into a smooth stream.

There are two variants. As a **queue**, the bucket is a FIFO drained by a worker at rate r: output never exceeds r, and a request waits behind everything in front of it. As a **meter**, nothing is queued: the limiter checks whether a request conforms to the rate and admits or rejects it, which makes it equivalent to a token bucket. The meter form is the Generic Cell Rate Algorithm (GCRA), born in ATM networks, which stores one timestamp per key: the theoretical arrival time (TAT) of the next request.

In the queue form the q-th queued request waits q/r, at most b/r: with r = 10 per second and room for 20, the last request in a full bucket waits 2 seconds. That is the cost of smoothing: latency instead of rejection.

```python
class GCRA:  # leaky bucket as a meter: one timestamp per key, times in ms
    def __init__(self, per_second: int, burst: int):
        self.interval = 1000 // per_second
        self.tolerance, self.tat = burst * self.interval, 0

    def allow(self, now: int) -> bool:
        tat = max(self.tat, now)              # theoretical arrival time
        if tat - now > self.tolerance:        # too far ahead of schedule
            return False
        self.tat = tat + self.interval
        return True


g = GCRA(per_second=10, burst=20)
print(sum(g.allow(0) for _ in range(30)), g.allow(50), g.allow(100))
```

The meter admits 21 requests at t = 0 (one on schedule plus 20 of tolerance), refuses one at 50 ms and accepts the next at 100 ms. nginx's `limit_req` uses the leaky-bucket method: `rate=10r/s burst=20` queues up to 20 excess requests and releases them at the rate, `nodelay` admits the burst at once while keeping the slots occupied, and rejected requests get `limit_req_status`, which defaults to 503 (set it to 429). The redis-cell module offers GCRA as a single command, `CL.THROTTLE`. The queue variant appears wherever a downstream has a hard rate: Encore's notification workers drain the SMS queue at the rate the SMS provider allows, so a flash sale produces a longer queue rather than a wall of provider errors (Chapter 14).

Choose the queue form when the downstream needs a smooth rate and callers tolerate delay (outbound calls to vendors, batch jobs). Choose the meter form, or a token bucket, for interactive APIs, where a fast 429 beats a slow success.

11.5 Window counters

Window algorithms count requests instead of tokens. Picture a nightclub clicker that the doorman resets on the hour (fixed window), a guest book with the arrival time of everyone inside (sliding log), and a doorman who glances at last hour's tally and this hour's tally and blends them (sliding window counter).

- **Fixed window.** One counter per key per window (`INCR` plus `EXPIRE` in Redis). Cheap and easy, but a client can send L requests at the end of one window and L more at the start of the next: $2L$ in a moment.
- **Sliding log.** Keep a timestamp per admitted request; drop those older than W; admit if fewer than L remain. Exact, but memory grows with the limit: $O(L)$ per key.
- **Sliding window counter.** Keep the counts of the current and previous fixed windows and weight the previous one by the part of it still inside the sliding window.

$$\text{estimate} = c_{\text{prev}} \quad (1 - \frac{t_{\text{elapsed}}}{W}) + c_{\text{curr}} \qquad \text{admit if estimate} < L$$

With L = 100 per minute, 80 requests last minute and 30 so far in this one, 15 seconds in: $80 \times 0.75 + 30 = 90$, so the request is admitted. The estimate assumes the previous window's requests were spread evenly; it is wrong only when they were bunched, and it never allows the full $2L$ burst. The demo sends 100 requests at 0:59 and 100 at 1:00: the fixed window admits all 200, the sliding counter 100.

```
from collections import defaultdict


class WindowCounter:
    """Fixed window (blend=False) or sliding window counter."""

    def __init__(self, limit: int, window: float, blend: bool = True):
        self.limit, self.window, self.blend = limit, window, blend
        self.counts = defaultdict(int)

    def allow(self, now: float) -> bool:
        slot, offset = divmod(now, self.window)
        prev = self.counts[slot - 1] * (1 - offset / self.window)
        if self.counts[slot] + self.blend * prev >= self.limit:
            return False
        self.counts[slot] += 1
        return True


times = [59.0] * 100 + [60.0] * 100           # 0:59 and 1:00
for blend in (False, True):
    lim = WindowCounter(limit=100, window=60, blend=blend)
    print(sum(lim.allow(t) for t in times))   # 200, then 100
```

Memory is where they differ in production. A sliding log for “100 per minute” across 1,000,000 active fans holds up to 100 million timestamps; at a rough 50 bytes per sorted-set entry that is about 5 GB of Redis. The sliding window counter needs two integers per key. Cloudflare’s rate limiting uses the sliding window counter for this reason, and AWS WAF rate-based rules count over a sliding evaluation window of 60, 120, 300 or 600 seconds (default 300) with a minimum limit of 10.

Algorithm	State per key	Bursts	Accuracy
Token bucket	2 numbers	Up to b, by design	Exact rate
Leaky bucket (GCRA)	1 timestamp	Up to tolerance	Exact rate
Fixed window	1 counter	Up to $2L$ at edges	Coarse
Sliding log	Up to L entries	None	Exact
Sliding counter	2 counters	Small	Approximate

11.6 Distributed limits with Redis and Lua

Encore’s gateway runs on dozens of nodes. If each node keeps its own bucket, a fan who is spread across 40 nodes by the load balancer gets 40 buckets. The limit has to live in one shared place: picture a scoreboard above all the turnstiles that every steward reads and updates.

Redis is the usual scoreboard (Chapter 9). The danger is the read-modify-write: two gateway nodes read “1 token left” at the same moment, both admit, and the limit leaks. A Lua script fixes this, because Redis executes one script at a time with no other command interleaved, so the read, the refill, the decision and the write happen as one step. Since Redis 7, scripts replicate their effects rather than the script text, so calling `TIME` inside the script is safe and removes clock skew between gateway nodes.

```
from concurrent.futures import ThreadPoolExecutor

import fakeredis

LUA = """
local rate, cap, cost = tonumber(ARGV[1]), tonumber(ARGV[2]),
                        tonumber(ARGV[3])
local t = redis.call('TIME')
local now = tonumber(t[1]) + tonumber(t[2]) / 1e6
local b = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens, ts = tonumber(b[1]) or cap, tonumber(b[2]) or now
tokens = math.min(cap, tokens + math.max(0, now - ts) * rate)
local ok = 0
if tokens >= cost then tokens = tokens - cost; ok = 1 end
redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('EXPIRE', KEYS[1], math.ceil(cap / rate) + 1)
return {ok, math.ceil((cost - tokens) / rate)}
"""
take = fakeredis.FakeRedis().register_script(LUA)


def allow(user: str, rate=0.5, cap=5, cost=1) -> tuple[bool, int]:
    ok, wait = take(keys=[f"rl:{{user:{user}}}:holds"],
                    args=[rate, cap, cost])
    return bool(ok), max(0, wait)              # wait = Retry-After, s


with ThreadPoolExecutor(50) as pool:
    results = list(pool.map(lambda _: allow("42"), range(50)))
print(sum(ok for ok, _ in results), "of 50 admitted")   # 5 of 50
print(allow("42"))                                      # (False, 2)
```

Fifty concurrent attempts against a bucket of five admit exactly five, and the script returns how long the caller must wait, which becomes the `Retry-After` header. The simpler algorithms need no script. A fixed window is two commands on a key named by the minute number since the epoch, and a sliding window counter reads two keys that share a hash tag so they land in the same Redis Cluster slot; a sliding log is a sorted set of timestamps, trimmed, counted and appended (inside one script, so the three steps are atomic):

```
INCR rl:{user:42}:m:29338880          -> 1
EXPIRE rl:{user:42}:m:29338880 120 NX -> 1   (set the TTL only once)
MGET rl:{user:42}:m:29338879 rl:{user:42}:m:29338880
ZREMRANGEBYSCORE rl:{user:42}:log -inf 1760435940123   (sliding log)
ZCARD rl:{user:42}:log                -> admit if below the limit
ZADD rl:{user:42}:log 1760436000123 1760436000123-7f3a
```

The numbers decide the deployment. In the first minute of the flash sale the gateway checks about 30,000 hold attempts per second plus the rest of the API; each check is one round trip of a few hundred microseconds inside an availability zone and one short script. A small Redis Cluster spreads keys over its 16,384 slots, and a hot key is rare because each key is one user. Where the check rate is too high for a round trip per request, each gateway node leases tokens in batches (take 10 at once, spend them locally), trading a little accuracy for ten times fewer calls.

Envoy's global rate limiting calls an external gRPC rate limit service; the reference implementation (`envoyproxy/ratelimit`) keeps its counters in Redis. On AWS the store is ElastiCache (GCP Memorystore, Azure Managed Redis). Decide what happens when the store is down: **fail open** (admit, maybe with a per-node fallback bucket) keeps the site up for real users; **fail closed** protects a fragile backend. Encore fails open at the gateway and relies on the service's own concurrency limit as the second line.

11.7 What to key on, and where it runs

The key decides whose bucket a request drains, and a request usually passes several limits at once, like a traveller who shows a passport, a boarding pass and a bag tag:

- **IP address** for anonymous traffic. Coarse: carrier-grade NAT and campus networks put thousands of people behind one IPv4 address, so IP limits must be generous; for IPv6, key on the /64 prefix, since one host can own billions of addresses.
- **User ID** after sign-in: the fairest key for a consumer product.
- **API key** for partners and integrations; each key carries a plan (rate, burst, daily quota).
- **Tenant** (an organizer's account): one bucket for the whole organization, often with per-user buckets nested inside, so one busy analyst cannot starve colleagues.
- **Route and cost**: an expensive endpoint (search, export, an LLM call) gets its own limit or a higher token cost per call.
- **Global**: a ceiling for the whole service, sized from its tested capacity, regardless of who is calling.

Where each limit runs follows from what that layer knows:

Layer	Knows	Good for	Examples
Edge, WAF	IP, path, headers	Floods, bots, cheap rejection	AWS WAF, Cloud Armor, Cloudflare
API gateway	API key, user from token	Per-client plans	API Gateway, Envoy, Kong, nginx
Service	Business context, cost	Per-endpoint, per-tenant	Library plus Redis
Client	Its own error rate	Slowing itself down	AWS SDK adaptive mode

The edge is AWS WAF on CloudFront (Google Cloud Armor, Azure Front Door WAF); Cloud Armor offers `throttle` and `rate_based_ban` actions, and Azure API Management has a `rate-limit-by-key` policy. On Amazon API Gateway, per-client plans are usage plans bound to API keys:

```
aws apigateway create-usage-plan --name organizer-standard \
  --throttle burstLimit=50,rateLimit=20 \
  --quota limit=200000,period=DAY \
  --api-stages apiId=a1b2c3d4e5,stage=prod
```

API Gateway documents these throttles and quotas as best-effort targets, not exact ceilings, which is true of every distributed limiter: replicas, batching and clock differences let a few requests through over the limit. Never use a rate limiter to enforce a rule that must hold exactly, such as the ticket cap per fan.

> **Design question.** A scalper's script opens 200 accounts and fires hold requests as fast as it can. How do you stop it without slowing the real fan who clicks five seats in a second?
>
> Layer the keys. At the edge, a per-IP rule (say 300 hold requests per minute, illustrative) rejects crude floods for microseconds of work per request. At the gateway, a per-account token bucket in Redis (r = 0.5/s, b = 5) lets the fan's five clicks through and caps any account at 35 hold attempts in the first minute. In the seat inventory service, a durable business rule caps active holds at 8 seats per account per event, checked in the same transaction as the hold. Two hundred accounts still multiply the bot's share, so the remaining defences are not rate limits: account-creation limits, bot detection and one waiting-room token per verified account (Chapter 22). The trade-off is a Redis round trip per hold attempt (about 30,000 per second at peak), which a small Redis Cluster absorbs. See also *50 System Designs* #2 (Rate Limiter).

11.8 Telling the client: 429 and Retry-After

A rejection is a message, and a good one tells the client what happened and when to come back, like a restaurant host who says "a table in 20 minutes" instead of closing the door.

- **429 Too Many Requests** (RFC 6585) means *you* sent too much: a rate limit on your key.
- **503 Service Unavailable** means *I* am overloaded or down, whoever you are: load shedding, maintenance.
- **Retry-After** (RFC 9110) goes with either, as a number of seconds or an HTTP date. The limiter computes it from its own state: for a token bucket, the wait for the next token is (cost − tokens)/r.

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/problem+json
Retry-After: 2
RateLimit-Policy: "holds";q=30;w=60
RateLimit: "holds";r=0;t=2

{"type": "https://encore.example/problems/rate-limited",
 "title": "Too many hold attempts", "status": 429,
 "detail": "Try again in 2 seconds."}
```

The body is an RFC 9457 problem document (Chapter 5). The `RateLimit-Policy` and `RateLimit` fields come from the IETF HTTPAPI working group's draft (version 11, May 2026, not yet an RFC): the policy says 30 units per 60-second window, and the current state says 0 remaining with 2 seconds until quota returns. Many APIs still send the older de facto `X-RateLimit-Limit`, `X-RateLimit-Remaining` and `X-RateLimit-Reset` headers; GitHub is a well-known example.

Three rules keep rejections useful. Make them cheap: the decision must not touch the database, and rejection logs should be sampled, or the limiter becomes the load. Keep the status honest: a business rule ("you already hold 8 seats") is a 409 or 422, not a 429, because retrying will not help. And make clients obey: Encore's apps honour `Retry-After` plus a little random jitter, and disable the button with a countdown, which turns a stream of retries into one.

11.9 Load shedding and admission control

A hospital emergency room does not treat patients in arrival order when the waiting room overflows; triage sends the sprained ankles home and keeps the beds for the heart attacks. Load shedding is the server doing triage on itself: rejecting work early, by priority, so that what it accepts still meets its deadline. Admission control is the same idea at the door of the whole system.

The best overload signal is not requests per second but **concurrency**, the requests in flight. By Little's law, in-flight work is arrival rate times latency, $L = \lambda W$, and a slowdown shows up as concurrency before anything else breaks. Encore's seat inventory fleet (tested at 6,000 holds per second, illustrative) at 20 ms per hold has $6,000 \times 0.020 = 120$ requests in flight. If it slows to 200 ms because the database is struggling, the same arrival rate means 1,200 requests in flight, every thread busy and every new request queued behind them. A cap of about twice the normal concurrency, enforced per instance, sheds the excess with a fast 503 instead. Shed by priority: confirming a paid order beats a new hold, which beats a seat-map refresh, which beats analytics. Google tags every request with a criticality (`CRITICAL_PLUS`, `CRITICAL`, `SHEDDABLE_PLUS`, `SHEDDABLE`) and sheds from the bottom.

Clients can shed too. In Google's adaptive throttling, each client tracks its recent requests and the accepts among them and drops requests locally with the probability below, with $K = 2$; at 100 requests and 40 accepts it drops about 20 percent before they leave the machine.

$$p_{\text{reject}} = max(0, \frac{\text{requests} - K \quad \text{accepts}}{\text{requests} + 1})$$

Fixed caps need tuning; adaptive limiters find the cap by watching latency. Envoy's adaptive concurrency filter (a gradient controller against a measured minimum RTT) and Netflix's concurrency-limits library work this way, and Kubernetes' API Priority and Fairness (stable since 1.29) splits the API server's concurrency into priority levels with fair queuing among flows.

11.10 Backpressure

On a sushi conveyor, the chef stops placing plates when the belt is full; nobody piles plates on the floor. Backpressure is a full stage telling the stage before it to slow down, all the way back to the source, instead of buffering without limit.

The rule is that **every queue has a bound**, and the bound picks a behaviour when it is reached: block the producer (an in-process `asyncio.Queue(maxsize=100)`, whose `put` waits), reject upstream (a 503 or 429 to the caller), or drop the oldest or least important items (metrics samples, seat-map refreshes). An unbounded queue does not remove

overload; it converts it into latency and then into an out-of-memory crash, and every request at the back is already past its deadline when it is served. A queue of Q items drained at μ per second adds Q/μ of waiting: 10,000 queued holds at 6,000 per second is 1.7 seconds before any work starts.

Backpressure is built into the protocols under you: TCP's receive window, HTTP/2 and QUIC per-stream flow control (Chapter 4), Reactive Streams' `request(n)`, gRPC streaming flow control, and pull-based consumers such as Kafka's, which fetch only as fast as they process and show overload as consumer lag rather than as dropped messages (Chapter 14). In services, backpressure travels as fast rejections plus the client's obedience to them.

11.11 Timeouts, retries and jitter

A phone call that rings forever ties up the caller. Every remote call needs a **timeout**, and the timeouts along a chain must fit inside the user's patience. Encore gives a hold request a 2-second deadline at the gateway and propagates the *remaining* time downstream (gRPC sends it as the `grpc-timeout` header), so a service never works on a request whose caller has already given up. Set each timeout from the dependency's measured latency, near its p99.9 plus a margin, not from a round number.

A **retry** is safe only when the operation is idempotent (a hold with a client-generated hold ID, a payment with an idempotency key) and the error is transient (timeouts, 503, 429 with `Retry-After`). Retrying a 400 repeats the mistake. Space retries with exponential backoff and **full jitter**: attempt k sleeps a uniform random time up to an exponentially growing, capped ceiling.

$$\text{sleep}_k \quad U(0,\ \min(\text{cap},\ \text{base} \quad 2^k))$$

With base 100 ms and cap 5 s the ceilings are 0.1, 0.2, 0.4, 0.8, 1.6, 3.2 and then 5 seconds. Without jitter, 1,000 clients that failed together retry together, in the same millisecond, and fail together again.

```
import random
from collections import Counter


def backoff(attempt: int, base=0.1, cap=5.0) -> float:
    """Full jitter: uniform in [0, min(cap, base * 2**attempt)]."""
    return random.uniform(0, min(cap, base * 2 ** attempt))


random.seed(7)
plain = Counter(int(0.1 * 1000) for _ in range(1000))       # ms buckets
jitter = Counter(int(backoff(0) * 1000) for _ in range(1000))
print(max(plain.values()), max(jitter.values()))   # 1000 vs 17
```

AWS SDKs use full jitter by default (standard mode: 3 attempts, a 20-second cap, and a 500-token retry quota that stops retries when failures are widespread); gRPC's retry policy caps `maxAttempts` at 5 by default and jitters each backoff by plus or minus 20 percent; Envoy and Istio configure retries per route.

11.12 Retry budgets and amplification

Retries multiply. If every layer of a chain makes up to a attempts and the bottom layer is failing, one user click becomes a^n calls at the bottom of n layers. With 3 tries each in the app, the gateway, orders and inventory, a failing database receives $3^4 = 81$ calls per click.

That is how a ten-second database hiccup becomes an hour-long outage: the retries keep the database at 81 times its normal load long after the original cause is gone, a **retry storm**. Three fixes, used together:

- **Retry at one layer**, normally the one just above the failing dependency; every other layer passes the failure up.
- **Retry budgets.** Allow retries only while they stay below a fraction of traffic. Google's SRE guidance is at most 3 attempts per request and a per-client retry ratio below 10 percent, which caps the extra load at 1.1 times. Envoy's `retry_budget` defaults to 20 percent of active requests (with a floor of 3 concurrent retries); gRPC keeps a

`token_count` that each failure lowers by 1 and each success raises by `tokenRatio`, and stops retrying when it falls to half of `maxTokens`.

- **Do not retry overload.** A 503 from load shedding with a long `Retry-After`, or a "do not retry" marker from a layer that already retried, ends the chain.

Hedged requests, which send a second copy after a short delay to cut tail latency, are retries that happen before failure; give them the same budget.

11.13 Circuit breakers and bulkheads

A fuse blows so that the house wiring does not burn. A **circuit breaker** wraps calls to one dependency, counts recent failures and slow calls, and when they cross a threshold stops calling for a while, failing fast instead of waiting on timeouts. After a cool-down it lets a few probe calls through to test recovery.

```
CLOSED    --(failure rate >= threshold)--> OPEN
OPEN      --(cool-down, e.g. 30 s)-------> HALF-OPEN
HALF-OPEN --(probes succeed)-------------> CLOSED
HALF-OPEN --(a probe fails)--------------> OPEN
```

Resilience4j's defaults are a 50 percent failure threshold over the last 100 calls (with at least 100 recorded), 60 seconds open, and 10 probe calls in half-open; Polly plays the same role in .NET, and Envoy and Istio's outlier detection ejects individual failing hosts from a pool, a breaker per host. A **bulkhead** is the compartment wall of a ship: each dependency gets its own bounded pool of connections, threads or concurrency, so one slow dependency floods only its own compartment. Envoy's cluster "circuit breakers" are in fact bulkheads (`max_connections`, `max_pending_requests` and `max_requests` default to 1,024, `max_retries` to 3); AWS Lambda reserved concurrency (Cloud Run maximum instances, Azure Functions maximum scale-out) does the same for functions.

> **Design question.** During the flash sale the payment provider's p99 latency jumps from 300 ms to 8 s and 30 percent of calls fail. How do you keep checkout and the rest of Encore alive?
>
> At an illustrative 300 payments per second, Little's law gives 90 calls in flight normally and 2,400 at 8 s, enough to exhaust the orders service's threads and stall holds too. A bulkhead caps payment calls at 500 in flight per region and fails the rest fast. A 3-second timeout bounds each call; a breaker opens at 50 percent failures over 20 calls and probes after 30 seconds. While it is open, checkout tells the fan "payment is delayed, your seats stay held", since the hold lasts 10 minutes. A timed-out charge may have succeeded, so the retry reuses the same idempotency key, and the order is settled by the provider's webhook or a status query, never by a blind second charge. The trade-off: some fans wait or lose their hold, and nobody is charged twice. See also *50 System Designs* #24 (Payment System).

11.14 In Encore: the waiting room

The waiting room is admission control for a whole event: instead of letting 1,000,000 fans hit seat inventory at 10:00, Encore queues them in front of it and lets them in at a rate the system has proven it can serve. Fans who arrive between 09:55 and 10:00 get a random position at 10:00, so refreshing early buys nothing; later arrivals join the back. Each fan's position is one atomic counter, and a scheduler advances an "admitted up to" counter every second:

```
INCR wr:{evt:9001}:issued            -> 734512   (my position)
INCRBY wr:{evt:9001}:admitted 1000   (scheduler, once per second)
GET wr:{evt:9001}:admitted           -> in when it reaches my position
```

An admitted fan receives a short-lived signed token (event, user, expiry) that an edge function checks on every seat-map and hold request (Chapter 10), so requests without a token never reach the origin, and the queue page polls a status object that the CDN caches for a few seconds. Products exist for this: Cloudflare Waiting Room, Queue-it, and the Virtual Waiting Room on AWS solution.

> **Design question.** How fast should the waiting room let fans in, and what does a fan in position 734,512 see?

> Start from the bottleneck. Seat inventory is tested at 6,000 holds per second (illustrative); keep it at half that, 3,000. An admitted fan makes about 3 hold attempts in a session, so admit $\lambda = 1,000$ fans per second: a tenth of the 30,000 holds per second an open door would bring. A fan at position p waits p/λ: 40 seconds at position 40,000, and about 17 minutes at the back of a million. But if half the admitted fans buy two seats, the 50,000 seats are gone after about 50,000 admissions, or 50 seconds; the room then stops admitting and tells everyone else the truth at once. Position 734,512 sees "sold out", not a 12-minute spinner. Raise λ only after a load test proves the next step. See also *50 System Designs* #25 (Ticket Booking) and #34 (E-Commerce Checkout and Inventory).

11.15 Token limits for LLM APIs

LLM APIs are rate-limited in tokens as well as requests, because a call's cost is its size. The Claude API, for example, limits each model by requests, input tokens and output tokens per minute (RPM, ITPM, OTPM), enforces them with token buckets, and returns 429 with `retry-after` plus `anthropic-ratelimit-*` headers showing what remains; for most models only uncached input tokens count toward ITPM, so prompt caching raises effective throughput. Encore's support agent calls the model through a model gateway that holds a token bucket per feature and tenant with `cost` set to the estimated tokens, then corrects the bucket with the actual usage in the response:

```
# reuses TokenBucket from the token bucket section
itpm = TokenBucket(rate=2_000_000 / 60, capacity=200_000)
if not itpm.allow(cost=4_000):        # estimated uncached input tokens
    raise RuntimeError("over budget: queue the step, do not call")
```

With an illustrative 2,000,000 ITPM and 4,000 uncached input tokens per agent step, the budget is 500 steps a minute, about 100 conversations taking 5 steps a minute each. Live chats get priority over batch jobs such as ticket summaries, long agent tasks wait on a queue instead of failing, and the gateway falls back to a smaller model when the bucket is empty (Chapter 24; *Designing Agentic Systems* covers agent budgets in depth). The same pattern applies to every metered external API: the SMS provider, the e-mail provider and the payment provider all publish limits that Encore should enforce on its own side, before the vendor does.

11.16 Numbers to remember

Figure	Value
Token bucket, any window T	at most $b + rT$ admitted
Encore hold limit per fan	0.5/s, burst 5: 35 in the first minute
Fixed window worst case	$2L$ around a boundary
Amazon API Gateway default	10,000 req/s, burst 5,000, per account per Region
In flight (Little's law)	6,000/s x 20 ms = 120
Retry amplification	a^n : 3 tries at 4 layers = 81x
Retry budget of 10 percent	at most 1.1x load
Full-jitter backoff	$U(0, min(\text{cap}, \text{base} \quad 2^k))$
Waiting room	1,000 fans/s in: 3,000 holds/s, not 30,000

11.17 Pitfalls

- Per-node limits behind a load balancer multiply the limit by the node count -> keep shared state in Redis, or divide the limit by the node count.
- Read-then-write limiter logic leaks under concurrency -> one Lua script or atomic command per decision.
- Retrying at every layer, without jitter -> retry at one layer, full jitter, a retry budget.
- Unbounded queues and missing timeouts hide overload until memory or patience runs out -> bound every queue, propagate deadlines.

- Using a rate limiter for exact business rules, or answering them with 429 -> durable counts in the domain service, 409 or 422.

11.18 Remember

- Say “not now” early and cheaply: edge, then gateway, then service, each rejecting what the next cannot afford.
- Token bucket for bursty users, leaky bucket for smooth outbound rates, sliding window counter for cheap accurate windows; Redis plus Lua for shared, atomic state.
- 429 means the client sent too much, 503 means the server is full; both carry `Retry-After`.
- Watch concurrency, not requests per second: $L = \lambda W$ reveals a slowdown first; shed by priority.
- Retries need idempotency, jitter and a budget; breakers fail fast and bulkheads contain the damage.
- The waiting room turns a stampede into a queue at a tested rate and tells the truth when seats run out.

12 Storage and databases

Every service eventually has to remember something: a seat that is held, an order that is paid, a PDF that must be downloadable next year. This chapter answers one question — where should each piece of data live, and what does the store do with it underneath? — by following a write from the application down to the disk and back, and then matching Encore's data to the right kind of database.

12.1 The picture

Think of a library. New books arrive all day. One librarian shelves each book in its exact place at once (slow per book, easy to find later); another drops new books on a cart, sorts the cart when it is full and merges it into the shelves at night (fast to accept, a little slower to search). Both keep a logbook of every arrival so that nothing is lost if the lights go out. The first librarian is a **B-tree**, the second an **LSM-tree**, and the logbook is the **write-ahead log**.

```
app: INSERT order ...           (connection pool, transaction)
       |
       v
+----------------- database process -------------------+
| parser/planner -> executor -> indexes                |
| buffer pool (pages in RAM)      WAL buffer           |
+-------|-------------------------------|--------------+
        | write pages later             | fsync at COMMIT
        v                               v
data files (B-tree pages            write-ahead log
or SSTables)                        (append only)
        \                               /
         +----- OS page cache ----------+
                      |
          block device: NVMe SSD / EBS volume
          (or: replicas, object storage for backups)
```

The rule that makes a database durable is in the picture: **the log is written and flushed before the commit is acknowledged; the data pages can follow later.** After a crash, the database replays the log. Everything else in this chapter — engines, indexes, isolation, pools — is about doing that quickly and correctly while many clients read and write at once.

12.2 Storage media and access patterns

A disk is a warehouse with a loading dock. Fetching one box from a random aisle costs a walk; taking the next box on the same shelf is almost free. Every storage design is shaped by that difference between **random** and **sequential** access, and by the unit the device moves: a 4 KB or 16 KB block, never a single byte.

Medium	Random read latency	Random IOPS	Sequential throughput
RAM	~100 ns	not the limit	tens of GB/s
NVMe SSD (local)	tens of microseconds	100,000s	several GB/s
Network block (EBS gp3)	single-digit ms	3,000 baseline	125 MiB/s baseline
HDD, 7,200 rpm	~8 ms	~120	~200 MB/s

The rows are orders of magnitude, not benchmarks. A hard disk's random IOPS falls straight out of its mechanics: an average seek of about 4 ms plus half a rotation.

$$\mathrm{IOPS}_{\mathrm{HDD}} \approx \frac{1}{t_{\mathrm{seek}} + \frac{1}{2}\,\frac{60}{\mathrm{rpm}}} = \frac{1}{4\,\mathrm{ms} + 4.17\,\mathrm{ms}} \approx 122$$

For any device, throughput = IOPS × I/O size, and whichever limit you hit first wins. A gp3 volume at its 3,000 baseline IOPS with 16 KB pages moves about 47 MiB/s of random I/O, well under its 125 MiB/s baseline throughput; the IOPS budget, not bandwidth, is the ceiling. AWS lets you provision gp3 up to 80,000 IOPS and 2,000 MiB/s, and io2 Block Express for sub-millisecond latency. Four consequences shape every engine below:

- **Write sequentially when you can.** Appending to a log turns random writes into sequential ones; this is why every database has a write-ahead log and why LSM-trees exist.
- **`fsync` is the price of durability.** A commit is durable only after the log reaches stable storage. If one flush takes 1 ms, committing one transaction per flush caps a single writer at about 1,000 commits/s; **group commit** flushes many transactions together, so 30 commits per flush give about 30,000 commits/s (illustrative).
- **RAM hides the disk.** The database buffer pool and the OS page cache keep hot pages in memory. A working set that fits in RAM makes almost any engine fast; once it spills, the access pattern decides everything.
- **Name the access pattern first**: point lookups by key (seat by id), range scans (orders of a user by date), full scans (analytics), append-mostly streams (clickstream), large immutable blobs (ticket PDFs); then pick the engine and data model that serve it.

12.3 Storage engines: B-trees and LSM-trees

The storage engine is the part of the database that lays bytes on disk. Two designs dominate, and the choice between them is a choice of which operation pays.

```
B-tree (update in place)              LSM-tree (append, merge later)
       [ 40 | 80 ]   root              put -> WAL (append) + memtable (RAM)
      /     |     \                              |  full
[10|25] [50|65] [90|99] internal                 v
  |  ...                                L0: [sst] [sst] [sst]  (overlap)
[leaf pages: sorted keys -> rows]       L1: [sst----][sst----]  sorted,
                                        L2: [sst][sst][sst][sst] 10x larger
write: find leaf, rewrite its page      read: memtable, then each level
read: root -> leaf, 3 to 4 pages              (Bloom filter skips most)
```

12.3.1 B-trees

A B-tree keeps keys sorted in fixed-size pages (8 KB in Postgres, 16 KB in MySQL InnoDB). Each internal page holds hundreds of keys and child pointers, so the tree is shallow. A write finds the leaf page, changes it in the buffer pool, records the change in the WAL and lets a background checkpoint write the page later. A full page splits in two. The height is the number of pages a lookup touches:

$$h \approx \log_f N \ , \qquad f \approx \frac{\text{page size}}{\text{key+pointer}}$$

With 8 KB pages and 20-byte entries, the fanout is about 409. Five years of Encore orders, 250 million rows, need $\log_{409} 2.5 \times 10^8$ = 4 levels. The leaves take about 611,000 pages (5 GB) but the three levels above them only about 1,500 pages (12 MB), which always stay in RAM, so a lookup costs at most one disk read for the leaf (plus one for the row in Postgres, whose table is a separate heap).

12.3.2 LSM-trees

A log-structured merge-tree never updates in place. A write appends to the WAL and inserts into an in-memory sorted **memtable**. When the memtable fills (64 MB by default in RocksDB) it is flushed as an immutable sorted file, an **SSTable**. Background **compaction** merges SSTables into larger sorted levels, discarding overwritten values and **tombstones** (delete markers). A read checks the memtable, then the levels from newest to oldest; a per-file **Bloom filter** answers “definitely not here” for most files, so a point read touches about one file. This toy version runs as written:

```
TOMBSTONE = None                            # a delete is a write too


class TinyLSM:
    def __init__(self, memtable_limit=3):
        self.wal, self.mem, self.runs = [], {}, []   # runs: newest first
        self.limit = memtable_limit

    def put(self, key, value):
        self.wal.append((key, value))       # 1. append to the log
        self.mem[key] = value               # 2. update the memtable
        if len(self.mem) >= self.limit:     # 3. full: flush a sorted run
            self.runs.insert(0, dict(sorted(self.mem.items())))  # SSTable
            self.mem, self.wal = {}, []     # run is durable: drop log

    def get(self, key):
        for table in [self.mem, *self.runs]:    # newest wins
            if key in table:
                return table[key]
        return None

    def compact(self):                      # merge runs, drop old values
        merged = {}
        for run in reversed(self.runs):     # oldest first, newer overwrite
            merged.update(run)
        self.runs = [{k: v for k, v in sorted(merged.items())
                      if v is not TOMBSTONE}]


db = TinyLSM()
for seat, state in [("A1", "free"), ("A2", "free"), ("A3", "free"),
                    ("A1", "held"), ("A2", "sold"), ("A3", TOMBSTONE)]:
    db.put(seat, state)
print(len(db.runs), db.get("A1"), db.get("A3"))  # 2 held None
db.compact(); print(db.runs)  # [{'A1': 'held', 'A2': 'sold'}]
```

12.3.3 Amplification

Three ratios compare engines. **Write amplification** is bytes written to disk per byte the application wrote; **read amplification** is I/Os per lookup; **space amplification** is bytes on disk per byte of live data. A B-tree that changes one 100-byte seat row eventually rewrites its whole 8 KB page: up to 8192/100 ≈ 82×, plus the WAL record (Postgres also logs the full page on its first change after each checkpoint). Many updates to the same hot page share one page write, which is why B-trees shine when the working set is cached.

For a leveled LSM-tree with size ratio T between levels and L levels, each byte is rewritten about T times per level on its way down:

$$L \approx \log_T \frac{\text{data size}}{\text{level-1 size}} + 1, \qquad WA_{\text{leveled}} \approx T \cdot L, \qquad SA_{\text{leveled}} \approx \frac{T}{T-1}$$

With RocksDB's defaults (T = 10, a 256 MB first level), 1 TB of data fills five levels (256 MB, 2.56 GB, 25.6 GB, 256 GB, 2.56 TB), giving a worst-case write amplification near 50 and space amplification near 1.11. **Size-tiered** compaction (Cassandra's classic default) merges files of similar size: write amplification drops to about L, but a read may check T files per level and compaction can temporarily need twice the space.

	B-tree	LSM, leveled	LSM, size-tiered
Writes	random page writes	sequential, rewritten ~T*L	sequential, ~L
Point reads	~1 I/O (cached top), best ranges	~1 I/O with Bloom filters	several I/Os
Space	page fill ~70 percent	~1.1x	up to 2x
Runs in	Postgres, InnoDB, SQL Server	RocksDB, LevelDB, Pebble	Cassandra (STCS)

Choose a B-tree for read-heavy, transactional data with range queries and a cacheable working set (catalog, orders). **Choose an LSM-tree** for write-heavy data, time-ordered events, and large key-value stores on SSDs (clickstream counters, seat-event logs, DynamoDB-style workloads). The cost of an LSM-tree is compaction: it competes with foreground traffic for I/O, and a backlog shows up as rising read latency and write stalls.

12.4 Indexes

An index is the back-of-book index: a second, smaller structure, sorted by something other than page order, that points into the main data. It turns a scan of *N* rows into a search of about *logN* steps, and every index costs extra work on every write and extra space: a table with six indexes does seven writes per insert. Index the columns your hot queries filter and sort on, in that order, and drop indexes nobody uses (Postgres reports them in `pg_stat_user_indexes`).

A **composite index** is sorted by its first column, then its second, like a phone book sorted by last name, then first name. It serves queries that constrain a **leftmost prefix** of its columns, and it can return rows already in the order of the next column. If it contains every column the query needs, it is a **covering index** and the table is never touched. SQLite shows all three cases (Postgres and MySQL behave the same way here):

```
import sqlite3

db = sqlite3.connect(":memory:")
db.executescript("""
CREATE TABLE orders (
  order_id INTEGER PRIMARY KEY, user_id INTEGER NOT NULL,
  created_at TEXT NOT NULL, status TEXT NOT NULL, total INTEGER NOT NULL);
CREATE INDEX orders_user_time ON orders (user_id, created_at);
""")
for q in ["SELECT * FROM orders WHERE user_id = 42 ORDER BY created_at",
          "SELECT * FROM orders WHERE created_at > '2026-10-01'",
          "SELECT user_id, created_at FROM orders WHERE user_id = 42"]:
    print(db.execute("EXPLAIN QUERY PLAN " + q).fetchone()[-1])
# SEARCH orders USING INDEX orders_user_time (user_id=?)
# SCAN orders                       <- created_at alone is not a prefix
# SEARCH orders USING COVERING INDEX orders_user_time (user_id=?)
```

Index	Answers	Structure	Where it runs
B-tree	equality, range, sort	sorted pages	every relational DB, the default
Hash	equality only	hash table	Postgres `USING hash`, Redis, Memcached
Inverted	“documents with word w”	term -> posting list	Lucene, Elasticsearch, OpenSearch, Postgres GIN
Geospatial	“within 5 km”, “inside polygon”	R-tree, geohash, S2, H3	PostGIS (GiST), Redis GEO, MongoDB 2dsphere
Vector	“nearest embeddings”	HNSW graph, IVF clusters	pgvector, OpenSearch, Pinecone, S3 Vectors

Vector indexes are approximate: HNSW walks a layered graph of neighbours and trades recall for speed through its search width. In Postgres with the pgvector extension:

```
CREATE EXTENSION IF NOT EXISTS vector;
CREATE TABLE policy_chunks (
  id bigserial PRIMARY KEY, body text, embedding vector(1024));
CREATE INDEX ON policy_chunks USING hnsw (embedding vector_cosine_ops);
SET hnsw.ef_search = 100;          -- wider search: better recall, slower
SELECT id, body FROM policy_chunks
ORDER BY embedding <=> $1 LIMIT 5; -- $1: the query embedding
```

12.5 Data models

A data model is the shape the database lets you think in: tables, documents, a giant hash map, a graph. Pick it from the access pattern, not from fashion. The table is the overview; Chapter 13 takes each family in turn, with NoSQL data modelling worked through for Encore.

Model	Shape	Best for	Weak at	Examples
Relational	tables, joins, SQL	invariants, ad-hoc queries	huge write scale on one node	Postgres, MySQL, Aurora
Distributed SQL	relational, sharded, consensus	SQL plus horizontal scale	cost, cross-region latency	Spanner, CockroachDB, Aurora DSQL
Key-value	key -> value	lookups by id at any scale	queries by anything else	DynamoDB, Redis, Riak
Document	key -> JSON tree	self-contained aggregates	cross-document joins	MongoDB, Firestore, DocumentDB
Wide-column	partition -> sorted rows	huge writes, time-ordered rows	ad-hoc queries, joins	Cassandra, Bigtable, ScyllaDB
Graph	nodes and edges	multi-hop relationships	bulk scans	Neo4j, Neptune, Spanner Graph
Time-series	(series, time) -> value	metrics, downsampling	random updates	Timestream, InfluxDB, TimescaleDB
Search	inverted index	full text, facets, ranking	being the source of truth	OpenSearch, Elasticsearch
Columnar (OLAP)	compressed columns	scans, aggregates	single-row updates	BigQuery, Redshift
Vector	id -> embedding	similarity search	exact filters at scale	pgvector, S3 Vectors, Pinecone

Two habits make the choice easy. First, **start relational** for anything with money, inventory or invariants: constraints, transactions and SQL are worth more than theoretical scale you do not need yet; a single Postgres primary on a large instance handles thousands of transactions per second. Second, **design key-value and wide-column tables from the queries backwards**: in DynamoDB or Cassandra you choose a partition key (which machine) and a sort key (order within it), and every query you will ever run must be answerable from those keys or a secondary index. DynamoDB hashes the partition key to place items, limits an item to 400 KB, and offers transactions across up to 100 items; Cassandra orders rows within a partition by its clustering columns. Chapter 17 covers how to choose those keys. On the cloud, managed Postgres is Amazon RDS or Aurora (Google Cloud SQL or AlloyDB; Azure Database for PostgreSQL), and DynamoDB maps roughly to Firestore or Bigtable on Google Cloud and Cosmos DB on Azure. Search and vector stores are almost always **derived** stores: the source of truth lives in a relational or key-value database and change data capture keeps the index fresh (Chapter 14).

12.6 Transactions and ACID

A transaction is a shopping trolley at a checkout: you load several items, and either everything goes through the till or you walk away with nothing. **ACID** names four guarantees:

- **Atomicity**: all writes of a transaction take effect or none do. The engine uses the WAL (and undo information) to roll back a half-finished transaction after an error or crash.
- **Consistency**: the database moves from one valid state to another as defined by *your* constraints (primary keys, foreign keys, `CHECK`, `UNIQUE`). It is the application's property, enforced with the database's tools.
- **Isolation**: concurrent transactions do not see each other's half-done work; how strictly is the isolation level, next section.
- **Durability**: once `COMMIT` returns, the data survives a crash — the WAL is flushed to disk, and in a replicated setup usually to a second machine too.

Buying two seats must mark both sold and create one order, or do nothing:

```
import contextlib
import sqlite3

db = sqlite3.connect(":memory:")
db.executescript("""
CREATE TABLE seats (seat_id TEXT PRIMARY KEY, order_id INTEGER,
  status TEXT NOT NULL CHECK (status IN ('free', 'held', 'sold')));
CREATE TABLE orders (order_id INTEGER PRIMARY KEY, user_id INTEGER);
INSERT INTO seats VALUES ('A1', NULL, 'held'), ('A2', NULL, 'held');
""")


def buy(user_id, seat_ids, crash_before=None):
    with db:                          # BEGIN; COMMIT, or ROLLBACK on error
        cur = db.execute("INSERT INTO orders (user_id) VALUES (?)",
                         (user_id,))
        for seat in seat_ids:
            if seat == crash_before:
                raise RuntimeError("worker crashed")
            db.execute("UPDATE seats SET status = 'sold', order_id = ? "
                       "WHERE seat_id = ?", (cur.lastrowid, seat))

with contextlib.suppress(RuntimeError):
    buy(7, ["A1", "A2"], crash_before="A2")
print(db.execute("SELECT seat_id, status FROM seats").fetchall())
# [('A1', 'held'), ('A2', 'held')]   -- nothing half-sold
print(db.execute("SELECT count(*) FROM orders").fetchone())   # (0,)
```

Keep transactions short and local. Never hold one open across a call to the payment provider or an LLM: the rows stay locked for seconds and the external call cannot be rolled back anyway. Charge the card outside the transaction with an idempotency key, then record the result in a short transaction (Chapter 5, Chapter 14 for the outbox). Durability itself has knobs: Postgres `synchronous_commit = off` acknowledges before the WAL flush (faster, may lose the last few hundred milliseconds of commits on a crash, never corrupts), and Aurora acknowledges a commit once four of its six storage copies across three availability zones have it.

12.7 Isolation levels and anomalies

Isolation is how thick the walls are between concurrent transactions. Thin walls are fast but let you hear the neighbours; each level removes a class of **anomalies**:

Anomaly	Encore example	Read Committed	Repeatable Read / snapshot	Serializable
Dirty read	see a hold that later rolls back	prevented	prevented	prevented
Non-repeatable read	price changes between two reads	possible	prevented	prevented
Phantom	“free seats in row K” count changes	possible	prevented in Postgres	prevented
Lost update	two read-modify-writes, one vanishes	possible	Postgres aborts one; InnoDB allows	prevented
Write skew	two orders each pass the 8-ticket check	possible	possible	prevented

Defaults differ: Postgres, SQL Server and Oracle default to Read Committed, MySQL InnoDB to Repeatable Read; CockroachDB, Spanner and SQLite default to Serializable. Postgres treats Read Uncommitted as Read Committed, implements Repeatable Read as snapshot isolation, and implements Serializable as **serializable snapshot isolation**

(SSI), which aborts one of two conflicting transactions with SQLSTATE `40001` ("could not serialize access"); the application must retry it.

The two anomalies that bite services are lost updates and write skew. The first comes from reading a value into application code and writing back a computed value; the cure is to let the database do the check and the write in one statement, so the row lock serialises the writers even at Read Committed:

```
-- Lost update: both fans read status = 'free', both write 'held'.
SELECT status FROM seats WHERE seat_id = 'A1';            -- app sees 'free'
UPDATE seats SET status = 'held', holder = 2 WHERE seat_id = 'A1';

-- Fix: a conditional write; exactly one caller gets rowcount = 1.
UPDATE seats SET status = 'held', holder = 2
WHERE seat_id = 'A1' AND status = 'free';
-- Or lock first (pessimistic): SELECT ... FOR UPDATE, then decide.
```

Write skew is subtler: two transactions read the same set, then write *different* rows, so no row conflict exists. A fan with six tickets opens the app on two phones and buys two more on each; each transaction counts six, inserts its own order, and the fan ends with ten.

> **Design question.** How do you enforce "at most 8 tickets per fan per event" while 30,000 holds per second arrive in the flash sale?
>
> The rule spans many order rows, so a per-row conditional write cannot see it, and snapshot isolation lets both purchases through (write skew). Option one: run the purchase at Serializable and retry on `40001`; correct, but at 30,000 attempts/s the abort rate on popular events adds load. Option two, the usual choice: **materialise the conflict** as one row per (fan, event) and update it conditionally, `UPDATE fan_limits SET bought = bought + 2 WHERE fan_id = 7 AND event_id = 1 AND bought + 2 <= 8`, inside the same transaction as the order. Now both phones contend on one row; the second sees `bought = 8` and gets zero rows. The cost is one extra row write per purchase, spread across millions of fans, so it creates no hot spot. See also *50 System Designs* #25 (Ticket Booking).

12.8 MVCC

Multi-version concurrency control is how most databases give readers a stable view without locking writers out: an update does not overwrite a row, it writes a **new version** stamped with the writing transaction's id, and each transaction reads through a **snapshot** that says which versions were committed when it started.

```
seat A1 versions          created_by  deleted_by
  ('free')                 tx 100      tx 205
  ('held', fan 2)          tx 205      -
snapshot of tx 204 (started before 205 committed) -> sees 'free'
snapshot of tx 206 (started after)                -> sees 'held'
```

Readers never block writers and writers never block readers; only two writers to the same row wait for each other. Postgres keeps old versions in the table itself (visible as the hidden `xmin` and `xmax` columns) and reclaims them with **VACUUM**; MySQL InnoDB and Oracle keep them in undo logs; CockroachDB and Spanner key versions by timestamp, which also enables reads "as of" a past time. The trade-off is garbage: a transaction left open for an hour stops vacuum from removing any version its snapshot might still need, so tables bloat and vacuum falls behind. Set `idle_in_transaction_session_timeout` and alert on the oldest open transaction.

12.9 Object, block and file storage

Three ways to keep bytes, like three ways to park: **block storage** is a private garage attached to one house (a raw disk for one machine), **file storage** is a shared car park with numbered bays and a directory (a POSIX file system many machines mount), and **object storage** is valet parking: you hand over the whole car, get a ticket (the key), and later get the whole car back.

	Block	File	Object
Interface	a disk for one instance	shared directories (NFS, SMB)	HTTP PUT/GET whole objects
Latency	sub-ms to ms	ms	tens of ms per request
Use for	database volumes	shared config, legacy apps	blobs, backups, data lakes
AWS (GCP; Azure)	EBS (Persistent Disk, Hyperdisk; Managed Disks)	EFS (Filestore; Azure Files)	S3 (Cloud Storage; Blob Storage)

S3 stores objects under keys in a flat namespace; slashes in keys are only a naming convention. S3 Standard is designed for 99.999999999 percent (eleven nines) durability by storing data redundantly across at least three availability zones. Since December 2020 it is **strongly consistent**: a GET or LIST after a successful PUT or DELETE sees the change. There are no atomic updates across keys, and two concurrent PUTs to one key resolve by last writer wins unless you use **conditional writes** (`If-None-Match: *` to create only if absent, `If-Match: <etag>` to replace only an unchanged object). Each partitioned prefix sustains at least 3,500 writes and 5,500 reads per second, and S3 adds partitions as load grows.

A single PUT carries up to 5 GB. Larger objects use **multipart upload**: up to 10,000 parts of 5 MiB to 5 GiB each, uploaded in parallel and retried individually, for objects up to about 50 TB (48.8 TiB). Abandoned uploads keep their parts, and their bill, until aborted: add a lifecycle rule that aborts incomplete uploads after a few days. The part count fixes the minimum part size:

$$\text{part size} \geq \frac{\text{object size}}{10,000} \qquad \text{e.g. } \frac{1\ \text{TiB}}{10,000} \approx 105\ \text{MiB}$$

```
POST /video/aftermovie.mp4?uploads                -> UploadId=VXBs
PUT  /video/aftermovie.mp4?partNumber=1&uploadId=VXBs  -> ETag "a54"
PUT  /video/aftermovie.mp4?partNumber=2&uploadId=VXBs  -> ETag "0c7"
POST /video/aftermovie.mp4?uploadId=VXBs   body: <CompleteMultipartUpload>
  with the list of (PartNumber, ETag) pairs  -> one object, ETag "...-2"
```

> **Design question.** Where do Encore's ticket PDFs live, and how does a fan download one?
>
> Volume: at least one 50 KB PDF per order, 50 million orders a year, so about 2.5 TB a year — small for S3, wasteful in Postgres, whose backups, replicas and buffer pool would all carry the blobs. Store each PDF in S3 under a non-guessable key such as `tickets/<event_id>/<order_id>-<random>.pdf`, keep only the key and a checksum in the orders row, and serve downloads through short-lived **presigned URLs** (or signed CloudFront URLs, Chapter 10) issued after an ownership check. Eleven nines of durability means an expected loss of $5 \times 10^{7} \times 10^{-11} = 0.0005$ objects a year; the real risks are deletion and bugs, so enable versioning and regenerate PDFs from the order row if needed. See also *50 System Designs* #33 (Object Storage (S3)).

12.10 Connection pools

Opening a database connection costs a TCP and TLS handshake, authentication and, in Postgres, a whole server process, so applications keep a **pool** of open connections and lend them to requests. How many are busy is Little's law (Chapter 1): arrival rate times time held.

$$C_{\text{busy}} = \lambda \times t_{\text{held}}$$

At Encore's daily peak, 6,000 requests/s with two 2 ms queries each keep only $6000 \times 2 \times 0.002 = 24$ connections busy. The flash sale's 30,000 holds/s at 3 ms each need about 90. The danger is the multiplication on the other side: 40 pods with a pool of 20 open 800 connections against a Postgres whose `max_connections` defaults to 100. Put a pooler in between — PgBouncer in transaction mode, or Amazon RDS Proxy — so thousands of client connections share a few dozen server connections, and size the server side near the HikariCP rule of thumb of about two connections per CPU core. Transaction pooling breaks session state (session `SET`, advisory locks, `LISTEN`), so keep such state out of the session. A short `pool_timeout` turns database saturation into fast errors that load shedding can handle (Chapter 11) instead of a pile of waiting threads.

```python
from sqlalchemy import create_engine, text

# Production: "postgresql+psycopg://app@pgbouncer:6432/orders"
engine = create_engine(
    "sqlite:///encore.db",
    pool_size=10,         # connections kept open per process
    max_overflow=5,       # extra connections allowed under bursts
    pool_timeout=2,       # seconds to wait, then fail fast
    pool_pre_ping=True,   # test a connection before handing it out
)
with engine.connect() as conn:
    print(conn.execute(text("SELECT 1")).scalar())   # 1
```

12.11 Schema migrations

A migration changes the schema while old and new code run side by side during a deploy, so every step must be compatible with both. Use **expand and contract**: add the new column or table, deploy code that writes both, backfill in small batches, switch reads, then drop the old shape in a later release (Chapter 23). Keep migrations versioned in the repository (Alembic, Flyway, Liquibase) and know which statements lock. In Postgres:

```sql
SET lock_timeout = '2s';             -- give up rather than queue behind
ALTER TABLE orders ADD COLUMN currency text DEFAULT 'EUR';  -- metadata only
CREATE INDEX CONCURRENTLY orders_event_idx ON orders (event_id);
ALTER TABLE orders ADD CONSTRAINT orders_total_pos
  CHECK (total >= 0) NOT VALID;      -- new rows checked at once
ALTER TABLE orders VALIDATE CONSTRAINT orders_total_pos;  -- light lock
```

Adding a column with a constant default has been metadata-only since Postgres 11; `CREATE INDEX CONCURRENTLY` builds without blocking writes (it cannot run inside a transaction block); MySQL offers `ALGORITHM=INSTANT` for many column changes and tools such as gh-ost for the rest. Never run a schema migration during the flash sale.

12.12 In Encore

Service	Store (AWS)	Why
Catalog	Aurora PostgreSQL + read replicas, cache	relational, read-heavy, joins
Search	OpenSearch, fed by CDC	full text, facets; derived
Seat inventory	Redis holds with TTL + Postgres seat rows	fast holds, durable truth
Orders, payments	Aurora PostgreSQL	ACID, constraints, ledger
Notifications	SQS + DynamoDB delivery log	key-value status, dedup
Waiting room	Redis (ElastiCache)	counters, queue position
Ticket PDFs	S3	immutable blobs
Clickstream	S3 + Iceberg tables	columnar scans (Chapter 21)
Support agent	Postgres + pgvector, DynamoDB	policy search, run checkpoints

The hot spot is not size. The flash sale's 50,000 seats at 100 bytes are 5 MB, and even the upper bound for all events (300,000 events of 80,000 seats) is 2.4 TB; orders grow by $5 \times 10^7 \times 2$ KB = 100 GB a year. The problem is 30,000 conditional writes per second on one event's rows, which is why holds go to Redis first (Chapter 9) with a durable record behind them, and why the inventory is partitioned by event and section (Chapter 17).

The support agent from *Designing Agentic Systems* is one more database client and should be treated as an untrusted one: it reads orders through a read replica with a read-only role scoped to the customer in the conversation, and it never writes rows directly — a refund is a tool call to the orders API, carrying an idempotency key, that runs the same short transaction as any other refund. Its retrieval index (policy documents as embeddings) lives in pgvector next to the data, and its run state is checkpointed in DynamoDB so that a crashed agent worker resumes instead of repeating a refund.

12.13 Numbers to remember

Figure	Value
EBS gp3 baseline	3,000 IOPS, 125 MiB/s
Postgres / InnoDB page	8 KB / 16 KB
B-tree height for 250 M rows	4 levels
LSM leveled write amplification	~T x L (worst ~50 at 1 TB)
S3 durability (designed)	11 nines
S3 single PUT / multipart	5 GB / 10,000 parts, ~50 TB
S3 per-prefix rate	3,500 writes/s, 5,500 reads/s
Encore orders	100 GB a year
Encore ticket PDFs	2.5 TB a year

12.14 Pitfalls

- Read-modify-write in application code -> a conditional `UPDATE ... WHERE status = 'free'` or `SELECT ... FOR UPDATE`.
- Assuming Repeatable Read prevents every anomaly -> it allows write skew; materialise the conflict or use Serializable with retries.
- Holding a transaction open across an external call -> charge outside, record the result in a short transaction.
- Blobs in the database -> object storage plus a key in the row.
- One pool per pod with no pooler -> PgBouncer or RDS Proxy, and a short pool timeout.
- A blocking `ALTER TABLE` at peak -> `lock_timeout`, concurrent index builds, expand and contract.
- Using search or a cache as the source of truth -> keep one system of record; derive the rest.

12.15 Remember

- The WAL is flushed before a commit is acknowledged; after a crash, replaying it restores every committed change.
- B-trees favour reads and ranges; LSM-trees favour writes and pay with compaction.
- Choose the data model from the access pattern; start relational for money and inventory.
- Isolation levels differ by database; fix lost updates with conditional writes and write skew with a materialised conflict or Serializable.
- Put blobs in object storage, connections behind a pooler, and migrations through expand and contract.

13 Database types: relational, NoSQL and beyond

Chapter 12 opened the box: engines, indexes, transactions and isolation. This chapter steps back to the shelf of boxes. There are a dozen families of database, each built around one shape of data and one kind of question, and "NoSQL" is the name for the half that gave up SQL tables to gain something else. The question it answers: for each piece of Encore's data, which family fits, what does that family make cheap, what does it make impossible, and how do you model data when the database will not join tables for you? The whole chapter fits in one picture: the same order, stored six ways.

13.1 The picture

Think of a city's shops. A department store (the relational database) sells everything, and its staff will fetch items from three floors and assemble them for you; past a certain crowd size the single building is the limit. A key-cutting kiosk (key-value) does one thing in seconds and you can open a thousand of them. A filing cabinet of folders (document) keeps everything about one customer together. A library's card index (search) finds every book that mentions "stadium". You do not pick one shop for the whole city; you pick a shop for each errand.

```
one Encore order, stored by six families

relational   orders(id, user_id, total)  order_items(order_id, seat_id)
             -> joined at query time

key-value    "ORDER#o9" -> {status, total, seats: [...]}  (opaque blob)

document     { _id: "o9", user: "ana", seats: [{sec: "A12", ...}] }
             -> fields indexable, nested, one aggregate per document

wide-column  partition user_id=ana | rows sorted by order_time desc

graph        (ana)-[:BOUGHT]->(o9)-[:FOR]->(seat A12-R05-S17)

search       inverted index: "stadium" -> [o9, o12, ...]
```

The relational row is normalised and assembled by joins; every other shape stores data the way one particular query will read it. That trade — query flexibility for a predictable, scalable access path — runs through the whole chapter.

13.2 What NoSQL means

"NoSQL" was the name, and the Twitter hashtag, chosen for a 2009 San Francisco meetup on distributed, non-relational databases; it is now read as "not only SQL". The ideas came earlier, from two papers: Google's **Bigtable** (2006), a sorted, range-partitioned wide-column store, and Amazon's **Dynamo** (2007), a hash-partitioned, leaderless key-value store built to keep the shopping cart writable through failures. Three pressures drove them:

1. **Scale-out.** A relational primary scales up, one bigger machine at a time. Web-scale write rates needed data partitioned across hundreds of cheap machines (Chapter 17), and joins and multi-row transactions are exactly what partitioning makes expensive. Dropping them made the scale-out simple.
2. **Flexible schema.** A document or wide-column row can carry new fields without an `ALTER TABLE`; the schema lives in the application ("schema on read").
3. **Availability.** Dynamo-style stores accept writes on any replica during a network partition and reconcile later (eventual consistency, Chapter 16), choosing A over C in CAP terms (Chapter 18).

The labels have since blurred. PostgreSQL stores and indexes JSON (`jsonb`); MongoDB has had multi-document ACID transactions since 4.0; DynamoDB offers transactions and strongly consistent reads; and distributed SQL systems (Spanner, CockroachDB) partition relational tables with consensus. So the useful question is not "SQL or NoSQL?" but

three narrower ones: **what is the data's shape, what are the queries, and what consistency does each write need?**

> **Design question.** SQL or NoSQL for Encore's orders? A teammate says "NoSQL scales, SQL doesn't, so start on DynamoDB".
>
> Scale is a number, so compute it. Encore's orders are 100 GB and about 1.6 writes per second on average, 83 per second when a stadium sells out in ten minutes: one Postgres primary handles that with room to spare. The hard number is the flash sale's 30,000 hold attempts per second at the door, which the waiting room meters down to the 3,000 holds per second the seat store runs at, and those land on the seat store whichever family you pick. Relational wins where invariants and ad-hoc queries matter (orders, payments, refunds, organizer reports); a partitioned key-value store wins where the access path is one key and the volume is huge (sessions, delivery logs, agent checkpoints). Choose per dataset, and choose NoSQL for a measured limit, not a slogan. See also *50 System Designs* #25 (Ticket Booking).

13.3 A map of database families

Each family is defined by its data model, and the data model decides which queries are one cheap lookup and which are a full scan. The table is the map; the subsections give each family's picture, query style, consistency and scaling model, and its job in Encore.

Family	Best for	Bad at	Encore use
Relational	invariants, joins, ad-hoc SQL	write scale beyond one primary	orders, payments, catalog
Key-value	lookups by key at any scale	queries by anything else	sessions, holds, delivery log
Document	self-contained aggregates	cross-document joins	venue seat-map layouts
Wide-column	huge write rates, ordered rows	ad-hoc queries	scan and check-in log
Graph	multi-hop relationships	bulk aggregation	fraud rings of scalpers
Time-series	metrics, rollups, retention	updates, joins	service and sales metrics
Search	full text, facets, ranking	being the source of truth	event search
Vector	similarity search	exact queries	support-agent retrieval
Analytical (OLAP)	scans and aggregates	single-row writes	organizer dashboards
Distributed SQL	SQL plus scale-out, multi-region	single-row latency, cost	a global orders tier later

13.3.1 Relational

```
users              orders                    order_items
id | email         id | user_id | total      order_id | seat_id
---+-------        ---+---------+------      ---------+--------
u1 | ana@...  <--  o9 | u1      | 190   <--  o9       | A12-R05-S17
```

Rows in tables, one fact in one place, relationships by foreign key, and a declarative query language that joins at read time. Constraints (`UNIQUE`, `CHECK`, foreign keys) and ACID transactions enforce invariants inside the database, which is why money and inventory start here. The query planner lets you ask questions nobody anticipated, the property every other family gives up first. **PostgreSQL** and **MySQL** dominate; managed on AWS as RDS and Aurora (Cloud SQL and AlloyDB on GCP; Azure Database for PostgreSQL or MySQL). Scaling: vertical first, read replicas for reads (Chapter 16), then partitioning with Citus, Vitess or Aurora Limitless Database (Chapter 17). Weak at: one primary absorbs every write, and schema changes on huge tables need care (Chapter 12).

13.3.2 Key-value

```
"SESSION#s81"      -> {user: u1, exp: 1760000000}
"HOLD#h1"          -> {seats: [...], expires: 1000600}
hash(key) -> partition -> replica set
```

A distributed hash map: `GET`, `PUT`, `DELETE` by key (in Redis, `SET hold:42:A12:R05-S017 h1 NX EX 600` takes a seat for ten minutes), sometimes with a sort key for ranges. Because every operation names its key, the system can route it to one partition and scale almost linearly. **DynamoDB** (hash on partition key, sorted by sort key, 400 KB items; Bigtable or Firestore on GCP, Cosmos DB on Azure) is the managed default; **Redis** and **Valkey** keep everything in memory for sub-millisecond reads (Chapter 9); **etcd** is a small, strongly consistent Raft store for configuration and coordination (1.5 MiB request limit, 2 GiB default storage quota, 8 GiB suggested maximum), not for application data. Consistency ranges from eventual (default DynamoDB reads) to linearizable (etcd). Weak at: anything that is not "give me this key", which the modelling section below turns into an art.

13.3.3 Document

```
{"_id": "venue-7", "name": "Riverside Stadium",
 "sections": [{"id": "A12", "rows": 20, "seats_per_row": 25,
               "price_band": "gold", "polygon": [[0, 0], [40, 0]]}]}
```

A key-value store whose values the database understands: JSON-like documents with nested arrays, indexable on any field, queried by example (`find({"sections.price_band": "gold"})`) or through aggregation pipelines. A document holds a whole **aggregate** that is read and written together, so one read returns the venue with all its sections, no join. **MongoDB** (current release 9.0; documents up to 16 MiB; Amazon DocumentDB is AWS's compatible service), **Firestore** (1 MiB documents; now also offered with MongoDB compatibility), **Couchbase** (queried with SQL++) and Azure Cosmos DB. MongoDB scales with a hashed or ranged shard key and replicates through replica sets; its default write concern is `w: "majority"`, and read concerns range from `local` to `linearizable`. Weak at: data shared across documents (duplicate it and keep the copies in step), and documents that grow without bound, such as an array of every ticket sold.

13.3.4 Wide-column

```
partition key: event_id=42          (hashed -> nodes)
clustering:    scanned_at DESC, ticket_id
  10:41:07 t811 gate-3 | 10:41:06 t044 gate-1 | 10:41:06 t530 gate-3 ...
```

A two-level map: a partition key that is hashed to a node, then rows sorted by clustering columns inside the partition. Writes go to an LSM-tree (Chapter 12), so append-heavy workloads at hundreds of thousands of writes per second are routine. **Apache Cassandra** (5.0 is the current major release; Amazon Keyspaces is AWS's compatible serverless service) and **ScyllaDB** (a C++ rewrite; its last open-source AGPL release was 6.2, and it is now source-available) are leaderless with tunable consistency; **Bigtable** and **Apache HBase** are range-partitioned with one server per tablet or region, so single-row operations are strongly consistent within a cluster. The query language (CQL) looks like SQL but only answers queries that name a partition. Weak at: ad-hoc queries, joins, and partitions that grow without bound; Encore would use it for the gate scan log at the stadium, written once per ticket and read per event.

13.3.5 Graph

```
(fan:ana)-[:USED_CARD]->(card:c7)<-[:USED_CARD]-(fan:bot91)
(fan:bot91)-[:SAME_DEVICE]->(fan:bot92)-[:HOLDS]->(seat:A12-R05-S18)
```

Nodes and edges with properties, stored so that following an edge is a pointer hop rather than an index lookup and join. Queries are patterns: Cypher and the ISO standard **GQL** (2024), Gremlin, SPARQL for RDF. **Neo4j**, **Amazon Neptune** (Neptune Analytics for in-memory graph algorithms), Spanner Graph on GCP and Cosmos DB's Gremlin API on Azure. Graphs scale up more easily than out, because a multi-hop query on a partitioned graph crosses machines at every hop. Weak at: bulk aggregation and simple key lookups. For Encore, a graph links accounts, cards, devices and IP addresses to find scalper rings before a flash sale. The sibling book *Designing Agentic Systems* has a full chapter on graph databases and knowledge graphs.

```
MATCH (a:Fan)-[:USED_CARD|SAME_DEVICE*1..3]-(b:Fan)
WHERE a.id = 'bot91' RETURN DISTINCT b.id
```

13.3.6 Time-series

```
series: holds_per_second{service="seat-inventory", region="eu"}
(t, v): (10:00:00, 2981) (10:00:15, 3010) (10:00:30, 2994) ...
```

Append-only points keyed by a series (a metric name plus labels) and a timestamp, compressed by delta-of-delta encoding of times and XOR encoding of values, kept hot for days and rolled up or dropped after that. Queries are windows and aggregates, such as PromQL's `sum by (code) (rate(http_requests_total[1m]))` or a 99th percentile per hour. **Prometheus** (pull-based metrics with PromQL; Amazon Managed Service for Prometheus), **InfluxDB 3** (Parquet files on object storage, SQL and InfluxQL; Amazon Timestream for InfluxDB), **TimescaleDB** (a Postgres extension; the company is now called Tiger Data). Weak at: updates, joins, and high-cardinality labels: a label per user ID multiplies the series count by 20 million. Encore's service metrics (request rates, latency, queue lag) live here; per-event sales analytics go to the columnar store below.

13.3.7 Search engines

An inverted index from terms to documents (Chapter 12), with scoring (BM25), facets, typo tolerance and geo filters (Chapter 19), queried with a JSON query DSL that combines scored `match` clauses with exact `filter` clauses. **Elasticsearch** (AGPL added as a license option in 2024) and **OpenSearch** (the Apache-licensed fork, now under the Linux Foundation; Amazon OpenSearch Service) shard each index and replicate each shard; refresh makes new documents searchable after about a second by default, so search is near-real-time, not read-your-writes. Weak at: being the system of record. Encore's event search is fed from the catalog by change data capture (Chapter 14) and can be rebuilt from it at any time.

13.3.8 Vector databases

Each item is an embedding, a vector of a few hundred to a few thousand numbers, and the query is "the *k* nearest vectors to this one", answered approximately by an HNSW graph or IVF index (Chapter 19). **pgvector** keeps vectors next to relational data in Postgres; **Pinecone** (serverless, managed only), **Milvus** (open source, Zilliz Cloud managed) and **Weaviate** are dedicated engines; **Amazon S3 Vectors** stores vectors in a bucket type built for cheap, infrequent queries, and DynamoDB and OpenSearch both offer vector indexes. Weak at: exact answers and selective filters, since approximate search plus a strict filter can return fewer than *k* results. Encore's support agent retrieves refund-policy passages by meaning; *Designing Agentic Systems* covers retrieval in depth.

13.3.9 Analytical and columnar

```
row store:     [o9, ana, 190, 10:00] [o10, ben, 95, 10:00] ...
column store:  user:  ana ben cat ...    total: 190 95 95 ...
               -> read 2 columns of 40, compressed 5-10x
```

Data stored column by column, compressed, and scanned by vectorized engines that read only the columns a query touches. The workload is OLAP: few queries, each aggregating billions of rows. **ClickHouse**, **Apache Druid** and **Apache Pinot** serve sub-second dashboards on fresh data; **BigQuery**, **Snowflake**, **Amazon Redshift** and **Databricks SQL** are warehouses and lakehouses (Chapter 21). Weak at: single-row updates and high-concurrency point lookups. The organizers' live sales dashboard reads a ClickHouse or Pinot table fed from the order stream, never the orders primary.

```
SELECT toStartOfMinute(paid_at) AS m, count() AS tickets, sum(total)
FROM orders_stream WHERE event_id = 42
GROUP BY m ORDER BY m;
```

13.3.10 Distributed SQL

Relational tables cut into ranges, each range replicated by Raft or Paxos (Chapter 18), with SQL and serializable or snapshot transactions spanning ranges. **Spanner** (TrueTime, external consistency), **CockroachDB** (Postgres wire protocol; no longer open source since 2024, with a free enterprise tier for small companies), **YugabyteDB** (Postgres-

compatible, Apache 2.0), **TiDB** (MySQL-compatible) and **Aurora DSQL** (serverless, Postgres-compatible, multi-region strong consistency, optimistic concurrency that aborts conflicting transactions with SQLSTATE `40001`). The cost is latency: every commit waits for a quorum, across regions if the replicas span them, and hot keys still serialize. Encore would reach for this when orders must be writable in both regions at once.

13.3.11 In-memory, embedded, ledger and multi-model

- **In-memory.** Redis, Valkey, Memcached and Amazon MemoryDB (Redis-compatible with a durable multi-AZ log) trade capacity for microsecond access; SAP HANA does the same for SQL (Chapter 9).
- **Embedded.** A library inside the process, no server: **SQLite** (the most deployed database; mobile apps, edge caches), **RocksDB** (the LSM engine under Kafka Streams state stores and Flink's large state), **DuckDB** (in-process OLAP over Parquet). Encore's mobile app caches tickets in SQLite so a QR code opens with no signal at the gate.
- **Ledger.** An append-only, cryptographically chained history that proves nothing was altered. Amazon QLDB reached end of support in July 2025, and AWS points users to Aurora PostgreSQL; Azure SQL Database ledger tables and immudb remain. Encore keeps its money ledger as append-only double-entry rows in Postgres.
- **Multi-model.** One engine, several APIs: Azure Cosmos DB (NoSQL, MongoDB, Cassandra, Gremlin and PostgreSQL APIs), ArangoDB, SurrealDB, and Postgres itself with `jsonb`, PostGIS, pgvector and TimescaleDB. Fewer systems to run, but each model is rarely the best of its kind.

13.4 Modelling NoSQL from access patterns

A relational schema is designed from the entities and queried any way you like later. A NoSQL schema is designed the other way round, from the queries, because a query that does not match a key is a full scan. The method:

1. **List every access pattern with its rate and consistency need**, before drawing a single table.
2. **Choose the partition key** for even spread and for locality: everything the hot query needs shares one partition key (Chapter 17 covers hot keys and salting).
3. **Choose the sort key** so the query is a range inside that partition: a prefix (`begins_with`), a date range, newest first.
4. **Denormalise.** Copy the attributes a query displays into the item it reads (the section's price band onto each seat, the event name onto each order), and accept that an update must touch every copy.
5. **Add a secondary index** for each remaining access path, knowing it is another write and, in DynamoDB, eventually consistent.

The duplication has a price you can compute. With g global secondary indexes that each project an item of size s, one logical write costs

$$\text{WCU per write} = (1 + g)\ \frac{s}{1\ \text{KB}}$$

so a 2 KB order with two GSIs costs 6 write units instead of 2. Denormalisation moves work from every read to every write; it pays when reads outnumber writes, which is almost always.

13.4.1 DynamoDB single-table design for Encore

Single-table design puts several entity types in one DynamoDB table, with generic key names `PK` and `SK` and type prefixes in the values, so that items read together share a partition key and come back in one `Query`, the NoSQL replacement for a join. Here are Encore's events, sections, seats, holds and orders:

PK	SK	Attributes	GSI1PK / GSI1SK
`EVENT#42`	`META`	name, venue, starts	`VENUE#7` / `2026-11-20#EVENT#42`
`EVENT#42`	`SEC#A12`	price 95, seats 500	
`EVENT#42#SEC#A12`	`SEAT#R05#S017`	status, hold, expires	
`HOLD#h1`	`META`	user, seats, expires (TTL)	
`ORDER#o9`	`META`	status, seats, total	`USER#ana` / `ORDER#2026-11-01T10:00:03Z`

And the queries they serve:

Access pattern	Operation	Key condition	Read
Event page with sections	`Query`	`PK = EVENT#42`	eventual, cached
Events at a venue by date	`Query` GSI1	`VENUE#7`, `SK >= 2026-10-14`	eventual
Seat map of a section	`Query`	`EVENT#42#SEC#A12`, `SEAT#` prefix	eventual, cached
Hold seats	`TransactWriteItems`	seats free + new `HOLD#`	conditional
Buy	`TransactWriteItems`	seats held by me + new `ORDER#`	conditional
Payment webhook	`GetItem`	`ORDER#o9`, `META`	strongly consistent
My orders, newest first	`Query` GSI1	`USER#ana`, descending	eventual

Three choices carry the design. Seats are partitioned by `event#section`, not by event, so the stadium show spreads over 100 partition keys and a section's 500 seats come back in one `Query`. Orders are keyed by order ID in the base table, because the payment provider's webhook arrives milliseconds after the order is written and needs a strongly consistent read, which a GSI cannot give; the user's order history, which tolerates a second of lag, uses the GSI. And holds carry an `expires` attribute that the conditions check, because DynamoDB's TTL deletes expired items "within a few days", not on the second. The simulation below models the table in memory, with all-or-nothing conditional transactions and a GSI that catches up asynchronously:

```
class Table:
    """Toy DynamoDB table: items by (PK, SK), one GSI updated async."""
    def __init__(self):
        self.parts, self.gsi, self.lag = {}, {}, []

    def get(self, pk, sk):
        return self.parts.get(pk, {}).get(sk)

    def query(self, pk, prefix="", forward=True, index=False):
        coll = (self.gsi if index else self.parts).get(pk, {})
        keys = sorted(k for k in coll if k.startswith(prefix))
        return [coll[k] for k in (keys if forward else reversed(keys))]

    def transact(self, ops):
        """All-or-nothing. ops: (pk, sk, condition(old), new or None)."""
        for pk, sk, cond, _ in ops:
            if not cond(self.get(pk, sk)):
                raise ValueError(f"ConditionalCheckFailed {pk} {sk}")
        for pk, sk, _, new in ops:
            if new is None:
                self.parts[pk].pop(sk, None)
                continue
            self.parts.setdefault(pk, {})[sk] = new
            if "GSI1PK" in new:              # index catches up later
                self.lag.append(new)

    def replicate(self):
        for it in self.lag:
            self.gsi.setdefault(it["GSI1PK"], {})[it["GSI1SK"]] = it
        self.lag.clear()


ANY = lambda old: True
NEW = lambda old: old is None              # attribute_not_exists(PK)
SEC = "EVENT#42#SEC#A12"                    # one section's partition
db = Table()
db.transact([("EVENT#42", "META", NEW, {"name": "Stadium show"}),
             ("EVENT#42", "SEC#A12", NEW, {"price": 95, "seats": 500})]
            + [(SEC, "SEAT#R05#S01" + n, NEW, {"status": "free"})
               for n in "78"])

def hold(user, hold_id, seats, now, ttl=600):
    def free(old):                           # free, or a hold that expired
        return old["status"] == "free" or (
            old["status"] == "held" and old["expires"] < now)
    ops = [(SEC, "SEAT#" + s, free, {"status": "held", "hold": hold_id,
            "expires": now + ttl}) for s in seats]
    ops.append(("HOLD#" + hold_id, "META", NEW,
                {"user": user, "seats": seats, "expires": now + ttl}))
    db.transact(ops)

def buy(hold_id, order_id, ts, now):
    if db.get("ORDER#" + order_id, "META"):  # retried request: done
        return "already bought"
    h = db.get("HOLD#" + hold_id, "META")
    mine = lambda old: old["hold"] == hold_id and old["expires"] >= now
    ops = [(SEC, "SEAT#" + s, mine, {"status": "sold", "order": order_id})
           for s in h["seats"]]
    ops += [("ORDER#" + order_id, "META", NEW,
             {"status": "pending_payment", "seats": h["seats"],
              "GSI1PK": "USER#" + h["user"], "GSI1SK": "ORDER#" + ts}),
            ("HOLD#" + hold_id, "META", ANY, None)]
    db.transact(ops)
    return "bought"

hold("ana", "h1", ["R05#S017", "R05#S018"], now=1000)
try:
    hold("ben", "h2", ["R05#S018"], now=1001)          # same seat
```

```
except ValueError as e:
    print("ben:", e)
print(buy("h1", "o9", "2026-11-01T10:00:03Z", now=1200))
print(buy("h1", "o9", "2026-11-01T10:00:03Z", now=1201))  # client retry
print([i["status"] for i in db.query(SEC, "SEAT#")])
print(db.get("ORDER#o9", "META")["status"])            # webhook: base table
print(len(db.query("USER#ana", "ORDER#", False, index=True)))  # GSI lags
db.replicate()
print(len(db.query("USER#ana", "ORDER#", False, index=True)))
print([k for k in db.parts["EVENT#42"]])               # event + sections
```

```
ben: ConditionalCheckFailed EVENT#42#SEC#A12 SEAT#R05#S018
bought
already bought
['sold', 'sold']
pending_payment
0
1
['META', 'SEC#A12']
```

Ben's hold fails as a whole, not seat by seat; the client's retried purchase is answered from the existing order, and the `attribute_not_exists` condition on `ORDER#o9` would reject a duplicate even if two retries raced; the webhook sees the order at once; the GSI shows nothing until it catches up; and one `Query` on `EVENT#42` returns the event with its sections, pre-joined. The real calls are `TransactWriteItems` with a `ConditionExpression` such as `#s = :free OR (#s = :held AND expires < :now)`, at most 100 items and 4 MB per transaction, each item costing two write units.

> **Design question.** How would you model the seat map in DynamoDB, and can it survive the flash sale?
>
> Use the table above: one item per seat under `EVENT#42#SEC#A12`. A section is 500 seats of 100 bytes, about 49 KB, so one `Query` costs 13 read units strongly consistent or 6.5 eventually consistent, and the whole 100-section map 650. Served straight from DynamoDB, 200,000 map views per second would need 130 million read units per second, 1.3 million on each section's partition against a limit of 3,000: impossible. So one refresher per section reads it about once a second (650 read units per second in all) into a cache and the CDN. Writes: a hold is a transaction of a seat item and a hold item, 4 write units. Unmetered, the 30,000 attempts per second at the door would need 120,000 write units, three times the default per-table quota of 40,000. Behind the waiting room the store runs at 3,000 holds per second (12,000 write units) and is load-tested at 6,000 (24,000): inside the quota, provided the table is pre-warmed before 10:00. Per partition, a floor section drawing 10 percent of 6,000 holds uses 1,200 write units against a limit of 1,000, so salt it by row block; failed conditional writes still consume capacity. The design works only with admission control in front; Encore takes holds in Redis first and records them durably behind it. See also *50 System Designs* #25 (Ticket Booking).

13.4.2 Cassandra: one table per query

Cassandra's secondary indexes, including the storage-attached indexes (SAI) of 5.0, are local to each node, so a query without the partition key still asks every node; on the hot path the rule is one table per query, written together. The partition key is in the inner parentheses, clustering columns order the rows inside it, and a time bucket keeps any one partition bounded (a common guideline is under about 100 MB):

```
CREATE TABLE scans_by_event_day (
  event_id int, day date, scanned_at timestamp, ticket_id text,
  gate text,
  PRIMARY KEY ((event_id, day), scanned_at, ticket_id)
) WITH CLUSTERING ORDER BY (scanned_at DESC, ticket_id ASC);

CREATE TABLE scans_by_ticket (
  ticket_id text PRIMARY KEY, event_id int, scanned_at timestamp,
  gate text);

-- "has this ticket been scanned?" needs a different partition key
INSERT INTO scans_by_ticket (ticket_id, event_id, scanned_at, gate)
VALUES ('t811', 42, '2026-11-20 19:41:07', 'gate-3') IF NOT EXISTS;
```

The `IF NOT EXISTS` turns the insert into a lightweight transaction, a Paxos round at `SERIAL` consistency, so a ticket copied to two phones gets in once. Querying `scans_by_event_day` without both `event_id` and `day` is rejected unless you add `ALLOW FILTERING`, which means a scan.

13.5 Consistency options

NoSQL systems let you choose consistency per request, and the default is usually the weaker one.

DynamoDB. Reads are eventually consistent by default and cost half a read unit per 4 KB; `ConsistentRead=true` returns the latest committed write for one full unit. Strong reads work on the table and local secondary indexes only; global secondary indexes and streams are always eventually consistent. Every read sees only committed data (read-committed), conditional writes are atomic per item, and transactions are ACID within one region. Global tables replicate across regions in one of two modes: multi-Region eventual consistency (MREC, the default, typically within a second, last writer wins) or multi-Region strong consistency (MRSC, the write is replicated synchronously to another region before it returns).

Cassandra. Every request carries a consistency level, the number of replicas that must answer. With replication factor *RF*, a quorum is

$$Q = \frac{RF}{2} + 1, \qquad R + W > RF \quad \text{reads see the latest write}$$

With $RF = 3$, `QUORUM` reads and writes each wait for 2 replicas, and $2 + 2 > 3$ guarantees they overlap (Chapter 16 simulates it).

Level	Must answer (2 DCs, RF 3 each)	Use
`ONE` / `LOCAL_ONE`	1 replica (local DC)	metrics, logs
`QUORUM`	4 of all 6	single-DC strong reads
`LOCAL_QUORUM`	2 in the local DC	the usual multi-DC default
`EACH_QUORUM`	2 in every DC	writes that must be everywhere
`ALL`	6	rarely: one replica down fails it
`ANY`	a stored hint (writes only)	almost never
`SERIAL` / `LOCAL_SERIAL`	Paxos quorum	`IF` lightweight transactions

MongoDB. Writes default to `w: "majority"`; reads choose a read concern (`local`, `majority`, `linearizable`, `snapshot`) and a read preference, and reading from secondaries returns data that may lag the primary.

13.6 Limits that bite

Limit	Value	Fix
DynamoDB item	400 KB	large fields to S3, key in the item
MongoDB / Firestore document	16 MiB / 1 MiB	bounded arrays; split documents
DynamoDB partition (and one item)	3,000 RCU, 1,000 WCU per s	finer keys, salting, cache
DynamoDB table, on demand	40,000 read and write units (default)	quota increase, pre-warming
DynamoDB `Query` page	1 MB	paginate with `LastEvaluatedKey`
DynamoDB transaction	100 items, 4 MB	smaller units, sagas
DynamoDB indexes	20 GSIs (default), 5 LSIs	fewer access paths, overloaded GSIs
Item collection with an LSI	10 GB, cannot split	prefer GSIs
Cassandra partition	~100 MB (guideline)	time buckets in the key
Joins	none (DynamoDB, Cassandra)	denormalise, pre-joined collections

Two more costs are easy to miss. A `Scan` or a `FilterExpression` reads, and bills, every item it examines before discarding most of them. And a hot partition is throttled even when the table as a whole is far below its capacity: adaptive capacity can isolate one busy item on its own partition, but that partition still tops out at 1,000 writes per second.

13.7 When NoSQL is the wrong choice

- **The queries are not known yet,** or analysts will ask new ones every week. Relational or a warehouse answers questions nobody planned; a key-designed table answers only the ones you designed.
- **Invariants span entities:** a refund that must reverse a payment, release seats and post ledger entries together, or uniqueness on something other than the key. Constraints and multi-row transactions do this in one statement.
- **Many-to-many relationships with reporting** (organizers, events, promoters, payouts): duplicated data drifts out of sync.
- **The data is small.** Under a few terabytes and a few thousand writes per second, one Postgres primary with replicas is simpler, cheaper and more flexible.
- **The team has not written down its access patterns.** Without that list, a single-table design turns into scans and filter expressions.

The reverse also holds: relational is the wrong choice for a firehose of appends keyed by device or time, for unbounded key-value lookups at hundreds of thousands per second, and for full-text or similarity search.

13.8 Choosing a database

Requirement	Family	Why
Money, inventory, constraints	relational	ACID, constraints, any query
Same, but multi-region writes	distributed SQL	consensus per range
Lookup by key, any volume	key-value	one partition per request
Sub-millisecond, loss-tolerant	in-memory	RAM, no disk on the path
Aggregate read as a whole	document	one read, flexible schema
Append firehose by key and time	wide-column	LSM writes, bounded partitions
Paths through relationships	graph	edge hops, not joins
Metrics with retention	time-series	compression, rollups
Text, facets, typos	search	inverted index, ranking
Meaning, not keywords	vector	nearest-neighbour index
Aggregates over billions of rows	columnar OLAP	reads only needed columns
Offline, on device	embedded	no server, no network

13.9 In Encore

Encore runs **polyglot persistence**: one system of record per dataset, chosen from that dataset's access pattern, with every other copy derived from it by change data capture (Chapter 14).

Data	System of record (AWS)	Derived copies
Orders, payments, ledger	Aurora PostgreSQL	OpenSearch (support), ClickHouse
Catalog: events, venues	Aurora PostgreSQL	OpenSearch, CDN pages
Venue seat-map layouts	`jsonb` documents in Aurora	JSON on the CDN
Seat holds	Redis (ElastiCache), TTL	Postgres seat rows (durable)
Notification delivery log	DynamoDB	none

Data	System of record (AWS)	Derived copies
Gate scans at the stadium	Amazon Keyspaces	S3 for analytics
Scalper graph	Neptune	none
Service metrics	Managed Prometheus	none
Sales dashboards	ClickHouse, fed by Kafka	none
Tickets on the phone	SQLite on device	from the orders API
Agent policies, checkpoints	pgvector, DynamoDB	none

The DynamoDB seat design above is the alternative when the seat writes outgrow one Postgres primary; the storage chapter's choice of Redis plus Postgres rows holds today. Every extra family is one more system to run, secure, back up and page someone for, so each must earn its place with a requirement the existing ones cannot meet.

External systems and agents are database clients too. The payment provider only knows Encore's order ID, so the webhook handler needs a key lookup with a strongly consistent read, never a GSI or a search index. The support agent reads orders through a narrow, read-only tool that issues one keyed query per call; a tool that lets a model compose free-form queries against DynamoDB turns into `Scan` bills and against Postgres into a security review. Its memory spans three families at once: run checkpoints in a key-value table keyed by conversation, policy passages in a vector index, and the order data itself in relational tables. *Designing Agentic Systems* covers agent memory and retrieval.

13.10 Numbers to remember

Fact	Value
DynamoDB item limit	400 KB
DynamoDB per partition	3,000 RCU, 1,000 WCU per s
Read unit / write unit	4 KB strong (8 KB eventual) / 1 KB
Transactional item	2 units
DynamoDB transaction	100 items, 4 MB
`Query` page	1 MB
On-demand table default	40,000 read and write units per s
MongoDB / Firestore document	16 MiB / 1 MiB
Cassandra quorum, RF 3	2 replicas
etcd request / default quota	1.5 MiB / 2 GiB
Encore section seat map	~49 KB, 13 RCU strong
Encore holds, run / tested	3,000/s, 12,000 WCU / 6,000/s, 24,000 WCU

13.11 Pitfalls

- Choosing NoSQL for "scale" before computing the load -> estimate size and write rate; start relational unless a number says otherwise.
- Designing DynamoDB tables from the entity diagram -> list access patterns first, then keys.
- Reading a GSI right after a write -> base-table `GetItem` with `ConsistentRead` when correctness depends on it.
- Relying on TTL to expire holds on time -> store `expires` and check it in the condition.
- Unbounded partitions or documents -> time buckets, bounded arrays, child items.
- `Scan` and `FilterExpression` on the hot path -> a key or index that answers the query.
- Search, cache or vector index as the system of record -> derive them by CDC and keep them rebuildable.
- Picking a product without checking its status -> verify licence and support; QLDB and ScyllaDB open source both ended.

- One family per new feature -> each store must justify its operational cost.

13.12 Remember

- “NoSQL” traded joins, multi-row transactions and ad-hoc queries for scale-out, flexible schema and availability; the lines have blurred since.
- Pick a family from the data’s shape, the queries and the consistency each write needs, one dataset at a time.
- In NoSQL, design from access patterns: partition key for spread and locality, sort key for ranges, denormalise, add indexes knowingly.
- Know the limits that bite: 400 KB items, 3,000 and 1,000 units per partition, eventual GSIs, no joins.
- Relational stays the default for money and inventory; Encore is polyglot, with one system of record per dataset and everything else derived.

14 Message queues, streams and workers

When a fan buys a ticket, Encore has to charge the card, render a ticket PDF, send an e-mail and a push notification, update the organizer's dashboard and feed the analytics pipeline. Only the first must happen before the fan sees "You're in". This chapter answers one question: how do you move the rest of the work off the request path, so that it happens later, exactly as often as it should, in the right order, and without losing anything when a machine dies? The picture is a conveyor belt between the service that produces work and the workers that consume it.

14.1 The picture

A **queue** is the ticket rail in a restaurant kitchen. Waiters clip orders on one end; whichever cook is free takes the next one; once the dish leaves the pass, the slip goes in the bin. A **log** (or stream) is a ship's logbook. Entries are appended in order and never torn out; every reader keeps a bookmark, and two readers can be on different pages of the same book.

```
QUEUE: a message is taken by ONE worker, deleted after the ack
  producer -> [ m5 m4 m3 m2 m1 ] -+-> worker A
                                  +-> worker B
LOG: records stay; every consumer group keeps its own offset
  producer -> | 0 | 1 | 2 | 3 | 4 | 5 | -> (appends at the end)
                    ^           ^
         analytics offset=1    e-mail offset=4
```

14.2 Why decouple

Putting a buffer between two services buys four things:

- **Latency.** The checkout handler writes one message (a few milliseconds) instead of waiting for a 50 KB PDF render and an e-mail provider round trip.
- **Burst absorption.** Producers can briefly outrun consumers; the queue holds the difference and workers drain it at their own pace.
- **Failure isolation.** When the e-mail provider is down, checkout keeps selling; messages wait.
- **Fan-out and independent scaling.** One "order paid" event feeds tickets, e-mail, push and analytics, each scaled on its own.

The numbers show why. A synchronous chain fails if any link fails, so availabilities multiply. Checkout at 99.95% calling an e-mail provider at 99.9% inline is available

$$A_{\text{chain}} = A_1 \quad A_2 = 0.9995 \times 0.999 = 0.9985$$

which is about 13 hours of downtime a year instead of 4.4. With a queue in between, the provider's outages become delay, not errors.

A queue only buffers the *difference* between arrival rate λ and service capacity $c\mu$ (c workers, each serving μ messages per second). Over a burst of length T the backlog grows by

$$B = (\lambda - c\mu)\,T \quad \text{when } \lambda > c\mu$$

Illustrative Encore flash sale: 50,000 seats sell in 10 minutes, so ticket PDFs arrive at $\lambda = 50,000/600 \approx 83$ per second. With 10 PDF workers at 200 ms each ($c\mu = 50$ per second), the backlog grows by 33 per second and reaches 20,000 PDFs after the sale. Nothing is lost, but the last fan waits almost 7 minutes for a ticket (Chapter 2 explains why the wait explodes as utilization nears 100%).

Decoupling has a price: the caller no longer knows when, or whether, the work finished. You need status tracking, idempotent consumers, monitoring of lag and a story for messages that never succeed. Use a synchronous call when the caller needs the answer to continue (the payment authorization, the seat hold); use a queue when it needs only the promise that the work will happen.

14.3 Queues and logs

Two families of brokers look alike from the producer's side and behave very differently on the consumer's side.

	Queue	Log / stream
After consumption	Message deleted on ack	Record kept until retention
Consumers	Compete for messages	Each group reads everything
Ack unit	Per message	An offset per partition
Ordering	Best effort, or per group (FIFO)	Strict within a partition
Replay	No (once deleted, gone)	Yes, rewind the offset
Scaling unit	Add workers	Add partitions, then consumers
Examples	SQS, RabbitMQ, Celery	Kafka, Kinesis, Redis Streams

Point-to-point means one message, one consumer: a work queue. **Publish/subscribe** means one message, many independent subscribers: a topic. Queues become pub/sub through fan-out (an SNS topic or RabbitMQ exchange copying each message into one queue per subscriber); logs are pub/sub by nature because every consumer group keeps its own offset. Google Cloud Pub/Sub is a topic with per-subscription delivery and acks, so it behaves like fan-out queues with optional ordering and replay by seek.

Where they run:

- **Amazon SQS** (GCP Pub/Sub, Azure Service Bus queues): managed queue; standard queues are at-least-once with best-effort ordering; FIFO queues order per `MessageGroupId` and deduplicate within a 5-minute window. Messages up to 1 MiB, retained 4 days by default and up to 14.
- **RabbitMQ** (Amazon MQ): AMQP broker with exchanges routing into queues; quorum queues replicate with Raft; it also offers append-only streams.
- **Apache Kafka** (Amazon MSK; Google Cloud Managed Service for Apache Kafka; Azure Event Hubs speaks the Kafka protocol; Confluent Cloud): partitioned, replicated log; default retention 7 days. Since 4.0 it runs on KRaft with no ZooKeeper, and since 4.2 *share groups* (KIP-932) give queue-style per-record acks on top of a topic.
- **Amazon Kinesis Data Streams** (GCP Pub/Sub, Azure Event Hubs): managed log; a provisioned shard takes 1 MB/s or 1,000 records/s in and serves 2 MB/s out; retention 24 hours by default, up to 365 days.
- **Redis Streams**: a log inside Redis with consumer groups, good for modest volumes where Redis is already present (Chapter 9).

Choose a queue for *tasks* (send this e-mail, render this PDF): each must be done once by someone, and per-message retry and dead-lettering matter. Choose a log for *facts* (order paid, seat held, page viewed): many consumers want them, in order, and you want to replay history into a new consumer or rebuild a view.

14.4 Delivery semantics and idempotent consumers

Every consumer does two things: it processes a message and it tells the broker it is done. A crash can fall between them, and the order you choose decides what a crash costs.

```
at-most-once:     receive -> ACK -> process   crash after ack: lost
at-least-once:    receive -> process -> ACK   crash before ack: redelivered
effectively-once: at-least-once + idempotent processing
                  (a duplicate has no effect)
```

- **At-most-once** suits data where a loss is cheaper than a duplicate: metrics samples, typing indicators.
- **At-least-once** is the default of SQS, RabbitMQ with manual acks, Pub/Sub and Kafka consumers that commit after processing. Duplicates appear on consumer crashes, ack timeouts, rebalances and producer retries.
- **Exactly-once delivery** over a network is impossible in general: the broker cannot tell a slow consumer from a dead one. What systems sell as exactly-once is *effectively-once processing*: at-least-once delivery plus deduplication. Kafka's idempotent producer (on by default since 3.0, together with `acks=all`) drops producer retries using a producer ID

and sequence numbers; Kafka transactions commit output records and consumer offsets atomically for read-process-write pipelines inside Kafka; SQS FIFO drops a resend with the same deduplication ID within 5 minutes; Pub/Sub offers exactly-once delivery on pull subscriptions in one region. None of these covers a side effect outside the broker, such as an e-mail.

So the consumer itself must be **idempotent**: processing a message twice has the same effect as once. Three ways, in order of preference:

1. **Naturally idempotent writes**: `SET seat:A-12 sold`, an upsert keyed by order ID, "mark ticket issued" rather than "add one ticket".
2. **A dedupe record in the same transaction** as the side effect, keyed by the message ID or a business key.
3. **An idempotency key passed downstream** to external systems (the payment provider, the e-mail provider) so they deduplicate for you (Chapter 5).

```
import sqlite3

db = sqlite3.connect(":memory:")
db.executescript("""
CREATE TABLE processed (message_id TEXT PRIMARY KEY);
CREATE TABLE tickets (order_id TEXT, seat TEXT, PRIMARY KEY (order_id, seat));
""")

def handle(msg: dict) -> str:
    """Apply a message at most once, even if the broker delivers it twice."""
    with db:  # one transaction: dedupe mark and side effect commit together
        cur = db.execute(
            "INSERT INTO processed VALUES (?) ON CONFLICT DO NOTHING",
            (msg["id"],))
        if cur.rowcount == 0:
            return "duplicate, skipped"
        db.execute("INSERT INTO tickets VALUES (?, ?)",
                   (msg["order_id"], msg["seat"]))
        return "applied"

m = {"id": "evt-91", "order_id": "ord-7", "seat": "A-12"}
print(handle(m))   # applied
print(handle(m))   # duplicate, skipped (redelivery after a lost ack)
print(db.execute("SELECT count(*) FROM tickets").fetchone()[0])  # 1
```

The same SQL runs on Postgres. The dedupe table only needs to remember IDs for as long as a duplicate can arrive (the broker's retention or redelivery window). For Encore's notifications, about 137,000 orders a day with two messages each, kept 7 days at about 100 bytes per entry, is $137,000 \times 2 \times 7 \times 100$ bytes, about 192 MB: small enough for a Redis set with a TTL or a Postgres table pruned nightly.

14.5 Ordering, partitions and keys

A single ordered log does not scale past one machine, so logs are cut into **partitions**, each an independent ordered sequence on its own leader broker. Order is guaranteed only *within* a partition. The producer picks the partition from the record's key, so all records with one key land in one partition, in the order they were written:

$$p = \text{hash(key)} \bmod N$$

Kafka's default partitioner uses murmur2 on the key bytes; records without a key are spread in batches across partitions (sticky partitioning) and have no relative order. Kinesis hashes the partition key with MD5 into shard ranges; SQS FIFO orders per `MessageGroupId`; Pub/Sub per ordering key; RabbitMQ orders per queue, and a *single active consumer* keeps it that way.

```python
import zlib

def partition(key: str, n: int) -> int:
    # Kafka uses murmur2(key) & 0x7fffffff; crc32 shows the same idea
    return zlib.crc32(key.encode()) % n

for key in ("ord-7", "ord-9", "ord-7"):    # same key -> same partition
    print(key, partition(key, 12))
keys = [f"ord-{i}" for i in range(100_000)]
moved = sum(partition(k, 12) != partition(k, 13) for k in keys)
print(f"12 -> 13 partitions moves {moved / len(keys):.0%} of keys")
```

The last line prints 92%: with modulo hashing, going from N to $N + 1$ partitions moves about $N/(N + 1)$ of the keys, so per-key order breaks across the change. Size partitions up front. The count must cover the slower of producing and consuming:

$$N \geq max(\frac{T}{t_{\text{produce}}}, \frac{T}{t_{\text{consume}}})$$

where T is peak throughput and t the throughput one partition sustains on each side. Encore's clickstream is 1 billion events a day, 11,574 per second on average; at an illustrative 5x flash-sale peak and 1 KB per event that is 58 MB/s. If one consumer processes 5 MB/s (illustrative), $N \geq 12$; Encore creates 48 to leave room for 4x growth without repartitioning. On Kinesis the same load needs 58 provisioned shards at 1 MB/s each, or on-demand mode. Storage: 1 TB a day, so 7 days of retention with replication factor 3 holds 21 TB.

The key choice is a trade between order and spread. A key with few values, or one value much hotter than the rest, creates a hot partition that one consumer must handle alone (Chapter 17). Producer retries can also reorder records; Kafka's idempotent producer preserves order with up to 5 in-flight requests per connection.

> **Design question.** Every hold, release and sale of a seat publishes a `seat-changed` event that feeds seat maps, organizer dashboards and analytics. Which key?
>
> Consumers need the history of each seat in order: held, released, held by another fan, sold. Keying by `event_id` gives order across the whole show, but every event of the flash-sale show then lands in one partition: if failed hold attempts are published too, up to 30,000 per second in the first minute, which one consumer must handle alone. Keying by `seat_id` keeps per-seat order and spreads the 50,000 seats over all 48 partitions, about 625 events per second each at that rate. Dashboards sum counts across partitions, which needs no cross-seat order. The trade-off: no consumer sees a consistent snapshot of a whole section from the stream; anything that needs one (never overselling) reads the seat-inventory database, the source of truth. See also *50 System Designs* #12 (Distributed Log).

14.6 Consumer groups and rebalancing

A **consumer group** is a set of workers that share one subscription. In Kafka each partition is assigned to exactly one member of the group, which gives parallelism and per-key order at once; different groups (tickets, analytics) each read the whole topic independently.

```
topic order-events (6 partitions)       group "tickets" (3 members)
  P0 P1  ------------------------------>  consumer A
  P2 P3  ------------------------------>  consumer B
  P4 P5  ------------------------------>  consumer C
a 7th member in a 6-partition topic sits idle
```

$$\text{parallelism} = min(\text{consumers},\ \text{partitions})$$

Each member commits its position (the offset of the next record to read) to the internal `__consumer_offsets` topic. A **rebalance** reassigns partitions when a member joins, leaves, misses heartbeats for `session.timeout.ms` (45 s by default) or fails to call `poll()` within `max.poll.interval.ms` (5 minutes). The new owner resumes from the last

committed offset, so everything processed since that commit runs again. With auto-commit every 5 seconds (the default) and Encore's peak of about 1,200 clickstream records per second per partition, a crash replays up to about 6,000 records per partition: one more reason for idempotent consumers.

The classic protocol stopped every member during a rebalance. Cooperative assignment moves only the partitions that change owner, and Kafka 4.0 made the broker-driven consumer protocol of KIP-848 generally available (`group.protocol=consumer`), which reconciles assignments incrementally on the broker. *Static membership* (`group.instance.id`) lets a pod restart without triggering a rebalance at all, which matters during rolling deployments.

Redis Streams has the same idea in a small package: `XREADGROUP` hands each entry to one consumer, the entry stays in a *pending entries list* until `XACK`, and a healthy consumer takes over a dead one's entries with `XAUTOCLAIM`:

```
import fakeredis

r = fakeredis.FakeRedis(decode_responses=True)
for order in ("ord-1", "ord-2", "ord-3"):
    r.xadd("orders", {"order_id": order, "type": "paid"})

r.xgroup_create("orders", "tickets", id="0")      # group reads from the start
batch = r.xreadgroup("tickets", "w1", {"orders": ">"}, count=2)
ids = [mid for mid, _ in batch[0][1]]
r.xack("orders", "tickets", ids[0])               # w1 finishes ord-1, crashes
print(r.xpending("orders", "tickets")["pending"])  # 1: ord-2 is stuck on w1

# w2 claims entries idle longer than min_idle_time (60000 ms in production)
_, claimed, _ = r.xautoclaim("orders", "tickets", "w2",
                             min_idle_time=0, start_id="0-0")
print([f["order_id"] for _, f in claimed])        # ['ord-2']
```

Unlike Kafka, Redis Streams acks per entry and does not pin keys to consumers, so it is a queue on a log with no per-key order across consumers. Kafka's share groups make the same trade on Kafka: many consumers per partition, per-record acks and delivery counts, no ordering.

14.7 Acks, visibility timeouts and dead letters

A queue does not delete a message when a worker receives it. SQS hides it for a **visibility timeout** (30 s by default, up to 12 hours); the worker deletes it when done, and if the worker dies the message reappears for someone else. Pub/Sub calls this the ack deadline (10 s by default, up to 600 s, extended automatically by the client libraries); RabbitMQ redelivers when the channel closes, and its 30-minute delivery acknowledgement timeout (on quorum queues) closes channels that hold a message too long. Set the timeout above the p99 processing time times a safety factor, and extend it (heartbeat) for long jobs. Too short and a slow-but-alive worker gets a duplicate running beside it; too long and a crashed worker's message waits that long before anyone retries it.

A **poison message** fails every time: a malformed payload, a deleted order, a bug. Retried forever, it burns workers and, in an ordered queue or a Kafka partition, blocks everything behind it. Count deliveries and, after k attempts, move the message to a **dead-letter queue** (DLQ) for inspection and later redrive. If a transient failure hits a fraction p of attempts independently, a healthy message reaches the DLQ with probability

$$P(\text{dead-lettered}) = p^k \qquad p = 0.1,\ k = 5 \qquad 10^{-5}$$

so five attempts make false dead letters rare while a true poison message leaves within five tries. On SQS the DLQ is a redrive policy on the source queue (CloudFormation properties of `AWS::SQS::Queue`):

```
{
  "QueueName": "email-send",
  "VisibilityTimeout": 60,
  "RedrivePolicy": {
    "deadLetterTargetArn": "arn:aws:sqs:eu-central-1:111122223333:email-dlq",
    "maxReceiveCount": 5
  }
}
```

Pub/Sub has dead-letter topics (5 to 100 delivery attempts), RabbitMQ dead-letter exchanges (quorum queues cap redeliveries with a delivery limit); Sidekiq moves failing jobs through a retry set with exponential backoff into a dead set, and Celery re-queues with a countdown through `autoretry_for`. Kafka has no per-record ack, so a failing record stops its partition: the usual pattern publishes it to a retry topic (consumed after a delay) and then to a DLQ topic, then commits and moves on, accepting that this key's later records may overtake it. Alarm on DLQ depth above zero; a DLQ nobody watches is a slow way to lose data.

14.8 Lag and backpressure

Lag is how far consumers trail producers. In Kafka it is counted in records per partition (log-end offset minus committed offset); in SQS as visible messages and the age of the oldest one. Users feel lag in *seconds*, which Little's law converts ($L = \lambda W$):

$$W_{\text{lag}} = \frac{\text{lag in messages}}{\text{consumption rate}} \qquad c = \frac{\lambda S}{\rho} \qquad t_{\text{drain}} = \frac{B}{c\mu - \lambda}$$

c is the number of workers for arrival rate λ, service time S and target utilization ρ; t_{drain} is the time to clear a backlog B while new work keeps arriving. Encore's flash sale makes ticket PDFs at 83 per second at 200 ms each; at 70% utilization that is 24 workers, and the 7-minute wait from the start of the chapter disappears.

```
import math

def workers_needed(rate: float, service_s: float, util: float = 0.7) -> int:
    return math.ceil(rate * service_s / util)   # keep each below util

def drain_seconds(backlog: int, capacity: float, arrivals: float) -> float:
    if capacity <= arrivals:
        return math.inf          # the backlog never drains: scale out
    return backlog / (capacity - arrivals)

print(workers_needed(83.3, 0.2))                 # 24 PDF workers
print(round(drain_seconds(20_000, 100, 2)))      # 204 s of e-mail backlog
```

Autoscale on lag rather than CPU: KEDA scales Kubernetes deployments on Kafka lag or SQS depth, and AWS Lambda (GCP Cloud Run functions, Azure Functions) event source mappings for SQS and Kafka add pollers as the backlog grows. Alert on the *age* of the oldest message, which is what the fan experiences.

Backpressure is how a slow stage tells a fast one to slow down instead of drowning. Inside a consumer, bound the work in flight: RabbitMQ prefetch (`basic.qos`), Celery's prefetch multiplier, at most 10 messages per SQS receive, `pause()` on a Kafka partition. Toward producers, a bounded buffer blocks or rejects (Kafka's producer blocks on a full `buffer.memory`, then fails after `max.block.ms`), and the API returns 429 or 503 (Chapter 11). A queue without a bound converts overload into unbounded latency and, eventually, a full disk.

> **Design question.** Encore's e-mail provider is down for the 10 minutes of the flash sale. What happens to 20,000 order confirmations, and how fast do they go out afterwards?
>
> Checkout keeps selling: it only writes to the queue. Workers' sends fail, retry with exponential backoff and jitter, and the messages stay in SQS (4-day default retention, far above the outage). Retrying hard against a dead provider wastes calls and can trip its abuse limits, so a circuit breaker pauses the workers and probes. When the provider returns, the workers send at its illustrative account limit of 100 e-mails per second while about 2 new orders per second arrive: $20,000/(100 - 2) \approx 204$ s, under 4 minutes. Cap worker concurrency with a token bucket at the provider's limit, not at the queue depth, or the recovery itself triggers throttling. Push notifications use a separate queue so they are not stuck behind e-mail. See also *50 System Designs* #20 (Notification System).

14.9 The transactional outbox and change data capture

The payments service must mark an order paid *and* publish `order-paid`. Doing both as two separate writes is the **dual-write problem**: crash between them and either the database says paid with no event (no ticket ever issued) or the event goes out for a transaction that rolled back. There is no transaction spanning Postgres and Kafka.

The **transactional outbox** removes the second write from the request. The service inserts the event into an `outbox` table in the same local transaction as the state change; a separate relay publishes outbox rows and marks them sent.

```
 payments service --(one DB transaction)--> orders + outbox
                                                  |
                          relay or CDC reads outbox -+-> Kafka order-events
                                                       |-> ticket workers
                                                       |-> e-mail workers
```

```
import json, sqlite3

db = sqlite3.connect(":memory:")
db.executescript("""
CREATE TABLE orders (id TEXT PRIMARY KEY, status TEXT);
CREATE TABLE outbox (seq INTEGER PRIMARY KEY, topic TEXT, payload TEXT,
                     sent INTEGER DEFAULT 0);
""")
with db:  # the state change and the event commit together, or neither does
    db.execute("INSERT INTO orders VALUES ('ord-7', 'paid')")
    db.execute("INSERT INTO outbox (topic, payload) VALUES (?, ?)",
               ("order-events", json.dumps({"order": "ord-7", "type": "paid"})))

def relay(publish) -> None:  # runs in a loop; a crash here means a resend
    rows = db.execute("SELECT seq, topic, payload FROM outbox "
                      "WHERE sent = 0 ORDER BY seq LIMIT 100").fetchall()
    for seq, topic, payload in rows:
        publish(topic, key=json.loads(payload)["order"], value=payload)
        with db:
            db.execute("UPDATE outbox SET sent = 1 WHERE seq = ?", (seq,))

relay(lambda topic, key, value: print(topic, key, value))
```

The relay is at-least-once (it can crash after publishing and before marking), so consumers deduplicate, as in the idempotent consumer above. On Postgres, several relays can share the table with `SELECT ... FOR UPDATE SKIP LOCKED`, at the cost of strict order across rows; delete or partition old rows so the table stays small.

Change data capture (CDC) replaces the polling relay by tailing the database's own log. Debezium on Kafka Connect reads the Postgres write-ahead log through logical decoding (or the MySQL binlog) and publishes each committed change, in commit order; its outbox event router turns outbox inserts into clean domain events. Managed versions: DynamoDB Streams (24-hour retention) feeding Lambda, AWS DMS, GCP Datastream, Azure Cosmos DB change feed. CDC adds no load to the write path and misses nothing that committed, but it exposes table rows; publishing outbox rows through CDC gives you both a stable event contract and log-based delivery. The same feed carries changes into search indexes, caches and the lakehouse (Chapter 21).

14.10 Sagas: choreography and orchestration

Buying a ticket touches three services and an external provider, and no transaction spans them. A **saga** is a sequence of local transactions, each with a **compensation** that semantically undoes it if a later step fails (Chapter 18 contrasts it with two-phase commit).

```
step            action                    compensation
1 inventory     hold seats (10 min TTL)   release hold
2 orders        create order "pending"    mark order "cancelled"
3 payments      charge via provider       refund via provider
4 tickets       issue tickets             void tickets
```

- **Choreography**: each service reacts to events and emits the next (`seats-held` -> orders -> `order-created` -> payments ...). No central component and loose coupling, but the flow lives in no single place, and cycles or missing compensations are hard to see once there are more than three or four steps.
- **Orchestration**: one orchestrator holds the saga's state and sends commands ("charge order 7"), waiting for replies. The flow is explicit, testable and observable; the orchestrator must be durable, which is what workflow engines provide.

Compensations must be idempotent and retried until they succeed, and a step after the *pivot* (the charge, which cannot be taken back silently) should be retriable forward rather than compensated.

> **Design question.** The provider charged the card, then ticket issuing failed. Refund, or retry?
>
> Charging is Encore's pivot: the fan has paid and expects tickets, so the saga retries issuing forward (rendering is idempotent per order and seat) with backoff for minutes. Only if the seats are truly gone, for example a hold that expired before the charge confirmed, does it compensate: refund with an idempotency key derived from the order ID, mark the order cancelled, notify the fan. An orchestrator records each step, so a crash mid-saga resumes rather than re-charging. Daily reconciliation against the provider's report catches charges the saga never saw. See also *50 System Designs* #24 (Payment System) and #25 (Ticket Booking).

14.11 Workers, background jobs and schedules

A **worker** is a process that loops: take a job, run it, ack it. Job frameworks add retries with backoff, timeouts, result storage and dashboards on top of a broker: **Celery** (Python; RabbitMQ, Redis or SQS), **RQ** (Python; Redis lists), **Sidekiq** (Ruby; Redis), and Postgres-backed queues (Oban, River, Solid Queue) that claim rows with `SELECT ... FOR UPDATE SKIP LOCKED`, so a job and the data it changes can commit in one transaction. Two Celery settings matter for correctness: `task_acks_late=True` acks after the task instead of before (at-least-once rather than at-most-once), and with the Redis broker an unacked task is redelivered after the `visibility_timeout` (1 hour by default), so a task longer than that runs twice.

Priorities. One queue per priority with workers that drain the high one first starves the low one under load; reserve a share of workers per queue instead (Sidekiq's weighted queues, Celery routing), and keep slow jobs (PDF batches) apart from fast ones (push) so a backlog of one cannot delay the other.

Delayed and scheduled jobs. SQS delays a message up to 15 minutes; beyond that, keep due times in a Redis sorted set and claim due entries atomically. Encore uses this to expire seat holds after 10 minutes, as a backstop to the Redis key TTL:

```
import fakeredis

r = fakeredis.FakeRedis(decode_responses=True)
CLAIM_DUE = r.register_script("""
local due = redis.call('ZRANGEBYSCORE', KEYS[1], '-inf', ARGV[1],
                       'LIMIT', 0, ARGV[2])
if #due > 0 then redis.call('ZREM', KEYS[1], unpack(due)) end
return due
""")  # atomic: two workers can never claim the same job

now = 1_000_000                                  # seconds since epoch
r.zadd("jobs:delayed", {"expire-hold:h-41": now + 600,   # hold TTL 10 min
                        "expire-hold:h-17": now - 5})    # already due
print(CLAIM_DUE(keys=["jobs:delayed"], args=[now, 100]))  # ['...h-17']
print(CLAIM_DUE(keys=["jobs:delayed"], args=[now + 600, 100]))
```

The claimed job is then pushed onto a work queue, so a worker that crashes after claiming still leaves a redeliverable copy. Recurring jobs (nightly payment reconciliation, purging the outbox) belong to a scheduler that fires once per tick even with many replicas: EventBridge Scheduler (GCP Cloud Scheduler, Azure Functions timer triggers) or a Kubernetes CronJob with `concurrencyPolicy: Forbid`. A crontab on every web server runs the job once per server.

14.12 Durable workflows and long agent jobs

A queue handles one step well. A process of many steps that lasts minutes to days (a saga, a refund awaiting approval, an agent run) needs its *progress* to survive crashes. A **durable workflow engine** records every step's result in an event history; when a worker dies, another replays the history and continues from the last completed step instead of starting over.

- **Temporal** (open source or Temporal Cloud): workflows are ordinary code that must be deterministic; side effects run as *activities* with retry policies and timeouts; durable timers sleep for days; *signals* deliver outside events such as an approval.
- **AWS Step Functions** (GCP Workflows, Azure Durable Functions): state machines in JSON. Standard workflows run up to one year with exactly-once workflow execution and can pause on a task token until a callback arrives; Express workflows run up to 5 minutes, at-least-once when asynchronous, priced for high volume.

The engine makes each step's *outcome* durable, not the step itself: an activity can still run twice (crash after the charge, before the record), so activities carry idempotency keys like any consumer.

Long-running **agent jobs** fit the same mould. Encore's support agent may take a minute of model calls and tool use, and a refund waits for a human. The API accepts the request with 202 and a status URL (Chapter 5); the job goes on a queue; a worker runs the agent loop, checkpointing state after each step; each tool call (look up order, request refund) is an activity with an idempotency key and its own authorization; the approval arrives as a signal or task-token callback. Worker concurrency is capped by the model provider's token-per-minute limit, not by CPU. *Designing Agentic Systems* covers the agent side in depth; Chapter 24 shows the end-to-end flow.

14.13 In Encore

```
checkout -> payments DB (+outbox) -> CDC -> Kafka order-events (key order_id)
                                            |-> SQS pdf -> workers -> S3
                                            |-> notify fan-out -> SQS email
                                            |                  -> SQS push
                                            |-> dashboard stream consumer
```

Flow	Transport	Why
Order events	Kafka via outbox and CDC	Facts, many consumers, replay
E-mail, SMS, push	SQS standard, one queue each + DLQ	Tasks, per-message retry
Ticket PDFs	SQS + 24 workers at the peak	Burst of 83/s, idempotent per seat
Hold expiry	Redis sorted set + TTL	Delays, atomic claim
Clickstream	Kafka, 48 partitions	58 MB/s peak, 7-day replay
Refund saga, agent jobs	Step Functions or Temporal	Long, multi-step, approval

14.14 Numbers to remember

Figure	Value
SQS visibility timeout	30 s default, 12 h max
SQS retention, message size	4 days (max 14), 1 MiB
SQS FIFO dedupe window	5 minutes
Pub/Sub ack deadline	10 s default, 600 s max
Kafka default retention	7 days
Kinesis shard	1 MB/s or 1,000 rec/s in, 2 MB/s out
Step Functions max duration	Standard 1 year, Express 5 min
Workers needed	$\lambda S/\rho$

Figure	Value
Drain time	$B/(c\mu - \lambda)$

14.15 Pitfalls

- Expecting exactly-once delivery -> design for duplicates: dedupe table, upserts, idempotency keys downstream.
- Dual write to the database and the broker -> transactional outbox or CDC.
- A low-cardinality or hot partition key -> key by the entity that needs order (seat, order), and size partitions for growth up front.
- Visibility timeout shorter than the slowest job -> heartbeat or extend it, and set it above the p99.
- Retrying poison messages forever -> a delivery limit and a DLQ with an alarm and a redrive tool.
- Unbounded queues and prefetch -> bound work in flight and push back with 429.
- Autoscaling workers on CPU -> scale on lag and the age of the oldest message, capped by downstream limits.

14.16 Remember

- Queues carry tasks that one worker does once; logs carry facts that many consumers read, in order per partition, with replay.
- Delivery is at-least-once in practice; “exactly-once” means idempotent processing on top.
- Partitions bound parallelism and order; the key decides both.
- Lag in seconds (lag divided by consumption rate) is the user-facing number; size workers with $\lambda S/\rho$.
- Publish events through an outbox or CDC, and run multi-step processes as sagas on a durable workflow engine.

15 Compute in the cloud: containers, Kubernetes and functions

Every box on a design diagram has to run somewhere: on a virtual machine you patch, in a container that Kubernetes keeps alive, on a managed container platform, or as a function that exists only while an event is being handled. This chapter answers where Encore's code actually runs, how many copies it needs at 10:00 on sale day, what it costs, and how the whole estate is written down as code so it can be rebuilt. The picture is a ladder: each rung up hands one more job (servers, operating system, scaling) to the cloud provider and takes away one more knob.

15.1 The picture

Think of shipping goods. Owning a **ship** is owning a physical server. Renting a **cabin** on someone's ship is a virtual machine: your space, your furniture, your cleaning. Packing goods in a **standard shipping container** means any ship or crane can move them unopened. A **harbor master** who keeps the agreed number of containers on ships, and replaces any that fall overboard, is Kubernetes. A **courier paid per parcel** is a serverless function: nothing to rent, nothing to pay when idle, but at rush hour you wait for a courier to arrive.

```
                you manage                          billed per
rung            code  runtime  OS   scaling  servers
-------------   ----  -------  ---  -------  -------  ------------------
VM (EC2)        you   you      you  you      cloud    instance-second
Kubernetes      you   you      img  you+HPA  you*     node-second
managed cont.   you   you      img  cloud    cloud    vCPU-s + GB-s
function        you   cloud    -    cloud    cloud    request + GB-s

img = you ship the OS userland inside the image
*   = managed control plane; nodes are yours unless Auto Mode/Autopilot
```

Two rules carry most decisions. **Steady traffic is cheapest on long-running capacity** (VMs, containers), because you pay for capacity, not per request. **Spiky, sparse or event-shaped work is cheapest and simplest on functions**, because idle time costs nothing. Encore has both shapes: an API that serves 1,500 requests/s all day, and workers that wake up when a ticket is sold.

15.2 Virtual machines

Picture. A physical host with 96 cores is sliced by a **hypervisor** into virtual machines, each of which believes it owns a computer: virtual CPUs, memory, disks and network cards.

Mechanism. On AWS the hypervisor is the Nitro system (KVM-based, with networking and storage offloaded to dedicated cards), so a VM loses almost nothing to virtualization. You pick an **instance type** — a family (general purpose, compute-, memory- or storage-optimized, accelerated with GPUs) and a size — and an **image** (an AMI on AWS) holding the operating system. On x86 instances a vCPU is one hyperthread of a core; on Graviton (Arm) a vCPU is a whole core. Boot takes tens of seconds; your code starts after that. An **Auto Scaling group** (Google managed instance groups, Azure Virtual Machine Scale Sets) keeps *N* identical VMs from a launch template running across zones and replaces unhealthy ones.

Three ways to pay: **on-demand** by the second, **commitments** (Savings Plans or reserved instances) for a discount in exchange for one or three years of use, and **spot** capacity, spare machines at up to 90 percent off that the provider reclaims with a two-minute warning. Spot suits stateless, interruptible work: batch jobs, CI runners, analytics, a share of a stateless web fleet. **MicroVMs** shrink the VM itself: Firecracker is a minimal KVM-based virtual machine monitor that AWS built for serverless services such as Lambda: each function environment gets its own lightweight VM, booted in a fraction of a second, so tenants are separated by a hypervisor rather than by a shared kernel. **Where it runs:** Amazon

EC2 (Google Compute Engine, Azure Virtual Machines). Choose VMs when you need the kernel, special hardware, licensed software, very large memory, or a stateful system you operate yourself (a self-managed Kafka or Cassandra cluster). The cost is operations: patching the OS, rotating images, and building scaling and deployment around them.

15.3 Containers: images, layers and registries

Picture. A container is the shipping box: the application with its runtime and libraries, sealed so it behaves the same on a laptop, a CI runner and a production node.

Mechanism. A container is an ordinary Linux process with two fences around it. **Namespaces** give it its own view of process IDs, network interfaces, mount points and hostname; **cgroups** cap how much CPU and memory it may use. All containers on a host share the host's kernel, which is why a container starts in well under a second (no OS boots) and why the isolation is weaker than a VM's; multi-tenant platforms wrap containers in microVMs or sandboxes such as gVisor.

An **image** is a stack of read-only **layers** (tar archives of file changes) plus a small JSON config (entry point, environment, user), defined by the OCI image specification. Every layer is named by the SHA-256 **digest** of its content, so identical layers are stored and transferred once. At run time an overlay filesystem stacks the layers and adds one thin writable layer per container, which vanishes with the container: anything worth keeping goes to a database, a volume or object storage. A **registry** (Amazon ECR; Google Artifact Registry, Azure Container Registry) stores layers by digest; a push or pull sends only the layers the other side lacks. **Tags** such as `:v42` or `:latest` are movable pointers; a **digest** (`@sha256:...`) names one exact image forever. Deploy by digest. The order of instructions in a Dockerfile is therefore a performance decision: put what changes rarely (base image, dependencies) first and the application code last, so a code change rebuilds and ships one small layer. A **multi-stage build** compiles in a fat builder image and copies only the result into a slim runtime image:

```
# Dockerfile for seat-inventory (multi-stage)
FROM python:3.12-slim AS build
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.12-slim
COPY --from=build /install /usr/local
WORKDIR /app
COPY src/ ./src/
# run as an unprivileged user, never root
USER 10001
EXPOSE 8080
CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Numbers. Image size sets how fast a new replica can start on a node that has never pulled it:

$$t_{\text{pull}} \approx \frac{\text{bytes not cached on the node} \times 8}{\text{network bandwidth (bit/s)}}$$

At 1 Gbit/s an 80 MB slim image takes about 0.64 s; a 500 MB image with build tools left inside takes about 4 s, before decompression. During a scale-out of 40 pods onto fresh nodes, that difference repeats on every node. Lazy loading (Seekable OCI on Fargate, image streaming on GKE) starts a container before the whole image arrives. **Trade-offs:** containers give one artifact from laptop to production, fast starts and dense packing; they do not give strong isolation or persistence by themselves. Scan images for vulnerable packages in CI, pin base images by digest, and rebuild regularly so patches land.

15.4 Kubernetes: a control loop for containers

Picture. A thermostat. You do not tell it to switch the heater on; you write down 21 degrees and it keeps comparing the room with the target. Kubernetes works the same way: you declare "ten replicas of this image, spread over three zones", and controllers keep comparing the cluster with that declaration and act on the difference, forever.

```
kubectl / CI --> API server <--> etcd (desired + observed state, Raft)
                  ^ watch
     +-------------+--------------+
 scheduler: bind pods     controllers: Deployment, ReplicaSet, HPA ...
 to nodes                 make actual == desired
     v
 node (zone a)            node (zone b)            node (zone c)
 kubelet -> containerd    kubelet -> pods          kubelet -> pods
```

Mechanism. The **API server** is the only door: every change is an object written to **etcd**, a Raft-replicated key-value store (Chapter 18). The **scheduler** watches for pods without a node and places each one on a node with enough unreserved resources that satisfies its constraints. On each node the **kubelet** watches for pods assigned to it, starts their containers through the container runtime (containerd), runs their health probes and reports status. **Controllers** close the loop: if a node dies, its pods vanish from the observed state, the ReplicaSet controller sees nine replicas where ten are wanted, and creates one more.

Object	What it declares
Pod	one or more containers sharing an IP and volumes
Deployment	N replicas of a pod template; rolling updates
StatefulSet, Job, CronJob	stable identity; run to completion
Service	a stable virtual IP and DNS name for a pod set
ConfigMap, Secret	configuration and credentials for pods
HPA	replica count driven by a metric
PodDisruptionBudget	how many pods voluntary evictions may take

Services and ingress. Pods come and go with new IPs; a **Service** gives a set of pods (selected by labels) one stable name, `seat-inventory.default.svc`, and kube-proxy or an eBPF data plane spreads connections over the ready pods. Type `LoadBalancer` asks the cloud for a load balancer. HTTP traffic from outside arrives through an **Ingress** or, the newer and more expressive standard, the **Gateway API**; the community ingress-nginx controller was retired in March 2026, so new clusters should use a Gateway API implementation (Envoy Gateway, Istio, the cloud's own controller). Note that a Service balances *connections*, not requests, which matters for gRPC (Chapter 8). **Config and secrets:** A ConfigMap holds non-secret settings, a Secret holds credentials; both reach the container as environment variables or files, keeping the image identical across environments (the twelve-factor rule from Chapter 6). A Secret is only base64-encoded: turn on encryption at rest (managed clusters use the cloud KMS) and prefer syncing from a secrets manager (AWS Secrets Manager through the External Secrets Operator or the Secrets Store CSI driver).

Requests and limits. Each container declares a **request** (what the scheduler reserves) and optionally a **limit** (the cgroup ceiling). The scheduler packs pods by requests only. Exceeding a CPU limit throttles the container (the kernel withholds CPU time for the rest of each 100 ms period), which shows up as mysterious tail latency; exceeding a memory limit kills it (`OOMKilled`). A common production rule: set the memory limit equal to the memory request, set a CPU request but no CPU limit for latency-sensitive services. Pods whose requests equal their limits for every resource get the **Guaranteed** quality-of-service class and are evicted last under node pressure.

$$\text{pods per node} = \min\left(\frac{\text{allocatable CPU}}{\text{CPU request}}, \frac{\text{allocatable memory}}{\text{memory request}}\right)$$

A 16-vCPU, 64 GB node keeps some capacity for the kubelet and system daemons; with about 15.5 vCPU allocatable and seat-inventory pods requesting 2 vCPU and 2 GiB, it fits $\lfloor 15.5/2 \rfloor = 7$ pods, bound by CPU (illustrative). Requests that are too high waste nodes; requests that are too low let pods crowd a node and starve each other. **Probes:** A **readiness** probe failing removes the pod from its Service (no traffic, no restart); a **liveness** probe failing restarts the container; a **startup** probe holds the other two off while a slow application boots. Never make liveness depend on a database: a database blip would then restart every pod at once.

The seat-inventory Deployment and its autoscaler (validated with kubeconform in strict mode; the image digest is shortened):

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: seat-inventory
spec:
  replicas: 10                      # HPA owns this after creation
  selector:
    matchLabels: {app: seat-inventory}
  strategy:
    rollingUpdate: {maxSurge: 25%, maxUnavailable: 0}
  template:
    metadata:
      labels: {app: seat-inventory}
    spec:
      topologySpreadConstraints:    # even spread over three zones
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: DoNotSchedule
        labelSelector:
          matchLabels: {app: seat-inventory}
      containers:
      - name: app
        image: 123456789012.dkr.ecr.us-east-1.amazonaws.com/seat-inv@sha256:9f3c
        ports: [{containerPort: 8080}]
        envFrom:
        - configMapRef: {name: seat-inventory-config}   # HOLD_TTL_S=600
        - secretRef: {name: seat-inventory-db}          # DB password
        resources:
          requests: {cpu: "2", memory: 2Gi}   # what the scheduler packs
          limits: {memory: 2Gi}               # no CPU limit: no throttling
        readinessProbe:                       # gate traffic
          httpGet: {path: /ready, port: 8080}
          periodSeconds: 5
        livenessProbe:                        # restart if wedged
          httpGet: {path: /live, port: 8080}
          periodSeconds: 10
          failureThreshold: 3
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: seat-inventory
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: seat-inventory}
  minReplicas: 10
  maxReplicas: 90
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: {type: Utilization, averageUtilization: 75}
```

`maxUnavailable: 0` makes a rolling update add new pods before removing old ones (defaults are 25 percent surge and 25 percent unavailable). The topology spread constraint keeps the replica counts per zone within one of each other, so losing a zone loses a third of the pods, not all of them. **Where it runs:** Amazon EKS (Google Kubernetes Engine, Azure Kubernetes Service) runs the control plane for you; you still choose and pay for nodes, unless you hand them over too with EKS Auto Mode or GKE Autopilot. Nodes scale with Karpenter or the Cluster Autoscaler. Kubernetes earns its complexity when you run many services with one deployment model, need portability across clouds, or run things serverless platforms cannot (long-lived connections, sidecars, stateful sets, GPUs). For three services and one team it is often too much; a managed container platform does the same job with fewer moving parts.

15.5 Serverless functions

Picture. The courier paid per parcel. You hand over a function and a trigger; the platform finds a machine, runs the code once per event, and bills you for the milliseconds it ran. No event, no courier, no bill.

Mechanism. On AWS Lambda (Google Cloud Run functions, Azure Functions) each copy of a function (an *Event Function* in AWS's current documentation, next to the newer Lambda Web Functions and Lambda Managed Instances) lives in an **execution environment**, a Firecracker microVM. The first event that finds no idle environment triggers a **cold start**: download the code, start the runtime, run the initialization code outside the handler. The environment then serves events **one at a time**, stays warm for reuse, and is reclaimed after a period of idleness. Concurrency grows by adding environments. Events arrive three ways:

- **Synchronous**: API Gateway, an Application Load Balancer or a function URL waits for the response.
- **Asynchronous**: S3, SNS and EventBridge hand the event to an internal queue; Lambda retries a failed event twice by default, then sends it to a failure destination or dead-letter queue.
- **Poll-based** (event source mappings): Lambda polls SQS, Kinesis, DynamoDB Streams or Kafka and invokes the function with batches.

Lambda limit (October 2026)	Value
Memory	128 MB to 10,240 MB; 1 vCPU at 1,769 MB
Timeout	15 minutes
Concurrency per account per region	1,000 by default, raisable
Scaling rate per function	1,000 environments every 10 s
Payload, synchronous	6 MB request and response
Payload, asynchronous	1 MB

Numbers. Concurrency is Little's law (Chapter 3), and cost is requests plus memory-time:

$$C = \lambda d \qquad \text{cost} = N p_{\text{req}} + N d m p_{\text{GB-s}}$$

with λ events per second, d seconds per event, N events per month, m memory in GB, and on x86 in us-east-1 p_{req} = \$0.20 per million and $p_{\text{GB-s}}$ = \$0.0000166667 (Arm is 20 percent cheaper per GB-second). Initialization time counts as billed duration. Encore's ticket-PDF worker: during the flash sale tickets are sold at $50,000/600 \approx 83$ per second, each PDF takes 200 ms, so $C \approx 17$ environments, far below the quota; the backlog that ten fixed workers built up in Chapter 14 never forms. Over a year 50 million orders, about 4.17 million a month, at 1 GB and 200 ms cost about \$13.89 in duration plus \$0.83 in requests: under \$15 a month for a worker that is idle most of the day. Now put Encore's whole API on Lambda: 1,500 requests/s on average is 3.94 billion requests a month; at 20 ms and 512 MB that is \$657 of duration plus \$788 of requests, about \$1,445 a month, or \$0.37 per million requests. A Fargate task with 1 vCPU and 2 GB costs about \$0.0494 an hour (\$36 a month) and serves up to 300 requests/s at a 60 percent target. Break-even is where a task's hourly price equals the Lambda bill for the requests it serves:

$$\lambda^* = \frac{c_{\text{task-hour}}}{3600\,(p_{\text{req}} + d\,m\,p_{\text{GB-s}})} = \frac{0.0494}{3600 \times 3.67 \times 10^{-7}} \approx 37 \text{ requests/s}$$

A container that averages more than about 37 requests/s is cheaper than Lambda; Encore's API averages far more per container, so it runs on containers, and eight always-on tasks (\$288 a month) cover its average with headroom, with autoscaling to 20 for the daily peak. List prices, illustrative workload.

Cold starts. Tens to hundreds of milliseconds for a small Python or Node.js function, seconds for a large Java or ML image. Mitigations, cheapest first: keep the package small and import lazily; create clients (database, Redis, HTTP) outside the handler so warm invocations reuse them; turn on **SnapStart** (Java 11+, Python 3.12+, .NET 8+), which restores a snapshot of an initialized environment; or pay for **provisioned concurrency**, environments kept initialized in advance, for the minutes that matter.

Code. A ticket-PDF worker fed by SQS. It deduplicates because SQS delivers at least once, and it reports partial batch failures so one poisoned message does not make SQS redeliver the nine good ones (fakeredis stands in for ElastiCache):

```
import json
import fakeredis

r = fakeredis.FakeRedis()        # init code: runs once per environment

def render_and_store(order_id: str) -> None:
    if order_id == "o-bad":
        raise ValueError("template error")   # poison message
    # put to s3://encore-tickets/{order_id}.pdf: same key, safe to repeat

def handler(event, context):
    failures = []
    for rec in event["Records"]:
        order_id = json.loads(rec["body"])["order_id"]
        if r.exists(f"pdf-done:{order_id}"):  # SQS is at-least-once
            continue
        try:
            render_and_store(order_id)
            r.set(f"pdf-done:{order_id}", 1, ex=86_400)
        except Exception:
            failures.append({"itemIdentifier": rec["messageId"]})
    return {"batchItemFailures": failures}   # SQS retries only these

event = {"Records": [
    {"messageId": "m1", "body": json.dumps({"order_id": "o-1"})},
    {"messageId": "m2", "body": json.dumps({"order_id": "o-bad"})},
    {"messageId": "m3", "body": json.dumps({"order_id": "o-1"})}]}
print(handler(event, None))  # {'batchItemFailures': [{'itemIdentifier': 'm2'}]}
```

Trade-offs. Functions remove servers, patching and idle cost and scale per event; in exchange you accept cold starts, a 15-minute ceiling, one request per environment (so a function waiting 18 ms on a database pays for the wait), per-account concurrency shared by every function, and many short-lived database connections (put RDS Proxy in front of Postgres). **Agents and external systems:** An LLM call is mostly waiting: a support-agent step that waits 8 seconds for a model bills 8 seconds of function memory doing nothing. Keep short tool calls (look up an order) in functions or containers, and run long agent tasks as checkpointed workflows (AWS Step Functions or Lambda durable functions) that sleep for free while a human approves a refund; *Designing Agentic Systems* covers the agent side. Callbacks from the payment provider are a good function fit: rare, bursty, and each one is a short, idempotent update.

> **Design question.** Should Encore's seat-inventory hold path run on Lambda or on containers?
>
> At 30,000 holds/s and 20 ms each, Little's law gives C = 600 concurrent environments: within the default 1,000 quota, and the scaling rate (1,000 every 10 s) could reach it. But the quota is shared with the PDF and notification functions, the first wave of fans at 09:55 would meet 600 cold starts, each environment pays while waiting 18 ms on Redis and Postgres, and 600 environments open 600 database connections. Containers pre-scaled at 09:40 have warm connection pools, warm caches and known p99s; Lambda would need reserved and provisioned concurrency for the window, which removes most of its advantage. Run holds on containers, PDFs and notifications on functions. The trade-off: containers bill all day, and someone must remember to pre-scale. See also *50 System Designs* #25 (Ticket Booking).

15.6 Managed containers

Between Kubernetes and functions sits the rung most teams should try first: give the platform a container image, and it runs, scales and replaces copies without nodes or a cluster to manage. **AWS Fargate** runs ECS tasks or EKS pods on capacity you never see, billed per vCPU-second and GB-second with a one-minute minimum (from 0.25 vCPU up to 32 vCPU per task; Fargate Spot is up to 70 percent cheaper). **Google Cloud Run** runs request-driven containers that scale to zero, and lets one instance serve many requests at once: by default 80 concurrent requests per instance from the

console (80 per vCPU from gcloud or Terraform), up to 1,000, with request timeouts up to 60 minutes. **Azure Container Apps** offers the same model with KEDA-based scaling. Unlike Lambda, a multi-concurrency instance serves other requests while one waits on the database:

$$\text{instances} = \frac{\lambda\, d}{\text{concurrency per instance}}$$

Encore's catalog API at the 6,000 requests/s peak with 40 ms responses has 240 requests in flight, three instances at concurrency 80 by this formula; CPU per request usually binds first, so load-test and set concurrency from the measured p99. On Cloud Run the knobs are `--concurrency`, `--min-instances` (warm instances that trade a small always-on bill for no cold starts) and `--max-instances` (a ceiling that protects the database).

15.7 Autoscaling

Picture. A supermarket opening checkouts as queues grow. Opening one takes a minute (someone has to walk over), so a good manager watches the door, not only the queue, and opens checkouts before the coach party arrives.

Mechanism and numbers. Every autoscaler computes how many copies the current load needs and moves toward that number. Sizing from first principles, with λ the load, c the capacity of one copy and u the target utilization:

$$N = \frac{\lambda}{c\,u} \qquad N_{\text{HPA}} = N_{\text{current}} \times \frac{\text{current metric}}{\text{target metric}}$$

Encore's API pods (1 vCPU, 2 ms of CPU per request, so 500 requests/s at 100 percent) at the 6,000 requests/s peak and a 60 percent target need $N = 6000/300 = 20$. Seat-inventory pods (2 vCPU, 1,000 holds/s at 100 percent) at 30,000 holds/s and a 75 percent target need 40. If ten API pods run at 90 percent CPU against a 60 percent target, the HPA asks for $10 \times 90/60 = 15$.

Kubernetes' HPA evaluates every 15 seconds, ignores ratios within 10 percent of 1 (the tolerance), by default may at most double the replica count or add four pods per 15 seconds, whichever is more, and waits for a 5-minute stabilization window before scaling down, so a dip does not remove pods that the next spike needs. Other styles: **target tracking** (AWS Auto Scaling keeps a metric at a value, as HPA does), **step scaling** (add k copies per alarm band), **scheduled** scaling (a known event at a known time), **predictive** scaling (forecast from history), and **queue-based** scaling with KEDA (Kubernetes event-driven autoscaling), which sizes workers from backlog: to drain backlog B in time T while new messages arrive at λ, with μ messages per second per worker, run $(\lambda + B/T)/\mu$ workers. **Scale to zero** is native to Lambda and Cloud Run; Kubernetes' HPA does not scale to zero by default, KEDA adds it. Under the pods, **cluster autoscaling** (Karpenter, Cluster Autoscaler, GKE and AKS node autoprovisioning) adds nodes when pods cannot be scheduled.

Code. A reactive HPA meeting the flash sale. Demand jumps from 10 pods' worth to 30,000 holds/s; CPU on full nodes cannot rise far above 100 percent of the request, so the HPA sees only 133 percent of target and grows in steps of a third, each waiting 90 s (illustrative) for a node, an image pull and a boot:

```
import math
def hpa(spec, ready, util, target=75, tol=0.10):
    ratio = ready * util / (spec * target)   # unready pods count as 0%
    if abs(ratio - 1) <= tol:
        return spec
    want = math.ceil(spec * ratio)
    return min(want, max(2 * spec, spec + 4))   # default scale-up cap

demand, per_pod = 30_000, 1_000      # holds/s; one pod at 100% CPU
spec, ready, t, starts = 10, 10, 0, []
while ready < 40:
    util = min(100, 100 * demand / (ready * per_pod))   # full nodes cap CPU
    new = hpa(spec, ready, util)
    if new > spec:
        starts.append((t + 90, new - spec))   # node + pull + boot: 90 s
        spec = new
    ready += (came := sum(n for at, n in starts if at == t))
    if came or t == 0:
        served = min(demand, ready * per_pod)
        print(f"t={t:3}s spec={spec:3} ready={ready:3} served={served}")
    t += 15
```

Ready pods go 10, 14, 19, 26, 35 at 0, 90, 195, 300 and 405 seconds: nearly seven minutes of a ten-minute sale pass before capacity meets demand. Two lessons: scale on a **demand** metric that does not saturate (requests per second at the gateway, waiting-room queue length), which here would ask for 40 pods in the first evaluation; and for known events, do not react at all.

> **Design question.** How many seat-inventory pods should run at 09:55, and when should they start?
>
> Demand is bounded by the waiting room (Chapter 11): it admits about 1,000 fans/s, which keeps the hold load near 3,000/s, half of the 6,000 holds/s the inventory was load-tested at. Size for the tested ceiling rather than the operating point, so raising the admission rate needs no redeploy: 6,000 holds/s at 1,000 holds/s per pod and a 75 percent target is 8 pods. To survive the loss of one of three zones, each zone must hold half the load: 4 pods per zone, 12 in total, on 2 nodes of 16 vCPU. Without the waiting room the open door's 30,000 holds/s would need 40 pods, 60 with the zone spare, and the database behind them would still be the limit. Start them at 09:40 with a scheduled action (a CronJob that raises the HPA's `minReplicas`, or a KEDA cron trigger), warm their connection pools and caches with synthetic reads, and let the HPA handle only the surprise. Lower `minReplicas` again at 10:30. The trade-off: about an hour of 12 pods per sale, small next to oversold seats or a stalled sale. See also *50 System Designs* #34 (E-Commerce Checkout and Inventory).

15.8 Regions, zones and placement

A **region** is a metropolitan area with its own control planes (us-east-1 in Virginia, eu-central-1 in Frankfurt for Encore). Inside it, **availability zones** are separate data centers with their own power, cooling and network, up to about 100 km apart on AWS yet a round trip of 0.5 to 2 ms from each other, close enough for synchronous replication. Zones fail independently; regions almost never fail together. Design for the loss of one zone without anyone noticing (static stability): with Z zones each must carry $1/(Z-1)$ of the peak, so

$$\text{deployed capacity} = \text{peak capacity} \times \frac{Z}{Z-1}$$

Three zones cost 50 percent extra, two zones 100 percent: Encore's 20 API pods become 30 and its 8 seat-inventory pods become 12. Place copies with topology spread constraints or pod anti-affinity, protect them during node upgrades with a PodDisruptionBudget, and keep chatty services zone-local (Kubernetes topology-aware routing), because AWS bills traffic between zones, about $0.01 per GB in each direction. EC2 **placement groups** pack instances close for low latency (cluster), apart for independence (spread), or in failure-isolated groups for Kafka- and Cassandra-style replicas (partition). A second region is for disasters, data residency and nearby users; each event's seats keep one home region, and failover is Chapter 16 and Chapter 23.

15.9 Infrastructure as code and CI/CD

Clicking resources into a console is cooking without a recipe: nobody can repeat it, review it or roll it back. **Infrastructure as code** writes the estate as declarative files in Git. Terraform (or its open-source fork OpenTofu) compares the files with a **state** file and the real cloud, prints a `plan` of creates, updates and deletes, and `apply` executes it; state lives remotely (an S3 bucket with locking) so a team shares one view. Native alternatives are CloudFormation and the CDK (Google Infrastructure Manager, Azure Bicep). The ticket-PDF pipeline, validated with `terraform validate`:

```
resource "aws_sqs_queue" "pdf_dlq" { name = "ticket-pdf-dlq" }
resource "aws_sqs_queue" "pdf" {
  name                       = "ticket-pdf"
  visibility_timeout_seconds = 60 # at least 6 x the function timeout
  redrive_policy = jsonencode({   # after 5 failed receives, park it
    deadLetterTargetArn = aws_sqs_queue.pdf_dlq.arn, maxReceiveCount = 5
  })
}
resource "aws_lambda_function" "pdf" {
  function_name = "ticket-pdf"
  package_type  = "Image"
  image_uri     = "${var.ecr_repo}@${var.image_digest}" # by digest
  role          = var.pdf_role_arn
  memory_size   = 1024
  timeout       = 10
  architectures = ["arm64"]
}
resource "aws_lambda_event_source_mapping" "pdf" {
  event_source_arn        = aws_sqs_queue.pdf.arn
  function_name           = aws_lambda_function.pdf.arn
  batch_size              = 10
  function_response_types = ["ReportBatchItemFailures"]
  scaling_config { maximum_concurrency = 50 } # cap the fan-out
}
```

The visibility timeout follows AWS's rule of at least six times the function timeout, so a slow batch is not redelivered while still running; `maximum_concurrency` caps how many environments the queue may drive. **CI/CD** turns a commit into running code with no hand steps:

```
push -> test -> build image -> scan -> push to ECR (digest) -> plan IaC
-> staging + smoke tests -> canary 5% -> watch SLOs -> 100% (or roll back)
```

With **GitOps** (Argo CD, Flux) the pipeline commits the new digest to a deployment repository and an agent in the cluster pulls and applies it, so Git is the record of what runs. Canary and blue-green releases are covered in Chapter 23; a full pipeline is in Chapter 25.

15.10 In Encore: where each component runs

Component	Runs on	Why
Web app, event pages, images	S3 + CloudFront	static, cacheable, global
Waiting-room token check	CloudFront Functions	a signature check at the edge
API gateway	ALB -> Envoy pods on EKS	auth, routing, rate limits
Catalog, search API, orders	EKS Deployments, 20 to 30 pods	steady load, autoscaled
Seat inventory	EKS, 12 pods pre-scaled at 09:40	hot path, warm pools
Holds and seat state	ElastiCache (Valkey) + Aurora PostgreSQL	fast TTL plus durable record
Payment webhooks	API Gateway -> Lambda	rare, bursty, idempotent
Ticket PDFs, notifications	SQS -> Lambda	event-shaped, idle most of the day
Clickstream	Kinesis -> Managed Flink -> S3 (Iceberg)	managed streaming at 1 billion/day

Component	Runs on	Why
Support agent	ECS on Fargate + Bedrock + Step Functions	containers, models, approvals
Scheduled jobs	EventBridge Scheduler, CronJobs	pre-scaling, reconciliation

The pieces meet in code through three things. **Names**: each pod gets endpoints (queue URL, Redis host, database host) from a ConfigMap or environment variables, never baked into the image. **Identity**: pods assume an IAM role through EKS Pod Identity (GKE Workload Identity, Azure Workload Identity), so the SDK finds short-lived credentials without a stored key. **Contracts**: services talk over the APIs and events of Chapter 5 and Chapter 14, so any box can move between rungs (a worker from Lambda to Fargate) without its callers noticing.

15.11 Building blocks to managed services

Building block	AWS	Google Cloud	Azure
VM fleet	EC2 + Auto Scaling	Compute Engine + MIGs	VMs + Scale Sets
Kubernetes	EKS	GKE	AKS
Serverless containers	Fargate	Cloud Run	Container Apps
Functions	Lambda	Cloud Run functions	Functions
Load balancer (L7)	ALB	Cloud Load Balancing	Application Gateway
API gateway	API Gateway	API Gateway, Apigee	API Management
CDN and edge code	CloudFront	Cloud CDN	Front Door
Cache	ElastiCache	Memorystore	Managed Redis
Relational database	RDS, Aurora	Cloud SQL, AlloyDB	Database for PostgreSQL
Key-value	DynamoDB	Firestore, Bigtable	Cosmos DB
Object storage	S3	Cloud Storage	Blob Storage
Queue	SQS	Pub/Sub, Cloud Tasks	Service Bus
Stream	Kinesis, MSK	Pub/Sub, Managed Kafka	Event Hubs
Workflows	Step Functions	Workflows	Durable Functions
Secrets	Secrets Manager	Secret Manager	Key Vault
LLM models	Bedrock	Vertex AI	Microsoft Foundry

The full list, with analytics, security and observability services, is in Appendix C, the cloud service map.

15.12 Numbers to remember

Figure	Value
Lambda timeout, memory	15 min; 128 MB to 10 GB
Lambda default concurrency	1,000 per region
Lambda scaling rate	1,000 environments per 10 s
Lambda price (x86)	$0.20/M requests + $0.0000166667/GB-s
Fargate 1 vCPU + 2 GB	about $0.049 per hour
Cloud Run concurrency	80 default, 1,000 max
HPA loop, tolerance	15 s; 10 percent
HPA scale-down window	5 minutes
Zone round trip	0.5 to 2 ms

Figure	Value
Capacity for zone loss	$Z/(Z-1)$: 1.5x for 3 zones

15.13 Pitfalls

- Reactive autoscaling for a known spike -> scheduled pre-scaling, plus a demand metric that does not saturate.
- No resource requests, or CPU limits on latency-critical pods -> requests from load tests, memory limit = request, no CPU limit.
- Liveness probes that call the database -> liveness checks the process only; readiness may check dependencies.
- Deploying `:latest` -> deploy immutable digests so every node runs the same bits and rollback is exact.
- Lambda in front of Postgres without a proxy -> RDS Proxy or a connection-light store; cap concurrency per function.
- All replicas in one zone after a rollout -> topology spread constraints and a PodDisruptionBudget.
- Console-built infrastructure -> everything in Terraform, applied by the pipeline, reviewed like code.

15.14 Remember

- Climb the ladder only as far as you must: functions for events, managed containers for services, Kubernetes for many services and special needs, VMs for kernels and stateful software you operate.
- Containers are processes with namespaces and cgroups; images are content-addressed layers; deploy by digest.
- Kubernetes is a set of control loops comparing declared state with reality; the scheduler packs by requests.
- Size with $N = \lambda/(c\,u)$, concurrency with $C = \lambda d$, and multiply by $Z/(Z-1)$ to survive a zone.
- Functions are cheaper while a container's share of traffic stays below the break-even (about 37 requests/s with Encore's numbers); above it, pay for capacity.
- Write the estate as code and ship it through a pipeline; the cloud is rebuilt, not repaired.

Part IV. Distribution

What changes when data lives on many machines: replication, partitioning, consistency and consensus, the algorithms inside real systems, blockchains as Byzantine-tolerant replicated ledgers, and big data.

In this part:

- Replication
- Partitioning and consistent hashing
- Consistency, clocks and consensus
- The algorithms inside real systems
- Web3: blockchains as distributed systems
- Big data: batch, streams and lakehouses

16 Replication

When Encore's seat-inventory database dies at 10:01 during the flash sale, which copy of the data takes over, how many confirmed holds does that copy lack, and how did the copies get there in the first place? Replication is keeping the same data on several machines so that a crash, a lost zone or a lost region costs neither data nor uptime, and so that reads can come from a nearby copy. Every replicated system answers the same three questions: who accepts writes (one leader, several leaders, or anyone), when a write counts as done (after one copy, a quorum, or all copies), and what a reader may see while the copies disagree.

16.1 The picture

Picture a box office with a master ledger of sold seats. Clerks at branch kiosks cannot see the master ledger, so a courier carries each new page to them. If the courier waits for every kiosk to sign before the box office tells the buyer "sold", buyers wait; if the box office answers first, a kiosk may still show a seat as free. If two kiosks are allowed to sell independently, they will sometimes sell the same seat, and someone must decide later who wins. Those three choices are the three replication models.

```
single-leader          multi-leader            leaderless
 writes                writes     writes       writes and reads
   |                     |          |           |     |     |
   v                     v          v           v     v     v
  [L] -+-> [F1]        [L1] <----> [L2]        [A]   [B]   [C]
       |                 |          |
       +-> [F2]          v          v          a write waits for W of N
                       [F]        [F]          a read asks R of N
 reads from L, F1, F2  each region has one     no node is special
```

The arrows are the replication stream: an ordered log of changes (the write-ahead log in Postgres, the binlog in MySQL, the commit log in Kafka) that each copy replays. A **replica** is any copy; the **leader** (primary, writer) accepts writes; **followers** (replicas, standbys, readers) apply the leader's log. **Replication lag** is how far a follower's log trails the leader's, in bytes, transactions or seconds.

16.2 Why replicate

Four reasons, and each pulls the design in a different direction:

Goal	What the copy buys	Where the copy goes
Availability	a survivor takes over when a node fails	another zone
Durability	a disk or zone loss loses no data	another zone or region
Latency	reads served near the user	another region, an edge
Read scale	reads spread over many machines	anywhere, same region

Replication is not backup. A replica faithfully copies a `DELETE FROM orders` within milliseconds; only point-in-time backups and snapshots undo mistakes (see Chapter 23). Replication is also not partitioning: every replica holds all of its data set, whereas partitioning splits the data so that each node holds a part (Chapter 17). Real systems do both: each partition is replicated three times. Copies cost money in three places: storage and compute for every replica, network transfer for the stream (inter-region traffic is billed per gigabyte), and write latency whenever the leader waits for a copy. The rest of the chapter is about spending those three wisely.

16.3 Single-leader replication

One node, the leader, takes every write, orders it, and appends it to its log. Followers connect, receive the log as a stream and replay it in the same order, so every follower passes through exactly the leader's sequence of states, only later. Because there is one order, there are no write conflicts; that is why single-leader is the default for relational databases and the right first answer in an interview.

What travels in the log matters:

- **Physical (WAL) shipping**: the bytes changed on disk pages. Exact and cheap to apply, but leader and follower must run the same major version. Postgres streaming replication works this way.
- **Logical (row-based)**: "row with key 42 now has these column values". Survives version differences and can feed other systems, which is what change data capture builds on (Chapter 14). MySQL's binlog in `ROW` format, Postgres logical replication.
- **Statement-based**: the SQL text. Compact, but `NOW()`, `RANDOM()` and auto-increment races make followers diverge; avoid it.

A new follower starts from a snapshot taken at a known log position (a Postgres LSN, a MySQL GTID set) and then streams everything after that position. The same mechanism lets a follower that was down for an hour catch up, provided the leader still has the log it needs; that is what replication slots in Postgres guarantee, at the price of WAL piling up on the leader's disk while a slot's consumer is gone.

Followers scale reads, never writes: each follower replays every write. If λ_r reads per second reach the database, each replica serves c reads per second, and you run replicas at utilization u, then

$$n_{\text{replicas}} = \frac{\lambda_r}{c\,u} + 1$$

where the +1 keeps capacity when one replica fails. Encore's seat map peaks at 200,000 reads per second; with a 95 percent cache hit ratio (Chapter 9), 10,000 per second reach Postgres. At an illustrative 10,000 simple reads per second per replica and u = 0.6, that is 1.67 + 1 = 3 replicas. Without the cache it would be 35, and every one of them would still replay the full write stream. Cache first, replicas second, partitioning third is the usual order of scaling a database (Chapter 7).

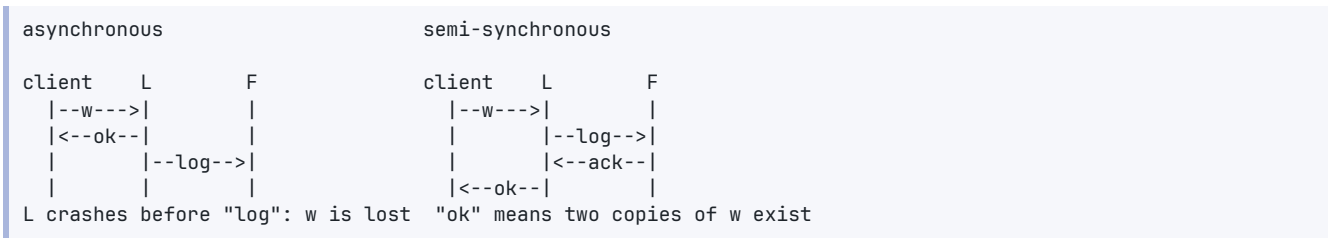
The leader exposes lag per follower; this is the first query to run when someone says "the replica is behind":

```
-- On the Postgres leader (10+): one row per connected standby.
SELECT application_name, state, sync_state,
       pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn) AS bytes_behind,
       replay_lag
FROM pg_stat_replication;
```

Every relational database and most others work this way: Postgres and MySQL (Amazon RDS, Google Cloud SQL, Azure Database for PostgreSQL), MongoDB replica sets (majority write concern by default since 5.0), Redis replicas, and each Kafka partition with its in-sync replica set. Aurora is a variant: the writer ships redo records to a storage layer that keeps six copies across three zones, a write needs four of the six and a read three, and up to 15 Aurora Replicas read the same volume, usually well under 100 ms behind.

16.4 Synchronous, asynchronous and semi-synchronous

The leader can answer the client before or after a follower has the write. That one decision fixes both commit latency and how much data a failover can lose.



- **Asynchronous**: the leader commits locally and answers; followers catch up when they can. Fastest, and a follower outage never blocks writes, but a leader crash loses whatever had not yet shipped.
- **Synchronous**: the leader waits until a follower confirms the write. Pure synchronous replication to *all* followers turns any one slow or dead follower into a write outage, so nobody runs it that way.
- **Semi-synchronous**: wait for *some* followers (one, or any *k* of *n*), and keep the rest asynchronous. This is what "synchronous" means in practice: Postgres `synchronous_standby_names = 'ANY 1 (...)'`, MySQL semi-sync, RDS Multi-AZ DB clusters (one of two readers must acknowledge), Kafka `acks=all` with `min.insync.replicas`.

A synchronous commit costs the local flush plus one round trip to the *k*-th fastest synchronous follower:

$$t_{\text{commit}} \approx t_{\text{flush}} + \text{RTT}_{(k)} \qquad \text{writes/s on one hot row} \le \frac{1}{t_{\text{commit}}}$$

The second bound bites because a row lock is held until commit. With illustrative figures of 1 ms to flush and 1 ms round trip between zones, a commit takes about 2 ms and one row can change about 500 times per second. A synchronous follower across the ocean (Virginia to Frankfurt, about 90 ms round trip) stretches the commit to about 91 ms and the same row to about 11 changes per second.

An asynchronous leader that dies loses the writes in flight:

$$\text{writes lost} \approx \lambda_w \times \text{lag}$$

At the flash-sale peak of 30,000 hold attempts per second and an illustrative 200 ms of lag, a crash forgets about 6,000 holds; at one second of cross-region lag, about 30,000. This product is the **RPO** (recovery point objective) of the design, expressed in writes rather than seconds.

```
# postgresql.conf on the leader (Virginia, zone a): commit waits for
# the first of two standbys; the unlisted Frankfurt standby stays async.
synchronous_standby_names = 'ANY 1 (seats_az_b, seats_az_c)'
synchronous_commit = on   # off | local | remote_write | on | remote_apply
```

`synchronous_commit` is settable per transaction, so one database can mix guarantees: `SET LOCAL synchronous_commit = off` for a page-view counter, `remote_apply` for a write that the next request will read from a standby. `on` waits until the standby has flushed the WAL to disk, `remote_write` until it has handed it to its operating system, `remote_apply` until it has replayed it and queries there can see it. MySQL's equivalents are `rpl_semi_sync_source_enabled` and `rpl_semi_sync_source_wait_for_replica_count`; its default wait point `AFTER_SYNC` waits for a replica's acknowledgement before the commit becomes visible, so no client can see a write that a failover would lose. Redis offers `WAIT numreplicas timeout`, which blocks until that many replicas acknowledge earlier writes; it narrows the loss window but does not make Redis a strongly consistent store, because a replica that acknowledged can still lose the write on failover in some sequences.

One trap: MySQL semi-sync falls back to asynchronous when no replica answers within `rpl_semi_sync_source_timeout` (10 seconds by default), and returns to semi-sync when one catches up. Postgres does the opposite: commits wait indefinitely for a listed standby. Know which you are running, because one silently trades durability for availability and the other trades availability for durability.

> **Design question.** Should a seat hold count as confirmed only after a second copy exists?
>
> A hold is worth 10 minutes of a fan's attention; an order is worth money. At 30,000 hold attempts per second, losing 200 ms of holds in a crash means about 6,000 fans whose hold vanished, and worse, whose seat may then be held by someone else. Semi-sync inside the region costs about 1 ms per commit and keeps the loss at zero for a single-node failure, so use `ANY 1` over two standbys in two other zones. Do not make the Frankfurt copy synchronous: 90 ms per commit would cap a contended seat row at about 11 changes per second. Orders and payments get the same in-region semi-sync; the cross-region copy stays asynchronous and the RPO for a full region loss is "under a second of writes". See also *50 System Designs* #25 (Ticket Booking).

16.5 Replication lag and its anomalies

An asynchronous follower is a little in the past, usually by milliseconds, sometimes by minutes when it replays a large batch or a long query holds it back. A system that serves reads from followers is therefore **eventually consistent**: stop writing and every copy converges, but while writes flow a reader can see the past. Three named anomalies cover what users notice, and each has a cheap fix.

```
read-your-writes         monotonic reads          consistent prefix
fan: pay -> L            read F1 -> 4 seats left  A asks "Sold out?"
fan: read F (behind)     read F2 -> 6 seats left  B answers "Yes"
  -> "no tickets yet"    (F2 lags F1: time runs   reader sees B before A:
                          backwards)              answer before question
```

Anomaly	What the user sees	Fix
Read-your-writes	own write missing	read own data from the leader for a while, or wait for the write's log position
Monotonic reads	data goes back in time	pin each user to one replica (hash the user ID)
Consistent prefix	effect before cause	keep causally related writes in one partition, in one log

16.5.1 Read-your-writes with a log position

The robust fix is to remember *where* the user's last write landed in the log and read only from a copy that has replayed that far. After the commit the application reads the leader's log position (`pg_current_wal_lsn()` in Postgres; on a standby, `pg_last_wal_replay_lsn()` says how far it has replayed); the application keeps it in the session or a cookie and the router compares:

```
import random

log = []                                  # the leader's log of (key, value)

class Copy:                               # the leader, or a replica
    def __init__(self):
        self.applied, self.data = 0, {}
    def catch_up(self, n):                # replay up to n more records
        for key, value in log[self.applied:self.applied + n]:
            self.data[key] = value
            self.applied += 1
def write(leader, key, value):
    log.append((key, value))
    leader.catch_up(1)
    return len(log)                       # the commit's LSN, kept in session
def read(leader, replicas, key, min_lsn=0):
    fresh = [r for r in replicas if r.applied >= min_lsn]
    return (random.choice(fresh) if fresh else leader).data.get(key)

leader, replicas = Copy(), [Copy(), Copy()]
lsn = write(leader, "order:9001", "PAID")
replicas[0].catch_up(1)                   # replicas[1] still behind
print("any replica:", random.choice(replicas).data.get("order:9001"))
print("lsn-aware:", read(leader, replicas, "order:9001", min_lsn=lsn))
# any replica: PAID or None, at random; lsn-aware: always PAID
```

Cheaper variants: route all of a user's reads to the leader for a fixed window after a write (simple, but the window must exceed the worst lag), or make the write itself wait with `synchronous_commit = remote_apply` so the standby has applied it before the client hears "ok". Azure Cosmos DB builds this in: its default **session** consistency hands the client a session token after every write and serves reads only from replicas that have caught up to it. DynamoDB (Google Cloud Firestore or Bigtable, Azure Cosmos DB) offers `ConsistentRead=true` per request, which reads from the partition's leader at twice the read cost. Aurora replicas share the writer's storage and lag usually under 100 ms, but "usually" is not a guarantee, so the same rule applies.

> **Design question.** A fan pays, the app jumps to "My tickets", and the list is empty. Fix it.
>
> The order was written to the leader in Virginia; "My tickets" reads a replica that is 300 ms behind. Reading everything from the leader would undo the read scaling, and only a small fraction of reads follow the reader's own write. So: the order service returns the commit LSN, the API gateway stores it in the session, and the tickets endpoint reads from a replica whose replayed LSN is at least that value, falling back to the leader. Alternatives: read from the leader for 5 seconds after any write by this user, or show the order from the response of the payment call itself (the client already has it). Add monotonic reads by hashing the user ID to a replica. See also *50 System Designs* #24 (Payment System).

The same anomaly hits AI agents harder than people, because an agent acts on what it reads. Encore's support agent that issues a refund and then checks "is this order refunded?" against a lagging replica may see "not refunded" and try again. Route the agent's read-after-write tool calls to the leader or pass the log position, and make the refund tool idempotent with a key derived from the order, so a second attempt is a no-op at the payment provider. *Designing Agentic Systems* covers tool design in depth.

16.6 Failover and split brain

Failover turns a follower into the leader when the leader dies. It has four steps, and each one has its own way of going wrong:

1. **Detect**: the leader misses heartbeats for a timeout. Too short and a garbage-collection pause or a network blip triggers a needless failover; too long and writes are down meanwhile.
2. **Choose**: promote the follower with the most of the log, decided by a majority of voters so that two nodes cannot both win.
3. **Redirect**: clients find the new leader through DNS, a virtual IP, a proxy (RDS Proxy, PgBouncer behind HAProxy) or a cluster-aware driver.
4. **Fence**: stop the old leader from accepting writes when it comes back. Without this you get **split brain**: two leaders, two divergent histories, and writes on one side that must be thrown away or merged by hand.

The outage lasts RTO ≈ $t_{\text{detect}} + t_{\text{elect}} + t_{\text{promote}} + t_{\text{redirect}}$ (recovery time objective). With illustrative figures of 15 s to detect, 2 s to elect and promote, and 5 s for a DNS TTL to expire, writes are down for about 22 s. At the flash-sale rate that is 660,000 failed hold attempts that clients must retry with backoff (Chapter 11), which is why the waiting room pauses admission during a failover instead of letting the retry wave arrive at once.

With asynchronous replication, failover also has a data cost: the old leader's last writes never reached the new one. When the old leader returns, those writes must be discarded (Postgres `pg_rewind` rewinds it to the point where histories diverged). Discarding is dangerous when other systems saw the lost writes: in a well-known GitHub incident, a promoted MySQL follower reissued auto-increment IDs that a Redis cache still associated with the old rows, and some users saw other users' data. Use IDs that do not depend on the leader's counter (see *50 System Designs* #21 (Unique ID Generator)).

The standard way to run Postgres failover is Patroni, which keeps a leader key with a TTL in etcd, Consul or ZooKeeper, so the election rides on a consensus store and only one node can hold the key:

```
scope: seats                # patroni.yml excerpt, seat inventory
etcd3:
  hosts: etcd-1:2379,etcd-2:2379,etcd-3:2379
bootstrap:
  dcs:
    ttl: 30                 # leader key expires 30 s after last renewal
    maximum_lag_on_failover: 1048576   # bytes; laggier nodes never promote
    synchronous_mode: true  # only a synchronous standby may be promoted
```

Managed databases do the same internally: RDS Multi-AZ, Aurora, Cloud SQL high availability and Azure Database for PostgreSQL flexible server all detect, promote and move the endpoint for you. Why a majority is required, how leases expire and how fencing tokens stop a paused old leader belong to Chapter 18.

16.7 Multi-leader replication

Several nodes accept writes, typically one leader per region, and each ships its changes to the others asynchronously. Writes become local in every region (a few milliseconds instead of a 90 ms ocean crossing) and a region can keep writing while the link is down. The price is **write conflicts**: two regions change the same item before either hears of the other, and both changes commit.

```
Virginia leader                        Frankfurt leader
  t=0   A12 := HELD by ana, "ok"     t=0   A12 := HELD by ben, "ok"
        ------------ replicate ------------->
        <------------ replicate --------------
  t=90ms both regions see the conflict: two fans were told "yours"
```

Conflict resolution after the fact (next sections) works for shopping carts and profiles; it cannot work for a seat, because two fans were already told yes. The cleanest multi-leader design avoids conflicts by **homing** each key: every record has one region that owns its writes (a user's profile in their home region, an event's seats in the event's region), and other regions forward writes to it. That is single-leader per key, multi-leader per system.

Multi-leader also appears in disguise: an offline mobile app is a leader with very long replication lag, and a collaborative editor has one leader per open tab (see *50 System Designs* #11 (Collaborative Editor)). Where it runs: DynamoDB global tables in the default multi-Region eventual consistency mode (replication typically within a second, last-writer-wins per item), Azure Cosmos DB with multiple write regions, MySQL Group Replication in multi-primary mode, EDB Postgres Distributed, CouchDB. For one data set that must be strongly consistent across regions, DynamoDB's multi-Region strong consistency mode replicates each write synchronously to another region before acknowledging; it requires exactly three regions (or two plus a witness) and pays a cross-region round trip on every write. Spanner (multi-region configurations) and CockroachDB take the same route with consensus per range.

16.8 Leaderless replication and quorums

No node is special. The client (or a coordinator node acting for it) sends each write to all N replicas of the key and calls it done when W acknowledge; a read asks R replicas and keeps the answer with the newest version. Amazon's 2007 Dynamo paper made the design famous; Cassandra, ScyllaDB and Riak run it today. (DynamoDB, despite the name, is not leaderless: each partition has three replicas in three zones with an elected leader.)

```
N = 3, W = 2, R = 2

write v2:  [A] v2 ack    [B] v2 ack    [C] still v1   -> ok after 2 acks
read:      ask A                       ask C          -> v2 and v1: return
                                                         v2, repair C
```

If every write reaches W replicas and every read asks R of the same N, the two sets must share at least one replica whenever

$$R + W > N$$

so at least one answer carries the latest acknowledged version. A write tolerates $N - W$ replicas down and a read tolerates $N - R$. When $R + W \le N$ (a *partial* quorum), a read issued the instant a write is acknowledged, before the remaining replicas have it, misses the new value with probability

$$P_{\text{stale}} = \frac{\binom{N-W}{R}}{\binom{N}{R}}$$

For $N = 3$: $W = R = 1$ gives 2/3; $W = 2, R = 1$ gives 1/3. Staleness then falls quickly as the slow replicas catch up. The chance that a quorum of K out of N replicas is reachable, with each replica independently up with probability p, is

$$P(\text{quorum}) = \sum_{k=K}^{N} \binom{N}{k} p^k (1-p)^{N-k}$$

With an illustrative $p = 0.99$ and $N = 3$: $K = 2$ gives 0.999702, $K = 3$ only 0.970299. Requiring every copy turns three good machines into one bad one; requiring a majority makes them better than any single machine.

The simulation writes a new version to three replicas whose apply times are random (mean 10 ms, illustrative), acknowledges after *W*, and reads *R* random replicas either at once or 10 ms later:

```
import random

def stale_rate(n, w, r, delay_ms=0.0, trials=100_000, seed=7):
    """Write v2 over v1 to n replicas; read r replicas delay_ms after ack."""
    rng = random.Random(seed)
    stale = 0
    for _ in range(trials):
        # time at which each replica applies v2 (exponential, mean 10 ms)
        arrive = sorted(rng.expovariate(1 / 10) for _ in range(n))
        ack = arrive[w - 1]                # client hears ok after w copies
        t = ack + delay_ms                 # the read happens at time t
        has_v2 = [a <= t for a in arrive]
        asked = rng.sample(range(n), r)    # coordinator picks r replicas
        if not any(has_v2[i] for i in asked):
            stale += 1                     # newest answer is still v1
    return stale / trials
for w, r in [(1, 1), (2, 1), (1, 2), (2, 2), (3, 1)]:
    print(f"N=3 W={w} R={r}: stale {stale_rate(3, w, r):.3f} at once,"
          f" {stale_rate(3, w, r, 10):.3f} after 10 ms")
```

It prints stale-read rates of 0.664, 0.331 and 0.334 for the three partial quorums, matching the formula's 2⁄3, 1⁄3 and 1⁄3, falling to 0.245, 0.123 and 0.045 ten milliseconds later; both strict quorums print 0.000. Even with $R + W > N$, a quorum system is not linearizable: two concurrent writes can land in different orders on different replicas, a write that failed after reaching one replica can still be read later, and last-writer-wins depends on clocks. For a compare-and-set on a seat you need consensus (Chapter 18); Cassandra's lightweight transactions (`IF NOT EXISTS`) run Paxos for exactly that reason.

16.8.1 Repairing the copies

- **Read repair**: when the replicas asked by a read disagree, the coordinator writes the newest version back to the stale ones. In Cassandra 4.0 and later this happens when a read consults several replicas and their digests mismatch (`read_repair = 'BLOCKING'`, the default); the old probabilistic `read_repair_chance` option is gone.
- **Hinted handoff**: when a replica is down, the coordinator stores the write as a hint and replays it when the replica returns. Cassandra keeps hints for `max_hint_window` (3 hours by default); a node down longer needs a full repair. Cassandra hints do not count toward the consistency level, except for `ANY`.
- **Sloppy quorum**: Dynamo and Riak go further and let a write count acknowledgements from healthy nodes *outside* the key's *N* home replicas during a partition, handing the data back later. Writes stay available, but $R + W > N$ no longer guarantees overlap until the hand-back.
- **Anti-entropy**: a background process compares replicas with Merkle trees and copies the differences (`nodetool repair`); the tree itself is in Chapter 19.

Cassandra makes *N*, *W* and *R* explicit. Per data center replication factors set *N*; the consistency level of each request sets *W* or *R*:

```
CREATE KEYSPACE demo WITH replication = {   -- N = 3 per data center
  'class': 'NetworkTopologyStrategy', 'virginia': 3, 'frankfurt': 3};
CONSISTENCY LOCAL_QUORUM;   -- cqlsh: 2 of 3 in the local data center only
```

`QUORUM` across both regions means 4 of 6 replicas, so every request crosses the ocean, and `EACH_QUORUM` waits for 2 of 3 in every region; `LOCAL_QUORUM` keeps latency local and replicates across regions asynchronously. See also *50 System Designs* #3 (Distributed Key-Value Store).

16.9 Conflict resolution

Multi-leader and leaderless systems must decide what happens when two writes to one item are **concurrent**: neither saw the other. There are four answers, from crudest to richest.

- **Last writer wins (LWW)**: attach a timestamp, keep the highest. Simple and convergent, and it silently drops the other write; clock skew between nodes decides which one. DynamoDB global tables (default mode, per item) and Cassandra (per column value) resolve conflicts this way. Fine when writes are whole-value replacements of data nobody shares.
- **Version vectors**: each replica keeps a counter per replica that has written the item. Comparing two vectors says whether one write saw the other (keep the later) or they are concurrent (keep both as *siblings* and let the application merge). Riak and the original Dynamo work this way.
- **CRDTs** (conflict-free replicated data types): data types whose merge is commutative, associative and idempotent, so replicas can merge in any order, any number of times, and converge without coordination: counters, sets (add-wins OR-Set), registers, maps and text sequences. Redis Enterprise Active-Active and Riak use them; Automerge and Yjs use sequence CRDTs for collaborative editing.
- **Application merge**: custom code reads the siblings and decides (union two shopping carts, keep the larger refund).

Version vectors detect concurrency; a G-Counter is the simplest CRDT, one slot per region, merged by taking the maximum of each slot:

```
def compare(a: dict, b: dict) -> str:
    """Compare two version vectors {replica: counter}."""
    keys = a.keys() | b.keys()
    a_le = all(a.get(k, 0) <= b.get(k, 0) for k in keys)
    b_le = all(b.get(k, 0) <= a.get(k, 0) for k in keys)
    if a_le and b_le:
        return "equal"
    return "a before b" if a_le else "b before a" if b_le else "concurrent"
def merge_counter(a: dict, b: dict) -> dict:
    """G-Counter CRDT: each region increments only its own slot."""
    keys = sorted(a.keys() | b.keys())
    return {k: max(a.get(k, 0), b.get(k, 0)) for k in keys}
print(compare({"va": 2, "fra": 1}, {"va": 3, "fra": 1}))   # a before b
print(compare({"va": 2, "fra": 1}, {"va": 1, "fra": 2}))   # concurrent
# Page views for one event counted in both regions during a partition:
va, fra = {"va": 700, "fra": 200}, {"va": 650, "fra": 260}
merged = merge_counter(va, fra)
print(merged, sum(merged.values()))      # {'fra': 260, 'va': 700} 960
assert merge_counter(va, fra) == merge_counter(fra, va)   # order-free
```

The counter cannot lose increments, however the copies are exchanged; a plain integer with LWW would have reported 900 or 910 and lost the rest. Clocks, version vectors and their cousins are developed in Chapter 18; a seat, however, has no meaningful merge, so no conflict resolution scheme can make it multi-leader.

16.10 Where each model runs

Model	Open-source systems	Managed services
Single leader	Postgres, MySQL, MongoDB, Redis, Kafka partitions	RDS, Aurora, ElastiCache, MemoryDB (Cloud SQL, AlloyDB, Azure Database for PostgreSQL)
Leader per shard, by consensus	CockroachDB, TiDB, YugabyteDB, etcd	DynamoDB, Spanner
Multi-leader	MySQL Group Replication (multi-primary), EDB Postgres Distributed, CouchDB	DynamoDB global tables, Cosmos DB multi-region writes
Leaderless	Cassandra, ScyllaDB, Riak	self-managed; Keyspaces offers the Cassandra API on different internals
Async object copies	MinIO site replication	S3 Cross-Region Replication (GCS dual-region, Azure GRS)

Two patterns dominate modern designs. Inside a region, a *leader per shard with synchronous copies in three zones* (Aurora's six-copy storage, DynamoDB's three replicas, a CockroachDB range's Raft group) gives zero data loss on a zone failure at about a millisecond per write. Across regions, *asynchronous copies with one write region per data item* give

local reads everywhere and an RPO of about a second. S3 Replication Time Control, for example, commits to replicating 99.99 percent of new objects within 15 minutes. Pay for synchronous cross-region writes (Spanner multi-region, DynamoDB multi-Region strong consistency) only for data where losing a second of writes is worse than adding a round trip to every write.

16.11 In Encore: two regions

Encore runs in us-east-1 (Virginia) and eu-central-1 (Frankfurt), about 90 ms apart. Each data set gets the cheapest replication that meets its loss and staleness budget:

Data	Model	In the region	To the other region
Seat inventory	leader per event	semi-sync, 1 of 2 zones	async standby
Orders, payments	single leader (Aurora)	6 copies, 3 zones	Aurora Global Database
Catalog, search	leader + read replicas	3 replicas	read replicas
Profiles, sessions	DynamoDB global table	3 zones	multi-leader, LWW
Ticket PDFs	S3	3 or more zones	Cross-Region Replication
Clickstream	Kafka, RF 3, `acks=all`	`min.insync.replicas=2`	mirrored to the lake

The replication stream is cheap even at the peak: 30,000 hold writes per second at an illustrative 400 bytes of WAL each is 12 MB/s, about 96 Mbit/s to each standby, and only for the minutes of the sale.

> **Design question.** Should Encore run seat inventory active-active in both regions?
>
> Active-active means a fan in Frankfurt and a fan in Virginia can both hold seat A12 before either region hears of the other; no merge rule can un-sell a seat. So each event has a home region (the venue's), which holds the only writable copy of its seats; the other region forwards holds there, and a Frankfurt fan pays one 90 ms round trip on a 10-minute flow. Reads (seat maps, catalog) are served locally from replicas and caches, slightly stale and corrected at hold time. If Virginia is lost, promote the Frankfurt standby: under a second of writes may be missing, so holds re-validate on the next click, and orders reconcile against the payment provider, whose records and idempotency keys say what was really paid. See also *50 System Designs* #34 (E-Commerce Checkout and Inventory) and #38 (Globally Distributed Database).

16.12 Numbers to remember

Figure	Value
Quorum overlap	$R + W > N$; majority of 3 is 2, of 5 is 3
Stale-read chance, $N = 3$, $W = R = 1$	2/3 right after the write
Majority of 3 up, nodes at 99%	99.97%
Sync commit, cross-zone vs cross-ocean	about 2 ms vs about 91 ms (illustrative)
Writes lost on async failover	write rate x lag
Aurora storage	6 copies, 3 zones, write 4/6, read 3/6
Aurora Replicas / Global Database lag	under 100 ms / under 1 s, typically
S3 Replication Time Control	99.99% of objects in 15 minutes

16.13 Pitfalls

- Treating a replica as a backup -> keep point-in-time backups; replicas copy mistakes instantly.
- Reading your own write from a lagging replica -> route by log position, or read from the leader just after a write.

- Making a cross-region copy synchronous for everything -> keep synchronous copies in the region, asynchronous across, and pay the round trip only where RPO must be zero.
- Failover without fencing -> elect through a consensus store, fence the old leader, and cap how far behind a promoted follower may be.
- Believing $R + W > N$ means linearizable -> use consensus or lightweight transactions for compare-and-set.
- Last-writer-wins on data that merges -> use a CRDT (counters, sets) or keep siblings.

16.14 Remember

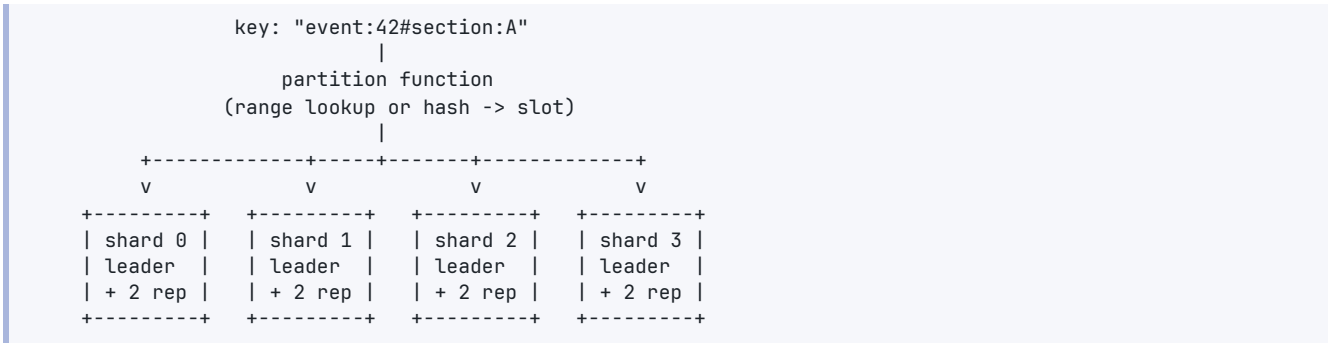
- Replicate for availability, durability, latency and read scale; partition for size and write scale.
- Single leader is the default; multi-leader and leaderless buy write availability with conflicts.
- Synchronous copies inside the region, asynchronous across regions; lost writes = rate x lag.
- Replication lag has three faces (read-your-writes, monotonic reads, consistent prefix), each with a cheap fix.
- Quorums need $R + W > N$ and still are not consensus.
- Give every item one home for writes when no merge rule exists, as for a seat.

17 Partitioning and consistent hashing

Replication copies the same data to many machines; partitioning cuts the data into pieces so each machine holds only some of it. This chapter answers one question: when one database can no longer hold Encore's data or absorb its writes, how do you split the data so every request still knows exactly where to go, the load stays even when one stadium show draws a million fans, and adding a machine moves as little data as possible? The whole chapter fits in one picture: a function from key to shard, and the choices you make about that function.

17.1 The picture

Think of a cloakroom at a stadium with ten counters. If the attendant decides "surnames A to C go to counter 1", coats are easy to find in alphabetical runs, but the counter holding "S" is mobbed. If instead every ticket number is scrambled into a counter number, the queues are even, but nobody can ask for "all coats from row 12" without visiting every counter. When an eleventh counter opens, you want to move a few coats to it, not reshuffle the whole room.



Every shard is itself replicated (Chapter 16), so a production cluster is a grid: partitions across, replicas down. The words differ by system — shard (MongoDB, Vitess, Elasticsearch), partition (DynamoDB, Kafka), region (HBase), range (CockroachDB), tablet (Bigtable), split (Spanner), vnode or token range (Cassandra) — but the idea is the same.

17.2 Why shard

You shard for one of three reasons, and you should be able to say which one:

1. **Size.** The data no longer fits on one machine, or fits but makes backups, restores and index rebuilds take too long.
2. **Write throughput.** Replicas scale reads, but every write still lands on one leader. When the leader's CPU, disk or log is saturated, only splitting the writes helps.
3. **Isolation and locality.** Keep a noisy tenant away from the others, or keep European users' data in a European region.

The number of partitions you need is set by whichever limit binds first:

$$P = max\left(\frac{S}{S_p}, \frac{T}{T_p}\right)$$

where S is the data size, S_p the size one partition should hold, T the peak operation rate and T_p the rate one partition sustains. Take Encore's seat state with DynamoDB's documented limit of 1,000 write units per second per partition (one unit is one write of an item up to 1 KB). An upper bound on size is 300,000 events × 80,000 seats × 100 bytes = 2.4 TB a year, if every event were a stadium; at about 10 GB per DynamoDB partition, that is 240 partitions by size. Throughput is the sharper constraint: the flash sale's 30,000 hold attempts per second, each a conditional write of a 100-byte item, need at least 30,000 / 1,000 = 30 partitions *for one event*, and only if the attempts spread evenly across them.

Sharding is not free. You give up cheap joins, cross-shard transactions and global uniqueness checks, and you take on routing, rebalancing and a harder operational life. Before sharding, exhaust the cheaper options: a bigger instance, read replicas, a cache, archiving cold data, or moving a hot workload to a store built for it. Encore's orders are a good

example of data that does *not* need sharding, as a design question below shows.

17.3 Range and hash partitioning

```
range:  [a .. f) -> s0   [f .. p) -> s1   [p .. z] -> s2    order kept
hash:   h(key) mod 64 -> partition 0..63 -> node map         order lost
```

Range partitioning assigns each shard a contiguous interval of keys, like volumes of an encyclopedia. The router keeps a sorted list of split points and binary-searches it. Range scans are cheap ("all orders between 09:55 and 10:05" touches one or two shards), and shards can split at any key when they grow. The danger is a monotonically increasing key: if the key is a timestamp or an auto-increment ID, every new write goes to the last shard while the others idle.

Hash partitioning applies a stable hash to the key and maps the hash to a partition. Load spreads evenly however skewed the keys look, but neighbouring keys land on unrelated shards, so a range query becomes a scatter-gather across all of them. Use a stable hash such as MD5, Murmur3 or xxHash; never a language's built-in `hash()`, which Python salts per process for strings.

```
import bisect
import hashlib

# Range partitioning: sorted split points; shard i holds [b[i-1], b[i])
BOUNDS = ["event:100000", "event:200000"]          # 3 shards
def range_shard(key: str) -> int:
    return bisect.bisect_right(BOUNDS, key)

# Hash partitioning: a stable hash, then a fixed number of partitions
PARTITIONS = 64
def hash_partition(key: str) -> int:
    digest = hashlib.md5(key.encode()).digest()
    return int.from_bytes(digest[:8], "big") % PARTITIONS

for k in ("event:004217", "event:150000", "event:299999"):
    print(k, range_shard(k), hash_partition(k))   # 0 8 / 1 21 / 2 37
```

The IDs are zero-padded because range routers compare keys as bytes: unpadded, "event:9" would sort after "event:150000".

Most real systems combine the two in a **compound key**: hash the first part to choose the partition, sort by the rest inside it. In DynamoDB the *partition key* is hashed and the *sort key* orders items within the partition; in Cassandra the *partition key* is hashed by the Murmur3 partitioner and *clustering columns* order rows within it. A key of `(event_id, seat_id)` puts one event's seats together, in seat order, while different events spread across the cluster. Bigtable, HBase, Spanner and CockroachDB are range-partitioned; they avoid the hot tail by asking you not to lead keys with a timestamp, or by hashing a prefix yourself.

17.4 Choosing a partition key

The partition key is the most consequential decision in a sharded design, because it is expensive to change later. A good key has three properties:

- **High cardinality**, so there are far more distinct values than partitions.
- **Even access**, so no single value receives a large share of reads or writes.
- **Locality for the common query**, so the hot path touches one partition. Pick the key from the access patterns, not from the entity diagram.

$$\text{load on hottest partition} \approx \max\left(\frac{T}{P},\ f_{\max} T\right)$$

where $f_{\max}$ is the share of traffic on the single most popular key. Adding partitions shrinks the first term but not the second: a key that draws 10 percent of traffic needs 10 percent of the capacity, however many partitions you have. This is the **hot key**, or **celebrity**, problem: one celebrity's timeline, one viral product, one stadium show (see *50 System Designs* #5 (News Feed) and #50 (Like and View Counter at Scale)).

Fixes, from cheapest to most invasive:

- **Cache the reads.** Hot reads belong in a cache or CDN in front of the partition (Chapter 9, Chapter 10).
- **Make the key finer.** Partition by `event#section` instead of `event`, so one event spreads over many partitions.
- **Salt the key (write sharding).** Append a suffix `0..k-1` to a hot key so its writes spread over *k* partitions; a read of the whole key fans out to all *k* and merges. Reserve salting for keys known to be hot.
- **Split counters.** A counter incremented on every sale becomes *k* sub-counters summed on read.
- **Isolate it.** Give the known-hot tenant or event its own dedicated partition or cluster.

> **Design question.** Encore's seat inventory is keyed by `event_id`. What happens at 10:00 on the day of the stadium sale, and how do you fix it?
>
> One event takes 30,000 hold attempts per second in the first minute; with `event_id` as the partition key, all of them hit one partition that sustains 1,000 writes per second (DynamoDB) or one Redis primary. Re-key to `event_id#section` with seat ID as the sort key: a 50,000-seat stadium with an illustrative 100 sections of 500 seats gives 300 attempts per second per section if spread evenly. Fans are not even: if the floor section draws 10 percent, that is 3,000 per second, so salt the hottest sections by row block into 4 sub-partitions (750 per second each). A hold touches one seat, so it stays a single-partition conditional write; the seat map read assembles 100 sections from a cache, not from the partitions. The trade-off is a wider fan-out to rebuild the full map, paid once per cache refresh rather than per fan. See also *50 System Designs* #25 (Ticket Booking).

17.5 Consistent hashing

17.5.1 Why `hash mod N` breaks

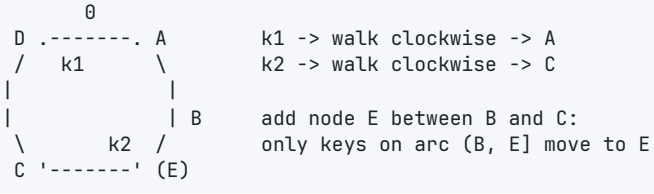
The obvious hash scheme is `shard = hash(key) mod N`. It spreads load perfectly and fails the moment *N* changes: a key stays put only if $h \bmod N = h \bmod (N+1)$, which holds for a fraction $1/(N+1)$ of keys. Growing from 10 to 11 shards moves

$$\text{moved}_{\text{mod}} = \frac{N}{N+1} = \frac{10}{11} \approx 90.9\% \text{ of all keys}$$

For a cache this is a cold restart; for a database it is a full reshuffle over the network. The ideal is to move only the share the new node should own, $1/(N+1)$, or 9.1 percent.

17.5.2 The ring

Picture a clock face numbered from 0 to $2^{64} - 1$. Hash each node's name to a point on the clock; hash each key to a point too, and walk clockwise: the first node you meet owns the key. Adding a node claims only the arc between it and its counter-clockwise neighbour; removing one hands its arc to the next node clockwise. Karger and colleagues introduced the scheme in 1997 for web caches.



With one point per node, arcs are very uneven. The arcs behave like independent exponential gaps, so the largest node's share is about $H_N = 1 + \frac{1}{2} + \ldots + \frac{1}{N} \approx \ln N + 0.58$ times the average: 2.9 times for 10 nodes, 5.2 times for 100. The fix is **virtual nodes**: give each physical node *V* points on the ring. A node's share is then a sum of *V* gaps, and its relative spread shrinks with the square root:

$$\frac{\sigma_{\text{load}}}{\mu_{\text{load}}} \approx \frac{1}{\sqrt{V}} \qquad \text{keys moved on adding a node} \approx \frac{K}{N+1}$$

With $V = 16$ the typical node is 25 percent off the mean; with $V = 100$, 10 percent; with $V = 256$, about 6 percent. Virtual nodes also let a bigger machine take more points (weighting), and when a node fails its load scatters across many survivors instead of doubling one neighbour. The cost is a larger routing table: $N \times V$ points to store and search, which is trivial at hundreds of nodes.

```
import bisect
import hashlib
from collections import Counter


def h64(s: str) -> int:
    """Stable 64-bit hash (Python's hash() is salted per process)."""
    return int.from_bytes(hashlib.md5(s.encode()).digest()[:8], "big")


class Ring:
    def __init__(self, nodes, vnodes=100):
        self.vnodes = vnodes
        self.points = []          # sorted list of (position, node)
        for n in nodes:
            self.add(n)

    def add(self, node):
        for i in range(self.vnodes):
            bisect.insort(self.points, (h64(f"{node}#{i}"), node))

    def remove(self, node):
        self.points = [p for p in self.points if p[1] != node]

    def owner(self, key):
        i = bisect.bisect(self.points, (h64(key), ""))
        return self.points[i % len(self.points)][1]   # wrap around


keys = [f"event:{i}" for i in range(100_000)]
ring = Ring([f"db{i}" for i in range(10)], vnodes=100)
before = {k: ring.owner(k) for k in keys}
ring.add("db10")
moved = sum(before[k] != ring.owner(k) for k in keys)
print(f"moved {moved / len(keys):.1%}")             # ideal 1/11 = 9.1%
load = Counter(ring.owner(k) for k in keys)
print(f"max/mean load {max(load.values()) / (len(keys) / 11):.2f}")
```

One run prints `moved 8.0%` and `max/mean load 1.22`: close to the ideal 9.1 percent, and the busiest of 11 nodes carries 22 percent more than average. A Monte Carlo check of the spread formula (300 random rings of 10 nodes) gives 0.23 at $V = 16$, 0.09 at $V = 100$ and 0.06 at $V = 256$, matching $1/\sqrt{V}$.

Consistent hashing runs in Cassandra (each node owns `num_tokens` token ranges on a Murmur3 ring; the default is 16 since Cassandra 4.0, with a token allocator that keeps them balanced), in Amazon's original Dynamo, in Riak, in client-side Memcached libraries (Ketama), and in load balancers that pin requests to cache servers (Chapter 8). Note what it does *not* fix: a single hot key still lands on one node.

17.6 Rendezvous and jump hashing

Two later schemes reach the same goal, minimal movement, with no ring at all.

Rendezvous hashing, or highest random weight (HRW; Thaler and Ravishankar, 1996), is an auction: for each key, every node computes a score `hash(node, key)` and the highest bid wins. Remove a node and only the keys it won go to their second-highest bidder; add a node and it wins about $1/(N+1)$ of the keys from everyone. Load is even without virtual nodes, because each key is an independent contest, and the runner-up list gives you replica placement for free (the top three bidders hold the three copies). The cost is $O(N)$ hashes per lookup, which is nothing for 10 nodes and noticeable for 10,000.

Jump consistent hash (Lamping and Veach, Google, 2014) maps a 64-bit key to a bucket in $[0, N)$ with no table, in a few lines and $O(\ln N)$ time. As N grows to $N+1$, each key jumps to the new bucket with probability $1/(N+1)$ and otherwise stays. Its restriction: buckets are numbered, and you can only add or remove at the end. That fits partitions of a storage system (bucket i is shard i, itself replicated) better than a pool of servers that die in any order.

```
import hashlib


def h64(s: str) -> int:
    return int.from_bytes(hashlib.md5(s.encode()).digest()[:8], "big")


def hrw_owner(key: str, nodes: list[str]) -> str:
    """Rendezvous hashing: every node bids, the highest score wins."""
    return max(nodes, key=lambda n: h64(f"{n}|{key}"))


def jump_hash(key: int, buckets: int) -> int:
    """Lamping and Veach (2014): bucket in [0, buckets), no memory."""
    b, j = -1, 0
    while j < buckets:
        b = j
        key = (key * 2862933555777941757 + 1) % 2**64
        j = int((b + 1) * (2**31 / ((key >> 33) + 1)))
    return b


nodes = [f"db{i}" for i in range(10)]
keys = [f"event:{i}" for i in range(100_000)]
a = {k: hrw_owner(k, nodes) for k in keys}
b = {k: hrw_owner(k, nodes + ["db10"]) for k in keys}
print(f"HRW moved {sum(a[k] != b[k] for k in keys) / len(keys):.1%}")
moved = sum(jump_hash(h64(k), 10) != jump_hash(h64(k), 11) for k in keys)
print(f"jump moved {moved / len(keys):.1%}")
```

It prints `HRW moved 8.9%` and `jump moved 9.1%`, both at the ideal 1/11.

Scheme	Lookup	Memory	Balance	Any node removable
`hash mod N`	$O(1)$	none	even	moves almost all
Ring + vnodes	$O(\log NV)$	$N \times V$ points	$\approx 1/\sqrt{V}$	yes
Rendezvous	$O(N)$	node list	even	yes
Jump hash	$O(\ln N)$	none	even	only the last
Maglev	$O(1)$	lookup table	near even	yes, small churn

Maglev hashing, Google's load-balancer table (covered in Chapter 8), trades a little extra churn for constant-time lookups. All five spread *keys* evenly; none of them splits one hot key.

17.7 Rebalancing strategies

Rebalancing moves data when nodes join, leave or run hot. It should keep the cluster serving during the move, move no more than necessary, and never be fully automatic on a hair trigger: a slow node mistaken for a dead one can start a rebalance that overloads everyone else. Four strategies cover almost every system:

- **Many fixed partitions.** Create far more partitions than nodes up front and move whole partitions, never re-hash keys. Redis Cluster has exactly 16,384 hash slots (`CRC16(key) mod 16384`); six primaries own about 2,731 each, and adding a seventh migrates about 2,341 slots to it. Couchbase uses 1,024 vBuckets; Elasticsearch fixes an index's primary shard count at creation (change it by split, shrink or reindex). Choose the count for the largest cluster you foresee, since it is hard to change.
- **Dynamic splitting.** Range-partitioned stores split a partition when it grows past a size or load threshold and merge small ones: HBase regions, Bigtable tablets, Spanner splits, CockroachDB ranges (512 MiB by default), MongoDB

ranges (128 MB by default, moved by the balancer). DynamoDB splits partitions for size and for heat on its own. Splits follow the data, but a new table starts with one partition unless you pre-split.

- **Proportional to nodes.** Each node owns a fixed number of ranges (Cassandra's `num_tokens`), so a new node takes ranges from everyone and the partition count grows with the cluster.
- **Fixed partitions that cannot move keys.** A Kafka topic's partition count can grow but never shrink, and the default producer picks `murmur2(key) mod partitions`, so adding partitions remaps keys and breaks per-key ordering across the change. Provision partitions for peak consumer parallelism on day one (Chapter 14).

```
> CLUSTER KEYSLOT foo
(integer) 12182
> CLUSTER KEYSLOT hold:{ev42:A}:12
(integer) 13180
> MSET hold:{ev42:A}:12 u1 hold:{ev42:B}:7 u2
(error) CROSSSLOT Keys in request don't hash to the same slot
```

The `{...}` **hash tag** tells Redis to hash only the text inside the braces, so keys that share a tag share a slot and can be used together in `MULTI`, Lua scripts and multi-key commands (Chapter 9). Tags are a partition key in disguise: tag by event and the event becomes a hot slot.

17.8 Request routing

Somebody must turn "key" into "node address", and that somebody must learn when partitions move.

```
1. smart client          2. routing tier          3. any node
client (has map)         client                   client
  |                        |                        |
  v                        v                        v
shard B                  proxy (has map)          node A (coordinator)
                           |                        |  forwards
                           v                        v
                         shard B                  shard B
```

1. **Smart client.** The driver caches the partition map and connects straight to the owner: Redis Cluster clients, Cassandra's token-aware drivers. A stale map is corrected by the server: Redis replies `-MOVED 13180 10.0.1.12:6379` when a slot has moved for good (update the map) and `-ASK` while a slot is mid-migration (retry there once). Fastest path, but every client language needs the logic.
2. **Routing tier.** A proxy owns the map and clients stay simple: MongoDB's `mongos`, Vitess's VTGate, Envoy's Redis proxy, Citus's coordinator. One extra hop and one more fleet to scale.
3. **Any node.** Clients reach any node, which acts as coordinator and forwards: Cassandra coordinators, DynamoDB's request routers behind one endpoint.

The map itself lives in a strongly consistent store or spreads by gossip: Vitess keeps it in a topology service (etcd, ZooKeeper or Consul), MongoDB in its config server replica set, Kafka in the KRaft controller quorum, Cassandra in gossip (Chapter 18).

17.9 Secondary indexes

A query by the partition key goes to one shard. A query by anything else ("orders for this e-mail address") needs a secondary index, and there are two ways to partition it.

```
local (by document)                global (by term)
shard 0: orders + index of its own  index shard "a-m": email -> order ids
shard 1: orders + index of its own  index shard "n-z": email -> order ids
read: ask every shard, merge        read: ask one index shard
write: one shard                    write: data shard + index shard
```

- **Local index**: each shard indexes only its own rows. Writes stay on one shard and the index is always consistent, but every read by the indexed column becomes a scatter-gather. DynamoDB local secondary indexes, Cassandra's secondary indexes and SAI, MongoDB indexes on non-shard-key fields, Elasticsearch shards.

- **Global index**: the index is partitioned by the indexed value itself. Reads hit one index partition; a write touches two places, so the index is usually updated asynchronously. DynamoDB global secondary indexes are eventually consistent, have their own partitions and capacity, and a throttled index throttles writes to the base table.

Scatter-gather has a tail problem. If each shard is slow on 1 percent of requests, a query that waits for all n shards is slow with probability

$$P(\text{slow}) = 1 - (1 - p)^n \qquad 1 - 0.99^{32} \approx 27.5\%$$

for Citus's default of 32 shards, and 63 percent at 100 shards. Use global indexes for frequent lookups and keep scatter-gather for rare or analytical queries.

17.10 Cross-shard queries and transactions

The best cross-shard transaction is the one you design away. **Co-locate** data that changes together under the same partition key: Citus places rows of co-located tables with the same distribution column value on the same node, Spanner interleaves child rows under their parent's key, DynamoDB keeps items with the same partition key together, and Redis hash tags put keys in one slot. Then most transactions are single-shard and run at single-node speed.

When a transaction must span shards, the options are, in rising cost:

- **Avoid atomicity**: make each step idempotent and order them so a crash leaves a recoverable state.
- **Bounded multi-item transactions**: DynamoDB `TransactWriteItems` covers up to 100 items across tables and partitions.
- **Two-phase commit** across shards: Spanner and CockroachDB run it over Paxos or Raft groups, so a coordinator crash does not block forever; plain 2PC can.
- **Sagas** with compensating steps for long business flows such as hold, pay, issue ticket (Chapter 18).

Cross-shard *reads* follow the same rule: joins on the partition key push down to each shard; joins on anything else move data. Analytical questions belong in the lakehouse, fed by change data capture (Chapter 21), not in scatter-gathers against the serving shards.

> **Design question.** Suppose Encore, grown tenfold, shards orders by `user_id` so "my tickets" is one shard. How does an organizer's live sales dashboard for one event work, and how does the support agent find an order from an e-mail?
>
> Sales per event cross every user shard: scattering 200 dashboard refreshes a minute across all shards multiplies load and inherits the tail of the slowest one. Instead, publish each order event to a stream keyed by `event_id` and keep per-event counters in a store partitioned by event, updated within seconds (Chapter 14). For the agent, keep a global index `email -> user_id` (and `order_id -> user_id`, or embed the shard in the order ID); one index lookup, then one shard read. Both are eventually consistent by a second or two, which is acceptable for dashboards and support but not for seat holds. See also *50 System Designs* #28 (Ad Click Aggregator).

17.11 Where it runs

System	Scheme	Unit	Routing
DynamoDB	hash of partition key	partition, auto-split	request router
Cassandra	Murmur3 ring, 16 tokens/node	token range	any node
Redis Cluster	CRC16 mod 16,384	hash slot	client, MOVED/ASK
Kafka	murmur2(key) mod count	partition	client metadata
Vitess	vindex to keyspace ID range	shard (`-80`, `80-`)	VTGate
Citus	hash of distribution column	shard (32 default)	coordinator
MongoDB	ranged or hashed shard key	range (128 MB)	`mongos`

System	Scheme	Unit	Routing
Spanner	key range, load splits	split (Paxos group)	managed front end

On AWS, DynamoDB is the default for partitioned key-value data (Bigtable or Firestore on GCP, Cosmos DB on Azure, which caps a logical partition at 20 GB). ElastiCache and MemoryDB run Redis OSS and Valkey in cluster mode (Memorystore on GCP, Azure Managed Redis). For sharded relational data, Aurora Limitless Database shards Postgres on AWS, Spanner is native on GCP, and Azure Cosmos DB for PostgreSQL is managed Citus.

17.12 In Encore

Each Encore store gets the partitioning its access pattern asks for:

Data	Partition key	Why
Seat state and holds	`event_id#section`, sort by seat	spreads one show, holds stay single-key
Hold TTLs in Redis	slot tag `{event:section}`	Lua script on one slot
Orders	not sharded yet; later `user_id`	fits one Postgres primary
Clickstream	Kafka, keyed by `session_id`	per-session order, even spread
Search index	Elasticsearch shards by document	scatter-gather is the design

> **Design question.** Does Encore need to shard its orders database today?
>
> 50 million orders a year at 2 KB is 100 GB a year: ten years fit on one Postgres instance. The average write rate is 1.6 orders per second; selling all 50,000 stadium seats in ten minutes is about 83 seats, or 33 orders, per second, which one primary absorbs easily because the 30,000 hold attempts per second land on the seat store, not on orders. So: one primary with read replicas, partitioned *by time inside* Postgres for archiving, and order IDs that already carry a shard field so a later split needs no ID change. Shard when write rate or size, not fashion, demands it. See also *50 System Designs* #24 (Payment System).

External systems and agents touch partitioning at the edges. The payment provider's webhook carries only Encore's order ID, so put the shard into the ID (Instagram's Snowflake-style IDs carry a 13-bit logical shard; *50 System Designs* #21 (Unique ID Generator)) and the webhook handler routes without a lookup. The support agent's conversation state is partitioned by `conversation_id`, and its order-lookup tool must hit the global `email -> user_id` index rather than scatter across shards on every turn; a tool that fans out to 32 shards per call multiplies cost and tail latency on every agent step. Vector stores for retrieval shard by document and merge the top-k results of every shard, the same scatter-gather trade-off (*Designing Agentic Systems* covers retrieval in depth).

17.13 Numbers to remember

Fact	Value
Keys moved, `mod N` to $N+1$	$N/(N+1)$, 91% at 10 to 11
Keys moved, consistent hashing	$1/(N+1)$, 9% at 10 to 11
Load spread with V vnodes	$\approx 1/\sqrt{V}$ (16: 25%, 256: 6%)
Hottest node, one token each	$\approx \ln N + 0.58$ times mean
Redis Cluster slots	16,384, CRC16
DynamoDB partition	3,000 RCU, 1,000 WCU per second
Cassandra `num_tokens` default	16 (since 4.0)
Citus default shard count	32
MongoDB range, CockroachDB range	128 MB, 512 MiB
Scatter over 32 shards, 1% slow	27.5% of queries slow

Fact	Value
Encore holds, first minute	30,000/s, at least 30 partitions

17.14 Pitfalls

- Partitioning by a timestamp or auto-increment ID -> all writes hit the newest range; lead with a hashed or high-cardinality key.
- Choosing the key from the entity diagram -> list the hot queries first and key for them.
- Ignoring the celebrity key -> finer keys, salting, split counters, and a cache in front.
- `hash(key) mod N` for a cluster that grows -> fixed slots, a ring with vnodes, rendezvous or jump hash.
- Too few fixed partitions (Kafka, Elasticsearch) -> provision for the largest foreseeable cluster and consumer count.
- Python's built-in `hash()` for routing -> it is salted per process; use a stable hash.
- Global secondary index read as if consistent -> it lags; read the base table when correctness needs it.
- Scatter-gather on the hot path -> a global index, co-location, or precomputed views.
- Cross-shard transactions by default -> co-locate by partition key; sagas or bounded transactions for the rest.
- Sharding too early -> a bigger box, replicas and a cache first; shard for a measured limit.

17.15 Remember

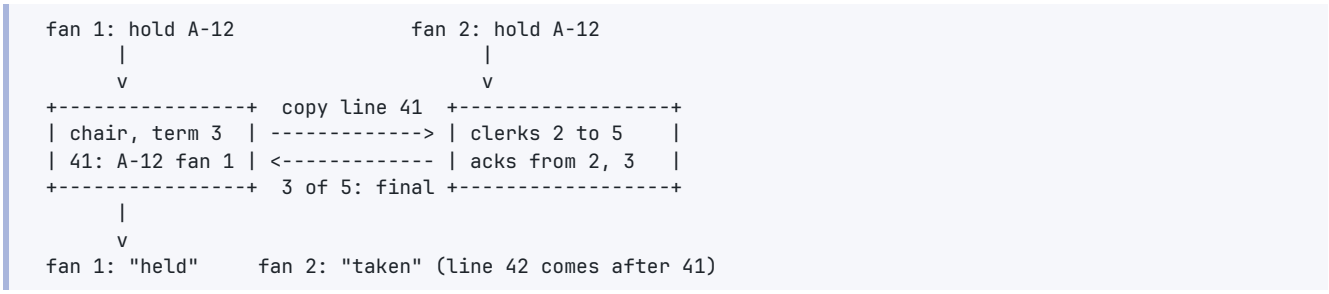
- Shard for size, write throughput or isolation; size it with $P = max(S/S_p,\ T/T_p)$.
- The partition key decides everything: high cardinality, even access, one partition for the hot query.
- Range keeps order and risks hot tails; hash spreads load and loses range scans; compound keys get both.
- Consistent hashing, rendezvous and jump hash move only $1/(N+1)$ of keys; virtual nodes even the load to about $1/\sqrt{V}$.
- No hash function fixes a hot key: Encore's flash sale needs `event#section` keys, salting for the hottest sections, and a cache for the seat map.
- Design cross-shard work away with co-location; route with smart clients, proxies or coordinators backed by a consistent map.

18 Consistency, clocks and consensus

When Encore's data lives on several machines in two regions, how do they agree on who holds seat A-12, in what order things happened, and what a reader is allowed to see — even while messages are delayed, clocks disagree and servers crash? This chapter answers with one picture: a committee that writes every decision into a numbered minute book, where a decision counts only once a majority has copied it.

18.1 The picture

Picture five clerks in five offices, each with a copy of the minute book for the stadium show. Phones are unreliable and the wall clocks in the offices disagree by a few seconds. One clerk is the chair: requests go to the chair, who writes line 41 ("A-12 held by fan 1") and phones the others. As soon as three of five (a majority) have copied line 41, it is final. If the chair falls silent, the others elect a new chair; any new majority overlaps the old one in at least one clerk, so no final line is ever lost. The minute book is the replicated log, the majority rule is consensus, and the line numbers stand in for clocks.



18.2 CAP and PACELC

Picture two box offices selling the same show, connected by one phone line. The line goes dead. Each office can keep selling (and risk selling seat A-12 twice) or stop selling until the line is back (and turn fans away). There is no third choice. That is CAP.

CAP stated correctly (Brewer's conjecture, proved by Gilbert and Lynch in 2002): when a network **partition** separates replicas, a system that keeps answering on both sides cannot also guarantee **linearizable consistency**; it must choose, per operation, between consistency (C, refuse or wait) and availability (A, answer from local, possibly stale data). Three common misreadings:

- "Pick two of three" is wrong. Partitions are not optional, so the real choice is C or A *during* a partition. With no partition you can have both.
- C in CAP means linearizability, not the C of ACID (which means "invariants hold").
- A in CAP means every non-failed node answers. A CP system can still be highly available in practice, since partitions are rare and a majority side keeps working.

PACELC (Abadi) adds the case that matters most of the day: *if Partition, choose Availability or Consistency; Else, choose Latency or Consistency*. Even with a healthy network, a strongly consistent write must wait for other replicas, and a consistent read must reach the leader or a quorum. That wait is the price.

$$t_{\text{write}} \approx t_{\text{local}} + \text{RTT}_{\text{to the slowest replica in the quorum}}$$

With replicas in three zones of one region (RTT about 1 ms, illustrative) a consistent write costs a millisecond or two. With a quorum that must cross the Atlantic (Virginia to Frankfurt, about 90 ms RTT, illustrative) every write pays about 90 ms. That is why systems choose differently:

System	Partition	Else	Note
DynamoDB (default reads)	A	L	Opt into strong reads

System	Partition	Else	Note
Cassandra	A	L	Tunable per query
Spanner, CockroachDB	C	C	Majority quorum per range
etcd, ZooKeeper	C	C	Minority side refuses
DynamoDB global tables MRSC	C	C	Exactly three Regions

Amazon DynamoDB (GCP Firestore, Azure Cosmos DB) global tables default to multi-Region eventual consistency (MREC) with last-writer-wins conflicts; the multi-Region strong consistency (MRSC) mode replicates each write synchronously to at least one other Region and must span exactly three Regions (or two replicas plus a witness). The choice is per data item, not per company: Encore can be CP for seat holds and AP for the seat map.

18.3 Consistency models

A consistency model is a contract: given the writes that happened, which values may a read return? Picture one write and three readers on a timeline. Seat A-12 changes from `free` to `held` at the moment marked `W`.

```
time ------------------------------------------------------>
writer   |--- W: A-12 = held ---|
reader 1                          R -> ?   (after W finished)
reader 2              R -> ?               (overlaps W)
reader 3 (the writer)                R -> ? (its own write)
```

Model	Picture	Reader 1 may see
Linearizable	One copy, one instant per op	held, always
Sequential	One order, maybe not real time	free (if ordered before W)
Causal	Effects never before causes	free, unless it saw a cause
Read-your-writes	You see your own writes	free; reader 3 sees held
Eventual	Replicas converge, some day	free, for a while

- **Linearizable** (strong): the system behaves like a single copy, and every operation takes effect at one instant between its start and its end. Once any read returns `held`, no later read returns `free`. Needed for locks, leader election, seat holds and uniqueness (“one username”).
- **Sequential**: all nodes see all operations in one total order that respects each client’s own order, but that order may lag real time. A reader can be “behind” yet never sees things out of order.
- **Causal**: if write B was made by someone who had seen write A, everyone sees A before B. Concurrent writes may appear in different orders. Picture a reply that never shows up before the message it answers. It is close to the strongest model that can stay available during a partition.
- **Session guarantees** (read-your-writes, monotonic reads, consistent prefix): promises to one client, covered in Chapter 16. A fan who held a seat must see it held on the next page load.
- **Eventual**: if writes stop, replicas converge. No promise about when, and no promise about order before that.

The models nest: linearizable implies sequential implies causal implies the session guarantees, and all imply eventual. Stronger costs coordination, and coordination costs round trips (PACELC’s L). Serializability is a different axis: it is about multi-object *transactions* appearing in some serial order; **strict serializability** (Spanner’s “external consistency”) is serializability plus linearizable real-time order. See Chapter 12 for isolation levels.

In DynamoDB, a read is eventually consistent unless you pass `ConsistentRead=True`; a strong read uses one read capacity unit per 4 KB, twice the eventually consistent rate, and is not available on global secondary indexes. Postgres gives linearizable reads only on the primary; replicas lag (see Chapter 16).

18.4 Physical clocks and skew

Every server keeps time with a quartz oscillator that runs slightly fast or slow (**drift**, tens of parts per million; Spanner budgets a worst case of 200 microseconds per second). NTP corrects it by asking a time server and halving the round trip, so the error is bounded by about half the RTT plus jitter: around a millisecond inside a data center, tens of milliseconds over the internet. The difference between two servers' clocks at one instant is **skew**.

$$\text{skew after } t \text{ seconds} \le 2\rho t + \varepsilon_{\text{sync}}, \qquad \rho = 200 \times 10^{-6}$$

Worked: two unsynchronized servers drifting in opposite directions at 200 ppm diverge by up to $2 \times 200 \times 10^{-6} \times 600\ \text{s} = 0.24\ \text{s}$ over one 10-minute seat hold. Three rules follow:

- Measure durations with a **monotonic clock** (`time.monotonic()` in Python), never the wall clock, which NTP can step backwards.
- Never order events on different machines by wall-clock timestamps alone. Last-writer-wins with skewed clocks silently drops the write that really came last.
- Treat a timestamp from another machine as an interval, not a point.

Cloud clocks are better than they used to be. The Amazon Time Sync Service (GCP and Azure run their own NTP services for VMs) is reachable from every EC2 instance, and supported instances expose a PTP hardware clock for microsecond-level accuracy; the open-source ClockBound daemon reports the current error bound with each reading, an interval like TrueTime's.

18.5 Logical clocks: Lamport, vector and hybrid

Picture a chain of letters: you can always tell that a reply was written after the letter it answers, even if nobody owns a watch. Logical clocks capture that **happened-before** relation: A happened before B if they are on the same machine in that order, or A is a send and B the matching receive, or by transitivity.

- **Lamport clock**: one counter per node. Increment on each event; stamp messages; on receive, set `t = max(t, msg_t) + 1`. If A happened before B then L(A) < L(B). The converse fails: L(A) < L(B) does not prove causality. Adding the node id breaks ties into a total order.
- **Vector clock**: one counter *per node* in every stamp. Compare entrywise: all less-or-equal means before, the reverse means after, anything else means **concurrent**, a real conflict to resolve. Cost: N entries per stamp, so it suits few writers (two regions, a handful of replicas). Dynamo-style stores and Riak used version vectors.
- **Hybrid logical clock (HLC)**: a pair (physical milliseconds, counter). It stays within the clock skew of real time, yet never goes backwards and always respects happened-before. CockroachDB and YugabyteDB stamp transactions with HLCs; MongoDB uses a similar cluster time.

```
def vc_compare(a, b):
    keys = a | b
    le = all(a.get(k, 0) <= b.get(k, 0) for k in keys)
    ge = all(a.get(k, 0) >= b.get(k, 0) for k in keys)
    if le and ge:
        return "equal"
    return "before" if le else "after" if ge else "concurrent"


us = {"us": 2, "eu": 0}             # hold written in us-east-1
eu = {"us": 1, "eu": 1}             # edit in eu-central-1 that saw us:1
print(vc_compare(us, eu))           # concurrent -> a real conflict
print(vc_compare({"us": 1}, us))    # before -> safe to overwrite
```

The HLC rules are short: on send, take `max(wall clock, last l)` and bump the counter if the wall clock did not move; on receive, take the max of the local, message and wall times and bump the counter past whichever matched. A receiver whose clock is 50 ms behind still stamps `(1000050, 1)` after a message stamped `(1000050, 0)`.

CockroachDB still needs bounded skew: its `--max-offset` defaults to 500 ms, and a node whose clock drifts beyond the limit crashes itself rather than risk serving inconsistent reads.

18.6 TrueTime and commit wait

Spanner turns clock uncertainty into a number and then waits it out. TrueTime (GPS receivers and atomic clocks in every data center) returns an interval `[earliest, latest]` guaranteed to contain true time, of width 2 . The 2012 Spanner paper reports varying between about 1 and 7 ms, about 4 ms most of the time.

To commit, the leader picks timestamp s = `TT.now().latest`, then **commit-waits** until `TT.now().earliest > s` before making the write visible. Any transaction that starts later, anywhere on Earth, gets a larger timestamp, so timestamp order equals real-time order (external consistency) with no messages between the two transactions.

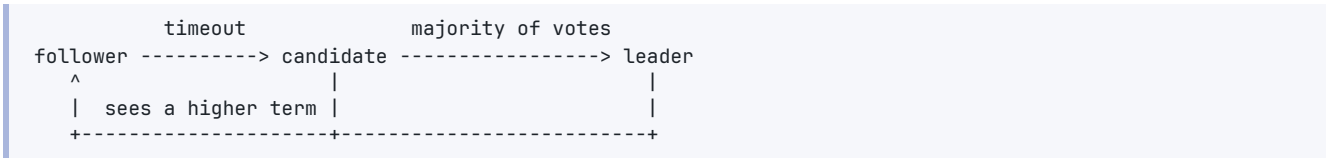
$$t_{\text{commit wait}} \approx 2 \quad = 2 \times 4 \text{ ms} = 8 \text{ ms}$$

The wait overlaps the Paxos replication round, so it often costs little extra. Without bounded-uncertainty clocks, systems settle for less: CockroachDB combines HLCs with an uncertainty window (restarting reads that fall inside it) and guarantees serializability, not Spanner's full external consistency. Spanner runs on GCP; the closest managed options elsewhere are Amazon Aurora DSQL (PostgreSQL-compatible, strongly consistent across peered Regions within one continent's Region set) and Azure Cosmos DB at its strong consistency level; see *50 System Designs* #38 (Globally Distributed Database).

18.7 Consensus and Raft

Consensus is a group of nodes agreeing on one value, and then on a sequence of values (a replicated log), such that a decided value is never undone, even when nodes crash and messages are lost or delayed. Most coordination problems reduce to it: electing one leader, granting one lock, assigning one seat, committing one configuration. A theorem (Fischer, Lynch and Paterson) shows no algorithm can guarantee progress in a fully asynchronous network with even one crash, so practical protocols are always *safe* and make progress whenever the network behaves for long enough, using timeouts.

Raft (Ongaro and Ousterhout, 2014) is the protocol most systems use today because it splits the problem into steps you can hold in your head.



- **Terms.** Time is divided into numbered terms, each with at most one leader. Every message carries the sender's term; a node that sees a higher term steps down to follower. Terms act as a logical clock that exposes stale leaders.
- **Election.** A follower that hears no heartbeat within its randomized election timeout (the paper suggests 150 to 300 ms; etcd defaults to 100 ms heartbeats and a 1,000 ms election timeout) becomes a candidate, increments the term, votes for itself and sends `RequestVote`. Each node grants one vote per term, and only to a candidate whose log is at least as up to date as its own (higher last term, or same last term and at least as long). A majority of votes makes a leader. Randomized timeouts make split votes rare.
- **Log replication.** Clients send commands to the leader. The leader appends the entry to its log and sends `AppendEntries` carrying the index and term of the entry just before it. A follower accepts only if its log matches at that point, so logs that agree on one entry agree on everything before it. On a mismatch the leader backs up and resends.
- **Commit.** An entry is **committed** once it is stored on a majority, and the leader only counts replicas for entries of its *current* term (older entries commit along with them). Committed entries are applied to the state machine in log order and the client gets its answer.
- **Safety.** Any two majorities intersect, and the vote rule ensures the winner already holds every committed entry. So a committed entry survives every future leader.

```python
def commit_index(match, log_terms, term):
    """Highest index stored on a majority whose entry is from this term.

    match: last replicated index per node, leader included.
    log_terms: term of each entry, log_terms[i] for index i (1-based).
    """
    majority = len(match) // 2 + 1
    for n in sorted(set(match), reverse=True):
        replicated = sum(1 for m in match if m >= n)
        if n and replicated >= majority and log_terms[n] == term:
            return n
    return 0


terms = [None, 1, 1, 2, 3, 3]          # entries 1..5, current term 3
print(commit_index([5, 5, 4, 2, 1], terms, 3))   # 4: on 3 of 5 nodes
print(commit_index([5, 3, 3, 1, 1], terms, 3))   # 0: entry 3 is term 2
```

The numbers. To survive f crashed nodes you need a majority that excludes them:

$$n = 2f + 1, \qquad \text{quorum} = \lfloor n/2 \rfloor + 1, \qquad A = \sum_{i=f+1}^{n} \binom{n}{i} p^i (1-p)^{n-i}$$

Three nodes tolerate one failure, five tolerate two; four nodes still tolerate only one, so even sizes add cost and no safety. With independent nodes each up 99 percent of the time ($p = 0.99$), a 3-node group is available 99.970 percent of the year (about 157 minutes down) and a 5-node group 99.999 percent (about 5 minutes). Independence is the catch: place the nodes in different zones. Each commit costs one round trip from the leader to the fastest majority plus a disk fsync, so a group inside one region commits in a millisecond or two, while a group whose majority spans regions pays the inter-region RTT on every write. Throughput is bounded by one leader per group, which is why databases run many groups: one per range or shard.

18.8 Paxos in one paragraph

Paxos (Lamport, written in 1989, published in 1998) came first and proves the same idea in two phases. A **proposer** picks a unique ballot number and asks a majority of **acceptors** to *prepare*: each promises to ignore lower ballots and reports any value it already accepted. The proposer must then propose the value with the highest ballot among those reports (or its own value if none), and the value is **chosen** once a majority *accepts* it. Multi-Paxos keeps one stable proposer (a leader) so later values skip the prepare phase, which makes it look much like Raft; Raft adds a strong leader, the log-matching rule and a precise membership-change procedure. Google's Chubby and Spanner use Paxos; ZooKeeper uses its own Zab protocol with the same majority idea.

18.9 Where consensus runs

System	Protocol	Used for
etcd	Raft	Kubernetes state, config, locks
ZooKeeper	Zab	Kafka (legacy), HBase, coordination
Consul	Raft	Service catalog, KV, sessions
Kafka KRaft	Raft variant	Cluster metadata, controller
Spanner	Paxos per split	Every write
CockroachDB, TiKV, YugabyteDB	Raft per range	Every write
MongoDB replica sets	Raft-like	Primary election, oplog

Kafka 4.0 (2025) removed ZooKeeper entirely; metadata lives in a Raft quorum of controllers (KRaft). In the cloud you rarely run consensus yourself: Amazon EKS (GKE, AKS) runs etcd for you, and DynamoDB conditional writes, Spanner and Aurora DSQL give you consensus-backed decisions as a database call.

Leader election as a service. Most applications never implement Raft; they borrow it. A Kubernetes controller with three replicas elects a leader through a `Lease` object in the `coordination.k8s.io` API: whoever creates or renews it within its lease duration is leader, and etcd's consensus makes the update atomic. The Lease records `holderIdentity`, `leaseDurationSeconds` and `renewTime`, and client-go's `leaderelection` package does the renewing. The same pattern works with an etcd lease, a ZooKeeper ephemeral node or a DynamoDB conditional write.

> **Design question.** Encore runs in two regions. Should the seat-inventory database run one consensus group with replicas in both?
>
> A majority must survive a failure. With two regions, one of them holds the majority (say 2 of 3 nodes in us-east-1); lose that region and the survivor cannot commit, so you gained no availability for writes. Spreading a 5-node group 3 + 2 has the same flaw. Every write would also pay the inter-region RTT (about 90 ms to Frankfurt, illustrative), against 30,000 hold attempts per second at the flash sale.
>
> Better: one strongly consistent store (a Multi-AZ primary with a synchronous standby, or a 3-zone Raft group) per event in its **home region**, with asynchronous replication to the other region for disaster recovery and seat-map reads. Fans abroad pay one cross-region round trip per hold, not per read. True multi-region strong consistency needs a third region or a witness, as DynamoDB MRSC requires. See also *50 System Designs* #25 (Ticket Booking).

18.10 Locks, leases and fencing tokens

Picture a hotel key card that stops working at noon. If the guest falls asleep and wakes at 12:05, the card no longer opens the door, whatever the guest believes. A distributed lock must be a **lease** like that card, because a holder can crash and never release. But a lease alone has a hole: the lock service has no way to stop a paused holder (a 30-second garbage-collection pause, a frozen VM, a slow network) from waking up and writing after its lease expired and someone else took over.

```
client A  [lease 1 ......... pause .............] write(token 1) -> REJECT
client B                 [lease 2] write(token 2) -> ok
storage                        highest token seen: 2
```

The fix is a **fencing token**: a number that grows every time the lock is granted, sent with every write, and checked by the resource itself, which rejects any token lower than the highest it has seen. The lease provides liveness; the token provides safety.

A holder must also stop early enough to absorb drift. If it asked for a lease of length L at local time t_0 (before the request left):

$$t_{\text{stop}} = t_0 + L\,(1 - \rho) - \delta, \qquad L = 30\ \text{s},\ \rho = 200\ \text{ppm} \quad L\rho = 6\ \text{ms}$$

where δ is a safety margin for the work in flight. The margin protects against drift, not against pauses; only fencing covers those.

```
import fakeredis

r = fakeredis.FakeRedis(decode_responses=True)
ACQUIRE = r.register_script(  # lock and token in one atomic step
    "if redis.call('set', KEYS[1], ARGV[1], 'NX', 'PX', ARGV[2]) then "
    "return redis.call('incr', KEYS[2]) else return 0 end")


def acquire(name, owner, ttl_ms):  # owner: a unique random id
    token = ACQUIRE(keys=[f"lock:{name}", f"fence:{name}"],
                    args=[owner, ttl_ms])
    return (owner, token) if token else (None, None)


class Storage:  # the protected resource checks every token
    def __init__(self):
        self.highest, self.value = 0, None

    def write(self, token, value):
        if token < self.highest:
            raise PermissionError(f"stale token {token} < {self.highest}")
        self.highest, self.value = token, value


store = Storage()
a_owner, a_tok = acquire("refund-42", "A-7f3e", ttl_ms=30_000)
r.delete("lock:refund-42")            # simulate: A pauses, lease expires
b_owner, b_tok = acquire("refund-42", "B-91c0", ttl_ms=30_000)
store.write(b_tok, "B")               # B writes with token 2
try:
    store.write(a_tok, "A")           # A wakes up, writes with token 1
except PermissionError as e:
    print(e)                          # stale token 1 < 2
```

Release with a compare-and-delete so you never delete someone else's lock: the Lua script `if get == owner then del`, or `DELEX key IFEQ owner` from Redis 8.4.

Why a Redis lock alone is not safe for correctness. Redis replication is asynchronous: the primary can grant the lock, crash before the replica copies it, and the promoted replica grants it again. Redlock (majority of five independent Redis primaries) narrows that window but still assumes bounded pauses and well-behaved clocks; Redis expires keys by wall clock, so a clock jump can free a lock early. Redis's own documentation now tells you to add fencing tokens. Use a Redis lock for **efficiency** (avoid two workers doing the same expensive job; a rare duplicate is harmless). For **correctness** (never two refunds, never two owners of a seat), take the lock from a consensus store (an etcd lease with its revision, or a ZooKeeper ephemeral sequential node with its zxid, both natural fencing tokens) or, better, skip the lock and make the write itself conditional: a version check in Postgres, a DynamoDB `ConditionExpression`, S3 `If-Match`. The token in the example above lives in Redis only for illustration; in production it must come from a store that cannot lose it on failover.

> **Design question.** Encore releases expired seat holds every second. How do you make sure exactly one releaser runs, and what happens when two run?
>
> Run three replicas of the hold-expirer and elect a leader through a Kubernetes `Lease` (15 s lease, retry every 2 s, as Kubernetes' own controllers do); a crashed leader is replaced within about 15 s, and holds last 10 minutes, so a short gap only delays releases. Do not trust the election for safety: during a pause two replicas may both think they lead. Make each release a conditional write, `UPDATE seat SET state = 'free' WHERE hold_id = ? AND state = 'held' AND hold_until < now()`, so a second releaser changes zero rows and a seat sold in the meantime is untouched. Elect for efficiency; fence or compare-and-set for correctness. See also *50 System Designs* #30 (Distributed Job Scheduler).

18.11 Distributed transactions: two-phase commit and sagas

Picture a wedding: the officiant asks each partner "do you?" and only after both say yes pronounces them married. Once a partner says yes, they cannot walk out until the officiant speaks. That is **two-phase commit (2PC)**.

```
coordinator             inventory DB, orders DB
    | --- 1. PREPARE ---------> |   write intent, keep locks
    | <-- 2. YES / NO --------- |   a YES is a promise
    | --- 3. COMMIT / ABORT --> |   all YES -> commit
```

In phase one each participant writes everything durably, keeps its locks and votes. In phase two the coordinator logs the decision and tells everyone. The cost is two round trips and several forced log writes per transaction, and the **blocking problem**: a participant that voted yes may not decide alone, so if the coordinator dies after the votes, participants hold their locks until it recovers. Postgres supports the participant side with `PREPARE TRANSACTION` and `COMMIT PREPARED` (disabled by default: `max_prepared_transactions = 0`); XA does the same across vendors. Spanner and CockroachDB make 2PC practical by running the coordinator and each participant as a replicated consensus group, so a "crashed coordinator" is replaced by its group's next leader.

2PC needs every participant to speak the protocol. Encore's payment provider does not, and holding seat locks across a card authorization that takes seconds is unacceptable. Across services, use a **saga**: a sequence of local transactions, each with a **compensation** that semantically undoes it, run in reverse when a later step fails.

```
def run_saga(steps):
    """steps: (action, compensation) pairs; undo done steps in reverse."""
    done = []
    for action, undo in steps:
        try:
            action()
        except Exception:
            for comp in reversed(done):
                comp()
            return "aborted"
        done.append(undo)
    return "completed"


def decline():
    raise RuntimeError("card declined")


log = []
print(run_saga([
    (lambda: log.append("held"), lambda: log.append("released")),
    (lambda: log.append("order"), lambda: log.append("cancelled")),
    (decline, lambda: log.append("refunded")),
]))                     # aborted
print(log)              # ['held', 'order', 'cancelled', 'released']
```

A saga gives up isolation: between steps, others can see the half-done state (a seat shown as held, an order pending). Design for it with explicit states (`PENDING`, `CONFIRMED`, `CANCELLED`), make every step and compensation idempotent and retryable, and put irreversible steps (sending the ticket e-mail) last. Run the saga in a durable workflow engine such as AWS Step Functions (GCP Workflows, Azure Durable Functions) or Temporal, so a crash mid-saga resumes instead of leaving seats stranded. Orchestration versus choreography is covered in Chapter 14.

> **Design question.** A fan's checkout touches seat inventory, orders and the external payment provider. Two-phase commit or a saga?
>
> 2PC is impossible (the provider cannot prepare) and would block: seat rows locked for the seconds a card authorization takes, at 30,000 hold attempts per second. Use a saga: hold seats (a 10-minute lease, one conditional write) -> create a `PENDING` order -> charge with an idempotency key -> confirm seats with the hold's version ->

> `CONFIRMED`, then e-mail via the outbox. Compensations: refund the charge, cancel the order, release the hold. The 10-minute hold bounds how long the half-done state lasts. The trade-off is visible intermediate states and compensation code to test. See also *50 System Designs* #24 (Payment System).

18.12 Exactly-once is idempotency plus deduplication

Networks give you at-most-once (send once, maybe lost) or at-least-once (retry until acknowledged, maybe duplicated). Nobody can deliver a message exactly once across a crash, because a sender that hears nothing cannot tell "lost" from "done, ack lost". What systems call **exactly-once** is *exactly-once effect*: at-least-once delivery plus a receiver that recognizes duplicates.

exactly-once effect = at-least-once delivery + idempotent or deduplicated processing

The dedup record and the effect must commit in the same transaction, or a crash between them breaks the guarantee:

```
import sqlite3

db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE processed (message_id TEXT PRIMARY KEY)")
db.execute("CREATE TABLE email_job (order_id TEXT)")


def handle(message_id, order_id):
    with db:  # dedup record and effect commit in one transaction
        cur = db.execute("INSERT INTO processed VALUES (?) "
                         "ON CONFLICT DO NOTHING", (message_id,))
        if cur.rowcount == 1:
            db.execute("INSERT INTO email_job VALUES (?)", (order_id,))
        return cur.rowcount == 1


print([handle("m-1", "o-9") for _ in range(3)])   # [True, False, False]
```

The same SQL runs on Postgres. Where it runs: Kafka's idempotent producer (on by default since 3.0) plus transactions and `read_committed` consumers give exactly-once *within* Kafka; Amazon SQS FIFO queues drop duplicates sent with the same deduplication ID within 5 minutes (Azure Service Bus duplicate detection works alike); payment providers accept an `Idempotency-Key` header (Stripe keeps keys for at least 24 hours). An effect outside your database (an e-mail, a charge) needs the external system's own idempotency key, derived from your message id. Encore's numbers are small: about 137,000 orders a day (50 million a year), so a day of idempotency keys at 200 bytes each is about 27 MB.

The support agent is a distributed client like any other. An LLM agent that times out and retries `request_refund` must not refund twice, so the tool takes an idempotency key derived from the conversation and order (`refund:{order_id}:{conversation_id}`) and the refunds service enforces it with the same unique constraint; *Designing Agentic Systems* treats idempotent tool calls in depth.

18.13 In Encore: never oversell a seat

The invariant is one sentence: a seat is sold at most once, and a held seat has exactly one holder. Each idea in this chapter guards one part of it.

- **One authority per event (CP).** All seat rows of an event live in one partition in the event's home region: a Postgres primary whose commits are durable in more than one zone (an RDS Multi-AZ synchronous standby or Aurora's 4-of-6 storage quorum; Cloud SQL HA, Azure Database for PostgreSQL zone-redundant HA). A four-seat hold is then a single-partition transaction, with no 2PC. During a partition the other region serves the seat map but refuses holds.
- **A hold is a lease, taken by compare-and-set.** `UPDATE ... WHERE state = 'free' OR hold_until < now()` changes one row for the winner and zero for everyone else. Expiry uses the database's clock only, so application servers' skew never matters.
- **Confirmation is fenced.** The hold id plus the row's `version` is the fencing token; a payment that succeeds after its hold expired cannot sell a seat someone else now holds.

- **Payment is a saga with idempotency keys.** A late success becomes a refund (the compensation), and a retried charge is deduplicated by the provider.
- **Reads are AP.** The seat map comes from caches and replicas up to a second stale; only the hold result is authoritative.

```
CREATE TABLE seat (event_id INT, seat_id TEXT, state TEXT, hold_id TEXT,
  hold_until INT, version INT, PRIMARY KEY (event_id, seat_id));
-- Hold h1 expired at 1600 s; fan 2 took the seat over as hold h2.
INSERT INTO seat VALUES (7, 'A-12', 'held', 'h2', 2300, 2);

-- Fan 1's payment succeeds late; confirm is fenced by hold id + version.
UPDATE seat SET state = 'sold', version = version + 1
 WHERE event_id = 7 AND seat_id = 'A-12'
   AND state = 'held' AND hold_id = 'h1' AND version = 1;
SELECT changes();          -- 0: stale hold, so refund fan 1
```

Numbers (illustrative): 30,000 hold attempts per second for the first minute is 1.8 million attempts for 50,000 seats, 36 per seat on average and far more for the front rows. Nearly all lose, and losing costs one indexed update that changes zero rows. Redis can pre-filter seats already held to spare the database, but Redis never decides: the conditional write on the durable primary does. See *50 System Designs* #25 (Ticket Booking) and #34 (E-Commerce Checkout and Inventory); Chapter 25 runs the race test.

18.14 Numbers to remember

Quantity	Value
Nodes to tolerate f failures	2f + 1 (3 for 1, 5 for 2)
3 nodes at 99 percent each	99.97 percent, about 157 min a year down
5 nodes at 99 percent each	99.999 percent, about 5 min a year down
Raft commit	1 RTT to a majority + fsync
etcd defaults	100 ms heartbeat, 1,000 ms election
Clock drift budget	200 ppm: 6 ms per 30 s lease
NTP error	about 1 ms in a DC, tens of ms on the internet
TrueTime epsilon (2012 paper)	1 to 7 ms, about 4 ms; wait about 2 epsilon
CockroachDB max offset	500 ms default
SQS FIFO dedup window	5 minutes

18.15 Pitfalls

- Reading CAP as “pick two” -> the choice is C or A during a partition, per operation; PACELC adds the latency cost the rest of the time.
- Ordering events, or resolving last-writer-wins, by wall clocks across machines -> lost updates under skew; use one authority’s clock, HLCs, version vectors, conditional writes or CRDTs (Chapter 16).
- An even-sized or two-region quorum -> no extra fault tolerance; use 3 or 5 nodes in 3 zones or regions.
- A Redis lock guarding correctness -> fencing tokens checked by the resource, or a conditional write instead of a lock.
- Measuring a lease with `time.time()` -> use a monotonic clock and stop early by $L\rho$ plus a margin.
- 2PC across services or with an external provider -> a saga with compensations and idempotent steps in a durable workflow.
- Believing a queue’s “exactly-once” -> at-least-once plus a dedup record committed with the effect.

18.16 Remember

- Linearizable means “one copy, one instant per operation”; it costs a round trip to a quorum on every write.

- Logical clocks order events by causality; physical clocks only bound it. TrueTime and HLCs combine the two.
- Raft: terms, randomized elections, log matching, commit on a majority of the current term. A majority of 2f + 1 survives f failures.
- Leases give liveness; fencing tokens or compare-and-set give safety.
- Prefer one partition per invariant over distributed transactions; across services use sagas.
- Exactly-once is idempotency plus deduplication, end to end, including the external provider.

19 The algorithms inside real systems

Open the source code of Redis, Cassandra, Kafka or Elasticsearch and you find the same few dozen algorithms again and again: a hash ring to place keys, a Bloom filter to skip disk reads, a sketch that counts a billion visitors in 12 KB, a tree of hashes that finds the one row two replicas disagree on. This chapter is the catalogue: for each algorithm, the problem it solves, the intuition, the one equation that governs it, and the real systems and cloud services that run it. It ends with a single table, algorithm by algorithm, that maps each one to the systems that use it and the *50 System Designs* case studies where it appears.

19.1 The picture

A large library works the same way. A **shelf map** at the door tells you which room holds which call numbers (hashing keys to machines). A **"definitely not here" stamp** on each room saves you from walking in to search for a book the room never had (a Bloom filter). The librarian estimates "about a million visitors this year" from a tally sheet the size of a postcard (HyperLogLog). Two branch libraries compare one checksum per shelf instead of every book to find the shelf that differs (a Merkle tree). The staff pass gossip at lunch about who is off sick (failure detection). The card catalogue is kept sorted so any title is a few steps away (B-trees and skip lists), and the subject index lists, for every word, the books that contain it (an inverted index).

Almost every algorithm here trades one of three things for speed or memory: **exactness** (sketches and filters answer approximately), **coordination** (gossip and CRDTs avoid a leader), or **reorganisation work later** (LSM compaction, timing wheels):

```
question a system asks            algorithm family         gives up
--------------------------------  -----------------------  -----------------
which machine owns this key?      consistent/HRW/Maglev    perfect balance
have I seen this item?            Bloom / cuckoo filter    false positives
how many distinct items?          HyperLogLog              ~1% error
how often / which are the most?   Count-Min, top-k heap    overcounting
which replicas differ?            Merkle tree              tree upkeep
who is alive?                     gossip, SWIM, phi        detection delay
keep it sorted and searchable     skip list, B-tree, LSM   write or read amp
find words, places, meanings      inverted index, S2, HNSW recall, precision
agree or merge without conflicts  Raft, vector clocks, CRDT latency, metadata
shape rate and time               token bucket, wheels     timer precision
name things uniquely              Snowflake IDs            clock dependence
```

19.2 Placing keys on machines: consistent, rendezvous, jump and Maglev hashing

Problem. Map millions of keys onto N machines so that each machine gets a fair share and adding or removing a machine moves as few keys as possible. The naive `hash(key) % N` fails the second test: going from 10 to 11 cache nodes remaps $N/(N+1)$, about 91 percent of keys, and the cold cache stampedes the database.

Consistent hashing places nodes and keys on the same circle of hash values; a key belongs to the first node clockwise. Adding a node takes over one arc, so only about $1/(N+1)$ of keys move. With one point per node the arcs are uneven, so each physical node gets v **virtual nodes**:

```
          A1 .---- key "event:9001" -> walks clockwise -> B2
        /      \
      C2        B2        A, B, C = physical nodes
      |   ring   |        A1, B2, ... = their virtual nodes
      B1        A2        a new node D takes over only the arcs
        \      /         that end at its own virtual nodes
          C1 '
```

$$\text{keys moved} \approx \frac{1}{N+1}, \qquad \frac{\sigma_{\text{load}}}{\text{mean load}} \approx \frac{1}{\sqrt{v}}$$

In a run of a small ring like the one in Chapter 17, with 300,000 event keys on 10 nodes, the busiest node held 3.15 times the mean load with $v = 1$, 1.38 times with $v = 16$ and 1.13 times with $v = 256$, while 8 to 9 percent of keys moved when an eleventh node joined (hash-mod-N moved 91 percent). It runs in Cassandra's token ring (Cassandra's shipped `num_tokens` is 16), Riak, Envoy's `RING_HASH` load-balancing policy, and Memcached clients (ketama). Amazon DynamoDB and Azure Cosmos DB also partition by a hash of the partition key (GCP's Bigtable splits by key range instead), though their internal maps from hash ranges to partitions are not client-visible rings. Redis Cluster deliberately uses a fixed table instead: 16,384 slots chosen by CRC16 of the key, moved between nodes as whole slots.

Rendezvous (highest-random-weight) hashing needs no ring: score every node against the key and take the highest. Removing a node moves only that node's keys; weights fit naturally:

$$\text{owner}(k) = \mathit{argmax}_n h(k, n), \qquad \text{weighted score} = -\frac{w_n}{\ln h(k, n)},\ h \quad (0, 1)$$

Lookup costs $O(N)$ hash computations, fine for tens of nodes. It runs in Apache Ignite's affinity function, Ceph's CRUSH `straw2` buckets (a weighted variant) and the old CARP cache-array protocol; it is also the easy way to pick the top three replicas for a key.

Jump consistent hash (Lamping and Veach, Google, 2014) computes a bucket number $0 \ldots N-1$ in $O(\ln N)$ time with no memory at all, and when a bucket is added it moves the minimum possible, $1/(N+1)$ of keys on average. The catch: buckets are numbered, so you can add or remove only at the end; it suits shards of a storage system, not a pool of servers that fail at random. Guava's `Hashing.consistentHash` implements it.

Maglev hashing (Google, 2016) is built for load balancers that must look up a backend for every packet. Each backend derives a permutation of a prime-sized table from two hashes of its name, offset and skip; backends take turns claiming their next preferred empty slot until the table is full. A packet's 5-tuple hash indexes the table in $O(1)$:

$$\text{perm}_i[j] = (\text{offset}_i + j \quad \text{skip}_i) \bmod M, \qquad M \text{ prime},\ M \quad N$$

Every backend owns almost exactly M/N slots, and when one leaves, most slots keep their owner, so open connections stay on the same backend. Google's passthrough Network Load Balancer is built on Maglev, and Envoy offers a `MAGLEV` policy with a fixed table of 65,537 entries.

Trade-offs. Use a ring with virtual nodes when membership changes and clients route; rendezvous for small node sets or replica selection; jump hash for numbered shards; Maglev for per-packet speed; fixed slots (Redis) or fixed partitions (Kafka) when you want rebalancing to be an explicit, operator-driven move.

19.3 Have I seen this before? Bloom and cuckoo filters

Problem. Before an expensive lookup (a disk read, a network call), answer "is this item possibly in the set?" with a few bytes per item. A **Bloom filter** is a bit array of m bits and k hash functions. Adding an item sets its k bits; a query checks them. If any bit is 0 the item was **definitely never added**; if all are 1 it is **probably** there, because other items may have set those bits. There are no false negatives, and you cannot delete.

```
add "ana"   -> bits 3, 9, 14        0 0 0 1 0 0 0 0 0 1 0 0 0 0 1 0
query "bo"  -> bits 3, 6, 14        bit 6 is 0 -> definitely absent
query "cy"  -> bits 3, 9, 14        all 1 -> "maybe" (false positive)
```

With n items the false-positive rate, the best k and the bits needed for a target rate p are:

$$p \approx (1 - e^{-kn/m})^k, \qquad k^* = \frac{m}{n}\ln 2, \qquad \frac{m}{n} = \frac{-\ln p}{(\ln 2)^2} \approx 1.44 \log_2 \frac{1}{p}$$

So 1 percent costs 9.6 bits and 7 hashes per item, and each further factor of 10 adds 4.8 bits. Encore's sign-up form asks "is this e-mail already registered?" for 20 million users: a 1 percent filter needs about 24 MB, small enough to live in every API process, and only the 1 percent "maybe" answers go to the database. The implementation below uses the standard double-hashing trick (two hashes generate all k positions):

```python
import hashlib
import math


class BloomFilter:
    def __init__(self, n: int, p: float):
        self.m = math.ceil(-n * math.log(p) / math.log(2) ** 2)  # bits
        self.k = round(self.m / n * math.log(2))                 # hashes
        self.bits = bytearray((self.m + 7) // 8)

    def _positions(self, item: str):
        d = hashlib.blake2b(item.encode(), digest_size=16).digest()
        h1, h2 = int.from_bytes(d[:8]), int.from_bytes(d[8:])
        return ((h1 + i * h2) % self.m for i in range(self.k))  # double hash

    def add(self, item: str) -> None:
        for p in self._positions(item):
            self.bits[p // 8] |= 1 << (p % 8)

    def might_contain(self, item: str) -> bool:
        return all(self.bits[p // 8] >> (p % 8) & 1
                   for p in self._positions(item))


bf = BloomFilter(n=200_000, p=0.01)
for i in range(200_000):
    bf.add(f"fan{i}@example.com")
assert all(bf.might_contain(f"fan{i}@example.com") for i in range(200_000))
fp = sum(bf.might_contain(f"new{i}@example.com") for i in range(200_000))
print(f"m={bf.m} bits ({bf.m / 8 / 1024:.0f} KiB), k={bf.k}, "
      f"false positives={fp / 200_000:.2%}")
```

It prints `m=1917012 bits (234 KiB), k=7, false positives=0.96%`. In Redis Open Source 8 the same structure is built in (in earlier versions it came from the RedisBloom module):

```
BF.RESERVE emails 0.01 20000000
BF.ADD emails ana@example.com          -> (integer) 1
BF.EXISTS emails ana@example.com       -> (integer) 1
BF.EXISTS emails bo@example.com        -> (integer) 0
```

Where it runs. LSM stores keep a Bloom filter per SSTable so a read skips files that cannot hold the key: Cassandra (`bloom_filter_fp_chance` per table), HBase (row filters on by default), RocksDB and LevelDB (configurable bits per key; RocksDB also offers the more compact Ribbon filter). Parquet files can carry Bloom filters per column chunk, Spark builds runtime Bloom filters to prune joins, Akamai used one to cache objects only on their second request, and web crawlers keep seen-URL sets this way (*50 System Designs* #17, Web Crawler).

Cuckoo filters (Fan et al., 2014) store a short **fingerprint** of each item in one of two candidate buckets, $i_1 = h(x)$ or $i_2 = i_1 \quad h(\text{fp}(x))$; on a collision the resident fingerprint is kicked to its other bucket, as a cuckoo chick evicts eggs. Because the fingerprint is stored, items can be **deleted**, and lookups touch at most two buckets. With buckets of 4 entries filled to $\alpha \approx 95$ percent:

$$\text{bits per item} \approx \frac{\log_2(1/\varepsilon) + 3}{\alpha}$$

That is 10.1 bits at 1 percent against Bloom's 9.6, but 13.6 against 14.4 at 0.1 percent; the basic form wins below about 0.4 percent, and the paper's semi-sorting variant below about 3 percent. Redis has `CF.ADD`, `CF.DEL` and `CF.EXISTS`. Choose cuckoo when items leave the set (revoked tickets, expired sessions); choose Bloom for append-only sets and when inserts must never fail (a cuckoo filter can fill up).

19.4 How many distinct? HyperLogLog

Problem. Count distinct users, IPs or search terms in a stream of billions without remembering them. Picture flipping coins: if someone tells you their longest run of heads was 20, you guess they flipped about 2^{20} times. **HyperLogLog** hashes each item, uses the first p bits to pick one of $m = 2^p$ registers, and stores in that register the longest run of leading zeros (plus one) seen in the rest of the hash. One register is noisy; the harmonic mean of m registers is not:

$$E = \alpha_m \, m^2 / \sum_{j=1}^{m} 2^{-M[j]}, \qquad \alpha_m \approx \frac{0.7213}{1 + 1.079/m}, \qquad \sigma \approx \frac{1.04}{\sqrt{m}}$$

Redis uses $m = 16,384$ registers of 6 bits: 12 KB at most, a standard error of 0.81 percent, for up to 2^{64} items. Two sketches **merge** by taking the register-wise maximum, so counts from shards, hours or regions combine without double-counting users seen in both. The sketch below merges a European and a US sketch that share 200,000 fans:

```
import hashlib
import math


class HyperLogLog:
    def __init__(self, p: int = 14):          # m = 2**14, as in Redis
        self.p, self.m = p, 1 << p
        self.reg = bytearray(self.m)

    def add(self, item: str) -> None:
        x = int.from_bytes(hashlib.blake2b(item.encode(),
                                           digest_size=8).digest())
        j = x >> (64 - self.p)                # first p bits: register
        rest = x & ((1 << (64 - self.p)) - 1)  # remaining bits
        rank = (64 - self.p) - rest.bit_length() + 1  # leading zeros + 1
        self.reg[j] = max(self.reg[j], rank)

    def count(self) -> int:
        alpha = 0.7213 / (1 + 1.079 / self.m)
        e = alpha * self.m ** 2 / sum(2.0 ** -r for r in self.reg)
        zeros = self.reg.count(0)
        if e <= 2.5 * self.m and zeros:        # small range: linear count
            e = self.m * math.log(self.m / zeros)
        return round(e)

    def merge(self, other: "HyperLogLog") -> None:
        self.reg = bytearray(map(max, self.reg, other.reg))


eu, us = HyperLogLog(), HyperLogLog()
for i in range(600_000):
    eu.add(f"fan:{i}")
for i in range(400_000, 1_000_000):            # 200,000 fans in both
    us.add(f"fan:{i}")
eu.merge(us)
est = eu.count()
print(f"estimate={est:,}  true=1,000,000  error={est / 1e6 - 1:+.2%}")
```

It prints an estimate of 992,453 for 1,000,000 true distinct fans, an error of −0.75 percent (this code stores a register per byte; Redis packs them into 6 bits). Below about 2.5m items the code switches to linear counting, as the original algorithm does; HyperLogLog++ (Google, 2013) improves that small range further and uses a sparse encoding.

Where it runs. Redis `PFADD`, `PFCOUNT` and `PFMERGE`; BigQuery's `HLL_COUNT` functions (HyperLogLog++) alongside `APPROX_COUNT_DISTINCT`; Amazon Redshift `APPROXIMATE COUNT(DISTINCT ...)`; Spark `approx_count_distinct`; Trino and Presto `approx_distinct` (2.3 percent default standard error); Apache Druid and Apache DataSketches; Postgres through the `hll` extension.

> **Design question.** Encore's organizers want "unique visitors to your event page" per hour and per day, across both regions, for 300,000 events. How do you build it?

One exact set per event and hour would hold up to 1,000,000 user IDs for the flash-sale show, tens of megabytes each in Redis, and two regions would have to share it to avoid counting a fan twice. Instead keep a HyperLogLog per event per hour in each region (`uv:<event>:<hour>`). Small events stay in Redis's sparse encoding at a few hundred bytes; a dense sketch never exceeds 12 KB. The daily figure is `PFMERGE` of 24 hourly sketches, and the global figure merges the two regions' sketches, because merge is a register-wise maximum. The standard error is 0.81 percent, about ±8,100 on a million fans, which no organizer will notice; ticket sales themselves stay exact in the orders database. See also *50 System Designs* #50 (Like and View Counter at Scale).

19.5 How often, and which are the most? Count-Min sketch, top-k heaps and reservoirs

Count-Min sketch. To estimate how often each item occurs without a counter per item, keep d rows of w counters, each row with its own hash. An arrival increments one counter per row; a query returns the **minimum** of its d counters. Collisions only add, so the answer never undercounts, and the smallest of several noisy counters is close to the truth. With N total arrivals:

$$\hat{f}(x) \le f(x) + \varepsilon N \quad \text{with probability } 1 - \delta, \qquad w = \lceil e/\varepsilon \rceil, \quad d = \lceil \ln(1/\delta) \rceil$$

For $\varepsilon = 0.1$ percent and $\delta = 1$ percent: $w = 2,719$, $d = 5$, 13,595 counters, 54 KB at 4 bytes each. In the flash sale's first minute ($N = 1,800,000$ hold attempts) any count is, with 99 percent probability, at most 1,800 too high. Sketches with the same dimensions and hashes merge by adding counters, so per-node, per-minute sketches roll up.

```
CMS.INITBYDIM holds:0955 2719 5
CMS.INCRBY holds:0955 evt:9001 3 evt:42 1        -> 1) 3  2) 1
CMS.QUERY holds:0955 evt:9001 evt:42 evt:7       -> 1) 3  2) 1  3) 0
```

Top-k with a heap. A sketch answers "how many?" for a key you name; to *list* the heaviest keys, keep a min-heap of the current top k beside it: after each update, if the item's estimate beats the heap's minimum, swap it in. That costs $O(\log k)$ per update and $O(n \log k)$ for a stream, against $O(n \log n)$ to sort. Redis `TOPK.ADD` and `TOPK.LIST` implement HeavyKeeper (a decaying counter table plus a min-heap); BigQuery has `APPROX_TOP_COUNT`; Lucene and Elasticsearch collect the best-scoring hits in a bounded priority queue, and Spark's `takeOrdered` keeps one per partition.

Reservoir sampling. To keep a uniform sample of k items from a stream of unknown length, keep the first k; then keep item i with probability k/i, replacing a random slot. Every item ends up in the sample with probability exactly k/n. Postgres `ANALYZE` builds column statistics from a reservoir of rows (300 times `default_statistics_target`, so 30,000 rows by default), and Dropwizard Metrics' uniform histogram uses one. Encore keeps a 1,000-event reservoir per hour of the clickstream for engineers to inspect, out of about 42 million events an hour.

Design question. Encore wants a "trending now" strip: the 10 events with the most hold attempts in the last 5 minutes, refreshed every few seconds. How do you compute it?

Hold attempts arrive at up to 30,000/s through the API gateway. Each gateway node keeps one Count-Min sketch and a 10-entry heap per minute; every 5 seconds it ships them to an aggregator, which adds the sketches for the last 5 minutes and merges the heaps (re-querying candidates against the summed sketch). The overcount is at most εN, about 9,000 on 9 million attempts in 5 minutes, irrelevant for a trending list. Be honest about scale: with only 300,000 events, an exact counter per event in Flink or Redis (`ZINCRBY` per minute bucket) also fits in a few megabytes, so choose the sketch when keys are seats, users or search terms by the million. See also *50 System Designs* #23 (Top-K Trending Topics) and #16 (Search Autocomplete).

19.6 Which replicas differ? Merkle trees

Problem. Two replicas each hold millions of rows; find the few that differ without shipping either copy. A **Merkle tree** hashes blocks of rows into leaves, then hashes pairs of hashes up to a single root. Equal roots mean equal data. If roots differ, compare the two children and descend only into the side that differs:

```
                root  (differs)
              /                \
        h(0-1) equal          h(2-3) differs
        skip whole half       /            \
                        h(2) equal        h(3) differs -> rows 192..255
```

With L leaves and d differing leaves, the replicas exchange about $2d\log_2 L$ hashes instead of every row. For one stadium event (80,000 seat rows of 100 bytes, 8 MB) in leaves of 64 rows there are 1,250 leaves and 11 levels below the root:

```
import hashlib


def H(b: bytes) -> bytes:
    return hashlib.sha256(b).digest()


def build(rows: list[str], leaf_size: int = 64) -> list[list[bytes]]:
    """Return tree levels, leaves first; each leaf hashes 64 seat rows."""
    level = [H("\n".join(rows[i:i + leaf_size]).encode())
             for i in range(0, len(rows), leaf_size)]
    levels = [level]
    while len(level) > 1:
        if len(level) % 2:
            level = level + [level[-1]]        # duplicate the odd one out
        level = [H(level[i] + level[i + 1]) for i in range(0, len(level), 2)]
        levels.append(level)
    return levels


def diff(a, b) -> tuple[list[int], int]:
    """Walk down from the root; return differing leaves and comparisons."""
    todo, found, compared = [(len(a) - 1, 0)], [], 0
    while todo:
        depth, i = todo.pop()
        if i >= len(a[depth]):
            continue
        compared += 1
        if a[depth][i] == b[depth][i]:
            continue                           # whole subtree agrees
        if depth == 0:
            found.append(i)
        else:
            todo += [(depth - 1, 2 * i), (depth - 1, 2 * i + 1)]
    return found, compared


seats = [f"seat:{n}:sold" for n in range(80_000)]
replica = seats.copy()
replica[41_234] = "seat:41234:held"     # one row diverged
leaves, comparisons = diff(build(seats), build(replica))
print(f"differing leaf={leaves}, rows {leaves[0] * 64}.."
      f"{leaves[0] * 64 + 63}, hash comparisons={comparisons}")
```

It prints `differing leaf=[644], rows 41216..41279, hash comparisons=23`: 23 hashes of 32 bytes located one changed row among 80,000. The same structure gives **inclusion proofs**: to prove an entry is in a log of n entries, send the $\log_2 n$ sibling hashes on its path to the root.

Where it runs. Dynamo-style anti-entropy: Cassandra's `nodetool repair` builds Merkle trees per token range and streams only mismatched ranges; Riak's active anti-entropy does the same. Git names every blob, tree and commit by the hash of its content, so a commit hash covers the whole history (a Merkle DAG) and `git fetch` transfers only missing objects. Certificate Transparency logs (RFC 9162) are Merkle trees that browsers audit with inclusion and consistency proofs. ZFS checksums blocks up a Merkle tree; Bitcoin and IPFS are built on them; cloud file sync compares chunk hashes the same way (*50 System Designs* #10, Cloud File Storage).

> **Design question.** Encore's seat inventory replicates asynchronously from the primary region to the standby region. Before a planned failover drill, how do you prove the standby's seat state matches for all 300,000 events?
>
> Copying everything would move up to 8 MB per stadium event, gigabytes in total, while sales continue. Instead each region maintains a Merkle tree per event over seat rows ordered by seat ID (leaves of 64 seats, rehashed on write or rebuilt by a nightly job). Exchange the 300,000 roots first, 9.6 MB of hashes; for each event whose root differs, walk down, about 23 hash comparisons per differing row, and replay only those rows from the primary's log. Rows still in flight because of replication lag will differ legitimately, so compare at a fixed log position (a snapshot), not "now". See also *50 System Designs* #3 (Distributed Key-Value Store).

19.7 Who is alive? Gossip and SWIM

Gossip spreads news the way a rumour crosses an office: every second each node tells a random peer what it knows (membership, heartbeats, schema versions), and the peer merges it. No node talks to all others, yet with fanout 1 the news reaches all N nodes in about $\log_2 N + \ln N$ rounds; a simulation of 200 nodes averaged 14 rounds, about 14 seconds at one round per second. **SWIM** (2002) separates failure *detection* from *dissemination*: each period a member pings one random peer; if no ack arrives it asks k others to ping it indirectly (so one bad link does not condemn a node), then marks it *suspect*, and declares it dead only after a suspicion timeout; updates ride piggyback on the pings. Load per member stays constant as the cluster grows. The **phi accrual detector** replaces "dead or alive" with a suspicion level computed from the history of heartbeat gaps, $\varphi = -\log_{10} P(\text{gap} > t)$, and convicts above a threshold.

Where it runs. Cassandra gossips once a second with a random peer and uses a variant of phi accrual (`phi_convict_threshold`, default 8); Consul uses Serf, built on HashiCorp memberlist, which implements SWIM with the Lifeguard extensions; Redis Cluster nodes exchange PING and PONG messages with gossip sections on the cluster bus, and a primary is marked failed when a majority of primaries report it unreachable; Akka Cluster and ScyllaDB gossip too. *Trade-off:* gossip is eventually consistent and slow to converge (seconds), so it carries membership and hints, never the decision of who holds a lock; that needs consensus (Chapter 18).

19.8 Keeping data sorted: skip lists, B-trees and LSM compaction

Skip lists are a sorted linked list with express lanes: every node is on level 1, and each node is promoted to the next level with probability p. A search runs along the top lane and drops down when it would overshoot, so the expected height is $\log_{1/p} n$ and search, insert and delete cost $O(\log n)$, with no rebalancing and simple lock-free variants. Redis sorted sets use a skip list plus a hash table ($p = 1/4$, up to 32 levels; small sets stay in a compact listpack). Encore's hold expiry is one: `ZADD holds:exp <expiry-ms> <hold-id>` and a sweeper calling `ZRANGEBYSCORE holds:exp -inf <now>`; 50,000 flash-sale holds need about $\log_4 50,000 \approx 8$ levels. RocksDB and LevelDB memtables and Java's `ConcurrentSkipListMap` are skip lists too; leaderboards are the classic case (*50 System Designs* #31, Leaderboard).

B-trees (in practice B+-trees) keep sorted keys in pages of a few kilobytes, each page holding hundreds of keys, so the tree is shallow: height $= \log_f N$ for fanout f. With an illustrative $f \approx 400$, Encore's 50 million orders a year need 3 levels and five years (250 million) need 4; the upper levels stay in memory, so a lookup costs about one disk read. Postgres's default index (8 KB pages), MySQL InnoDB's clustered primary key (16 KB pages), SQLite, MongoDB's WiredTiger and etcd's bbolt are B-trees. **LSM trees** instead buffer writes in a memtable, flush sorted immutable SSTables and merge them in the background; **compaction** is the algorithm that decides which files to merge. *Leveled* compaction (RocksDB's default, size ratio $T = 10$ between levels) keeps reads and space tight at a write amplification of up to about T per level; *size-tiered* (Cassandra's default) writes each byte only about once per tier but can temporarily need twice the space and leaves more files per read, which the per-file Bloom filters soften. Chapter 12 derives the amplification equations; choose B-trees for read-heavy, update-in-place workloads (orders, seat state in Postgres) and LSM trees for write-heavy ingest (clickstream, time series, Cassandra, HBase and RocksDB-based stores).

19.9 Finding words: tries, inverted indexes and BM25

A **trie** stores strings by shared prefix, one edge per character, so finding every key that starts with "tay" costs *O*(prefix length) to reach the node plus the size of the answer. For autocomplete, each node also caches its top 5 completions, so Encore's search box answers "tay" with the five most-searched artists without scanning anything (*50 System Designs* #16, Search Autocomplete). Compressed forms are everywhere: radix trees in HTTP routers (Go's httprouter and Gin) and in Redis Streams (`rax`), an LC-trie for longest-prefix matching in the Linux IP routing table, and Lucene's term dictionary as a finite-state transducer that shares prefixes and suffixes.

An **inverted index** is the index at the back of a book: for every term, a sorted **posting list** of the documents (and positions) containing it. A query intersects or unions posting lists, using skip pointers to leap ahead; ranking then scores each match. **BM25**, the default similarity in Lucene, Elasticsearch, OpenSearch and Solr, rewards rare terms, saturates repeated terms and penalises long documents:

$$\text{score}(D,Q) = \sum_{q \in Q} \text{IDF}(q)\,\frac{f(q,D)\,(k_1+1)}{f(q,D) + k_1\,(1 - b + b\,\frac{|D|}{\text{avgdl}})}, \qquad \text{IDF}(q) = \ln\!\left(1 + \frac{N - n_q + 0.5}{n_q + 0.5}\right)$$

Defaults are $k_1 = 1.2$ and $b = 0.75$. Worked example over Encore's 300,000 event documents (average 60 terms): "taylor" appears in 600, so IDF = 6.21. A 40-term event page that mentions it once scores 7.19, twice 9.43, ten times only 12.54 (the ceiling is IDF($k_1 + 1$) = 13.67), and a 200-term page with two mentions scores 5.16. Postgres full-text search keeps the same inverted structure in a GIN index but ranks with `ts_rank`, not BM25. Search engines shard the index by document and merge per-shard top-k lists (*50 System Designs* #22, Web Search Engine).

19.10 Finding places: geohash, quadtrees, S2 and H3

"Venues near me" needs a one-dimensional key for two-dimensional points, so an ordinary sorted index can answer it. **Geohash** halves the longitude range, then the latitude range, alternately, writing a bit per halving, and spells the interleaved bits in base 32 (5 bits per character). Nearby points share prefixes: 6 characters is a cell of about 1.2 km by 0.6 km, 7 characters about 153 m by 153 m. Two points across a cell edge share no prefix, so a radius query scans the cell and its 8 neighbours. Redis GEO stores a 52-bit geohash as a sorted-set score:

```
GEOADD venues -0.2795 51.5560 wembley -0.1145 51.4651 brixton
GEOSEARCH venues FROMLONLAT -0.1276 51.5072 BYRADIUS 10 km ASC WITHDIST
  -> 1) 1) "brixton"  2) "4.7700"
GEOHASH venues wembley                          -> 1) "gcpv39krrn0"
```

A **quadtree** splits a square into four whenever it holds more than a set number of points, so dense central London gets small cells and the North Sea one big one; it adapts to density but lives in memory and must be rebuilt as points move (*50 System Designs* #27, Proximity Search). Google's **S2** projects the sphere onto a cube, orders each face's cells along a Hilbert curve and names them with 64-bit IDs across 31 levels, so any region becomes a few ID ranges in a B-tree; MongoDB `2dsphere` indexes, CockroachDB spatial indexes and BigQuery geography functions (`S2_CELLIDFROMPOINT`) use it. Uber's **H3** tiles the globe with hexagons at 16 resolutions, $2 + 120 \quad 7^r$ cells at resolution r (resolution 9 averages 0.105 square kilometres); every neighbour of a hexagon is equidistant, which suits surge-pricing zones and heat maps (*50 System Designs* #13, Ride Hailing). Databricks, Snowflake, ClickHouse and Postgres (`h3-pg`) ship H3 functions. PostGIS indexes geometries with R-trees over GiST, and Lucene with BKD trees.

19.11 Finding similar meaning: HNSW

Embeddings turn text into vectors whose distance tracks meaning; the problem is finding the nearest of millions of vectors without comparing against all of them. **HNSW** (hierarchical navigable small world, Malkov and Yashunin, 2016) builds a stack of proximity graphs, like an airline network: the top layer has few nodes and long hops (hubs), lower layers add more nodes and shorter hops, and layer 0 holds every vector. A query greedily walks toward the target on the top layer, drops a layer, repeats, and finishes with a beam search that keeps the best `ef_search` candidates on layer 0. Each insert gets a random top layer (exponentially rarer higher up) and links to its M nearest neighbours per layer ($2M$ on layer 0). A search visits roughly $O(\log N)$ nodes; memory is about $4D + 8M$ bytes per float32 vector of

dimension D plus upper layers. Encore's support agent searches 100,000 help and policy chunks of 1,024 dimensions (illustrative): 410 MB of vectors and about 13 MB of layer-0 links, a single-node index. Raising `ef_search` buys recall with latency; deletes and heavy filtering degrade the graph.

Where it runs. pgvector (`hnsw` beside `ivfflat`; defaults `m = 16`, `ef_construction = 64`, `hnsw.ef_search = 40`) on Amazon RDS and Aurora PostgreSQL (Cloud SQL and AlloyDB, Azure Database for PostgreSQL); Lucene's HNSW inside Elasticsearch and Amazon OpenSearch Service; Redis (`HNSW`, `FLAT` and, since 8.2, `SVS-VAMANA`); Qdrant, Weaviate, Milvus and FAISS. Alternatives: IVF clustering (faster builds, less memory), Google's ScaNN (Vertex AI Vector Search, AlloyDB) and DiskANN-style graphs on SSD (Azure Cosmos DB). For agents, the same index backs retrieval for tools and a **semantic cache** of model answers keyed by query embedding, which must be scoped to non-personal content and a strict similarity threshold; *Designing Agentic Systems* covers retrieval quality in depth.

19.12 Agreeing and merging: Raft, vector clocks and CRDTs

Raft makes a group of $n = 2f + 1$ servers act as one: an elected leader appends each command to its log, replicates it, and commits it once a majority $\lfloor n/2 \rfloor + 1$ has stored it, so 5 servers survive 2 failures; terms and randomized election timeouts prevent two leaders from both committing (Chapter 18 walks through it step by step). It runs in etcd (and so every Kubernetes control plane), Consul servers, CockroachDB and TiKV (one Raft group per range), Kafka's KRaft controller quorum and Redpanda; MongoDB replica sets use a Raft-inspired protocol, ZooKeeper uses Zab and Spanner uses Paxos. Encore uses consensus indirectly, through etcd and through the database that guarantees one writer per seat.

Vector clocks detect concurrency instead of preventing it. Each replica keeps a counter per writer; a write increments its own entry. Version a happened before b if $a_i \le b_i$ for every i (and they differ); if neither dominates, the writes were **concurrent** and both are kept as siblings for the application to merge. Dynamo used them and Riak uses dotted version vectors; the cost is metadata that grows with the number of writers. Cassandra deliberately chose per-cell timestamps and last-writer-wins instead.

CRDTs (conflict-free replicated data types) remove the conflict by design: the merge is commutative, associative and idempotent, so replicas that receive the same updates in any order, any number of times, converge. A **G-counter** keeps one count per replica, merges by element-wise maximum and reads the sum; a PN-counter pairs two of them; an OR-set tags each add so a concurrent add beats a remove. Encore's "fans interested" counter per event can be written in both regions at once: EU holds {eu: 120, us: 75}, US holds {eu: 110, us: 80}, the merge is {eu: 120, us: 80} and both read 200. Riak data types, Redis Active-Active databases (Redis Software and Redis Cloud), and Automerge and Yjs for collaborative editing (*50 System Designs* #11, Collaborative Editor) are CRDTs. They cannot enforce invariants such as "at most 50,000 seats sold"; seats need a single writer or consensus.

19.13 Shaping time: token buckets, leaky buckets and timing wheels

A **token bucket** holds up to b tokens and refills at r per second; a request spends a token or is rejected (HTTP 429). It allows bursts up to b while capping the long-run rate: tokens $= \min(b,$ tokens $+ r\,\Delta t)$, and in any window T at most $b + rT$ requests pass. Encore limits hold attempts per fan to $r = 1$/s with $b = 5$, so at most 65 in a minute, whatever a bot tries. Amazon API Gateway throttles with token buckets (rate and burst per stage, method and usage plan), as do Envoy's local rate limit filter and Linux `tc` with `tbf`. A **leaky bucket** queues requests and drains them at a constant rate, smoothing bursts instead of allowing them; Nginx `limit_req` uses it. LLM APIs apply the same bucket with tokens of text as the currency (requests and tokens per minute), so a model gateway debits the estimated prompt plus output size (Chapter 11 has the runnable Redis versions).

A **timing wheel** manages millions of timers in $O(1)$ each. Picture a clock face with W slots, each a list of timers; a pointer advances one slot per tick, and a timer due in Δ goes into slot $(\text{now} + \Delta)/\text{tick} \bmod W$. Insert and cancel are constant time, against $O(\log n)$ for a heap; delays longer than one revolution go into a coarser wheel, like the hour hand (hierarchical wheels, Varghese and Lauck, 1987). Kafka's request purgatory tracks delayed produce and fetch requests on hierarchical timing wheels, Netty's `HashedWheelTimer` drives connection timeouts, and the Linux kernel's timer wheel

handles network timeouts. For Encore's seat holds, a 600-slot wheel at one-second ticks expires each 10-minute hold in the second it lapses; the wheel is only an in-memory accelerator, and the hold's expiry still lives in Redis and the database so a restart loses nothing.

19.14 Naming things: Snowflake IDs

Orders, holds and tickets need IDs that are unique across many servers, issued without a round trip to a central counter, and roughly sorted by time so new rows land at the right edge of a B-tree. Twitter's **Snowflake** (2010) packs a 64-bit integer:

```
0 | 41 bits: ms since a custom epoch | 10 bits: worker | 12 bits: sequence
```

41 bits of milliseconds last 2^{41} ms ≈ 69.7 years; 10 bits name 1,024 workers; 12 bits allow 4,096 IDs per millisecond per worker, 4.1 million per second, far above Encore's 30,000 holds per second. The one real hazard is the clock: a generator must refuse to issue IDs (or wait) if NTP steps time backwards, and when 4,096 IDs are used within one millisecond it waits for the next. The ID is `(ms - epoch) << 22 | worker << 12 | sequence`.

Discord and Instagram use Snowflake-style layouts (Instagram generates them inside Postgres per logical shard); Sonyflake trades sequence bits for a longer lifetime. **UUIDv7** (RFC 9562, 2024) puts a 48-bit millisecond timestamp in front of random bits, giving time order with no worker registry, at 128 bits. Random UUIDv4 keys scatter inserts across the whole index. Snowflake IDs leak creation time and volume, so never expose them where order counts are sensitive (*50 System Designs* #21, Unique ID Generator, and #1, URL Shortener).

19.15 The catalogue: which systems run which algorithms

One row per algorithm: the problem it solves, the systems that run it, and the *50 System Designs* case studies where it carries the design.

Algorithm	Problem it solves	Runs in	*50 System Designs*
Consistent hashing, vnodes	Keys to nodes, few moves	Cassandra, Riak, Envoy ring hash, ketama	#3, #4
Hash slots	Operator-driven moves	Redis Cluster (16,384, CRC16)	#4
Rendezvous (HRW)	Owner or top-r replicas	Apache Ignite, Ceph CRUSH straw2	#4
Jump hash	Key to numbered shard	Guava `consistentHash`	#3
Maglev hashing	Backend per packet	Google Cloud passthrough NLB, Envoy	#40
Power of two choices	Balance with little state	Envoy least request, Nginx	#40
Bloom filter	Skip absent keys	Cassandra, HBase, RocksDB, Parquet, Redis	#3, #17
Cuckoo filter	Membership with deletes	Redis `CF.*`	-
HyperLogLog	Distinct counts	Redis, BigQuery, Redshift, Spark, Trino	#28, #50
Count-Min sketch	Frequencies in a stream	Redis `CMS.*`, stream jobs	#23, #28
Top-k heap, HeavyKeeper	Heaviest items	Redis `TOPK.*`, BigQuery, Lucene	#16, #23
Reservoir sampling	Uniform stream sample	Postgres `ANALYZE`, Dropwizard	#29, #48
Merkle tree	Find diffs, prove inclusion	Cassandra repair, Git, CT logs, ZFS	#3, #10
Gossip	Spread membership	Cassandra, Redis Cluster bus	#3

Algorithm	Problem it solves	Runs in	*50 System Designs*
SWIM, phi accrual	Detect failures	Consul (Serf), Cassandra	#3
Skip list	Sorted set in memory	Redis sorted sets, RocksDB memtable	#31
B+-tree	Sorted index on disk	Postgres, InnoDB, SQLite, WiredTiger	#24, #25
LSM tree, compaction	Write-heavy storage	RocksDB, Cassandra, HBase	#3, #29
Trie, radix tree, FST	Prefix lookup	Lucene, Redis Streams, Linux routing	#16
Inverted index, BM25	Ranked full-text search	Lucene, Elasticsearch, OpenSearch, Solr	#22, #42
Geohash	2-D point to sortable key	Redis GEO, Elasticsearch	#27
Quadtree	Density-adaptive cells	In-memory spatial indexes	#13, #27
S2 cells	Sphere cells, Hilbert order	MongoDB, CockroachDB, BigQuery	#13, #27
H3 hexagons	Equal-neighbour cells	Uber, Databricks, Snowflake, ClickHouse	#13
HNSW	Nearest vectors	pgvector, OpenSearch, Redis, Qdrant	#47
Raft	One replicated log	etcd, Consul, CockroachDB, Kafka KRaft	#30, #39
Vector clocks	Detect concurrent writes	Dynamo, Riak	#3
CRDTs	Merge without coordination	Riak, Redis Active-Active, Yjs	#11
Token bucket	Rate limit with bursts	API Gateway, Envoy, Linux `tc`	#2
Leaky bucket	Smooth output rate	Nginx `limit_req`	#2
Timing wheel	Millions of timers	Kafka purgatory, Netty, Linux	#12, #30
Snowflake, UUIDv7	Unique time-ordered IDs	Twitter, Discord, Instagram	#1, #21

19.16 In Encore

Read by service, Encore runs most of the catalogue:

- **Load balancers and API gateway:** Maglev or power of two choices to spread connections; token buckets per fan, IP and API key.
- **Cache tier:** Redis Cluster hash slots for seat-map and event-page keys; consistent hashing if the tier moves to Memcached.
- **Catalog and search:** an inverted index with BM25, a trie for autocomplete, geohash or S2 for “venues near me”.
- **Seat inventory:** a sorted set (skip list) and a timing wheel for hold expiry, Postgres B-trees for seat state, Merkle trees to verify the standby region, consensus underneath for one writer per seat.
- **Orders and payments:** Snowflake IDs for orders and holds, which also serve as idempotency keys sent to the payment provider.
- **Analytics:** HyperLogLog for unique visitors, Count-Min plus a heap for trending events, reservoir samples for debugging, LSM-based storage for the clickstream.
- **Platform and support agent:** Raft inside etcd for Kubernetes, gossip inside Consul or Cassandra if used, HNSW for the agent’s retrieval and semantic cache.

19.17 Numbers to remember

Figure	Value
Keys moved adding a node: ring vs mod N	$1/(N+1)$ vs $N/(N+1)$
Ring load spread with v vnodes	about $1/\sqrt{v}$
Bloom filter at 1 percent; each 10x better	9.6 bits, 7 hashes; +4.8 bits
Redis HyperLogLog	12 KB, 0.81 percent error
Count-Min for $\varepsilon = 0.1\%$, $\delta = 1\%$	2,719 x 5 counters, 54 KB
Merkle search for one diff in 80,000 rows	23 hash comparisons
Gossip to 200 nodes, fanout 1	about 14 rounds
B-tree levels for 50 million rows ($f \approx 400$)	3
Snowflake: bits, lifetime, rate per worker	41 + 10 + 12, 69.7 years, 4,096/ms
Maglev table in Envoy	65,537 entries

19.18 Pitfalls

- Using a Bloom filter for exact deduplication (payment webhooks, ticket scans) -> use it only as a fast "definitely new" path and confirm "maybe" answers against a unique key in the database.
- `hash(key) % N` for cache placement -> a ring with virtual nodes, slots or rendezvous hashing, so scaling out does not empty the cache.
- Exact distinct counts with sets per page per hour -> HyperLogLog with `PFMERGE`; keep money and seat counts exact.
- Trusting a sketch's count as the truth -> report it as an estimate with its bound εN, and re-count winners exactly if they drive payouts.
- Gossip or CRDTs to decide who owns a seat -> consensus or a single writer per key; gossip carries membership, CRDTs carry mergeable counters.
- Snowflake IDs without clock discipline -> refuse to issue on a backward step, monitor NTP offset, assign worker IDs from a registry.
- An HNSW index with default `ef_search` and heavy filters -> measure recall on your own queries and raise `ef_search` or pre-filter by tenant.

19.19 Remember

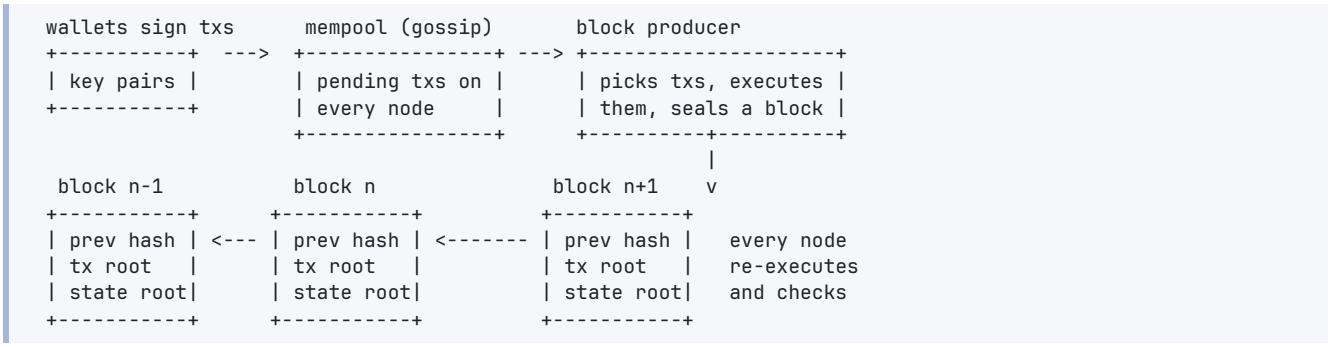
- Each algorithm buys speed or memory with one currency: exactness, coordination, or background work.
- Placement: ring with vnodes, rendezvous, jump hash, Maglev or fixed slots; all move about $1/(N+1)$ of keys, not all of them.
- Sketches answer at scale: Bloom for "seen?", HyperLogLog for "how many distinct?", Count-Min plus a heap for "how often and which?"; all merge across shards.
- Merkle trees find differences and prove inclusion in $O(\log n)$ hashes; gossip and SWIM spread membership; Raft, vector clocks and CRDTs agree, detect or merge.
- Indexes are algorithms too: skip lists and B-trees for order, LSM compaction for writes, inverted indexes, geocells and HNSW for words, places and meaning.
- In an interview, name the algorithm, give its equation and one real system that runs it; that is what turns a box on a diagram into a design.

20 Web3: blockchains as distributed systems

How do thousands of computers that do not know or trust each other keep one shared ledger, where nobody can rewrite history, nobody can be stopped from submitting a transaction and nobody runs the database? This chapter treats a blockchain as what it is underneath the vocabulary: a **replicated state machine** (Chapter 16) whose log is agreed by a consensus protocol that tolerates **Byzantine** nodes, ones that lie, not only ones that crash. It goes from blocks and Merkle proofs through proof of work, proof of stake and BFT, to fees, smart contracts, rollups, bridges, indexers and DeFi, with the numbers that bound each piece, and ends with an honest test: when is a blockchain the wrong tool, and should Encore put its tickets on one? Protocol figures were checked on ethereum.org, the EIPs and bitcoin.org on October 14, 2026; prices and costs are illustrative.

20.1 The picture

Picture a village ledger kept in a thousand houses. Anyone may shout a transaction ("Ana pays Ben 5"). Every few minutes one house is chosen, by a lottery that costs real money to enter, to write the next page; it seals the page with a wax stamp that includes the stamp of the previous page, and every other house checks the page and copies it. A forger who wants to change page 40 must redo the stamp of page 40 and of every page after it, faster than the whole honest village writes new ones. Nobody owns the ledger; the lottery and the chain of stamps replace the trusted clerk.



Compare it with Raft in Chapter 18: there, five known servers trust each other to follow the protocol and only crash. Here the members are unknown, may join or leave at any time, and some are actively trying to cheat. Everything that makes blockchains slow and expensive follows from that one change in the failure model.

Property	Raft cluster	Public blockchain
Members	Fixed, known, few (3 or 5)	Open, pseudonymous, thousands
Failure model	Crash, delay	Byzantine: lie, equivocate, censor
Who may write	Clients of one operator	Anyone who pays the fee
Sybil defence	Not needed	Cost: work or stake
Commit latency	About 1 ms in a region	12 s to 60 min
Throughput	10,000s of writes/s per group	7 to a few hundred tx/s on L1

20.2 What "Web3" means, and what is hype

"Web3" names an idea and a marketing wave at once. The idea: Web1 was **read** (static pages), Web2 is **read-write** (platforms host what users create, and own the data, the accounts and the rules), Web3 adds **own**: assets and identities are entries on a public ledger controlled by a user's private key, so no platform can freeze, copy or confiscate them, and programs (smart contracts) run on neutral infrastructure that anyone can call and nobody can quietly change. Gavin Wood, a co-founder of Ethereum, coined "Web 3.0" in this sense in 2014.

What has proved real: Bitcoin as a censorship-resistant bearer asset; stablecoins as dollars that settle around the clock between any two wallets; public, programmable settlement (DeFi exchanges and lending markets that hold billions with no operator); and verifiable, append-only public records. What was hype: “everything on the blockchain” (most data and computation are cheaper, faster and more private in an ordinary database), NFTs as proof of owning the image itself (the token usually points at a URL), and “decentralised” products that in practice run behind one company’s front end, one RPC provider and one admin key. Read every Web3 design by asking which parts are actually on chain, and who can still change or stop the rest.

20.3 Blocks, transactions and the mempool

A **transaction** is a message signed with the sender’s private key: on Ethereum it carries the sender’s next **nonce** (a per-account counter that orders the sender’s transactions and stops replays), the recipient, a value, optional call data for a contract, and fee limits. Nodes check the signature and the balance, then **gossip** valid transactions to their peers; each node keeps the ones not yet in a block in its **mempool**. There is no global order in the mempool: two nodes may hold different sets, and the block producer chooses which transactions to include and in what order, usually by fee. That freedom is the root of front-running (see MEV below).

A **block** is a header plus an ordered list of transactions. The header holds the hash of the previous header (which chains the blocks: changing any old block changes every later hash), the **Merkle root** of the transactions, and on Ethereum a **state root** that commits to every account balance and contract storage slot after the block runs. Merkle trees are taught in Chapter 19; what matters here is the **inclusion proof**: to show a transaction is in a block of n transactions you send the $\log_2 n$ sibling hashes on its path, and the verifier needs only the 32-byte root from a header.

$$\text{proof size} = 32 \ \log_2 n \ \text{bytes}, \qquad n = 4,000 \ \Rightarrow \ 12 \ \text{hashes} = 384 \ \text{bytes}$$

That is how a phone wallet (a **light client**) checks a payment without downloading blocks: Bitcoin headers are 80 bytes, one every 10 minutes, about 4.2 MB a year. This toy chain mines a block by proof of work (next section), proves one transaction among 1,000 with 10 hashes, and shows that tampering breaks the root in the header:

```
import hashlib
import json


def sha(b: bytes) -> bytes:
    return hashlib.sha256(hashlib.sha256(b).digest()).digest()  # Bitcoin


def up(level: list[bytes]) -> list[bytes]:
    """One Merkle level up; an odd last node is paired with itself."""
    if len(level) % 2:
        level = level + [level[-1]]
    return [sha(level[i] + level[i + 1]) for i in range(0, len(level), 2)]


def merkle(leaves: list[bytes], i: int) -> tuple[bytes, list]:
    """Return the root and the proof (sibling, is_left) for leaf i."""
    level, proof = leaves, []
    while len(level) > 1:
        padded = level + [level[-1]] if len(level) % 2 else level
        proof.append((padded[i ^ 1], i % 2 == 1))
        level, i = up(level), i // 2
    return level[0], proof


def verify(leaf: bytes, proof: list, root: bytes) -> bool:
    for sib, left in proof:
        leaf = sha(sib + leaf) if left else sha(leaf + sib)
    return leaf == root


def mine(prev: str, root: bytes, bits: int) -> dict:
    """Try nonces until the header hash is below 2 ** (256 - bits)."""
    for nonce in range(10**8):
        h = sha(json.dumps([prev, root.hex(), nonce]).encode()).hex()
        if int(h, 16) < 2 ** (256 - bits):
            return {"prev": prev, "root": root, "nonce": nonce, "hash": h}


txs = [f"transfer ticket {n} to fan {n % 7}" for n in range(1000)]
leaves = [sha(t.encode()) for t in txs]
root, proof = merkle(leaves, 417)
block = mine("0" * 64, root, bits=16)
print(block["hash"][:16], "nonce", block["nonce"])
print(len(proof), "hashes prove tx 417:", verify(leaves[417], proof, root))
leaves[417] = sha(b"transfer ticket 417 to scalper")           # tamper
print("tampered root matches header:", merkle(leaves, 0)[0] == root)
```

It prints a block hash starting with four zero hex digits (16 bits) at nonce 144,752 (the expected number of tries is $2^{16} = 65,536$; luck varies), `10 hashes prove tx 417: True`, and `tampered root matches header: False`. Real chains differ in detail (Bitcoin hashes an 80-byte binary header with double SHA-256; Ethereum uses Keccak-256 and a Merkle Patricia trie for state), not in principle.

20.4 Proof of work and probabilistic finality

Proof of work turns "who writes the next page" into a lottery whose tickets are hash attempts. A miner varies a nonce in the header until the header's hash, read as a number, falls below a **target**; the only strategy is to try, so the chance of winning is proportional to the share of the network's hashing power you own. That is the Sybil defence: a million fake identities win nothing without a million times the hardware and electricity. Every 2,016 blocks, Bitcoin compares the time they took with two weeks (1,209,600 seconds) and rescales the target, which holds the average block interval at **10 minutes** however much hardware joins. The subsidy halves every 210,000 blocks (about four years); since April 2024 it is 3.125 BTC per block, plus fees.

Blocks arrive as a Poisson process, so the gap between blocks is exponential with a mean of 10 minutes. Two miners sometimes find blocks seconds apart: the chain **forks**, nodes follow the branch with the most cumulative work, and the losing block's transactions return to the mempool. A transaction is therefore never final, only increasingly expensive to undo. Nakamoto's white paper (2008, section 11) computes the chance that an attacker with a share q of the hash power ($p = 1 - q$ honest) ever overtakes the honest chain after the merchant waited for z confirmations:

$$P(q, z) = 1 - \sum_{k=0}^{z} \frac{\lambda^k e^{-\lambda}}{k!}\left(1 - \left(\frac{q}{p}\right)^{z-k}\right), \qquad \lambda = z\,\frac{q}{p}$$

```
from math import exp, factorial


def attacker_success(q: float, z: int) -> float:
    """Nakamoto (2008), section 11: chance a share q ever catches up."""
    p, lam = 1 - q, z * q / (1 - q)
    return 1 - sum(lam**k * exp(-lam) / factorial(k)
                   * (1 - (q / p) ** (z - k)) for k in range(z + 1))


for q in (0.10, 0.20, 0.30, 0.40):
    z = next(z for z in range(1000) if attacker_success(q, z) < 0.001)
    print(f"q={q:.2f}: z=1 {attacker_success(q, 1):.4f}, "
          f"z=6 {attacker_success(q, 6):.6f}, P < 0.1% needs z={z}")
```

It reproduces the paper's table: with 10 percent of the hash power the attacker succeeds 20.5 percent of the time against one confirmation and 0.024 percent against six, while with 40 percent even six confirmations leave a coin flip (50.4 percent); to push the risk below 0.1 percent you wait 5 blocks against $q = 0.1$, 11 against 0.2, 24 against 0.3 and 89 against 0.4. At $q \geq 0.5$ the attacker always wins eventually: that is the **51 percent attack**, and selfish-mining strategies (Eyal and Sirer, 2014) earn more than a fair share with less. The convention of six confirmations (about an hour) is a choice of z for an assumed q, not a protocol rule; exchanges set their own depth per asset. The exponential gaps also mean waits are lumpy: a gap longer than 60 minutes has probability $e^{-6} = 0.25$ percent per block, so it happens about once every 400 blocks, roughly every 2.8 days.

Trade-offs. Proof of work is simple, permissionless and battle-tested since 2009, and its security is easy to state (rent or buy half the hash power). It pays for that with energy, with slow probabilistic finality, and with a security budget that depends on block rewards that keep halving.

20.5 Proof of stake: Ethereum's slots, epochs and finality

Proof of stake replaces burned electricity with locked capital. Since the Merge (September 2022) Ethereum is secured by **validators**, each backed by a deposit of at least 32 ETH; the Pectra upgrade (May 7, 2025) raised the maximum effective balance per validator to 2,048 ETH (EIP-7251), so large stakers can run fewer, bigger validators. The protocol is an ordinary distributed algorithm with a clock:

```
epoch N (32 slots x 12 s = 6.4 min)            epoch N+1
| s0 | s1 | s2 | ... | s31 | s0 | s1 | ... | s31 | s0 ...
  ^                          ^                    ^
  proposer per slot,         checkpoint N+1       checkpoint N+2:
  committees attest          justified (2/3)      N+1 finalized
```

- **Slots and proposers.** Time is cut into 12-second slots and 32-slot epochs. In each slot one validator, chosen pseudo-randomly (RANDAO) with odds proportional to its balance, proposes a block.
- **Attestations.** Every active validator attests once per epoch, in a randomly assigned committee for one slot: it votes for the head of the chain it sees (the fork choice, LMD-GHOST, follows the branch with the most attesting stake) and for a pair of epoch-boundary checkpoints.
- **Finality (Casper FFG).** When checkpoint votes from at least two-thirds of all staked ETH link two checkpoints, the newer one is **justified** and the older one **finalized**. In the normal case a block is finalized after about two epochs.
- **Slashing.** Signing two different blocks for one slot, or contradictory checkpoint votes, is provable from the signatures and destroys part of the stake, from under 0.1 percent for a lone offender to all of it when many

validators are slashed together. If the chain fails to finalize for more than four epochs an **inactivity leak** drains offline validators until the online ones again hold two-thirds.

$$t_{\text{final}} \approx 2 \times 32 \times 12\,\text{s} = 768\,\text{s} \approx 12.8\,\text{min}, \qquad \text{cost to revert finality} \geq \tfrac{1}{3} S_{\text{staked}}$$

That second term is **accountable safety**, the property proof of work lacks: two conflicting finalized blocks require at least one-third of the stake to have signed contradictory votes, and that stake is provably slashable. With an illustrative 30 million ETH staked, an attacker forfeits at least 10 million ETH. Applications choose their depth through the JSON-RPC block tags: `latest` (the head, may reorg), `safe` (unlikely to reorg, a strong majority attested) and `finalized`.

What is changing. Fusaka (December 3, 2025) added PeerDAS for blobs (below) and a 60 million gas default. Glamsterdam (enshrined proposer-builder separation, EIP-7732, and block-level access lists, EIP-7928) was testing on devnets with a Sepolia testnet fork scheduled for October 6, 2026; ethereum.org expects mainnet in Q4 2026 with no confirmed date. Finality in one slot is on the longer roadmap; until then design for about 13 minutes.

Trade-offs. Proof of stake uses a tiny fraction of the energy, gives explicit finality and punishes provable misbehaviour; in exchange it is more complex, concentrates influence in large staking pools and exchanges, and needs a **weak subjectivity** checkpoint: a node syncing from scratch must get a recent finalized block from a source it trusts, because old validators that withdrew can cheaply sign an alternative history.

20.6 BFT consensus and deterministic finality

Picture four referees scoring a match, one of whom has been bribed and may tell each colleague a different score. Raft's majority of 3 is not enough: the liar can vote on both sides. Byzantine fault tolerance needs quorums so large that any two of them overlap in at least one *honest* node:

$$n \geq 3f + 1, \qquad q = 2f + 1, \qquad |Q_1 \quad Q_2| \geq 2q - n = f + 1$$

Four nodes tolerate one liar, seven tolerate two, 100 tolerate 33. Since two quorums share $f + 1$ nodes and at most f of them lie, at least one honest node is in both and will never sign two conflicting decisions.

Tendermint (now CometBFT), used by the Cosmos chains, runs this as rounds: a proposer (rotating by voting power) broadcasts a block; validators **prevote**; on seeing prevotes from more than two-thirds of the voting power they **precommit**; once more than two-thirds precommit, the block is committed and final, typically within a few seconds. There are no forks to wait out. The price is the CAP choice from Chapter 18: if more than a third of the voting power is offline or partitioned the chain **halts** rather than fork. All-to-all voting also costs $O(n^2)$ messages per block, which keeps validator sets to the order of a hundred or two (HotStuff-style protocols, used by Aptos and others, make it linear with a leader that aggregates votes). PBFT (Castro and Liskov, 1999) is the classic ancestor; Ethereum's Casper FFG applies the same two-thirds rule to stake rather than to a fixed node list.

Finality style	Example	When is it final?	On attack
Probabilistic	Bitcoin	Never; risk falls with depth	Deep reorg
Head plus checkpoints	Ethereum	About 12.8 min (2 epochs)	Halt finality, slash
Deterministic, per block	CometBFT	In seconds, one block	Chain halts
Crash-fault (not BFT)	Raft, Paxos	One RTT to a majority	Trusts members

Choose a permissioned BFT chain (or a plain Raft cluster) when the participants are known and few; choose an open chain only when anyone must be able to join.

20.7 Throughput: why a chain does 7 or 200 transactions a second

A chain's throughput is set by how much it lets into each block and how often blocks come, because every full node must receive, execute and store everything:

$$\text{TPS} = \frac{\text{block capacity/transaction size}}{\text{block interval}}$$

Chain	Capacity per block	Tx size (illustrative)	Interval	TPS
Bitcoin	4M weight = 1M vbytes	250 vbytes	600 s	about 7
Ethereum L1, transfers	60M gas	21,000 gas	12 s	about 238
Ethereum L1, token transfers	60M gas	50,000 gas	12 s	about 100
All rollups, blob data	14 blobs x 128 KiB (target)	100 bytes compressed	12 s	about 1,500

Each line is a ceiling, not a measurement: the Ethereum rows assume blocks full to the 60 million gas limit, twice the fee-neutral target (next section), and the rollup row assumes every byte of the post-BPO2 blob target goes to compressed transactions, shared by all rollups together. Against these, Encore's flash sale makes 30,000 hold attempts per second and its normal peak is 6,000 requests per second; the seat inventory is load-tested at 6,000 holds per second. One flash sale's holds would need more than a hundred Ethereum L1s. The bottleneck is not the cryptography but replication by design: raising the gas limit or blob count raises the bandwidth, disk and CPU every validator needs, and pushes out home stakers. That is the scaling trade-off in one sentence.

20.8 Accounts, keys and wallets

On a blockchain the key *is* the account. An Ethereum **externally owned account** is a secp256k1 key pair; its address is the last 20 bytes of the Keccak-256 hash of the public key, and whoever can sign with the private key controls everything the address holds. There is no password reset, no support desk and no chargeback. A **contract account** has code instead of a key and acts only when called. Public-key signatures, key storage and KMS are taught in Chapter 22; the new parts are:

- **Seed phrases.** A wallet derives all its keys from one secret (BIP-32), written as 12 or 24 words from a 2,048-word list (BIP-39). Twelve words carry 128 bits of entropy plus a 4-bit checksum ($2048^{12} = 2^{132}$), far beyond brute force, so the realistic attacks are phishing, malware, and paper photographed or lost.
- **Custody.** *Custodial* wallets (an exchange holds the keys, you hold an IOU and a login) bring back the trusted operator and its failure modes; *self-custody* puts the whole risk on the user. Between them: **hardware wallets** that sign inside a secure chip and show the transaction on their own screen, **multisig** contracts such as Safe (m of n keys must sign), and **MPC wallets** where a threshold signature is computed from key shares on separate devices and the full key never exists anywhere.
- **Account abstraction.** Smart accounts make the wallet a contract with its own rules: spending limits, session keys that expire, social recovery, batched calls, gas paid by a sponsor. ERC-4337 does it without protocol changes (users sign *UserOperations*, bundlers submit them to an EntryPoint contract, paymasters pay gas); EIP-7702 (Pectra) lets an ordinary account delegate to contract code; and Fusaka's EIP-7951 added a secp256r1 precompile, so the passkeys of Chapter 22 can sign for smart accounts cheaply.

Agents with wallets. An AI agent that pays for an API or settles a trade on chain holds a key, and a key with no limits is the worst credential an agent can hold. Give it a smart account with a session key scoped to named contracts, a daily spending cap and an expiry, keep the master key with a human, and simulate every transaction before signing; payment protocols such as Coinbase's x402 (HTTP 402 Payment Required settled in stablecoins) are designed around exactly that. The sibling book *Designing Agentic Systems* covers tool permissions in depth.

20.9 Gas and fees: EIP-1559

Picture a toll road whose price per car rises automatically when the road is more than half full and falls when it is less, with an optional tip to jump the queue. **Gas** meters every EVM step (a plain transfer costs 21,000 gas; writing a new storage slot costs 20,000), and since EIP-1559 (London, August 2021) the price has two parts:

- A **base fee** per gas, set by the protocol and **burned**. Each block has a **target** of half the gas limit (elasticity 2, so 30 million of the 60 million gas limit Ethereum clients have defaulted to since the November 2025 raise ahead of

Fusaka). After a full block the base fee rises 12.5 percent; after an empty one it falls 12.5 percent; in between, proportionally.

- A **priority fee** (tip) per gas, paid to the block proposer. The user signs a *max fee* and a *max priority fee*; the effective tip is *min*(max tip, max fee − base fee) and the unused rest is never charged.

$$b_{n+1} = b_n\left(1 + \frac{1}{8}\,\frac{G_n - G_{\text{target}}}{G_{\text{target}}}\right), \qquad \text{fee} = g\,(b + \text{tip})$$

```python
GWEI = 10**9
DENOM, ELASTICITY = 8, 2                  # EIP-1559 constants


def next_base_fee(base: int, used: int, limit: int) -> int:
    """Base fee in wei for the next block (integer maths as in the EIP)."""
    target = limit // ELASTICITY
    if used > target:
        return base + max(base * (used - target) // target // DENOM, 1)
    return base - base * (target - used) // target // DENOM


def pay(gas: int, base: int, max_fee: int, max_tip: int) -> tuple[int, int]:
    """(burned, tip) in gwei; the rest of max_fee is never charged."""
    if max_fee < base:
        raise ValueError("waits in the mempool: max fee below base fee")
    tip = min(max_tip, max_fee - base)
    return round(gas * base / GWEI), round(gas * tip / GWEI)


LIMIT = 60_000_000                        # gas limit since late 2025
base = 1 * GWEI                           # illustrative quiet chain
for _ in range(10):                       # a mint storm fills 10 blocks
    base = next_base_fee(base, LIMIT, LIMIT)
print(f"base fee after 10 full blocks: {base / GWEI:.2f} gwei")
print(pay(21_000, base, max_fee=4 * GWEI, max_tip=2 * GWEI))
```

Ten full blocks multiply the base fee by $1.125^{10} = 3.25$, so a quiet 1 gwei becomes 3.25 gwei in two minutes; the transfer then burns 68,194 gwei and tips only 15,806 gwei because its 4 gwei cap squeezed the tip, about 0.000084 ETH in total (25 cents at an illustrative $3,000 per ETH). A transaction whose max fee falls below the base fee is not rejected; it waits in the mempool, which is why wallets set the max fee at about twice the current base fee. The design makes fees predictable block to block and removes most of the old first-price auction, but it cannot make block space cheaper when everyone wants it at once: during a mint storm prices still rise exponentially until demand gives up. Blob data has its own, separate base fee with the same shape (EIP-4844, below).

20.10 Smart contracts and the EVM

A **smart contract** is a program stored at an address whose code every node runs, on the same inputs in the same order, to reach the same state root: a deterministic state machine replicated thousands of times. The Ethereum Virtual Machine (EVM) is a 256-bit stack machine; contracts keep persistent **storage** (expensive), emit **events** into the block's logs (cheap, readable by off-chain programs but not by contracts) and call each other. Determinism rules out wall clocks, randomness and network calls: a contract sees only the block number and timestamp, its storage and its inputs. Contracts are usually written in Solidity. This sketch of a ticket registry (compiled with solc 0.8.30 and run against py-evm through web3.py) gives the organizer alone the right to mint, lets holders transfer, and freezes a ticket once it is scanned at the gate:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;

/// One token per seat; a teaching sketch, not a full ERC-721.
contract EncoreTickets {
    address public immutable organizer;
    mapping(uint256 => address) public ownerOf;    // seat id -> holder
    mapping(uint256 => bool) public used;          // scanned at the gate

    event Transfer(address indexed from, address indexed to,
                   uint256 indexed seat);           // same shape as ERC-721
    error NotAllowed();

    constructor() { organizer = msg.sender; }

    function mint(address to, uint256 seat) external {
        if (msg.sender != organizer || ownerOf[seat] != address(0))
            revert NotAllowed();                    // access control
        ownerOf[seat] = to;
        emit Transfer(address(0), to, seat);
    }

    function transfer(address to, uint256 seat) external {
        if (ownerOf[seat] != msg.sender || used[seat]) revert NotAllowed();
        ownerOf[seat] = to;
        emit Transfer(msg.sender, to, seat);
    }

    function checkIn(uint256 seat) external {
        if (msg.sender != organizer) revert NotAllowed();
        used[seat] = true;
    }
}
```

On the test chain a mint used 46,658 gas and a transfer 31,803, and calls from the wrong account reverted. Shown as `text` because the book's toolchain has no Solidity highlighter.

Immutability and upgrades. Deployed code cannot change, which is the point (users can read what they rely on) and the danger (a bug is permanent). Teams therefore deploy a **proxy** that forwards every call with `delegatecall` to an implementation contract whose address sits in a fixed storage slot (ERC-1967); upgrading swaps the address. That reintroduces a trusted operator, so serious protocols put the upgrade key behind a multisig and a **timelock** (say 48 hours) so users can leave before a change takes effect.

The bugs that cost the most.

- **Reentrancy.** A contract sends ETH before updating its balance; the recipient's code calls back in and withdraws again. The DAO lost 3.6 million ETH this way in 2016, and Ethereum hard-forked to undo it. Fix: checks, then effects, then interactions (update state before any external call), plus a reentrancy guard.
- **Access control.** A function that should be owner-only is public, or an initializer can be called twice; the 2017 Parity multisig library was "self-destructed" by a stranger and froze about 500,000 ETH. Fix: explicit roles, tests that every privileged function rejects strangers, audited libraries such as OpenZeppelin.
- **Oracle manipulation.** A lending contract reads an asset's price from one exchange pool; an attacker moves that pool with a flash loan (borrowed and repaid in the same transaction) and borrows against the fake price. Fix: time-weighted prices and multiple independent sources (see oracles below).

Audits, fuzzing, formal verification and bug bounties are standard practice because a contract that holds money is a public bounty for anyone who finds the bug first.

20.11 Tokens: ERC-20, ERC-721 and ERC-1155

A **token** is a contract that keeps a ledger and follows a standard interface, so every wallet, exchange and indexer can handle it without custom code.

Standard	What it records	Key calls	Typical use
ERC-20	Balances of interchangeable units	`transfer`, `approve`, `transferFrom`	Stablecoins, governance tokens
ERC-721	One owner per unique token ID	`ownerOf`, `safeTransferFrom`	NFTs: art, tickets, names
ERC-1155	Many token types in one contract	`balanceOf(id)`, batch transfers	Game items, ticket tiers

Two details matter for system design. First, `approve` grants another contract the right to move your tokens, often for an unlimited amount; a malicious or later-hacked spender can drain the wallet years later, so wallets show and revoke allowances and newer signatures (EIP-2612 permits) scope them. Second, an NFT usually stores only an ID and a URI; the image and metadata live off chain (see IPFS below), and royalties are only *advertised* (ERC-2981 returns a suggested royalty, which marketplaces may ignore). Stablecoins such as USDC are ERC-20 contracts whose issuer can freeze addresses: on-chain does not mean ungoverned.

20.12 Scaling: rollups, blobs and other chains

The **scalability trilemma** says a chain can easily have two of decentralisation (cheap nodes anyone can run), security and scale, but not all three on one layer. Ethereum's answer is to keep L1 small and verifiable and move execution to **rollups** (layer 2): a rollup's sequencer orders and executes transactions off chain, then posts the compressed transaction data and the new state root to L1, so anyone can recompute the L2 state from L1 alone. Users see a soft confirmation from the sequencer in about a second; L1 settlement follows in one of two ways:

- **Optimistic rollups** (Arbitrum One, OP Mainnet, Base) assume each state root is valid; anyone may submit a **fraud proof** during a **challenge window** of about seven days, which is why withdrawals to L1 take a week unless a liquidity provider fronts the money for a fee.
- **ZK rollups** (zkSync, Starknet, Scroll, Linea) post a succinct **validity proof** that L1 verifies; once it is accepted the batch is final, typically within hours, but proving needs heavy, specialised compute.
- **Honest caveat.** Most rollups still run one sequencer operated by one company and keep upgrade keys; users rely on an escape hatch (forcing a transaction through L1) and on the maturity "stages" that trackers such as L2BEAT publish. Soft confirmations are a promise by that sequencer, not L1 finality.

Data availability and blobs. A rollup is only as safe as the availability of its data: if the data were withheld, nobody could rebuild the state or prove fraud. EIP-4844 (Dencun, March 13, 2024) added **blobs**: 4,096 field elements of 32 bytes, 128 KiB each, carried beside the block, committed to by a KZG commitment, priced by their own EIP-1559-style blob base fee, and pruned after 4,096 epochs (about 18 days), long enough for any challenge. The target and maximum were 3 and 6 blobs per block, then 6 and 9 with Pectra, 10 and 15 at BPO1 (December 9, 2025) and 14 and 21 at BPO2 (January 7, 2026); Fusaka's PeerDAS (EIP-7594) lets each node check availability by sampling about one-eighth of the data, which is what made those raises safe.

$$14 \times 131{,}072\ \text{B} \times 7{,}200\ \tfrac{\text{blocks}}{\text{day}} \approx 13.2\ \text{GB/day}, \qquad \times 18\ \text{days} \approx 238\ \text{GB retained}$$

Other designs. Solana keeps one fast global chain (slots of about 400 ms) and pays with demanding validator hardware and bandwidth. **Cosmos** builds many application-specific chains on CometBFT, linked by the IBC protocol. **Sidechains** such as Polygon PoS, where Polymarket settles, run their own validator set: cheaper and faster, but secured by that set rather than by Ethereum. The sibling book *Dialogflow CX in Practice* has a Polymarket chapter that shows an off-chain order book settling on such a chain.

20.13 Bridges and oracles

A **bridge** moves value between chains that cannot see each other. The common design locks tokens in a contract on chain A and mints a wrapped copy on chain B once a set of signers attests to the lock; the reverse burns and unlocks. The bridge contract therefore holds the pooled assets of everyone who crossed, guarded by whatever verifies the attestation: an m-of-n multisig, a light client, or a validity proof. That makes bridges the most attacked component in

the ecosystem. In 2022 Ronin lost about $600 million when attackers obtained 5 of its 9 validator keys, Wormhole about $320 million through a signature-verification bug, and Nomad about $190 million after an initialization error let any message pass. In system terms, a bridge is a cross-system transaction without a shared consensus, the problem Chapter 18 solves with sagas: it needs the same care with verification, idempotency (never mint twice for one lock) and rate limits (cap withdrawals per hour so a bug cannot drain everything at once).

An **oracle** brings outside facts (prices, weather, match results) on chain. Because a contract cannot call an API, a network of reporters signs observations and an aggregation contract publishes, for example, the median of many sources when the price moves more than a deviation threshold or a heartbeat interval passes (Chainlink price feeds work this way). The oracle is a trusted input: design for stale or manipulated values with time-weighted averages, multiple sources, sanity bounds and circuit breakers, exactly as Chapter 11 treats any flaky dependency.

20.14 Reading the chain: nodes, indexers and reorgs

Nobody queries a blockchain like a database. An application talks to a **node** over JSON-RPC (`eth_call` to read contract state, `eth_sendRawTransaction` to submit, `eth_getLogs` to read events by contract and topic). Running Ethereum yourself means two programs, an execution client (Geth, Nethermind, Reth, Besu, Erigon) and a consensus client (Lighthouse, Prysm, Teku, Nimbus, Lodestar), with a few terabytes of fast SSD. Most teams rent RPC from providers such as Infura, Alchemy or QuickNode, or from the clouds: Google Cloud Blockchain Node Engine runs managed Ethereum nodes, while Amazon Managed Blockchain is closed to new customers from October 29, 2026 and ends on September 30, 2027 (Azure has no first-party offering). An RPC provider is a single point of failure and a censor in practice, so production systems use two providers or one provider plus their own node, and verify critical reads against a second source.

Logs are easy to append and hard to query, so applications run an **indexer**: a worker that follows new blocks, decodes the events it cares about (an ERC-721 `Transfer` has the event signature hash as topic 0 and from, to and token ID as topics 1 to 3) and writes them into Postgres or a search index, which serves the API. The Graph popularised this as hosted "subgraphs"; many teams run their own. The hard part is that the head of the chain can **reorganize**: a block the indexer already processed may be replaced.

> **Design question.** How would you index resale `Transfer` events of Encore tickets reliably, despite reorgs and RPC failures?
>
> Requirements: no missed or duplicated transfers, owners visible within seconds, and money-moving decisions only on settled data. Store for each processed block its number, hash and parent hash. On each new block, if its parent hash differs from the stored hash at height $n - 1$, walk back to the common ancestor, delete the rows written for the orphaned blocks (key every row by block hash and log index, so replays are idempotent upserts), and re-apply the new branch. Serve two views: `latest` for the app ("transfer pending"), and `finalized` (about 13 minutes on Ethereum, or the rollup's L1-settled state) for anything irreversible, such as paying a seller or re-issuing a gate QR code. Backfill with `eth_getLogs` in bounded block ranges from two RPC providers and compare counts per range. It is the outbox and change-data-capture pattern of Chapter 14 with a source that can rewind. See also *50 System Designs* #12 (Distributed Log).

20.15 Storage off chain: IPFS, Filecoin and Arweave

Storing data on Ethereum L1 costs about 20,000 gas per 32-byte word, so a 50 KB ticket PDF would cost over 30 million gas, half a block. Blockchains store **commitments** (hashes) and keep the bytes elsewhere. **IPFS** addresses content by its hash: a file is split into chunks, linked in a Merkle DAG, and named by a **CID** that is the multihash of the root, so any peer can serve it and the reader verifies what it gets (the same idea as Git and the content-hash dedup of *50 System Designs* #10, Cloud File Storage). IPFS does not promise that anyone keeps the data; someone must **pin** it. **Filecoin** adds a market where storage providers are paid for deals and must keep proving (proof of replication and of spacetime) that they still hold the data; **Arweave** charges once up front into an endowment meant to fund permanent storage. An NFT whose metadata is an `https://` URL on the issuer's server is only as permanent as that server.

20.16 DeFi in one equation: the AMM, and MEV

Automated market makers (AMMs) replaced the order book with a formula. A pool holds reserves x of token X and y of token Y and accepts any trade that keeps their product constant (Uniswap v2's rule, before a 0.3 percent fee that stays in the pool):

$$(x + \Delta x')(y - \Delta y) = xy \qquad \Delta y = \frac{y\,\Delta x'}{x + \Delta x'}, \qquad \text{price impact} \approx \frac{\Delta x}{x + \Delta x}$$

```python
def swap(x: float, y: float, dx: float, fee: float = 0.003):
    """Sell dx of token X into an x * y = k pool; return dy and new pool."""
    dx_in = dx * (1 - fee)                  # the fee stays in the pool
    dy = y * dx_in / (x + dx_in)            # keeps (x + dx_in)(y - dy) = k
    return dy, x + dx, y - dy


x, y = 1_000.0, 3_000_000.0                 # 1,000 ETH and 3M USDC
spot = y / x                                # 3,000 USDC per ETH
for dx in (1, 10, 100):
    dy, *_ = swap(x, y, dx)
    impact = 1 - (dy / dx) / spot
    print(f"sell {dx:>3} ETH -> {dy:>9,.0f} USDC, "
          f"{dy / dx:,.0f} each, {impact:.1%} below spot")
```

In a pool of 1,000 ETH and 3 million USDC, selling 1 ETH gets 2,988 USDC (0.4 percent below the 3,000 spot, mostly the fee), 10 ETH lose 1.3 percent and 100 ETH lose 9.3 percent: depth is everything. **Liquidity providers** deposit both tokens to earn the fees and bear the loss when the price moves away from their entry. Arbitrageurs keep the pool price in line with other markets, which is also why a single pool's spot price is a dangerous oracle.

MEV (maximal extractable value) is the profit a block producer, or anyone who bids for its ordering, can take by including, excluding or reordering transactions. A visible swap with 2 percent slippage tolerance in the public mempool invites a **sandwich**: buy immediately before it, let the victim push the price, sell right after. The ordering freedom of the mempool is the cause. Mitigations: tight slippage limits, private transaction relays, batch auctions; on Ethereum, block building is outsourced to specialised builders through MEV-Boost relays today, and Glamsterdam's enshrined proposer-builder separation moves that auction into the protocol. A stock exchange solves the same fairness problem with one trusted sequencer and price-time priority (*50 System Designs* #43, Stock Exchange Matching Engine).

20.17 Decentralized identity

Decentralized identifiers (DIDs, a W3C standard) are identifiers such as `did:web:encore.example` or `did:key:...` that resolve to a document listing the public keys that control them; **verifiable credentials** (also W3C) are claims signed by an issuer ("this wallet belongs to a verified fan over 18") that the holder presents and anyone can verify against the issuer's key without calling the issuer. **Sign-In with Ethereum** (ERC-4361) is the practical entry point: the site shows a structured message with its domain and a fresh nonce, the wallet signs it, and the server verifies the signature (EIP-1271 for smart accounts) and then issues an *ordinary* session or JWT (Chapter 22). The wallet replaces the password; everything after login is Web2.

20.18 How a dApp is built end to end

A real dApp is mostly ordinary distributed systems around a thin on-chain core:

```
browser (static app on S3 + CloudFront)
  |  wallet: signs txs and SIWE logins (keys stay on device)
  v
RPC providers x2 / own node <-------- indexer workers -> Postgres
  |  eth_call, eth_sendRawTransaction     ^ follows blocks, reorg-safe
  v                                       |
contracts on L2 (rollup) ---- batches ---> Ethereum L1
  ^
backend API (EKS / Lambda): auth, off-chain data, KYC, notifications,
relayer or paymaster that sponsors gas, signs nothing it cannot cap
```

Off chain sit the front end and its CDN (S3 and CloudFront here; Cloud Storage and Cloud CDN on GCP, Blob Storage and Front Door on Azure), the RPC gateway with failover and rate limits, the indexer and its database, search, notifications, fiat on-ramps through a payment provider (*50 System Designs* #24, Payment System), customer support, analytics and the keys of any relayer. Each is designed with the rest of this book; the chain changes only the source of truth for ownership.

20.19 Regulation, custody and security operations

Regulation (October 2026, in brief, not legal advice): in the EU, MiCA has applied in full since December 30, 2024 and licenses crypto-asset service providers; in the US, the GENIUS Act (July 2025) set rules for payment stablecoins; travel-rule obligations make exchanges pass sender and recipient details with transfers; tickets on chain stay subject to consumer-protection and resale laws. **Custody** turns key management into operations: cold storage for most funds, hot wallets with withdrawal limits and address allowlists, HSM- or MPC-backed signing, multi-person approval, and a signed record of every movement (*50 System Designs* #44, Digital Wallet). **Security operations** add transaction simulation before signing, monitoring of contracts and admin keys, incident runbooks that include pausing contracts, and supply-chain hygiene for signing interfaces: in February 2025 attackers took about $1.5 billion from the exchange Bybit by tampering with the multisig web interface its signers used, so the signers approved a transaction they could not see.

20.20 When a blockchain is the wrong tool

A blockchain buys one property, **agreement without a trusted operator**, and pays for it in throughput, latency, privacy, cost and operational risk. Before choosing one, walk the checklist:

- Do several parties that do not trust each other, and will not accept one operator, all need to write? If one organisation already runs the system (Encore does), use its database.
- Must anyone be able to join, verify or transact without permission, including against the operator's will? If not, a permissioned BFT ledger or a replicated database is simpler.
- Can the workload live with seconds to minutes per confirmation and tens to thousands of transactions per second, shared with the world?
- Can the data be public forever? Privacy law needs erasure (GDPR), which an immutable public ledger cannot give; store only hashes on chain, never personal data.
- Are mistakes acceptable as permanent? Refunds, chargebacks and support overrides need an admin path, which brings the operator back.
- Will users manage keys, or will you custody them for them (then you are the operator again)?

> **Design question.** A venue partner asks for Encore's ticket sales on "a blockchain" so that nobody can alter sales figures afterwards. What do you build?
>
> The real requirement is tamper evidence, not decentralisation. Keep the orders in Postgres, append every sale to a hash-chained audit table (each row stores the hash of the previous one), and once an hour compute the Merkle root of that hour's rows and publish it: to the partner, to a transparency log, or as one transaction on a public chain (one small transaction an hour costs cents, illustrative). Any later edit breaks the chain and the published

root, and a Merkle proof of 17 hashes shows any single sale of 100,000 is included. This is Certificate Transparency's design (Chapter 19). Managed ledger databases exist too (Azure SQL ledger tables; Amazon QLDB was retired in July 2025, with Aurora PostgreSQL as AWS's suggested path).

20.21 In Encore: NFT tickets on a rollup?

Design question. Should Encore issue tickets as NFTs on a rollup, to fight scalpers and earn royalties on resale?

Requirements. Fair primary sale, controlled resale with a royalty, fast gate entry, refunds and cancellations, 20 million mostly non-crypto users. **Numbers.** Minting the 50,000 seats costs about 46,700 gas each (the sketch above), 2.3 billion gas: 39 completely full L1 blocks, about 8 minutes of all of Ethereum, and the base fee would climb 12.5 percent per full block while it lasted. On a rollup the same mints cost a fraction of a cent to a few cents each (illustrative) and confirm in about a second, so a rollup is the only plausible venue. The 30,000 hold attempts per second at 10:00 and the 10-minute hold TTL stay off chain regardless: tickets are minted *after* payment, from the 20,000 orders.

Scalpers. NFTs make transfers easier, not harder: bots create wallets for free. Anti-scalping still comes from the waiting room, identity checks and per-fan caps of Chapter 22. Restricting transfers in the contract (allowlisted marketplace, price cap) works only because Encore controls the allowlist. **Royalties.** ERC-2981 merely advertises a royalty and several marketplaces made it optional; enforcement needs transfer restrictions, and selling the wallet or a private key bypasses even those.

Keys and gate. Fans lose keys and phones; Encore must offer custodial or smart accounts with passkey recovery, which makes Encore the operator again. At the gate, a rotating QR code signed by the holder's key is checked against an indexed snapshot, because scanning 50,000 fans in an hour cannot wait for a chain. **What remains off chain.** Waiting room, holds, fiat payment and refunds through the payment provider, the indexer, notifications, support, fraud review, GDPR deletion, and an admin key able to cancel and re-issue tickets.

Verdict. For the primary sale, no: every hard problem stays where it was and new ones (keys, fees, public data, a reorg-aware indexer) arrive. A defensible experiment is opt-in: after the show, let fans mint a collectible of their ticket on a rollup, or run resale through Encore's own marketplace with signed, re-issued tickets. See also *50 System Designs* #25 (Ticket Booking), #24 (Payment System) and #44 (Digital Wallet).

20.22 Numbers to remember

Quantity	Value
Bitcoin block interval	10 min; retarget every 2,016 blocks
Bitcoin, attacker q = 0.1	0.024 percent after 6 blocks
Ethereum slot, epoch	12 s; 32 slots = 6.4 min
Ethereum finality	about 2 epochs, 12.8 min
Validator stake	32 ETH min, 2,048 ETH max effective
BFT quorum	n >= 3f + 1, quorum 2f + 1
Gas limit, target	60M, 30M (since late 2025)
ETH transfer	21,000 gas
Blob	128 KiB, kept about 18 days
Blobs per block (BPO2)	target 14, max 21
Optimistic challenge window	about 7 days
Throughput ceilings	BTC about 7, ETH L1 about 100 to 240 tx/s

20.23 Pitfalls

- Treating one confirmation (or the `latest` block) as final -> act on `finalized`, or on a confirmation depth chosen from $P(q, z)$ and the value at risk.
- An indexer that only appends -> key rows by block hash and log index and roll back on reorgs.
- One RPC provider -> two providers or your own node, with timeouts and cross-checks.
- Sending ETH or calling out before updating state -> checks, effects, interactions; reentrancy guards.
- A single upgrade or bridge key -> multisig plus timelock, withdrawal rate limits, monitoring.
- Reading a price from one AMM pool -> time-weighted, multi-source oracles with bounds.
- Personal data or files on chain -> hashes on chain, bytes in object storage or IPFS with pinning, personal data in a deletable database.
- Assuming users will keep seed phrases -> smart accounts with passkeys and recovery, or honest custody.
- Choosing a blockchain where one trusted operator exists -> a database with a hash-chained audit log.

20.24 Remember

- A blockchain is a replicated state machine under a Byzantine failure model, with Sybil resistance bought by work or stake.
- Finality is probabilistic in proof of work ($P(q, z)$), checkpointed in Ethereum (about 12.8 minutes), immediate but halting in BFT ($n \geq 3f + 1$).
- Throughput is capped on purpose: block capacity over transaction size over block interval; rollups and blobs scale data, not trust.
- Fees are an auction for scarce replicated block space: EIP-1559 base fee (burned, 12.5 percent steps) plus a tip.
- The weakest links are keys, bridges, oracles, upgrade admins and the off-chain front end, not the hash function.
- Use a chain only when no single operator can be trusted; otherwise a database with a tamper-evident log wins.

21 Big data: batch, streams and lakehouses

Encore's apps emit a billion clickstream events a day, and its orders database changes thousands of times a minute during a flash sale. Organizers want a dashboard that moves within seconds; analysts want last quarter's funnel by city; the support agent wants fresh features about each customer. This chapter answers one question: how do you turn a firehose of events into answers, some tonight and some this second, without paying to read every byte twice? The answer is one log feeding two paths, a cheap columnar lake for the slow path and a stateful stream processor for the fast path.

21.1 The picture

Think of a post office. The **batch** path is the night shift: every letter of the day is dumped on big tables, sorted by postcode, counted and bundled before morning. The **stream** path is the conveyor belt: each letter is read and routed the moment it arrives, and a clerk keeps running tallies on a whiteboard. The **lake** is the warehouse at the back where every letter is kept, filed by date, so the night shift can re-sort a whole month if the rules change.

```
web, app --clicks--+                                    seconds
                   +--> Kafka --> Flink (windows, state) --> Redis/OLAP
orders DB ---CDC---+    (log) |                             dashboards
                              v
                 S3 lake: Parquet files + Iceberg tables
                              |                    minutes to hours
                              +--> Spark / SQL --> warehouse tables:
                                                   reports, ML, agents
```

Everything in this chapter is one box of that diagram: how the log is sized, how files are laid out in the lake, how batch engines split work, and how a stream processor keeps correct counts when events arrive late or twice.

21.2 Volume, velocity and variety

"Big data" means three pressures at once, the classic three Vs: **volume** (too much to fit on one machine's disk or to scan in one machine's lifetime), **velocity** (arriving faster than one machine can write or react) and **variety** (many shapes: JSON clicks, database rows, logs, images). Encore's clickstream has all three: every page view, seat-map refresh, hold, search and checkout step is an event of about 1 KB of JSON (illustrative).

$$r_{avg} = \frac{N_{day}}{86,400}, \qquad B = r \quad s, \qquad S_{year} = \frac{N_{day} \quad s \quad 365}{c}$$

With $N_{day} = 10^9$ events, s = 1 KB and a columnar compression ratio $c = 10$ (illustrative):

- average rate $10^9 / 86,400 \approx 11,600$ events/s, about 11.6 MB/s;
- the flash sale drives a 5x peak of about 58,000 events/s, or 58 MB/s; clients batch events and sample seat-map refreshes, so the 200,000 seat-map reads per second do not become 200,000 events per second;
- raw volume is 1 TB a day and 365 TB a year as JSON, but about 100 GB a day and 36.5 TB a year as compressed Parquet. At roughly $0.023 per GB-month in S3 Standard, a year of Parquet costs about $840 a month to keep.

The numbers decide the architecture. 58 MB/s of small writes is beyond one database's comfortable write rate but easy for a partitioned log: if one consumer processes an illustrative 5 MB/s, the peak needs 12 partitions, and Encore creates 48 to leave room for 4x growth (Chapter 14). 36.5 TB a year is far too much to scan per query on one machine, so the lake must let queries skip most of it.

Variety is handled with **schemas**. Clients send events with an `event_type`, a `schema_version`, an `event_uuid` (for deduplication), the concert's `event_id`, a `user_id` and an `event_time` set on the device. Producers register Avro or Protobuf schemas in a schema registry (Confluent Schema Registry, AWS Glue Schema Registry; Azure Event Hubs has its own), which rejects changes that would break consumers, the same evolution rules as in Chapter 5.

21.3 Batch processing: MapReduce

MapReduce (Dean and Ghemawat, Google, 2004) is the night shift made into a programming model. You write two pure functions and the framework runs them on thousands of machines:

```
input splits        map               shuffle (by key)        reduce
[e1 view, e2 view] -> (e1,1)(e2,1) --+--> reducer 0: e1 [1,1] -> e1 2
[e1 view, e3 view] -> (e1,1)(e3,1) --+--> reducer 1: e2 [1]   -> e2 1
                                                    e3 [1]   -> e3 1
```

1. **Split.** The input in the distributed file system is cut into splits (an HDFS block is 128 MB by default); one map task per split runs next to the data.
2. **Map.** Each record becomes zero or more key-value pairs: a page view of event `e1` becomes `(e1, 1)`.
3. **Combine** (optional). A mini-reduce on the mapper adds up its own `(e1, 1)` pairs before anything crosses the network.
4. **Shuffle.** A partitioner sends each key to reducer $p(k) = h(k) \bmod R$, so all values of a key meet in one place. Mappers sort and spill to local disk; reducers fetch and merge-sort.
5. **Reduce.** Each reducer folds the values of a key into a result and writes it back to the file system.

Because tasks are deterministic and inputs are immutable, fault tolerance is "run it again": a failed or slow task (a straggler) is re-executed elsewhere, and speculative execution races a backup copy against the slowest tasks. The shuffle is the expensive step. Without a combiner, one billion views cross the network as one billion pairs; with one, each mapper sends at most one pair per distinct key it saw.

```python
from collections import Counter, defaultdict
from zlib import crc32

R = 2                                          # number of reducers
splits = [["e1,view", "e2,view", "e1,hold"], ["e1,view", "e3,view"]]

def map_and_combine(split):                    # map, then pre-add locally
    pairs = (record.split(",") for record in split)
    return Counter(event for event, action in pairs if action == "view")

reducers = [defaultdict(list) for _ in range(R)]
for split in splits:                           # shuffle: key -> one reducer
    for key, n in map_and_combine(split).items():
        reducers[crc32(key.encode()) % R][key].append(n)

result = {k: sum(v) for red in reducers for k, v in red.items()}  # reduce
print(dict(sorted(result.items())))            # {'e1': 2, 'e2': 1, 'e3': 1}
```

Where it runs. Hadoop MapReduce on HDFS (the open-source clone of Google's MapReduce and GFS, *50 System Designs* #37) is now mostly legacy; Amazon EMR (GCP Dataproc, Azure HDInsight) still runs it. The model survives everywhere else: every SQL `GROUP BY` on a distributed engine is a map, a shuffle by key and a reduce. **Trade-off:** writing every intermediate result to disk makes MapReduce robust but slow for multi-step jobs, which is exactly what Spark fixed.

21.4 Spark: DAGs, partitions and lazy evaluation

Apache Spark keeps the map-shuffle-reduce idea but plans the whole job first and keeps intermediate data in memory. Picture a recipe card: you write down every step, and only when someone says "serve" does the kitchen look at the whole card, merge steps that can share a pan, and start cooking.

- **Lazy evaluation.** Transformations (`filter`, `select`, `groupBy`, `join`) only build a logical plan. An **action** (`show`, `count`, `write`) hands the plan to the Catalyst optimizer, which pushes filters down to the scan, prunes unused columns and picks join strategies.
- **DAG of stages.** The plan becomes a directed acyclic graph. *Narrow* steps (filter, map) run inside a partition and are pipelined together; *wide* steps (group by, join on a new key) need a shuffle, and each shuffle is a stage boundary.

- **Partitions and tasks.** Each stage runs one task per partition on executors; a lost partition is recomputed from its lineage in the DAG.

A scan stage has about D/p tasks for D bytes of input and a split size p of 128 MB (`spark.sql.files.maxPartitionBytes`): one day of Encore's lake, 100 GB of Parquet, gives about 745 scan tasks; on 50 executors with 4 cores each (200 slots, illustrative) that is four waves. After a shuffle Spark starts from `spark.sql.shuffle.partitions = 200`, and Adaptive Query Execution (on by default since Spark 3.2) coalesces small shuffle partitions and splits skewed ones at run time.

```
from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder.master("local[4]").getOrCreate()
clicks = spark.createDataFrame(
    [("u1", "e1", "view"), ("u2", "e1", "hold"),
     ("u1", "e2", "view"), ("u3", "e1", "view")],
    "user_id string, event_id string, action string")

funnel = (clicks                                    # nothing runs yet:
          .filter(F.col("action").isin("view", "hold"))   # narrow
          .groupBy("event_id", "action")                  # wide: shuffle
          .agg(F.countDistinct("user_id").alias("users")))
funnel.explain()                                    # print the plan
funnel.orderBy("event_id", "action").show()         # action: job runs
```

```
HashAggregate(keys=[event_id, action], count(distinct user_id))
+- Exchange hashpartitioning(event_id, action, 200)            <- shuffle 2
   +- HashAggregate(keys=[event_id, action, user_id])
      +- Exchange hashpartitioning(event_id, action, user_id)  <- shuffle 1
         +- Filter action IN (view, hold)    (abridged; e1 view has 2 users)
```

The plan shows a lesson worth carrying into interviews: an exact distinct count needs two shuffles. Replacing it with `F.approx_count_distinct` (HyperLogLog, Chapter 19) leaves one shuffle and a small sketch per key. **Where it runs:** Spark on Amazon EMR or AWS Glue (GCP Dataproc, Azure HDInsight or Azure Databricks), and Databricks on all three clouds. **Trade-off:** Spark wins on large multi-step jobs; for a few hundred GB, a single-node engine such as DuckDB or a serverless warehouse is often faster and cheaper than starting a cluster.

21.5 Storage formats: rows, columns and Parquet

A row store files each order as one card holding all its fields; a column store keeps one long strip per field. To add up the price of a million orders, the row store pulls every card, the column store reads one strip.

```
row layout (Postgres, Avro, JSON lines)   column layout (Parquet, ORC)
[t1,u7,e1,view,Paris]                      event_time: t1 t2 t3 ...
[t2,u9,e1,hold,London]                     user_id:    u7 u9 u7 ...
[t3,u7,e2,view,Paris]                      action:     view hold view ...
good for: fetch/insert one record          good for: scan few columns
```

Parquet, the default file format of the lake, splits a file into **row groups** (typically 128 MB to 1 GB), each holding one **column chunk** per column, divided into pages. Similar values sit together, so encodings work well: dictionary encoding turns `view, hold, view` into small integers, run-length encoding collapses repeats, and a general codec (Snappy, zstd) squeezes the rest. Each chunk stores min/max statistics, so a reader skips row groups that cannot match a filter (**predicate pushdown**) and never touches unused columns (**projection pushdown**). ORC is the same idea from the Hive world; Avro is a row format, good for messages and CDC streams.

$$\text{bytes scanned} = D \;\frac{\text{columns read}}{\text{columns}}\;\frac{\text{partitions read}}{\text{partitions}}$$

A year of Encore's lake is 36.5 TB. A query over one day reading 3 of 30 columns scans 36.5 TB $\frac{3}{30}\;\frac{1}{365}$ = 10 GB, which costs $0.05 at Amazon Athena's $5 per TB scanned; the same query on unpartitioned JSON would read 365 TB.

```
import json, os, random
import pyarrow as pa, pyarrow.parquet as pq

random.seed(7)
n, acts = 1_000_000, ["view", "search", "hold", "checkout", "pay"]
rows = {"event_time": [1_792_000_000_000 + i * 50 for i in range(n)],
        "user_id": [f"u{random.randrange(200_000)}" for _ in range(n)],
        "event_id": [f"e{random.randrange(500)}" for _ in range(n)],
        "action": random.choices(acts, [70, 15, 8, 4, 3], k=n),
        "city": random.choices(["London", "Paris", "Berlin"], k=n)}
with open("clicks.jsonl", "w") as f:
    for i in range(n):
        f.write(json.dumps({k: v[i] for k, v in rows.items()}) + "\n")
pq.write_table(pa.table(rows), "clicks.parquet", compression="zstd")
for path in ("clicks.jsonl", "clicks.parquet"):
    print(path, f"{os.path.getsize(path) / 1e6:.1f} MB")
# clicks.jsonl 107.3 MB, clicks.parquet 6.7 MB  (toy data: 16x)

-- DuckDB on the file above: reads only the 'action' and 'city' chunks
SELECT action, count(*) AS events
FROM 'clicks.parquet'
WHERE city = 'Paris'
GROUP BY action
ORDER BY events DESC;
-- EXPLAIN shows PARQUET_SCAN with Projections: action, Filters: city='Paris'
```

Trade-off: columnar files are write-once and slow to update one row, and each file has fixed overhead, so a lake of millions of tiny files is slow. Batch writes into large files, and leave row-at-a-time updates to the operational databases of Chapter 12.

21.6 Table formats: Iceberg, Delta and Hudi

A folder of Parquet files is not a table: two writers can clobber each other, a reader can see half a job's output, and "delete user 42" means rewriting files by hand. A **table format** adds a ledger beside the files. Readers trust the ledger, never the folder listing; a commit is one atomic swap of the ledger pointer.

```
catalog (Glue, S3 Tables, Nessie, Unity, Polaris)
  clicks -> metadata v3.json ------------------- one atomic pointer swap
              schema, partition spec, snapshots     per commit
              +-> snapshot 2: manifest list
                    +-> manifest A: file1.parquet  day=10-14  min/max stats
                    +-> manifest B: file2.parquet  day=10-14  min/max stats
```

Apache Iceberg (from Netflix) keeps metadata files, manifest lists and manifests with per-file statistics, so planning a query prunes files without listing S3. Writers commit with optimistic concurrency: write new files, then compare-and-swap the catalog pointer; on conflict, retry against the new snapshot. *Hidden partitioning* derives partitions from columns (`days(event_time)`, `bucket(16, event_id)`), so queries filter on `event_time` and never name a partition column. Every commit is a snapshot, giving time travel (`TIMESTAMP AS OF`) and rollback. **Delta Lake** (from Databricks) keeps an ordered `_delta_log` of JSON commit files with a Parquet checkpoint every 10 commits by default. **Apache Hudi** (from Uber) focuses on upserts, with copy-on-write tables (rewrite files on update) and merge-on-read tables (append change logs, merge at read time). All three give ACID commits, schema evolution, row-level deletes for GDPR and compaction of small files.

```sql
-- Spark SQL with the Iceberg runtime (tested with Spark 4.0, Iceberg 1.10)
CREATE TABLE lake.analytics.clicks (
  event_uuid STRING, user_id STRING, event_id STRING,
  action STRING, event_time TIMESTAMP)
USING iceberg
PARTITIONED BY (days(event_time), bucket(16, event_id));

-- idempotent load: a replayed batch inserts nothing twice
MERGE INTO lake.analytics.clicks t
USING staged_clicks s
ON t.event_uuid = s.event_uuid
WHEN NOT MATCHED THEN INSERT *;
CALL lake.system.rewrite_data_files(table => 'analytics.clicks');
```

Where it runs: Iceberg in Amazon S3 Tables, AWS Glue, Athena, EMR, Redshift, Snowflake, BigQuery and Trino; Delta in Databricks and Microsoft Fabric; Hudi on EMR. **Trade-off:** pick Iceberg for the widest engine support, Delta when Databricks is the main engine, Hudi for heavy upsert workloads; whichever you pick, schedule compaction and snapshot expiry or metadata and small files pile up.

21.7 Lake, warehouse and lakehouse

Three places to keep analytical data, like three ways to keep a library. A **data lake** is a vast basement of boxes: cheap, holds anything, and you need to know what is in each box (schema on read). A **warehouse** is a curated reading room: everything catalogued, fast to find, but only what the librarians admitted and in their own shelving system (schema on write, proprietary storage). A **lakehouse** puts a catalogue and lending rules on the basement itself: the open table formats of the previous section turn lake files into tables that many engines can read and write safely.

	Lake	Warehouse	Lakehouse
Storage	S3 files	Engine-managed	S3 files + table format
Schema	On read	On write	On write, evolvable
Transactions	No	Yes	Yes (table format)
Engines	Any	One	Many on one copy
Examples	S3 + Athena	BigQuery, Snowflake, Redshift	Databricks, Iceberg + Athena/Trino

The warehouses converged on the lakehouse: BigQuery (serverless, columnar Capacitor storage) reads and manages Iceberg tables through BigLake, Snowflake separates storage from elastic "virtual warehouses" and supports Iceberg tables, Amazon Redshift (RA3 managed storage, Redshift Serverless) queries the lake through Spectrum, and Microsoft Fabric (OneLake, Delta) is Azure's answer. A common layout is the **medallion** pattern: *bronze* tables hold raw events as ingested, *silver* tables are cleaned, deduplicated and typed, and *gold* tables are business aggregates that dashboards read.

> **Design question.** Where should Encore keep 36.5 TB a year of clickstream so analysts, data scientists and the finance team can all query it?
>
> Requirements: years of history, SQL for analysts, Spark and Python for data science, nightly finance reports, cost proportional to use. Store events as Parquet in S3, registered as Iceberg tables (Amazon S3 Tables or the AWS Glue Data Catalog), partitioned by `days(event_time)`: bronze `clicks_raw`, silver `clicks` (deduplicated on `event_uuid`), gold `funnel_daily`. Analysts use Athena or Redshift Serverless, data scientists use Spark on EMR, all on one copy. A day-scoped query reads about 10 GB, about $0.05 on Athena. Trade-off: you operate compaction and snapshot expiry yourself, and interactive latency is seconds, not milliseconds; a warehouse alone is simpler if only analysts query the data. See also *50 System Designs* #28 (Ad Click Aggregator).

21.8 Stream processing

Batch answers "what happened yesterday"; a stream processor answers "what is happening now" by running the query forever over an unbounded input. Picture the conveyor belt again: each operator is a clerk at a station, the belt is split into lanes by key, and every clerk keeps a whiteboard (state) for the keys in their lane.

```
Kafka clicks, 48 partitions (p0 .. p47), keyed by user_id
[source] -> [parse, pre-agg] -> keyBy(event_id) -> [window sum] -> [sink]
 offsets     partial counts     network shuffle    keyed state      Redis,
 checkpointed per partition                        in RocksDB       Kafka
```

The mechanism mirrors Spark's DAG, but the operators run continuously. A source reads partitions and tracks offsets; stateless operators parse and filter; `keyBy` repartitions the stream by hash of the key so all events of one concert reach one parallel instance. The flash-sale show would make that instance hot, so the topic is keyed by `user_id` for spread and each source instance pre-aggregates partial counts per second before the shuffle, the combiner idea from MapReduce; stateful operators keep counts, joins or deduplication sets in local state (in memory or RocksDB on local disk); sinks write results. Parallelism is capped by the number of source partitions, which is why Encore sizes the topic at 48.

Engine	Model	State	Runs on
Apache Flink	Event at a time	RocksDB + checkpoints to S3	Amazon Managed Service for Apache Flink, Kubernetes
Kafka Streams	Library in your service	RocksDB + changelog topics	Any JVM service
Spark Structured Streaming	Micro-batches	State store + checkpoint dir	EMR, Databricks, Glue
Apache Beam	Unified batch/stream API	Runner-managed	GCP Dataflow, Flink

The log under it is Kafka on Amazon MSK (GCP Managed Service for Apache Kafka, Azure Event Hubs with its Kafka endpoint) or Amazon Kinesis Data Streams (GCP Pub/Sub, Azure Event Hubs). Azure's managed SQL-on-streams service is Azure Stream Analytics. **Trade-off:** Flink gives the lowest latency and the richest event-time semantics but is a cluster to run; Kafka Streams needs no cluster but only reads Kafka; Spark Structured Streaming reuses batch code at the cost of micro-batch latency (typically seconds).

21.9 Windows, watermarks and state

Every stream answer is "count per key per window", and the hard part is time. **Event time** is when the fan tapped the seat (stamped by the device); **processing time** is when the operator sees it. They differ by network delay, retries and phones that were offline in the stadium car park, so events arrive out of order.

```
event time ->  0s        60s       120s
tumbling      [----w1----)[----w2----)          fixed, no overlap
sliding       [------10 min------)              size 10m, slide 1m:
                [------10 min------)            each event in 10 windows
session       [-a-a--a-]      [-a-a-]           per user, closes after
                         gap>30m                30 minutes of silence
```

A **watermark** is the operator's statement "I believe no more events older than W will arrive". The common bounded-out-of-orderness rule, and when a window fires:

$$W(t) = max(\text{event time seen}) - \delta, \qquad \text{window } [s, s+w) \text{ fires when } W \geq s + w$$

With w = 60 s and δ = 10 s, the window $[0, 60)$ fires once an event stamped 70 s or later arrives. Events for a fired window are **late**: drop them, send them to a side output for the batch path to correct, or keep windows open for an extra *allowed lateness* and emit updates. A larger δ means more complete answers and later answers; the watermark is the dial between correctness and latency.

State is what makes this a database problem. Windowed counts are small, but deduplication is not: remembering every `event_uuid` for one hour at a billion events a day is 41.7 million IDs, about 2 GB at 50 bytes each, so state needs a TTL and lives in RocksDB rather than the JVM heap. The sketch below is the whole idea in one operator: deduplicate,

assign to an event-time window, advance the watermark, fire closed windows, divert late events.

```
from collections import defaultdict

WINDOW, DELAY = 60, 10       # 1-minute tumbling windows, 10 s lateness bound
counts = defaultdict(int)    # keyed state: (event_id, window start) -> count
seen, late, max_ts = set(), [], 0

def on_event(uuid, event_id, ts):
    global max_ts
    if uuid in seen:                            # duplicate delivery: drop
        return
    seen.add(uuid)
    start = ts - ts % WINDOW
    if start + WINDOW <= max_ts - DELAY:        # its window already fired
        late.append(uuid)
        return
    counts[(event_id, start)] += 1
    max_ts = max(max_ts, ts)
    watermark = max_ts - DELAY
    for key in sorted(k for k in counts if k[1] + WINDOW <= watermark):
        print(f"{key[0]} [{key[1]},{key[1] + WINDOW}) -> {counts.pop(key)}")

# b is retried, d is out of order but on time, e moves the watermark to 65
# so [0,60) fires, and f arrives after its window fired
for uuid, ts in [("a", 5), ("b", 58), ("c", 63), ("b", 58), ("d", 50),
                 ("e", 75), ("f", 30)]:
    on_event(uuid, "e1", ts)
print("late:", late)          # e1 [0,60) -> 3, then late: ['f']
```

In Flink the same pieces are `WatermarkStrategy.forBoundedOutOfOrderness`, `TumblingEventTimeWindows`, `allowedLateness` and `sideOutputLateData`; in Spark Structured Streaming, `withWatermark("event_time", "10 seconds")` plus `window(...)` in a `groupBy`.

> **Design question.** Fans in a stadium with poor signal send events up to five minutes late. How should Encore's live funnel treat them?
>
> Requirements: the organizer dashboard must move within seconds; the official funnel report must count every event once. Use event time with a small watermark delay (10 s) so the live view stays fresh, and route late events to a side output that lands in the lake. The nightly batch job recomputes the day's funnel from the complete Iceberg table and overwrites the gold table. Raising δ to five minutes would make the live view five minutes stale for everyone to fix a few percent of events. Trade-off: two code paths for one metric, so share the counting logic as one SQL definition. See also *50 System Designs* #23 (Top-K Trending Topics).

21.10 Exactly-once in streams

Machines crash mid-window, so a processor must restart without losing or doubling counts. Flink takes periodic **checkpoints**: the sources inject barriers into the stream, and each operator snapshots its state to S3 when the barrier passes (asynchronous barrier snapshotting, a variant of Chandy–Lamport). A checkpoint is a consistent cut: source offsets plus every operator's state. On failure, Flink restores the last checkpoint and rewinds the sources to its offsets, so each event affects state **exactly once** even though it may be *processed* twice. Output is the harder half. Either the sink joins the checkpoint (Flink's Kafka sink writes in a Kafka transaction and commits when the checkpoint completes, a two-phase commit), or the sink is **idempotent**: upsert keyed by `(event_id, window_start)` so a replayed result overwrites itself. Kafka Streams gets the same guarantee with `processing.guarantee=exactly_once_v2`, which writes outputs, state changelogs and consumer offsets in one Kafka transaction. Downstream readers must use `isolation.level=read_committed`. Anything outside the transaction, such as sending an e-mail or calling the payment provider, still needs an idempotency key (Chapter 18). The checkpoint interval is the replay bill: with 30-second checkpoints at the 58,000 events/s peak, a crash replays up to about 1.7 million events.

21.11 Lambda and Kappa architectures

The **Lambda architecture** (Nathan Marz) runs both paths on purpose: a batch layer recomputes accurate views from all history, a speed layer covers the last hours approximately, and a serving layer merges them. It is robust, but every metric is written twice, in two engines, and the two drift. The **Kappa architecture** (Jay Kreps, 2014) keeps one path: everything is a stream, the log is retained long enough to replay, and a fix is deployed by running the new job version from offset zero beside the old one and switching readers when it catches up.

Replay time is retained events divided by replay rate: 30 days of Encore's clickstream is 3 10^{10} events, and at an illustrative 10^{6} events/s the replay takes about 8.3 hours, which is acceptable for a bug fix. Kafka keeps only 7 days, so most teams land in between: Kappa for live metrics, with Kafka tiered storage or the Iceberg table as the replay source, and batch jobs over the lakehouse for reports, as in the late-event design question above.

21.12 ETL, ELT and change data capture

ETL (extract, transform, load) cleans data on the way in, in a separate engine, so the warehouse only ever sees curated tables; it suits strict schemas and expensive warehouse storage. **ELT** loads raw data first and transforms inside the warehouse or lakehouse with SQL, usually managed by **dbt** as versioned, tested models; it is the default now because storage is cheap and raw history lets you rebuild any model. AWS Glue (GCP Dataflow or Dataproc, Azure Data Factory) runs both styles.

Operational data reaches the lake by **change data capture**, not by nightly `SELECT *` dumps that hammer the primary. A CDC connector reads the database's replication log (the Postgres write-ahead log through a logical replication slot, the MySQL binlog) and emits one event per inserted, updated or deleted row, in commit order per key.

```
orders (Postgres) --WAL--> Debezium (Kafka Connect) --> topic orders.cdc
  {"op": "u", "before": {...}, "after": {"order_id": 9, "status": "PAID"}}
                                  |                       |
                    Flink: live dashboard      Iceberg MERGE (upsert by key)
```

Debezium on Kafka Connect, AWS Database Migration Service, Google Datastream and Azure Data Factory are the usual connectors; the transactional outbox of Chapter 14 is CDC applied to an events table. **Trade-off:** CDC is near real time and spares the primary, but it couples consumers to table schemas, so publish a cleaned topic as the contract and watch the replication slot, which retains WAL on the primary if the connector stalls.

21.13 Orchestration and data quality

Batch pipelines are graphs of jobs with dependencies, schedules and retries, like a kitchen's prep list: stock before sauce, sauce before service, and if the stock burns, redo the stock, not the whole menu. **Apache Airflow** describes the graph in Python; the scheduler creates one run per interval and retries failed tasks. The rule that makes this safe is **idempotent tasks**: each task overwrites exactly its own partition (`silver/2026-10-14`), so a retry or a backfill of last month produces the same result. Airflow runs as Amazon MWAA (GCP Cloud Composer; Azure Data Factory covers the same ground); Dagster, Prefect and AWS Step Functions are alternatives.

Data quality is a task in the graph, placed before anything publishes. Check freshness (did today's data arrive?), volume (row count within, say, 30 percent of the last seven days), schema, uniqueness of keys, nulls and accepted values, and fail the run instead of shipping a wrong dashboard. dbt tests, Great Expectations, Soda, AWS Glue Data Quality and Google Dataplex automate these; a *data contract* is the same checks agreed with the producing team and enforced at the schema registry.

```
from datetime import datetime
import duckdb
from airflow.sdk import dag, task                 # Airflow 3 Task SDK

LAKE = "/tmp/lake"                                # s3://encore-lake in prod

@dag(schedule="@daily", start_date=datetime(2026, 10, 1), catchup=False)
def clicks_daily():
    @task(retries=3)
    def build_silver(ds=None):                    # dedupe one day, rerunnable
        duckdb.sql(f"""COPY (SELECT DISTINCT ON (event_uuid) *
            FROM '{LAKE}/bronze/{ds}/*.parquet')
            TO '{LAKE}/silver/{ds}.parquet'""")

    @task
    def check_quality(ds=None):                   # fail loudly, before gold
        rows, dupes, nulls = duckdb.sql(f"""SELECT count(*),
            count(*) - count(DISTINCT event_uuid),
            count(*) FILTER (WHERE user_id IS NULL)
            FROM '{LAKE}/silver/{ds}.parquet'""").fetchone()
        assert rows > 0 and dupes == 0 and nulls == 0, (rows, dupes, nulls)

    @task
    def build_gold(ds=None):
        duckdb.sql(f"""COPY (SELECT event_id, action, count(*) AS events
            FROM '{LAKE}/silver/{ds}.parquet' GROUP BY ALL)
            TO '{LAKE}/gold/funnel_{ds}.parquet'""")

    build_silver() >> check_quality() >> build_gold()

clicks_daily()
```

DuckDB stands in for Spark or Athena so the DAG runs on a laptop (`airflow dags test clicks_daily 2026-10-14`); in production each task submits an EMR or Athena job and the files are Iceberg tables.

21.14 The live sales dashboard

> **Design question.** Design the live sales dashboard organizers watch during Encore's flash sale: tickets sold, revenue, active holds and the funnel, per event, fresh within two seconds.
>
> Requirements: freshness under 2 s; numbers must never go backwards on a replay; each organizer sees only their own events; the final figures must match finance. Load: 50,000 seats sell as about 20,000 orders (2.5 seats each, illustrative) in ten minutes, about 33 orders/s, while the clickstream peaks at 58,000 events/s. Design: the orders CDC topic is the source of truth for sales and revenue; the clickstream feeds the funnel. One Flink job keys both by `event_id`, deduplicates on `order_id` and `event_uuid`, keeps running totals in state, and every second writes **absolute totals**, not increments, to Redis, so a replay after a crash rewrites the same values. The dashboard API checks that the organizer owns the event and pushes updates over Server-Sent Events. 58,000 events/s in becomes about one Redis write per concert per second out. Every night, a batch job recomputes totals from the Iceberg tables and reconciles them with the payment provider's settlement file. Trade-off: two paths for one number, but the fast path is disposable and the batch path is the record. See also *50 System Designs* #28 (Ad Click Aggregator) and #25 (Ticket Booking).

```
HSET dash:e42 sold 31250 revenue_cents 296875000 holds 4120 as_of 10:03:15
PUBLISH dash:e42 updated
HGETALL dash:e42
```

Writing `HSET` with totals computed in Flink state is idempotent; writing `HINCRBY sold 1` per order is not, and a checkpoint replay would count the same orders twice. With an illustrative average price of $95, the 31,250 seats sold so far are $2,968,750, stored in cents as an integer. For per-organizer drill-downs over history (sales by hour, by city, by

price tier), a real-time OLAP store such as Apache Pinot, Apache Druid or ClickHouse ingests the same topics and answers in sub-second time.

21.15 Feature pipelines and embeddings for agents

The support agent is a data consumer like any other. It answers better with **features** about the customer ("3 orders this year, 1 refund, attended 2 shows at this venue") and with **embeddings** of policies, FAQs and event descriptions for retrieval. A feature pipeline computes features in batch from the lakehouse (the offline store, used for training and evaluation) and keeps the latest values in a low-latency online store (DynamoDB or Redis) that the agent's tools read in milliseconds; streaming updates from the CDC topic keep fresh features fresh. Two rules carry over from this chapter: build training sets with *point-in-time* joins, so no feature uses data from after the moment it describes, and compute online and offline features from one definition to avoid training-serving skew. SageMaker Feature Store (Vertex AI Feature Store, Azure Machine Learning managed feature store) and the open-source Feast implement this split. Embeddings are a batch job with an incremental tail: embed everything once, then re-embed only rows whose text changed, driven by CDC on the catalog. Encore's 300,000 events a year at about 500 tokens each are 150 million tokens, about $3 a year at an illustrative $0.02 per million tokens, so the cost is in the pipeline, not the model. When an agent writes SQL against the lakehouse, give it a read-only role on gold tables and a per-query scan cap (an Athena workgroup data limit, BigQuery's `maximum_bytes_billed`). *Designing Agentic Systems* covers retrieval and tool design in depth; Chapter 24 places these services in the architecture.

21.16 Numbers to remember

Figure	Value
Clickstream average / flash-sale peak	11,600 / 58,000 events/s (5x, 58 MB/s)
Raw JSON vs Parquet	1 TB vs 100 GB a day (illustrative 10x)
Lake size after a year	36.5 TB of Parquet
Day-scoped query, 3 of 30 columns	10 GB, $0.05 on Athena ($5/TB)
Spark scan split size	128 MB; 100 GB = 745 tasks
Kafka partitions for the clickstream	12 needed, 48 provisioned
Replay 30 days at 1M events/s	about 8.3 hours
Flash-sale orders	about 33 orders/s

21.17 Pitfalls

- Grouping by processing time -> counts shift with every delay or replay; window on event time with a watermark.
- Incrementing counters in the sink (`HINCRBY`) -> double counts after a restart; write absolute totals or upsert by key.
- Millions of small files from streaming writes -> slow planning and scans; buffer to large files and schedule compaction.
- Partitioning by a high-cardinality column such as `user_id` -> one tiny file per partition; partition by day, bucket by key.
- A nightly full dump of the orders database -> load on the primary and stale data; use CDC.
- Unbounded deduplication state -> memory grows until the job dies; set a state TTL.
- One hot concert key in a stream job -> one overloaded task; pre-aggregate per partition, then combine, as in Chapter 17.

21.18 Remember

- One log, two paths: a stream processor for seconds-fresh answers, a columnar lakehouse for complete ones.
- Size everything from events/s times bytes per event; the flash sale, not the average, sets the partition count.

- Columnar files plus partition pruning turn a 36.5 TB table into a 10 GB query.
- A table format (Iceberg, Delta, Hudi) is what makes files a table: atomic commits, snapshots, schema evolution.
- Event time, windows and watermarks trade latency against completeness; late data goes to the batch path.
- Exactly-once is checkpointed state plus a transactional or idempotent sink, never a property of the network.

Part V. Safe and reliable

Keeping a distributed system secure, available and understandable while it changes.

In this part:

- Security
- Reliability, observability and operations

22 Security

Every request that reaches Encore carries a claim: "I am fan 42, and I want to hold seat C-17." Security is the set of mechanisms that let each component check that claim, limit what a true claim can do, and leave a record when something goes wrong, while 1,000,000 fans and an unknown number of scalper bots push on the doors at 10:00. This chapter walks the whole path: threat modelling, identity, permissions, encryption in transit and at rest, secrets, networks, the classic web attacks in system terms, bots, tenants, audit, compliance and agents.

22.1 The picture

Think of a stadium on concert night. A fence with gates surrounds it (the network perimeter). At the gate a steward checks your ticket and face (authentication). The ticket says Block C, so a second steward at the stairs turns you away from the VIP box (authorization). Cash moves in sealed bags whose keys sit in a safe at head office (encryption and key management). Cameras record who opened which door (audit logs). And the stewards do not trust you merely because you are already inside the fence: every door checks again (zero trust).

```
fan / bot          edge                 inside the VPC
---------    ----------------    ------------------------------------
browser  --> CDN + WAF + DDoS --> API gateway --> seat inventory --> DB
app          bot checks,           authn (JWT),    authz (owner?    encrypted
             rate limits           scopes, quotas  tenant? role?)   at rest
                  |                     |               |               |
                  +------ TLS ---------+---- mTLS -----+---- TLS -----+
                                                       |
                              secrets manager, KMS  <---+--> audit log
```

Each arrow crosses a **trust boundary**: a line where data moves from a less trusted zone to a more trusted one. Security design is mostly deciding what gets checked at each boundary.

22.2 Threat modelling with STRIDE

A **threat model** answers four questions before any code exists: what are we building, what can go wrong, what will we do about it, and did we do a good job. Draw the data flow (clients, services, stores, external systems), mark the trust boundaries, then walk each element through **STRIDE**, Microsoft's six-letter checklist. Each letter is the negation of a property you want.

Threat	Property broken	Encore example	Main control
Spoofing	authenticity	login with stolen password	MFA, passkeys
Tampering	integrity	edit price in the hold request	server-side prices, signatures
Repudiation	non-repudiation	"I never requested that refund"	audit log
Information disclosure	confidentiality	read another fan's order	ownership checks, encryption
Denial of service	availability	bot flood at 10:00	CDN, WAF, waiting room
Elevation of privilege	authorization	fan calls an organizer API	RBAC, deny by default

To decide which threats deserve engineering time, put a rough price on each. The classic formula multiplies the cost of one incident by how often it happens per year:

$$\text{ALE} = \text{SLE} \times \text{ARO}$$

Here ALE is the annualised loss expectancy, SLE the single loss expectancy and ARO the annual rate of occurrence. With illustrative numbers, if account takeover costs Encore $200 per case in refunds and support, and happens 2,000 times a year, ALE is $400,000 a year. A control that costs $100,000 a year and removes most of it (MFA for risky logins) pays for itself; a control against a $50,000-a-year threat at the same price does not. The estimates are crude, and that is fine: their job is to rank the list.

22.3 Authentication: proving who you are

Authentication (authn) establishes *who* is calling; authorization (authz, next section) decides *what* they may do. Keep the two apart in code and in your head: most breaches are an authorization bug behind a perfectly good login.

22.3.1 Passwords and slow hashes

Never store a password, and never store a fast hash of it. Store the output of a deliberately slow, salted, memory-hard function. The **salt** (16 random bytes per user) makes identical passwords hash differently, so an attacker who steals the table must attack each user separately. The **cost** makes every guess expensive. The current OWASP guidance: Argon2id with at least 19 MiB of memory, 2 iterations and 1 lane; scrypt with $N = 2^{17}$, $r = 8$, $p = 1$; bcrypt with work factor 10 or more (bcrypt reads only the first 72 bytes); PBKDF2-HMAC-SHA256 with 600,000 iterations where FIPS compliance requires it.

The defender pays the cost once per login; the attacker pays it once per guess:

$$T_{\text{crack}} = \frac{U \times G}{R}$$

with U users, G guesses tried per user and R hashes per second. If Encore's 20 million password hashes leak and the attacker tries the 10,000 most common passwords against each, that is 2×10^{11} guesses. At an illustrative 10^{10} fast SHA-256 hashes per second on one GPU the job takes 20 seconds; at an illustrative 10^{4} memory-hard hashes per second it takes about 231 GPU-days. Same leak, very different morning.

The defender's side of the same equation matters during the flash sale. Cores needed equal logins per second times CPU seconds per hash. If 300,000 of the 1,000,000 fans log in between 09:55 and 10:05 (500 per second) and a hash is tuned to 50 ms (illustrative; measure yours), login needs 25 cores. If they all arrive in the first minute, it needs 250. The fix is product, not hardware: invite fans to log in to the waiting room from 09:30, and keep sessions long-lived so that 10:00 is a token check, not a password check.

```
from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError

# OWASP minimum for Argon2id: 19 MiB memory, 2 passes, 1 lane
ph = PasswordHasher(time_cost=2, memory_cost=19_456, parallelism=1)

stored = ph.hash("correct horse battery staple")
print(stored[:31])                # parameters and salt travel with the hash

def login(password: str, stored: str) -> bool:
    try:
        ph.verify(stored, password)
    except VerifyMismatchError:
        return False
    return True                   # if ph.check_needs_rehash: re-hash

print(login("correct horse battery staple", stored), login("hunter2", stored))
```

This prints `$argon2id$v=19$m=19456,t=2,p=1$` and `True False` (it needs `pip install argon2-cffi`). Storing the parameters inside the hash string lets you raise the cost later and upgrade each user at their next login.

22.3.2 Sessions or tokens

After login the server hands the client a credential so that it need not send the password again. There are two shapes.

- **Session**: a random opaque ID (128 bits or more) in a cookie marked `HttpOnly; Secure; SameSite=Lax`. The server looks it up in a session store (Redis, well under a millisecond in-region) on every request. Revocation is instant: delete the key. The cost is a lookup per request and a store that every service must reach.
- **Self-contained token**: a signed JSON Web Token (JWT) that carries the user ID, scopes and expiry. Any service with the public key verifies it locally in microseconds, with no shared store. The cost is revocation: a stolen token stays valid until it expires.

The usual answer is both: a short-lived access token (5 to 15 minutes) for APIs, plus a long-lived refresh token kept server-side and rotated on every use. If you must revoke access tokens early (logout, stolen phone), keep a denylist of token IDs (`jti`) until they expire. Its size is the revocation rate times the token lifetime:

$$N_{\text{deny}} = \lambda_{\text{revoke}} \times \text{TTL}$$

If 1 percent of Encore's 4 million daily users log out explicitly (40,000 a day) and tokens live 300 seconds, the denylist holds about $40,000 \times 300 / 86,400 \approx 139$ entries: a tiny Redis set that every gateway can check.

22.3.3 OAuth 2.0 and OpenID Connect

OAuth 2.0 is a protocol for *delegated authorization*: a client obtains an access token to call a resource server on a user's behalf, without ever seeing the user's password. **OpenID Connect (OIDC)** adds authentication on top: an **ID token** (a JWT with `iss`, `sub`, `aud`, `exp` and a `nonce`) that tells the client who logged in. "Sign in with Google" is OIDC.

The flow for browsers and mobile apps is the **authorization code flow with PKCE** (RFC 7636). The OAuth 2.0 Security Best Current Practice (RFC 9700, January 2025) deprecates the older implicit and password grants.

```
app                          auth server                    Encore API
 | 1 make code_verifier v;      |                              |
 |   send challenge S256(v) --->| 2 user logs in, consents     |
 |<--- 3 redirect with code ----|                              |
 | 4 POST /token code + v ----->| 5 checks S256(v) == challenge|
 |<--- 6 access + refresh + ID -|                              |
 | 7 GET /orders  Authorization: Bearer <access token> ------->|
 |                              |   8 verify signature, aud,   |
 |<---------------------------------- exp, scope; answer ----|
```

PKCE matters because the code travels through a redirect that other apps on the phone may observe; without the secret verifier, a stolen code is useless. For service-to-service calls with no user, the **client credentials** grant issues a token to the service itself. For smart TVs and command-line tools, the **device authorization grant** (RFC 8628) shows a short code to type on a phone.

22.3.4 JWT: structure and pitfalls

A JWT is three base64url parts joined by dots: `header.payload.signature`. The payload is signed, **not encrypted**: anyone holding the token can read it, so never put personal data you would not print on the ticket. This stdlib version shows every moving part (use a maintained library such as PyJWT in production):

```
import base64, hashlib, hmac, json, time

def b64(data: bytes) -> str:
    return base64.urlsafe_b64encode(data).rstrip(b"=").decode()

def unb64(text: str) -> bytes:
    return base64.urlsafe_b64decode(text + "=" * (-len(text) % 4))

KEY = b"demo-only-load-from-a-secret-manager"

def sign(claims: dict) -> str:
    head = b64(json.dumps({"alg": "HS256", "typ": "JWT"}).encode())
    body = b64(json.dumps(claims).encode())
    mac = hmac.new(KEY, f"{head}.{body}".encode(), hashlib.sha256)
    return f"{head}.{body}.{b64(mac.digest())}"

def verify(token: str, audience: str) -> dict:
    head, body, sig = token.split(".")
    if json.loads(unb64(head)).get("alg") != "HS256":   # allowlist, never
        raise ValueError("unexpected alg")              # trust the header
    mac = hmac.new(KEY, f"{head}.{body}".encode(), hashlib.sha256)
    if not hmac.compare_digest(mac.digest(), unb64(sig)):
        raise ValueError("bad signature")
    claims = json.loads(unb64(body))
    if claims["exp"] < time.time() or claims["aud"] != audience:
        raise ValueError("expired or wrong audience")
    return claims

now = int(time.time())
tok = sign({"iss": "https://auth.encore.example", "sub": "user-42",
            "aud": "encore-api", "scope": "holds:write orders:read",
            "iat": now, "exp": now + 300})
print(verify(tok, "encore-api")["sub"])                 # user-42
```

The pitfalls are all visible here. Accepting `alg: none`, or letting the header choose the algorithm (the "algorithm confusion" attack, where an RS256 public key is used as an HS256 secret), lets an attacker forge tokens: pin the algorithm on the server. Skipping the `aud` check lets a token minted for one service unlock another. Comparing signatures with `==` leaks timing; use a constant-time compare. Across many services, prefer asymmetric signatures (RS256 or ES256): the auth server keeps the private key, services fetch public keys from a JWKS endpoint and pick one by the `kid` header, which makes key rotation routine.

22.3.5 MFA and passkeys

Multi-factor authentication adds something you have to something you know. A **TOTP** code (RFC 6238) is 6 digits that change every 30 seconds; servers usually accept the neighbouring time steps too, so about $w = 3$ codes are valid at once. After k guesses the chance of success is

$$P = 1 - (1 - \frac{w}{10^6})^k \approx \frac{k\,w}{10^6}$$

which with a lockout after $k = 5$ attempts is 1.5×10^{-5}. That is why MFA endpoints need their own strict rate limit (Chapter 11). SMS codes are weaker (SIM swaps), and both TOTP and SMS can be phished by a fake login page that relays the code in real time.

Passkeys (FIDO2 and WebAuthn) remove the shared secret. At registration the device creates a key pair for Encore's domain and sends only the public key. At login the server sends a random challenge; the device signs it, together with the origin, after a fingerprint or PIN. The private key never leaves the device (or the platform's synced keychain), and a phishing site on a different origin gets a signature that Encore rejects. A leaked passkey table contains only public keys.

Where it runs: Amazon Cognito (Google Identity Platform, Microsoft Entra External ID), Auth0, Okta and Keycloak implement OIDC and MFA, and increasingly passkeys; API gateways (Amazon API Gateway, Envoy, Kong) verify JWTs at the edge so services receive only authenticated calls.

22.4 Authorization: deciding what you may do

Authorization is a function `allowed(principal, action, resource)` evaluated on every request, on the server, with **deny by default**. Three models build it:

- **RBAC** (role-based): users get roles, roles get permissions. Simple and auditable; it explodes when every exception becomes a new role.
- **ABAC** (attribute-based): a policy reads attributes of the user, the resource and the context ("support agents may refund up to $150, during business hours, for orders in their region").
- **ReBAC** (relationship-based): permissions follow a graph ("fan 42 owns order 9, which belongs to event 3, which organizer 7 runs"), as in Google's Zanzibar and its open-source descendants SpiceDB and OpenFGA.

```
ROLES = {"fan": {"order:read"}, "organizer": {"sales:read"},
         "support": {"order:read", "order:refund"}}

def allowed(user: dict, action: str, res: dict) -> bool:
    perms = set().union(*(ROLES[r] for r in user["roles"]))
    if action not in perms:                          # RBAC: deny by default
        return False
    if "fan" in user["roles"] and res["owner"] != user["id"]:
        return False                                 # ownership (no IDOR)
    if "organizer" in user["roles"] and res["tenant"] != user["tenant"]:
        return False                                 # tenant boundary
    if action == "order:refund":                     # ABAC: attributes
        return res["amount"] <= user["refund_limit"]
    return True

order = {"owner": "user-42", "tenant": "org-7", "amount": 180}
fan42 = {"id": "user-42", "roles": {"fan"}}
fan99 = {"id": "user-99", "roles": {"fan"}}
bot = {"id": "support-bot", "roles": {"support"}, "refund_limit": 150}
print(allowed(fan42, "order:read", order),         # True
      allowed(fan99, "order:read", order),         # False
      allowed(bot, "order:refund", order))         # False: over the limit
```

The **ownership check** is the line that matters most. `GET /orders/9` with a valid token for fan 99 must not return fan 42's order; skipping it is an insecure direct object reference (IDOR), the most common form of broken access control, which tops the OWASP Top 10 (2025 edition). Random IDs do not fix it; the check does.

A **confused deputy** is a privileged service tricked into using its own authority for a caller who lacks it. Encore's ticket-PDF worker can read every ticket in S3; if it accepts "render the ticket at path X" from a fan, it becomes the fan's deputy. The fix is to authorize with the *caller's* identity: pass the user's token (or a scoped token derived from it) along the chain, and have each service check the original principal. AWS guards its own deputies the same way, with the `aws:SourceArn` and `aws:SourceAccount` condition keys and the external ID on cross-account roles.

Where it runs: policy engines such as Open Policy Agent (Rego), Amazon Verified Permissions (Cedar), SpiceDB and OpenFGA keep rules out of handler code; cloud IAM (AWS IAM, Google Cloud IAM, Azure RBAC) applies the same ideas to machines.

22.5 TLS, mTLS and certificates

TLS gives a connection three things: secrecy (encryption), integrity (tampering is detected) and server identity (a **certificate** binds a domain name to a public key, signed by a certificate authority the client already trusts). TLS 1.3 completes in one round trip (Chapter 4); its 0-RTT mode saves that trip on resumption but its early data can be replayed, so allow it only for idempotent requests.

Mutual TLS (mTLS) makes the client present a certificate too. Between services it replaces "anything inside the VPC may call me" with "only the orders service may call me". A service mesh (Istio, Linkerd, Consul) issues each workload a short-lived certificate carrying an identity such as `spiffe://encore/ns/prod/sa/orders` (the SPIFFE standard) and rotates it automatically; the sidecar or proxy does the handshake, so application code stays unchanged.

Certificate lifetimes are shrinking by industry rule. Under CA/Browser Forum ballot SC-081, the maximum lifetime of a public TLS certificate fell from 398 to 200 days on 15 March 2026, falls to 100 days in March 2027 and to 47 days in March 2029. Manual renewal no longer works: use ACME (Let's Encrypt) or a managed service such as AWS Certificate Manager (Google Certificate Manager, Azure Key Vault certificates), which renews certificates attached to load balancers and CloudFront automatically. Terminate public TLS at the CDN or load balancer, and re-encrypt to the services behind it; "TLS stops at the edge" is a finding in every audit.

22.6 Encryption at rest and envelope encryption

Encryption at rest protects data on disks, snapshots and backups from someone who gets the storage but not the keys. The cipher is the easy part (AES-256-GCM); the keys are the hard part. A **key management service** (AWS KMS; Google Cloud KMS, Azure Key Vault) holds master keys inside hardware security modules, never exports them, logs every use and enforces who may use which key.

KMS is a network call with a quota, so it does not encrypt your data directly (its `Encrypt` call takes at most 4 KB). **Envelope encryption** splits the job: KMS generates a fresh **data key**, returns it in plaintext and wrapped (encrypted under the master key); you encrypt the data locally with the plaintext key, store the wrapped key next to the ciphertext and forget the plaintext key.

```
import os
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

kms = AESGCM(AESGCM.generate_key(bit_length=256))  # stands in for KMS

def seal(pdf: bytes, order_id: str) -> dict:
    dek = AESGCM.generate_key(bit_length=256)        # kms.generate_data_key
    kn, n = os.urandom(12), os.urandom(12)           # never reuse a nonce
    return {"key": kn + kms.encrypt(kn, dek, None),  # wrapped data key
            "nonce": n,
            "blob": AESGCM(dek).encrypt(n, pdf, order_id.encode())}

def unseal(rec: dict, order_id: str) -> bytes:
    k = rec["key"]
    dek = kms.decrypt(k[:12], k[12:], None)          # kms.decrypt
    return AESGCM(dek).decrypt(rec["nonce"], rec["blob"], order_id.encode())

rec = seal(b"%PDF ticket for ord-42", "ord-42")
print(unseal(rec, "ord-42")[:11])                    # b'%PDF ticket'
unseal(rec, "ord-43")      # raises InvalidTag: bound to its order ID
```

The order ID is passed as *associated data*: it is authenticated but not encrypted, so a ciphertext copied onto another order fails to decrypt (KMS calls the same idea an encryption context). Rotation is cheap: KMS keeps old master-key versions to unwrap old data keys, and re-wrapping a data key moves 32 bytes, not terabytes.

The numbers say where KMS belongs. The default shared quota for symmetric KMS operations is 10,000 requests per second per account and region (20,000 in some regions and 100,000 in us-east-1, us-west-2 and eu-west-1; adjustable). Encore's 50 million orders a year average 1.6 per second, and even 50,000 ticket PDFs written in ten minutes is 83 per second: no problem. A design that calls KMS on *every API request* (say, to decrypt a session at 200,000 seat-map reads per second) breaks the quota at once. Keep KMS off the hot path: cache data keys for minutes (the AWS Encryption SDK does this) and enable S3 Bucket Keys, which cut S3's KMS calls by reusing a bucket-level key.

Where it runs: S3 has encrypted every new object by default (SSE-S3) since January 2023; choose SSE-KMS when you need key policies and per-key audit. RDS, Aurora, DynamoDB and EBS encrypt with KMS keys; Cloud SQL, Spanner and Azure SQL offer customer-managed keys the same way. Encryption at rest does nothing against an attacker who steals the application's credentials, because the application can decrypt; that is what authorization and secrets management are for.

22.7 Secrets management

A secret is anything that grants access: database passwords, the payment provider's API key, JWT signing keys, webhook signing secrets. The rules: never in code, images or Git; fetched at runtime from a **secrets manager** (AWS Secrets Manager or SSM Parameter Store; Google Secret Manager, Azure Key Vault; HashiCorp Vault anywhere); readable only by the one workload that needs it; rotated automatically; every read logged.

Better still, remove the secret. Workloads get identities instead of keys: an IAM role per Kubernetes service account (EKS Pod Identity or IRSA; GKE Workload Identity; Azure workload identity), short-lived database credentials (RDS IAM authentication tokens last 15 minutes; Vault issues dynamic database users), and CI pipelines that exchange an OIDC token from GitHub Actions for temporary cloud credentials instead of storing access keys. A credential that expires in minutes is worth little when it leaks. Rotation of the secrets that remain must be zero-downtime: accept old and new values during an overlap window, then retire the old.

22.8 Network security and zero trust

The network is the outer wall, not the only one. Encore's VPC (VPC network in GCP, VNet in Azure) splits each availability zone into tiers, and **security groups** (stateful, allow-only rules attached to instances; GCP firewall rules, Azure network security groups) let each tier accept traffic only from the tier above, by referencing the other group rather than IP ranges. Network ACLs add stateless subnet-level rules, including denies.

```
internet -> CloudFront + WAF + Shield
              |
public subnet:  ALB (sg-alb: 443 from CloudFront prefix list)
              |
private subnet: services (sg-app: 8080 from sg-alb only)
              |                \--> VPC endpoints: S3, KMS, Secrets Manager
data subnet:    Aurora, Redis (sg-db: 5432/6379 from sg-app only)
                no route to the internet at all
```

Private endpoints (AWS PrivateLink and gateway endpoints for S3 and DynamoDB; Google Private Service Connect, Azure Private Link) keep calls to cloud APIs off the public internet, and an egress allowlist stops a compromised container from talking to anything but the payment provider. At the edge, a **WAF** (AWS WAF; Google Cloud Armor, Azure WAF) blocks known attack patterns and applies rate-based rules, and **DDoS protection** absorbs floods: AWS Shield Standard is automatic for all AWS customers at layers 3 and 4, Shield Advanced adds layer-7 response and cost protection, and the CDN's spare capacity soaks up volume (Chapter 10).

Zero trust (Google's BeyondCorp, NIST SP 800-207) drops the idea that inside the network means trusted. Every request carries a verified identity (user token plus workload mTLS identity), every service authorizes it, and access for employees goes through an identity-aware proxy instead of a VPN. The wall still helps; it is no longer the reason a call is allowed.

22.9 Web risks in system terms

The OWASP Top 10 (2025) ranks broken access control first, then security misconfiguration, software supply chain failures, cryptographic failures and injection. In system terms most of them are one mistake: **data from outside is treated as code or as authority**.

Injection happens when input is pasted into a language the server runs: SQL, shell, templates, LDAP, or an LLM prompt. Parameterised queries send the statement and the data separately, so input can never change the statement:

```
import sqlite3

db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE orders (id TEXT, user_id TEXT, total INT)")
db.executemany("INSERT INTO orders VALUES (?, ?, ?)",
               [("ord-1", "user-42", 180), ("ord-2", "user-99", 90)])
attack = "x' OR '1'='1' --"

bad = f"SELECT * FROM orders WHERE id = '{attack}' AND user_id = 'user-42'"
print(db.execute(bad).fetchall())                # every order leaks

good = "SELECT * FROM orders WHERE id = ? AND user_id = ?"
print(db.execute(good, (attack, "user-42")).fetchall())   # []
```

The `AND user_id = ?` is the ownership check from the authorization section, pushed into the query.

Server-side request forgery (SSRF), now filed under broken access control, turns a feature that fetches URLs (an organizer's "import venue image from URL", a webhook target, an agent's browse tool) into a proxy into your private network. The prize is often the cloud metadata endpoint at `169.254.169.254`, which hands out the instance's credentials; the 2019 Capital One breach began that way. Resolve the host, refuse anything that is not a public address, and connect to the address you checked so DNS cannot change between check and use:

```
import ipaddress, socket
from urllib.parse import urlsplit

def safe_target(url: str) -> str:
    """Return a vetted IP for an outbound fetch, or raise."""
    parts = urlsplit(url)
    if parts.scheme != "https" or not parts.hostname:
        raise ValueError("https URLs only")
    infos = socket.getaddrinfo(parts.hostname, parts.port or 443,
                               proto=socket.IPPROTO_TCP)
    for *_, sockaddr in infos:
        ip = ipaddress.ip_address(sockaddr[0])
        if not ip.is_global:                     # private, loopback,
            raise ValueError(f"blocked {ip}")    # link-local (IMDS) ...
    return infos[0][4][0]   # connect to this IP, so DNS cannot change it

safe_target("https://169.254.169.254/latest/meta-data/")  # ValueError
```

Add the platform defences too: require IMDSv2 on EC2 (a session token fetched with a `PUT`, which a naive SSRF cannot send), and run URL fetchers in a separate network zone with egress through a proxy. Other risks follow the same pattern: cross-site scripting is injection into the browser (escape output, set a Content-Security-Policy), CSRF is a forged request riding a cookie (`SameSite` cookies plus a CSRF token), and supply-chain failures are untrusted code you chose to run (pin dependencies, scan images, sign builds).

22.10 Bots and scalpers in the flash sale

A scalper is a fan who is a thousand times faster and has a thousand accounts. Blocking floods is DDoS defence; stopping scalpers is about making speed and account count worthless. Per-IP limits barely help: a real fan refreshes the seat map about 0.2 times a second (200,000 reads/s over 1,000,000 fans), 60 requests per 5 minutes, but 100 fans behind one carrier-grade NAT share an address, so a safe per-IP limit is about 6,000 per 5 minutes, and a bot under it still makes 20 requests a second. Limits must key on the waiting-room token and the account, not the address.

Under first come, first served, the fastest clients win, so the bots' share of seats is set by their account count times the per-account cap:

$$\text{seats}_{\text{bot}} = \min(A_{\text{bot}} \times c,\ S) \qquad \text{versus} \qquad \text{seats}_{\text{bot}} \approx S \times \frac{I_{\text{bot}}}{I_{\text{total}}}$$

The second form holds when the waiting room gives everyone who arrives before 10:00 a *random* position and each verified identity (one phone number, one payment card) is capped at c seats. With 5,000 bot accounts and $c = 4$, first come first served hands bots 20,000 of 50,000 seats (40 percent); a random queue over 1,005,000 identities gives them

about 249. Verification then shrinks I_{bot} further.

> **Design question.** How would you stop scalpers taking Encore's 50,000-seat show without blocking a million real fans?
>
> Requirements: fairness, no false blocks at 30,000 hold attempts per second, cost of evasion higher than resale profit. Layers: (1) pre-registration days ahead with phone and card checks, scored by account age and history ("verified fan"); (2) edge filtering with AWS WAF Bot Control, JavaScript challenges and CAPTCHA only for suspicious scores, so most fans see nothing; (3) a waiting room that randomises arrival order before 10:00 and issues a signed, single-use pass bound to the account, checked at the edge (Chapter 10); (4) caps of 4 seats per account, card and phone, enforced atomically in Redis at hold time; (5) after the sale, named tickets and controlled resale, and cancelling orders that share cards or devices. Trade-off: each layer adds friction and some false positives, so apply the strict layers only to high-demand events. See also *50 System Designs* #25 (Ticket Booking).

22.11 Multi-tenancy isolation

Encore's organizers are tenants: each sees its own events, sales and buyers, and none may see a rival's. Isolation comes in three strengths: **pool** (shared tables with a `tenant_id` column), **bridge** (a schema or database per tenant) and **silo** (a separate account or cluster, for the largest tenants or regulated data). Whatever the model, take the tenant from the verified token, never from the request body, and let the database enforce it as a second wall. Postgres row-level security does that (it does not exist in SQLite, so this one is Postgres only):

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE orders FORCE ROW LEVEL SECURITY;   -- applies to the owner too
CREATE POLICY tenant_isolation ON orders
  USING (organizer_id = current_setting('app.organizer_id')::bigint);
-- per request, inside the transaction:
SET LOCAL app.organizer_id = '7';
```

Isolate keys and limits as well: a KMS key per silo tenant lets one organizer's data be revoked alone, and per-tenant rate limits stop one organizer's export job from starving the rest (Chapter 11).

22.12 Audit logs

An audit log answers "who did what, to which resource, when, from where, and was it allowed" for every security-relevant action: logins, permission changes, refunds, data exports, key use. It must be append-only and tamper-evident. Write it to storage the writers cannot delete (S3 Object Lock in compliance mode, a separate security account) and chain the records: each record stores the hash of the previous one, h_i = SHA256(r_i h_{i-1}), so editing record i breaks every hash after it. CloudTrail's log file integrity validation uses signed SHA-256 digest files in the same spirit (Google Cloud Audit Logs and Azure Monitor activity logs are the equivalents). Keep audit logs separate from debug logs, never log secrets or full card data, and alert on the rare events (a new admin, a key policy change) rather than reading them after the fact.

22.13 Compliance: PCI DSS and GDPR

PCI DSS (version 4.0.1) applies to every system that stores, processes or transmits card numbers, and to every system connected to those. The cheapest control is not to touch cards at all. Encore's checkout embeds the payment provider's hosted fields (or redirects to its page); the card goes from the browser straight to the provider, which returns a **token** that Encore stores and charges with an idempotency key. No Encore service ever sees a card number, so the assessment shrinks from the whole platform to the shortest self-assessment questionnaire (SAQ A). The page that embeds the form still matters: PCI DSS v4 expects merchants to protect it from tampered scripts.

GDPR gives EU fans rights over their personal data: access, correction, portability and erasure ("right to be forgotten", Article 17), answered within one month (extendable by two more for complex cases); breaches go to the regulator within 72 hours; fines reach 4 percent of global annual turnover or 20 million euros, whichever is higher. System

consequences: collect less, know where every field lives, keep EU data in the EU region, and make deletion a workflow rather than a script.

> **Design question.** A fan asks Encore to delete their account. Their data sits in Postgres, Redis, Kafka topics, the lakehouse, support transcripts and 35 days of backups. How do you comply?
>
> Keep a data map keyed by user ID. Publish one `user.erasure_requested` event; each owning service deletes or pseudonymises its rows and reports back, and an orchestrator (Chapter 14) tracks completion against the one-month deadline. Orders that tax law requires you to keep stay, stripped of name and e-mail. Kafka: short retention, or tombstones on compacted topics. Lakehouse: row-level deletes in Iceberg or Delta, then expire old snapshots (Chapter 21). Backups: **crypto-shredding**, encrypting each user's personal fields with a per-user data key and deleting the key, so every copy in every backup becomes unreadable at once. Trade-off: a key lookup per read of personal data, which a key cache absorbs.

22.14 Security for agents

Encore's support agent reads customer messages and order data, then calls tools that look up orders and request refunds. That makes it a confused deputy waiting to happen: **prompt injection**, ranked first in the OWASP Top 10 for LLM Applications, lets text the model reads (a customer's message, or an e-mail quoted into the conversation) override its instructions. The dangerous combination is private data, untrusted input and a way to act or send data out; the support agent has all three. Treat the model as an untrusted user with good manners, and put every control in the tool layer, which is ordinary code.

- **Scoped credentials**: tools run with a token derived from the customer's session through OAuth token exchange (RFC 8693), whose `sub` is the customer and whose `act` claim names the agent, so the agent can only see what the customer could.
- **Tool permissions**: a fixed allowlist per agent; read tools by default; write tools validated server-side (ownership, refund at most the order total).
- **Human approval** above a threshold, as a workflow step; idempotency keys on every write; every tool call in the audit log.

> **Design question.** How would you let the support agent issue refunds without letting a prompt injection drain money?
>
> The refund tool ignores any customer or order ID chosen by the model beyond what the session token allows: it loads the order, checks that the token's `sub` owns it, and caps the amount at the order total. Refunds up to $150 (illustrative) go straight to the payments service with the idempotency key `refund:{order_id}:{conversation_id}` and a running total capped at the order price, so a retried call returns the first answer and a looping agent cannot refund twice; larger ones create an approval task for a human. The payments service, not the agent, holds the provider's API key. Trade-off: some legitimate large refunds wait for a person. See also *50 System Designs* #24 (Payment System); Chapter 24 and *Designing Agentic Systems* cover the agent runtime in depth.

22.15 Numbers to remember

Figure	Value
Argon2id minimum (OWASP)	19 MiB, 2 iterations, 1 lane
bcrypt	work factor 10+, first 72 bytes only
Access token lifetime	5 to 15 minutes
TOTP	6 digits, 30-second step
Public TLS certificate maximum	200 days (2026), 100 (2027), 47 (2029)
KMS `Encrypt` payload	4 KB; use envelope encryption

Figure	Value
KMS symmetric quota	10,000/s default per account and region
GDPR	erasure in 1 month; breach notice in 72 hours
Encore flash-sale logins	500/s spread, 5,000/s if in one minute

22.16 Pitfalls

- Checking the role but not the owner -> add the ownership or tenant predicate to every query.
- Trusting the JWT header's `alg`, or skipping `aud` and `exp` -> pin algorithms and validate every claim.
- Long-lived tokens with no revocation -> short access tokens, rotating refresh tokens, a small `jti` denylist.
- Calling KMS on every request -> envelope encryption with cached data keys and S3 Bucket Keys.
- Secrets in environment files and CI variables -> workload identities, a secrets manager, short-lived credentials.
- Per-IP limits as bot defence -> limits per account and waiting-room pass, plus randomised queue order.
- Guardrails written only in the prompt -> authorization, caps and approvals in the tool code.

22.17 Remember

- Authenticate once at the edge, authorize everywhere, deny by default, and take identity and tenant from the verified token.
- Slow, salted password hashes; passkeys and MFA where takeover costs money; short-lived tokens.
- TLS on every hop, mTLS between services, envelope encryption with keys in KMS, no long-lived secrets.
- Most web attacks are outside data treated as code or authority: parameterise, validate destinations, check ownership.
- Shrink scope instead of defending it: tokenise cards with the provider, crypto-shred personal data, keep agents' power in tools.

23 Reliability, observability and operations

Every component in Encore will fail: a disk, a zone, a deploy, the payment provider, an engineer's command at 3 a.m. Reliability is not the absence of failure but the habit of surviving it: design so that one failure stays small, see it quickly, repair it fast, change the system without breaking it, and learn from each incident. This chapter turns the SLOs of Chapter 1 into daily practice: the availability arithmetic of a call chain, the ways systems fail, redundancy and degradation, logs, metrics and traces, alerts that page on burn rate, safe deployments and migrations, disaster recovery, chaos experiments, incident response, and the load test that decides whether 10:00 on flash-sale day is a celebration or a postmortem.

23.1 The picture

Think of an airline. Planes have two engines because one can fail (**redundancy**), and a plane that loses one engine is certified to fly to the nearest airport (**graceful degradation**). The cockpit shows a few instruments that summarize the whole aircraft (**metrics**), the flight recorder keeps every detail for later (**logs and traces**), and an alarm sounds on symptoms that matter to the flight, not on every sensor twitch (**alerting**). New aircraft fly test flights before passengers board (**canaries**); crews rehearse emergencies in simulators (**game days**); every incident ends in a report that fixes the system, not the pilot (**blameless postmortems**).

```
design for failure --> observe ---------> alert on burn --> respond
(redundancy, limits)   (logs, metrics,     rate              (incident
        ^               traces)                               roles)
        |                                                       |
change safely  <-----------------  learn (postmortem)  <--------+
(canary, flags, expand-contract)
```

The loop never stops. Most outages are caused by change (a deploy, a configuration push, a migration), so the same loop that recovers from failure must also govern how Encore changes.

23.2 Availability of a call chain

Chapter 1 gave the two composition rules: a serial chain multiplies availabilities, a parallel set of redundant copies fails only when all copies fail. Two more ideas make those rules usable in design: availability is set by how fast you repair, and redundant copies are only as independent as the things they share. **Repair time matters as much as failure rate.** A component that fails on average once every MTBF (mean time between failures) hours and takes MTTR (mean time to repair) hours to come back is up for the fraction

$$A = \frac{\text{MTBF}}{\text{MTBF}+\text{MTTR}} \qquad A_{\text{redundant}} = (1 - p_{\text{common}})\left(1 - \prod_i (1 - A_i)\right)$$

The second equation adds a **common-mode** failure probability p_{common}: the chance that something shared by all copies (a configuration push, a DNS record, a bad deploy, an expired certificate) takes them all down at once. It is a ceiling: no number of replicas lifts availability above $1 - p_{\text{common}}$.

Worked example (illustrative): a seat-inventory node fails about once a month (MTBF 720 hours). If a person must notice and replace it in 20 minutes, availability is 99.954%; with the automated failover of Chapter 16 in 22 seconds, it is 99.9992%. The hardware did not change; the repair did. Three zones at 99.9% each would reach nine nines if independent, but a 0.05% chance of a bad configuration reaching all three caps the set at 99.95%. Three design rules follow:

- **Shorten MTTR before buying MTBF**: automatic failover, fast rollback and good dashboards are cheaper than better hardware.
- **Attack the common modes**: roll out configuration zone by zone and region by region, so a bad change hits one copy first.

- **Shorten the chain**: Encore's checkout chain of 99.84% (Chapter 1) improves more by taking ticket issuing and e-mail out of the synchronous path (Chapter 14) than by adding replicas to any one box.

Large providers organize this as **cells**: independent, full copies of the stack, each serving a slice of customers, with no shared database and changes rolled out one cell at a time (Chapter 7). A failure or bad deploy then has a blast radius of one cell. AWS also asks for **static stability**: a system should keep working through a failure without needing to change anything (no new instances to launch, no control-plane calls), because the control plane is the part most likely to be busy or impaired during a large event.

23.3 Error budgets in practice

An error budget is a monthly allowance of unhappiness, like a household budget for repairs: spend it on risky changes while it lasts, and stop spending when it is gone. Chapter 1 defined it as (1 − SLO) of the valid events. In practice two choices shape it: what you count, and what you do when it runs out.

Count requests or count minutes. A time-based SLO counts bad minutes; a request-based SLO counts bad requests. A 99.95% SLO allows 21.6 bad minutes in 30 days (Chapter 1 has the table of nines). Requests are better for Encore, because they weight each minute by how many fans it hurt. The two views diverge sharply on sale day. The month's whole request budget at 99.95% is about 1.94 million bad requests (3.9 billion requests in 30 days); if seat inventory fails for one minute while it serves the 3,000 holds/s the waiting room lets through, that is 180,000 bad requests, 9% of the month; if the front door fails and all 30,000 raw attempts per second error, it is 1.8 million, 93%. By the clock, the same minute is under 5% of the 21.6-minute budget.

$$\text{budget share of an incident} = \frac{\lambda_{\text{incident}} \times t \times f_{\text{bad}}}{(1 - \text{SLO}) \times N_{\text{window}}}$$

Write the policy down before you need it. An error budget only changes behaviour when product and engineering agree the consequences in advance, in a short document next to the SLO definition. Encore's policy for the hold journey: below 25% of budget remaining, two reviewers per deploy and no Friday deploys; at zero, a feature freeze in which only reliability and security fixes ship; any single incident that spends more than 20% gets a postmortem with at least one high-priority action item; budget spent by a dependency's outage triggers no freeze but a review of the fallback and the vendor's SLA, because it should change the design (a fallback, a second provider), not the release cadence. The 20% rule follows the example policy in Google's SRE workbook. Tools that track the remaining budget include CloudWatch Application Signals (Google Cloud Monitoring SLOs, Azure Monitor SLIs), Grafana SLO, Datadog and Nobl9; OpenSLO is a vendor-neutral YAML format for the definitions.

> **Design question.** What SLO should Encore set for the flash sale itself?
>
> A monthly request-based SLO is the wrong tool: one bad minute at the door spends 93% of the month, and nothing in the month says whether the sale went well. Give the sale its own SLI per event: of the fans admitted by the waiting room, the fraction whose hold attempt got a definite answer (held, or "seat taken") in under 1 s, target 99% over the sale window. Count "seat taken" and waiting-room queueing as good events: they are correct answers, not errors. Count timeouts, 5xx and holds lost after confirmation as bad. Publish a second SLI for fairness (admission order versus arrival order). The trade-off: per-event SLOs need per-event dashboards and a review after each big sale, which is the point. See also *50 System Designs* #25 (Ticket Booking).

23.4 How systems fail

A crashed server is the easy case: it stops answering, the health check fails, the load balancer removes it. The hard failures are the ones that look healthy from inside. A shop whose till is broken but whose lights are on loses more customers than a shop with a closed sign, because people keep walking in.

Failure mode	What it looks like	Detect	Contain
Crash (fail-stop)	process or node gone	health checks, heartbeats	redundancy, failover

Failure mode	What it looks like	Detect	Contain
Slow (fail-slow)	answers in 5 s, not 50 ms	latency SLIs, p99	timeouts, outlier ejection
Partition	nodes alive, cannot talk	quorum loss, errors	majority quorums, fencing
Gray failure	probes pass, users fail	client-side SLIs	probe real user paths
Dependency outage	provider 5xx or timeouts	per-dependency metrics	breaker, fallback, queue
Retry storm	load far above traffic	attempts per request	retry budgets, one layer
Thundering herd	spike at recovery or TTL	synchronized peaks	jitter, coalescing, ramp
Bad change	errors start at a deploy	deploy markers	canary, fast rollback
Exhaustion	disk, FDs, pool, quota full	saturation metrics	limits, alerts on trend

Slow is worse than dead. By Little's law (Chapter 1), a dependency whose latency grows from 20 ms to 2 s holds 100 times as many threads and connections at the same request rate, so a slow payment provider exhausts the orders service's pool and takes down unrelated endpoints. Timeouts, bulkheads and circuit breakers (Chapter 11) turn slow failures into fast ones, which the rest of the system already knows how to handle. Envoy's outlier detection and the Kubernetes readiness probe (Chapter 8) remove slow instances as well as dead ones.

Gray failure is the gap between what the system thinks and what users see: a node whose health endpoint returns 200 while its disk is failing, a zone whose network drops 2% of packets, a cache that serves stale data. The fix is to measure from the user's side (load balancer logs, client telemetry, synthetic probes that run a real hold and cancel it, such as CloudWatch Synthetics canaries) and to compare a node with its peers, not with a fixed threshold.

Metastable failure is the worst pattern: a trigger (a brief database slowdown) pushes the system into overload, and a sustaining effect (retries, timed-out requests that the server still processes, a cold cache) keeps it there after the trigger is gone. The system stays down until load is cut below normal: shed traffic, drain queues, close the waiting room. Bronson and colleagues named the pattern in 2021, and a 2022 study of public incident reports found it behind major outages at several large cloud providers. Retry budgets (Chapter 11), bounded queues, server-side deadlines that drop requests the client has abandoned, and warm caches (Chapter 9) remove the sustaining effects. **Thundering herds** come from synchronization: a million fans' apps reconnecting the second Encore recovers, cache entries written together expiring together, cron jobs at the top of the hour. Add jitter to every timer, coalesce identical requests, and bring capacity back gradually (the waiting room reopens admission in steps rather than at once).

23.5 Redundancy across zones and regions

A building with three fire exits must be safe with one exit blocked, so each of the other two must be wide enough for half the crowd. Redundancy in the cloud works the same way: the spare capacity must already exist, in a place that does not fail together with the primary. The units of failure, from small to large, are the process, the node, the rack, the **availability zone** (one or more data centres with independent power and cooling, a few milliseconds from its neighbours), and the **region** (a group of zones, typically hundreds or thousands of kilometres from the next). AWS builds new regions with at least three zones (Google Cloud and Azure use the same three-zone pattern). Spread every tier across three zones; spread across regions what must survive a regional event.

Static stability across zones. If a zone is lost, the remaining $N - 1$ zones must carry the peak without launching anything, because during a zone outage the control plane is busy and capacity in the healthy zones is in demand by every other customer:

$$\text{capacity per zone} = \frac{\text{peak}}{N-1} \qquad \text{total provisioned} = \frac{N}{N-1} \times \text{peak}$$

Two zones cost twice the peak (50% utilization at peak); three zones cost 1.5 times (67%); four, 1.33 times. That is why three is the usual answer: much cheaper than two, and the third zone also gives quorum systems their majority (Chapter 18). Seat inventory is load-tested at 6,000 holds/s; an illustrative pod handles 1,000 holds/s and runs at 75% of that, so the peak needs $6,000/750 = 8$ pods. Surviving a zone loss needs 16 pods in two zones (8 per zone) and 12 in

three (4 per zone, 3,000/750). In Kubernetes, a topology spread constraint keeps the pods even across zones and a PodDisruptionBudget stops voluntary evictions (node upgrades) from taking too many at once (Chapter 15); the data tier keeps synchronous copies in the other zones (Chapter 16).

Across regions the same arithmetic applies with bigger numbers: if Frankfurt must absorb Virginia's traffic, it needs the capacity, the quotas and the warm caches ready, or Encore must accept a degraded mode while it scales. Cross-region redundancy also needs every dependency to be regional: a region whose login depends on a global identity service in the other region has not gained much. Chapter 16 decided how each data set crosses regions; the section on disaster recovery below decides what runs where.

> **Design question.** A zone fails at 10:01, one minute into the flash sale. What must already be true for fans not to notice?
>
> Capacity: seat inventory runs 4 pods per zone in three zones (12 pods for an 8-pod peak), so two zones carry the tested 6,000 holds/s alone; the gateway and waiting room likewise. Data: each inventory partition (`event_id#section`) has its Postgres leader in one zone and a synchronous standby in another, so failover loses no confirmed hold and takes about 22 s (Chapter 16); Redis keeps the hold TTLs with a replica in another zone. Behaviour: the waiting room pauses admission during the failover, so the 66,000 holds (3,000/s for 22 s) that failed do not retry at once. The cost is 50% more inventory pods than the peak needs, every day: scaling up is the step most likely to fail mid-event. See also *50 System Designs* #34 (E-Commerce Checkout and Inventory).

23.6 Graceful degradation and fallbacks

A restaurant whose oven breaks does not close; it serves the cold menu. Graceful degradation is deciding in advance which parts of Encore may break, what each falls back to, and who flips the switch. The rule is to protect the **critical path** (see seats, hold, pay, receive the ticket) by sacrificing everything else first.

Feature	Failure	Fallback
Seat map	inventory slow	CDN copy up to 5 s stale, re-checked at hold
Search	cluster down	browse by city and date from the catalog cache
Recommendations	model service slow	last good list, then top sellers
Payment	provider outage	second provider, or keep the hold and retry
Tickets by e-mail	provider quota	queue; ticket visible in the app at once
Support agent	LLM provider down	smaller model, then a human queue
Organizer dashboard	lake lagging	show "data as of 09:58"

Three mechanisms implement the table. **Fallbacks in code**: every call to a non-critical dependency has a timeout and a cheaper answer. **Stale-if-error at the edge**: `Cache-Control: stale-if-error=300` lets CloudFront or Fastly keep serving a cached page while the origin fails (Chapter 10). **Kill switches and brownouts**: feature flags that turn off expensive features under load (recommendations, live seat-count animations) so their capacity goes to holds. A fallback wrapper runs the call under a deadline (`asyncio.wait_for` with 150 ms), saves each good answer, and on a timeout or error returns the last good answer marked `stale`, or a default marked `default`, instead of making the fan wait 2 s. Export that marker as a metric: a fallback that serves silently for a week is an outage nobody saw.

Fallbacks have costs. Each one is a code path that runs rarely, so it rots; exercise it on purpose (in tests, and with chaos experiments below). A fallback must never be heavier than the primary: a fallback that queries the main database when the cache fails can take down the database too. And not everything can degrade: a hold cannot be "approximately" confirmed, so on the critical path the fallback is a clear error with `Retry-After`, never a guess.

LLM services are dependencies like any other, with long and variable latency and per-token quotas. Encore's support agent calls a model gateway with a timeout, falls back from a large model to a smaller one or a second provider, and finally hands the conversation to a human queue with the transcript attached; a refund tool call that timed out is retried with the same idempotency key, never re-issued blind (Chapter 24; *Designing Agentic Systems* covers agent fallbacks in depth).

23.7 Observability: logs, metrics and traces

A doctor reads a patient three ways: vital signs on a monitor (cheap, continuous, a few numbers), the notes in the chart (detailed events), and a contrast dye that shows the path blood takes through the body. Those are **metrics**, **logs** and **traces**. Monitoring answers questions you thought of in advance ("is the error rate above 1%?"); observability is being able to answer new ones ("why do only Android fans in Frankfurt fail to hold seats in section C?") from the data you already collect.

- **Metrics** are numbers aggregated over time: counters (requests), gauges (queue depth), histograms (latency buckets, from which p99 is computed). Cheap to store and to alert on, but every label multiplies the number of time series. A latency histogram with 20 routes, 5 status classes, 12 pods and 12 buckets is 14,400 series; adding a `user_id` label with 20 million users makes it billions. Put high-cardinality detail in traces and logs, never in metric labels.
- **Logs** are timestamped events. Write them as structured JSON with fixed keys (`trace_id`, `event_id`, `outcome`), never as free text, so they can be filtered and joined; keep secrets and card data out (Chapter 22).
- **Traces** follow one request across services. Each hop is a **span** with a start, a duration, attributes and a parent; the W3C `traceparent` header (`00-<trace id>-<span id>-<flags>`) carries the context between services. Traces show where the 180 ms went, which no single service's metrics can.

Three checklists tell you what to measure. **RED** for every request-driven service: Rate, Errors, Duration. **USE** for every resource (CPU, memory, disk, connection pool, queue): Utilization, Saturation, Errors. Google's four **golden signals** combine both: latency, traffic, errors, saturation. Measure latency of successful and failed requests separately, since fast errors hide slow successes.

OpenTelemetry (OTel) is the vendor-neutral standard for all three signals: APIs and SDKs per language, semantic conventions for attribute names, the OTLP wire protocol, and the Collector, which receives, samples, batches and forwards telemetry to any backend. The snippet records a trace with a child span, a latency histogram and a JSON log line carrying the trace ID (it uses in-memory exporters to run offline; production uses the OTLP exporter):

```
import json, logging, time
from opentelemetry import metrics, trace
from opentelemetry.sdk.metrics import MeterProvider
from opentelemetry.sdk.metrics.export import InMemoryMetricReader
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import SimpleSpanProcessor
from opentelemetry.sdk.trace.export.in_memory_span_exporter import (
    InMemorySpanExporter)

spans = InMemorySpanExporter()               # production: OTLP exporter
tp = TracerProvider()
tp.add_span_processor(SimpleSpanProcessor(spans))
trace.set_tracer_provider(tp)
reader = InMemoryMetricReader()
metrics.set_meter_provider(MeterProvider(metric_readers=[reader]))

tracer = trace.get_tracer("seat-inventory")
latency = metrics.get_meter("seat-inventory").create_histogram("hold.ms")
logging.basicConfig(level=logging.INFO, format="%(message)s")

def hold(event_id: str, seat: str) -> str:
    start = time.perf_counter()
    with tracer.start_as_current_span("POST /holds") as span:
        span.set_attribute("encore.event_id", event_id)
        with tracer.start_as_current_span("redis SET NX"):
            outcome = "held"                 # the real work goes here
        tid = f"{span.get_span_context().trace_id:032x}"
        logging.info(json.dumps({"seat": seat, "outcome": outcome,
                                 "trace_id": tid}))
    ms = (time.perf_counter() - start) * 1000
    latency.record(ms, {"route": "/holds", "outcome": outcome})
    return outcome

hold("ev-123", "A12")
print([s.name for s in spans.get_finished_spans()])
m = reader.get_metrics_data().resource_metrics[0].scope_metrics[0].metrics[0]
print(m.name, m.data.data_points[0].count)
```

It prints the JSON log line with a 32-hex-digit trace ID, `['redis SET NX', 'POST /holds']` (children finish first) and `hold.ms 1`. Trace volume needs a budget too: at Encore's average 1,500 requests/s, with an illustrative 10 spans of 500 bytes per request, full tracing is 7.5 MB/s, about 650 GB a day. **Head sampling** keeps a fixed fraction chosen at the first hop (10% gives 65 GB a day); **tail sampling** in the Collector waits for the whole trace and keeps all errors and slow requests plus a sample of the rest, which is what you want during an incident.

Where it runs: Prometheus and Grafana (with Loki for logs and Tempo or Jaeger for traces), or managed: CloudWatch and X-Ray with the AWS Distro for OpenTelemetry (Google Cloud Monitoring, Logging and Trace; Azure Monitor and Application Insights), Amazon Managed Service for Prometheus, Datadog, Honeycomb. For the support agent, the OTel GenAI semantic conventions name model calls and their token counts (`gen_ai.request.model`, `gen_ai.usage.input_tokens`), so a slow answer can be traced through retrieval, model and tool spans like any other request.

23.8 Alerting on symptoms and burn rates

A smoke alarm that rings whenever someone makes toast gets its battery removed. Pages must be rare, urgent and actionable, so page on **symptoms users feel** (holds failing, checkout slow) and send **causes** (CPU at 90%, one pod restarting, disk at 70%) to dashboards and tickets. A cause that hurts users will show up as a symptom; a cause that does not hurt users should not wake anyone.

The best symptom alert asks one question: at the current error rate, how soon is the error budget gone? With burn rate b (Chapter 1) sustained over an alert window w in an SLO period T:

$$\text{budget consumed} = \frac{b \times w}{T} \qquad \text{error-ratio threshold} = b \times (1 - \text{SLO})$$

Google's SRE workbook recommends paging when 2% of a 30-day budget burns in 1 hour (b = 14.4) or 5% in 6 hours (b = 6), and opening a ticket when 10% burns in 3 days (b = 1). Each long window is paired with a short one (5 minutes for the 1-hour window, 30 minutes for 6 hours) and both must exceed the threshold, so the alert also stops soon after the problem does. For Encore's 99.95% hold SLO, the fast-burn threshold is an error ratio of $14.4 \times 0.0005 = 0.72\%$; a total outage crosses it in about 26 seconds of the hour, and at that rate the whole budget would last 50 hours.

```yaml
# hold-slo.rules.yaml: SLO 99.95%, so the error-ratio budget is 0.0005
groups:
  - name: hold-slo
    rules:
      - record: slo:hold_errors:ratio_rate5m
        expr: |
          sum(rate(holds_total{outcome="error"}[5m]))
            / sum(rate(holds_total[5m]))
      # ... the same for 30m, 1h and 6h windows
      - alert: HoldBudgetFastBurn        # 2% of the budget in 1 hour
        expr: |
          slo:hold_errors:ratio_rate1h > (14.4 * 0.0005)
            and slo:hold_errors:ratio_rate5m > (14.4 * 0.0005)
        labels: {severity: page}
      # HoldBudgetSlowBurn: 6h and 30m windows above (6 * 0.0005)
```

`promtool check rules` accepts the file. CloudWatch composite alarms, Google Cloud Monitoring SLO burn-rate alerts and Azure Monitor alerts express the same rule on managed metrics. Route pages through an on-call tool (PagerDuty, incident.io) with a runbook link in every alert. Count pages per on-call shift; Google's SRE book suggests no more than two incidents per 12-hour shift, so that each one gets a proper follow-up.

23.9 Deployments: rolling, blue-green, canary and flags

Miners once carried a canary into the pit: if the air turned bad, the bird showed it before the miners felt it. A canary release sends a small share of real traffic to the new version and watches it before the rest of the users are exposed. Because most incidents start with a change, the deployment method is a reliability decision.

- **Rolling**: replace instances a few at a time (Kubernetes' default, Chapter 15). Cheap, but a bad version reaches everyone at the pace of the rollout, and both versions serve at once.
- **Blue-green**: run the new version as a full second environment and switch the load balancer or DNS. Rollback is one switch back; the price is double capacity during the switch, and a shared database that both versions must understand.
- **Canary**: route 1%, then 5%, 25%, 100%, comparing the canary's error rate and latency with the old version's at each step, and roll back automatically when they diverge.
- **Feature flags**: deploy the code dark and turn the feature on separately, for 1% of fans, for one organizer, or for staff. Deploy and release become two decisions, and a kill switch turns the feature off in seconds without a deploy.

The numbers of a canary. The canary must see enough traffic to reveal a regression. At 6,000 requests/s, a 5% canary for 10 minutes serves 180,000 requests; if the version doubles the error rate from 0.1% to 0.2%, the canary shows about 360 errors where 180 are expected, about 13 standard deviations ($\sqrt{180} \approx 13.4$). At 1% the counts are 36 and 72, still 6 standard deviations, while exposing five times fewer fans. Rare errors and low-traffic endpoints need longer bake times, not bigger canaries. The blast radius of a bad change is roughly the exposed fraction times the time to roll back times the request rate, so small canaries and fast rollbacks shrink it in both dimensions.

Flags must bucket users deterministically, so a fan does not flicker between versions and the fans enabled at 1% stay enabled at 5%:

```python
import hashlib

def bucket(flag: str, user_id: str) -> float:
    digest = hashlib.sha256(f"{flag}:{user_id}".encode()).digest()
    return int.from_bytes(digest[:8], "big") / 2**64   # uniform in [0, 1)

def enabled(flag: str, user_id: str, percent: float) -> bool:
    return bucket(flag, user_id) < percent / 100

users = [f"u{i}" for i in range(100_000)]
early = {u for u in users if enabled("bitmap-seat-map", u, 1)}
for pct in (1, 5, 25):
    on = {u for u in users if enabled("bitmap-seat-map", u, pct)}
    print(f"{pct:>2}%: {len(on):>6} users, early ones kept: {early <= on}")
```

It prints about 1,000, 5,000 and 25,000 users with `early ones kept: True` at each step. Hashing the flag name with the user ID gives each flag an independent sample, so the same 1% of fans are not the guinea pigs for every experiment.

Where it runs: Argo Rollouts and Flagger automate Kubernetes canaries against Prometheus queries; AWS CodeDeploy shifts traffic for ECS and Lambda (for example `CodeDeployDefault.LambdaCanary10Percent5Minutes`), Cloud Run and Cloud Deploy split traffic between revisions (Azure Container Apps revisions, App Service deployment slots); flags come from AWS AppConfig, LaunchDarkly, Unleash or Flagsmith, ideally behind the CNCF OpenFeature API (*50 System Designs* #49 (Feature Flag and Config Service)). Treat configuration exactly like code: versioned, reviewed, canaried region by region. And freeze non-urgent changes for the hours around a flash sale: the safest deploy at 09:55 is none.

23.10 Safe schema changes: expand and contract

You cannot swap a bridge while traffic crosses it, but you can build the new bridge beside it, move the traffic, and then demolish the old one. During every deploy, old and new code run side by side against one database, so every schema step must work with both. **Expand and contract** (Chapter 12) splits one breaking change into compatible steps, each deployed and verified separately. Here Encore moves order amounts from a floating-point column to integer cents:

```sql
-- expand: a new nullable column; old code ignores it
ALTER TABLE orders ADD COLUMN amount_cents INTEGER;
-- release 1 writes both columns; backfill old rows in small batches
UPDATE orders SET amount_cents = CAST(ROUND(amount * 100) AS INTEGER)
 WHERE id IN (SELECT id FROM orders
              WHERE amount_cents IS NULL LIMIT 1000);
-- release 2 reads cents, release 3 stops writing amount; weeks later:
ALTER TABLE orders DROP COLUMN amount;
```

Run in SQLite 3.53 (which supports `DROP COLUMN` since 3.35), it converts 19.99 to 1999 and `0.1 + 0.2` to 30. In Postgres, adding a nullable column is a metadata-only change; build indexes with `CREATE INDEX CONCURRENTLY` and set a short `lock_timeout`, so a migration waiting for a lock does not queue every query behind it. Every step can be rolled back except the last, which is why the contract step waits until no running version, cached job or replica still reads the old shape. Tools such as Flyway, Liquibase, Alembic and gh-ost (for MySQL) run the steps; the discipline is yours.

23.11 Disaster recovery

Replication keeps a second house furnished and lived in; a backup is the photo album in a fireproof safe. A fire in one house is survived by the second house, but a flood of bad writes reaches both houses within a second; only the album lets you go back to yesterday (Chapter 16). Disaster recovery (DR) plans for both: losing a region, and losing data to a bug, a bad migration, an attacker or a human.

Two targets set the plan. The **RPO** (recovery point objective) is how much recent data you may lose; the **RTO** (recovery time objective) is how long you may be down. Chapter 16 derived both for a failover; the strategy chooses how much of the second region is already running:

Strategy	Second region runs	RPO / RTO (AWS guidance)	Cost
Backup and restore	nothing; backups copied there	hours	lowest

Strategy	Second region runs	RPO / RTO (AWS guidance)	Cost
Pilot light	data replicas; app servers off	tens of minutes	low
Warm standby	everything, scaled down	minutes	medium
Active-active	everything, full size, serving	near zero	highest

Backups. Keep at least three copies on two kinds of storage with one off-site (the 3-2-1 rule), in a separate account, and immutable (S3 Object Lock, AWS Backup Vault Lock), so that the credentials that can delete production cannot delete the backups. Point-in-time recovery replays logs onto a snapshot: Aurora and RDS restore to any point in a retention window of up to 35 days, and DynamoDB PITR keeps 1 to 35 days and restores to a new table. Restore to a side cluster, never over the original, and copy back only the damaged rows:

```
aws rds restore-db-cluster-to-point-in-time \
  --source-db-cluster-identifier orders \
  --db-cluster-identifier orders-before-bad-migration \
  --restore-to-time 2026-10-14T09:58:00Z
```

For Aurora this restores the cluster only; add an instance with `create-db-instance`. A backup that has never been restored is a hope, not a backup. Restore one every week automatically, check row counts and checksums, and record how long it took: that duration is the real RTO of a data disaster. Failover itself should use data-plane operations that work when the control plane is struggling: Amazon Application Recovery Controller routing controls or Route 53 health checks flip traffic (Google Cloud DNS routing policies, Azure Traffic Manager), whereas creating resources and changing DNS records in a crisis are control-plane calls.

> **Design question.** Virginia goes dark at 10:01 during the flash sale. What does Encore lose, and how long until fans can buy again?
>
> Encore runs warm standby for writes and active-active for reads: seat maps, catalog and pages keep serving from Frankfurt and the CDN. Each event's seats have a home region, so the stadium event's inventory leader is in Virginia with an asynchronous Frankfurt standby; orders sit in Aurora Global Database, whose secondary typically lags under a second and can be promoted in under a minute. RPO: under a second of holds and orders. RTO target: about 5 minutes, dominated by the decision to fail over, so make it a rehearsed runbook behind one routing-control switch. Meanwhile the waiting room keeps fans queued instead of failing them; after promotion, holds re-validate and orders reconcile against the payment provider's records by idempotency key. Frankfurt runs, not launches, its 12 inventory pods. See also *50 System Designs* #25 (Ticket Booking).

23.12 Chaos engineering and game days

Fire drills find the locked exit before the fire does. **Chaos engineering** injects failures on purpose, in a controlled way, to test the hypothesis that the system keeps its steady state (holds succeed, p99 stays under 500 ms) when a part fails. Start in staging with a small blast radius and an abort condition tied to the SLO; move to production once the experiments pass, because only production has production's traffic and configuration. This one makes half of the payment adapter's pods 2 s slower for 10 minutes, to prove that the orders service's timeouts, breaker and pool limits fail payments fast and keep holds working:

```
apiVersion: chaos-mesh.org/v1alpha1
kind: NetworkChaos
metadata: {name: slow-payment-adapter, namespace: encore-staging}
spec:
  action: delay
  mode: fixed-percent
  value: "50"                      # half of the adapter pods
  selector:
    namespaces: [encore-staging]
    labelSelectors: {app: payment-adapter}
  delay: {latency: "2s", jitter: "500ms", correlation: "50"}
  duration: "10m"
```

AWS Fault Injection Service (Azure Chaos Studio; on Google Cloud, Chaos Mesh or LitmusChaos on GKE) stops instances, injects API errors, throttles or partitions networks and runs a whole zone-impairment scenario, with CloudWatch alarms as stop conditions. Netflix's Chaos Monkey started the practice by terminating random production instances. A **game day** is a rehearsed exercise with people: fail Virginia's seat inventory over to Frankfurt in staging with the on-call team, time each step against the RTO, and fix the runbook where people hesitated.

23.13 Incidents and postmortems

When something breaks, a hospital emergency room does not let everyone treat the patient at once: one person leads, one treats, one talks to the family, one writes it down. Incident response borrows the same roles: the **incident commander** coordinates and decides, the **operations lead** changes the system, the **communications lead** updates the status page and support, a scribe keeps the timeline. Declare incidents early and by severity (SEV1: holds or checkout failing for many fans; SEV3: the organizer dashboard is late), and **mitigate before you diagnose**: roll back the last deploy, fail over, shed load, turn off the flag. Root cause can wait until fans can buy again.

$$\text{MTTR} = t_{\text{detect}} + t_{\text{engage}} + t_{\text{mitigate}}$$

Burn-rate alerts shrink detection to minutes; an on-call rotation with a 5-minute acknowledgement target shrinks engagement; rollbacks, flags and failover switches shrink mitigation. The **postmortem** follows within days, and it is **blameless**: it asks how the system let a reasonable person make the mistake, not who made it, so people keep reporting near misses. Keep postmortems searchable and review them across teams; the same three causes (change, capacity, dependency) recur, and the action items are the reliability roadmap.

```
Postmortem: holds failed 10:02-10:09, flash sale ev-stadium (SEV1)
Impact     : 31% of holds failed for 7 min; 20% of month's budget
Timeline   : 10:02 burn-rate page | 10:04 IC declared | 10:07 flag off
Causes     : seat-map query moved to the primary; deployed 09:58 in freeze
Actions    : [P0] enforce freeze in CI   [P1] seat map to replicas
             [P1] canary analysis on hold SLI   owner + due date each
```

23.14 Capacity planning and load tests for the flash sale

A stadium does not find out on match day how many people its exits can clear; it measures. Capacity planning starts from the demand forecast, divides by what one unit can do at a safe utilization, and adds the zone spare:

$$\text{units} = \frac{\text{peak load}}{\text{capacity per unit} \times u_{\text{target}}} \times \frac{N}{N-1}$$

For Encore's sale, the demand at the door is 30,000 hold attempts per second; nothing behind the door is sized for that. The waiting room admits about 1,000 fans per second, each making about 3 hold attempts, so seat inventory runs at 3,000 holds/s and is load-tested at 6,000 (twice the run rate); the back of a 1,000,000-fan queue waits about 1,000,000 / 1,000 = 1,000 s, about 17 minutes. Inventory then needs $\lceil 6{,}000/750 \rceil = 8$ pods plus the zone spare, 12 in all. Seat-map reads (200,000/s) are a CDN problem: a 1-bit-per-seat bitmap cached for a second is about 10 Gbit/s from the edge (Chapter 10). A load test must reproduce the shape, not only the size: the ramp before 10:00, the step at 10:00, ten minutes at the plateau. This Locust file drives inventory to the tested 6,000 holds/s (run it distributed, for example with `locust --processes -1` on several machines):

```python
import random                  # locustfile.py: admitted fans holding seats
from locust import HttpUser, LoadTestShape, constant_throughput, task

class Fan(HttpUser):
    wait_time = constant_throughput(1)         # at most 1 hold/s per user

    @task
    def hold(self):
        seat = f"{random.choice('ABCDEFGH')}{random.randint(1, 6250)}"
        self.client.post("/holds", json={"event_id": "ev-stadium",
                                         "seat": seat}, name="hold")

class SaleShape(LoadTestShape):                # (until s, users, spawn/s)
    stages = [(120, 3_000, 100), (180, 6_000, 1_000), (780, 6_000, 1_000)]

    def tick(self):
        t = self.get_run_time()
        return next(((u, r) for end, u, r in self.stages if t < end), None)
```

Measure throughput, p99 and errors per stage, and watch the USE metrics of every tier to find which saturates first. Three traps: Locust's users wait for each response (a **closed** model), so a slow server receives less load and the test flatters it; use `constant_throughput` and check the achieved rate, or an **open** arrival-rate generator such as k6's `constant-arrival-rate` executor. Test with production-sized data (a cold cache and a 50,000-seat event behave differently from ten seats). And include the dependencies: the payment provider's sandbox has its own rate limits, so stub it at its contracted capacity. Repeat the test before every major sale; Distributed Load Testing on AWS, Azure Load Testing and Google Cloud's documented Locust-on-GKE pattern host the generators.

23.15 Numbers to remember

Figure	Value (Encore figures illustrative)
Availability from repair	MTBF / (MTBF + MTTR)
99.95% budget, 30 days	21.6 minutes, or 0.05% of requests
Page / ticket burn rates	14.4 (1 h + 5 min), 6 (6 h + 30 min) / 1 (3 d + 6 h)
Zone spare, N zones	N / (N - 1): 1.5x for three zones
Encore seat inventory	3,000 holds/s run, 6,000 tested, 12 pods
Waiting room	about 1,000 fans/s; 17 min for the back of the queue
Canary at 1%	36,000 requests in 10 min at 6,000/s
Aurora Global Database	lag typically under 1 s; promote in under 1 min
On-call load	at most 2 incidents per 12-hour shift

23.16 Pitfalls

- Paging on causes (CPU, one pod) -> page on SLO burn rates; send causes to dashboards.
- Multiplying nines of copies that share a config, DNS or deploy -> include the common mode, roll changes out one zone, cell and region at a time.
- Health checks that test the process, not the user path -> synthetic transactions and client-side SLIs.
- Fallbacks that are never exercised, or heavier than the primary -> test them with chaos experiments; keep them cheaper than the call they replace.
- Metric labels with user or order IDs -> keep cardinality in traces and logs.
- Treating replicas as backups -> immutable, cross-account, regularly restored point-in-time backups.
- Breaking schema changes in one deploy -> expand, migrate, contract over several releases.
- Closed-model load tests that slow down with the server -> open arrival-rate tests with production-sized data.

23.17 Remember

- Availability is set by repair time and by what copies share: shorten MTTR, attack common modes, shorten the synchronous chain.
- Error budgets turn SLOs into decisions; count requests, write the policy first, and give the flash sale its own SLI.
- Page on burn rate over a long and a short window; everything else is a ticket or a dashboard.
- Most incidents start with a change: canary it, flag it, migrate by expand and contract, and roll back first.
- Redundancy must be statically stable: spare zone capacity and data-plane failover already in place.
- Rehearse: restore backups, run chaos experiments and game days, and load test the sale's exact shape.

Part VI. Putting it together

External systems and AI agents in the architecture, one system built end to end in code and cloud, the hard problems each kind of service raises and how to solve them, and a method for the design interview.

In this part:

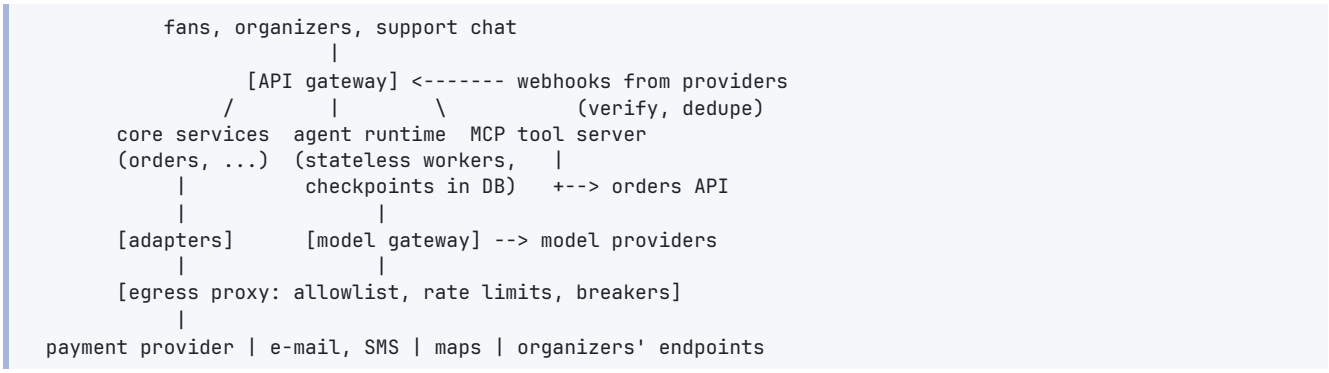
- External systems, tools and AI agents
- Encore end to end: from requirements to code and cloud
- Hard problems by service, and how to solve them
- The system design interview

24 External systems, tools and AI agents

Encore does not move money, deliver e-mail, draw maps or run language models. A payment provider, an e-mail and SMS vendor, a maps API and one or two model providers do, and a support agent built on a model now looks up orders and requests refunds on a fan's behalf. Each of these is a service Encore calls but cannot deploy, scale, fix or see inside, and the agent is a client that improvises. This chapter answers one question: where do external systems, tools and AI agents sit in a distributed design, and which ordinary mechanisms (idempotency keys, queues, limits, breakers, workflows, authorization) make them safe? The picture is a theatre that hires outside contractors and one very fast, very literal new assistant.

24.1 The picture

A theatre sells tickets but leaves card payments to the bank, posters to the printer and deliveries to the courier. Every contractor has a counter with opening hours, a daily limit and a form in its own language; the theatre keeps one clerk per contractor who speaks that language and files a copy of every form. When the bank phones back to confirm a payment, the clerk checks the caller is really the bank. Then the theatre hires a new assistant who answers customers at superhuman speed but takes instructions too literally: the assistant may fill in a refund form, but the form goes through the same office, large refunds need the manager's signature, and the assistant never holds the key to the safe.



Read it from the bottom. Everything below the egress proxy belongs to someone else and fails on its own schedule. Adapters translate their language into Encore's; the egress proxy enforces how much and how fast Encore may call them; the gateway is the only way in, for fans, for providers' webhooks and for the agent's tools. The agent runtime is one more service: it calls models through a gateway of their own and reaches data only through tools that are ordinary, authorized APIs. *Designing Agentic Systems* builds the agent itself; this chapter places it in the system.

24.2 External systems are dependencies you do not control

An external system brings four things into your design, and each needs a mechanism on your side:

It brings	Example	Your mechanism
A contract you do not own	provider renames a status	adapter, contract tests
Failures you cannot fix	30 s of 503s, a slow region	timeouts, breakers, fallbacks
Limits and a bill	100 e-mails/s, per-call fees	outbound rate limits, budgets
Calls back into you	payment webhooks	verify, deduplicate, reconcile

The deepest of these is the **ambiguous outcome**. A call to the payment provider that times out has three possible results, not two: the charge failed, the charge succeeded and the reply was lost, or the charge is still in progress. Treating a timeout as "failed" and charging again is how systems bill customers twice. Every design in this chapter keeps the third state, *unknown*, explicit and resolves it later with an idempotent retry or a lookup.

Dependencies also cap your availability. Serial dependencies multiply, and redundant ones fail only together:

$$A_{\text{serial}} = \prod_i A_i \qquad A_{\text{either of two}} = 1 - (1 - A_1)(1 - A_2)$$

With Encore's checkout path at 99.95 percent and a payment provider at 99.9 percent (both illustrative), checkout is available $0.9995 \times 0.999 = 99.85$ percent of the time, about 13 hours of failed checkouts a year, and most of them are the provider's. A second, independent provider behind a failover would lift the provider layer to $1 - 0.001^2 = 99.9999$ percent and checkout to about 99.95 percent (4.4 hours). The price is real: saved cards are tokenised in each provider's vault, so a fan's card may exist at only one of them, and two providers mean two ledgers to reconcile. Many marketplaces keep one provider and degrade gracefully instead: when the breaker opens, Encore extends a fan's 10-minute seat hold by a few minutes and shows "payment is slow, your seats are safe".

Keep every external system behind an **adapter**, the anti-corruption layer of Chapter 6: the provider's JSON, status names and error codes stop at one module, and the rest of Encore sees its own small vocabulary. The adapter is also where *unknown* is born, so it must never round it to success or failure:

```
from enum import Enum


class Outcome(Enum):
    CAPTURED, DECLINED, UNKNOWN = "captured", "declined", "unknown"


STATUS_MAP = {"succeeded": Outcome.CAPTURED, "canceled": Outcome.DECLINED,
              "requires_payment_method": Outcome.DECLINED}


def translate(raw: dict | None) -> Outcome:
    """Anti-corruption layer: provider JSON in, Encore's terms out."""
    if raw is None:                          # the call timed out
        return Outcome.UNKNOWN
    return STATUS_MAP.get(raw.get("status"), Outcome.UNKNOWN)


print(translate({"status": "succeeded"}).name,
      translate({"status": "processing"}).name, translate(None).name)
```

It prints `CAPTURED UNKNOWN UNKNOWN`: a status the adapter has never seen (here `processing`, which a Stripe-like provider uses while a payment settles) is unknown, not declined. Swapping providers, or adding a second, touches the adapter and its contract tests, which run against the provider's sandbox in CI.

24.3 The payment provider: keys, webhooks and reconciliation

Picture a cheque with a serial number. If the bank receives the same cheque twice, it pays once, because it remembers the number. An **idempotency key** is that serial number on an API call, and it turns the ambiguous outcome into a safe retry.

The charge runs in three steps, none of them inside a database transaction that is held open across the network (Chapter 12):

1. **Record intent.** Insert a `payments` row for the order, status `pending`, with the key `pay:<order_id>`. One order, one key, so any number of retries can only ever mean one charge.
2. **Call the provider** with the header `Idempotency-Key: pay:<order_id>`, a timeout of a few seconds, and the amount. A timeout or a 5xx leaves the row at `unknown`; a retry with the same key returns the first result instead of charging again (Stripe keeps keys for at least 24 hours, and rejects a reused key sent with different parameters).
3. **Record the outcome** from the response, from the webhook, or from a sweeper that looks up every `pending` or `unknown` payment older than a minute and retries it with its original key.

The **webhook** confirms what the response may not have: the provider POSTs `payment.succeeded` seconds or minutes later, signed with a shared secret. Chapter 5 shows signature and timestamp verification; the receiver's other half is applying events safely. Webhooks arrive **at least once and in any order**, so deduplicate by event ID and let a status only move forward:

```
import sqlite3

RANK = {"pending": 0, "processing": 1, "succeeded": 2, "refunded": 3}
db = sqlite3.connect(":memory:")
db.executescript("""
CREATE TABLE seen_events (event_id TEXT PRIMARY KEY);
CREATE TABLE payments (order_id TEXT PRIMARY KEY, status TEXT, rank INT);
INSERT INTO payments VALUES ('o_77', 'pending', 0);""")


def on_event(event_id: str, order_id: str, status: str) -> str:
    with db:                                    # one transaction
        new = db.execute("INSERT OR IGNORE INTO seen_events VALUES (?)",
                         (event_id,)).rowcount
        if not new:
            return "duplicate"
        moved = db.execute(
            "UPDATE payments SET status = ?, rank = ? "
            "WHERE order_id = ? AND rank < ?",
            (status, RANK[status], order_id, RANK[status])).rowcount
        return "applied" if moved else "stale"


print(on_event("evt_2", "o_77", "succeeded"))   # arrives first
print(on_event("evt_1", "o_77", "processing"))  # older, arrives late
print(on_event("evt_2", "o_77", "succeeded"))   # provider retry
print(db.execute("SELECT status FROM payments").fetchone())
```

It prints `applied`, `stale`, `duplicate` and `('succeeded',)`: the late `processing` event cannot drag a paid order backwards, and the retry is a no-op. On Postgres the same shape is `INSERT ... ON CONFLICT DO NOTHING` and a conditional `UPDATE`. In production the handler does only this (verify, record, return 200 within a second or two) and publishes `OrderPaid` through the outbox (Chapter 14); ticket rendering happens downstream.

Reconciliation is the safety net under both. Webhooks can be lost, endpoints can be down longer than the provider retries (Stripe retries for up to three days), and bugs happen on both sides. Once a day the provider publishes a settlement report of what it actually charged; Encore compares it with its own ledger:

```
SELECT COALESCE(l.order_id, s.order_id) AS order_id,
       CASE WHEN l.order_id IS NULL THEN 'charged, not in ledger'
            WHEN s.order_id IS NULL THEN 'in ledger, not charged'
            ELSE 'amount differs' END AS problem
FROM ledger l FULL OUTER JOIN settlement s ON l.order_id = s.order_id
WHERE l.amount_cents IS NOT s.amount_cents
ORDER BY 1;
```

Run in SQLite 3.39 or later on three sample rows per side, it lists `o_2` (amount differs), `o_3` (in the ledger, never charged) and `o_4` (charged, but Encore never recorded it). Postgres spells the null-safe comparison `IS DISTINCT FROM`. Each class has an owner: a charge without an order is refunded or turned into tickets, a ledger entry without a charge is a bug to page on. The numbers justify the machinery: at about 137,000 orders a day, losing even 0.1 percent of webhooks (illustrative) would leave 137 paying fans a day without tickets if the sweeper and the daily reconciliation did not exist.

> **Design question.** At 10:01 in the flash sale, the charge call for an order times out. The fan's seat hold expires at 10:09. What does Encore do?
>
> The payment is *unknown*, not failed. Encore tells the fan "confirming your payment", keeps the hold, and retries with the same key `pay:<order_id>` under exponential backoff with jitter; the provider either returns the original charge or performs it once. A webhook may settle it first, and the monotonic update makes the order of arrival irrelevant. If the outcome is still unknown when the hold is about to expire, Encore extends that one hold rather than releasing seats a fan may have paid for, and the sweeper keeps polling the provider. If the provider finally

> reports “declined”, the hold is released and the fan asked to retry. Trade-off: a few seats stay held a little longer during a provider incident, in exchange for never selling a paid seat twice. See also *50 System Designs* #24 (Payment System).

24.4 E-mail, SMS, maps and other third-party APIs

Not every vendor sits on the critical path, and the first design decision is which ones do. A vendor the fan waits for needs a breaker and a fallback; a vendor that works after the sale needs a queue.

Vendor	Fan waits for it?	When it fails
Payment provider	yes, at checkout	extend holds, retry, reconcile
E-mail	no, after the sale	queue, back off, second vendor
SMS, push	no	push first, SMS for critical only
Maps, geocoding	no, event page	cached result, plain address
Model provider	support chat only	second provider, human handoff

E-mail is asynchronous twice over. Amazon SES (Azure Communication Services Email; Google Cloud has no first-party sending service, so GCP shops use a vendor such as SendGrid) accepts a message in milliseconds, but *accepted* is not *delivered*: bounces, complaints and deliveries come back later as events (SES publishes them to SNS or EventBridge), and an address that bounces goes on a suppression list. Quotas are per account and per region: a new SES account starts in a sandbox limited to 200 messages a day at 1 per second, and production limits are granted on request. Send tickets as a short-lived signed link to the 50 KB PDF in S3 rather than as an attachment: the e-mail stays small and a lost ticket can be reissued without resending anything.

SMS and push are priced and policed differently. Each SMS costs money and carriers limit throughput per sender number and require sender registration in many countries, so Encore sends push notifications (APNs and Firebase Cloud Messaging) first and SMS only for messages a fan must not miss, such as a cancelled show. AWS End User Messaging SMS (Azure Communication Services SMS) or vendors such as Twilio carry them.

Maps and other lookups are cache candidates. A venue’s coordinates do not change, so Encore geocodes an address once when the organizer creates the venue and stores the result, within the provider’s terms (some restrict how long results may be kept). The event page then needs the maps vendor only for an interactive map, which degrades to a plain address and a static image served by the CDN.

Vendors rarely deduplicate for you, so the sender does. A queue redelivers a message whose worker died, and without a marker every redelivery is another e-mail. A short claim before the send and a long marker after it skip almost all of them:

```
SET sent:o_77:tickets sending NX EX 300   -> OK   (claim, then send)
SET sent:o_77:tickets done XX EX 604800   -> OK   (after the send)
SET sent:o_77:tickets sending NX EX 300   -> nil  (redelivery: skip)
```

A worker that dies before sending leaves a claim that expires in 5 minutes, and the redelivered message sends. Only a worker that dies between the send and `done` causes a duplicate, because the send and the marker live in two systems and no marker can close that window. For e-mail that is acceptable: a duplicate confirmation is cheaper than a lost ticket. For money it is not, which is why the payment provider gets an idempotency key it enforces on *its* side.

24.5 Vendor limits, quotas and circuit breakers

A vendor’s limit is a property of your *account*, not of your worker. Encore’s e-mail quota of 100 messages per second (illustrative) is shared by every worker in both regions, so a limit enforced per worker overshoots as soon as autoscaling adds a pod. Enforce it once, centrally: a token bucket per vendor in Redis (Chapter 11), set at about 90 percent of the quota to leave headroom, and every 429 from the vendor honoured through its `Retry-After`. The in-flight calls that rate needs follow from Little’s law:

$$L = \lambda \times W = 100\ \text{s}^{-1} \times 0.1\ \text{s} = 10\ \text{calls in flight}$$

That number sizes the **bulkhead**: a connection pool of about twice L for the e-mail vendor, a separate one for SMS, another for the payment provider, so a vendor that slows to 10 seconds per call exhausts its own pool and not the threads every other vendor needs. A **circuit breaker** per vendor stops calling one that is failing (open after, say, half of the last 20 calls failed; one probe after 30 seconds), which protects both sides: retries against a dead vendor waste money and can trip its abuse controls. Both live naturally in an **egress proxy** through which all outbound calls leave, Envoy here, with one cluster per vendor:

```
clusters:
- name: email_vendor            # one cluster, one bulkhead per vendor
  connect_timeout: 1s
  circuit_breakers:
    thresholds:
    - max_connections: 20       # about 2x Little's law (10 in flight)
      max_pending_requests: 50  # beyond this, fail fast with 503
      max_requests: 20
      max_retries: 2            # retries in flight, cluster-wide
```

Envoy's "circuit breakers" are concurrency caps, a bulkhead in this chapter's terms; the open-and-probe breaker sits in the client library or the worker. The proxy is also the single place that holds vendor credentials, allowlists destination hosts (AWS Network Firewall domain lists or a Squid or Envoy forward proxy; Cloud NGFW on GCP, Azure Firewall), and emits per-vendor latency and error metrics, which is how you notice the vendor's bad morning before your users do.

24.6 Outbound webhooks: Encore as the provider

Organizers want `order.paid` pushed to their own systems, so Encore becomes somebody else's payment provider and owes them what it expects from its own: signed messages, retries, and a way to catch up. The pipeline reuses the building blocks of Chapter 14: the outbox publishes the event, a fan-out step writes one delivery job per subscribed endpoint, and **one queue per endpoint** (or a per-endpoint concurrency cap on a shared one) keeps a slow organizer from delaying everyone else. Each attempt has a 15 to 30 second timeout, as the Standard Webhooks specification recommends; any `2xx` is success, anything else is retried on a capped exponential schedule:

$$d_k = min(c,\ b \quad 2^k) \qquad b = 60\ \text{s},\ c = 6\ \text{h},\ \text{horizon } 72\ \text{h}$$

With these parameters an undeliverable event is tried 20 times, the last about 68.5 hours after the first, and then the endpoint is disabled and its owner e-mailed. The signing scheme follows Standard Webhooks: HMAC-SHA256 over `id.timestamp.body`, base64-encoded, with the `webhook-id` doubling as the receiver's deduplication key:

```
POST /hooks/encore HTTP/1.1
Host: tickets.organizer.example
Content-Type: application/json
webhook-id: msg_1
webhook-timestamp: 1792000000
webhook-signature: v1,aXFi406xj...

{"type": "order.paid", "order": "o_77", "status": "paid", "version": 3}
```

The signature header may carry several space-separated signatures, which is how a secret is rotated without downtime. Managed options exist (Amazon EventBridge API destinations deliver to HTTP endpoints with retries; Svix is a hosted webhook sender built on the same specification), but the per-endpoint isolation and the replay API are design decisions either way.

> **Design question.** An organizer's endpoint is down for the whole flash sale and returns a day later. What happens to its 25,000 `order.paid` events?
>
> They wait in that organizer's queue; no other organizer notices, because each endpoint has its own queue and concurrency cap. Retries back off to one attempt every 6 hours, so a dead endpoint costs little. When it recovers, Encore must not flood it: a cap of 50 deliveries per second (illustrative) drains 25,000 events in $25,000/50 = 500$ s, about 8 minutes. Events arrive out of order, so each carries the order's current status and a version, and the

> receiver deduplicates by `webhook-id`. For anything beyond the 3-day horizon, Encore offers a replay API (`GET /events?after=<cursor>`) over the event log, so an organizer can always catch up. See also *50 System Designs* #20 (Notification System).

24.7 LLM services as dependencies: the model gateway

A model API is an external system with unusual numbers. A call takes seconds, not milliseconds; it costs money per token, not per call; the same input can give different outputs; and the provider limits you in requests *and tokens* per minute. Calling it directly from every service scatters keys, limits and bills across the codebase. A **model gateway** is the travel agent between Encore and the airlines: one desk that knows every carrier's prices and schedules, books the best available seat, rebooks when a flight is cancelled, and keeps the receipts. It does six jobs. **Routing** sends each task to a model tier (a small model classifies a ticket, a large one plans a refund). **Fallbacks** go to a second provider or region when the first times out, but only to models evaluated on the same task, because a fallback that picks the wrong tool is worse than an error. **Token-based limits** per feature and tenant reserve the estimated tokens before the call and settle the actual count after it. **Cost accounting** tags tokens times price by feature, conversation and customer. **Caching** covers exact-match, provider prompt caching and scoped semantic caching (Chapter 9). **Credentials and logs**: provider keys live only here, and prompts are logged with personal data redacted.

Take a support run of six model calls (illustrative). As *Designing Agentic Systems* shows, the context grows each step, so six steps that start at 3,000 input tokens and add 1,000 per step process 33,000 input tokens, plus 200 output tokens per step. At illustrative prices of $3 per million input and $15 per million output tokens, without prompt caching:

$$\text{cost per run} = \frac{33,000 \times 3 + 1,200 \times 15}{10^6} = 0.099 + 0.018 = \$0.117$$

At 20,000 runs a day (illustrative) that is $2,340 a day. The token limit bites before the bill does. With prompt caching, about 4,000 *uncached* input tokens per step count against the provider's limit, and the 2,000,000 input tokens a minute of Chapter 11 serve 500 steps, about 83 runs a minute or 1.4 runs per second. Flash-sale morning brings 5 runs per second, $5 \times 60 \times 6 \times 4,000 = 7.2$ million tokens a minute: more than three times the quota. The core of a gateway is small:

```
PRICE = {"big": (3.0, 15.0), "small": (0.8, 4.0)}  # $ per M tokens in/out
budget = {"support": 2_000_000}                    # tokens left this minute


def call(model: str, prompt: int) -> int:          # returns output tokens
    if model == "big":
        raise TimeoutError("provider A slow")       # simulated outage
    return 200


def complete(feature: str, prompt: int, max_out: int = 400) -> str:
    if budget[feature] < prompt + max_out:          # reserve before calling
        return "429 Retry-After: 20"
    budget[feature] -= prompt + max_out
    for model in ("big", "small"):                  # fallback chain
        try:
            out = call(model, prompt)
        except TimeoutError:
            continue                                # counted as spent
        budget[feature] += max_out - out            # settle actual usage
        cost = (prompt * PRICE[model][0] + out * PRICE[model][1]) / 1e6
        return f"{model}: {out} tokens, ${cost:.4f}"
    return "503"


print(complete("support", 8000), budget)
```

It falls back to the small model and prints `small: 200 tokens, $0.0072`, with 1,991,800 tokens left in the minute's budget. In production the budget is a Redis token bucket shared by all gateway replicas (Chapter 11), and a timed-out call stays counted as spent, because the provider may have billed it. Where it runs: LiteLLM proxy, Agent Router (formerly Envoy AI Gateway, which also routes MCP traffic), Kong AI Gateway and Cloudflare AI Gateway; Azure API Management's `llm-token-limit` policy enforces tokens per minute and quotas per key and answers 429 with `Retry-After`. Amazon Bedrock's cross-region inference (Vertex AI's global endpoint, Azure AI Foundry's global deployments) spreads one model's traffic over several regions' capacity.

24.8 Streaming, timeouts and partial results

A model writes its answer a token at a time, so the gateway streams it to the chat client over Server-Sent Events (Chapter 4). With an illustrative 1 second to the first token and 50 tokens per second after it, a 300-token answer takes 1 + 300/50 = 7 seconds, and streaming turns 7 seconds of blank screen into 1. One request now has three clocks, and a single timeout cannot express them: a **time-to-first-token** timeout catches a stuck or queued provider (5 seconds, then fall back), an **idle timeout between chunks** catches a stream that stalls midway (10 seconds), and a **total deadline** with `max_tokens` stops a runaway answer (60 seconds; all illustrative).

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
X-Accel-Buffering: no

data: {"type": "delta", "text": "Order o_77 is paid; tickets went out."}

data: {"type": "done", "output_tokens": 9}
```

Every hop must pass chunks through unbuffered (`X-Accel-Buffering: no` for Nginx-style proxies), and idle timeouts on load balancers must exceed the longest silence (an AWS ALB closes idle connections after 60 seconds by default), so long answers send SSE comment lines as heartbeats. When a stream breaks midway, text already shown is a **partial result**: mark it, and regenerate or resume on a fallback model, remembering that a retry pays for the input tokens again. A **tool call** is never acted on from a partial stream: the runtime waits for the complete, schema-valid call, because half a refund request is not a refund request.

24.9 Agents as distributed components

To a system designer an agent is a loop: read state, ask the model what to do, run the tools it chose, save state, repeat. Everything distributed about it follows from where that state lives. Keep it out of the process, and the **agent runtime becomes a stateless service** like any other: any worker can run any step, a crashed worker's run resumes elsewhere from the last checkpoint, and scaling is adding workers.

```
chat -> API (202 + status URL) -> queue -> any worker <-> checkpoints
worker -> model gateway                          (DynamoDB, S3 if large)
worker -> tools (MCP) -> orders API
approval needed -> workflow waits on a token -> human decides -> resume
```

- **Checkpoints** go to a durable store after every step, keyed by `conversation_id` (the partition key, Chapter 17) and step number. A DynamoDB item holds at most 400 KB, so a long transcript goes to S3 and the item keeps a pointer.
- **Long tasks go on queues.** An interactive chat turn streams; a task that may take minutes (a refund awaiting approval) is accepted with 202 and a status URL, queued on SQS, and run by workers whose concurrency is capped by the token budget, not by CPU (Chapter 14).
- **Human approval is a workflow step**, not a sleeping thread: AWS Step Functions pauses on a task token (`.waitForTaskToken`) for up to a year, Temporal waits for a signal, and nothing runs, or bills, while the supervisor decides. The approval is bound to the exact proposal (tool, order, amount), and the tool re-checks everything after it.

The numbers are small, which is the point of the design. At the 1.4 runs per second the token budget allows, and six model calls of about 8 seconds each, Little's law gives $1.4 \times 48 \approx 67$ runs in flight and about $1.4 \times 6 \approx 8$ checkpoint writes per second. The model and its quota are the bottleneck; the runtime is a few containers. Managed runtimes package the same shape: Amazon Bedrock AgentCore (Vertex AI Agent Engine, Azure AI Foundry Agent Service).

24.10 Tools as idempotent, authorized APIs

A tool is an API whose client is a model: fast, tireless, literal, and occasionally wrong or manipulated. So the tool, which is ordinary code, carries every guarantee, and the model only proposes. Four rules, each a mechanism from an earlier chapter:

- **Authorize from the session.** The tool runs with a token exchanged from the fan's session (its `sub` is the fan, its `act` claim names the agent; Chapter 22), and checks ownership itself. An order ID typed by the model is a request, not a permission.
- **Compute limits in code.** The refund cannot exceed what the order cost, summed over all refunds, whatever amount the model asks for.
- **Make writes idempotent.** The key `refund:{order_id}:{conversation_id}` (Chapter 18) makes a retried call return the first answer; the running total makes a call with a *fresh* key harmless.
- **Read your writes.** A tool that checks a refund it just issued reads from the leader, not a lagging replica (Chapter 16).

```
import sqlite3

APPROVAL_ABOVE = 15_000                      # cents: $150, illustrative
db = sqlite3.connect(":memory:")
db.executescript("""
CREATE TABLE orders (id TEXT PRIMARY KEY, owner TEXT, total INTEGER);
CREATE TABLE refunds (key TEXT PRIMARY KEY, order_id TEXT, amount INT,
                      status TEXT);
INSERT INTO orders VALUES ('o_77', 'u_42', 16000);""")


def request_refund(actor: str, order_id: str, amount: int, key: str):
    with db:                          # actor: from the session token only
        row = db.execute("SELECT owner, total FROM orders WHERE id = ?",
                         (order_id,)).fetchone()
        if row is None or row[0] != actor:
            return "403 not your order"
        old = db.execute("SELECT status FROM refunds WHERE key = ?",
                         (key,)).fetchone()
        if old:
            return f"200 {old[0]} (same key, same answer)"
        done = db.execute("SELECT COALESCE(SUM(amount), 0) FROM refunds "
                          "WHERE order_id = ?", (order_id,)).fetchone()[0]
        if done + amount > row[1]:
            return "422 exceeds order total"
        status = "awaiting_approval" if amount > APPROVAL_ABOVE else "issued"
        db.execute("INSERT INTO refunds VALUES (?, ?, ?, ?)",
                   (key, order_id, amount, status))
        return f"201 {status}"


for who, amount, conv in [("u_42", 16000, "c_9"), ("u_42", 16000, "c_9"),
                          ("u_42", 16000, "c_10"), ("u_7", 100, "c_11")]:
    print(request_refund(who, "o_77", amount, f"refund:o_77:{conv}"))
```

It prints `201 awaiting_approval` ($160 is above the $150 threshold), the same answer for the retry, `422` for a second conversation trying to refund the same order again, and `403` for another fan. On Postgres, run it as one transaction with `SELECT ... FOR UPDATE` on the order row so two concurrent refunds cannot both pass the total check (Chapter 3). The tool never calls the payment provider itself: an issued refund goes through the payments service and its outbox, with the provider's own idempotency key derived from the refund key.

24.11 MCP servers, A2A and retrieval services

MCP servers are microservices. The Model Context Protocol exposes tools over JSON-RPC 2.0, and its 2026-07-28 revision made the HTTP transport stateless: no protocol sessions, every request its own POST to one endpoint, answered with JSON or a request-scoped SSE stream. The method and tool name are mirrored into headers, so the API gateway can route, rate-limit and audit per tool without parsing bodies (the server rejects a header that disagrees with the body):

```
POST /mcp HTTP/1.1
Host: tools.encore.internal
Authorization: Bearer <token issued for this server>
MCP-Protocol-Version: 2026-07-28
Mcp-Method: tools/call
Mcp-Name: get_order

{"jsonrpc": "2.0", "id": 7, "method": "tools/call",
 "params": {"name": "get_order", "arguments": {"order_id": "o_77"}}}
```

So Encore's MCP tool server is a stateless HTTP service behind the gateway, load-balanced without stickiness and scaled like any other. MCP authorization is OAuth 2.1: tokens are bound to the server's own URI (resource indicators), and a server must not pass a client's token through to the APIs behind it, the confused-deputy rule of Chapter 22. **A2A** is for agents someone else owns, a venue's accessibility desk for example: read its Agent Card at `/.well-known/agent-card.json`, send a message that opens a *task* (working, input-required, completed), and receive *artifacts*. Treat it like a partner API: allowlisted cards, short-lived tokens scoped to one skill, timeouts, and an idempotency key on any side effect, because a retry opens a new task. **Retrieval** (the policy documents the agent cites) is a service with its own SLO: embeddings in pgvector next to the data, refreshed from the policy store by change data capture, filtered to public documents in the query itself; a fan's own data comes through tools, never through the index. *Designing Agentic Systems* covers when to choose a tool, an MCP server, a subagent or an A2A agent.

24.12 Guardrails and audit

Guardrails are checkpoints placed in the request path, each a component you can name in a design: at the **entrance**, personal data is redacted and outside text (a fan's message, a quoted e-mail) labelled untrusted; at the **gate**, every proposed tool call meets the checks above plus call limits and a token budget per run; at the **exit**, replies are validated against a schema and the egress proxy refuses unknown hosts. Managed filters help at the edges (Amazon Bedrock Guardrails; Google Cloud Model Armor, Azure AI Content Safety), but none replaces authorization in the tool. The **audit log** records every model call and tool call with its trace ID, actor chain (fan, agent, tool), arguments and result, written append-only (S3 with Object Lock; bucket lock on Cloud Storage, immutable Blob Storage), so "why was this refunded?" has an answer a year later.

24.13 Developer tools in the loop

The build pipeline is an external system with production power. CI should reach the cloud through short-lived OIDC federation rather than stored keys, pin third-party actions by commit, and apply infrastructure as code only from a reviewed `terraform plan`. Coding agents fit the same frame as the support agent: they propose pull requests from a sandbox that holds no production credentials, and CI, tests and a human review decide what ships. Contract tests against each vendor's sandbox, run nightly, catch the renamed status field before a fan does.

24.14 In Encore: the support agent end to end

A fan whose show moved asks the support agent for a refund of a $160 order. Every arrow is a mechanism from this chapter or an earlier one:

```
fan -> chat API          "Refund order o_77, I can't make the new date"
chat API -> queue        run r_5 accepted: 202 + status URL, text by SSE
worker -> model gateway  step 1: model proposes get_order(o_77)
worker -> MCP tools      get_order as u_42 (exchanged token) -> paid, $160
worker -> model gateway  step 2: model proposes request_refund($160)
worker -> MCP tools      request_refund, key refund:o_77:c_9
refunds service          owner ok, total ok, $160 > $150 -> approval task
workflow -> supervisor   approve / edit / reject; the run sleeps for free
supervisor -> workflow   approve, bound to (o_77, $160, key)
workflow -> refunds      re-check, status issued, outbox: RefundIssued
payments -> provider     POST /refunds, Idempotency-Key: refund:o_77:c_9
provider -> webhook      refund.succeeded (signed) -> ledger updated
worker -> fan            "Refunded $160" in chat, confirmation by e-mail
```

The model chose two tool calls and wrote two sentences; code decided who the fan is, how much may be refunded, whether a human must approve, and that the money moves exactly once. If the model provider fails mid-run, the checkpoint survives and the chat offers a human; an approved refund completes without any model at all. Where each piece runs on AWS (equivalents in Appendix C): the chat API and the MCP tool server on ECS or EKS behind the API gateway; agent workers on ECS with Fargate, fed by SQS; the model gateway (LiteLLM or Agent Router) on EKS, in front of Bedrock and a second provider; checkpoints in DynamoDB with long transcripts in S3; the approval in Step Functions with a task token; policy retrieval in Aurora PostgreSQL with pgvector; the webhook receiver on API Gateway and Lambda, handing off to SQS; egress through Network Firewall or Envoy; the audit log in S3 with Object Lock.

> **Design question.** On flash-sale morning, 5 support runs per second arrive and the token budget allows 1.4. What do you do?
>
> Cut demand before buying capacity. Most questions that morning are “where are my tickets?” and “what is the refund policy for this show?”: the first is a deterministic status lookup that needs no model, the second a scoped semantic-cache hit on public policy. If those absorb 60 percent (illustrative), 2 runs per second still need the large model. A second provider with its own 2 million input tokens a minute doubles capacity to 2.8 runs per second; refund and payment issues get priority, and everything else is queued with an honest wait time or handed to a human. Ask the provider to raise the quota a week ahead, as you would load-test any other dependency. Trade-off: a two-provider gateway needs both models evaluated on Encore’s tasks. See also *50 System Designs* #25 (Ticket Booking).

24.15 Numbers to remember

Figure	Value
Checkout with a 99.9% provider	99.85%, about 13 h down a year
Stripe idempotency keys, webhook retries	at least 24 h; up to 3 days
SES sandbox	200 messages a day, 1 per second
Vendor calls in flight	$L = \lambda W$: 100/s x 0.1 s = 10
Outbound webhook schedule (Encore)	20 attempts over 68.5 h
Support run, 6 steps	33,000 input tokens, $0.117
2M input tokens/min, 4,000 per step	about 1.4 runs per second

24.16 Pitfalls

- Treating a timeout as a failure and charging again -> keep the payment *unknown*, retry with the same idempotency key, reconcile.
- Trusting webhooks to arrive once and in order -> verify, deduplicate by event ID, let status only move forward, reconcile daily.

- Enforcing a vendor quota per worker -> one shared bucket per vendor account, below the quota, honouring `Retry-After`.
- One pool or queue for every vendor and endpoint -> a bulkhead per vendor and a queue per webhook endpoint.
- Letting the model choose identity, amounts or permissions -> actor from the session, limits computed in the tool, approval above a threshold.
- Agent state in worker memory, approvals in sleeping threads -> checkpoint every step; approval as a workflow task.
- Calling model APIs directly from every service -> one model gateway with token limits, evaluated fallbacks and cost per feature.

24.17 Remember

- Every external call has three outcomes, success, failure and unknown; design for the third.
- Adapters translate, the egress proxy limits, the gateway admits, and reconciliation catches what slipped through.
- A model provider is a dependency priced and limited in tokens; the model gateway routes, limits, caches and counts.
- An agent is a stateless loop over checkpointed state: long tasks go on queues, approvals into workflows.
- Tools are ordinary APIs: authorized from the session, limited by code, idempotent by key, and audited.

25 Encore end to end: from requirements to code and cloud

Every earlier chapter taught one part. This one assembles them into a single design and carries it all the way to running code: from the requirements on a page, through the API, the schema and the architecture, to a race-condition test that really fails and really passes, and then to the files that put the system into the cloud (repository, Dockerfile, Kubernetes manifest, Terraform, CI pipeline). The question it answers is the one an interviewer and a new team member both ask: how do all these ideas fit together in one system, and where does each one live? The picture is a building project: each stage hands a concrete document to the next.

25.1 The picture

A house goes from a brief ("four bedrooms, a budget, near a school") to a floor plan, to wiring diagrams, to a building permit, to keys. Each stage produces a document the next stage builds on, and a mistake found late (a missing staircase) costs far more than one found early. A system design works the same way, and every stage of Encore's design leans on chapters of this book:

```
stage                        artifact                       chapters
requirements, estimates   ->  numbers that rule designs out  requirements
API, data model           ->  contracts and tables           apis, storage
high-level architecture   ->  boxes, arrows, owners          app. architecture
flash-sale deep dive      ->  waiting room, holds, race test concurrency
payments, queues, caches  ->  idempotency, outbox, CDN plan  queues, caching
shards, regions, SLOs     ->  where it breaks, how we know   partitioning
code and cloud            ->  repo, image, manifests, IaC    cloud-compute
```

The stages run top to bottom, but the work loops: the cost estimate at the bottom can send you back to the requirements to ask whether a seat map one second stale is acceptable (it is, and it lets the CDN absorb 200,000 reads a second).

25.2 Requirements and estimates

The core flows are four: view the seat map, hold seats for 10 minutes, pay, receive tickets. Search, organizer dashboards and the support agent are real but standard; resale and dynamic pricing are out of scope (Chapter 1). The non-functional targets, each with a number (illustrative): seat map p99 under 200 ms; 99.95% of holds answered in under 500 ms; checkout available 99.95% of each 30-day month; a seat sold at most once and a hold with one holder; a paid order never lost; a seat map at most about a second stale, and tickets in the app within a minute of payment.

The estimates that decide the design are the flash-sale numbers. One million fans arrive between 09:55 and 10:05; at the door they would make 30,000 hold attempts a second, while seat inventory is load-tested at 6,000 holds/s and run at 3,000. The waiting room closes that gap by admitting fans at a fixed rate:

$$\lambda_{\text{holds}} = \lambda_{\text{admit}} \times a = 1,000 \times 3 = 3,000 \text{ holds/s} \qquad T_{\text{back of queue}} = \frac{N}{\lambda_{\text{admit}}} = \frac{10^6}{1,000} \approx 17 \text{ min}$$

where a is about three hold attempts per admitted fan. On a normal day Encore serves 1,500 requests/s, 6,000 at the daily peak, which a modest stateless fleet handles. The rest of the page of numbers, all carried over from earlier chapters:

Quantity	Value	Consequence
Seat-map reads, flash sale	200,000/s	bitmap at the edge, 1 s TTL
Seat map as a bitmap	6.25 KB, 10 Gbit/s in total	CDN, not origin
Holds after the waiting room	3,000/s (6,000 tested)	12 inventory pods

Quantity	Value	Consequence
Orders at the peak	about 33/s (83 tickets/s)	one Postgres primary
Orders, ticket PDFs per year	100 GB, 2.5 TB	Postgres, S3
Hot seat state	50,000 x 100 B = 5 MB	fits one Redis shard

Without the waiting room every downstream figure is ten times larger: the 30,000 holds/s that Chapter 15 sizes for is the cost of an open door.

25.3 The API

Five calls carry the core flows; Chapter 5 derives each contract. Every write carries an `Idempotency-Key`, and the hold call also needs the waiting room's admission token, a signed cookie that the edge checks before the request reaches the origin; the traces of a retried hold and a `409` are in that chapter.

Call	Answer	Contract
`GET /v1/events/42/seat-state`	bitmap, 6.25 KB	CDN, `s-maxage=1`
`POST /v1/events/42/holds`	201 hold, or 409	admission token, idempotency key
`POST /v1/orders`	202, `Location`	idempotency key, pays one hold
`GET /v1/orders/o_77`	status, tickets	ETag; push as the fast path
`POST /webhooks/payments`	200	signature, dedupe by event id

25.4 The data model

Seat inventory owns `seats` and `holds`; orders owns `orders` and `tickets` (Chapter 6 draws the ownership lines). IDs are generated by the services (Snowflake-style, so they carry a shard field), the seat id spells section, row and seat, and the `section` column is what the inventory is partitioned on. The schema below is Postgres DDL that also runs unchanged on SQLite, where the tests in this chapter executed it:

```
CREATE TABLE events (event_id BIGINT PRIMARY KEY, venue_id BIGINT NOT NULL,
  starts_at TIMESTAMPTZ NOT NULL, home_region TEXT NOT NULL); -- 10 KB in all
CREATE TABLE seats (                       -- about 100 bytes a row
  event_id    BIGINT NOT NULL REFERENCES events,
  seat_id     TEXT NOT NULL,               -- 'A-12-7': section, row, seat
  section     TEXT NOT NULL,               -- shard key: event_id#section
  price_cents INT NOT NULL,
  status      TEXT NOT NULL DEFAULT 'free'
              CHECK (status IN ('free', 'held', 'sold')),
  hold_id     TEXT, held_until TIMESTAMPTZ,
  version     INT NOT NULL DEFAULT 0,      -- fencing token for confirms
  PRIMARY KEY (event_id, seat_id));
CREATE TABLE holds (
  hold_id     TEXT PRIMARY KEY,            -- 'h_91'
  event_id    BIGINT NOT NULL REFERENCES events, user_id BIGINT NOT NULL,
  expires_at  TIMESTAMPTZ NOT NULL,        -- created + 10 minutes
  idem_key    TEXT NOT NULL UNIQUE);       -- Idempotency-Key of the POST
CREATE TABLE orders (                      -- about 2 KB with line items
  order_id    TEXT PRIMARY KEY,            -- 'o_77'
  hold_id     TEXT NOT NULL UNIQUE REFERENCES holds,
  user_id     BIGINT NOT NULL, amount_cents INT NOT NULL,
  status      TEXT NOT NULL DEFAULT 'pending_payment',
  idem_key    TEXT NOT NULL UNIQUE);
CREATE TABLE tickets (ticket_id TEXT PRIMARY KEY,
  order_id TEXT NOT NULL REFERENCES orders, event_id BIGINT NOT NULL,
  seat_id TEXT NOT NULL, status TEXT NOT NULL DEFAULT 'valid');
CREATE UNIQUE INDEX one_live_ticket_per_seat      -- the backstop
  ON tickets (event_id, seat_id) WHERE status = 'valid';
```

Three details carry the correctness argument. The seat row is the unit of contention, so the hold is a conditional update of that row (Chapter 3). `version` increases on every change and serves as the fencing token when a payment confirms a hold, so a confirmation that arrives after its hold expired changes zero rows (Chapter 18). And the partial unique index allows at most one valid ticket per seat: whatever bug slips past the application, the database refuses a second ticket, which the test run confirmed with `UNIQUE constraint failed: tickets.event_id, tickets.seat_id`.

25.5 The high-level architecture

```
fans, organizers -> Route 53 (latency records) -> nearest healthy region
     v
CloudFront: pages, images, seat bitmap (1 s); token check at the edge
     |
waiting room: Redis sorted set, positions over SSE, 1,000 fans/s
     v
ALB -> Envoy gateway: auth, limits, home-region routing <- MCP tools <- agent
     |               |                    |                 |
  catalog       seat inventory          orders -------> payment adapter
  Aurora +      Redis holds (TTL)       Aurora + outbox     <-> provider
  Redis cache   + Aurora seat rows        |    ^ signed webhooks
     |                                    v
  OpenSearch <- CDC          CDC -> Kafka order-events
                                |-> SQS -> Lambda ticket PDF -> S3
                                |-> SQS -> notifications -> SES, SNS, push
                                |-> dashboards (SSE), lakehouse (Iceberg)
```

Read the diagram by its two kinds of arrow. **Synchronous** arrows are where a fan waits for an answer: seat map, hold, the `202` from checkout; there are as few of them as possible, each with a timeout that shrinks inward (Chapter 3). **Asynchronous** arrows carry consequences: once an order is paid, tickets, e-mails, dashboards and analytics follow through the outbox and Kafka, so a slow e-mail provider can never fail a checkout (Chapter 14). Each service owns its tables, every external system sits behind an adapter, and the support agent is one more client of the orders API: its refund tool carries an idempotency key and runs the same transaction as a refund from the support console (Chapter 24; *Designing Agentic Systems* covers the agent itself).

25.6 The flash sale deep dive

The numbers point at one place: one event, 50,000 rows, 30,000 attempts a second at the door. Three mechanisms work in sequence, each shrinking the problem for the next.

25.6.1 The waiting room

A theme park's virtual queue: you get a return time on your phone instead of standing in line, and the ride sees a steady stream. Fans who open the event page from 09:55 join a Redis sorted set; those who arrive before 10:00 get a random score, so refreshing early buys nothing, and later arrivals queue in order. Every second an admitter pops 1,000 fans and gives each a signed admission token (event, user, 15-minute expiry) as a cookie; a CloudFront Function checks it at the edge and the hold API checks it again (Chapter 10, Chapter 11). Positions stream over SSE (Chapter 4). The queue empties in about 17 minutes, but 50,000 seats can sell out sooner, so the moment the inventory reports sold out the waiting room tells everyone still in line. Tested with fakeredis:

```
ZADD wr:{42} NX 1791960001.1 user:501   -> 1     (joined; NX keeps the score)
ZRANK wr:{42} user:503                  -> 3     (fans ahead of user:503)
ZPOPMIN wr:{42} 1000                    -> up to 1,000 fans admitted now
```

25.6.2 Holds: a TTL in Redis and a durable record

A cloakroom ticket and the ledger behind the counter: the ticket is handed out instantly, the ledger is what settles disputes. A hold takes two steps:

1. **Redis filters.** One Lua script checks every requested seat key, `hold:{42#A}:A-12-7` and its neighbours, and sets all of them with `EX 600` or none (Chapter 9). Seats already held are rejected in microseconds with `409` and nearby free

seats. The `{42#A}` hash tag is the partition key `event_id#section`, so all keys of one script share a Redis Cluster slot.
2. **Postgres decides.** For the winner, one transaction inserts the `holds` row and runs the conditional `UPDATE seats ... WHERE status = 'free' OR held_until < now()`; it commits only if every seat changed, otherwise it rolls back and deletes the Redis keys it set.

Redis absorbs the losers, which are almost all attempts; Postgres carries the truth, so a Redis failover that loses a few acknowledged keys (its replication is asynchronous) costs fans a retry, never a double sale. Expiry needs no timer for correctness: the Redis key vanishes after 600 s, and the conditional write treats an expired `held_until` as free; a sweeper resets expired rows in batches only to keep the seat map accurate. Postgres sees winners only; even if all 3,000 attempts a second reached it, the pool would need about $3,000 \times 5$ ms = 15 busy connections, against the 225 that Chapter 4 computes for the open door.

25.6.3 Never oversell: the race test

A design that never oversells must be shown not to, under the worst timing, before 10:00 on sale day. The test below follows the recipe from Chapter 3: eight threads play eight fans, a barrier releases them in the same instant, and a 5 ms sleep between check and act stands in for a network round trip, widening the window the way production load does. Each fan uses its own database connection, as requests on different pods would. The first function is the bug; the next three are the fixes. The Redis fans ask for overlapping pairs of seats (A-12-7 with A-12-8, A-12-8 with A-12-9), so at most one pair can win:

```
import sqlite3, threading, time
import fakeredis

GAP, SEAT = 0.005, (42, "A-12-7")     # event 42, section A, row 12, seat 7
READ = "SELECT status FROM seats WHERE event_id = ? AND seat_id = ?"
TAKE = ("UPDATE seats SET status = 'held', hold_id = ?, held_until = ? "
        "WHERE event_id = ? AND seat_id = ?")
redis = fakeredis.FakeServer()        # one Redis server, many clients

def sql():                            # every fan's request has its own conn
    return sqlite3.connect("encore.db", timeout=10, isolation_level=None)

def race(hold, fans=8):
    sql().executescript("""DROP TABLE IF EXISTS seats;
      CREATE TABLE seats (event_id INT, seat_id TEXT, status TEXT,
        hold_id TEXT, held_until INT, PRIMARY KEY (event_id, seat_id));
      INSERT INTO seats VALUES (42, 'A-12-7', 'free', NULL, NULL);""")
    fakeredis.FakeRedis(server=redis).flushall()
    gate, wins = threading.Barrier(fans), []
    def fan(i):
        gate.wait()
        if hold(f"fan{i}", [["A-12-7", "A-12-8"], ["A-12-8", "A-12-9"]][i % 2]):
            wins.append(i)
    ts = [threading.Thread(target=fan, args=(i,)) for i in range(fans)]
    for t in ts: t.start()
    for t in ts: t.join()
    return len(wins)

def buggy(fan, _):                    # check, wait, act
    db = sql()
    if db.execute(READ, SEAT).fetchone()[0] != "free":
        return False
    time.sleep(GAP)                   # a network round trip, a log call
    db.execute(TAKE, (fan, time.time() + 600, *SEAT))
    return True

def conditional(fan, _):              # fix 1: the check lives in the write
    now = time.time()
    cur = sql().execute(TAKE + " AND (status = 'free' OR (status = 'held'"
                       " AND held_until < ?))", (fan, now + 600, *SEAT, now))
    return cur.rowcount == 1

def pessimistic(fan, _):              # fix 2: lock first, then check and act
    db = sql()
    db.execute("BEGIN IMMEDIATE")     # Postgres: SELECT ... FOR UPDATE
    free = db.execute(READ, SEAT).fetchone()[0] == "free"
    if free:
        time.sleep(GAP)
        db.execute(TAKE, (fan, time.time() + 600, *SEAT))
    db.execute("COMMIT")
    return free

HOLD_ALL = """
for _, k in ipairs(KEYS) do
  if redis.call('EXISTS', k) == 1 then return 0 end
end
for _, k in ipairs(KEYS) do redis.call('SET', k, ARGV[1], 'EX', 600) end
return 1"""

def lua(fan, seats):                  # fix 3: all seats or none, atomically
    r = fakeredis.FakeRedis(server=redis)
    keys = [f"hold:{{42#A}}:{s}" for s in seats]   # tag = partition key
    return r.eval(HOLD_ALL, len(keys), *keys, fan) == 1

for fix in (buggy, conditional, pessimistic, lua):
    print(f"{fix.__name__:11} fans told 'yours': {race(fix)}")
```

Every run (ten in a row, with `sqlite3` and `fakeredis` from the book's environment) printed the same four lines: `buggy` told 8 fans the seat was theirs, and `conditional`, `pessimistic` and `lua` each told exactly 1. In CI the same test runs against real Postgres and Valkey containers, with `FOR UPDATE` in place of `BEGIN IMMEDIATE`, on every change to the hold path.

- **The bug.** All eight fans read `free` before any of them wrote, so all eight were told the seat is theirs and the last writer silently owns it. The same race happens in Redis with `GET` followed by `SET`; moving to a faster store does not remove a gap between check and act.
- **Fix 1, conditional write.** The check moves into the `WHERE` clause and the row count says who won: compare-and-set in SQL. It needs no lock held across a round trip, which is why Encore uses it. In Postgres under Read Committed, an `UPDATE` that waited for a concurrent writer re-checks its `WHERE` clause against the new row version, so the second fan updates zero rows there too.
- **Fix 2, pessimistic lock.** Lock first, then check and act. SQLite's `BEGIN IMMEDIATE` takes the single database-wide write lock; Postgres's `SELECT ... FOR UPDATE` locks only the seat rows, so other seats proceed in parallel. Correct, but every fan behind a hot seat waits a round trip; use it when the decision spans several rows and needs application logic in between.
- **Fix 3, an atomic script.** Redis runs one Lua script at a time, so "check every seat, then set every seat" cannot interleave with another fan's script, and the hold is all-or-nothing with its 600 s TTL built in. Under all three fixes stays the backstop: the unique index on valid tickets.

> **Design question.** A fan's Lua script succeeds in Redis, but the Postgres transaction that should record the hold times out. What does the fan see, and what state is left?
>
> The invariant is that Postgres decides, so a timeout is a failure, not a hold. The service cannot know whether the commit happened, so it retries once with the same hold id: either the insert into `holds` succeeds (the first attempt never committed) or it hits the primary key `h_91` (it did, so the seats are already held by this fan). Both outcomes are safe, because the hold id makes the write idempotent. If the retry also fails within the request's budget (about 500 ms), the service deletes its Redis keys and answers `503` with `Retry-After`, and the fan's app retries with the same `Idempotency-Key`. The worst leftover is a Redis key without a Postgres row, which blocks the seat for at most 600 s and which a sweeper can clear by comparing the two. The trade-off: briefly unsellable seats instead of a second source of truth that could disagree. See also *50 System Designs* #25 (Ticket Booking).

25.7 Payments: idempotency keys and the outbox

Checkout is a saga in three local steps (Chapter 18). `POST /v1/orders` inserts the order with the client's `Idempotency-Key` under a unique constraint (`ON CONFLICT DO NOTHING`, so a retried click finds the first order), calls the provider with Encore's own idempotency key derived from the order id, and answers `202`. The provider's signed webhook then lands in one transaction that dedupes the event, sells the seats fenced by the hold id, marks the order paid, issues tickets and writes the outbox row; no step can happen without the others. Tested on SQLite (with `INTEGER` for `BIGSERIAL`), including a repeated webhook and a retried order:

```
CREATE TABLE payment_events (provider_event_id TEXT PRIMARY KEY,
  order_id TEXT NOT NULL REFERENCES orders);      -- webhook dedupe
CREATE TABLE outbox (seq BIGSERIAL PRIMARY KEY, topic TEXT NOT NULL,
  msg_key TEXT NOT NULL, payload JSONB NOT NULL, sent_at TIMESTAMPTZ);
-- POST /webhooks/payments, event evt_3 'payment.succeeded' for o_77
BEGIN;
INSERT INTO payment_events VALUES ('evt_3', 'o_77') ON CONFLICT DO NOTHING;
-- 0 rows inserted: a repeat delivery; ROLLBACK and answer 200
UPDATE seats SET status = 'sold', version = version + 1
 WHERE event_id = 42 AND hold_id = 'h_91' AND status = 'held';
-- rows changed must equal the seats in the hold, or roll back and refund
UPDATE orders SET status = 'paid'
 WHERE order_id = 'o_77' AND status = 'pending_payment';
INSERT INTO tickets (ticket_id, order_id, event_id, seat_id)
  SELECT 't_' || seat_id, 'o_77', event_id, seat_id
    FROM seats WHERE event_id = 42 AND hold_id = 'h_91';
INSERT INTO outbox (topic, msg_key, payload)
  VALUES ('order-events', 'o_77', '{"type": "order.paid", "order": "o_77"}');
COMMIT;
```

If the seat update changes fewer rows than the hold has seats, the hold expired and someone else holds a seat now: the transaction rolls back, the order becomes `refund_due`, and the compensation refunds the fan through the provider with its own idempotency key. Debezium (or a polling relay) moves outbox rows to the Kafka topic `order-events`, keyed by order id, at least once, so every consumer dedupes (Chapter 14). A nightly job reconciles Encore's orders against the provider's settlement report, which catches the webhooks that never arrived. The card numbers never touch Encore: the provider's hosted fields tokenise them, which keeps PCI DSS scope small (Chapter 22).

25.8 Notifications through a queue

A consumer of `order-events` fans each paid order out to one SQS queue per channel (e-mail, SMS, push), each with a dead-letter queue. The ticket-PDF queue feeds a Lambda function: at the peak of about 83 tickets a second and 200 ms per PDF, Little's law gives about 17 concurrent environments (Chapter 15). Consumers dedupe by `order_id` because SQS delivers at least once, and the e-mail sender respects the provider's sending quota with a token bucket, so a sell-out delays e-mails by minutes instead of getting the account throttled (Chapter 11). The app does not wait for e-mail: the order page shows the tickets as soon as the PDF lands in S3, behind a signed URL.

25.9 Caching and the CDN plan

Data	Where	TTL	Freshness rule
Images, JS, CSS	CloudFront, browser	1 year, immutable	versioned URLs
Event page HTML	CloudFront	60 s, browser 10 s	purge by tag on edit
Seat bitmap per event	CloudFront + Origin Shield	1 s	rebuilt every second
Event and venue details	Redis, cache-aside	60 s + jitter	delete on write
Holds, waiting line	Redis	600 s, sale window	Postgres decides
Orders, holds, account	nowhere shared	`no-store`	always origin

The rule behind the table: public and slow-changing data goes to the edge, shared objects to Redis with short jittered TTLs, and anything that decides money or seats is read from its owner (Chapter 9, Chapter 10). With a 1-second TTL behind Origin Shield, the 200,000 seat-map reads a second cost the origin about one fetch per second per event; the edge carries 10 Gbit/s, and 200,000 requests a second sits near CloudFront's default quota of 250,000 per distribution, so the quota is raised and the path load-tested before the sale.

25.10 Sharding the inventory

One event is the hot key, so the partition key is `event_id#section` (Chapter 17). The stadium's 100 sections of 500 seats spread over Redis Cluster's 16,384 slots by their hash tags; at the 3,000 admitted holds a second that is 30 a second per section on average, and even a floor section drawing 10 percent sees 300, which a Redis primary handles without noticing. The tags still matter: without the waiting room the same floor section would see 3,000 a second, and splitting by section is what keeps one shard from becoming the single queue of the whole sale (Chapter 2). Postgres needs no sharding today: winners only reach it, and the seat rows of all live events fit one primary, list-partitioned by event inside Postgres for cheap archiving. Orders stay unsharded too (100 GB a year, about 33 a second at the peak); their ids already carry a shard field for the day they need one.

25.11 Two regions and failover

Encore runs in us-east-1 (Virginia) and eu-central-1 (Frankfurt), about 90 ms apart (Chapter 16). Reads are active-active: event pages, search and seat maps are served in both regions from replicas and caches. Writes for one event's seats are not: each event has a **home region**, the venue's, which holds the only writable copy of its seat rows and holds, and the gateway forwards a Frankfurt fan's hold there over a warm connection, at the price of one ocean round trip on a 10-minute flow. Orders use Aurora Global Database (replication lag typically under a second), sessions and profiles a DynamoDB global table, ticket PDFs S3 Cross-Region Replication.

> **Design question.** Virginia fails at 10:02, in the middle of the stadium sale. What happens, step by step?
>
> Route 53 health checks fail over within a few minutes (detection plus a 60 s TTL; Global Accelerator does it in seconds). The waiting room pauses admission and tells fans the sale is paused; each fan's signed queue cookie carries their join time, so Frankfurt can rebuild the line without Virginia's Redis. Frankfurt's async standby of the inventory is promoted by an operator runbook, not automatically, because an automatic promotion during a partition is how two regions end up selling the same seats. Writes from the last second may be missing: holds re-validate on the next click, and paid orders reconcile against the provider, whose idempotency keys and records say what was really charged. The sale resumes in Frankfurt minutes later. The trade-off: an RTO of minutes and a paused sale, in exchange for never running two writable copies of one seat. See also *50 System Designs* #25 (Ticket Booking) and #38 (Globally Distributed Database).

25.12 Observability and SLOs

The SLIs are measured at the gateway, close to the fan: seat-map latency, hold answers in under 500 ms, checkout availability (Chapter 23). One detail decides whether the hold SLO means anything: a `409 Conflict` is a *good* event, the correct answer to a fan who lost the seat; only 5xx responses and slow answers are bad. During the sale, 3,000 attempts a second for ten minutes is 1.8 million attempts, and a 99.95% SLO allows 900 bad ones. A multi-window burn-rate alert pages when both the last hour and the last five minutes burn the 30-day budget 14.4 times too fast, a pace that spends 2 percent of it in an hour; in PromQL the condition is the ratio of `hold_requests_total{outcome="bad"}` to all hold requests, over `[1h]` and over `[5m]`, each above `14.4 * 0.0005`. A ten-minute sale is shorter than the long window, so on sale day the on-call team also watches a dashboard of 1-minute rates. Traces (OpenTelemetry, propagated by Envoy) follow one hold from the edge through Redis and Postgres. Two invariant monitors sit beside the SLOs: confirmations that lost their fencing check (each one a refund) and a nightly query that finds sold seats without exactly one valid ticket, which must always return zero.

25.13 The cost estimate

Cost follows the same arithmetic as capacity: units times unit price. For one stadium sale, with us-east-1 list prices (CloudFront $0.085 per GB and $0.0100 per 10,000 HTTPS requests; Fargate-equivalent $0.0899 an hour for a 2 vCPU, 2 GB pod; SES $0.10 per 1,000 e-mails) and illustrative volumes:

$$\text{cost} = \sum_i \text{units}_i \times \text{price}_i \qquad \text{seat maps} = 200{,}000 \times 600\ \text{s} \times 6.25\ \text{KB} = 750\ \text{GB}$$

Item	Units	Cost
CDN transfer	750 GB seat maps + 200 GB pages	about $81
CDN requests	120 M seat maps + 30 M page objects	about $150
Seat inventory	12 pods, 09:40 to 10:30	about $1
Waiting room, Redis	20 SSE nodes, 6 cache nodes, 1 hour	about $4
E-mail, PDFs	20,000 orders, 50,000 PDFs	about $2
Total	50,000 tickets	**about $240, 0.5 cents a ticket**

> **Design question.** The finance team asks what a flash sale costs and where to save. What do you answer?
>
> Infrastructure is about $240 for 50,000 tickets, half a cent each, and the largest line is CDN *requests*, not bytes: a 1-second seat-map TTL is 120 million requests. Raising the TTL to 2 s halves the seat-map requests at the price of a staler map, and pushing seat deltas over the SSE connections fans already hold would remove most of it. Compute is noise: even the open door's 60 inventory pods would cost about $5. The number that matters is elsewhere: card fees at an illustrative 2.9 percent plus 30 cents on 20,000 orders of 2.5 seats at $90 are about $137,000, nearly 600 times the infrastructure. The real cost lever is availability, not the cloud bill: a stalled sale puts $4.5 million of revenue (illustrative) at risk. Day to day, about 60 always-on 1 vCPU tasks across both regions cost about $2,160 a month at Fargate prices, before databases, caches and the CDN.

25.14 From repository to cloud

The design becomes real through five files a reviewer can read: a repository layout, an image, a manifest, infrastructure code and a pipeline. Chapter 15 explains each tool; here is what Encore writes.

25.14.1 Repository layout

One repository with one directory per deployable keeps a change to the hold path, its migration, its manifest and its test in one reviewable pull request:

```
encore/
  api/openapi.yaml               the public contract; gateway checks, SDKs
  services/seat-inventory/       src/ (api, holds, repo, hold_all.lua),
                                 tests/test_race.py, Dockerfile, locks
  services/{orders,catalog,waiting-room,notifications}/
  functions/ticket-pdf/          Lambda handler, built as an image
  libs/encore-common/            idempotency, outbox relay, telemetry
  db/migrations/                 expand-and-contract SQL per service
  deploy/k8s/{base,overlays/{staging,use1,euc1}}/   Kustomize manifests
  infra/terraform/{modules,envs/{staging,prod}}/    Terraform
  loadtest/flash_sale.py         replays 09:55 to 10:05 at 1/10 scale
  .github/workflows/
```

25.14.2 The Dockerfile

`services/seat-inventory/Dockerfile` has two stages, dependencies before code so a code change rebuilds one small layer, a non-root user, and a graceful shutdown shorter than Kubernetes' 30-second grace period, so in-flight holds finish while the pod drains (Chapter 8). CI pins both base images by digest:

```
FROM python:3.12-slim AS build
COPY requirements.lock .
RUN pip install --no-cache-dir --prefix=/install -r requirements.lock

FROM python:3.12-slim
COPY --from=build /install /usr/local
WORKDIR /app
COPY src/ ./src/
USER 10001
EXPOSE 8080
CMD ["uvicorn", "src.inventory.api:app", "--host", "0.0.0.0", \
     "--port", "8080", "--timeout-graceful-shutdown", "20"]
```

25.14.3 The Kubernetes deployment

The Deployment itself (three zones, probes, requests without CPU limits, image by digest) is the one in Chapter 15. What the sale adds is scaling that does not wait for CPU to rise: a KEDA `ScaledObject` combines a cron trigger, which raises the floor to 12 pods at 09:40, with a demand trigger on hold requests per second at the gateway, and a PodDisruptionBudget with `maxUnavailable: 1` keeps node upgrades from draining the fleet mid-sale. The pod template also gets a `preStop` sleep of 10 s, so balancers stop sending before the process stops listening. KEDA creates and owns the HorizontalPodAutoscaler, so the plain HPA of Chapter 15 is removed: two autoscalers on one Deployment fight each other. Validated as YAML:

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata: {name: seat-inventory}
spec:
  scaleTargetRef: {name: seat-inventory}
  minReplicaCount: 3                  # one per zone on a normal day
  maxReplicaCount: 24
  triggers:
  - type: cron                        # pre-scale for the 10:00 sale
    metadata:
      timezone: America/New_York
      start: "40 9 14 10 *"
      end: "30 10 14 10 *"
      desiredReplicas: "12"           # 8 for 6,000 holds/s + zone spare
  - type: prometheus                  # demand, measured at the gateway
    metadata:
      serverAddress: http://prometheus.monitoring:9090
      query: sum(rate(hold_requests_total[1m]))
      threshold: "750"                # per pod: 1,000 holds/s at 75%
```

25.14.4 The Lambda alternative, in Terraform

Bursty, short, idempotent work runs on Lambda: the ticket-PDF worker, the notification senders and the payment-webhook receiver, each fed by a queue and idle most of the day. The hold path stays on containers: warm pools, warm caches and known p99s matter more at 10:00 than paying nothing at 03:00, and the whole API on Lambda only pays off below about 37 requests a second per container (Chapter 15). A team without Kubernetes can run the same images on ECS with Fargate, with the cron pre-scale as a scheduled scaling action.

The Terraform fragment below declares the PDF queue with its dead-letter queue, wires it to the function with partial-batch failures, and creates the Valkey cluster for holds: three shards, each with a replica in another zone. It passes `terraform fmt` and `terraform validate` against the AWS provider, with stubs for the function, the dead-letter queue and the subnet group. The visibility timeout is six times the function timeout, as the Lambda documentation advises, so a batch still running is not delivered twice; `maximum_concurrency` caps the function well above the 17 environments the peak needs, so a runaway backlog cannot exhaust the account's concurrency or the e-mail quota:

```
resource "aws_sqs_queue" "pdf" {
  name                       = "encore-ticket-pdf"
  visibility_timeout_seconds = 180 # at least 6 x the function timeout
  redrive_policy = jsonencode({
    deadLetterTargetArn = aws_sqs_queue.pdf_dlq.arn # a second queue
    maxReceiveCount     = 5
  })
}
resource "aws_lambda_event_source_mapping" "pdf" {
  event_source_arn        = aws_sqs_queue.pdf.arn
  function_name           = aws_lambda_function.ticket_pdf.arn # timeout 30
  function_response_types = ["ReportBatchItemFailures"]
  scaling_config {
    maximum_concurrency = 50 # 17 needed at the peak
  }
}
resource "aws_elasticache_replication_group" "holds" {
  replication_group_id       = "encore-holds"
  description                = "Seat holds and the waiting room"
  engine                     = "valkey"
  node_type                  = "cache.r7g.xlarge"
  cluster_mode               = "enabled"
  parameter_group_name       = "default.valkey9.cluster.on"
  num_node_groups            = 3 # shards
  replicas_per_node_group    = 1 # each in another zone
  automatic_failover_enabled = true
  multi_az_enabled           = true
  transit_encryption_enabled = true
  subnet_group_name          = aws_elasticache_subnet_group.data.name
}
```

25.14.5 The CI pipeline

Every pull request runs the race test against real Postgres and Valkey containers; every merge to `main` builds, scans and ships one image by digest through staging, a flash-sale load test and a canary. AWS credentials come from OIDC federation, so the repository stores no keys:

```
name: seat-inventory
on:
  pull_request: {paths: ["services/seat-inventory/**", "libs/**"]}
  push: {branches: [main]}
jobs:
  test:                                # real stores for the race test
    runs-on: ubuntu-latest
    services:
      postgres: {image: "postgres:17", env: {POSTGRES_PASSWORD: test},
                 ports: ["5432:5432"]}
      valkey: {image: "valkey/valkey:9", ports: ["6379:6379"]}
    steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with: {python-version: "3.12"}
    - run: pip install -r services/seat-inventory/requirements-dev.lock
    - run: ruff check . && pytest services/seat-inventory -q
  ship:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    permissions: {id-token: write, contents: read}   # OIDC, no AWS keys
    steps:
    - uses: actions/checkout@v4
    - uses: aws-actions/configure-aws-credentials@v4
      with: {role-to-assume: "${{ vars.DEPLOY_ROLE }}", aws-region: us-east-1}
    - run: make image scan push        # build, Trivy scan, push by digest
    - run: make deploy ENV=staging     # terraform apply, digest to overlay
    - run: make loadtest ENV=staging   # 1/10-scale flash sale, SLO gate
```

Production is a protected environment with a manual approval, then a 5 percent canary per region that is promoted only while the burn rate stays low (Chapter 23). Coding agents fit into this loop as one more contributor: they open pull requests, and the same tests, plans and canaries judge their changes; *Designing Agentic Systems* covers that workflow.

25.15 Numbers to remember

Figure	Value (illustrative)
Hold attempts at the door	30,000/s
Waiting room	1,000 fans/s; 1 M fans in about 17 min
Seat inventory	6,000 holds/s tested, 3,000 run, 12 pods
Seat map	6.25 KB bitmap, 200,000 reads/s, 10 Gbit/s, 1 s TTL
Stadium sale	50,000 seats, 20,000 orders, 83 tickets/s, 33 orders/s
Sale error budget at 99.95%	900 bad of 1.8 M hold attempts
Infrastructure per sale	about $240, 0.5 cents a ticket

25.16 Pitfalls

- Testing holds one fan at a time -> a barrier race test with an injected gap, against the real stores, in CI.
- Letting Redis decide a sale -> Redis filters, the durable conditional write decides, a unique index backs it up.
- Counting `409` as an error -> a conflict is a correct answer; bad events are 5xx and slow answers.
- Scaling on CPU for a sale at a known time -> a cron pre-scale at 09:40 plus a demand metric, under one autoscaler.
- Publishing events with a second write after the commit -> the outbox row in the same transaction.
- Promoting the other region's inventory automatically -> one writable home region per event, promoted by runbook.

25.17 Remember

- Requirements and estimates find the hard part: for Encore it is contention on one event's 50,000 rows, not data size.
- Shrink the problem in stages: the CDN takes reads, the waiting room meters fans, Redis rejects losers, Postgres records winners.
- Prove the invariant with a test that fails on the bug: eight winners before, one after each of the three fixes.
- Call synchronously only where a fan waits; carry every consequence through the outbox and queues.
- A design is finished when it is code: a repository, an image, manifests, Terraform, and a pipeline that runs the race test on every change.

26 Hard problems by service, and how to solve them

Every design question hides a second question. “Design a ticketing site” is really “how do you never sell a seat twice while a million fans try at once”; “design a chat app” is really “how does a message reach every device once, in order, after the phone was offline”. Interviewers and production incidents rarely ask you to implement an algorithm; they ask what breaks in *this kind of service*, and how you would stop it. This chapter answers that question for twenty-one kinds of service. It opens with one method that works on any challenge, then gives each service a table of its hard questions (why each is hard, how to solve it, where this book teaches the mechanism) and one worked example solved step by step, mostly on Encore. It closes with a one-page matrix of service against challenge type. The mechanisms are taught elsewhere; this chapter is the map that tells you which one to reach for.

26.1 The picture

A good building inspector walks into a hospital, a stadium or a bank and asks the same five questions: what must never happen here, where do people crowd, how many of them, what holds the door shut, and what happens when the power fails. The buildings differ; the questions do not. The same is true of services: a payment service and a chat gateway fail in different places, but you find those places the same way.

```
service type
    |
    v
1 invariant    what must never happen?     "never sell a seat twice"
    |
2 pressure     where do requests collide?  one lock per event
    |          or where can a step fail?   provider call times out
    v
3 numbers      how hard is it?             rho = 3,000/s x 1 ms = 3
    |
4 mechanism    what enforces it?           conditional write per seat
    |
5 failure      what if it half-happens?    retry with the same key,
               and the retry?              expiry releases the hold
```

26.2 A method for any challenge

Five steps, always in this order. Say them out loud in an interview; they turn a vague “what about scale?” into a specific answer.

1. **Name the invariant.** One sentence that must stay true whatever happens: never oversell a seat, never charge a card twice, never lose a paid order’s tickets, never show a fan another fan’s order. If you cannot name one, the service probably tolerates staleness, and that is an answer too (Chapter 1).
2. **Find the pressure point.** Either a *contention point*, where many requests touch the same thing (one row, one key, one partition, one connection pool), or a *failure point*, where a step can half-happen (a call that times out, a worker that crashes after the side effect but before the acknowledgement).
3. **Quantify it.** Turn the pressure into a number: requests per second per key, fan-out per write, bytes per second, cost per request. The numbers decide whether the naive design is fine; often it is.
4. **Pick the mechanism.** Choose the smallest thing that enforces the invariant at that rate: a constraint, a conditional write, a single writer per key, a queue, a cache, a split key.
5. **Handle the failure and the retry.** Ask what happens if the step fails halfway, and what happens when the caller retries. Every distributed operation is retried somewhere, so every one must be safe to repeat.

Two equations do most of the quantifying. A key that serializes its writers (a lock, a single row, a single Redis key) is busy for a fraction

$$\rho_{\text{key}} = \lambda_{\text{key}} \quad t_{\text{hold}}$$

of the time, where λ_{key} is the request rate on that key and t_{hold} how long each request holds it. At $\rho \geq 1$ the queue on that key grows without bound; aim well below 1, because waiting time climbs steeply as ρ approaches it (Chapter 2). Encore's waiting room admits about 1,000 fans per second, who make about 3,000 hold attempts per second (30,000 per second arrive at the door). With an illustrative 1 ms critical section, one lock per event gives $3,000 \times 0.001 = 3$: impossible even behind the waiting room. One lock per section (100 sections, if the attempts spread evenly) gives $30 \times 0.001 = 0.03$: fine. A conditional write per seat removes the shared lock altogether. The second equation is retry amplification: if each of L layers makes up to a attempts, the deepest dependency sees up to

$$A = a^{L} \qquad 3 \text{ attempts}, 3 \text{ layers}: A = 27$$

times the original load exactly when it is weakest (Chapter 11).

Most challenges fall into nine types, and each type has a short list of standard answers:

Challenge type	The question	Standard mechanisms
Consistency	Who sees what, when?	single leader, quorums, read-your-writes
Contention	Many writers, one thing	conditional writes, split keys, queues
Hot spots	One key gets most traffic	cache, salting, local aggregation
Ordering	Must B follow A?	partition key, sequence numbers
Duplicates	What if it runs twice?	idempotency keys, unique constraints
Fan-out	One write, many readers	push vs pull, hybrid, pub/sub
Latency	Is the slow path on the hot path?	async, precompute, edge, parallel calls
Cost	What does one request cost?	caching, batching, tiered storage
Security	Who may do this?	authn, ownership checks, least privilege

Step 5 needs the same tool so often that it is worth having in your hands. This helper runs a side effect at most once per key and replays the stored result to every retry:

```
import json
import fakeredis

r = fakeredis.FakeRedis(decode_responses=True)

def once(key: str, ttl_s: int, work):
    """Run work() at most once per key; replay the stored result."""
    if r.set(f"idem:{key}", "running", nx=True, ex=ttl_s):
        try:
            result = work()
        except Exception:
            r.delete(f"idem:{key}")          # let a retry try again
            raise
        r.set(f"idem:{key}", json.dumps(result), ex=ttl_s)
        return result
    stored = r.get(f"idem:{key}")
    if stored == "running":
        raise RuntimeError("in progress, retry later")   # HTTP 409
    return json.loads(stored)

calls = []
def send_tickets():
    calls.append(1)
    return {"order": "o-77", "emailed": True}

print(once("tickets:o-77", 86_400, send_tickets))   # runs the work
print(once("tickets:o-77", 86_400, send_tickets))   # replays the result
print(len(calls))                                   # 1
```

Redis here is the fast filter, not the guarantee: its replication is asynchronous, so a failover can forget a key. Where a duplicate costs money, back it with a unique constraint in the database that owns the record (Chapter 3, Chapter 5).

26.3 API gateway and edge

The front door: every request passes through it, so its problems are about protecting what lies behind it. *50 System Designs* #40 (Load Balancer and API Gateway) and #2 (Rate Limiter) are the case studies.

Challenge question	Why it is hard	How to solve it	Chapter
Stop one client starving the rest?	limits must hold across many pods	token buckets in Redis, 429 + Retry-After	Chapter 11
A backend slows down: then what?	retries multiply load, threads pile up	deadlines, retry budgets, circuit breakers	Chapter 11
A million fans in ten minutes?	demand far exceeds capacity	waiting room, signed token checked at the edge	Chapter 10
Change routing safely?	one config fronts every service	staged config rollout, canary gateway pods	Chapter 23
Where does trust begin?	TLS ends before the services	TLS at the edge, mTLS inside, auth at the gateway	Chapter 22

Worked example: the retry storm. At 10:01 the seat-inventory database slows down. The mobile app retries three times, the gateway retries three times, and the inventory service retries its query three times. (1) *Invariant*: a slow dependency must not be turned into a dead one. (2) *Pressure*: the database at the bottom of three retrying layers. (3) *Numbers*: $A = 3^3 = 27$, so the 3,000 hold attempts per second the waiting room admits can become up to 81,000 queries per second on a database that was already struggling (and an open door without the waiting room, 30,000 per second, would become 810,000). (4) *Mechanism*: retry at one layer only (the client, with exponential backoff and jitter, because it alone can show "try again" to a human); every other layer fails fast within a deadline propagated from the edge (gRPC deadlines, or a header the gateway sets). Add a retry budget, retries at most 10 percent of requests, which caps amplification at 1.1, and a circuit breaker that answers "busy, trying again in a few seconds" at once while the database recovers. (5) *Retry safety*: retry only idempotent requests or those carrying an `Idempotency-Key`, so a retried hold never takes two seats. The trade-off is that some requests fail that a retry would have saved; that is the price of keeping the database alive.

26.4 Authentication and sessions

Authentication is a read-heavy service with one rare, expensive write (the login) and one hard operation (taking access away). The mechanisms live in Chapter 22; the design questions are about state and failure.

Challenge question	Why it is hard	How to solve it	Chapter
Revoke a stateless token?	a JWT is valid until it expires	5 to 15 minute access tokens, rotating refresh tokens, `jti` denylist	Chapter 22
Credential stuffing?	attackers hold real leaked passwords	limits per IP and account, breached-password check, passkeys	Chapter 11
Where do sessions live?	any pod may serve the next request	Redis session store or signed tokens	Chapter 7
Survive a region failover?	session state is regional	keys verifiable everywhere, replicated refresh state	Chapter 16
Why is login slow at peak?	argon2 and bcrypt are slow on purpose	size CPU for login bursts, keep tokens long enough	Chapter 2

Worked example: a failover that logs nobody out. (1) *Invariant*: a signed-in fan stays signed in through a regional failover, and a revoked session stays revoked. (2) *Pressure*: session state in `us-east-1` only. (3) *Numbers*: if the region fails during the flash sale and 1,000,000 fans must log in again, at an illustrative 50 ms of CPU per password hash that is 50,000 core-seconds, about 833 cores busy for a whole minute, at the worst possible moment. (4) *Mechanism*: sign access tokens with an asymmetric key (ES256) and publish the public keys (a JWKS document) in both regions, so either region verifies a token without any lookup; keep refresh tokens in a table replicated to both regions (a DynamoDB global table, or Aurora Global Database). (5) *Failure and retry*: replication is asynchronous, so a refresh rotated in the last second before failover may be missing in the new region. Accept the *previous* refresh token once within a short grace window; outside it, reuse of an old token means theft, so revoke the whole token family. The grace window is a small, bounded replay risk bought for a failover that 1,000,000 fans never notice.

26.5 User profiles

Profiles look trivial, a row per user, until you meet the invariants that span all users and the readers that every other service sends.

Challenge question	Why it is hard	How to solve it	Chapter
Unique e-mail or username?	uniqueness spans all users and regions	one authority: unique index or conditional put	Chapter 13
"My edit vanished on refresh"?	replica lag and cached copies	read-your-writes: primary after own write, versioned keys	Chapter 16
Every service reads profiles?	hot read path, fan-in from many callers	cache-aside; small stable fields as token claims	Chapter 9
"Delete my account"?	copies live in many stores	deletion events, crypto-shredding, retention limits	Chapter 22

Worked example: one account per e-mail in two regions. (1) *Invariant*: no two accounts share an e-mail. (2) *Pressure*: both regions accept sign-ups. In an active-active store such as a DynamoDB global table, which replicates asynchronously and by default resolves conflicts by last writer wins, two sign-ups for the same address in two regions both succeed locally, and one silently overwrites the other. (3) *Numbers*: 20 million users gathered over, say, five years is about 11,000 sign-ups a day, or 0.13 per second. The rate is tiny, so sending every sign-up to one home region costs one extra cross-region round trip (about 90 ms from Frankfurt) on a once-in-a-lifetime action. (4) *Mechanism*: claim the address with a conditional write in that one region (an item `email#ana@example.com` written only if it does not exist) or a unique index on the primary Postgres, then create the profile. (5) *Failure and retry*: store the sign-up ID with the claim, so a retry by the same sign-up succeeds and a different sign-up is told the address is taken; a claim whose profile never appears is cleaned up after a timeout. The same shape, one authority for a global invariant and local replicas for everything else, fixes usernames, short-URL keys (*50 System Designs* #1, URL Shortener) and unique ID ranges (#21, Unique ID Generator).

26.6 Catalog and search

The catalog (events, venues, prices) is the source of truth; search is a copy of it shaped for queries. Almost every hard question is about keeping the copy right. See *50 System Designs* #22 (Web Search Engine) and #16 (Search Autocomplete).

Challenge question	Why it is hard	How to solve it	Chapter
Search shows a cancelled show?	the index is fed asynchronously	outbox or CDC into the index, a lag SLO	Chapter 14
Updates applied out of order?	parallel indexers race	key the stream by event, external versions	Chapter 17
Change the mapping, no downtime?	a new mapping needs a full rebuild	build a new index, replay, swap the alias	Chapter 13

Challenge question	Why it is hard	How to solve it	Chapter
Relevance, typos, autocomplete?	ranking mixes text and business	BM25 plus boosts, cached top-k per prefix	Chapter 19
One event page, a million fans?	a read storm on one item	CDN, short TTL, versioned URLs	Chapter 10

Worked example: the cancelled show that came back. (1) *Invariant*: the index never moves backwards; a cancelled show never reappears. (2) *Pressure*: event 42 changes price (version 7) and then is cancelled (version 8); two indexer workers pick up the two changes and the version-7 write lands last, resurrecting the show. (3) *Numbers*: 300,000 events a year with a few edits each is a trickle; the problem is ordering, not rate, so do not reach for more machines. (4) *Mechanism*: publish changes from the transactional outbox (or Debezium reading the Postgres WAL) to a Kafka topic keyed by `event_id`, so one partition, and so one worker, sees an event's changes in order; and write to Elasticsearch or OpenSearch with `version_type=external` and the row's version, so the engine rejects any write whose version is not higher than the stored one. Mark a cancelled show `status: cancelled` rather than deleting the document, so the version guard keeps working. (5) *Retry*: a replayed change returns a version conflict (HTTP 409), which the indexer counts as success. For a mapping change, build `events_v2` from a replay of the topic, then move the `events` alias to it in one `_aliases` call.

26.7 Inventory and booking

Encore's hot spot and the classic of the genre: *50 System Designs* #25 (Ticket Booking), #26 (Hotel Booking) and #34 (E-Commerce Checkout and Inventory). The whole design is worked in Chapter 25; here are the questions it must answer.

Challenge question	Why it is hard	How to solve it	Chapter
Never oversell?	3,000 holds/s admitted, 30,000 at the door	conditional write per seat (Lua or `UPDATE ... WHERE`)	Chapter 18
One event is the hot partition?	every fan wants the same show	key by `event_id#section`, salt the hottest	Chapter 17
Seat map for 200,000 readers/s?	reads collide with writes	1-bit-per-seat snapshot, cached about a second	Chapter 9
Abandoned holds?	fans leave mid-checkout	10-minute lease, one elected expirer	Chapter 18
At most 8 tickets per fan?	the invariant spans many seats	per-fan counter in the same atomic step	Chapter 12
Bots and fairness?	scripts outrun people	waiting room, per-account limits	Chapter 22

Worked example: the hold expires while the card is paying. (1) *Invariant*: a fan who paid owns the seat, and no seat is sold twice. (2) *Pressure*: the hold ends at 10:09:59; the provider's success webhook arrives at 10:10:01; in between the expirer freed the seat and another fan held it. (3) *Numbers*: the provider takes 1 to 30 seconds to confirm, so every checkout that starts in the last 30 seconds of its hold is at risk; if 1 percent of 50,000 checkouts do (illustrative), that is 500 unhappy fans per show. (4) *Mechanism*: when the fan presses Pay, freeze the hold with a conditional update that succeeds only if it is still this fan's live hold, and give it a payment deadline the expirer respects; ask the provider to *authorize* (not capture) the card; sell the seat; then capture.

```
-- Pay pressed: freeze my own live hold for the payment window
UPDATE seats SET state = 'paying', hold_until = :now + 180
 WHERE event_id = :e AND seat_id = :s AND hold_id = :h
   AND state = 'held' AND hold_until > :now;
-- Webhook: sell only if it is still my hold; 0 rows -> void the auth
UPDATE seats SET state = 'sold'
 WHERE event_id = :e AND seat_id = :s AND hold_id = :h
   AND state IN ('paying', 'sold');
```

5. *Failure and retry*: the expirer only frees `held` rows past their deadline, so a `paying` seat is safe; a seat stuck in `paying` after 180 seconds is resolved by asking the provider. The provider retries webhooks, and the second update matches the already-sold row, so it is idempotent. If the first update changed 0 rows, the hold was already lost: offer an equivalent seat or void the authorization, which costs the fan nothing, unlike a refund. The statements run as written on SQLite with named parameters; in Postgres use your driver's placeholders.

26.8 Payments and ledgers

Money has the strictest invariants and the least forgiving external dependency. See *50 System Designs* #24 (Payment System) and #44 (Digital Wallet); the provider integration is in Chapter 24.

Challenge question	Why it is hard	How to solve it	Chapter
Never charge twice?	clients and workers retry	an idempotency key per attempt, sent to the provider	Chapter 5
Did the timed-out call charge?	the outcome is unknown	write `pending` first, query by key, sweep	Chapter 24
Balances that always add up?	concurrent transfers, partial failures	double-entry ledger, append-only, entries sum to zero	Chapter 12
Provider and ledger disagree?	two systems of record	daily reconciliation against settlement reports	Chapter 24
Card data in scope?	PCI DSS applies to every system that touches it	tokenise through the provider's hosted fields	Chapter 22

> **Design question.** Encore calls the provider to charge order o-77 and the call times out after 10 seconds. Did the fan pay?
>
> Nobody knows yet, and the design must make that state explicit. Invariant: each attempt charges at most once and every charge is recorded. Numbers: at the peak of about 33 orders per second (83 tickets), an illustrative 0.1 percent timeout rate is about 2 unknown outcomes a minute, more than 100 in a busy hour, far too many for humans. Mechanism: before the call, insert a payment row in state `pending` with an idempotency key for this attempt (`pay:o-77:1`), and pass that key to the provider, which returns the original result for a repeated key instead of charging again. Retry with the *same* key and backoff; if retries keep failing, a sweeper looks the payment up by key or metadata, and the provider's webhook resolves it too, whichever comes first. On success append the double-entry rows in the same transaction that marks the payment `succeeded`. Mint a new key only when the fan deliberately pays again, and only after the first attempt is known to have failed. Nightly reconciliation against the provider's settlement report catches the rest. See also *50 System Designs* #24 (Payment System).

26.9 Orders and checkout

Checkout stitches inventory, payment, tickets and notifications into one user action with no shared transaction. See *50 System Designs* #34 (E-Commerce Checkout and Inventory).

Challenge question	Why it is hard	How to solve it	Chapter
Double-click on Buy?	two requests, one intent	client-generated order key, unique constraint	Chapter 5
One action, four services?	no transaction spans them	saga with compensations, orchestrated	Chapter 18
Paid, but no event published?	DB write and publish are not atomic	transactional outbox	Chapter 14
Webhooks late, twice, out of order?	the provider promises neither order nor once	state machine, dedup by event ID	Chapter 24

Challenge question	Why it is hard	How to solve it	Chapter
"My tickets" empty after paying?	the read hits a lagging replica	read-your-writes routing	Chapter 16

Worked example: the refund that arrived before the payment. (1) *Invariant*: an order moves only forward along allowed transitions (`paying -> paid -> ticketed -> refunded`), and a refunded order never becomes paid again. (2) *Pressure*: webhooks. Payment providers retry deliveries and do not promise order (Stripe's documentation says so explicitly), so `charge.refunded` can arrive before `payment_intent.succeeded`, and either can arrive twice. (3) *Numbers*: about 33 orders per second at the peak, each with three or four provider events, is about a hundred webhooks per second, easily handled by one service; the difficulty is correctness, not load. (4) *Mechanism*: record each provider event ID under a unique constraint and skip IDs already seen; apply a transition with a guarded update (`UPDATE orders SET state = 'paid' WHERE id = :o AND state = 'paying'`); when an event does not fit the current state, do not guess: fetch the payment object from the provider's API, which is the source of truth, and move the order to the state it reports. (5) *Retry*: acknowledge the webhook with 2xx only after the event row is committed, and put the slow work (issuing tickets, e-mail) in the outbox in that same transaction, so a crash at any point leads to a redelivery that is harmless.

26.10 Notifications

Notifications are a fan-out through queues to external providers that have their own limits and failures. See *50 System Designs* #20 (Notification System) and #42 (Email Service).

Challenge question	Why it is hard	How to solve it	Chapter
Send exactly once?	queues and providers deliver at least once	dedup key per user and message	Chapter 14
Provider down or throttling?	an external quota you do not control	queue, backoff, fallback provider, breaker	Chapter 24
50,000 messages at 18:00?	a burst far above the provider rate	rate-limited workers, a shared token bucket	Chapter 11
Opt-outs and quiet hours?	preferences change after enqueue	check preferences at send time	Chapter 6
Did it arrive?	delivery is asynchronous	delivery webhooks, status per channel	Chapter 24

Worked example: "doors open" to 50,000 ticket holders. (1) *Invariant*: every ticket holder who has not opted out gets the push once. (2) *Pressure*: the campaign job and its workers can crash and re-run, so messages repeat; the push provider throttles bursts. (3) *Numbers*: at an illustrative quota of 1,000 sends per second, 50,000 pushes take 50 seconds, so "at 18:00" means "between 18:00:00 and 18:00:50"; 50,000 dedup keys of about 100 bytes are 5 MB in Redis. (4) *Mechanism*: the campaign job enqueues one message per fan, in batches, each with the key `doors:{event}:{user}`; a worker claims the key with the `once()` pattern above, reads the fan's preferences, takes a token from the shared bucket and sends with a collapse key (APNs `apns-collapse-id`, FCM `collapse_key`), so a rare duplicate replaces the first notification instead of stacking beside it. (5) *Failure and retry*: claim with a short TTL (60 seconds) so a worker that crashes mid-send lets the message retry instead of losing it; back off on throttling errors; delete a device token the provider reports as invalid instead of retrying it; route the rest to a dead-letter queue. The choice is deliberate: for a reminder, a rare duplicate is better than a miss.

26.11 Chat and presence

Real-time services keep a connection open to every online device, so state lives in the gateway layer and the questions are about order, delivery and reconnection. See *50 System Designs* #6 (Chat System), #7 (Messenger) and #32 (Live Streaming and Live Comments).

Challenge question	Why it is hard	How to solve it	Chapter
Messages in order?	many senders, many servers	one sequencer per conversation	Chapter 18
Offline phones, many devices?	devices come and go	durable inbox, sync from last sequence	Chapter 14
Millions of open connections?	gateways become stateful	connection registry in Redis, drain on deploy	Chapter 4
Presence for 10 million online?	heartbeats are a write storm	in-memory TTL keys, push changes to watchers only	Chapter 9
Group of 1,000 members?	every message fans out	fan out per online member, pull for the rest	Chapter 14

Worked example: once and in order on every device. (1) *Invariant*: every device shows each message once, in the same order. (2) *Pressure*: the sender's acknowledgement is lost, so it retries; the recipient's phone drops in a tunnel and reconnects to a different gateway. (3) *Numbers*: even a busy 1,000-member group sends a few messages per second, so a single writer per conversation is idle most of the time ($\rho \quad 1$), and a single writer is the simplest way to get one order. Presence is the real load: 10 million online users with 30-second heartbeats are about 333,000 writes per second, which belong in memory with TTLs, never in the message store. (4) *Mechanism*: the sender attaches a client message ID; the conversation's owner (the leader of the partition keyed by `conversation_id`) ignores an ID it has already seen, assigns `seq = last + 1`, appends to the conversation log and pushes to recipients' gateways. Each device remembers the highest contiguous `seq` it holds; on reconnect it asks for "conversation c after 1041", and a gap in the numbers triggers the same fetch. (5) *Retry*: a retried send returns the original `seq`, so it is harmless. Encore's support chat uses the same idea over SSE, where the browser's `Last-Event-ID` header tells the server where to resume.

26.12 News feeds and timelines

A feed is a fan-out problem: one write must appear in many readers' timelines, or many writers' posts must be merged at read time. See *50 System Designs* #5 (News Feed), #19 (Photo Sharing) and #47 (Recommendation System).

Challenge question	Why it is hard	How to solve it	Chapter
A post from a 5-million-follower account?	push means 5 million writes	hybrid: push for most, pull for celebrities	Chapter 14
Feed reads in milliseconds?	merging many sources is slow	precomputed timelines of post IDs in Redis	Chapter 9
Deleted or blocked posts?	copies sit in millions of timelines	store IDs only, filter when hydrating	Chapter 9
Ranked, not chronological?	scoring per read is expensive	candidates, then a ranking service, cached per user	Chapter 21

Worked example: an artist announces a tour. Encore's home screen shows news from artists a fan follows. (1) *Invariant*: followers see the announcement within a minute, and no feed read takes longer than about 200 ms. (2) *Pressure*: one artist has 5,000,000 followers. (3) *Numbers*: fan-out on write at an illustrative 50,000 timeline writes per second takes 100 seconds and blocks every other artist's posts meanwhile; a typical artist with 2,000 followers takes 40 ms. (4) *Mechanism*: push for artists below a threshold (say 100,000 followers): a worker appends the post ID to each follower's timeline, a Redis sorted set capped at a few hundred entries, and only for followers active in the last 30 days. Pull for the few celebrity artists: at read time, merge the fan's timeline with the recent posts of the celebrities they follow, which are a handful of hot keys served from cache. Hydrate post IDs into posts at read time, so an edit or deletion shows everywhere at once. (5) *Retry*: `ZADD timeline:{fan} {time} {post}` is idempotent, so a fan-out batch that is redelivered writes the same members again and changes nothing.

26.13 Media upload and processing

Media services move big, untrusted files and run slow, failure-prone processing on them. See *50 System Designs* #8 (Video Platform), #19 (Photo Sharing) and #9 (Streaming Service).

Challenge question	Why it is hard	How to solve it	Chapter
2 GB over a flaky phone?	long transfers drop	multipart upload, resume per part	Chapter 12
Keep bytes off the API servers?	bandwidth and cost	presigned URLs straight to object storage	Chapter 12
Transcode, thumbnail, scan?	slow, CPU-heavy, can fail	event -> queue -> idempotent workers	Chapter 14
Malicious files?	uploads are untrusted input	quarantine bucket, scan, then publish	Chapter 22
Serve it to millions?	egress cost and latency	CDN, adaptive bitrate renditions	Chapter 10

Worked example: an organizer's 2 GB promo video. (1) *Invariant*: an accepted upload is never lost, and a half-processed video is never shown. (2) *Pressure*: a phone on a train, and transcoders that crash. (3) *Numbers*: 2 GB at 10 Mbit/s takes about 27 minutes; in 16 MiB parts that is 128 parts, so a dropped connection costs one part (about 13 seconds) instead of a restart. S3 allows parts from 5 MiB to 5 GiB and at most 10,000 parts per upload. (4) *Mechanism*: the API records the upload, starts an S3 multipart upload (GCS resumable uploads, Azure block blobs) and returns presigned URLs per part; the phone uploads parts in parallel and completes the upload; the S3 event notification goes through SQS to transcode workers (or AWS Elemental MediaConvert), which write renditions under keys derived from the source object's version, and a final conditional update flips the video from `processing` to `ready`. (5) *Failure and retry*: S3 notifications are at least once, but a duplicate event regenerates the same output keys and the status flip happens once; a lifecycle rule (`AbortIncompleteMultipartUpload` after 7 days) deletes the parts of uploads nobody finished, which otherwise cost storage forever.

26.14 Counters, likes and view counts

Counters look like `x += 1`. At scale they are the purest hot-key problem in this book. See *50 System Designs* #50 (Like and View Counter at Scale) and #28 (Ad Click Aggregator).

Challenge question	Why it is hard	How to solve it	Chapter
200,000 increments/s on one key?	one key serializes its writers	local aggregation, sharded counter keys	Chapter 17
One like per user?	retries and double taps	unique (user, item) row; the count is derived	Chapter 12
Exact or approximate?	exact unique counts cost memory	HyperLogLog, 0.81% standard error in Redis	Chapter 19
Survive crashes?	in-memory buffers lose increments	short flush windows, events in a stream	Chapter 14

Worked example: views of the stadium show's page. (1) *Invariant*: the displayed count is close (within a fraction of a percent) and never goes backwards; nobody needs it exact. (2) *Pressure*: one Redis key, `views:42`, during the flash sale. (3) *Numbers*: at an illustrative 10 microseconds of server time per `INCR`, 200,000 increments per second give $\rho = 2$ on one key: saturated. (4) *Mechanism*: each of 500 pods counts in memory and flushes `INCRBY views:42 n` once a second, which is 500 writes per second, $\rho = 0.005$. When even the flushes are too many, split the key into `views:42:{0..15}` and sum the 16 keys on read (or cache the sum for a second). For unique visitors use `PFADD` and `PFCOUNT`. (5) *Failure and retry*: a crashed pod loses at most one second of its share, about 400 views, acceptable for a display counter; do not retry a timed-out `INCRBY` blindly, because it may have applied. Where money depends on the count (ad clicks), count from the event stream with deduplication instead (#28).

26.15 Analytics and metrics

Analytics and monitoring ingest far more data than the product itself and must stay correct when events arrive twice, late or with too many labels. See *50 System Designs* #29 (Metrics and Monitoring), #28 (Ad Click Aggregator) and #48 (Distributed Logging).

Challenge question	Why it is hard	How to solve it	Chapter
1 billion events a day?	11,574/s on average, bursts above	collectors -> Kafka or Kinesis -> Parquet	Chapter 21
Duplicates and late events?	phones retry and reconnect hours later	event IDs, event-time windows, watermarks	Chapter 21
Metrics cardinality explosion?	series multiply by label values	bounded labels; per-user detail in traces and logs	Chapter 23
Keep a year cheaply?	raw resolution grows forever	downsampling, tiered storage	Chapter 21

Worked example: the `user_id` label. A developer adds `user_id` to the request-latency metric "to debug one fan's slow checkout". (1) *Invariant*: the monitoring system must stay up when the product is in trouble, because that is when it is needed. (2) *Pressure*: every distinct combination of label values is a separate time series with its own memory and index entry. (3) *Numbers*: 20 endpoints × 5 status classes × 500 pods is 50,000 series, comfortable; multiply by 4,000,000 daily users and it is 2×10^{11}, which no Prometheus (or managed equivalent: Amazon Managed Service for Prometheus, Google Cloud Managed Service for Prometheus, Azure Monitor) will hold. (4) *Mechanism*: label metrics only with bounded dimensions (endpoint, status class, region, pod); answer per-user questions with traces, linked from the latency histogram by exemplars, and with structured logs in the lakehouse. (5) *Failure*: guard with per-scrape `sample_limit`; Prometheus then fails the whole scrape of an offending target rather than ingesting it, so alert on failed scrapes, and review new labels in code review like schema changes.

26.16 Leaderboards and ranking

A leaderboard asks for order over millions of entries, updated live. See *50 System Designs* #31 (Leaderboard) and #18 (Discussion Forum) for hot ranking.

Challenge question	Why it is hard	How to solve it	Chapter
Rank of any of 20 million users?	rank needs order over all entries	Redis sorted set, `ZREVRANK` in $O(\log n)$	Chapter 9
Ties?	equal scores order by member name	composite score: points, then who got there first	Chapter 19
Too big or too hot for one node?	memory and write rate	shard by score range, or approximate rank from a histogram	Chapter 17
Weekly reset?	rebuilding is costly	one key per period with a TTL	Chapter 9

Worked example: Encore's fan-loyalty board. Fans earn points for tickets bought; the app shows the top 100 and "you are number 48,213". (1) *Invariant*: ranks are exact, update within a second, and ties go to whoever reached the score first. (2) *Pressure*: rank queries from millions of app opens. (3) *Numbers*: 20 million entries at an illustrative 100 bytes each is about 2 GB, so one Redis node holds the board; a sorted set is a skip list, so a rank costs about $\log_2(2 \times 10^7) \approx 24$ steps. Points change only on purchase, at most 83 times a second. (4) *Mechanism*: encode ties into one number, score $= p \times 10^{10} + (10^{10} - t)$, with p the points and t the seconds since the programme began; a double holds integers exactly up to 2^{53}, so this works for fewer than 900,000 points. Cache the top 100 (`ZREVRANGE board 0 99`) for a second; serve "my rank" with `ZREVRANK` directly. (5) *Retry*: `ZINCRBY` is not idempotent, so a retried increment double-counts; instead compute the fan's total from the orders database and write it with `ZADD`, which sets the same score however often it is repeated, and rebuild the whole board from the database if Redis is lost.

26.17 Schedulers and background jobs

Schedulers and job runners turn "at 10:00" and "eventually" into work that runs once, on time, despite crashes. See *50 System Designs* #30 (Distributed Job Scheduler) and #46 (Online Judge).

Challenge question	Why it is hard	How to solve it	Chapter
Run each job exactly once?	workers crash mid-job, leases expire	at-least-once delivery, idempotent job bodies	Chapter 18
A single point of failure?	one scheduler process	leader election, or many workers polling with `SKIP LOCKED`	Chapter 18
Thousands due in the same second?	a burst on the due-time index	durable job table, pre-scaling ahead of the burst	Chapter 14
Jobs that last hours?	state across retries and deploys	a workflow engine (Temporal, AWS Step Functions)	Chapter 14

Worked example: opening sales at 10:00:00. (1) *Invariant*: every on-sale opens on time, once, even if a scheduler dies at 09:59:59. (2) *Pressure*: Friday mornings, when hundreds of on-sales fall on the same second. (3) *Numbers*: three jobs per event (open sales, a 24-hour reminder, a post-show survey) for 300,000 events is 900,000 jobs a year, about 2,500 a day: a Postgres table holds this easily. (4) *Mechanism*: a `jobs` table indexed on `due_at`; several workers poll every second with `SELECT ... WHERE state = 'queued' AND due_at <= now() ORDER BY due_at LIMIT 50 FOR UPDATE SKIP LOCKED` (Postgres; MySQL 8 has the same clause), which hands each row to one worker without blocking the others, then mark it running with a lease. No leader election is needed because the row lock is the election. A second job at 09:50 pre-scales seat inventory and the waiting room. (5) *Failure and retry*: a worker that dies lets its lease expire and another runs the job again, so the body must be idempotent: `UPDATE events SET on_sale = true WHERE id = :e AND on_sale = false` opens the sale once however many times it runs.

26.18 File storage and sync

Sync services keep many devices' copies of the same files in agreement over unreliable networks. See *50 System Designs* #10 (Cloud File Storage), #11 (Collaborative Editor) and #37 (Distributed File System).

Challenge question	Why it is hard	How to solve it	Chapter
Upload only what changed?	big files, small edits	chunks named by content hash	Chapter 19
Two devices edit offline?	concurrent versions, no winner	commit with a base version; conflicted copy, or CRDTs for text	Chapter 16
The same file stored a million times?	duplicates waste storage	content-addressed chunks with reference counts	Chapter 12
Tell the other devices?	many devices per user	change cursor, long poll or push	Chapter 4

Worked example: an organizer edits a 1 GB venue file on two laptops. (1) *Invariant*: no edit is silently lost. (2) *Pressure*: the second laptop was offline and edited the same file. (3) *Numbers*: in illustrative 4 MB chunks a 1 GB file is 250 chunks; a 1 MB edit in place changes one chunk (two if it straddles a boundary), so the sync sends 4 MB, not 1 GB. An *insertion* shifts every later byte, and fixed-size chunks would all change; content-defined chunking (boundaries chosen by a rolling hash of the content) keeps most boundaries where they were. (4) *Mechanism*: a file version is a manifest, the list of chunk hashes. A device uploads the missing chunks (a `PUT` keyed by hash, skipped if the chunk exists) and then commits the new manifest *with the version it started from*; the server accepts the commit only if that base is still current, a compare-and-set. The offline laptop's commit fails, so the client saves its version as a "conflicted copy" next to the original, which is what Dropbox does, instead of overwriting. (5) *Retry*: chunk uploads and a repeated commit of the same manifest are idempotent, so a flaky connection retries freely.

26.19 Location and proximity

Proximity queries ask a two-dimensional question of one-dimensional indexes. See *50 System Designs* #27 (Proximity Search), #13 (Ride Hailing) and #14 (Maps and Navigation).

Challenge question	Why it is hard	How to solve it	Chapter
Events near me?	B-trees index one dimension	geohash or S2 cells, PostGIS GiST, Redis `GEOSEARCH`	Chapter 19
A neighbour across a cell edge?	close points, different prefixes	search the cell and its 8 neighbours, then filter by distance	Chapter 19
250,000 position updates/s?	the index churns constantly	in-memory index sharded by cell, positions with a TTL	Chapter 17
Dense cities, empty countryside?	fixed cells are uneven	quadtrees or adaptive cell levels	Chapter 19

Worked example: let the numbers choose the design. Encore shows "events near you" within 25 km. (1) *Invariant*: no event inside the radius is missing; order by distance. (2) *Pressure*: query rate, not data size. (3) *Numbers*: an illustrative 50,000 upcoming events at about 100 bytes each is 5 MB, and if 10 percent of the 6,000 requests per second at the daily peak ask, that is 600 queries per second. (4) *Mechanism*: one Redis geo set (`GEOSEARCH events FROMLONLAT 13.4 52.5 BYRADIUS 25 km ASC`) or a PostGIS `ST_DWithin` query on a GiST index, cached per geohash cell for a minute; nothing more. Now ask the same question of a ride-hailing service: 1,000,000 drivers reporting every 4 seconds is 250,000 writes per second of data that is stale within seconds. That design keeps positions in memory, sharded by city or cell, with a TTL so a silent driver disappears, and never puts a durable database on the update path. (5) *Failure*: a lost in-memory shard rebuilds itself within one reporting interval, because every driver reports again. Same question, opposite answers, decided by step 3.

26.20 Caches

Caches are in every design, and each brings the same four questions. The mechanisms are all in Chapter 9; see *50 System Designs* #4 (Distributed Cache).

Challenge question	Why it is hard	How to solve it	Chapter
Stale after a write?	a delete races a slow read	TTL as a bound, leases, CDC-driven invalidation	Chapter 9
One hot key?	one key overwhelms one node	in-process cache, key replicas, request coalescing	Chapter 9
Stampede on expiry?	many misses at once	rebuild lock, early refresh, jittered TTLs	Chapter 9
A cache node dies?	the database takes the misses	replicas, sharding, admission control at the database	Chapter 23

Worked example: the cache that was holding the database up. (1) *Invariant*: losing a cache node must not take the database down; the cache is an optimisation, never a dependency for correctness. (2) *Pressure*: the database is sized for the misses, not for the traffic. (3) *Numbers*: at the daily peak of 6,000 reads per second and a 95 percent hit ratio, the database serves 300 per second. Lose the whole cache and it receives 6,000, twenty times its normal load. Spread the cache over 10 shards and lose one: misses become $6,000 \times (0.05 + 0.95 \times 0.1) = 870$ per second, 2.9 times normal. (4) *Mechanism*: shard the cache (Redis Cluster; Amazon ElastiCache, or Memorystore on GCP and Azure Managed Redis) with a replica per shard and automatic failover, so a node loss costs seconds of misses on one shard; coalesce concurrent misses for the same key; and cap concurrent queries at the database so the excess gets a fast 503 instead of a slow collapse. (5) *Recovery*: a replacement node warms from its replica or from reads; capacity-test the database at the "one shard lost" load, not at 300 per second.

26.21 Queues and streams

Queues and logs absorb bursts and decouple services, and in exchange they deliver duplicates, reorder work and fall behind. The mechanisms are in Chapter 14; see *50 System Designs* #12 (Distributed Log).

Challenge question	Why it is hard	How to solve it	Chapter
A poison message blocks a partition?	an in-order consumer retries forever	bounded retries, then a dead-letter or parking topic	Chapter 14
Order per key, and parallelism?	one ordered stream is one consumer	partition by the key that needs order	Chapter 17
Duplicates?	at-least-once delivery	idempotent consumers, a dedup table	Chapter 14
Consumers fall behind?	the backlog compounds	scale on lag, up to the partition count	Chapter 14

Worked example: catching up after an outage. The clickstream consumer that loads the lakehouse was down for two hours. (1) *Invariant*: no event is lost, and the dashboards catch up before the evening shows. (2) *Pressure*: events keep arriving while the backlog drains. (3) *Numbers*: the backlog drains only at the *spare* capacity, so the catch-up time is

$$T_{\text{catch-up}} = \frac{B}{C - \lambda}$$

with backlog B, consumption capacity C and arrival rate λ. At 1 billion events a day, $\lambda \approx 11,574$ per second and two hours left $B \approx 83.3$ million. Twelve consumers at an illustrative 2,000 events per second each give $C = 24,000$ and $T \approx 6,706$ s, almost two more hours; twenty-four give $C = 48,000$ and $T \approx 2,288$ s, 38 minutes. (4) *Mechanism*: autoscale consumers on lag (KEDA, or Lambda event source mappings), but a classic Kafka consumer group uses at most one consumer per partition, so 24 consumers need at least 24 partitions, chosen when the topic was created; and the sink must absorb 48,000 writes per second too. (5) *Retry*: replayed events carry their event IDs, so the load deduplicates on the way into the lakehouse.

26.22 The database layer

Under every service sits a database that must change while it serves traffic. The operational questions are often harder than the modelling ones; see *50 System Designs* #3 (Distributed Key-Value Store) and #38 (Globally Distributed Database).

Challenge question	Why it is hard	How to solve it	Chapter
Change the schema, no downtime?	DDL locks; old and new code run together	expand and contract	Chapter 23
Backfill 50 million rows?	long transactions, replica lag	small batches by key range, throttled on lag	Chapter 16
A hot partition?	one key takes the load	re-key, split, cache in front	Chapter 17
Connections run out?	pods × pool > `max_connections`	a pooler (PgBouncer, RDS Proxy), sized by Little's law	Chapter 4

Worked example: a new required column on orders. Encore adds `buyer_country`, derived from the billing address, and wants it `NOT NULL`. (1) *Invariant*: checkout never stops, and old and new application versions both work throughout. (2) *Pressure*: a table rewrite or a long lock on the orders table, and replica lag from a giant update. (3) *Numbers*: 50 million rows in batches of 5,000 every half second is 10,000 rows per second, about 83 minutes. (4) *Mechanism*, in Postgres: *expand* with `ALTER TABLE orders ADD COLUMN buyer_country text` (no rewrite); deploy code that writes it on every new order; backfill with `UPDATE ... WHERE id BETWEEN :a AND :b AND buyer_country IS NULL`, pausing whenever replica lag passes a few seconds; add `CHECK (buyer_country IS NOT NULL) NOT VALID`, then `VALIDATE`

`CONSTRAINT`, which scans without blocking writes; then `SET NOT NULL`, which since Postgres 12 uses the validated constraint instead of scanning again. *Contract* last: delete the old code path that tolerated nulls. (5) *Retry*: each batch is idempotent because of `IS NULL`, so a backfill that dies resumes from where it stopped.

26.23 AI agents and LLM services

An LLM-backed service is a slow, expensive, non-deterministic dependency whose output can trigger actions. The architecture is in Chapter 24; *Designing Agentic Systems* covers agents in depth.

Challenge question	Why it is hard	How to solve it	Chapter
Cost per conversation?	tokens × calls × users	budgets, small models for routing, prompt caching	Chapter 24
Multi-step latency?	model calls run in sequence	stream, parallel tool calls, fewer steps	Chapter 24
Tool safety?	model output is untrusted input	scoped credentials, idempotent tools, human approval	Chapter 22
Is it any good?	answers vary run to run	offline evaluation sets, sampled review, release gates	Chapter 23
Provider limits or outage?	quotas counted in tokens	model gateway, token buckets, fallback models	Chapter 11

> **Design question.** What will Encore's support agent cost, how fast will it answer, and how do you keep it from refunding the wrong order?
>
> Numbers, all illustrative: 1 percent of 4 million daily users is 40,000 conversations a day, each with 6 model calls of 4,000 input and 300 output tokens. At $3 per million input tokens and $15 per million output, a call costs $0.0165, a conversation $0.099, a day $3,960, about $1.4 million a year. Cache the stable 3,000-token prefix (instructions and tool definitions) at a tenth of the input price and a call drops to $0.0084, about $2,016 a day. Six sequential calls at 1.5 s each is 9 s, too slow, so stream the final answer, run independent lookups in parallel, and send the commonest intent ("where are my tickets?") down a deterministic path with no model at all. Safety: the refund tool takes an idempotency key per conversation and order, checks that the order belongs to the signed-in fan, caps the amount, and sends anything above the cap to a human approval step. Quality: replay a fixed set of recorded conversations against every prompt or model change and block the release if it scores worse.

26.24 The challenge matrix

One page to revise from. Read a row as "when you design this service, these are the questions to expect"; read a column as "these services share this kind of problem, and so share its mechanisms". **X** marks the headline challenge, **x** a real one; a blank means it is rarely the hard part.

Service	Consistency	Contention	Hot spots	Order	Duplicates	Fan-out	Latency	Cost	Security
Gateway, edge			x		x		X	x	X
Auth, sessions	x						x	x	X
User profiles	X		x						x
Catalog, search	x		x	X			x		

Service	Consistency	Contention	Hot spots	Order	Duplicates	Fan-out	Latency	Cost	Security
Inventory, booking	X	X	X		x				x
Payments, ledgers	X				X		x		X
Orders, checkout	x			X	X				
Notifications					X	X		x	
Chat, presence				X	x	x	X		
Feeds, timelines			x			X	x	x	
Media					x		x	X	x
Counters		X	X		x				
Analytics, metrics				x	x			X	
Leaderboards	x	x	x	x			x		
Schedulers, jobs			x		X				
File sync	X				x			x	
Location		x	x				X		
Caches	X		X				x		
Queues, streams				X	X		x		
Database layer	x	x	X						
AI agents, LLMs					x		X	X	X

Three patterns stand out. Duplicates appear almost everywhere work crosses a network, so idempotency keys and unique constraints are the most reused tools in this book. Consistency and contention concentrate in the few services that own scarce things (seats, money, names), which is where to spend strong guarantees, single writers and careful failure handling; everything else can usually be eventually consistent. And latency and cost dominate at the two ends of the system, the edge and the model calls, where caching, streaming and doing less work pay most.

26.25 Numbers to remember

Fact	Value
Per-key utilization	$\rho = \lambda t$; keep well below 1
Retry amplification	a^L : 3 attempts, 3 layers = 27
Flash sale, at the door	30,000 hold attempts/s
Flash sale, admitted	about 1,000 fans/s, 3,000 holds/s
Unknown payment outcomes	0.1% of 33 orders/s = 2 a minute
Celebrity fan-out	5M followers at 50,000/s = 100 s
Local counter aggregation	200,000 incr/s -> 500 flushes/s

Fact	Value
Metrics series with `user_id`	50,000 -> 2×10^{11}
Catch-up time	$B/(C - \lambda)$: 38 min with 24 consumers
One of 10 cache shards lost	database load 2.9 times normal
Backfill, 50M rows	83 min at 10,000 rows/s
S3 multipart parts	5 MiB to 5 GiB, at most 10,000
Support agent (illustrative)	about $0.10 per conversation

26.26 Pitfalls

- Answering “how does it scale?” before naming the invariant -> state what must never happen, then design for it.
- Designing for the scary number without computing it -> quantify first; “events near me” and the job scheduler need no exotic machinery.
- Retrying at every layer -> retry at one layer with backoff, fail fast elsewhere, cap retries with a budget.
- Treating a timeout as a failure -> it is an unknown outcome; record `pending`, retry with the same key, reconcile.
- Redis as the only guard against duplicates -> back it with a unique constraint where a duplicate costs money.
- Last-writer-wins for a global invariant -> one authority for uniqueness, local replicas for the rest.
- Expiring a hold that is paying -> freeze it with a conditional update; authorize, sell, then capture.
- Non-idempotent updates under retries (`ZINCRBY`, `INCRBY`, “send e-mail”) -> absolute writes, dedup keys, or accept and bound the error.
- Unbounded metric labels -> bounded labels; per-user detail in traces and logs.
- Sizing the database for the cache hit ratio -> size it for losing a cache shard.
- Agents with broad tools and no budget -> scoped, idempotent tools, approval above a cap, cost per conversation on a dashboard.

26.27 Remember

- Every hard question follows one method: invariant, pressure point, numbers, mechanism, failure and retry.
- $\rho = \lambda t$ per key and a^L for retries settle most contention and overload arguments in one line.
- Duplicates are everywhere; idempotency keys, unique constraints and absolute writes are the universal fix.
- Spend strong consistency only where something scarce is owned: seats, money, names, job runs.
- Fan-out, hot keys and lag are numbers to compute, and the numbers often say the simple design is enough.
- Use the matrix to predict an interviewer’s follow-up questions, then answer them with the mechanism from the chapter the table points to.

27 The system design interview

A system design interview gives you about 40 minutes, a whiteboard and a one-line prompt such as “design a ticketing site”, and asks one question underneath: can this engineer turn an open problem into a sound, defensible design while thinking aloud with a colleague? This chapter is the method for that hour. It reuses the design sequence of Chapter 1, adds a clock, a rubric and a map from common prompts to the chapters of this book and to the case studies of *50 System Designs*, and ends with three mini interviews in dialogue form. The picture to hold on to is a flight plan: fixed legs, a fuel budget for each, and a captain who announces every change of course.

27.1 The picture

A pilot does not take off and improvise. She files a flight plan with legs (taxi, climb, cruise, descent), a fuel budget for each, and checkpoints where she confirms position with the tower. If weather forces a detour she says so on the radio, and the tower helps. A design interview is the same flight: the legs are the steps of the design method, the fuel is minutes, the tower is the interviewer, and the radio calls are you saying what you are doing and why.

```
min  0     5     9         15             23                 37   40
     |-----|-----|---------|--------------|------------------|----|
     | req | est | API+data| high-level   | deep dives       | TO |
     |     |     |  model  | design       | (one or two)     |    |
     +-----+-----+---------+--------------+------------------+----+
       ^ checkpoint: "Here is what I will build. OK to go on?"
                     ^ checkpoint: "These numbers say X is the hard part."
                                   ^ checkpoint: "Which part should I deepen?"
TO = trade-offs and wrap-up
```

Candidates who struggle usually know enough. What goes wrong is the flight: they spend 20 minutes on requirements, or draw boxes before they know the load, or dive into one component and never return to the whole. The flight plan prevents all three: it tells you, at every minute, which leg you should be on.

27.2 The 45-minute budget

System design rounds at Google and most large companies are typically 45 minutes; some run 60. Introductions at the start and your questions at the end take about five, so a 45-minute slot holds about 40 minutes of design. Split those 40 minutes by share, not by habit, so the same plan scales to any slot:

$$t_i = f_i \times (T_{\text{slot}} - T_{\text{overhead}}), \qquad \sum_i f_i = 1$$

Leg	Share f_i	45-minute slot	What must exist at the end
Requirements	12.5%	0 to 5	core flows, targets, out of scope
Estimates	10%	5 to 9	peak QPS, storage, the hot number
API and data model	15%	9 to 15	3 to 5 calls, entities and keys
High-level design	20%	15 to 23	one diagram covering every core flow
Deep dives	35%	23 to 37	one or two hard parts, solved
Trade-offs, wrap-up	7.5%	37 to 40	what you gave up, what you would add

Deep dives get the largest share because that is where seniority shows: anyone can draw a load balancer in front of servers; the signal is what you do with the one component that the numbers say will break. The plan as a script, so you can rehearse with a timer:

```python
# Minutes for each leg of the design, scaled to any slot.
STEPS = {                  # share of the design time
    "requirements": 0.125,
    "estimates": 0.10,
    "API and data model": 0.15,
    "high-level design": 0.20,
    "deep dives": 0.35,
    "trade-offs, wrap-up": 0.075,
}
assert abs(sum(STEPS.values()) - 1) < 1e-9


def plan(slot_min: int, overhead_min: int = 5) -> list[tuple[str, int, int]]:
    """Return (leg, start, end) in minutes after the design begins."""
    design = slot_min - overhead_min     # intros and your questions
    t, out = 0.0, []
    for step, share in STEPS.items():
        start, t = t, t + share * design
        out.append((step, round(start), round(t)))
    return out


for leg, start, end in plan(60):
    print(f"{leg:22} {start:2}-{end:2}")
```

For a 60-minute slot it prints 0 to 7 for requirements, 7 to 12 for estimates, 12 to 21 for API and data model, 21 to 32 for the high-level design, 32 to 51 for deep dives and 51 to 55 for the wrap-up. The budget is a guide, not a contract. The interviewer may steer you into a deep dive at minute 12 ("let's talk about the seat holds now"); follow, and keep a mental note of what you skipped so you can name it in the wrap-up. Wear a watch or keep the clock in view, and check it at each checkpoint in the picture.

27.3 Clarifying questions

A tailor measures before cutting, and asks what the suit is for: a wedding and a winter commute want different cloth. The prompt "design Ticketmaster" is the customer saying "a suit, please". Your first five minutes are the measurements: questions whose answers change the design, asked in an order that builds the requirements list of Chapter 1 on the whiteboard as you go.

Ask in four groups, and write each answer down where the interviewer can see it:

Group	Question	Why it changes the design
Users and flows	Who uses it, and which three to five flows are core?	Every flow is a component
Scale	Daily users, peak events, data size, growth?	One box or a fleet
Qualities	Latency, availability, consistency per flow?	Cache or transaction
Boundaries	External systems, existing stack, out of scope?	Adapters, what not to draw

Three habits make the five minutes count. **Propose rather than interrogate**: "I'll assume 4 million daily users and a flash sale of a million fans for one stadium show; does that match what you have in mind?" gives the interviewer something to correct and shows you know what typical numbers look like. **Name the invariant early**: "a seat is sold at most once" or "a message is never lost" is the sentence the whole design will be judged against (Chapter 26 starts every hard problem from it). **Close the scope aloud**: "resale, dynamic pricing and venue management are out of scope"; every dropped feature is a component you will not have to defend.

External systems deserve one explicit question, because they carry requirements you do not control: a payment provider with its own availability and at-least-once webhooks, an e-mail provider with a sending quota (Encore's accepts 100 messages a second), a maps or identity API with rate limits (Chapter 24). A prompt that involves an AI

assistant (“add a support chatbot”, “design a ChatGPT-like service”) adds its own questions: tokens per request, acceptable seconds to the first token, which tools the agent may call and who approves a refund. The method does not change; the units do (tokens per second instead of requests per second, dollars per conversation instead of dollars per server), and *Designing Agentic Systems* covers those designs in depth.

> **Design question.** The interviewer says “Design Ticketmaster” and nothing else. Which questions do you ask in the first five minutes?
>
> Ask what decides the architecture and propose numbers for each. Flows: browse, seat map, hold, pay, receive ticket; resale out of scope. Scale: “4 million daily users normally, but the design point is a 50,000-seat show with a million fans in ten minutes; agreed?” Qualities: seat map can be a second stale, a hold must be exact, checkout 99.95%, a paid order never lost. Boundaries: payments through an external provider (so idempotency keys and webhooks), tickets by e-mail through a provider with a quota. Then state the invariant: never sell a seat twice. Five minutes, one board of requirements, and the interviewer already sees the hard part coming. See also *50 System Designs* #25 (Ticket Booking).

27.4 Estimation drills

Estimation in an interview is a carpenter’s tape measure, not a laser survey: it decides whether the beam is two metres or twenty, and that is all it must do. Spend about four minutes, say every assumption aloud, round to powers of ten, and finish each line with a decision (“fits one Postgres primary”, “needs a CDN”). An estimate that ends in a number and not a decision has wasted the minutes.

Memorise the conversions so the arithmetic is instant:

Per day	Per second	Per day	Per second
1 million	about 12	100 million	about 1,200
10 million	about 120	1 billion	about 12,000

The equations are those of Chapter 1: QPS from daily users, a peak factor, storage per year, bandwidth and cache memory. The step candidates skip is the last one, comparing each result with what one machine does (Chapter 2): a Python API worker serves hundreds to thousands of requests a second, a Postgres primary thousands to tens of thousands of durable commits, a Redis instance on the order of 100,000 simple commands.

$$\text{machines} \approx \frac{\text{peak rate}}{\text{capacity of one}} \qquad \text{storage} = \text{items per day} \times 365 \times \text{years} \times \text{bytes}$$

Rehearse with a drill script and change one input at a time (all inputs illustrative):

```
DAY = 86_400
CAPACITY = {              # one instance, orders of magnitude
    "python api worker": 1_000,
    "postgres commits": 5_000,
    "redis ops": 100_000,
}


def drill(name, per_day, read_ratio, item_bytes, years, peak=2):
    writes = per_day / DAY * peak
    reads = writes * read_ratio
    storage_tb = per_day * 365 * years * item_bytes / 1e12
    pg = writes / CAPACITY["postgres commits"]
    print(f"{name}: {writes:,.0f} writes/s, {reads:,.0f} reads/s peak,"
          f" {storage_tb:,.0f} TB in {years} y, {pg:.1f} Postgres primaries")


drill("url shortener", 100e6, 100, 500, 5)
drill("chat messages", 500e6 * 40, 1, 100, 1)
```

It prints about 2,300 writes and 231,000 reads a second at the peak and 91 TB over five years for a URL shortener with 100 million new links a day, and about 463,000 messages a second and 730 TB a year for a chat service with 500 million daily users sending 40 messages of 100 bytes. The decisions follow: the shortener's writes fit one primary but its reads need a cache, and 91 TB needs a partitioned key-value store; the chat service's writes need on the order of a hundred Postgres primaries, so it wants a store built for partitioned writes, such as Cassandra or Bigtable (Chapter 13). For Encore the drill is already done: 6,000 requests a second on a normal day, 200,000 seat-map reads and 30,000 hold attempts a second in the flash sale, and the 7:1 read/write ratio on one event that makes it a contention problem.

27.5 API and data model

A restaurant's menu and its pantry come before the kitchen layout: what can be ordered, and what must be stored to cook it. The API is the menu, the data model the pantry, and together they take about six minutes. Write only the calls that carry the core flows, three to five of them, each with its method, the parameters that matter and the status it returns; then the entities with their keys and the store that owns each. Signal depth with details that prevent bugs, not with completeness: an idempotency key on every call that creates something, cursor pagination on every list, 202 Accepted for anything slow (Chapter 5).

```
GET  /v1/events/{id}/seatmap      -> 200 bitmap, 1 bit/seat, CDN TTL 1 s
POST /v1/events/{id}/holds        Idempotency-Key, {seat_ids}
                                  -> 201 {hold_id, expires_at} | 409
POST /v1/orders                   Idempotency-Key, {hold_id}
                                  -> 202 {order_id, status: pending}
GET  /v1/orders/{id}              -> 200 {status: pending|paid|ticketed}
POST /v1/webhooks/payments        provider-signed, processed idempotently
```

Entity	Key	Store	Why
Event, venue	`event_id`	Postgres + cache + CDN	read-heavy, rarely changes
Seat state	`event_id#section`, seat	Postgres rows, Redis holds	the contended core
Hold	`hold:{42:A}:A-12-7`, TTL 600 s	Redis	expires by itself
Order	`order_id` (carries shard bits)	Postgres	transactions, 100 GB a year
Ticket PDF	`order_id/seat_id`	S3	50 KB blobs
Click event	`session_id`	Kafka, then lakehouse	append-only

The keys are the part interviewers probe. Say why each one spreads load or keeps related data together: `event_id#section` splits the stadium show over partitions (Chapter 17), the Redis hash tag puts a multi-seat hold on one slot so a Lua script can check all seats atomically (Chapter 9), and an order ID that carries its shard lets the payment webhook route without a lookup. Choose each store from the access pattern in one sentence, and save the comparison of database families for a question that asks for it (Chapter 13).

27.6 The high-level design

An architect's first sketch of a house shows rooms and doors, not wiring. The high-level design is that sketch: every box a component with one job, every arrow a call labelled synchronous or asynchronous and, on the hot paths, with its rate. Draw it in about eight minutes, then walk each core flow along the arrows aloud ("a hold goes from the client through the waiting room and the gateway to seat inventory, which runs one Redis script"); a walk-through finds the missing arrow faster than staring does.

```
fans --> CDN (pages, seat-map bitmap, 1 s TTL)       200,000 reads/s
  |
  +--> waiting room (admits ~1,000 fans/s) --> API gateway
                                                  |
          +---------------+---------------------+------------+
          v               v                     v            v
      catalog         seat inventory          orders       search
      Postgres       Redis holds (3,000/s)   Postgres     OpenSearch
      + cache        Postgres seats          + outbox
                                                | async (Kafka, SQS)
                        +-----------------------+---------+
                        v                       v         v
                notification workers      PDF workers  analytics
                e-mail, SMS, push         -> S3        lakehouse
orders <-- webhook -- payment provider (external, idempotency keys)
```

Three rules keep the diagram honest. Start with the simplest design that meets the requirements, then add a box only when a number demands it; an unexplained cache or queue invites "why is that there?". Keep stateless services stateless and put state in named stores, so scaling is a replica count (Chapter 7). And mark the hot spot on the board itself, here the seat-inventory box, where 30,000 hold attempts a second arrive at the door and the waiting room lets through about 3,000, because it tells the interviewer where you will go next.

Name real products where they help, with the equivalent on the other clouds the first time, because it shows you have run these things: CloudFront (Cloud CDN, Azure Front Door) for the edge, ALB and Envoy for load balancing (Chapter 8), ElastiCache for Valkey (Memorystore, Azure Managed Redis) for holds, Aurora or RDS for PostgreSQL (Cloud SQL, Azure Database for PostgreSQL), SQS and MSK (Pub/Sub, Service Bus and Event Hubs) for asynchronous work (Chapter 14), S3 (Cloud Storage, Blob Storage) for tickets. The appendix Appendix C lists the rest. A product name is not a design, though: "we use Kafka" earns nothing until you say what the key is and what happens when a consumer falls behind.

27.7 Choosing deep dives

A doctor examining a patient does not test every organ; she follows the symptoms to the one or two that matter. Your estimates are the symptoms. A deep dive belongs where a number says the simplest design breaks, or where the invariant is at risk, and nowhere else. Fourteen minutes is room for one deep dive done well or two done briskly, never five.

Rank candidates by **pressure**, the peak demand divided by what the simplest design can take, and add any component that guards the invariant even at low pressure:

$$\text{pressure} = \frac{\text{peak demand}}{\text{capacity of the simplest design}}$$

```python
# Pressure = peak demand / capacity of the simplest design (illustrative).
candidates = [
    # name, peak per second, simplest design, its capacity per second
    ("seat-map reads", 200_000, "normal-day fleet", 6_000),
    ("hold attempts", 30_000, "one DynamoDB partition", 1_000),
    ("order writes", 33, "one Postgres primary", 5_000),
    ("ticket e-mails", 33, "e-mail provider quota", 100),
]
ranked = sorted(candidates, key=lambda c: c[1] / c[3], reverse=True)
for name, peak, simple, cap in ranked:
    print(f"{name:15} {peak / cap:7.2f}x  vs {simple}")
```

It ranks seat-map reads at 33x (200,000 a second against the normal-day fleet, all figures illustrative), hold attempts at 30x (30,000 a second at the door, before any waiting room, against one DynamoDB partition's 1,000 writes a second), the ticket e-mails at 0.33x (33 a second against the provider's 100) and order writes at 0.01x (33 a second). So the deep dives are the seat map and the holds, and the holds win the tie because they also guard the invariant. Order writes need one sentence ("one Postgres primary absorbs them"), and saying so is a signal too: knowing what is *not* hard is half of judgement.

Then ask. “The numbers say the hold path and the seat map are the hard parts; I’d like to start with holds, unless you’d rather see something else?” Interviewers often have a deep dive in mind, and this question surfaces it while showing you can prioritise. Run each deep dive in five beats, out loud:

1. **Restate the problem with its number**: 30,000 hold attempts a second on 50,000 seats of one event.
2. **Show the naive design and how it fails**: one partition keyed by `event_id` takes 1,000 writes a second.
3. **Bring the mechanism**: key by `event_id#section`, hold with `SET NX EX 600` in Redis, a Lua script for multi-seat holds, the conditional `UPDATE` in Postgres at sale time (Chapter 3).
4. **Break it on purpose**: the Redis primary fails mid-sale, a client retries, a hold expires during payment (Chapter 18).
5. **Name the trade-off**: a lost hold costs a fan ten minutes; a lost sale is impossible because Postgres is the source of truth for sales.

For the hard problems each kind of service raises (the feed’s celebrity fan-out, the chat gateway’s reconnect storm, the payment’s double charge), the catalogue in Chapter 26 gives a table per service and a worked solution; use it to prepare the deep dives for the prompts you expect.

> **Design question.** In a URL-shortener interview you have 14 minutes left. Which deep dives do you choose?
>
> Run the pressure numbers from the drill: about 231,000 redirects a second at the peak against one Redis instance’s 100,000 operations is about 2.3x; about 2,300 creates a second against one Postgres primary’s thousands of commits is under 1x, but the invariant (no two links share a code) lives there. So: first the read path, a cache in front of a partitioned key-value store, with a Redis Cluster spreading hot codes and 302 versus 301 decided by whether click analytics are needed (a 301 lets browsers skip the server next time). Second, ID generation: counter ranges handed to each server or Snowflake-style IDs encoded in Base62, rather than hashing with collision checks. Analytics gets one sentence: clicks go to a stream and are counted asynchronously. See also *50 System Designs* #1 (URL Shortener) and #21 (Unique ID Generator).

27.8 Trade-offs and bottlenecks

Buying a car is a set of trade-offs everyone understands: more space costs fuel, more speed costs money, and the right answer depends on whether you have three children or a long commute. Design choices are the same, and interviewers listen for the *depends on what*. Every choice you make gets one sentence in a fixed shape:

```
I chose  X  over  Y  because  <requirement or number>.
It costs <what X gives up>.  I would switch to Y if <condition>.
```

“I chose fan-out on write over fan-out on read because feed reads outnumber posts 20 to 1; it costs write amplification for authors with many followers; I would switch to reading at request time for any author above 10,000 followers.” The sentence shows you know the alternative, the price and the boundary, which is the whole of judgement in one breath. The pairs that recur in almost every interview:

Choice	Pick the first when	Chapter
Strong vs eventual consistency	a stale read breaks the invariant	Chapter 18
Push vs pull (fan-out)	reads far outnumber writes	Chapter 26
Relational vs NoSQL	joins, transactions, modest scale	Chapter 13
Synchronous vs asynchronous	the caller needs the result now	Chapter 14
Cache longer vs fresher	staleness is cheap, load is not	Chapter 9
At-least-once + idempotency vs exactly-once	almost always	Chapter 14
One region vs several	the SLO or the users demand it	Chapter 16
Exact vs approximate counts	money, inventory, limits on rights	Chapter 19

After the deep dives, sweep the diagram for **bottlenecks** before the interviewer does. Ask four questions of every box: is it a single point of failure; does one key or one tenant concentrate its load; does any queue or connection pool grow without a bound; and what happens when its slowest dependency gets ten times slower (by Little's law, the work in flight grows tenfold, Chapter 1)? Then answer the classic closing question, "what breaks first at ten times the load?", with a number: for Encore at 300,000 hold attempts a second, the Redis shards for the hottest sections saturate first, and the fix is finer salting and a slower waiting-room admission rate, not more application servers (Chapter 11).

> **Design question.** At minute 30 the interviewer changes a requirement: Encore must keep selling even if a whole region fails during the flash sale. What changes?
>
> Restate the cost first. Each event's seats have a home region with asynchronous replication to the other, so a region loss costs up to about a second of writes: with the waiting room running inventory at 3,000 holds a second, up to about 3,000 holds and up to about 80 sold seats. Lost holds are acceptable (fans retry). Lost sales are not: the promoted region could sell those seats again. Two options. Keep asynchronous replication, and after failover pause the event, reconcile sold seats against the payment provider's records of captured payments, then reopen: zero oversell, a pause of minutes. Or make sales synchronous across regions with a consensus database spanning three locations (two regions plus a witness, as Spanner, CockroachDB and DynamoDB multi-Region strong consistency do): no pause, but about 90 ms more per sale commit, which 83 sale writes a second absorb easily while holds stay regional. State which you pick and why: the second, because a pause at 10:01 on the biggest sale of the year is itself an outage. See also *50 System Designs* #25 (Ticket Booking) and #38 (Globally Distributed Database).

27.9 What interviewers look for

A driving examiner does not grade whether you reached the destination; she grades mirror checks, lane discipline and how you handled the cyclist. Design interviewers likewise grade the way you work, not whether your diagram matches a reference answer, because there is none. The dimensions below are a synthesis of how large companies describe their rubrics (names and weights vary, and no company publishes its exact form):

Dimension	Weak signal	Strong signal
Problem navigation	starts drawing at once	scopes, names the invariant, checks in
Estimation	skips it or gets lost	four minutes, rounded, ends in decisions
Design soundness	boxes without reasons	simplest design that meets every flow
Technical depth	names products	explains keys, failure and recovery
Trade-offs	one option, no cost	alternatives, price, switching condition
Operations	ignores failure	timeouts, retries, monitoring, rollout
Communication	monologue or silence	thinks aloud, drives, takes hints well

The bar rises with level. A mid-level candidate is expected to produce a working design with some guidance; a senior candidate drives the whole interview, finds the hard part unprompted and goes deep on it; a staff-level candidate also reasons about cost, operations, migration from what exists today and the organisation that will run it. Hints are part of the format, not a penalty in themselves: an interviewer who asks "what if the cache node dies?" is giving you a deep dive. Take it, answer it, and say what it changes in the diagram. Arguing with a hint, or ignoring it, scores worse than any missing component.

27.10 Common questions mapped to this book

Interview prompts are a small family wearing many costumes: most are a read-heavy cache problem, a fan-out problem, a contention problem, a stream-counting problem or a storage-engine problem in disguise. The table maps the common prompts to their case studies in *50 System Designs* (by number) and to the chapters that teach the

concepts underneath. Read across a row before a mock interview: the case study shows the finished design, the chapters let you defend it.

Prompt	*50 System Designs*	Chapters
URL shortener, Pastebin	#1, #41	Chapter 9, Chapter 17
Rate limiter	#2	Chapter 11, Chapter 9
Key-value store	#3	Chapter 16, Chapter 17
Distributed cache	#4	Chapter 9, Chapter 17
News feed, photo sharing	#5, #19	Chapter 14, Chapter 9
Chat, messenger	#6, #7	Chapter 4, Chapter 8
Video upload, streaming	#8, #9, #32	Chapter 10, Chapter 14
File storage and sync	#10, #33, #37	Chapter 12, Chapter 16
Collaborative editor	#11	Chapter 16, Chapter 18
Distributed log, logging	#12, #48	Chapter 14, Chapter 21
Ride hailing, nearby places	#13, #27	Chapter 19, Chapter 17
Maps and navigation	#14	Chapter 19, Chapter 10
CDN	#15	Chapter 10, Chapter 8
Autocomplete, trending	#16, #23	Chapter 19, Chapter 21
Web crawler, search engine	#17, #22	Chapter 14, Chapter 19
Notification system	#20	Chapter 14, Chapter 24
Unique ID generator	#21	Chapter 17, Chapter 19
Payments, digital wallet	#24, #44	Chapter 18, Chapter 24
Tickets, hotels, flash sale	#25, #26, #34	Chapter 3, Chapter 25
Ad clicks, metrics, counters	#28, #29, #50	Chapter 21, Chapter 19
Job scheduler, online judge	#30, #46	Chapter 14, Chapter 15
Leaderboard	#31	Chapter 9, Chapter 17
Spanner, ZooKeeper	#38, #39	Chapter 18, Chapter 16
Load balancer, API gateway	#40	Chapter 8, Chapter 11
Stock exchange	#43	Chapter 3, Chapter 2
Feature flags, config	#49	Chapter 23, Chapter 18
Recommendations	#47	Chapter 21, Chapter 13
LLM assistant, support agent	*Designing Agentic Systems*	Chapter 24, Chapter 11

Every row also leans on Chapter 1 for the estimates and on Chapter 26 for the hard questions of its service type. If time is short, practise the Tier 1 prompts of *50 System Designs* first (#1 to #17) and add #20, #24 and #25: between them they exercise nearly every chapter of this book.

27.11 Mini interview: a rate limiter

The three interviews below are compressed to their turning points; all numbers are illustrative. Read the candidate's lines aloud and time yourself.

Interviewer: Design a rate limiter for our public API.

Candidate: A few questions first. Is it a library inside each service or a shared service in front? What do we limit by: user, API key, IP? How many clients and how much traffic? And when the limiter itself fails, do we let traffic through or block it?

Interviewer: Shared, at the gateway. Per API key and per user. Ten million users, 100,000 requests a second at the peak. You choose the failure behaviour.

Candidate: Then the invariant is soft: a limit protects capacity and fairness, so being off by a few percent is fine, but adding latency to every call is not. Estimates: one counter state of about 100 bytes per active user is 1 GB in memory; 100,000 checks a second is one Redis instance's worth of simple commands, so I'll run a Redis Cluster of three shards for headroom and spread keys by hash slot. Algorithm: a token bucket, because it allows bursts up to b and caps any interval at $b + rT$; say r = 10 per second and b = 20 per user. The check is one Lua script per request that refills, decides and decrements atomically, about half a millisecond inside one zone. The gateway answers 429 with `Retry-After` computed from the bucket.

Interviewer: What happens if the Redis cluster goes down?

Candidate: Fail open for ordinary endpoints with a coarse local limiter in each gateway instance, so an outage of the limiter is not an outage of the API; fail closed for the expensive ones, such as SMS sending or model calls, where a missing limit costs money. I'd alert on the fallback and watch the 429 rate per key as the SLI.

What a strong answer contains. Clarified placement, keys and failure mode in the first two minutes; turned 10 million users and 100,000 requests a second into 1 GB and three shards; chose an algorithm with a reason and an equation; made the check atomic; returned a correct 429 with `Retry-After`; decided fail-open versus fail-closed per endpoint. Extensions worth one sentence each: layered limits at the edge, per-tenant quotas, and token-based limits for an LLM endpoint (Chapter 11). See also *50 System Designs* #2 (Rate Limiter).

27.12 Mini interview: a news feed

Interviewer: Design the home timeline for a social network.

Candidate: Core flows: post, follow, read my feed in reverse time order, newest first. I'll assume 200 million daily users who open the feed 10 times a day and post once every two days, an average of 200 followers, and a feed that shows the newest 20 posts within a few seconds of their creation. Ranking, ads and search are out of scope. Does that work?

Interviewer: Yes. Go on.

Candidate: That is about 23,000 feed reads a second on average, 46,000 at a 2x peak, against 1,200 posts a second: reads outnumber posts 20 to 1, so I'll precompute feeds. Fan-out on write means each post is inserted into every follower's feed: 1,200 posts times 200 followers is about 231,000 inserts a second, done by workers reading a queue. Each feed is a Redis sorted set of the newest 500 post IDs, 4 KB a user, so 1.6 TB with a 2x overhead across a Redis Cluster of about 25 nodes of 64 GB, plus replicas. Reading a feed is one range query plus a batched lookup of 20 posts from a post cache.

```
ZADD feed:{u7} 1792000000 p9001        # worker inserts a new post
ZREMRANGEBYRANK feed:{u7} 0 -501       # keep the newest 500
ZREVRANGE feed:{u7} 0 19               # read the first page
```

Interviewer: A celebrity with 50 million followers posts. What happens?

Candidate: Fan-out on write would make 50 million inserts, about 50 seconds even at a million inserts a second, and a burst of them every time she posts. So I'd use the hybrid: authors above a threshold, say 10,000 followers, are not fanned out; at read time the feed service merges the precomputed feed with the recent posts of the few celebrities the reader follows, each cached once for everyone. The cost is a slightly more expensive read for readers who follow celebrities; the threshold is a tuning knob, and inactive users can skip fan-out and get their feed built on demand.

What a strong answer contains. A read/write ratio that justifies precomputing; the fan-out arithmetic and the memory sizing; the celebrity problem raised before or immediately when prompted, with the hybrid as its fix; the queue between posting and fan-out so a post returns fast; and the trade-off sentence for push versus pull. Deeper follow-ups (ranking, deletes that must disappear from feeds, cursor pagination) are listed with the other feed challenges in Chapter 26. See also *50 System Designs* #5 (News Feed) and #19 (Photo Sharing).

27.13 Mini interview: a ticketing flash sale

Interviewer: A stadium show goes on sale at 10:00. Design the sale.

Candidate: The invariant is that a seat is sold at most once. I'll assume 50,000 seats, a million fans arriving between 09:55 and 10:05, a 10-minute hold, and payment through an external provider. The numbers I'd design for: 200,000 seat-map reads a second and 30,000 hold attempts a second in the first minute, all on one event. The seat map I serve as a bitmap, one bit per seat, 6.25 KB, cached at the CDN for one second: 10 Gbit/s at the edge instead of 1.6 Tbit/s of JSON, and the origin sees one refresh per second per edge location. It is a hint; the hold is the truth.

Interviewer: And the holds?

Candidate: First I'd stop the crowd from reaching them at once. A waiting room admits fans at the rate the core can serve. Seat inventory is load-tested at 6,000 holds a second and run at 3,000 for headroom; an admitted fan makes about 3 hold attempts, so the room admits about 1,000 fans a second, and the back of a million-fan queue waits about 17 minutes, which the queue page says honestly. Inventory runs 12 pods: 8 carry the tested 6,000 holds a second at 1,000 per pod and 75 percent utilization, and 4 more cover the loss of a zone. Behind it, a hold is one atomic command, with the event split by section so one stadium spreads over many shards:

```
SET hold:{42:A}:A-12-7 fan_ana NX EX 600   # OK: Ana holds the seat
SET hold:{42:A}:A-12-7 fan_ben NX EX 600   # (nil): Ben gets 409
```

Multi-seat holds run as one Lua script on one slot. At payment, Postgres is the source of truth: the order insert and a conditional `UPDATE seats SET status = 'sold' ... WHERE status = 'held' AND holder = ...` commit in one transaction, and a unique index on tickets `(event_id, seat_id)` is the backstop.

Interviewer: The fan pays, and the provider's confirmation arrives twice.

Candidate: Every call to the provider carries an idempotency key derived from the order, and the webhook handler records the provider's event ID under a unique constraint before acting, so the second delivery is a no-op. Payment returns 202 and the app polls or gets a push. `OrderPaid` goes out through the outbox, so a crash between commit and publish loses nothing; 20,000 orders make 20,000 confirmation e-mails, which the provider's 100 a second sends in about 200 seconds, and 24 workers render the PDFs.

What a strong answer contains. The invariant named first; the two flash-sale numbers and their decisions (bitmap at the edge, admission control); an atomic hold with a TTL and a durable, conditional sale; the hot event split by section; idempotency on every external call and webhook; asynchronous tickets through the outbox and a queue; and a word on bots (one waiting-room token per verified account, Chapter 22). The full build, with code and cloud, is Chapter 25. See also *50 System Designs* #25 (Ticket Booking) and #34 (E-Commerce Checkout and Inventory).

27.14 Numbers to remember

Figure	Value
Design time in a 45-minute slot	about 40 minutes
Budget (req, est, API, HLD, deep, wrap)	5, 4, 6, 8, 14, 3 minutes
1 million / 1 billion a day	about 12 / 12,000 a second
One Python worker, Postgres primary, Redis	1,000s req/s, 1,000s commits/s, 100,000 ops/s
Pressure	peak demand / capacity of the simplest design
Encore flash sale	200,000 reads/s; 30,000 holds/s at the door, 3,000 inside
Encore waiting room	about 1,000 fans/s admitted, 17-minute wait at the back
Feed drill	46,000 reads/s, 231,000 inserts/s, 1.6 TB of feeds
Rate-limiter drill	100,000 checks/s, 1 GB, 3 Redis shards

27.15 Pitfalls

- Drawing before scoping -> spend the first five minutes on flows, numbers and the invariant, and write them on the board.
- Estimates that end in a number -> end every line with a decision (“one primary”, “needs a CDN”).
- Ten minutes on the load balancer -> draw the standard parts in one breath and spend the time where pressure is highest.
- Five shallow deep dives -> one or two, in five beats: number, naive design, mechanism, failure, trade-off.
- Product names as answers -> say the key, the consistency model and what happens on failure.
- Silence while thinking -> narrate; the interviewer grades reasoning they can hear.
- Ignoring or fighting a hint -> treat it as the deep dive the interviewer wants, and update the diagram.
- Running out of time with no wrap-up -> at minute 37, stop and summarise trade-offs and what you would add next.

27.16 Remember

- The interview is a flight plan: requirements, estimates, API and data model, high-level design, deep dives, trade-offs, on a clock.
- Ask questions that change the design, propose numbers, and name the invariant early.
- Every estimate ends in a decision; every choice gets the sentence “X over Y because…, costs…, switch if…”.
- Choose deep dives by pressure and by the invariant, and ask the interviewer before you go deep.
- Most prompts are a few problems in disguise; the mapping table and Chapter 26 prepare them.
- Interviewers grade how you work: scoping, depth, trade-offs and communication, more than any box.

Appendix A. Numbers to know

The figures worth carrying into a design discussion, collected from the chapters. Orders of magnitude matter more than digits: a design that is off by 10 percent works, one that is off by a factor of 1,000 does not. Hardware and service figures are typical values as of October 2026; service limits are defaults that can often be raised. Encore figures are illustrative and identical everywhere in the book.

Time and scale

Figure	Value
Seconds in a day / a year	86,400 (about 10^5) / about 3.15×10^7
1 request/s sustained	about 86,000 a day, 31.5 million a year
1 million / 1 billion a day	about 12 / 12,000 a second
Powers of two	$2^{10} \approx 10^3$, $2^{20} \approx 10^6$, $2^{30} \approx 10^9$, $2^{40} \approx 10^{12}$
Bytes to bits	multiply by 8
Light in fiber	about 200 km per millisecond

Latency ladder

Operation	Typical time
L1 / L2 / L3 cache, DRAM	1 / 4 / 10 to 40 / 100 ns
Python `Lock` acquire and release	about 135 ns
Thread handoff / asyncio task switch	about 5 us / 1.6 us
Random 4 KiB read on NVMe	20 to 100 us
Round trip within one zone	0.1 to 0.5 ms
Redis round trip in one zone	about 0.5 ms
Round trip between zones	0.5 to 2 ms
`fsync` on network block storage	1 to 3 ms
HDD seek	5 to 10 ms
Round trip to a nearby CDN PoP	about 10 ms
Round trip Frankfurt to Virginia	about 90 ms (65 ms speed-of-light floor)
New TCP + TLS 1.3 connection	2 round trips before the request; QUIC 1 (0 on resumption)

What one machine or service does

Component	Order of magnitude
One web worker process	thousands of requests/s for light handlers
One Postgres primary on a large instance	thousands of commits/s
One Redis instance	about 100,000 simple commands/s; 1,000,000+ pipelined
One DynamoDB partition	3,000 read units and 1,000 write units per second
One Kinesis shard	1 MB/s or 1,000 records/s in, 2 MB/s out
One EBS gp3 volume	3,000 IOPS and 125 MiB/s included; up to 80,000 IOPS and 2,000 MiB/s

Component	Order of magnitude
One HDD	about 120 random IOPS
S3 per prefix	3,500 writes/s and 5,500 reads/s
Largest EC2 instance	1,920 vCPU and 32 TiB of memory

Availability

Availability	Downtime a year	Downtime a 30-day month
99%	3.65 days	7.2 hours
99.9%	8.8 hours	43 minutes
99.99%	53 minutes	4.3 minutes
99.999%	5.3 minutes	26 seconds

Five components in series at 99.95 percent each give 99.75 percent, about 22 hours down a year. A majority of three nodes at 99 percent each is up 99.97 percent of the time; of five, 99.999 percent. M/M/1 response time is 2, 5 and 10 service times at 50, 80 and 90 percent utilization.

Default limits of common services

Service	Limit
AWS Lambda	15 minutes, up to 10,240 MB, 1,000 concurrent executions per Region by default, +1,000 environments per 10 s per function
Amazon API Gateway	10,000 requests/s with a burst of 5,000 per account and Region; 29 s integration timeout; 10 MB payload
Amazon SQS	30 s default visibility timeout (12 h maximum); 4-day retention (14 maximum); 1 MiB messages; 5-minute FIFO deduplication window
DynamoDB	400 KB items; transactions of 100 items and 4 MB; 1 MB query pages
Amazon S3	11 nines designed durability; 5 GB single PUT; multipart of 10,000 parts from 5 MiB to 5 GiB, about 50 TB per object
Amazon CloudFront	250,000 requests/s and 150 Gbit/s per distribution by default; CloudFront Functions under a millisecond
Application Load Balancer	60 s idle timeout; 300 s deregistration delay; health check every 30 s
Kubernetes	30 s termination grace period; HPA checks every 15 s and waits 5 minutes before scaling down
Redis Cluster	16,384 hash slots, CRC16 of the key or its hash tag
Kafka	7-day default retention; order is kept within a partition
Postgres	`max_connections` 100 by default; 8 KB pages

Sizes of the clever structures

Structure	Size
HyperLogLog (Redis)	12 KB, 0.81 percent standard error
Bloom filter at 1 percent false positives	9.6 bits and 7 hashes per item; each 10x better adds 4.8 bits
Count-Min sketch, 0.1 percent error, 99 percent confidence	2,719 x 5 counters, about 54 KB

Structure	Size
Merkle search for one difference in 80,000 rows	about 23 hash comparisons
Gossip to 200 nodes	about 14 rounds
B-tree for 50 million rows	3 to 4 levels
Snowflake ID	41 + 10 + 12 bits: 69.7 years, 4,096 IDs per millisecond per worker
Consistent hashing, adding the ($N + 1$)th node	moves about $1/(N + 1)$ of keys; `mod N` moves $N/(N + 1)$

Security

Figure	Value
Argon2id minimum (OWASP)	19 MiB memory, 2 iterations, 1 lane
bcrypt	work factor 10 or more; only the first 72 bytes count
Access token lifetime	5 to 15 minutes
Public TLS certificate maximum lifetime	200 days (2026), 100 days (2027), 47 days (2029)
AWS KMS symmetric requests	10,000/s by default per account and Region; 4 KB `Encrypt` payload
GDPR	erasure within one month; breach notice within 72 hours

Encore's numbers

Figure	Value
Users	20 million registered, 4 million daily active
Normal load	1,500 requests/s average, 6,000/s at the daily peak
Flash sale	50,000 seats; 1,000,000 fans between 09:55 and 10:05
Seat-map reads at the peak	200,000/s, served as a 6.25 KB bitmap (10 Gbit/s)
Hold attempts at the door	30,000/s in the first minute
Waiting room	admits about 1,000 fans/s; about 17 minutes at the back of a million
Seat inventory	load-tested at 6,000 holds/s, run at 3,000/s; 12 pods; keyed by `event_id#section`
Flash-sale sales	about 20,000 orders (2.5 seats each): about 83 tickets and 33 orders per second
Ticket PDFs	200 ms each; 24 workers keep up
E-mail provider	100 messages/s
Storage per year	orders 100 GB, ticket PDFs 2.5 TB
Clickstream	1 billion events a day at about 1 KB; 58 MB/s at a 5x peak; 48 Kafka partitions
Regions	us-east-1 and eu-central-1, about 90 ms apart

Appendix B. Formulas

The equations of the book, grouped by the question they answer. Each comes with one line on when to use it and the chapter that derives it with a worked Encore example. Symbols: λ is an arrival rate (per second), W a time spent in the system, S a service time, ρ utilization, N a count of machines, replicas or partitions, p a probability.

Estimation and capacity

Requests per second from daily users, and the peak you must design for (Chapter 1):

$$\text{QPS}_{\text{avg}} = \frac{\text{DAU} \times \text{requests per user per day}}{86,400} \qquad \text{QPS}_{\text{peak}} = \text{QPS}_{\text{avg}} \times \text{peak factor}$$

Storage and bandwidth (Chapter 1):

$$\text{storage per year} = \text{items per year} \times \text{bytes per item} \times \text{replicas} \qquad \text{bandwidth} = \text{requests/s} \times \text{bytes per response}$$

Little's law: items in a system equal arrival rate times time inside. It sizes pools, workers, queues and the waiting room (Chapter 1, Chapter 3):

$$L = \lambda W$$

Machines for a load, at a target utilization u, with one zone's worth of spare capacity across Z zones (Chapter 7, Chapter 15):

$$N = \frac{\lambda}{c\,u} \quad \frac{Z}{Z-1}$$

Threads for an I/O-bound pool, where W/C is the ratio of waiting time to compute time (Chapter 3):

$$N_{\text{threads}} = N_{\text{cores}} \times \left(1 + \frac{W}{C}\right)$$

Interview shortcut: how much a component is overloaded in the simplest design tells you where to spend the deep dive (Chapter 27):

$$\text{pressure} = \frac{\text{peak demand}}{\text{capacity of the simplest design}}$$

Availability and reliability

Downtime allowed by an availability A over a period T, and availability of components in series and in parallel (Chapter 1):

$$\text{downtime} = (1 - A)\,T \qquad A_{\text{serial}} = \prod_i A_i \qquad A_{\text{parallel}} = 1 - \prod_i (1 - A_i)$$

Availability from failure and repair times, and the error budget an SLO allows (Chapter 23):

$$A = \frac{\text{MTBF}}{\text{MTBF}+\text{MTTR}} \qquad \text{error budget} = (1 - \text{SLO}) \times \text{valid events in the window}$$

Burn rate: how fast an incident spends the budget; alert when it would empty the budget long before the window ends (Chapter 1, Chapter 23):

$$\text{burn rate} = \frac{\text{observed error ratio}}{1 - \text{SLO}}$$

Availability of a majority quorum of $n = 2f + 1$ nodes, each up with probability p (Chapter 18, Chapter 16):

$$A = \sum_{i=f+1}^{n} \binom{n}{i} p^i (1-p)^{n-i}$$

Performance and queueing

Utilization and the M/M/1 queue: response time explodes as utilization approaches 1 (Chapter 2):

$$\rho = \lambda S \qquad W = \frac{S}{1-\rho} \qquad L = \frac{\rho}{1-\rho}$$

Amdahl's law: the serial fraction caps the speed-up (Chapter 2):

$$\text{speedup}(N) = \frac{1}{(1-p) + p/N} \qquad \lim_{N \to \infty} \text{speedup} = \frac{1}{1-p}$$

Universal Scalability Law: contention σ and coherency κ make throughput peak at N^* and then fall (Chapter 7):

$$X(N) = \frac{\lambda N}{1 + \sigma(N-1) + \kappa N(N-1)} \qquad N^* = \sqrt{\frac{1-\sigma}{\kappa}}$$

Tail at scale: a request that waits for n calls, each slow with probability p, is slow with probability (Chapter 6, Chapter 17):

$$P(\text{slow}) = 1 - (1-p)^n$$

Networks

Speed of light in fiber sets a floor on round-trip time over distance d (Chapter 4):

$$RTT_{min} = \frac{2d}{V}, \qquad v \approx 200,000 \text{ km/s}$$

Time to fetch a response of size S over bandwidth B after k handshake round trips, and the bandwidth-delay product a connection must keep in flight (Chapter 4):

$$T = T_{\text{dns}} + k \quad RTT + \text{rounds} \quad RTT + \frac{S}{B} + T_{\text{server}} \qquad BDP = B \times RTT$$

Caching and CDNs

Effective latency and the load that reaches the database with hit ratio h (Chapter 9):

$$L_{\text{eff}} = L_c + (1-h)\,L_d \qquad R_{\text{db}} = (1-h)\,R$$

Cache memory for the hot set, with an overhead factor f (Chapter 9):

$$M = N_{\text{hot}} \times (s_{\text{key}} + s_{\text{value}} + s_{\text{overhead}}) \times f$$

Hit ratio of a TTL cache for an object requested at rate λ, and origin load through an edge and a shield tier (Chapter 10):

$$h = \frac{\lambda T}{1 + \lambda T} \qquad R_{\text{origin}} = R\,(1-h_e)(1-h_s)$$

Concurrent rebuilds when a hot key expires, the stampede that coalescing prevents (Chapter 9):

$$N_{\text{rebuild}} = R_{\text{key}} \times t_{\text{rebuild}}$$

Redis Cluster slot of a key (Chapter 9):

$$\text{slot}(k) = \text{CRC16}(k) \bmod 16384$$

Rate limiting and resilience

Token bucket with rate r and capacity b: the most it admits in any window T (Chapter 11):

$$\text{tokens}(t) = \min(b,\ \text{tokens}(t_0) + r\,(t - t_0)) \qquad \text{admitted}(T) \le b + rT$$

Sliding window counter estimate for a limit L per window W (Chapter 11):

$$\text{estimate} = c_{\text{prev}} \quad \left(1 - \frac{t_{\text{elapsed}}}{W}\right) + c_{\text{curr}}$$

Exponential backoff with full jitter, and retry amplification when L layers each try a times (Chapter 11, Chapter 26):

$$\text{sleep}_k \quad U(0,\ \min(\text{cap},\ \text{base} \quad 2^k)) \qquad \text{amplification} = a^L$$

Storage and databases

B-tree height for N keys with fan-out f, and leveled LSM write amplification with size ratio T and L levels (Chapter 12):

$$h \approx \log_f N \qquad WA_{\text{leveled}} \approx T \cdot L$$

DynamoDB write units for an item of size s written to the table and g global secondary indexes (Chapter 13):

$$\text{WCU per write} = (1 + g) \cdot \frac{s}{1\ \text{KB}}$$

Queues and streams

Workers needed to keep up at target utilization ρ, backlog growth when arrivals exceed capacity, and time to drain it (Chapter 14, Chapter 26):

$$c = \frac{\lambda S}{\rho} \qquad B = (\lambda - c\mu)\,T \qquad T_{\text{drain}} = \frac{B}{c\mu - \lambda}$$

Partitions for a target throughput T, and the parallelism a consumer group gets (Chapter 14):

$$N \geq max(\frac{T}{t_{\text{produce}}}, \frac{T}{t_{\text{consume}}}) \qquad \text{parallelism} = min(\text{consumers}, \text{partitions})$$

Replication, partitioning and consensus

Quorum overlap and the chance a read misses the latest write when it does not hold (Chapter 16):

$$R + W > N \qquad P_{\text{stale}} = \frac{\binom{N-W}{R}}{\binom{N}{R}}$$

Data lost on an asynchronous failover (Chapter 16):

$$\text{writes lost} \approx \lambda_w \times \text{lag}$$

Nodes needed to survive f failures, and the quorum size (Chapter 18):

$$n = 2f + 1 \qquad \text{quorum} = \lfloor n/2 \rfloor + 1$$

Partitions needed for data size S and throughput T when each partition holds S_p and serves T_p (Chapter 17):

$$P = max(\ \frac{S}{S_p}\ ,\ \frac{T}{T_p}\ )$$

Keys moved when a node is added: almost all with `hash mod N`, about $1/(N+1)$ with consistent hashing; load spread with V virtual nodes per server (Chapter 17):

$$\text{moved}_{\text{mod}} = \frac{N}{N+1} \qquad \text{moved}_{\text{ring}} \approx \frac{1}{N+1} \qquad \frac{\sigma_{\text{load}}}{\mu_{\text{load}}} \approx \frac{1}{\sqrt{V}}$$

Maximum load with random placement versus the power of two choices (Chapter 8):

$$\max \text{load}_{\text{random}} \approx \frac{\ln n}{\ln \ln n} \qquad \max \text{load}_{d=2} \approx \frac{\ln \ln n}{\ln 2}$$

Clock skew bound for drift rate ρ, and Spanner-style commit wait (Chapter 18):

$$\text{skew after } t \leq 2\rho\, t + \varepsilon_{\text{sync}} \qquad t_{\text{commit wait}} \approx 2\varepsilon$$

Probabilistic data structures

Bloom filter false-positive rate for n items in m bits with k hashes, the best k, and bits per item for a target rate (Chapter 19):

$$p \approx (1 - e^{-kn/m})^k \qquad k^* = \frac{m}{n} \ln 2 \qquad \frac{m}{n} \approx 1.44 \log_2 \frac{1}{p}$$

HyperLogLog standard error with m registers, and Count-Min sketch error bounds (Chapter 19):

$$\sigma \approx \frac{1.04}{\sqrt{m}} \qquad \hat{f}(x) \le f(x) + \varepsilon N \text{ with probability } 1 - \delta, \quad w = \lceil e/\varepsilon \rceil, \; d = \lceil \ln(1/\delta) \rceil$$

Cloud and cost

Serverless concurrency for requests of duration d, and the function's cost for N requests with memory m (Chapter 15):

$$C = \lambda d \qquad \text{cost} = N p_{\text{req}} + N d m p_{\text{GB-s}}$$

Kubernetes horizontal autoscaler target (Chapter 15, Chapter 7):

$$N_{\text{desired}} = \left\lceil N_{\text{current}} \times \frac{\text{current metric}}{\text{target metric}} \right\rceil$$

Security

Time to crack leaked password hashes (U users, G guesses each, rate R), and the size of a token denylist (Chapter 22):

$$T_{\text{crack}} = \frac{U \times G}{R} \qquad N_{\text{deny}} = \lambda_{\text{revoke}} \times \text{TTL}$$

Appendix C. Cloud service map

The building blocks of this book exist as managed services on every large cloud. The table maps each block to its usual service on Amazon Web Services, Google Cloud and Microsoft Azure, so a design drawn in boxes can be named on any of them. Names are as of October 2026; providers rename and retire services often, so confirm a name in the provider's documentation before you rely on it. Open-source software you can run yourself on any cloud is listed where it is the common choice.

Compute

Building block	AWS	Google Cloud	Azure
Virtual machines	EC2	Compute Engine	Virtual Machines
Managed Kubernetes	EKS	GKE	AKS
Serverless containers	Fargate (ECS, EKS), ECS Express Mode	Cloud Run	Container Apps
Functions	Lambda	Cloud Run functions	Functions
Batch jobs	AWS Batch	Batch	Batch
Container registry	ECR	Artifact Registry	Container Registry

Networking and the edge

Building block	AWS	Google Cloud	Azure
Virtual network	VPC	VPC	Virtual Network
Layer 7 load balancer	Application Load Balancer	Application Load Balancer (Cloud Load Balancing)	Application Gateway, Front Door
Layer 4 load balancer	Network Load Balancer	Network Load Balancer (Cloud Load Balancing)	Load Balancer
DNS and global routing	Route 53	Cloud DNS	DNS, Traffic Manager
CDN	CloudFront	Cloud CDN, Media CDN	Front Door
Edge functions	CloudFront Functions, Lambda@Edge	Service Extensions (Media CDN, load balancers)	no direct equivalent (Front Door rule sets run no custom code)
API gateway	API Gateway	API Gateway, Apigee	API Management
Private connectivity	PrivateLink	Private Service Connect	Private Link
Web firewall and DDoS	WAF, Shield	Cloud Armor	Web Application Firewall, DDoS Protection
Service-to-service networking	VPC Lattice, ECS Service Connect	Cloud Service Mesh	Istio add-on for AKS

Storage and databases

Building block	AWS	Google Cloud	Azure
Object storage	S3	Cloud Storage	Blob Storage
Block storage	EBS	Persistent Disk, Hyperdisk	Managed Disks
File storage	EFS	Filestore	Files

Building block	AWS	Google Cloud	Azure
Relational (managed)	RDS, Aurora	Cloud SQL, AlloyDB	Database for PostgreSQL / MySQL, SQL Database
Globally distributed SQL	Aurora DSQL	Spanner	no direct equivalent
Key-value and document	DynamoDB	Firestore	Cosmos DB
Wide-column	Keyspaces (Cassandra)	Bigtable	Managed Instance for Apache Cassandra
In-memory cache	ElastiCache (Valkey, Redis OSS, Memcached), MemoryDB	Memorystore (Valkey, Redis Cluster, Redis)	Managed Redis
Search	OpenSearch Service	Agent Search (formerly Vertex AI Search)	AI Search
Graph	Neptune	Spanner Graph	Cosmos DB for Apache Gremlin
Time series	Timestream for InfluxDB	Bigtable, BigQuery	Data Explorer
Vectors	OpenSearch Service, Aurora PostgreSQL with pgvector	Vector Search (Agent Platform), AlloyDB	AI Search, Cosmos DB

Messaging and workflows

Building block	AWS	Google Cloud	Azure
Queue	SQS	Pub/Sub, Cloud Tasks	Service Bus queues, Queue Storage
Publish/subscribe and events	SNS, EventBridge	Pub/Sub, Eventarc	Service Bus topics, Event Grid
Streaming log	Kinesis Data Streams, MSK (Kafka)	Pub/Sub, Managed Service for Apache Kafka	Event Hubs
Durable workflows	Step Functions	Workflows	Durable Functions, Logic Apps
Scheduler	EventBridge Scheduler	Cloud Scheduler	Functions timer trigger, Logic Apps

Data and analytics

Building block	AWS	Google Cloud	Azure
Warehouse	Redshift	BigQuery	Fabric Data Warehouse, Synapse Analytics
Lake tables	S3 with Apache Iceberg (S3 Tables)	Cloud Storage with Lakehouse (formerly BigLake)	Data Lake Storage, OneLake
Spark	EMR, Glue	Managed Service for Apache Spark (formerly Dataproc)	Databricks, Fabric
Stream processing	Managed Service for Apache Flink	Dataflow	Stream Analytics
Pipeline orchestration	Managed Workflows for Apache Airflow	Managed Service for Apache Airflow (formerly Cloud Composer)	Data Factory
Change data capture	Database Migration Service	Datastream	Data Factory CDC, Fabric Mirroring

Security, identity and observability

Building block	AWS	Google Cloud	Azure
Identity and access for staff and services	IAM	IAM	Entra ID with Azure RBAC
Sign-in for customers	Cognito	Identity Platform	Entra External ID
Encryption keys	KMS	Cloud KMS	Key Vault
Secrets	Secrets Manager	Secret Manager	Key Vault
TLS certificates	Certificate Manager	Certificate Manager	Key Vault certificates
Metrics and logs	CloudWatch	Cloud Monitoring, Cloud Logging	Monitor, Log Analytics
Tracing	X-Ray, CloudWatch Application Signals	Cloud Trace	Application Insights
Managed Prometheus and Grafana	Managed Service for Prometheus, Managed Grafana	Managed Service for Prometheus	Monitor managed Prometheus, Managed Grafana

AI services

Building block	AWS	Google Cloud	Azure
Model APIs	Bedrock	Gemini Enterprise Agent Platform (formerly Vertex AI)	Microsoft Foundry (Foundry Models, Azure OpenAI)
Managed agent runtime	Bedrock AgentCore	Agent Runtime (formerly Vertex AI Agent Engine)	Foundry Agent Service
Conversational platform	Lex	Dialogflow CX	AI Bot Service, Copilot Studio

Building and deploying

Building block	AWS	Google Cloud	Azure
Infrastructure as code	CloudFormation, CDK	Infrastructure Manager	Resource Manager, Bicep
CI/CD	CodePipeline, CodeBuild	Cloud Build, Cloud Deploy	Azure Pipelines

Terraform and OpenTofu describe all three clouds, GitHub Actions and GitLab CI run pipelines for any of them, and Kubernetes, Kafka, PostgreSQL, Valkey/Redis, OpenSearch and Apache Iceberg run anywhere. Choosing open software over a managed service trades operating effort for portability; choosing the managed service trades portability for fewer pagers.

Appendix D. Glossary

The first section explains the handful of ideas that recur in almost every design, with the chapters where each one does real work. The second section defines every other term the book uses, alphabetically, with the chapter that teaches it.

Ideas that recur everywhere

Idempotency. An operation is idempotent when doing it twice has the same effect as doing it once: setting a seat to `HELD` by fan 42 is idempotent, adding one ticket to a cart is not. Networks lose replies, so every client retries, and every retry is a chance to do the work twice; idempotency is what makes retries safe. The usual tool is an **idempotency key**: the client sends a unique key with the request, the server records the key with the result in the same transaction as the work, and a repeat with the same key returns the stored result instead of acting again. Encore uses one for every hold, every payment and every refund the support agent requests. The mechanism is in Chapter 3, the API contract (the `Idempotency-Key` header, how replays and conflicts are answered) in Chapter 5, idempotent message consumers in Chapter 14, and the payment provider's keys in Chapter 24.

Exactly-once, honestly. No network can deliver a message exactly once; it can deliver at most once (and sometimes lose it) or at least once (and sometimes duplicate it). Systems that advertise exactly-once deliver at least once and make the *effect* happen once, through idempotent processing, deduplication by message ID, or transactions that commit the output and the consumer's position together. See Chapter 14 and Chapter 18.

The invariant. Every hard design question protects one sentence that must stay true: never sell a seat twice, never charge a card twice, never lose a paid order. Name the invariant first, find where concurrent requests or failures could break it, and put the guard there: a conditional write, a unique constraint, a lock with a fencing token, a single writer per key. See Chapter 3 and Chapter 26.

Consistency versus availability. When machines cannot talk to each other, a system must either refuse some requests (stay consistent) or answer with possibly stale data (stay available); when they can, it still trades latency for consistency on every request. Seat holds choose consistency; event pages choose availability. See Chapter 18 and Chapter 16.

Backpressure and admission control. A system that accepts more work than it can finish does not get faster; queues grow, latency explodes and timeouts cause retries that add more work. Pushing back early — rate limits, bounded queues, load shedding, a waiting room — keeps the work that is accepted fast. See Chapter 11 and Chapter 14.

Timeouts, retries and backoff. Every remote call needs a timeout, because a slow dependency is worse than a dead one; retries need exponential backoff with jitter and a budget, because synchronized retries from every layer multiply load. See Chapter 11 and Chapter 23.

Statelessness. A stateless server keeps nothing between requests that another copy could not rebuild, so any copy can serve any request and copies can be added or killed freely; the state moves to a database, a cache or the client. It is the default for horizontal scale, not a law. See Chapter 7.

Partitioning by key. Splitting data by a key spreads load across machines, but only as evenly as the key itself: one popular key — a flash-sale event, a celebrity — becomes a hot partition. Choosing and splitting keys is most of the work. See Chapter 17.

Little's law. In any stable system, the average number of items inside equals the arrival rate times the average time each spends inside, $L = \lambda W$. It sizes connection pools, worker counts, queues and the waiting room. See Chapter 1.

Tail latency. Users feel the slowest of the calls their request makes, so the 99th percentile, not the average, decides the experience; a request that fans out to many servers almost always meets a slow one. See Chapter 1 and Chapter 23.

Terms

ACID: The guarantees of a database transaction: atomicity (all or nothing), consistency (invariants hold), isolation (concurrent transactions do not interfere, to a chosen degree) and durability (committed data survives crashes). Chapter 12

Admission control: Deciding at the door how much work to accept, so that accepted work finishes on time; a waiting room is admission control for a whole event. Chapter 11

Amdahl's law: The speed-up from parallelism is limited by the part that stays serial: at most $1/(1 - p)$ when a fraction p is parallel. Chapter 2

Anti-entropy: Background comparison and repair of replicas, often with Merkle trees, so replicas converge even if updates were missed. Chapter 19

API gateway: The front door for APIs: routing, authentication, rate limits, request transformation and metrics in one place. Chapter 8

At-least-once / at-most-once delivery: Delivery guarantees of a message system: every message arrives but some may repeat, or no message repeats but some may be lost. Chapter 14

Autoscaling: Adding and removing capacity automatically from a measured signal such as CPU, requests per second or queue depth. Chapter 15

Availability: The fraction of time a service answers correctly, often written in nines; 99.9 percent allows about 8.8 hours of downtime a year. Chapter 1

Availability zone: A separate data center within a cloud region, with its own power and network, close enough for synchronous replication. Chapter 15

B-tree: The balanced, page-based index used by most relational databases; a few levels deep even for billions of rows. Chapter 12

Blockchain: A replicated, append-only ledger of blocks, each linked to the previous by its hash, kept consistent by a consensus protocol among nodes that do not trust each other. Chapter 20

Bloom filter: A compact bit array that answers "definitely not present" or "possibly present", used to skip disk reads and lookups. Chapter 19

Blue-green and canary deployment: Releasing by switching traffic between two complete environments, or by sending a small share of traffic to the new version first and widening it while metrics stay healthy. Chapter 23

Bulkhead: Isolating resources (pools, threads, instances) per dependency or tenant so that one failing part cannot exhaust capacity for the rest. Chapter 11

Byzantine fault: A failure in which a node keeps running but behaves arbitrarily or maliciously; tolerating f such nodes needs at least $3f + 1$ in BFT protocols. Chapter 20

Cache stampede (thundering herd): Many requests missing the same expired key at once and all rebuilding it, overloading the source; fixed with request coalescing, locks or early refresh. Chapter 9

Cache-aside: The application reads the cache, falls back to the database on a miss, and fills the cache itself; the most common caching pattern. Chapter 9

CAP theorem: During a network partition a distributed system must choose between consistency and availability for the operations that span it. Chapter 18

CDN (content delivery network): A network of edge servers that cache content close to users and absorb traffic before it reaches the origin. Chapter 10

Change data capture (CDC): Turning a database's change log into a stream of events for caches, search indexes and analytics. Chapter 14

Circuit breaker: A switch that stops calling a failing dependency for a while, failing fast instead, then probes before closing again. Chapter 11

Cold start: The delay before a new serverless function instance or container can serve its first request. Chapter 15

Compaction: Merging and rewriting sorted files in an LSM-tree to discard overwritten and deleted data. Chapter 12

Compare-and-set (CAS): An atomic update that succeeds only if the value still equals what the caller last read; the basis of optimistic concurrency. Chapter 3

Connection pool: A bounded set of open connections reused across requests, sized with Little's law. Chapter 4

Consensus: Agreement among a group of machines on a single value or log order despite failures, as in Raft and Paxos. Chapter 18

Consistent hashing: Placing keys and servers on a hash ring so that adding or removing a server moves only a small fraction of keys. Chapter 17

Consumer group: A set of consumers that share a stream's partitions, each partition read by one member at a time. Chapter 14

Container: A packaged process with its own file system and resource limits, sharing the host's kernel. Chapter 15

Count-Min sketch: A small table of counters that estimates how often items occur, never undercounting; used for heavy hitters and trending lists. Chapter 19

CQRS: Command query responsibility segregation: separate models for writing and for reading, kept in sync by events. Chapter 6

CRDT: A conflict-free replicated data type whose replicas can be updated independently and always merge to the same result. Chapter 16

Critical section: Code that must not run in two threads at once because it reads and writes shared state. Chapter 3

Dead-letter queue: Where messages go after repeated processing failures, so one poison message cannot block the rest. Chapter 14

Deadlock: Two or more workers each holding a lock the other needs, waiting forever; prevented by acquiring locks in a fixed order or with timeouts. Chapter 3

Denormalization: Storing data redundantly, shaped for the queries that read it, trading write work and consistency effort for fast reads; central to NoSQL modelling. Chapter 13

Distributed lock: A lock shared by processes on different machines; safe for correctness only with leases and fencing tokens. Chapter 18

DNS: The system that resolves names to addresses, with answers cached for their time to live. Chapter 4

Durability: The promise that acknowledged data survives failures, achieved with write-ahead logs, replication and backups. Chapter 12

Edge computing: Running small functions in CDN points of presence, close to users, for redirects, token checks and personalization. Chapter 10

Envelope encryption: Encrypting data with a data key and the data key with a master key held in a key management service. Chapter 22

Error budget: The amount of unreliability an SLO allows over a window, (1 − SLO) × window, spent on releases and incidents. Chapter 23

Event sourcing: Storing every change as an immutable event and deriving current state by replaying them. Chapter 6

Eventual consistency: Replicas may disagree for a while but converge when updates stop. Chapter 18

Failover: Moving service from a failed leader or region to a standby, automatically or by an operator. Chapter 16

Fan-out: One request or event becoming many: a feed post written to many timelines, or a query sent to many shards. Chapter 26

Fencing token: A number that increases with every lock grant; storage rejects writes carrying an older token, so a paused former lock holder cannot corrupt data. Chapter 18

Finality: The point after which a blockchain transaction cannot be reverted: probabilistic in proof of work (more confirmations, lower risk), explicit in proof-of-stake and BFT protocols. Chapter 20

Gas: The unit that meters computation and storage on Ethereum-style chains; the fee is gas used times the price per unit of gas. Chapter 20

Gossip protocol: Nodes periodically exchanging state with random peers, spreading information to all nodes in a logarithmic number of rounds. Chapter 19

gRPC: A remote procedure call framework using Protocol Buffers over HTTP/2, with streaming in both directions. Chapter 5

Head-of-line blocking: One slow or lost item holding up everything queued behind it, in TCP, HTTP/1.1 or a queue partition. Chapter 4

Health check: A periodic probe a load balancer or orchestrator uses to decide whether an instance should receive traffic. Chapter 8

Horizontal and vertical scaling: Adding more machines, or moving to a bigger machine. Chapter 7

Hot key, hot partition: A key or partition that receives a disproportionate share of traffic and limits the whole system. Chapter 17

HyperLogLog: A sketch that counts distinct items in about 12 KB with an error near one percent. Chapter 19

Isolation level: How much concurrent transactions may see of each other: read committed, repeatable read, snapshot, serializable. Chapter 12

JWT (JSON Web Token): A signed token carrying claims that a server can verify without a lookup; hard to revoke before it expires. Chapter 22

Kubernetes: A system that keeps containers running as declared: scheduling, restarts, service discovery, scaling and rollouts. Chapter 15

Lakehouse: Analytical tables in open formats (Parquet with Iceberg or Delta) on object storage, queried by many engines. Chapter 21

Latency and throughput: How long one operation takes, and how many operations complete per second. Chapter 2

Leader election: Choosing one node to coordinate, usually through a consensus system or a lease. Chapter 18

Lease: A lock or role granted for a limited time, which must be renewed; it expires on its own if the holder dies. Chapter 18

Linearizability: The strongest single-object consistency model: every operation appears to happen at one instant between its start and its end, as if there were one copy. Chapter 18

Load balancer (layer 4 and layer 7): A component that spreads connections (layer 4) or requests (layer 7) across healthy servers. Chapter 8

Load shedding: Deliberately rejecting some requests under overload so the rest succeed. Chapter 11

Lost update: Two read-modify-write cycles overlapping so that one overwrites the other's change. Chapter 3

LSM-tree: A storage engine that buffers writes in memory, flushes sorted files and merges them later; fast writes, used by Cassandra, RocksDB and many others. Chapter 12

MCP and A2A: Protocols for connecting AI agents to tools and data (Model Context Protocol) and to other agents (Agent2Agent). Chapter 24

Merkle tree: A tree of hashes in which each node hashes its children, so two replicas find their differences by comparing a few hashes. Chapter 19

Message queue and stream: A queue hands each message to one consumer and deletes it on acknowledgement; a stream is a retained, ordered log that many consumer groups read independently. Chapter 14

Microservice: A small, independently deployable service that owns its data and one business capability. Chapter 6

Model gateway: A service in front of language-model APIs that routes, rate-limits by tokens, caches, records cost and falls back between providers. Chapter 24

mTLS: Mutual TLS, in which both client and server present certificates, authenticating services to each other. Chapter 22

MVCC: Multi-version concurrency control: readers see a consistent snapshot of old versions while writers create new ones. Chapter 12

NoSQL: Databases that are not relational tables with SQL: key-value, document, wide-column, graph and others, usually designed to scale out by partitioning. Chapter 13

OAuth 2.0 and OpenID Connect: The protocols by which an application obtains a scoped access token on a user's behalf, and learns who the user is. Chapter 22

Optimistic and pessimistic concurrency control: Detecting conflicting updates at write time with a version check, or preventing them by locking first. Chapter 3

Outbox (transactional): Writing an event to an outbox table in the same transaction as the business change, then publishing it from there, so the change and the event never disagree. Chapter 14

PACELC: CAP's extension: if partitioned, choose availability or consistency; else, choose latency or consistency. Chapter 18

Pagination (offset and cursor): Returning results in pages by position, or by a cursor that remembers where the last page ended; cursors stay fast and stable on large, changing data. Chapter 5

Percentile (p50, p99): The value below which a given share of measurements fall; p99 latency is what the slowest one percent of requests exceed. Chapter 1

Power of two choices: Picking two servers at random and sending the request to the less loaded one, which nearly eliminates hot spots. Chapter 8

Proof of work and proof of stake: Ways to choose who appends the next block: by spending computation on a hash puzzle, or by locking up stake that can be destroyed for misbehaviour. Chapter 20

Quorum: The number of replicas that must answer a read or a write; with N replicas, reads and writes overlap when $R + W > N$. Chapter 16

Race condition: A result that depends on the timing of concurrent operations, such as two fans holding the same seat. Chapter 3

Raft: A consensus algorithm with an elected leader that replicates a log to a majority before committing each entry. Chapter 18

Rate limiter: A component that caps how many requests a client, key or tenant may make per unit of time. Chapter 11

Read replica: A copy of a database that serves reads, updated from the leader, usually with some lag. Chapter 16

Read-your-writes: The guarantee that a user always sees their own latest writes, even when reading from replicas. Chapter 16

Rebalancing: Moving partitions between nodes when nodes are added or removed, ideally without moving most of the data. Chapter 17

Rendezvous hashing: Choosing the server with the highest hash of (key, server); a simple alternative to a ring. Chapter 17

Replication lag: How far a replica trails its leader, in time or in log position. Chapter 16

Retry budget: A cap on retries as a share of normal traffic, so retries cannot multiply load during an outage. Chapter 11

Rollup: A layer-2 chain that executes transactions off the main chain and posts their data and a fraud or validity proof to it, inheriting its security at lower cost. Chapter 20

RPO and RTO: Recovery point objective (how much data may be lost) and recovery time objective (how long recovery may take). Chapter 23

Saga: A long business transaction split into local steps, each with a compensating action that undoes it if a later step fails. Chapter 18

Scale cube: Three ways to scale: clone the service (X), split it by function (Y), split it by data (Z). Chapter 7

Semantic cache: A cache that returns a stored answer to a question similar in meaning to an earlier one, used in front of language models. Chapter 9

Semaphore: A counter that allows up to *n* holders at once, used to bound concurrency. Chapter 3

Serverless function: Code the cloud runs on demand per event, scaling to zero and billing per request and duration. Chapter 15

Service mesh and sidecar: A layer of proxies next to every service instance that handles retries, mTLS and telemetry outside the application code. Chapter 6

Sharding: Another word for partitioning a dataset across machines. Chapter 17

Shuffle sharding: Assigning each customer a random small set of servers so that one bad customer affects only the few customers sharing that exact set. Chapter 7

SLA, SLI, SLO: A service level indicator is a measurement, an objective is the target for it, and an agreement is the contract with consequences. Chapter 1

Sliding window counter: A rate-limit algorithm that blends the previous and current window counts to approximate a true sliding window cheaply. Chapter 11

Smart contract: A program stored on a blockchain whose code and state every node executes and agrees on. Chapter 20

Snowflake ID: A 64-bit identifier made of a timestamp, a machine number and a sequence, sortable and generated without coordination. Chapter 19

Split brain: Two nodes both believing they are the leader and accepting writes, usually after a partition and a careless failover. Chapter 16

SSE, WebSocket and long polling: Ways to push data to clients: a one-way event stream over HTTP, a two-way persistent connection, or repeated requests held open until data arrives. Chapter 4

Sticky session: Routing a client to the same server every time; convenient for stateful servers, harmful for balance and failover. Chapter 8

Stream processing: Computing continuously over unbounded event streams, with windows, watermarks and state. Chapter 21

Token (fungible and non-fungible): A balance or a unique item recorded by a smart contract: interchangeable units (ERC-20) or one-of-a-kind items such as a ticket (ERC-721, NFT). Chapter 20

Token bucket: A rate-limit algorithm that refills tokens at a steady rate up to a capacity and spends one per request, allowing short bursts. Chapter 11

Tool call: A request from an AI agent to run a function or API on its behalf; in a distributed design it is an ordinary, authorized, idempotent API call. Chapter 24

Transaction: A group of reads and writes that the database applies all together or not at all. Chapter 12

TTL (time to live): How long a cached answer, DNS record or lease remains valid. Chapter 9

Two-phase commit (2PC): A protocol in which a coordinator asks all participants to prepare, then to commit; correct but blocking if the coordinator fails. Chapter 18

Universal Scalability Law: A model of throughput with contention and coherency terms that explains why adding nodes eventually stops helping, or hurts. Chapter 7

Vector clock: A set of counters, one per node, that detects whether two updates are ordered or concurrent. Chapter 18

Visibility timeout: How long a queue hides a delivered message from other consumers before redelivering it if no acknowledgement arrives. Chapter 14

Wallet: Software or hardware that holds the private keys controlling blockchain accounts and signs transactions; losing the keys loses the assets. Chapter 20

Watermark: A stream processor's estimate of how far event time has progressed, used to decide when a window is complete. Chapter 21

Webhook: An HTTP callback a system sends to notify another system of an event; must be signed, retried and idempotent. Chapter 5

Write skew: Two transactions each reading the same data, then writing different rows in a way that together breaks an invariant that each check alone passed. Chapter 12

Write-ahead log (WAL): An append-only log written before data pages change, so a crash can be recovered by replaying it. Chapter 12

Zero trust: Network security in which no request is trusted because of where it comes from; every call is authenticated and authorized. Chapter 22

Selected reading

Books

- Martin Kleppmann, *Designing Data-Intensive Applications* (O'Reilly). The deepest single treatment of storage, replication, partitioning, transactions and stream processing; a second edition with Chris Riccomini updates it for the cloud era.
- Alex Xu, *System Design Interview*, volumes 1 and 2 (with Sahn Lam). Interview-shaped walk-throughs of common designs.
- Betsy Beyer, Chris Jones, Jennifer Petoff and Niall Richard Murphy (eds.), *Site Reliability Engineering* (O'Reilly, 2016), and *The Site Reliability Workbook* (2018). SLOs, error budgets, monitoring and incident response as practised at Google; free online.
- Maarten van Steen and Andrew S. Tanenbaum, *Distributed Systems*, fourth edition. The textbook view: communication, naming, coordination, consistency, fault tolerance; free for personal use.
- Brendan Burns, *Designing Distributed Systems* (O'Reilly). Container-era patterns: sidecars, ambassadors, sharded services, work queues.
- Brian Goetz et al., *Java Concurrency in Practice*, and Luciano Ramalho, *Fluent Python*. Concurrency on one machine, explained with care.

Papers behind the building blocks

- Leslie Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System" (1978). Logical clocks and happened-before.
- Burton H. Bloom, "Space/Time Trade-offs in Hash Coding with Allowable Errors" (1970). The Bloom filter.
- David Karger et al., "Consistent Hashing and Random Trees" (1997). Consistent hashing.
- Patrick O'Neil et al., "The Log-Structured Merge-Tree" (1996). The storage engine behind most modern key-value stores.
- Sanjay Ghemawat, Howard Gobioff and Shun-Tak Leung, "The Google File System" (2003); Jeffrey Dean and Sanjay Ghemawat, "MapReduce" (2004); Fay Chang et al., "Bigtable" (2006).
- Mike Burrows, "The Chubby Lock Service for Loosely-Coupled Distributed Systems" (2006); Patrick Hunt et al., "ZooKeeper" (2010).
- Giuseppe DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store" (2007). Leaderless replication, quorums, vector clocks, Merkle trees, gossip.
- Philippe Flajolet et al., "HyperLogLog" (2007); Graham Cormode and S. Muthukrishnan, "The Count-Min Sketch" (2005).
- Jay Kreps, Neha Narkhede and Jun Rao, "Kafka: a Distributed Messaging System for Log Processing" (2011).
- Marc Shapiro et al., "Conflict-free Replicated Data Types" (2011).
- James C. Corbett et al., "Spanner: Google's Globally-Distributed Database" (2012). TrueTime and external consistency.
- Eric Brewer, "CAP Twelve Years Later" (2012); Seth Gilbert and Nancy Lynch, "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services" (2002); Daniel Abadi, "Consistency Tradeoffs in Modern Distributed Database System Design" (2012), which introduces PACELC.
- Jeffrey Dean and Luiz André Barroso, "The Tail at Scale" (2013). Why the slowest request decides latency at scale.
- Diego Ongaro and John Ousterhout, "In Search of an Understandable Consensus Algorithm" (2014). Raft.
- Daniel E. Eisenbud et al., "Maglev: A Fast and Reliable Software Network Load Balancer" (2016).
- Michael Mitzenmacher, "The Power of Two Choices in Randomized Load Balancing" (2001).
- Abhinandan Das, Indranil Gupta and Ashish Motivala, "SWIM: Scalable Weakly-consistent Infection-style Process Group Membership Protocol" (2002).

Online

- The Amazon Builders' Library: articles on timeouts, retries and backoff with jitter, shuffle sharding, load shedding and static stability.
- The Google SRE books online, and the Google Cloud Architecture Center.
- Jepsen analyses (jepsen.io): what real databases do under partitions, measured.
- The documentation of the systems named in each chapter — Redis, Kafka, PostgreSQL, Envoy, Kubernetes, DynamoDB, Spanner — which is usually the most precise description of their guarantees.